Web Development Toolkit for Java Developers

Build dynamic, secure, and scalable web applications with Java

Dr. Nirali Dabhi

Dr. Dharmendra Patel

Dr. Atul Patel

www.bpbonline.com

Copyright © 2023 BPB Online

All rights reserved. No part of this book may be reproduced, stored in a retrieval system, or transmitted in any form or by any means, without the prior written permission of the publisher, except in the case of brief quotations embedded in critical articles or reviews.

Every effort has been made in the preparation of this book to ensure the accuracy of the information presented. However, the information contained in this book is sold without warranty, either express or implied. Neither the author, nor BPB Online or its dealers and distributors, will be held liable for any damages caused or alleged to have been caused directly or indirectly by this book.

BPB Online has endeavored to provide trademark information about all of the companies and products mentioned in this book by the appropriate use of capitals. However, BPB Online cannot guarantee the accuracy of this information.

First published: 2023

Published by BPB Online
WeWork
119 Marylebone Road
London NW1 5PU

UK | UAE | INDIA | SINGAPORE

ISBN 978-93-55510-341

www.bpbonline.com

Dedicated to

My beloved Parents:

Late Nikhilkumar Johnbhai Honest

Nilaxi Nikhilkumar Honest

&

My Husband ***Bhavinkumar Williambhai Dabhi*** *and all my family members*

About the Authors

- **Dr. Nirali Dabhi** has worked as Senior Java Developer, Dakshya Freondtech, from March 2022 to June 2022. From January 2021 to February 2022, she was working as Associate Professor at Smt. Chandaben Mohanbhai Patel Institute of Computer Applications (CMPICA), CHARUSAT, Changa. January 2010 to December 2020, she has also worked as Assistant Professor at Smt. Chandaben Mohanbhai Patel Institute of Computer Applications (CMPICA), CHARUSAT, Changa. August 2005 to December 2009, she worked as a Lecturer at Smt. Chandaben Mohanbhai Patel Institute of Computer Applications (CMPICA) formerly known as Charotar Institute of Computer Application (CICA), CHARUSAT, Changa.

- **Dr. Dharmendra Patel** is currently working as Professor in Smt. Chandaben Mohanbhai Patel Institute of Computer Applications, Charusat, Changa. He possesses 19 years of progressive experience in Academics, Research and implementing academic policies. His area of research is Web Mining. He has published 49 research papers in various domains of computer science in reputed journals. He also acted reviewers and editors in various journals of repute. He has published 6 book chapters and 16 technical articles in reputed books and magazine respectively. He has received five research paper awards as contribution in research. He also acted resource person of various committees of conferences, STTPs, Workshops etc. He has organized more than 10 events as a coordinator. He also has professional body membership of various reputed organizations.

- **Dr. Atul Patel** is currently working as in charge Registrar of Charotar University of Science and Technology, Changa. He is Dean and Principal of Smt. Chandaben Mohanbhai Patel institute of Computer Applications since inception the university. Under his leadership, institution has achieved various milestones. The institution has received 5 stars ranking in GSIRF two consecutive years. He possesses 28 years of progressive experience in Industry, Academics, Research and implementing academic policies. He is also servicing as a controller of examination in Charotar University of Science and Technology. He also served as an E-Governance Coordinator, CHARUSAT for integration and smooth functioning of different digital services of entire University. He

provided several ICT services like Website and Web Application Development, Mobile Application Development to the various departments and institutions of entire University as and when required. His area of research is Wireless Communications. He has published more than 100 research papers in various domains of computer science. 6 students completed Ph.D. under his guide ship and 8 students are working under him. He is also an active member of NAAC committee. He possesses various roles in NAAC committee including chairman of NAAC peer team. He is awarded with various owners/ awards from reputed institutions and organizations. He is awarded with Rashtriya Vidya Gaurav Gold Medal Award, IIEM (ISC), New Delhi in 2020. He received Bharat Vidya Shiromani Award, Indian Solidarity Council, New Delhi in 2020. He is also senior member of ACM, USA and life member of CSI India. He has delivered more than 30 expert talks in various workshops, conferences, and seminars of repute. He is associated with several other universities/ Institutions as a member of Doctoral Advisory Committee, Boards of Studies, and Faculty Board. He is serving as an expert in Ph.D. evaluation and viva for many Universities. He is also serving as an expert in faculty recruitment for many institutions.

About the Reviewer

Dr. Pranav Vyas is a highly skilled and experienced Product Specialist at Caliber Technologies Pvt Ltd, a leading company that specializes in developing cutting-edge products for the pharmaceutical and chemical sector. His expertise is widely recognized by leading Indian and multinational companies in this domain.

Prior to joining Caliber Technologies Pvt Ltd, Dr. Vyas held an academic position for over a decade at a renowned university in Gujarat. As an academician, he authored numerous research papers which were published in top-tier journals and presented at both national and international conferences. Additionally, he has made notable contributions to several books in the form of book chapters, focusing on topics of his interest.

Dr. Vyas' diverse areas of expertise include IoT, Network Security, Software Engineering, Product Management and Blockchain. He is a frequent guest speaker and lecturer in leading universities and educational institutions, where he passionately shares his knowledge on these subjects.

Acknowledgement

There are a few people I want to thank for the continued and ongoing support they have given me during the writing of this book. First and foremost, I would like to thank my parents for continuously encouraging me for writing the book, my husband and my family motivated me to complete the work with all the support they can give me, I could have never complete this book without their support.

I am grateful to Dr. Atul Patel for providing me all the resources needed to complete the book, I would thank Dr. Dharmendra Patel for writing and guiding throughout the chapters, I gratefully acknowledge Mr. Rakesh Dobariya, Dr. Pranav Vyas for accepting and performing technical review for this book.

My gratitude also goes to the team at BPB Publications for being supportive enough to provide me quite a long time to finish the book and providing guidance and valuable inputs throughout the publication process.

Preface

Web development is one of the most demanding skills required in the field of computer application. Java supports the development of web application using simple model view controller as well as with support of advance frameworks based on java language.

This book takes a practical approach for web application development using java, it covers the tokens of concepts required to develop a web application. It covers the practical aspects of programming using simple examples.

This book is divided into **14 chapters**. Database connectivity from java, JSP and servlets, JSF, web services, Spring boot and aspect-oriented concepts.

Chapter 1: Database Connectivity From Java- The chapter covers the use of JDBC API, explain the major components of JDBC API, list and explain the types of driver available to communicate with the database.

Chapter 2: Performing Insert, Update, Delete and Select Operations- The chapter covers the steps used to access a database from a java application. JDBC API components with its methods are described briefly to understand the purpose and syntax before using them to build a program. The basic database operations like insert records, update records, selecting records, traversing on the ResultSet, deleting records, etc. are covered with suitable examples.

Chapter 3: Creating Properties File and Performing Batch Operations- The chapter covers the steps to create a properties file and access it in a java code. The use of properties file can be for different purposes here we use it to hold the database specific key and value pairs. The other part of chapter covers the concept of batch processing with the transactions and shows the concept of how to manually rollback and commit using the SavePoint. It also shows that instead of executing a program for one query, one can add the queries to batch and execute the batch single time, so it save time and effort, for example if you want to enter five records then have five insert queries added to a single batch and execute this batch once.

Chapter 4: Web Components and Web Application Directory Structure- This chapter covers the fundamentals knowledge required to understand client server communication, it includes the understanding of web client, server and protocols

used in the communication. The chapter covers a brief overview of TCP/IP and HTTP protocols with their role in web client server communication. The role of container is explained in brief and the details of how a web directory structure is organized is explained with the goal of making a web archive file. In the end of the chapter MVC design pattern is briefly described.

Chapter 5: Servlet Programming- This chapter covers servlet introduction, role and taks of servlet, working of servlet, servlet life cycle, generate plain text and html responses. Understand the generation of implicit and explicit data by understanding request and response headers, and its methods

Chapter 6: Managing Advance Features in Servlet- This chapter covers advance features that can be managed using servlets, the major features include managing cookies and sessions.

Chapter 7: Basics of Java Server Pages (JSP)- In this chapter we cover the basic syntax of JSP. JSP is used to write java in html, but how we write, where we write, and how much we write matters a lot. So understanding the basic scripting elements like scriptlets, expressions and declarations are very important. Then we understand the usage of directives like page, include and taglib directive. Apart from these we covered the include action, param element, forward action, and the XML syntax of using all the JSP tokens.

Chapter 8: Handling Beans and Using Expression Language- In this chapter we cover the concept of bean class, how to create and use it in a simple java application as well as in a web application, then we cover the usage of jsp:useBean, jsp:getProperty and jsp:getSetproperty. We cover the sharing of beans in different locations like request, session and application. In the second half part of chapter we cover the usage of expression language which is a short and easy way to invoke java code without writing core java syntax in html.

Chapter 9: Understand Model, View and Controller Pattern- In this chapter we cover the concept of model, view and controller in the development of the application. The MVC pattern suggest the separation of code based on the different role and purpose, so the error solving and managing of code become easier as the application size and complexity increases. Servlet is good at processing, JSP is good at presentation, and bean class and database can be used to represent the entity class. Technologies can be optimized in using it for right purpose, so Servlet is used as a Controller, JSP is used as a View layer and, Java Beans are used as Model layer.

Chapter 10: Overview of JSP Standard Tag Library (JSTL)- In this chapter we cover the concept of using an external library that support various functions required during programming. The use of this functions shows how easy it is with lesser programing we can continue to work without compromising functionalities.

Chapter 11: Outline of JSF for Building Component-Based, Event-Oriented Web Interfaces- In this chapter we cover the concept of using JSF, we compare its benefit over JSP, and explore the syntax and application of it.

Chapter 12: Working with Web Services (SOAP and RESTful)- In this chapter we cover the concept of web services both SOAP based and RESTful services.

Chapter 13: Aspect-Oriented Approach with Spring Framework- In this chapter we cover the aspect-oriented approach and explore the Spring Framework.

Chapter 14: Introduction to Spring Boot- In this chapter we cover the syntax and application of Spring Boot.

Code Bundle and Coloured Images

Please follow the link to download the
Code Bundle and the ***Coloured Images*** of the book:

https://rebrand.ly/v5hub5c

The code bundle for the book is also hosted on GitHub at **https://github.com/bpbpublications/Web-Development-Toolkit-for-Java-Developers**. In case there's an update to the code, it will be updated on the existing GitHub repository.

We have code bundles from our rich catalogue of books and videos available at **https://github.com/bpbpublications**. Check them out!

Errata

We take immense pride in our work at BPB Publications and follow best practices to ensure the accuracy of our content to provide with an indulging reading experience to our subscribers. Our readers are our mirrors, and we use their inputs to reflect and improve upon human errors, if any, that may have occurred during the publishing processes involved. To let us maintain the quality and help us reach out to any readers who might be having difficulties due to any unforeseen errors, please write to us at :

errata@bpbonline.com

Your support, suggestions and feedbacks are highly appreciated by the BPB Publications' Family.

Did you know that BPB offers eBook versions of every book published, with PDF and ePub files available? You can upgrade to the eBook version at www.bpbonline.com and as a print book customer, you are entitled to a discount on the eBook copy. Get in touch with us at :

business@bpbonline.com for more details.

At **www.bpbonline.com**, you can also read a collection of free technical articles, sign up for a range of free newsletters, and receive exclusive discounts and offers on BPB books and eBooks.

Piracy

If you come across any illegal copies of our works in any form on the internet, we would be grateful if you would provide us with the location address or website name. Please contact us at **business@bpbonline.com** with a link to the material.

If you are interested in becoming an author

If there is a topic that you have expertise in, and you are interested in either writing or contributing to a book, please visit **www.bpbonline.com**. We have worked with thousands of developers and tech professionals, just like you, to help them share their insights with the global tech community. You can make a general application, apply for a specific hot topic that we are recruiting an author for, or submit your own idea.

Reviews

Please leave a review. Once you have read and used this book, why not leave a review on the site that you purchased it from? Potential readers can then see and use your unbiased opinion to make purchase decisions. We at BPB can understand what you think about our products, and our authors can see your feedback on their book. Thank you!

For more information about BPB, please visit **www.bpbonline.com**.

Join our book's Discord space

Join the book's Discord Workspace for Latest updates, Offers, Tech happenings around the world, New Release and Sessions with the Authors:

https://discord.bpbonline.com

Table of Contents

1. Database Connectivity From Java 1
Introduction 1
Structure 1
Objectives 2
Introduction to JDBC 2
Core components of JDBC API 3
JDBC drivers 4
Types of JDBC drivers 4
Type 1: JDBC-ODBC Bridge 4
Type 2: Native-API driver 5
Type 3: Network-Protocol 6
Type 4: DatabaseProtocol driver 7
Type 5 JDBC Drivers 8
Conclusion 9
Points to remember 9
Multiple choice questions 9
Answers 9
Questions 10
Key terms 10

2. Performing Insert, Update, Delete and Select Operations 11
Introduction 11
Structure 11
Objectives 12
Steps to access the database from Java application 12
Import the packages 12
Register the JDBC driver 12
Open a connection 14
Creating a Statement 15

Creating a Statement object *15*
Creating the PreparedStatement objects *15*
Creating the CallableStatement objects *16*
Executing a query *17*
Extracting data from result set *18*
Clean up the environment *21*
MetaData 21
ResultSetMetaData *21*
DatabaseMetaData *22*
Creating application to access the database using IDE 22
Creating a table in Oracle from Java *23*
Alter table in Oracle from Java *32*
Insert records in table from Java *32*
Update records in table from Java *34*
Delete records in table from Java *36*
Conclusion 38
Points to remember 38
Multiple choice questions 38
Answers *40*
Questions 40
Key terms 41

3. Creating Properties File and Performing Batch Operations 43
Introduction 43
Structure 43
Objectives 44
Introduction to properties file 44
Constructors and methods of properties class 44
Properties file example 45
JDBC transactions 50
Commit and rollback *51*
Savepoints *51*

Batch processing in JDBC *51*
Advantages of batch processing *52*
Application of methods *52*
Conclusion 56
Multiple choice questions 56
Answers *57*
Questions 57
Key terms 57

4. Web Components and Web Application Directory Structure 59
Introduction 59
Structure 59
Objectives 60
Understanding web applications 60
Web server and client 60
TCP/IP - Transmission Control Protocol (TCP) and Internet Protocol (IP) 61
HTML and HTTP 62
Understanding the HTTP 62
Examples of HTTP parameters *63*
HTTP headers *65*
Message start-line 65
Header fields 65
Message body 66
HTTP - requests 66
Understanding URL 70
Need servlet and JSPs 71
Role of container in handling the request 73
Web application directory structure 75
WAR file 77
Deployment descriptor 77
Model View Controller (MVC) Overview 77
Conclusion 78

Points to remember 78
Multiple choice questions 78
Answers *79*
Questions 80
Key terms 80

5. Servlet Programming 81
Introduction 81
Structure 81
Objectives 82
Basics of the servlet 82
Role of servlets *82*
Common Gateway Interface (CGI) *83*
Servlet Life cycle 84
The init() method *84*
The service() method *85*
The doGet() method *85*
The doPost() method *85*
The destroy() method *85*
Servlet life cycle scenario *86*
Servlet programming 86
Servlet to generate plaintext *88*
Servlet to generate HTML *88*
Handling the client-side form data 89
Methods to read the data 89
Reading single values: getParameter *89*
Reading multiple values: getParameterValues *90*
Accessing parameter names: getParameterNames and getParameterMap *90*
Accessing the form fata : getReader or getInputStream *90*
Reading input in multiple character sets: setCharacterEncoding *91*
Reading form data *91*
Handling the request headers 93

Methods to read the request headers *93*
HTTP request headers *94*
Generate the server response 96
Setting random status codes: setStatus *97*
Setting 302 and 404 status codes: sendRedirect and sendError *97*
HTTP 1.1 status codes *97*
Handling the response headers 99
Setting response headers from servlets *100*
HTTP response headers *100*
Expires *102*
Last-modified *103*
Location *103*
Pragma *103*
Refresh *103*
Retry-after *104*
Set-Cookie *104*
WWW-Authenticate *104*
Conclusion 104
Points to remember 104
Multiple choice questions 105
Answers *105*
Questions 106
Key terms 106

6. Managing Advance Features in Servlet 107
Introduction 107
Structure 107
Objectives 107
Managing cookies 108
Sending and receiving cookies *108*
Sending cookies to the client *109*
Setting the maximum age *109*

Placing the Cookie in the Response Headers 109
Reading cookies from the client 110
Cookie attributes 110
Session and cookies 111
Cookie utilities 113
Utilities.java 113
LongLivedCookie.java 114
Cookie utilities 114
RepeatVisitor.java 115
Using cookies to handle user accounts and preferences 115
AccessCounts.java 116
RegistrationForm.java 117
RegistrationServlet.java 118
Managing session 118
Overview of session tracking 119
Cookies 119
URL rewriting 120
Hidden form fields 120
Session tracking in servlets 121
Accessing the session object associated with the current request 121
Looking up information associated with a session 122
Associating information with a session 122
Discarding session data 123
API for session tracking 124
Types of sessions 126
URL encoding 127
Conclusion 128
Points to remember 128
Multiple choice questions 128
Answers 129
Questions 129
Key terms 129

7. Basics of Java Server Pages (JSP) 131
Introduction 131
Structure 131
Objectives 132
Java Server Pages 132
JSP life cycle 132
The JSP page interface 134
The HttpJspPage interface 134
Overview of JSP 134
Basic tokens of JSP 135
Writing first JSP program 136
Basic scripting elements 136
JSP expressions 137
JSP scriptlets 141
JSP declarations 142
Comparing JSP expressions, Scriptlets, and declarations 144
JSP directives 146
Page directive 146
The import attributes 147
The contentType and pageEncoding attributes 148
The session attributes 158
The isELIgnored attribute 159
The buffer and autoFlush attributes 159
The info attributes 159
The errorPage and isErrorPage attributes 159
The extends attribute 167
The language attributes 167
Include files 167
The jsp:include action 167
The page attributes 168
The flush attribute 168
XML syntax and jsp:include 169

The include directive *175*
The file attributes *176*
XML syntax for the include directive *176*
Forwarding requests with jsp:forward 183
taglib directive 186
Conclusion 186
Points to remember 186
Multiple choice questions 187
Answers *188*
Questions 188
Key terms 188

8. Handling Beans and Using Expression Language 189
Introduction 189
Structure 189
Objectives 190
Introduction to JavaBean 190
Advantages of using Java bean *192*
Using Java beans in a web application *193*
JSP syntax for creating and accessing beans *197*
Creating beans *197*
Retrieving values from beans *198*
Assigning values to beans *199*
Assigning properties of bean with input parameter values *205*
Assigning all properties of bean with input parameter values *212*
Sharing Beans in different scopes 212
Unshared bean in Page scope *219*
Sharing bean in Request scope *222*
Sharing bean in Session scope *225*
Sharing bean in application scope 230
Introduction to expression language 232
Retrieving scoped variables 233

Retrieving Collections 236
Using Implicit objects 239
Retrieving Bean properties 242
Using Operators 248
Using Functions 256
Conclusion 256
Points to remember 256
Multiple choice questions 257
Answers 258
Questions 258
Key terms 258

9. Understand Model, View and Controller Pattern 261
Introduction 261
Structure 261
Objectives 262
Understanding the use of technology 262
Design pattern 262
Uses of design patterns 262
MVC design pattern 263
Need for MVC 263
MVC frameworks 264
Implementing MVC with servlets and JSP 264
Introduction to RequestDispatcher interface 267
Data sharing in request scope 269
Data sharing in session scope 269
Data sharing in application scope 270
MVC example 270
Conclusion 294
Points to remember 294
Multiple choice questions 294
Answers 295

Questions 295
Key terms 295

10. Overview of JSP Standard Tag Library (JSTL) 297
Introduction 297
Structure 297
Objectives 298
Introduction to JSTL 298
Installing JSTL library 298
Core tag library 299
c:out tag 300
c:set and c:remove tags 301
c:if tag 302
c:choose, c:when, and c:otherwise tag 304
c:forEach and c:forTokens tags 306
c:import tag 309
c:url and c:param tags 312
c:redirect tag 316
c:catch tag 318
SQL tags 319
sql:setDataSource tag 319
sql:query tag 320
sql:update tag 322
sql:param tag 322
sql:dateParam tag 323
sql:transaction tag 323
XML tags 324
x:out tag 325
x:parse tag 325
x:set tag 326
x:if tag 326
x:forEach tag 327

x:choose, x:when, x:otherwise tags *328*
x:transform tag *329*
JSTL functions 331
fn:contains() function *331*
fn:containsIgnoreCase() function *333*
fn:endsWith() function *334*
fn:escapeXml() function *335*
fn:indexOf() *336*
fn:join() *337*
fn:length() *338*
fn:replace() *339*
fn:split() *340*
fn:startsWith() *341*
fn:substring() *342*
fn:substringAfter() *343*
fn:substringBefore() *344*
fn:toLowerCase() *345*
fn:toUpperCase() *346*
fn:trim() *346*
Formatting tags 347
fmt:formatNumber () *347*
fmt:parseNumber *350*
fmt:formatDate *351*
fmt:parseDate *354*
fmt:bundle and fmt:setbundle *355*
fmt:setLocale *357*
fmt:timeZone and fmt:setTimeZone *358*
fmt:message 360
fmt:requestEncoding *360*
Conclusion 361
Points to remember 361
Multiple choice questions 361

Answers *362*
Questions 362
Key terms 362

11. Outline of JSF for Building Component- Based, Event-Oriented Web Interfaces 363
Introduction 363
Structure 363
Objectives 364
Introduction to JSF 364
Web interfaces in JSF 365
HTML components in JSF *366*
Nested tags *366*
Using Java objects in JSF *366*
Structure of a JSF application *367*
MVC in JSF 367
Controller beans *367*
Managing PrimeFaces and DataList *368*
Dependency injection and annotations *370*
Lifecycle methods *370*
Serving levels *371*
Bean information *372*
Events in JSF 372
Component-managed events *372*
Application of JavaServer faces 373
Project preparation and environment setup *374*
Pages prototype and design *374*
Development and debugging *375*
Performance Tuning *376*
Conclusion 377
Points to remember 377
Multiple choice questions 377
Answers *378*

Questions 378
Key terms 378

12. Working with Web Services (SOAP and RESTful) 379
Introduction 379
Structure 379
Objectives 380
Service-oriented architecture 380
Service connections 380
Service-oriented terminologies 380
Workings of service-oriented architecture 381
Functional operations 382
Qualitative operations 382
Introduction to web services 382
Simple Object Access Protocol (SOAP) 383
Types of web services 383
SOAP-based web services 384
SOAP Web services benefits 385
Problems with SOAP web services 385
RESTful web services 385
Advantages of RESTful web services 385
Web services in Java 386
Using JAX-WS to create a web service and client 386
Creating a web service 387
Choosing a container 387
Creating a web service from a Java class 388
Adding an operation to the web service 388
Executing and testing the web service 393
To test successful deployment to a GlassFish or WebLogic server 393
Consume a web service in Java 395
Web service interoperability 402
Creating web service using Java 402

Create ASP.net client *410*
Create the client *411*
Provide the web reference *411*
Call a web service *413*
Conclusion 415
Points to remember 415
Multiple choice questions 416
Answers *416*
Questions 417
Key terms 417

13. Aspect-Oriented Approach with Spring Framework 419
Introduction 419
Structure 419
Objectives 420
Software framework 420
Spring introduction 422
Spring architecture *422*
Spring directory structure *423*
IoC container 424
Containers and beans *424*
Resources *425*
Container settings *425*
The beans *426*
Using the container *426*
Dependencies 426
Using setter methods and parameterized constructor for injecting dependencies *427*
Ambiguous dependencies *427*
Collections *429*
Checking for dependencies *431*
Method injection *431*
Bean scopes 432

The singleton scope *432*
The prototype scope *433*
The request scope *433*
The session scope *434*
The global session scope *434*
Custom scopes *434*
Using a custom scope *435*
Bean definition inheritance 436
The ApplicationContext 437
Internationalization using MessageSources *437*
Events *439*
Applying validation 440
Aspect-oriented programming 448
The Solution through AOP *451*
AOP terminologies *452*
Aspects *452*
JoinPoint *452*
Pointcut *452*
Advice *453*
Conclusion 454
Points to remember 454
Multiple choice questions 454
Answers *455*
Questions 455
Key terms 456

14. Introduction to Spring Boot 457
Introduction 457
Structure 457
Objectives 458
Basics of Spring boot 458
Spring boot advantages and disadvantages *459*

Spring boot architecture *460*
Creating projects 460
Annotations 461
Dependency management 463
Inheriting starter parent *463*
Spring boot application properties *465*
YAML properties file *465*
Spring boot property groups *466*
Common application properties *467*
Spring boot starters *469*
External starters *469*
Spring boot production starters *471*
Spring boot technical starters *471*
Spring boot starter web *472*
Using another embedded web server 473
Jetty server *473*
Undertow server *474*
spring-boot-starter-web vs. spring-boot-starter-tomcat *474*
Database connectivity 476
Spring data repository *476*
Spring boot starter data JPA *477*
Hibernate vs. JPA *477*
Conclusion 477
Points to remember 477
Multiple choice questions 478
Answers *478*
Questions 478
Key terms 479

Index 481-489

Chapter 1
Database Connectivity From Java

Introduction

Handling data is very important for any programming language or application. Therefore, understanding how to access the database from the language is very significant. There are different approaches to access the database from a Java application. This chapter covers the major components and their structures, so one can understand and use them for accessing database from the application. Based on the requirements of accessing the database, one can select a different driver to handle the database. The chapter shows the merits and demerits of using different types of drivers so that one can select the right driver while programming the application for database usage.

Structure

In this chapter, we will cover the following topics:

- Introduction to JDBC API
- Core components of JDBC API
- JDBC drivers
- Types of JDBC drivers
- Summary

Objectives

The chapter covers the concept of drivers, types of drivers and core components of JDBC. The understanding of these components will help one select the right type of driver for establishing the database connectivity based on one's requirement.

Introduction to JDBC

Java application programming interface is a list of all classes that are part of the **Java Development Kit** (**JDK**). These prewritten classes provide a tremendous amount of functionality to a programmer. To connect any database and access data from Java environment, the support of **Java Database Connectivity** (**JDBC**) API is provided to the programmers. Using the JDBC API, you can access virtually any data source, from relational databases to different files. The JDBC API comprises two packages:

- `java.sql`
- `javax.sql`

You automatically get both packages when you download the **Java Platform Standard Edition** (**Java SE**) 8. To use the JDBC API with a particular database management system, you need a JDBC technology-based driver to mediate between JDBC technology and the database.

Java JDBC is a Java library to connect and execute query with the database.

JDBC API uses JDBC drivers to connect with the database; refer to *Figure 1.1*:

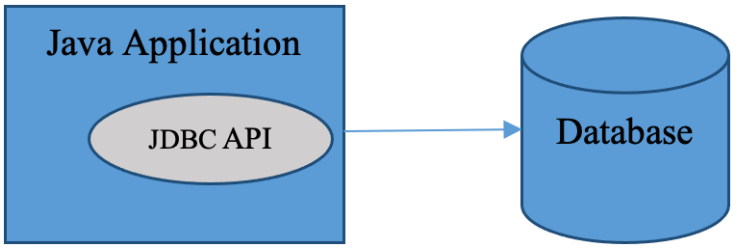


Figure 1.1: *Use of JDBC API in Java application to connect with a database*

The JDBC library includes APIs for performing the basic operations like the following:

- **Query the database**: Read the data from the database.
- **Query the database and result set meta data**: The database metadata contains information about the database itself, for instance, information about the tables defined the columns in each table, the data types, etc.
- **Update the database**: Write data to the database, which includes adding new records or modifying the existing records.

- **Perform transactions**: A transaction groups multiple updates and possible queries, into a single action. Either all the actions are executed or none of them are.

Core components of JDBC API

The JDBC API provides the interfaces and classes that allow you to perform the operations required to connect and communicate with a database from a Java application. The components include the following:

- **`DriverManager`**: This class manages a list of database drivers.
- **`Driver`**: This interface handles communications with the database server. A JDBC driver is a collection of Java classes that enables you to connect to a certain database. A JDBC driver implements many of the JDBC interfaces, which hide the complexity of connecting to a database.
- **`Connection`**: After the JDBC driver is loaded and initialized, we need to connect to the database; this interface has all methods for contacting a database. All communication with the database is done with a proper connection. An application can have more than one connection open to a database at a time.
- **`Statement`**: Objects created from this interface are used to submit the SQL statements to the database. A **`Statement`** is what you use to execute queries and updates against the database. There are a few different types of statements that we can use as per our requirements, but each statement corresponds to a single query or update at a time.
- **`ResultSet`**: These objects hold data retrieved from a database after you execute an SQL query using **`Statement`** objects. The result obtained after executing a query is held and allowed for navigation using the **`ResultSet`**.
- **`SQLException`**: This class handles any errors that occur in a database application. The following are the core components of JDBC API.

The below figure shows the major components used in establishing the database connection.

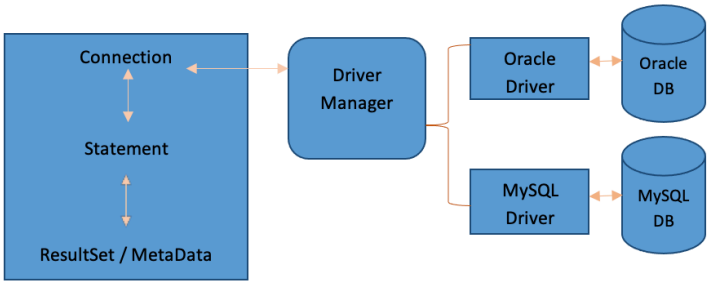


Figure 1.2: *Core components of JDBC API*

The Java application communicates with the core components to load the **`Driver`**, and then it establishes a connection, creates a statement, executes it, retrieves the **`ResultSet`**, traverses and processes the result. If anything goes wrong during the database communication after loading the driver, **`SQLException`** are thrown based on the type of error.

JDBC drivers

JDBC drivers implement the defined interfaces in the JDBC API for interacting with your database server. JDBC driver implementations vary because of the wide variety of operating systems and hardware platforms on which Java operates. Sun has divided the implementation types into four categories:

- **Type 1**: JDBC-ODBC (Open Database Connectivity) Bridge
- **Type 2**: Native-API driver
- **Type 3**: Network-Protocol driver (Middleware driver)
- **Type 4**: Database-Protocol driver (Pure Java driver)

Types of JDBC drivers

There are different types of drivers available for establishing the database connection; let's look at them.

Type 1: JDBC-ODBC Bridge

It requires you to configure a Data Source Name (DSN) on your system. A type 1 JDBC driver consists of a Java part that translates the JDBC interface calls to ODBC calls. An ODBC bridge then calls the ODBC driver of the given database. Type 1 drivers were mostly used when there were no type 4 drivers (all Java drivers).

Advantages

- Easy to use
- Can be easily connected to any database

Disadvantages

- Performance degraded because JDBC method call is converted into the ODBC function calls.
- The ODBC driver needs to be installed on the client machine.

Following is the flowchart of Type 1 Driver:

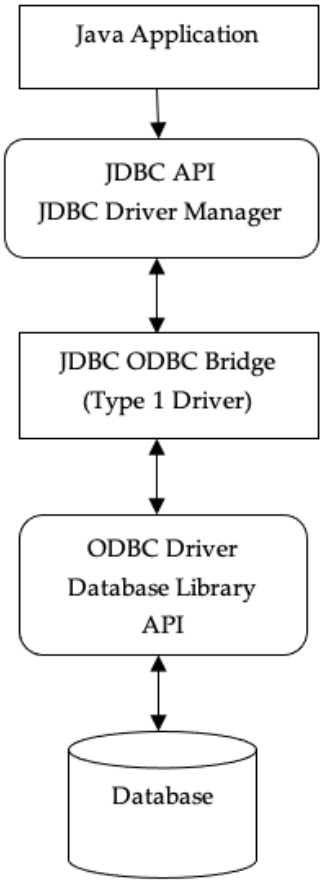


Figure 1.3: *Type 1 Driver*

Type 2: Native-API driver

A type 2 JDBC driver is like a type 1 driver, except the ODBC part is replaced with a native code part instead. The native code part is targeted at a specific database product, and JDBC API calls are converted into native C/C++ API calls.

Advantage

- Performance upgraded than JDBC-ODBC bridge driver

Disadvantage

- The Native driver needs to be installed on each client machine.
- The Vendor client library needs to be installed on the client machine.

Following is the flow chart of Type 2 Driver:

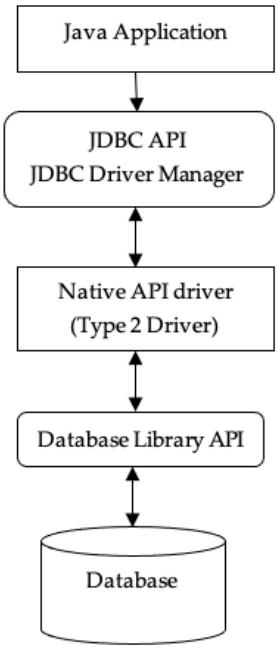


Figure 1.4: *Type 2 Driver*

Type 3: Network-Protocol

JDBC clients use standard network sockets. A type 3 JDBC driver is an all-Java driver that sends JDBC interface calls to an intermediate server. The intermediate server then connects to the database on behalf of the JDBC driver.

Advantage

- No client-side library is required because of the application server.

Disadvantages

- Network support is required on the client machine.
- It requires database-specific coding in the middle tier.
- Maintenance of **`NetworkProtocol`** driver becomes costly.

Following is the flow chart of Type 3 Driver:

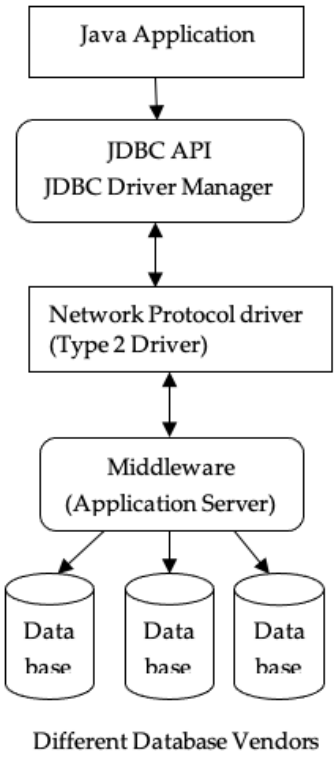


Figure 1.5: *Type 3 Driver*

Type 4: DatabaseProtocol driver

It communicates directly with the vendor's database through socket connection. A type 4 JDBC driver is an all-Java driver that connects directly to the database. It is implemented for a specific database product.

Advantages

- Better performance than all other drivers
- No software required at client or server side

Disadvantages

- Drivers depend on the database.

Following is the flow chart of Type 4 Driver:

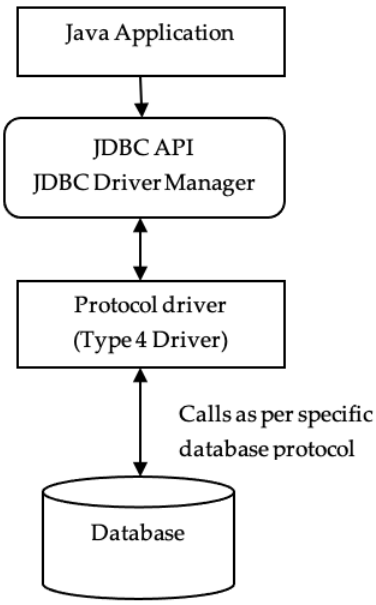


Figure 1.6: *Type 4 Driver*

Which driver should be used?

- If you are accessing one type of database, such as Oracle, Sybase, or IBM, the preferred driver type is 4.
- If your Java application is accessing multiple types of databases at the same time, type 3 is the preferred driver.
- Type 2 drivers are useful in situations where a type 3 or type 4 driver is not available for your database yet.
- The type 1 driver is not considered a deployment-level driver and is typically used for development and testing purposes only.
- Apart from the four drivers listed by the sun recently, there is a type 5 driver.

Type 5 JDBC Drivers

Type 5 JDBC drivers offer the same client-side, single tier, 100% Java architecture of type 4 JDBC drivers but address the limitations of many of the type 4 JDBC drivers available today. Progress DataDirect is the only vendor of Type 5 JDBC drivers. It offers superior application performance and scalability, codeless enhancement and all-in-one deployment.

Conclusion

The chapter covered the use of JDBC, explained the major components of JDBC, and listed and explained the types of drivers available to communicate with the database. The chapter is the foundation required to start with database connectivity from the Java language.

The next chapter will cover the details of how to access the database from the Java language.

Points to remember

- **Java Database Connectivity** (**JDBC**) is used to connect databases from the Java language; the type of driver can be selected based on the requirement of communication with the database.
- Java drivers allow you to interact with the database and access it.

Multiple choice questions

1. **What does JDBC stand for?**
    a. Java Database Connectivity
    b. Java Driver for Basic Connection
    c. Joint Database Connectivity
    d. Joint Driver for Basic Connection

2. **Which of the following type of JDBC driver should be used if your Java application is accessing multiple types of databases at the same time?**
    a. Type 1
    b. Type 2
    c. Type 3
    d. Type 4

Answers

1. **a**
2. **c**

Questions

1. What is JDBC?
2. What is a JDBC driver?
3. What are the core components of JDBC API?
4. Explain the types of JDBC driver.

Key terms

- **JDBC**: It refers to the Java API to connect and execute query with the database.
- **JDBC drivers**: JDBC drivers implement the defined interfaces in the JDBC API for interacting with your database server.

Join our book's Discord space

Join the book's Discord Workspace for Latest updates, Offers, Tech happenings around the world, New Release and Sessions with the Authors:

https://discord.bpbonline.com

Chapter 2
Performing Insert, Update, Delete and Select Operations

Introduction

This chapter covers writing the necessary queries to perform the basic database creation and manipulation tasks from the Java platform. The concept of database remains the same, but writing the query from Java language is little different than writing query from the database itself This chapters deals with the syntax basic queries that we can write from the Java language.

Structure

In this chapter, we will discuss the following topic:

- Steps to connect and access the database from a Java application
- Metadata
- Creating application to access the Oracle database using IDE Netbeans
    - Create a table in Oracle from Java
    - Alter table in Oracle from Java
    - Insert records in table from Java
    - Update records in table from Java
    - Delete records in table from Java

Objectives

After studying this chapter, you should be able to access the database and perform basic operations like creating a table, inserting records in the table, altering it, deleting the records, updating the records, and performing the basic database tasks from the Java program.

Steps to access the database from Java application

These are the steps involved in building a JDBC application:

- Import the packages
- Register the JDBC driver
- Open a connection
- Create a statement
- Execute a query
- Extract data from result set
- Clean up the environment

Import the packages

To use the classes and interfaces provided by API, we need to import the package in which our desired functionality falls. By default, **`java.lang`** is imported in the application, so all the other packages have to be imported explicitly. To use the JDBC API, we need to import **`java.sql`** or **`javax.sql`**, as they provide the primary and necessary classes for establishing the connection and communicating with the database. The import statement is as follows:

```
import java.sql. *;
```

It is a bad practice to write an '**`*`**' after the package name; instead, write the name of classes required to use as part of application programming.

Register the JDBC driver

We need to register the driver to initialize communication with the database. This step causes the JVM to load the desired driver implementation into memory so that it can fulfill your JDBC requests.

You must register the driver in your program before you use it. Registering the driver is the process by which the Oracle driver's class file is loaded into the memory so that it can be utilized as an implementation of the JDBC interfaces.

`Class.forName()`

- It dynamically loads the driver's class file into memory, which automatically registers it.
- This method is preferable because it allows you to make the driver registration configurable and portable.

The following figure shows the code snippet for registering the Oracle driver using **`the forName`** method:

```
01. try
02.  {
03.  Class.forName("oracle.jdbc.driver.OracleDriver");
04.  }
05. catch(ClassNotFoundException ex)
06.  {
07.     System.out.println("Error: unable to load driver class!");
08.  }
09.
```

Figure 2.1: *Register Oracle Driver using forName method*

`DriverManager.registerDriver()`

- The static **`DriverManager.registerDriver()`** method can be used to register the driver.
- Use it if you are using a non-JDK compliant JVM, such as the one provided by Microsoft.

The following figure shows the code snippet for registering the Oracle driver using **`the registerDriver`** method:

```
01. try
02.  {
03.            Driver myDriver = new oracle.jdbc.driver.OracleDriver();
04.            DriverManager.registerDriver( myDriver );
05.  }
06.  catch(ClassNotFoundException ex)
07.  {
08.            System.out.println("Error: unable to load driver class!");
09.  }
10.
11.
```

Figure 2.2*: Register Oracle Driver using registerDriver method*

Open a connection

After loading the driver, you can establish a connection using the **`DriverManager.getConnection()`** method, as follows:

- **`getConnection(String url)`**
- **`getConnection(String url, Properties prop)`**
- **`getConnection(String url, String user, String password)`**

A database URL is an address that points to your database; based on the database, the **`url`** can be formed as shown in the following table:

RDBMS	JDBC driver name	URL format
MySQL	`com.mysql.jdbc.Driver`	`jdbc:mysql://hostname:port/databaseName`
	`oracle.jdbc.driver.OracleDriver`	`jdbc:oracle:thin:@hostname:port Number:databaseName`
DB2	`COM.ibm.db2.jdbc.net.DB2Driver`	`jdbc:db2:hostname:port Number/databaseName`
Sybase	`com.sybase.jdbc.SybDriver`	`jdbc:sybase:Tds:hostname: port Number/databaseName`

***Table 2.1**: Database and URL format*

Based on the different forms of **`getConnection`**, the following approaches can be used. The following snippet provides the details of **`url`**, **`username`** and **`password`** to set up the connection:

```
String URL = "jdbc:oracle:thin:@localhost:1521:EMP";
String USER = "username";
String PASS = "password"
Connection conn = DriverManager.getConnection(URL, USER, PASS);
```

The following snippet shows the use of the property file to read the username and password to set up the connection:

```
01. import java.util.*;
02. String URL = "jdbc:oracle:thin:@localhost:1521:EMP";
03. Properties info = new Properties( );
04. info.put( "user", "username" );
05. info.put( "password", "password" );
06. Connection conn = DriverManager.getConnection(URL, info);
```

***Figure 2.3**: Set up connection with property files*

Creating a Statement

It is used for building and submitting an SQL statement to the database. Once a connection is established, we can interact with the database. The JDBC **`Statement`**, **`CallableStatement`**, and **`PreparedStatement`** interfaces define the methods and properties that enable you to send SQL or PL/SQL commands and receive data from your database. They also define methods that help bridge data type differences between Java and SQL data types used in a database. Below table provides the type of Statements available:

Interfaces	Recommended use
`Statement`	It is used for general-purpose access to your database and is useful when you are using static SQL statements at runtime. The Statement interface cannot accept parameters.
`PreparedStatement`	Use this when you plan to use the SQL statements many times. The `PreparedStatement` interface accepts input parameters at runtime.
`CallableStatement`	Use it when you want to access the database stored procedures. The `CallableStatement` interface can also accept runtime input parameters.

***Table 2.2**: Types of Statements*

Creating a Statement object

Before we use a **`Statement`** object to execute a statement, we need to create a statement using the **`Connection`** object's **`createStatement()`** method, as shown here:

```
01. Statement stmt = null;
02. try
03. {
04.      stmt = con.createStatement( );
05. }
06. catch (SQLException e)
07. {  }
08. finally
09. {       }
10.
11.
12.
```

***Figure 2.4**: Create statement object*

Creating the PreparedStatement objects

The **`PreparedStatement`** interface extends the **`Statement`** interface, which gives you added functionality with a couple of advantages over a generic **`Statement`**

object. This statement gives you the flexibility of supplying arguments dynamically. We need to create a prepared statement using the **prepareStatement()** method, as shown in the below code. The following code snippet shows the use of **query** parameter:

```
01. PreparedStatement pstmt = null;
02. try
03. {
04.    String SQL = "Update Employees SET age = ? WHERE id = ?";
05.       pstmt = conn.prepareStatement(SQL);
06.     . . .
07. }
08. catch (SQLException e)
09. {  }
10. finally
11. {        }
12.
13.
14.
```

***Figure 2.5**: Create preparedstatement object*

Creating the CallableStatement objects

Just as a **Connection** object creates the **Statement** and **PreparedStatement** objects, it also creates the **CallableStatement** object, which would be used to execute a call to a database stored procedure. We need to create a callable statement using the **prepareCall()** method, as shown in the next section.

Creating CallableStatement object

Suppose you need to execute the following Oracle stored procedure:

```
CREATE OR REPLACE PROCEDURE getStudentName
    (STUDENT_ID IN NUMBER, STUDENT_NAME OUT VARCHAR) AS
    BEGIN
        SELECT name INTO STUDENT_NAME
        FROM Student
        WHERE ID = STUDENT_ID;
    END;

    CallableStatement cstmt = null;
        try {
            String SQL = "{call getStudentName (? ?)}";
            cstmt = con.prepareCall(SQL);
            . . .
```

```
}
catch (SQLException e) {
        . . .
}
```

The following code shows the creation of procedure and the use of callable statement to invoke the procedure:

```
01. CREATE OR REPLACE PROCEDURE getStudentName
02.     (STUDENT_ID IN NUMBER, STUDENT_NAME OUT VARCHAR) AS
03.  BEGIN
04.      SELECT name INTO STUDENT_NAME
05.      FROM Student
06.      WHERE ID = STUDENT_ID;
07.  END;
08.
09.
10. CallableStatement cstmt = null;
11.       try {
12.      String SQL = "{call getStudentName (? ?)}";
13.      cstmt = con.prepareCall(SQL);
14.      . . .
15.  }
16.  catch (SQLException e) {
17.      . . .
18.  }
```

Figure 2.6: *Create statement object*

The statement can be closed using the **`Statement`** object's **`close()`** method.

Executing a query

Once we have created a **`Statement`** object, we can use it to execute an SQL statement with one of its three execute methods:

- **`boolean execute (String SQL)`**: It returns a boolean value of true if a **`ResultSet`** object can be retrieved; otherwise, it returns false. Use this method to execute SQL **Data Definition Language** (**DDL**) statements or when you need to use truly dynamic SQL.
- **`int executeUpdate (String SQL)`**: It returns the number of rows affected by the execution of the SQL statement. Use this method to execute SQL statements for which you the total number of affected rows are returned. , this method is used with e, an **`INSERT`**, **`UPDATE`**, or **`DELETE`** statement.
- **`ResultSet executeQuery (String SQL)`**: It returns a **`ResultSet`** object. Use this method when you expect to get a result set, as you would with a **`SELECT`** statement.

Extracting data from result set

The SQL statements that read data from a database query return the data in a result set. The **`java.sql.ResultSet`** interface represents the result set of a database query. A **`ResultSet`** object maintains a cursor that points to the current row in the result set. The term *result set* refers to the row and column data contained in a **`ResultSet`** object. The methods of the **`ResultSet`** interface can be divided into three categories:

- Navigational methods
- Get methods
- Update methods

The cursor is movable based on the properties of the **`ResultSet`**. These properties are designated when the corresponding **`Statement`** that generates the **`ResultSet`** is created. JDBC provides the following connection methods to create statements with the desired **`ResultSet`**:

```
createStatement(int RSType, int RSConcurrency)

prepareStatement(String SQL, int RSType, int RSConcurrency)

prepareCall(String sql, int RSType, int RSConcurrency)
```

In the first method , the first argument indicates the type of a **`ResultSet`** object, and the second argument is one of two **`ResultSet`** constants for specifying whether a result set is read-only or updatable. The possible **`RSType`** are given as follows. If you do not specify any **`ResultSet`** type, you will automatically get one that is **`TYPE_FORWARD_ONLY.`**.

Type	Description
`ResultSet.TYPE_FORWARD_ONLY`	The cursor can only move forward in the result set.
`ResultSet.TYPE_SCROLL_INSENSITIVE`	The cursor can scroll forward and backward, and the result set is not influenced by the changes occurred in the database after the result set was created.
`ResultSet.TYPE_SCROLL_SENSITIVE.`	The cursor can scroll forward and backward, and the result set is influenced by the changes occurred in the database after the result set was created..

***Table 2.3**: Types of ResultSet*

- **Concurrency of ResultSet**: The possible **`RSConcurrency`** are given as follows. If you do not specify any concurrency type, you will automatically get one that is **`CONCUR_READ_ONLY`**.

Concurrency	Description
`ResultSet.CONCUR_READ_ONLY`	Creates a read-only result set; this is the default
`ResultSet.CONCUR_UPDATABLE`	Creates an updateable result set

***Table 2.4**: Types of Concurrency Support in ResultSet*

- **Navigation methods provided by the ResultSet**: These methods involve moving the cursor to the derived **`ResultSet`**. The most common navigation methods are listed in *Table 2.5*:

Sr.no.	Methods and description
1	`public void beforeFirst()`: Moves the cursor just before the first row
2	`public void afterLast()`: Moves the cursor just after the last row
3	`public boolean first()`: Moves the cursor to the first row
4	`public void last()`: Moves the cursor to the last row
5	`public boolean absolute(int row)`: Moves the cursor to the specified row
6	`public boolean relative(int row)`: Moves the cursor the given number of rows forward or backward from where it is currently pointing
7	`public boolean previous()`: Moves the cursor to the previous row and returns false if the previous row is off the result set
8	`public boolean next()`: Moves the cursor to the next row and returns false if there are no more rows in the result set
9	`public int getRow()`: Returns the row number that the cursor is pointing to
10	`public void moveToInsertRow()`: Moves the cursor to a special row in the result set that can be used to insert a new row into the database; the current cursor location is remembered
11	`public void moveToCurrentRow()`: Moves the cursor back to the current row if the cursor is currently at the insert row; otherwise, this method does nothing

***Table 2.5**: Navigation methods provided by ResultSet*

- **Viewing a ResultSet**: The **`ResultSet`** interface contains dozens of methods for getting the data of the current row. There is a **`get`** method for each of the possible data types, and each **`get`** method has two versions:
    - One that takes in a column name
    - One that takes in a column index

  For example, if the column you are interested in viewing contains an **`int`**, you need to use one of the **`getInt()`** methods of **`ResultSet`**.

Sr.no.	Methods and description
1	`public int getInt(String columnName) throws SQLException` returns the int in the current row in the column named `columnName`.
2	`public int getInt(int columnIndex) throws SQLException` returns the int in the current row in the specified column index. The column index starts at 1, meaning the first column of a row is 1, the second column of a row is 2, and so on.

***Table 2.6**: Retrieval methods provided by ResultSet*

For each primitive data type; for types like **`Object`**, **`String`**, and **`URL`**; and for SQL data types, there are **`get`** methods.

- **Updating a ResultSet**: The **`ResultSet`** interface contains a collection of update methods for updating the data of a result set.

Sr.no.	Methods and description
1	`public void updateString(int columnIndex, String s) throws SQLException` changes the `String` in the specified column to the value of `s`.
2	`public void updateString(String columnName, String s) throws SQLException` is similar to the previous method, except that the column is specified by its name instead of its index.

***Table 2.7**: Update methods provided by ResultSet*

Updating a row in the result set changes the columns of the current row in the **`ResultSet`** object, but not in the underlying database. To update your changes to the row in the database, you need to invoke one of the following methods:

Sr.no.	Methods and description
1	`public void updateRow()` updates the current row by updating the corresponding row in the database.
2	`public void deleteRow()` deletes the current row from the database.
3	`public void refreshRow()` refreshes the data in the result set to reflect any recent changes in the database.
4	`public void cancelRowUpdates()` cancels any updates made on the current row.
5	`public void insertRow()` inserts a row into the database. This method can only be invoked when the cursor is pointing to the insert row.

Table 2.8: *Update methods for changing row value provided by ResultSet*

Clean up the environment

To clean up the environment, we need to close the open connections, statements, etc. as we explicitly open the connection. It is a good practice to close the connection once the operations on the database are complete.

MetaData

Metadata refers to data about data. To get details about the results received after executing query, we use the **`ResultSetMetaData`**; likewise, to receive more information about the database being used, we can use **`DatabaseMetaData`**. These are provided as interfaces, with several methods that we can explore and use for our purpose.

ResultSetMetaData

MetaData means to get the more detailed data . In terms of the result that we obtain after executing a query, we may need to know column name, column type, total number of columns, etc. We can obtain this data from the methods provided by the **`ResultSetMetaData`** interface. Commonly used methods are listed in *Table 2.9*:

Method	Description
`public int getColumnCount()throws SQLException`	It returns the total number of columns in the `ResultSet` object.
`public String getColumnName(int index)throws SQLException`	It returns the column name of the specified column index.

Method	Description
`public String getColumnTypeName(int index) throws SQLException`	It returns the column type name of the specified index.
`public String getTableName(int index)throws SQLException`	It returns the table name of the specified column index.

***Table 2.9**: Commonly used methods of ResultSetMetaData interface*

DatabaseMetaData

In terms of database, we may need to know database name, version, driver name, etc. This data can be obtained from the methods provided by the **`DatabaseMetaData`** interface shown here:

Method	Description
`public String getDriverName() throws SQLException`	It returns the name of the JDBC driver.
`public String getDriverVersion() throws SQLException`	It returns the version number of the JDBC driver.
`public String getUserName() throws SQLException`	It returns the username of the database.
`public String getDatabaseProductName() throws SQLException`	It returns the product name of the database.
`public String getDatabaseProductVersion() throws SQLException`	It returns the product version of the database.
`public ResultSet getTables(String catalog, String schemaPattern, String tableNamePattern, String[] types) throws SQLException`	It returns the description of the tables of the specified catalog. The table type can be table, view, alias, system table, synonym, etc.

***Table 2.10**: Commonly used methods of DatabaseMetaData interface*

Creating application to access the database using IDE

As we have covered the basic tokens required to start communication with the database, now we can build an application using an IDE. For the applications

considered in this book, we use NetBeans, as it has easy interface for performing the required tasks.

Creating a table in Oracle from Java

The steps to create a Java application using NetBeans are listed here:

1. Open IDE, click on the **File** menu and select **New Project...**; refer to the following figure:

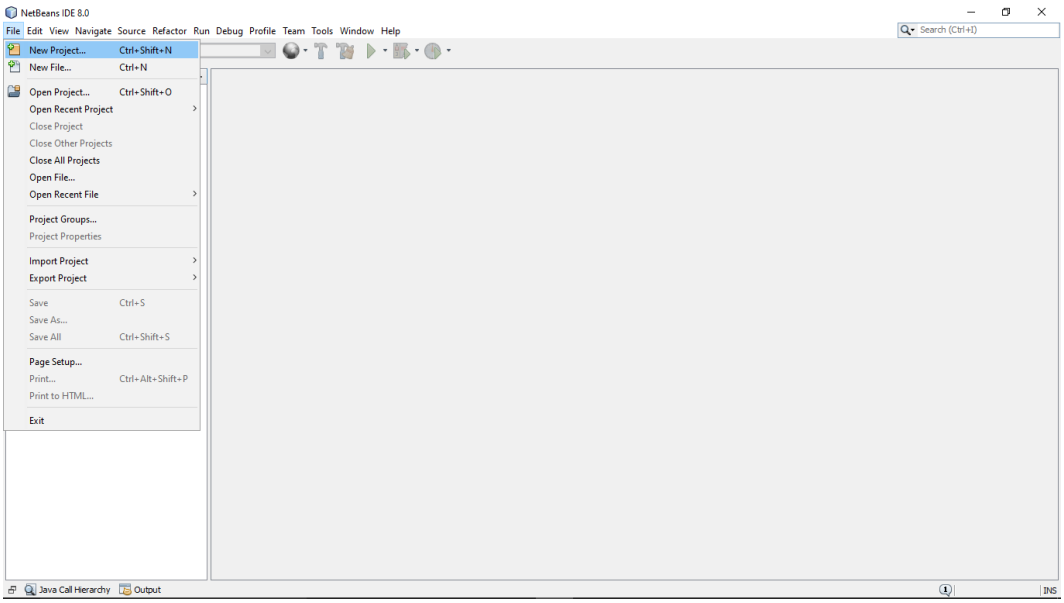


Figure 2.7: *Snapshot of NetBeans 8 to open File menu select new Project*

2. Select category as **Java** and within **Projects**, select **Java application**:

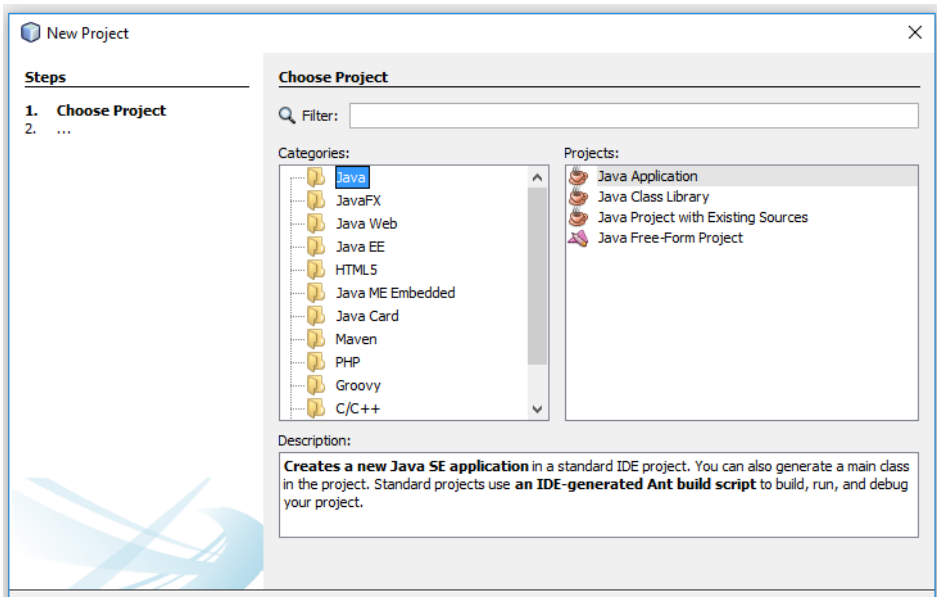


***Figure 2.8**: Snapshot of NetBeans 8 to select category and type of project under the category*

3. Click on the **Next** button to specify the project name and the location for storing the application:

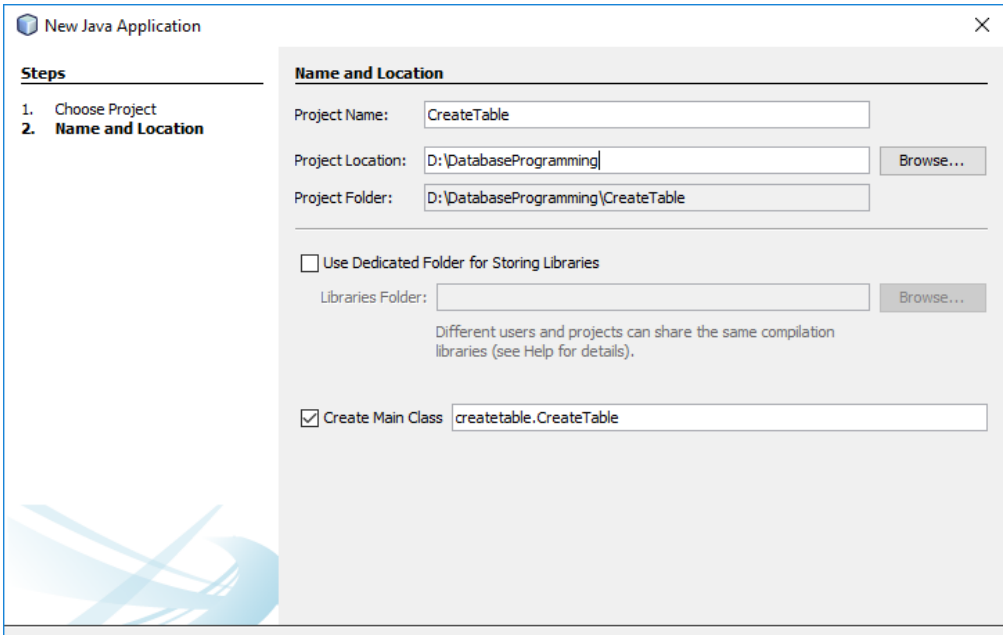


***Figure 2.9**: Snapshot of NetBeans 8 to specify project name and location*

4. Click on finish, and you will find the directory structure created for a Java application; explore the project window.

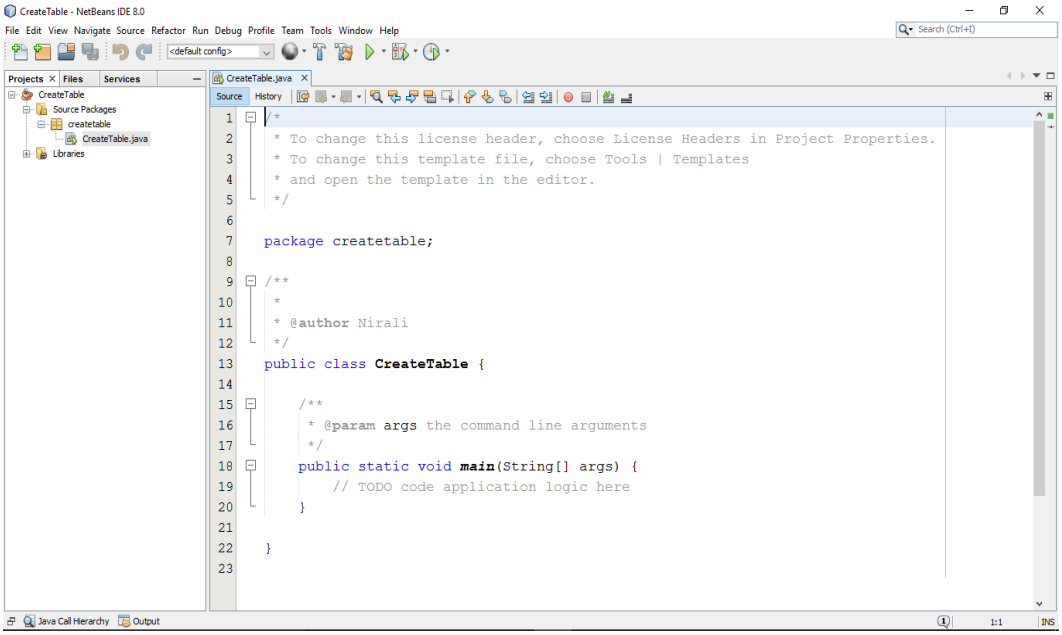


***Figure 2.10**: Snapshot of NetBeans 8 which shows the Java application*

The IDE creates a blank Java class, which we need to write as per our requirements. Let's consider creating a table in Oracle 10g with the below fields and datatype.

5. Using JDBC, create a table called **`CONTACT_INFO`** with the following fields:

```
01.  CONTACT_ID     INTEGER            NOT NULL          PRIMARY KEY,
02.  FIRST_NAME     VARCHAR(20)      NOT NULL
03.  MIDDLE_NAME    CHAR(1)
04.  LAST_NAME      VARCHAR(30)      NOT NULL
05.  STREET          VARCHAR(50)      NOT NULL
06.  CITY            VARCHAR(30)      NOT NULL
07.  STATE           VARCHAR(30)      NOT NULL
08.  ZIP             VARCHAR(6)       NOT NULL
09.
```

***Figure 2.11**: Table Contact_Info structure*

As per our discussion in the previous section, we will follow the given steps to communicate with the database:

- Import the required packages from the **`sql`** package, as shown in the following figure:

```
01. import java.sql.Connection;
02. import java.sql.DriverManager;
03. import java.sql.SQLException;
04. import java.sql.Statement;
```

***Figure 2.12**: Import statements to manage database from Java*

- Register the JDBC driver, as shown here:

```
01. String driver = "oracle.jdbc.OracleDriver";
02. Class.forName(driver);
```

***Figure 2.13**: Register the driver*

- Open a connection, as shown in the following figure:

```
01. String url = "jdbc:oracle:thin:@localhost:1521:xe";
02. String username = "system";
03. String password = "oracle";
04. Connection con = DriverManager.getConnection(url, username, password);
05. System.err.println("Connection Established");
```

***Figure 2.14**: Create the connection*

- Create a statement, as follows:

```
Statement stmt = con.createStatement();
String create_query = "CREATE TABLE CONTACT_INFO("
                    + "CONTACT_ID INTEGER NOT NULL, "
                    + "FIRST_NAME VARCHAR(20) NOT NULL, "
                    + "MIDDLE_NAME CHARACTER(1), "
                    + "LAST_NAME VARCHAR(30) NOT NULL, "
                    + "STREET VARCHAR(50) NOT NULL, "
                    + "CITY VARCHAR(30) NOT NULL, "
                    + "STATE VARCHAR(30) NOT NULL, "
                    + "ZIP VARCHAR(6) NOT NULL, " + "PRIMARY KEY
(CONTACT_ID) " + " ) ";
```

```
Statement stmt = con.createStatement();
String create_query = "CREATE TABLE CONTACT_INFO("
                    + "CONTACT_ID INTEGER NOT NULL, "
                    + "FIRST_NAME VARCHAR(20) NOT NULL, "
                    + "MIDDLE_NAME CHARACTER(1), "
                    + "LAST_NAME VARCHAR(30) NOT NULL, "
                    + "STREET VARCHAR(50) NOT NULL, "
                    + "CITY VARCHAR(30) NOT NULL, "
                    + "STATE VARCHAR(30) NOT NULL, "
                    + "ZIP VARCHAR(6) NOT NULL, " + "PRIMARY KEY (CONTACT_ID) " + " ) ";

```

***Figure 2.15**: Create a query from Java*

- Execute a query and clean up the environment:

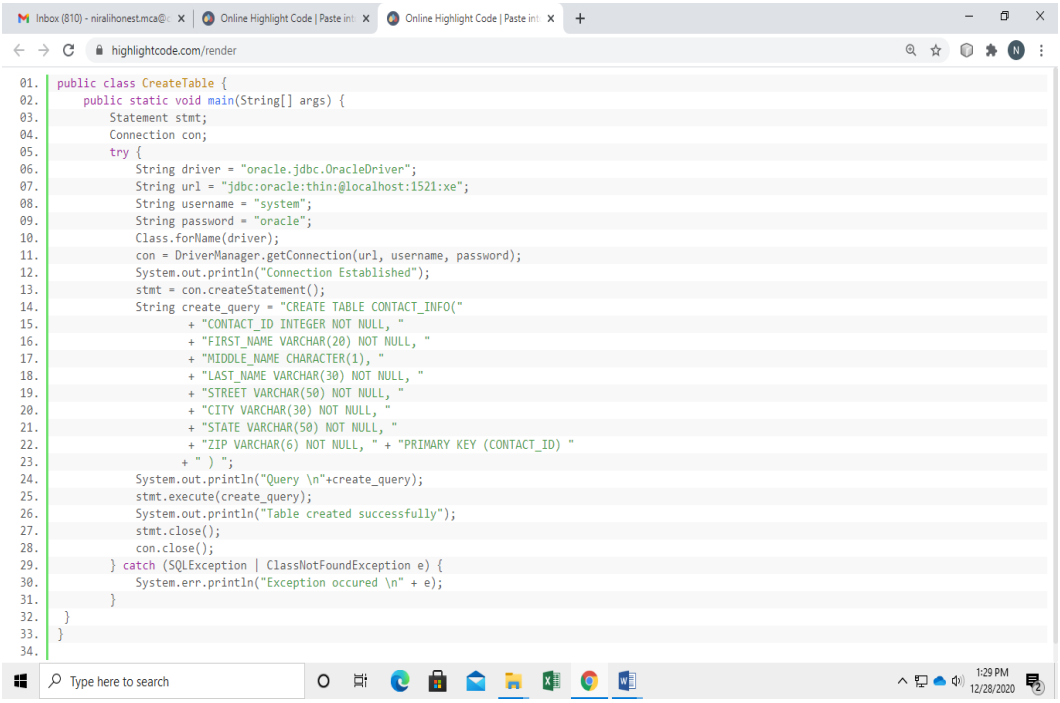
```
stmt.execute(create_query);
stmt.close();
```

```
stmt.execute(create_query);
```

```
stmt.close();
```

***Figure 2.16**: Execute query and close the statement*

We can encapsulate all the steps and place them in a single application that we have created, as shown in the following code snippet:




```
public class CreateTable {
    public static void main(String[] args) {
        Statement stmt;
        Connection con;
        try {
            String driver = "oracle.jdbc.OracleDriver";
            String url = "jdbc:oracle:thin:@localhost:1521:xe";
            String username = "system";
            String password = "oracle";
            Class.forName(driver);
            con = DriverManager.getConnection(url, username, password);
            System.out.println("Connection Established");
            stmt = con.createStatement();
            String create_query = "CREATE TABLE CONTACT_INFO("
                    + "CONTACT_ID INTEGER NOT NULL, "
                    + "FIRST_NAME VARCHAR(20) NOT NULL, "
                    + "MIDDLE_NAME CHARACTER(1), "
                    + "LAST_NAME VARCHAR(30) NOT NULL, "
                    + "STREET VARCHAR(50) NOT NULL, "
                    + "CITY VARCHAR(30) NOT NULL, "
                    + "STATE VARCHAR(50) NOT NULL, "
                    + "ZIP VARCHAR(6) NOT NULL, " + "PRIMARY KEY (CONTACT_ID) "
                   + " ) ";
            System.out.println("Query \n"+create_query);
            stmt.execute(create_query);
            System.out.println("Table created successfully");
            stmt.close();
            con.close();
        } catch (SQLException | ClassNotFoundException e) {
            System.err.println("Exception occured \n" + e);
        }
    }
}
```

***Figure 2.17**: Entire program to execute the table creation from Java*

When driver class is not found, the **ClassNotFoundException** is thrown; after the driver file is added, any problem in connecting to database or any communication with database is handled by **SQLException**. Note that it is good practice to write the code in try catch block as the code throws an exception. Also, we can include a finally block to place the cleanup code. Use intermediate **System.out.println** statements to debug when you find an exception. After successful execution of the program, remove the unnecessary **print** statements.

The mentioned program, when executed successfully, will create a table named **contact_info** in Oracle. You can check the table in Oracle. Once the program is executed and the table is created, you cannot execute it again, as it will try to create a table again. It has already created one, and objects with the same name cannot be created, so an **SQLException** will be thrown. As programmers, it is our responsibility to check whether the name of the table exists before we create it. So, we can alter the given code and add to check whether the table exists. If it exists, we want to delete it and then create our new table. We can use the **getTables** method of **DatabaseMetaData** to know whether the table exists. The following code snippet show how to create a table in Oracle from Java after checking that the table exists:

```
01.   DatabaseMetaData dbm = con.getMetaData();
02.   ResultSet rs = dbm.getTables(null, null, "CONTACT_INFO", null);
03.
04.   if (rs.next())
05.   {
06.       System.out.println("Table exists");
07.       stmt.execute("DROP TABLE CONTACT_INFO");
08.    }
09.       System.out.println("Table does not exist");
10.       System.out.println("Table \n" + create_query);
11.
12.       Boolean flag = stmt.execute(create_query);
13.       System.out.println("Table created successfully");
14.
```

***Figure 2.18**: Check the existence of table using DatabaseMetaData*

If the table exists, the **ResultSet** object retrieves the value and the table is dropped and then created; otherwise, the table is created directly.

In order to execute the program correctly, you need to add a JAR file for the Oracle driver. In the example, we have used the **ojdbc6.jar** file. You can add a JAR file to the project, as shown here:

1. In the project tab, select the project to which you want to add a JAR file.
2. Right-click on the **Libraries** tab and select **Add JAR**; then, browse and open the JAR file, as shown in the following figure:

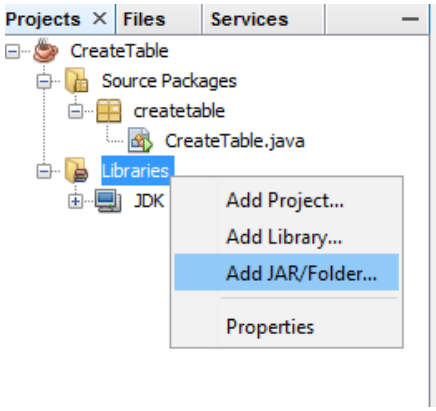


***Figure 2.19**: Snapshot of NetBeans 8 to add a jar file in the project*

3. After adding the JAR file, compile and run the application, as shown in the following figure:

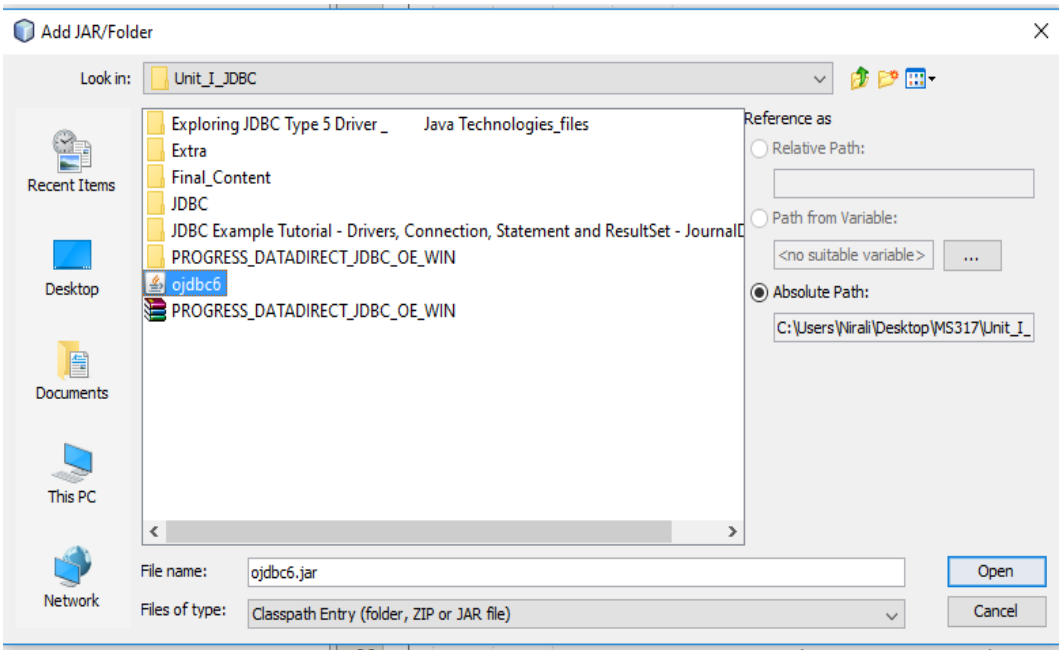


***Figure 2.20**: Snapshot of NetBeans 8 to browse and add a jar file in the project*

To check the table in Oracle, open the oracle home page and log in with your credentials. Select the command prompt and check for the table structure created.

4. The following figure shows the Oracle 10g express edition home page:

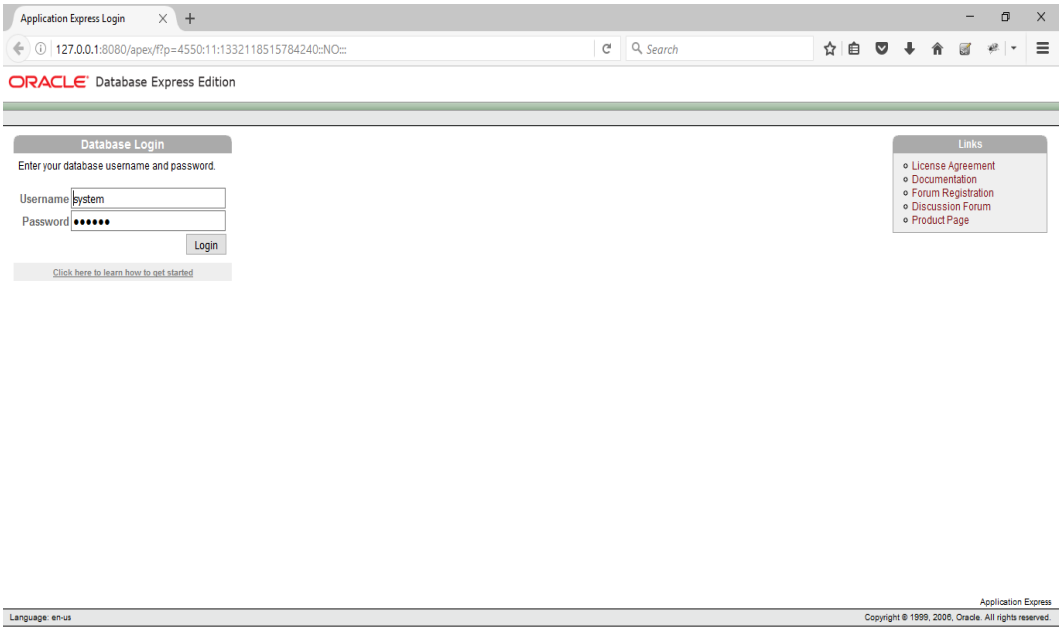


***Figure 2.21**: Snapshot of Oracle 10g express edition home page*

5. The following figure shows the Oracle 10g express edition Dashboard; select SQL:

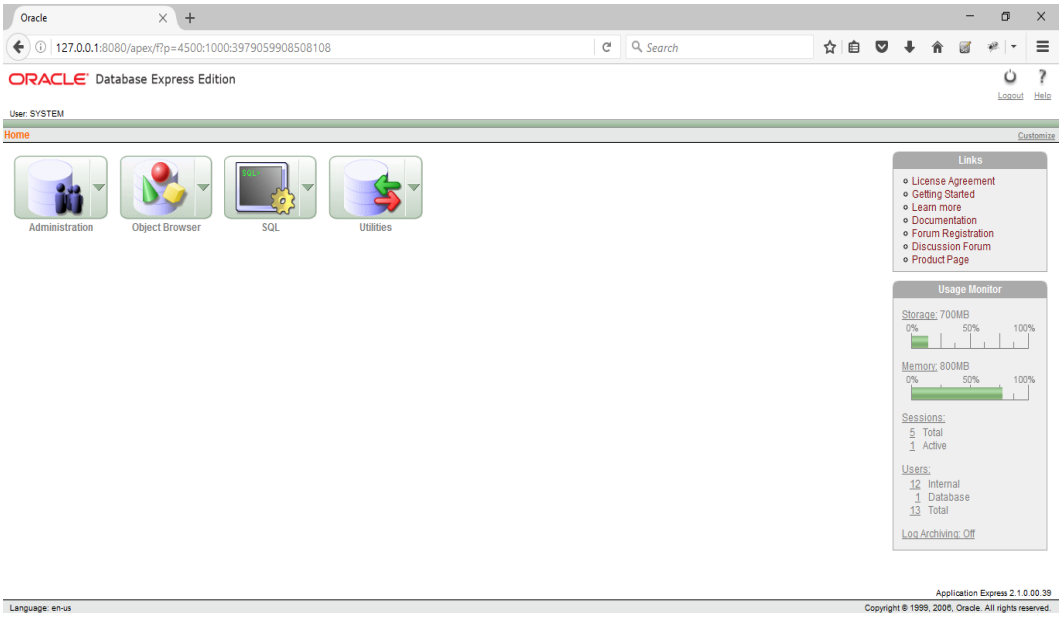


***Figure 2.22**: Snapshot of Oracle 10g express edition Dashboard, select SQL*

6. The following figure shows the Oracle 10g express edition **Dashboard**; select **SQL Commands**:

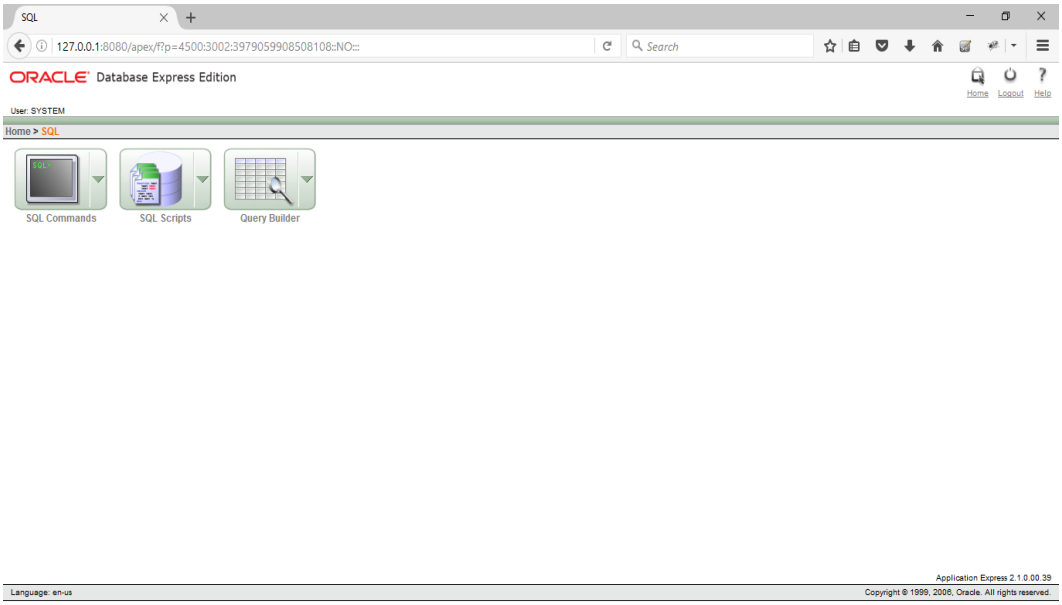


***Figure 2.23**: Snapshot of Oracle 10g express edition Dashboard, select SQL Commands*

7. The following figure shows Oracle 10g express edition **Dashboard**; type and run **SQL Commands**:

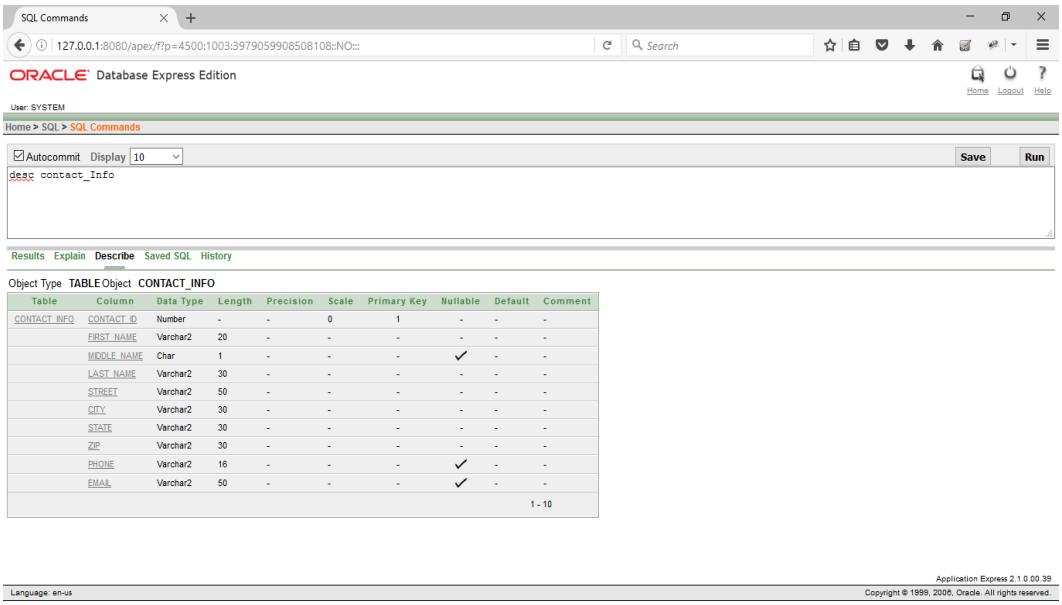


***Figure 2.24**: Snapshot of Oracle 10g express edition Dashboard, type and run SQL Commands*

Alter table in Oracle from Java

Create a new application and write the Java code to alter the table created in the previous section to include the following fields:

```
'PHONE' varchar(16)
'EMAIL' varchar(50)
```

The following code snippet shows how to alter a table in Oracle from Java:

```
01. package altertable;
02. import java.sql.Connection;
03. import java.sql.DatabaseMetaData;
04. import java.sql.DriverManager;
05. import java.sql.ResultSet;
06. import java.sql.SQLException;
07. import java.sql.Statement;
08.
09. public class AlterTable {
10.     public static void main(String[] args) {
11.         try {
12.             String driver = "oracle.jdbc.OracleDriver";
13.             String url = "jdbc:oracle:thin:@localhost:1521:xe";
14.             String username = "system";
15.             String password = "oracle";
16.
17.             Class.forName(driver);
18.
19.             Connection con = DriverManager.getConnection(url, username, password);
20.             System.out.println("Connection Established");
21.
22.             Statement stmt = con.createStatement();
23.             String alter_query = "ALTER TABLE CONTACT_INFO"
24.                                  + " ADD("
25.                                  + "PHONE VARCHAR(16),"
26.                                  + " EMAIL VARCHAR(50))";
27.
28.             System.out.println("Query \n"+alter_query);
29.             stmt.execute(alter_query);
30.             System.out.println("Table altered successfully");
31. stmt.close();
32.             con.close();
33.         } catch (SQLException | ClassNotFoundException e) {
34.             System.err.println("Exception occured \n" + e);
35.         }
36.     }
37. }
38.
```

***Figure 2.25**: Write a alter query from Java*

After the successful execution of this code, the table is altered to include two new fields in the **`contact_info`** table.

Insert records in table from Java

Create a new application and write the Java code to insert records in the table created in the previous section. The following code snippet shows how to insert static records in a table in Oracle from Java:

```
01. package insertrecords;
02. import java.sql.Connection;
03. import java.sql.DriverManager;
04. import java.sql.ResultSet;
05. import java.sql.SQLException;
06. import java.sql.Statement;
07. 
08. public class InsertRecords {
09.     public static void main(String[] args) {
10.         try {
11.             String driver = "oracle.jdbc.OracleDriver";
12.             String url = "jdbc:oracle:thin:@localhost:1521:xe";
13.             String username = "system";
14.             String password = "oracle";
15. 
16.             Class.forName(driver);
17.             Connection con = DriverManager.getConnection(url, username, password);
18.             System.out.println("Connection Established");
19. 
20.             Statement stmt = con.createStatement();
21.             String insert_query = "
22.             INSERT INTO CONTACT_INFO VALUES("
23. + "1 ," + "'Akash', 'B' , 'Patel','Near Valetva','Changa',
24.                     +'Gujarat','388421','02697265242','abc@gmail.com')";
25. 
26.             System.out.println("Query \n"+insert_query);
27.             stmt.execute(insert_query);
28.             System.out.println("Records inserted successfully");
29. 
30.         } catch (SQLException | ClassNotFoundException e) {
31.             System.err.println("Exception occured \n" + e);
32.         }
33.     }
34. }
```

***Figure 2.26**: Write an insert query from Java*

Here, the record inserted in the table has values, which are statically defined, meaning the values are written directly in the **`insert`** query. Often, we may have to insert records in the table, which are not known in advance when the **`insert`** query is written. In such a case, it is always desirable to write a dynamic query using the **`prepared`** statement. We can write the **`insert`** statement to include a dynamic

query; for that, you need to add the **`import`** statement, as shown in the following code snippet:

```
01.             import java.sql.PreparedStatement;
02.
03.             String pinsert_query="INSERT INTO CONTACT_INFO VALUES (?,?,?,?,?,?,?,?,?,?)";
04.             PreparedStatement pstmt=con.prepareStatement(pinsert_query);
05.
06.             pstmt.setInt(1,2);
07.             pstmt.setString(2,"Akshay");
08.             pstmt.setString(3,"H");
09.             pstmt.setString(4,"Bhatt");
10.             pstmt.setString(5,"Opp. Big Bazar");
11.             pstmt.setString(6,"Anand");
12.             pstmt.setString(7,"Gujarat");
13.             pstmt.setString(8,"388001");
14.             pstmt.setString(9,"0269225245");
15.             pstmt.setString(10,"xyz@gmail.com");
16.
17.             int i=pstmt.executeUpdate();
18.             System.out.println(i+" records inserted");
19.
20.
```

***Figure 2.27**: Write a parameterized query from Java*

Here, the insert statement includes placeholders **`?`**, which are replaced by values set using the setter methods. The first argument in methods indicates the placeholder you want to set for each column, and the second argument is the value you want to set for each column. We can use the appropriate setter method based on the type of column value we want to set. The **`?`** allows us to provide the value at the time of execution. We need not provide the values in the query while writing it; instead, they can be provided during execution, and the query can be replaced with the values given as parameters. This allows a way to write queries with dynamic values.

Update records in table from Java

Using JDBC, update city as Surat and PIN code value as 395003 where city is Anand, and display the number of rows updated.

Note: Before updating the original table, `contact_info` takes a backup.

1. Create the **`contact_info_old`** table as **`select * from contact_info order by contact_id`**.
2. If the changes are done and you want to retrieve the earlier records, run the following query:

   Delete from **`contact_infos`** table.

3. Insert records into the **`contact_info`** table using the **`select * from contact_info_old`** command.

The following code snippet shows how to update records in a table in Oracle from Java:

```
01. package updaterecord;
02.
03. import java.sql.Connection;
04. import java.sql.DriverManager;
05. import java.sql.ResultSet;
06. import java.sql.Statement;
07.
08. public class UpdateRecord {
09.     public static void main(String[] args) {
10.         String driver = "oracle.jdbc.OracleDriver";
11.             String url = "jdbc:oracle:thin:@localhost:1521:xe";
12.             String username = "system";
13.             String password = "oracle";
14.             try {
15.             Class.forName("driver");
16.            Connection con = DriverManager.getConnection("url", "username","password");
17.           Statement stmt = con.createStatement();
18.          String select_query = "select * from contact_info order by contact_id";
19.
20.        System.out.println("Query \n"+select_query);
21.
22.       ResultSet rs= stmt.executeQuery(select_query);
23.
24.             while(rs.next())
25.             {
26.               System.out.println(rs.getInt(1)+" "+rs.getString(2)+" "+rs.getString(3)+" "+rs.getString(4)+" "+rs.getString(5)+" "+rs.getString(6)+"
27.               "+rs.getString(7)+" "+rs.getString(8)+" "+rs.getString(9)+" "+rs.getString(10)+" \n ");
28.             }
29.
30.             Statement ustmt=con.createStatement();
31.             String update_query="UPDATE CONTACT_INFO "
32.                                 + "SET CITY='SURAT',"
33.                                 + "ZIP='395003' "
34.                                 + "WHERE CITY='CHANGA'";
35.             System.out.println(update_query);
36.             int i=ustmt.executeUpdate(update_query);
37.
38.             ResultSet rs1= ustmt.executeQuery("SELECT * FROM CONTACT_INFO");
39.             System.out.println("Total : "+i+" no. of rows updated.\n");
40.
41.             while(rs1.next())
42.             {
43.                 System.out.println(rs1.getInt(1)+""+rs1.getString(2)+""+rs1.getString(3)+""+rs1.getString(4)+" "+rs1.getString(5)+" "+rs1.getString(6)+"
44.                 "+rs1.getString(7)+""+rs1.getString(8)+" "+rs1.getString(9)+" "+rs1.getString(10)+" \n ");
45.             }
46.         } catch (Exception  e) {
47.             System.err.println("Exception occured \n" + e);
48.         }
49.     }
50. }
```

***Figure 2.28**: Write a select query from Java*

The preceding code finds the record where city is **`changa`** and updates it with **`surat`**, and it updates ZIP to "**`395003`**".

The **`executeUpdate`** method returns the number of rows affected by the query. So, you can print the number of rows changed because of the **`update`** query.

Delete records in table from Java

Using JDBC, delete a record having contact ID as **7**. For this type of query, only one record will be affected if the ID is present, or an exception will be thrown if the ID is not present. The following code snippet shows how to delete a record from a table in Oracle from Java:

```
01. package deleterecord;
02. import java.sql.Connection;
03. import java.sql.DriverManager;
04. import java.sql.ResultSet;
05. import java.sql.Statement;
06. 
07. public class UpdateRecord {
08.     public static void main(String[] args) {
09.         String driver = "oracle.jdbc.OracleDriver";
10.             String url = "jdbc:oracle:thin:@localhost:1521:xe";
11.             String username = "system";
12.             String password = "oracle";
13.             try {
14.             Class.forName("driver");
15.            Connection con = DriverManager.getConnection("url", "username","password");
16.           Statement stmt = con.createStatement();
17.          String delete_query = "delete from contact_info where contact_id=7";
18. 
19.         System.out.println("Query \n"+delete _query);
20.            int i=ustmt.executeUpdate(delete_query);
21.                         System.out.println("Total : "+i+" no. of rows deleted.\n");
22. 
23.         } catch (Exception  e) {
24.             System.err.println("Exception occurred \n" + e);
25.          }
26.     }
27. }
28. 
```

***Figure 2.29**: Write a delete query from Java*

The following figure shows the records of the table before running the **`delete`** query:

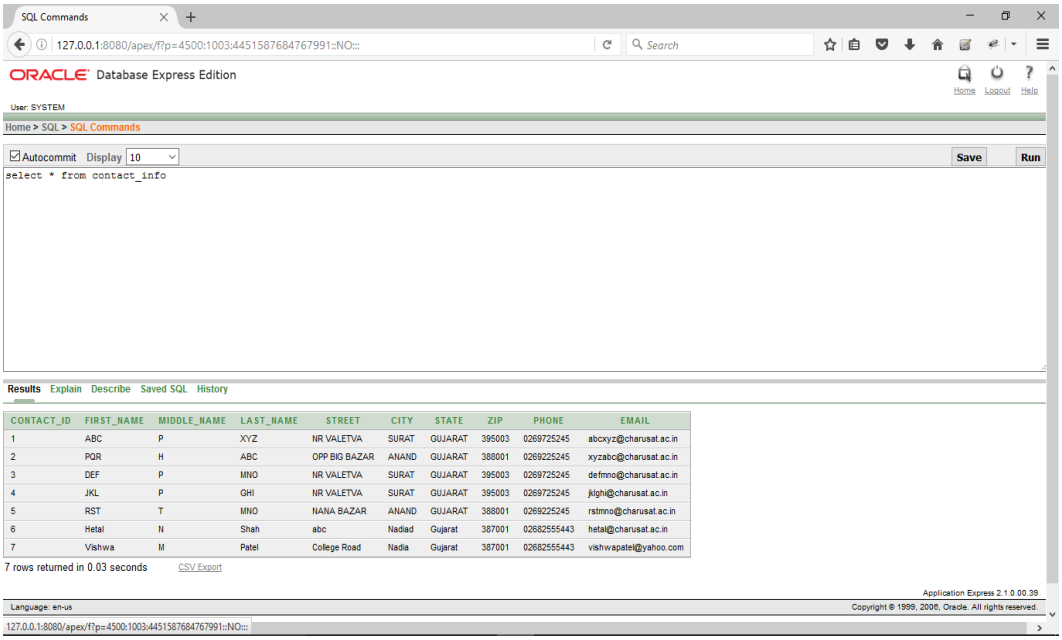


***Figure 2.30**: Seven records of the table before delete query*

The following figure shows the records of the table after running the **`delete`** query:

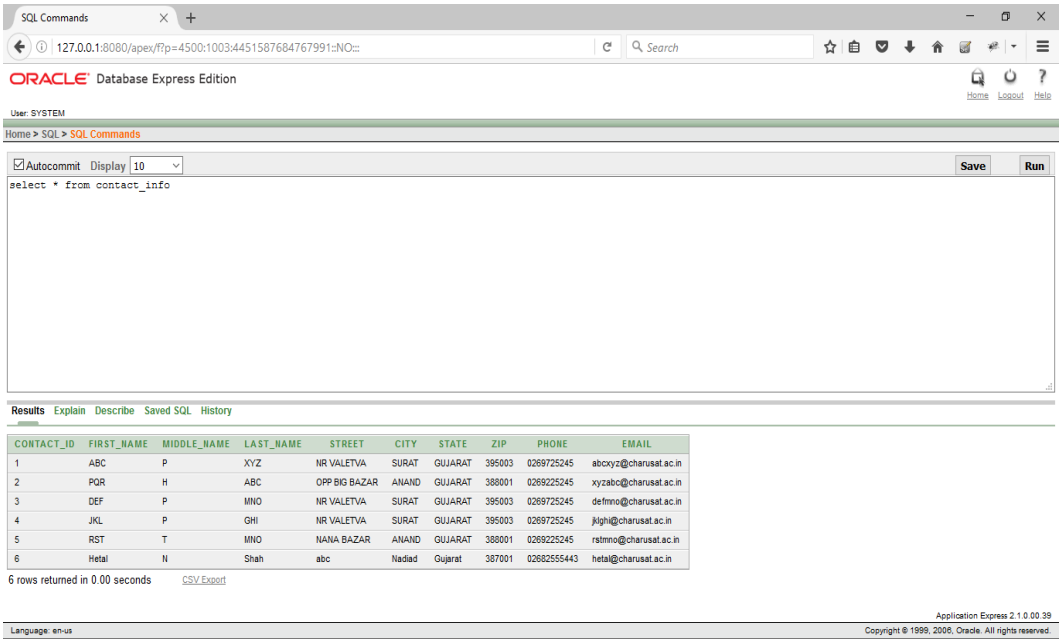


***Figure 2.31**: Six records of the table after delete query*

Conclusion

The chapter covered the steps to connect a database to a Java application. JDBC API components with its methods were described briefly to help you understand their purpose and syntax before using them to build a program. The basic database operations like inserting records, updating records, selecting records, traversing on the `ResultSet`, and deleting records were covered with suitable examples.

In the next chapter, we will see how we can separate the database attributes into a different file and access it from main code. We will also see how we can perform batch operations.

Points to remember

- The basic steps to establish the database connection include importing the packages to support the libraries to allow database management, registering the JDBC drivers, establishing a connection, creating a statement, executing a query, fetching the result from the result set and closing the connections when.
- The metadata about retrieved results and database itself can be accessed from the interface `ResultSetMetaData` and `DatabaseMetaData`.

Multiple choice questions

1. **Which of the following holds data retrieved from a database after you execute an SQL query using Statement objects?**
    a. ResultSet
    b. JDBC driver
    c. Connection
    d. Statement

2. **Which of the following is correct for the `Class.forName()` method call?**
    a. This method dynamically loads the driver's class file into memory, which automatically registers it.
    b. This method is preferable because it allows you to make the driver registration configurable and portable.
    c. Both of the above
    d. None of the above

3. Which packages contain the JDBC classes?

a. java.jdbc and javax.jdbc
b. java.jdbc and java.jdbc.sql
c. java.sql and javax.sql
d. java.rdb and javax.rdb

4. Which of the following statements about the different types of statements in JDBC is false?

a. Statement
b. Prepared Statement
c. Callable Statement
d. Interim Statement

5. Which of the following manages a list of database drivers in JDBC?

a. DriverManager
b. JDBC driver
c. Connection
d. Statement

6. In which of the following type of `ResultSet`, can the cursor scroll forward and backward, and the result set created is sensitive to changes made by others to the database that occur after the result set was created?

a. ResultSet.TYPE_FORWARD_ONLY
b. ResultSet.TYPE_SCROLL_INSENSITIVE
c. ResultSet.TYPE_SCROLL_SENSITIVE
d. None of the above

7. When is the message No Suitable Driver shown?

a. When the driver is not registered by the **`Class.forname()`** method
b. When the usernames, password and the database does not match
c. When the JDBC database URL passed is not constructed properly
d. When the type 4 driver is used

8. **_____ updates the current row by updating the corresponding row in the database.**
    a. insertRow()
    b. updateRow()
    c. refreshRow()
    d. deleteRow()

Answers

1. **a**
2. **c**
3. **c**
4. **d**
5. **a**
6. **b**
7. **c**
8. **b**

Questions

1. Explain the methods that can be used to establish JDBC connection (`DriverManager.getConnection()` and other approaches of `getConnection()`).
2. What is JDBC `Statement`? Explain the types of `Statement`.
3. Explain the three execute methods with examples.
4. How many types of methods does the `Result` interface provide?
5. Explain any three navigation methods provided by the `Result` interface.
6. Explain any three `get` methods provided by the `Result` interface.
7. Explain any three update methods provided by the `Result` interface.
8. List and explain the types of `ResultSet`.
9. List and explain the concurrency types supported by `ResultSet`.
10. What is `ResultSetMetaData`? Explain any three methods provided by `ResultSetMetaData`.

Key terms

- **Database URL**: A database URL is an address that points to your database.
- **Statement**: It is used for building and submitting an SQL statement to the database.
- **ResultSet**: The SQL statements that read data from a database query and returns the data in a **`ResultSet`**.
- **Navigational methods**: These methods allow you to move the cursor to the desired position in the retrieved result set.
- **Get methods**: These methods allow you to fetch or retrieve the values from the result set.
- **Update methods**: These methods allow you to update the content in the result set.

Join our book's Discord space

Join the book's Discord Workspace for Latest updates, Offers, Tech happenings around the world, New Release and Sessions with the Authors:

https://discord.bpbonline.com

Chapter 3
Creating Properties File and Performing Batch Operations

Introduction

This chapter covers the concept of properties file to showcase how one can separate the database connectivity code to apply changes easily when the database properties change. The next part of the chapter covers the concept of JDBC transactions to understand how the transactions can be rolled back to previously set point. The last part of the chapter covers how multiple transactions can be carried out in batches.

Structure

In this chapter, we will cover the following topics:

- Properties file
- Constructors and methods provided by the properties class
- Properties file example
- JDBC Transactions
    - Commit and rollback
    - Savepoints
    - Batch processing in JDBC

    - Application of methods
- Conclusion

Objectives

After studying this chapter, you should be able to understand the concept of properties file; use properties file in program code; perform batch transactions; apply commit, rollback and generate save points; and perform all operations on transactions.

Introduction to properties file

The **`java.util.Properties`** class represents a persistent set of properties The properties object contains key and value pair as a string. The **`Properties`** class is the subclass of **`Hashtable`**. It can be used to get property value based on the property key.

The main advantage of using properties file is that if any information is changed from the file, we don't need to recompile the Java class. It is used to store information that is to be changed frequently.

Constructors and methods of properties class

The **`Properties`** class provides methods to get data from properties file and store data into properties file. Moreover, it can be used to get the properties of application currently executed **`Properties`** defines an instance variable, which holds a default property list associated with a **`Properties`** object.

- **Constructors**: The following listed constructors are invoked to instantiate the properties object:
    - **`Properties()`**: This creates a **`Properties`** object that has no default values.
    - **`Properties(Properties propDefault)`**: This creates an object that uses **`propDefault`** for its default value.

- **Methods**: The commonly used methods of the **`Properties`** class are given in the following table.

Method	Description
`public void load (Reader r)`	Loads data from the `Reader` object
`public void load (InputStream is)`	Loads data from the `InputStream` object
`public String getProperty (String key)`	Returns value based on the key
`public void setProperty (String key,String value)`	Sets the property in the properties object
`public void store (Writer w, String comment)`	Writers the properties in the writer object
`public void store (OutputStreamos, String comment)`	Writes the properties in the `OutputStream` object
`storeToXML(OutputStreamos, String comment)`	Writers the properties in the writer object for generating an XML document
`public void storeToXML(Writer w, String comment, String encoding)`	Writers the properties in the writer object for generating XML document with specified encoding

Table 3.1: *Methods of Properties class*

Properties file example

In this example, we will show how to create a properties file to hold the database properties and to access this file from Java code. Create a new project as Java application with the name **`PropertyFile`**, right-click on the **`propertyfile`** package in **Source Packages**, and then click on **New** and select **Others...**; in **File** types, select **Properties File**.

We can add the properties file, as shown in the following figure, by selecting new and then **Other...**, as it is not available in the popup menu:

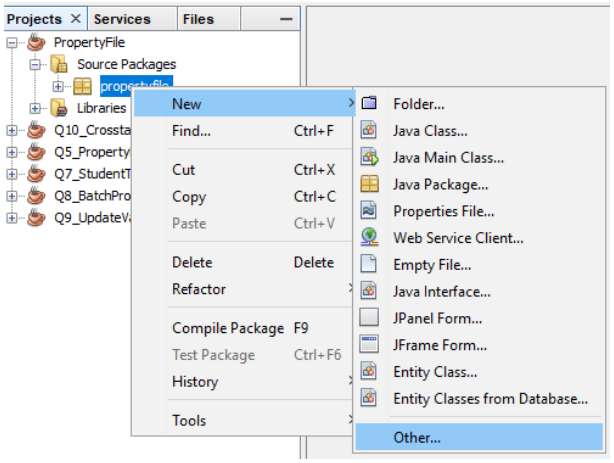


Figure 3.1: *Snapshot to select New -> Others*

In the others category, select the **Properties** file, as shown in the following figure:

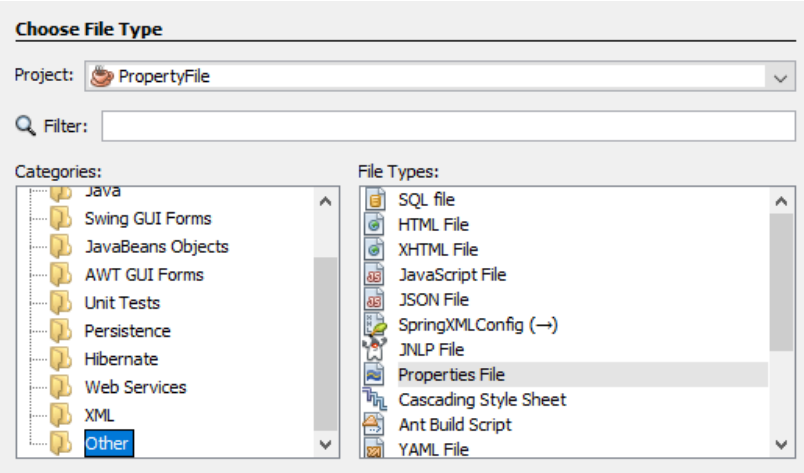


Figure 3.2: *In categories select Other -> Properties File*

Provide the name of the properties file, as shown in the following figure:

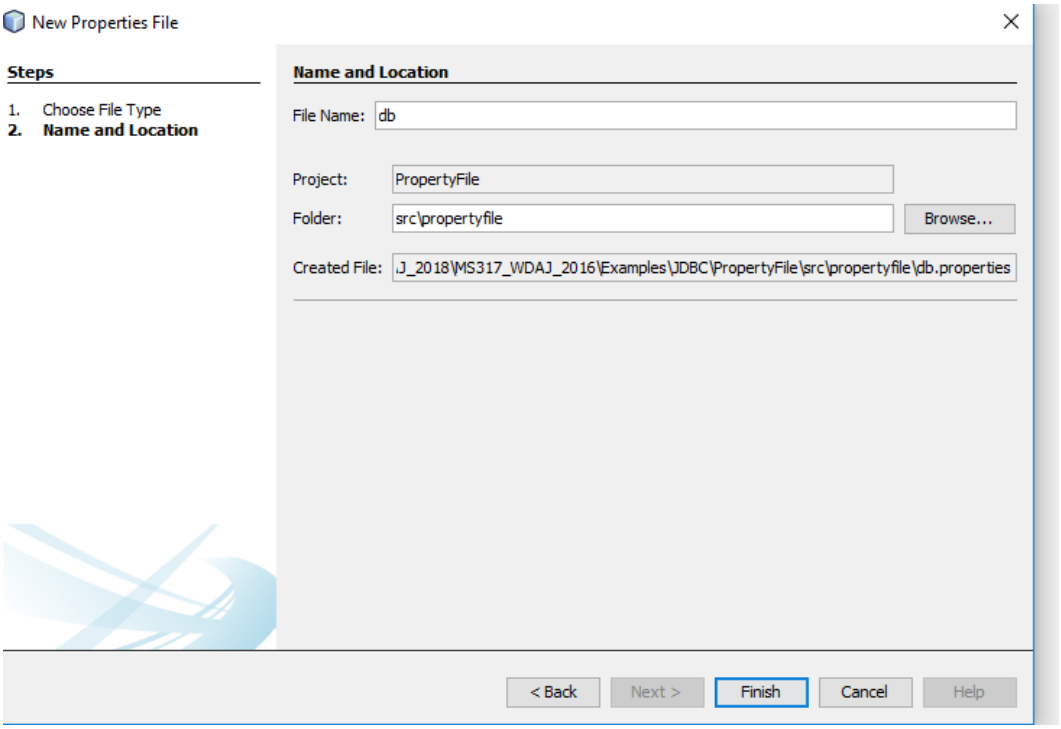


Figure 3.3: *Give the name of the properties file*

The newly created properties file will be as follows:

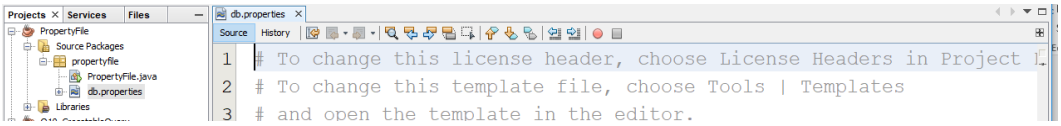


Figure 3.4: *Snapshot of New db.properties file created*

We can then add the key-value pair in the properties file, as shown here:

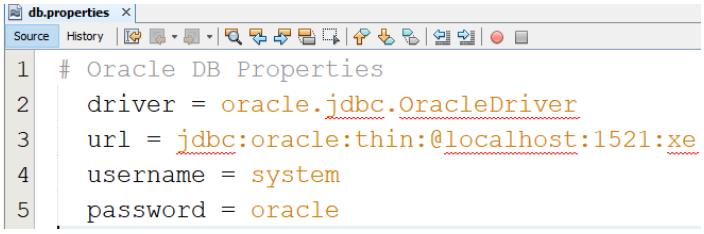


Figure 3.5: *Snapshot to add the content in terms of key-value pairs in the file db.properties*

Now we are writing the Java code to connect to the database using properties file and display the records in a formatted way. In this example, we are using the **`executeQuery`** method of the **`Statement`** class to select the data from above table into result set and display first name, last name, phone and email as formatted output. To display the content formatted as well as in different colors, use the following syntax:

Syntax for coloring output: **`Escape character[<<code>>m <<output text>>`**

Output text is the output that should be printed on the console. The following table represents the code effect that can be applied on console:

Code	Effect
0	Reset all the formatting
1	Bold
3	Italics
4	Underline
30-37	Each number represents one colors, i.e., 30 is for Black, 31 for Red, 32 for Green, 33 for Yellow, etc.

Table 3.2: *Code to set the output text from console*

Note that DOS doesn't support ANSI colors. You can use Linux terminal or NetBeans IDE to run this program. And *Escape* character in Java is ASCII code 33 (that is; \033); we use **`printf`** for formatted output.

We need to open the **`db.properties`** file for reading, so we take an object of **`FileInputStream`** and mention the absolute path of the **`db.properties`** file. We create a **`Properties`** class object using the default constructor and load the **`properties`** file using the **`load()`** method. Once the properties file is loaded, we can access the **`values`** described in the file using the relevant **`keyname`** in the **`getProperty()`** method.

We can access the properties file as shown in the following figure:

```
01. package propertyfile;
02. import java.io.FileInputStream;
03. import java.sql.Connection;
04. import java.sql.DriverManager;
05. import java.sql.ResultSet;
06. import java.sql.Statement;
07. import java.util.Properties;
08.
09. public class PropertyFile
10. {
11.    public static void main(String[] args)
12. {
13.         Properties props = new Properties();
14. FileInputStreamfis = null;
15.         Connection con = null;
16.         try
17. {
18. fis = new FileInputStream("D:\\PropertyFile\\src\\propertyfile\\db.properties");
19. props.load(fis);
20.
21. Class.forName(props.getProperty("driver"));
22.
23.             con = DriverManager.getConnection(props.getProperty("url"),
24. props.getProperty("uname"),
25. props.getProperty("pwd"));
26.
27.              Statement stmt =con.createStatement(ResultSet.TYPE_SCROLL_SENSITIVE,
28. ResultSet.CONCUR_UPDATABLE);
29.
30.             String select_query = "SELECT * FROM CONTACT_INFO";
31.
32. System.out.printf("Query :  " + select_query + "\n\n");
33.
34. ResultSetrs = stmt.executeQuery(select_query);
35.
36. System.out.printf("\033[34;1mFirst Name\033[0m \t\t\033[34;1mLast Name\033[0m\t\t
37. \033[34;1mPhone\t\t\t   \033[34;1mEmail Address \t\t \n");
38.
39.          while (rs.next())
40. {
41. System.out.printf("%s\t\t        %s\t\t         %s\t\t   %s\t\t \n ", rs.getString(2), rs.getString(4),
42. rs.getString(9), rs.getString(10));
43.            }
44.
45. System.out.printf("\n\n\n");
46.         }
47.  catch (Exception e)
48.  {
49. System.err.println("Exception occured \n" + e);
50.         }
51.     }
52.
53. }
54.
```

Figure 3.6: *Code to access the properties file and perform the select query operation using formatted output*

After writing this code in the main class, run the Java application to view the output as follows:

```
run:
Connection Established
Query :  SELECT * FROM CONTACT_INFO

First Name      Last Name       Phone           Email Address
 ABC              XYZ            0269725245      abcxyz@charusat.ac.in
  PQR             ABC            0269225245      xyzabc@charusat.ac.in
  DEF             MNO            0269725245      defmno@charusat.ac.in
  JKL             GHI            0269725245      jklghi@charusat.ac.in
  RST             MNO            0269225245      rstmno@charusat.ac.in
```

Figure 3.7: *Snapshot of displaying formatted output using printf*

The benefit we get by separating the database-related properties code in a **`db.properties`** file is that you need to update only this file if there are any changes related to driver name, **`url`** of the database, username or password. The file that accesses this properties file need not be changed, as it has the code to access the keys, and what we change is generally the values held by these keys. So, updating the code and maintaining code becomes easier by separating the database-specific key and values in a separate file using the **`java.util.Properties`** file. To create a properties file, we need to create a plain text file which can hold key and value pair in the format key=value, there is no symbol used to indicate the end of statement, for example we cannot use semicolon or full stop to indicate end of statement, also we cannot use single or double quote to hold the value.

To add meaningful comments, one can use the **#** symbol in the file.

JDBC transactions

In JDBC, the **`Connection`** interface provides methods to manage a transaction. If your JDBC Connection is in auto-commit mode, which it is by default, then every SQL statement is committed to the database upon its completion. That may be fine for simple applications, but there are three reasons why you may want to turn off the auto-commit and manage your own transactions:

- To increase performance
- To maintain the integrity of business processes
- To use distributed transactions

To enable manual transaction support instead of the auto-commit mode that the JDBC driver uses by default, use the **`Connection`** object's **`setAutoCommit()`** method.

```
con.setAutoCommit(false);
```

Commit and rollback

Once you are done with your changes and want to commit the changes, call the **`commit()`** method on the **`Connection`** object, as follows:

```
conn.commit( );
```

Otherwise, to roll back updates to the database made using the **`Connection`** named conn, use the following code:

```
conn.rollback( );
```

The below table list various methods used to perform transaction management:

Method	Description
`void setAutoCommit(boolean status)`	It is true by default, which means each transaction is committed by default.
`void commit()`	Commits the transaction.
`void rollback()`	Cancels the transaction.

Table 3.3: *Methods to manage transactions in your own way*

This table covers the methods to manage transactions. The **`setAutoCommit`** method can be used to set the committed mode to true or false; and to save or cancel the transaction, the commit and rollback methods can be used.

Savepoints

When you set a savepoint, you define a logical rollback point within a transaction. If an error occurs past a savepoint, you can use the rollback method to undo either all the changes or only the changes made after the savepoint.

The **`Connection`** object has two new methods that help you manage savepoints:

- **`setSavepoint(String savepointName)`**: It defines a new savepoint and also returns a **`Savepoint`** object.
- **`releaseSavepoint(SavepointsavepointName)`**: It deletes a savepoint.

Batch processing in JDBC

Instead of executing a single query, we can execute a batch (group) of queries for faster performance. The **`java.sql.Statement`** and **`java.sql.PreparedStatement`** interfaces provide methods for batch processing.

Advantages of batch processing

- **Fast performance**: The required methods for batch processing are given below:

Method	Description
`void addBatch(String query)`	It adds query into batch.
`int[] executeBatch()`	It executes the batch of queries.

Table 3.4: *Methods to add and execute batch operations*

Application of methods

Example 1: In this example, we create a **`student`** table with the following columns and insert four records:

```
int studentid;
String    name;
String    course;
String    grade;
```

Before adding the fifth record, set a savepoint, and then commit the transaction. Display the records with the total number of rows, and then roll back to the savepoint and display the records again with the total number of records.

The following code snippet shows the application of savepoints and performing commit and rollback operations:

```
01. package studenttable;
02.
03. import java.io.FileInputStream;
04. import java.io.IOException;
05. import java.sql.Connection;
06. import java.sql.DriverManager;
07. import java.sql.ResultSet;
08. import java.sql.SQLException;
09. import java.sql.Savepoint;
10. import java.sql.Statement;
11. import java.util.Properties;
12.
13.
14. public class StudentTable
15. {
16.      public static void main(String[] args) throws SQLException
17.     {
18.                 Properties props = new Properties();
19.                 FileInputStreamfis = null;
20.                 Connection con = null;
21.                 try
22.       {
23.                 fis=new FileInputStream("D:\\StudentTable\\src\\studenttable\\db.properties");
24.                 props.load(fis);
25.            Class.forName(props.getProperty("driver"));
26.                 con = DriverManager.getConnection(props.getProperty("url"),
27.                 props.getProperty("username"),
28.                 props.getProperty("password"));
29.                 Statement stmt = con.createStatement();
30.                 String insert_query = "INSERT INTO STUDENT VALUES(" + " 4,'ABC','MCA','AA')";
31.                 String select_query = "SELECT * FROM STUDENT";
32.                con.setAutoCommit(false);
33.               Savepoint savepoint1 = con.setSavepoint("Savepoint1");
34.             stmt.executeUpdate(insert_query);
35.         ResultSetrs = stmt1.executeQuery(select_query);
36.         inti = 0;
37.           while (rs.next())
38.            {
39.                       System.out.println(rs.getInt(1) + " " + rs.getString(2) + " " + rs.getString(3) + " " + rs.getString(4) + " \n ");
40.                       i++;
41.                }
42.
43.         System.out.println("\n rows before rollback statement :  " + i + "\n");
44.         con.rollback(savepoint1);
45.                con.commit();
46.                rs = stmt1.executeQuery(select_query);
47.                    i = 0;
48.               while (rs.next())
49.                    {
50.                        System.out.println(rs.getInt(1) + " " + rs.getString(2) + " " + rs.getString(3) + " " + rs.getString(4) + " \n ");
51.                           i++;
52.                     }
53.
54.          System.out.println("\n rows after rollback statement :  " + i + "\n");
55.
56.       }
57.     catch (SQLException | IOException | ClassNotFoundException e)
58.     {
59.                System.err.println("Exception occured \n" + e);
60.        }
61.     }
62.
63. }
```

Figure 3.8: *Code to add Savepoint and perform the commit and rollback operations*

In this example, we create the **Student** table when we execute the code for first time and then comment the code and insert three records. After inserting three records, the code to insert the fourth record is shown in *Figure 3.8*. Set the autocommit to false to manually commit or rollback the transactions. Create a **SavePoint** before inserting the fourth record, and then display the records by navigating the **Resultset**. Roll back to the **SavePoint** set and display all the records again; you can see that the last record inserted is not saved as we move to the **SavePoint**, which is present before inserting the record. The result is that the first navigation loop displays four records and the second navigation loop displays three records, as shown in *Figure 3.9*:

```
run:
Connection Established
1 ABC MCA AA
2 DEF MCA AB
3 CCC MCA AA
4 ABC MCA AA

 rows before rollback statement :  4

1 ABC MCA AA
2 DEF MCA AB
3 CCC MCA AA

 rows after rollback statement :  3
```

Figure 3.9: *Snapshot of records before and after rollback from Student table*

Example 2: In this example, we create the employee table with the following fields:

```
int empid;
String    firstname;
String    lastname;
String    designation;
```

Then perform the following sequence of operations:

- Create SQL statements with placeholders.
- Create the **PrepareStatement** object using the **prepareStatement()** method.
- Set auto-commit to false using **setAutoCommit()**.
- Add as many as SQL statements as you like into batch using the **addBatch()** method on the created statement object.
- Execute all the SQL statements using the **executeBatch()** method on the created statement object.

- Finally, commit all the changes using the **`commit()`** method if all transactions in batch are successfully executed, else roll back the transactions.

The following code shows the application of batch execution:

```
01. package studenttable;
02. import java.io.BufferedReader;
03. import java.io.FileInputStream;
04. import java.io.IOException;
05. import java.io.InputStreamReader;
06. import java.sql.Connection;
07. import java.sql.DriverManager;
08. import java.sql.PreparedStatement;
09. import java.sql.ResultSet;
10. import java.sql.SQLException;
11. import java.sql.Savepoint;
12. import java.sql.Statement;
13. import java.util.Properties;
14. public class StudentTable
15.   {
16.     public static void main(String[] args) throws SQLException
17.    {
18.           Properties props = new Properties();
19.           FileInputStreamfis = null;
20.           Connection con = null;
21.           try
22.          {
23.     fis = new FileInputStream("C:\\BatchProcessing\\src\\studenttable\\db.properties");
24.     props.load(fis);
25.     Class.forName(props.getProperty("driver"));
26.     con = DriverManager.getConnection(props.getProperty("url"),
27.     props.getProperty("username"),
28.     props.getProperty("password"));
29.     con.setAutoCommit(false);
30.     Statement stmt = con.createStatement();
31.     PreparedStatementps = con.prepareStatement("INSERT INTO EMPLOYEE VALUES(?,?,?,?)");
32.     BufferedReaderbr = new BufferedReader(new InputStreamReader(System.in));
33.     while (true)
34.      {
35.       System.out.println("Enter id");
36.                                           String s1 = br.readLine();
37.       int id = Integer.parseInt(s1);
38.       System.out.println("Enter first name");
39.                                      String fname = br.readLine();
40.       System.out.println("Enter last name");
41.                                                 String lname = br.readLine();
42.        System.out.println("Enter Designation");
43.                                           String designation = br.readLine();
44.       ps.setInt(1, id);
45.       ps.setString(2, fname);
46.       ps.setString(3, lname);
47.       ps.setString(4, designation);
48.                                                 ps.addBatch();
49.       System.out.println("Want to add more records y/n");
50.                                        String ans = br.readLine();
51.        if (ans.equals("n"))
52.         break;
53.
54.
55.                    }
56.      ps.executeBatch();
57.
58.           }
59.     catch (SQLException | IOException | ClassNotFoundException e)
60.    {
61.      System.err.println("Exception occured \n" + e);
62.      con.rollback();
63.
64.    }
65.   }
66. }
```

Figure 3.10: *Code to add the records in batch and execute the batch*

In the preceding example, we create the table for the first time the code is executed, and then we comment the code to insert two records manually, where the value of each field is known in advance. We add two records to the batch and then execute the batch. Then, we take the object of **`BufferedReaderand InputStreamReader`** to take input from the console, set each parameter in the dynamic query using

`PreparedStatement`, ask the user to input n number of records from user until the user presses **`n`**, add these n records to the batch and then execute the batch.

Conclusion

This chapter covered the steps to create a properties file and access it in Java code. A properties file can be used for different purposes; here, we used it to hold the database-specific key-value pairs. Moving on, we covered the concept of batch processing with the transactions and showed how to manually roll back and commit using **`SavePoint`**. We also saw that instead of executing a program for one query, one can add the queries to a batch and execute the batch at once to save time and effort; for example, if you want to enter five records, then have five insert queries added to a single batch and execute this batch together.

The next chapter will cover the concept of web components and help you understand the web directory structure.

Multiple choice questions

1. **Which of the following means that the ResultSet can be both read and updated?**
    a. ResultSet.CONCUR_READ_ONLY
    b. ResultSet.CONCUR_UPDATABLE
    c. ResultSet.READ_ONLY
    d. ResultSet.UPDATABLE

2. **Which method allows you to execute a batch of queries?**
    a. execute()
    b. executeUpdate()
    c. executeBatch()
    d. executeQuery()

3. **How can you start a database transaction in the database?**
    a. By asking a Transaction object to your Connection and calling the begin() method on it
    b. By asking a Transaction object to your Connection and setting the autoCommit property of the Transaction to false
    c. By calling the beginTransaction() method on the Connection object
    d. By setting the autoCommit property of the Connection to false and executing a statement in the database

4. **Why will you set auto commit mode to false?**
    a. To increase performance
    b. To maintain the integrity of business processes
    c. To use distributed transactions
    d. All of the above
5. **Which method is used to perform DML statements in JDBC?**
    a. execute()
    b. executeQuery()
    c. executeUpdate()
    d. executeResult()

Answers

1. **b**
2. **c**
3. **d**
4. **d**
5. **c**

Questions

1. Explain the purpose of using properties file.
2. How do you access the properties file froma program?
3. What is JDBC Transaction?
4. Explain the method for performing commit and rollback.

Key terms

- **Properties file**: It is a separate file used to provide key-value pairs to separate the properties, so they can be easily updated and maintained.
- **Commit**: It is used to apply the changes and save it.
- **Rollback**: It is used to undo the changes made in the database.
- **Savepoint**: It is used to define a logical rollback point within a transaction.

Join our book's Discord space

Join the book's Discord Workspace for Latest updates, Offers, Tech happenings around the world, New Release and Sessions with the Authors:

https://discord.bpbonline.com

Chapter 4

Web Components and Web Application Directory Structure

Introduction

The chapter covers the basic concept of web components used in creating a web application. It covers the core concept of web server and client and the protocols used to support communication between the server and the client.

Structure

In this chapter, we will discuss the following topics:

- Understanding web applications
- Web server and client
- TCP/IP -Transmission Control Protocol (TCP) and Internet Protocol (IP)
- HTML and HTTP
- Understanding the HTTP
    - Example of HTTP parameters
    - HTTP communication using request and response headers
- Message start-line, header fields, message body, HTTP - requests

- Understanding URL
- Need servlet and JSPs
- Role of container in handling the request
- Web application directory structure, WAR file
- Deployment descriptor web.xml servlet mapping with name
- Model View Controller (MVC) overview

Objectives

After studying this chapter, you should be able to understand the structure and components of web applications, which will help you in developing a web application using Java.

Understanding web applications

A web application or web app is a client-server software application that the client (or user interface) runs in a web browser. A standard website is meant to provide content for reading, so the static content is formed as part of web page creation. A web application provides interactive and dynamic web page content, so it can be loaded based on the activities of the client or user.

Java Web Application is used to create dynamic websites. Java provides support for web application through servlets and JSPs. We can create a website with static HTML pages, but we need a web application when we want information to be dynamic.

Web server and client

Web servers and clients are the major components of web application communication:

- **Web server** is a software that can process the client request and send the response back to the client. For example, Apache is one of the most widely used web servers. A web server runs on a physical machine and listens to client requests on a specific port.
- A **web client** is a software that helps the web client in communicating with the server.
    - Some of the most widely used web clients are Firefox, Google Chrome, and Safari. When we request something from the server (through URL), a web client takes care of creating a request and sending it to server, and then parsing the server response and presenting it to the user.

The following figure shows the basic communication between the a client and the server:

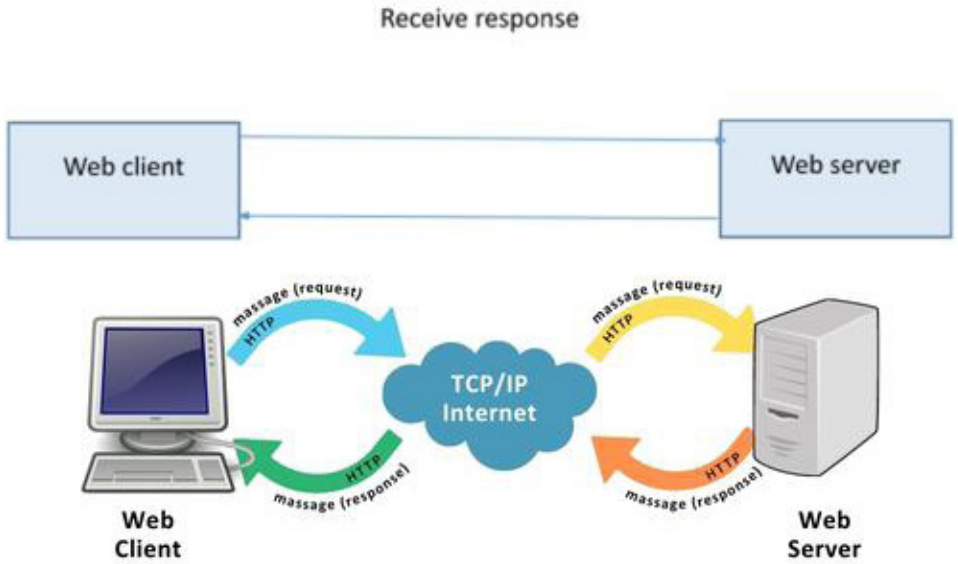


***Figure 4.1**: Communication between Client and Server*

TCP/IP - Transmission Control Protocol (TCP) and Internet Protocol (IP)

TCP/IP is one of the important communication protocols used to connect computer systems in a network. **Hypertext Transfer Protocol** (**HTTP**) is a member of the TCP/IP family.

The two main concerns are the two end systems:

- A web browser
- A web server

TCP provides the delivery of a stream of bytes from a program on one computer to another computer. It is also in charge of the following:

- Controlling size
- The rate of data exchange
- Network traffic congestion

The following figure shows the critical role of the TCP/IP protocol in sending and receiving data:

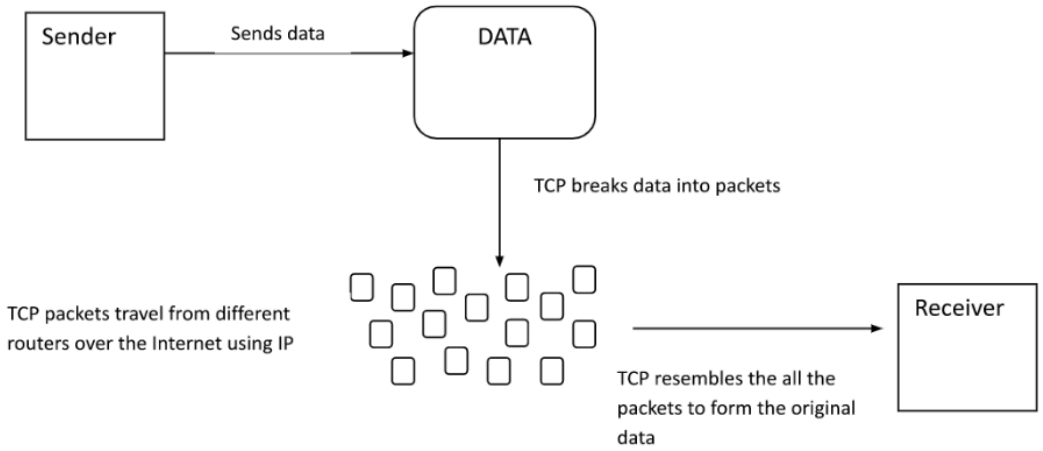


***Figure 4.2**: Role of TCP/IP in client-server communication*

HTML and HTTP

Web server and web client are two separate software, so there should be some common language for communication. HTML is the common language between the server and the client, and it stands for HyperText Markup Language.

The web server and the client needs a common communication protocol; HTTP is the communication protocol between the server and the client, and it runs on top of the TCP/IP communication protocol.

Understanding the HTTP

HTTP is the foundation for data communication for the **World Wide Web** (that is, internet) since 1990. There are three basic features that make HTTP a simple but powerful protocol:

- **HTTP is connectionless**: The HTTP client, i.e., a browser, initiates an HTTP request, and then the client disconnects from the server and waits for a response. The server processes the request and re-establishes the connection with the client to send a response.
- **HTTP is media independent**: It means any type of data can be sent by HTTP as long as both the client and the server know how to handle the data content. Both the client and the server need to specify the content type using

appropriate **MIME**-type. (**Multipurpose Internet Mail Extensions**), It's a way of identifying files on the internet based on their nature and format.

- **HTTP is stateless**: As mentioned earlier, HTTP is connectionless, and it is a direct result of HTTP being a stateless protocol. The server and client are aware of each other only during a request. Afterward, both of them forget about each other. Due to this nature of the protocol, neither the client nor the browser can retain information that is shared between different. requests across the web pages.
- The HTTP protocol is a request/response protocol based on the client/server architecture.

  The HTTP client sends a request to the server in the form of a Request method, URI, Protocol version followed by a MIME-like message containing request modifiers, client information, and possible body content over a TCP/IP connection
- The HTTP server responds with aStatus line including the message's protocol version and a success or error code followed by a MIME-like message containing server information, entity meta information, and possible entity-body content.

The basic protocol layer in client-server communication is shown in the following figure:

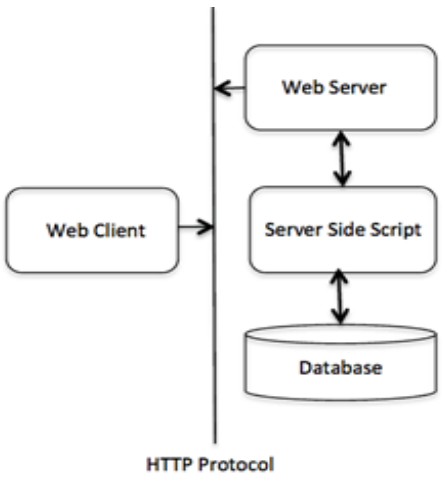


Figure 4.3: *HTTP Protocol layer in client-server communication*

Examples of HTTP parameters

HTTP is a communication protocol used to support client requests and server responses. It is a stateless protocol that is used to exchange messages over a consistent

TCP/IP connection. The HTTP client is used to begin a connection to the server (generally, the client is a web browser); once the connection is established, the client can send requests to the server.

An HTTP server is used to listen to and handle the connection requests, process each request, and send the response messages.

Some of the HTTP parameters used are as follows:

- HTTP Version for example, HTTP/1.1
- Uniform Resource Identifiers
  - URI = "http:" "//" host [":" port] [abs_path ["?" query]]
  - http://abc.com:80/xyz/home.php?id=1
- Date/Time Formats
- Character Sets, for example, US-ASCII
- Content Encodings
- Media Types
  - media-type = type "/" subtype *(";" parameter)
  - Accept: image/gif
- Language Tags
  - language-tag = primary-tag *("-" subtag)
  - For example, en

The following figure lists the major components of request and response headers:

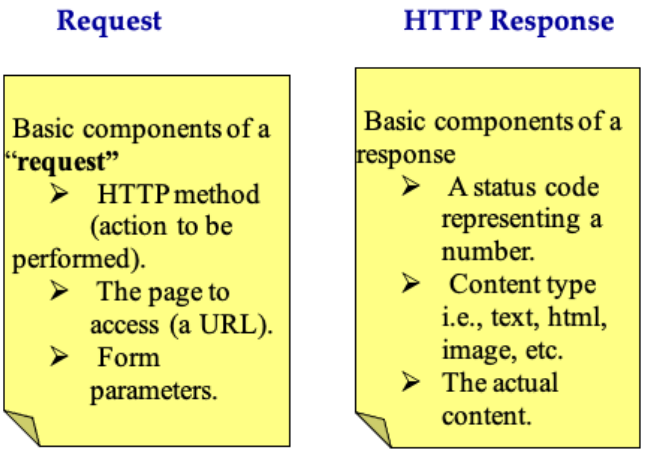


Figure 4.4: *HTTP request and response roles*

HTTP headers

In the client-server communication process, various request and response headers are used. The basic steps in communication are listed as follows:

- Type the Uniform Resource Identifier (URI) in the address bar of the browser to identify a given resource. Then, the connection is established.
- Send the HTTP messages; these messages include the following:
    - Requests from client to server
    - Responses from server to client

```
HTTP-message   = <Request> | <Response> ; HTTP/1.1 messages
```

HTTP requests and HTTP responses use a generic message format of RFC 822 for transferring the required data. This generic message format consists of the following items:

- Message Start-Line
- Header Fields
- Message Body

Message start-line

A start-line will have the following generic syntax:

- start-line = Request-Line | Status-Line

The examples of start line in case of request and response:

- **`GET /hello.htm HTTP/1.1`**: This is the Request-Line sent by the client.
- **`HTTP/1.1 200`**: This is the Status-Line sent by the server.

Header fields

There are four types of HTTP message headers:

- **General-header**: These header fields have general applicability for both request and response messages, for example, Date.
- **Request-header**: These header fields have applicability only for request messages, for example, User-Agent.
- **Response-header**: These header fields have applicability only for response messages, for example, Accept-Ranges.

- **Entity-header**: These header fields define meta information about the entity-body or, if nobody is present, about the resource identified by the request, for example, Content-Language.

Following are the example of Header Fields:

- message-header = field-name ":" [field-value]
- User-Agent: Mozilla/5.0 (Windows NT 10.0; WOW64; rv:47.0) Gecko/20100101 Firefox/47.0 Host: www.example.com
- Accept-Language: en, en-us
- Date: Tue, 24 Jul 2018 12:28:53 GMT
- Server: Apache
- Last-modified: Mon, 23 Jul 2018 19:15:56 GMT
- Accept-ranges: bytes
- Content-length: 51
- Content-type: text/plain

Message body

This is optional. In case the message body is present, it will carry the request or response content. When the content is present as a message, it uses the content-type and content-length header lines to state the type of message associated.

HTTP - requests

An HTTP client sends an HTTP request to a server in the form of a request message that has the following format:

- A Request-line
- Zero or more header (General | Request | Entity) fields followed by CRLF
- An empty line (a line with nothing preceding the CRLF) indicating the end of the header fields
- Optionally, a message-body

The detail format and content of request is given here:

- **Request-line**
    - The Request-Line begins with a method token, followed by the Request-URI and the protocol version, and it ends with CRLF.

- The elements are separated by space SP characters.

```
Request-Line = Method SP Request-URI SP HTTP-Version CRLF
```

- **Request method**
  - This indicates the type of method to be applied on the given resource mentioned in the request URI.
  - The method is case-sensitive and should always be mentioned in uppercase.
  - The following table lists all the supported methods in HTTP/1.1:
    - GET
    - HEAD
    - POST
    - PUT
    - DELETE
    - CONNECT
    - OPTIONS
    - TRACE

The following table shows the major methods used to handle the given request:

S.N.	Method and description
1	**`GET:`** The `Get` method is used to retrieve information from the server using the URI.
2	**`HEAD:`** It is same as `GET`, but it transfers the status line and the header section only.
3	**`POST:`** A `POST` request is used to send data to the server, such as student details and file upload, using HTML forms.
4	**`PUT:`** It changes all the current data of the given resource with the uploaded content.
5	**`DELETE:`** It deletes all the current data of the given resource given by URI.
6	**`CONNECT:`** It establishes a tunnel to the server identified by a given URI.
7	**`OPTIONS:`** It describes the communication options for the given resource.
8	**`TRACE:`** It performs a message loop back test along with the path to the given resource.

***Table 4.1**: HTTP request methods*

- **Request-URI**
  - The Request-URI is a Uniform Resource Identifier and identifies the resource upon which to apply the request. Following are the most commonly used forms to specify an URI:

    ```
    Request-URI = "*" | absoluteURI | abs_path |
    ```

- **Request header fields**
  - The request-header fields allow the client to pass additional information about the request, and about the client itself, to the server.
  - These fields act as request modifiers.
  - The following list provides information about some significant request headers:
    - Accept-Charset
    - Accept-Encoding
    - Accept-Language
    - Host
    - Referer
    - User-Agent
  - Examples of Request Message
    - GET /hello.htm HTTP/1.1
    - Accept-Encoding: gzip, deflate

      Accept-Language en-US,en;q=0.5

      Host: localhost:8083

      Referer: **http://localhost:8083/RequestHeaders/**

      User-Agent: Mozilla/5.0 (Windows NT 10.0; WOW64; rv:47.0) Gecko/20100101 Firefox/47.0

- **Response header fields**
  - After receiving and interpreting a request message, a server responds with an HTTP response message:
    - A Status-line
    - Zero or more header (General|Response|Entity) fields, followed by CRLF
    - An empty line indicating the end of the header fields
    - Optionally, a message-body

- **Message status-line**
  - A Status-Line consists of the protocol version, followed by a numeric status code and its associated textual phrase.
  - The elements are separated by space SP characters.

    ```
    Status-Line = HTTP-Version SP Status-Code SP Reason-Phrase
    CRLF
    ```

- **HTTP version**
  - A server supporting HTTP version 1.1 will return the following version information:

    ```
    HTTP-Version = HTTP/1.1
    ```

- **Status code**
  - The Status-Code element is a 3-digit integer, where the first digit of the Status-Code defines the class of response and the last two digits do not have any categorization role.
  - There are 5 values for the first digit.

    The following table shows the range of status code with the general meaning for the starting number in each range.

S.N.	Code and description
1	**1xx: Informational** This code range indicates that the request has been received by the server and is under process.
2	**2xx: Success** This code range indicates that the request has been received and processed successfully.
3	**3xx: Redirection** This code range indicates that some extra action needs to be taken to process the request.
4	**4xx: Client Error** This code range indicates that the request in incorrect or incomplete and cannot be processed.
5	**5xx: Server Error** This code range indicates that the server cannot accomplish the request, and it failed to process the request completion.

***Table 4.2**: Status code supported by HTTP*

- **HTTP Status code**: Here are a few examples of HTTP status code:
    - 100 (Continue)
    - 200 (OK)
    - 301 (Moved Permanently)
    - 400 (Bad Request)
    - 401 (Unauthorized)
    - 403 (Forbidden)
    - 404 (Not Found)
    - 405 (Method Not Allowed)
    - 500 (Internal Server Error)
    - 503 (Service Unavailable)

- **Response Header Fields**
    - The response-header fields allow the server to pass additional information about the response, which cannot be placed in the Status-Line. These header fields give information about the server and about further access to the resource identified by the Request-URI.

      the response header includes,
        - Accept-Ranges
        - Age
        - Location
        - Retry-After

Understanding URL

Universal Resource Locator, shortly known as URL, is used to locate the server and resource. Every resource on the web has its own unique address. Let's understand the parts of a URL with an example:

```
http://localhost:8080/FirstServletProject/jsps/hello.jsp
```

- **`http://`**: This is the first part of URL and provides the communication protocol to be used in server-client communication.
- **`localhost`**: This is the unique address of the server, and it's mostly the hostname of the server that maps to the unique IP address. Sometimes, multiple hostnames point to the same IP addresses and the web server virtual host takes care of sending request to the particular server instance.

- **`8080`**: This is the port on which the server is listening; it's optional, and if we don't provide it in URL then the request goes to the default port of the protocol. Port numbers 0 to 1023 are reserved ports for well-known services, for example, 80 for HTTP, 443 for HTTPS, 21 for FTP, etc.
- **`FirstServletProject/jsps/hello.jsp`**: This is the resource requested from server. It can be static HTML, PDF, JSP, servlets, PHP, etc.

Need servlet and JSPs

Static pages can be developed using HTML. We can generate dynamic web page content and handle the database using several server-side programming languages like PHP, Python, Ruby, Java servlets, and JSP. The need of servlets and JSP is to provide the web servers with the facility to generate dynamic response and save data in the database. We must use servlets and JSP to create sites that include server-side response that is dynamic in terms of content using Java. A brief introduction of these technologies is given as follows:

- **Servlet**: Servlet technology is used to create a web application. The Java servlet API allows you to communicate and do programming with HTTP-specific classes. A servlet is used to create applications that can read the request and generate the response. The applications can be hosted on the web server, and the request can be handled and processed by the servlet. The **`javax.servlet`** and **`javax.servlet.http`** packages provide the classes and interfaces to define servlets.

  HTML servlet classes extend the **`javax.servlet.http.HttpServlet`** abstract class, which provides a framework for handling HTTP protocol.

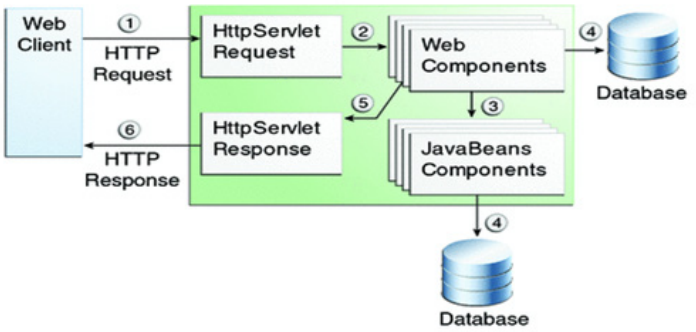


Figure 4.5: *HTTP Servlet role*

- **JavaServer Pages Technology**: The **JavaServer Pages** (**JSP**) technology provides a simple and fast way to create dynamic web content. JSP enables you to create web-based applications that are server- and platform-independent.

Typically, a JSP page is a text-based document that contains two types of text:

- Static data, which can be expressed in any text-based format, such as HTML, Wireless Markup Language (WML), or XML.
- JSP technology elements, determine how the page is built with the dynamic content.
- The packages involved in creating JSP pages are **`javax.el`**, **`javax.servlet.jsp`**, **`javax.servlet.jsp.el`**, and **`javax.servlet.jsp.tagext`**, though you will rarely have to import these directly.
- A JSP page is a combination of HTML code and JSP code, with the **`extension.jsp`**. Servlets have a life cycle which is execution of several methods phase wise; it does not have the **`main()`** method. Servlets are handled by another Java application known as a Container. Tomcat is an example of a Container. On receiving the servlet request by web server application, it hands the request over to the container in which the servlet is deployed. The container calls the servlets life cycle methods and handles the HTTP request and response. Web container (also known as a Servlet container) is the component of a web server that interacts with Java servlets. A web container is responsible for managing the life cycle of servlets, mapping a URL to a particular servlet and ensuring that the URL requester has the correct access rights.

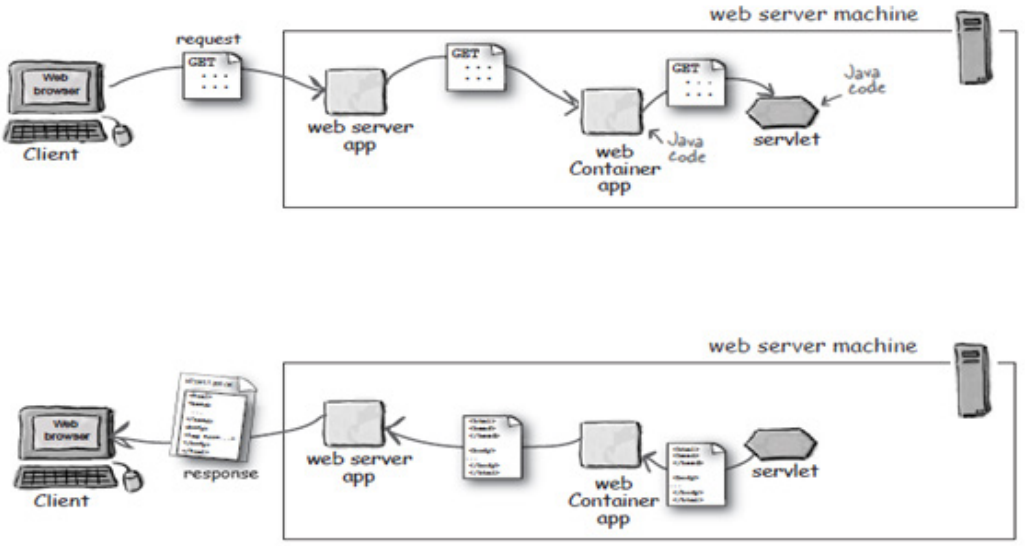


Figure 4.6: *Client-Server Communication*

- **Communications support**: The container provides an easier way for servlets to communicate with the web server. A **`ServerSocket`** is not required to build and listen on a particular port. The API used to handle the communication makes it easy to work between the server and the application code. We need to provide the business logic as per the requirements.
- **Lifecycle management**: The container manages the servlet upon its creation and manages all its phases until it is alive. It takes care of loading the classes, instantiating and initializing the servlets, invoking the servlet methods, and making servlet instances eligible for garbage collection.
- **Multithreading support**: A new thread is created by the container for handling every servlet request it receives. When the servlet is done processing the request, the thread terminates.
- **Declarative security**: A container provides a deployment descriptor file to configure the major settings related to application.
- **JSP support**: JSPs are nothing but Java inside HTML. A container translates JSP into Java automatically.

Role of container in handling the request

When a request is made from a client to the web server, it passes the request to a web container. Then, it's the web container's job to find the correct resource to handle the request (servlet or JSP) and then use the response from the servlet or JSP to generate the response and provide it to web server. Then, the web server sends the response back to the client. When the web container gets the request, if it's for the servlet, the container creates two objects: **`HTTPServletRequest`** and **`HTTPServletResponse`**. Then, it finds the correct servlet based on the URL and creates a thread for the request. Next, it invokes the servlet **`service()`** method and based on the HTTP request **`service()`** method invokes **`doGet()`** or **`doPost()`** methods. Servlet methods generate the dynamic page and write it to response. Once the servlet thread is complete, the container converts the response to HTTP response

and sends it back to client. The container follows six steps, as shown in the following figure:

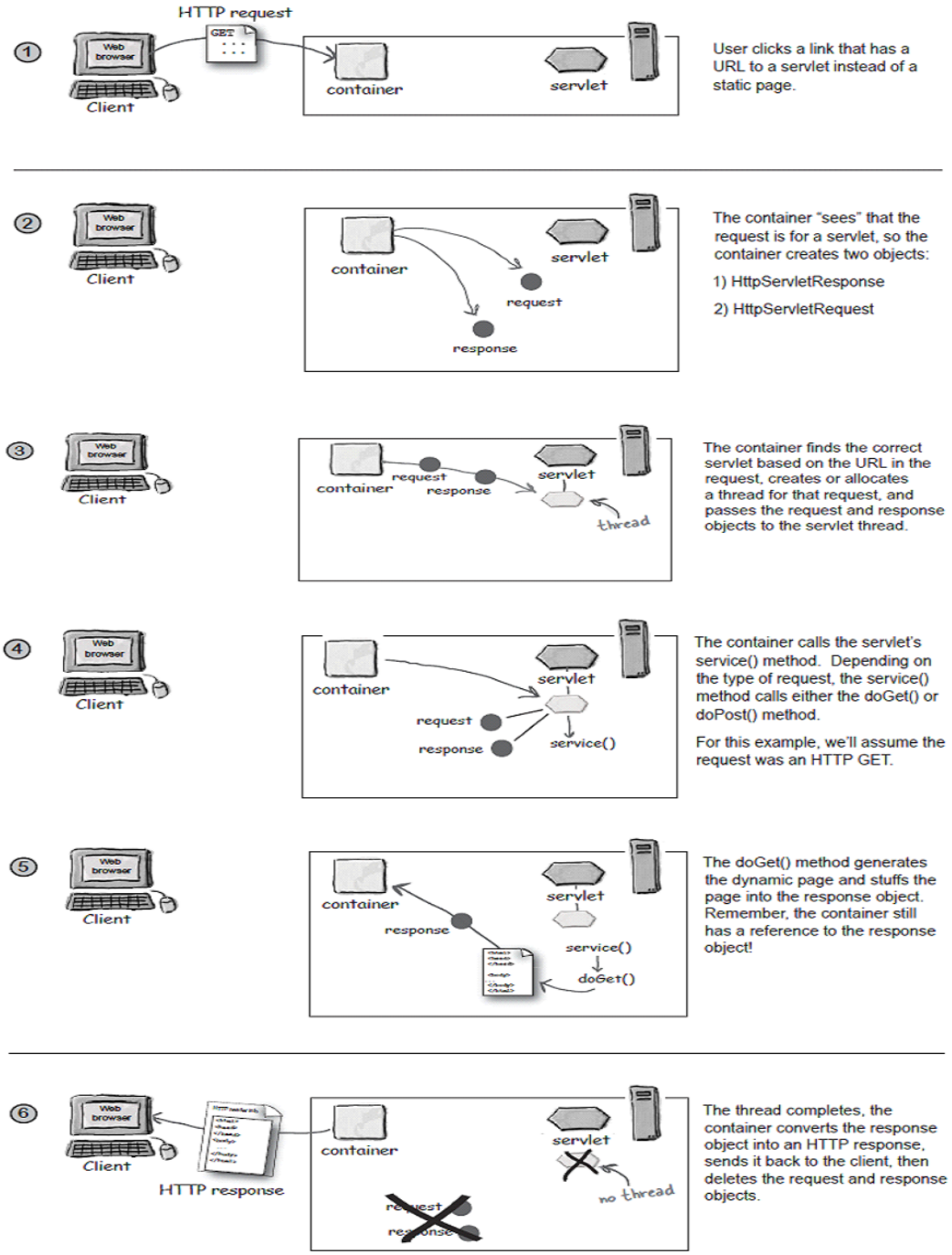


Figure 4.7: *Role of container in Client-Server Communication*

Web application directory structure

Java web applications are packaged as **Web Archive** (**WAR**), which has a defined structure. The directory structure of a web application is shown in the following figure:

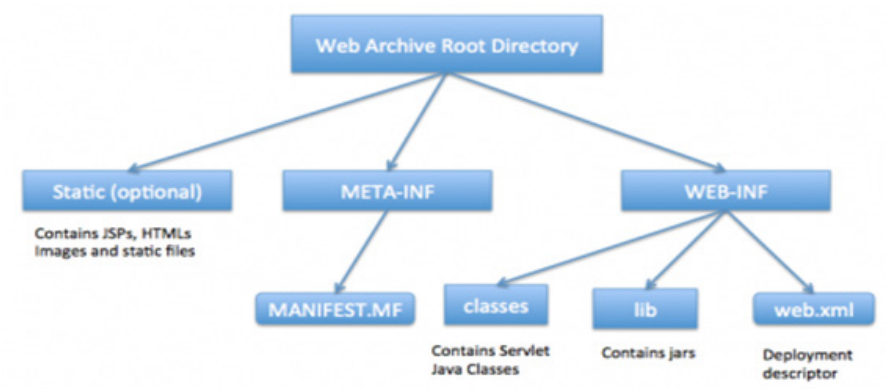


Figure 4.8: *Web Application Directory Structure*

In order to run your Java web application (servlet, JSP, and so on), we need web server. Before running the application, you need to package the resources inside it (servlets, JSP's, XML files and, so on) in a standardized way, as shown in the following figure:

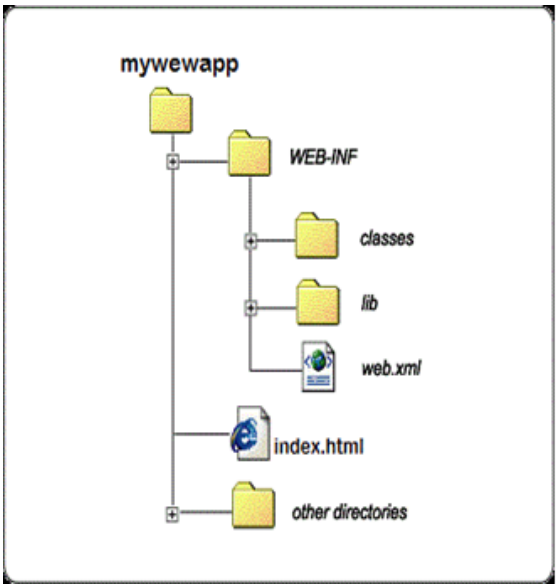


Figure 4.9: *Hierarchy of files organized in the web application structure*

The following is the hierarchy of files organized in the web application structure:

- **The root directory**
    - The root directory is the name of the web application; it can be any valid name. In the previous example, the root directory name is **`mywebapp`**. Under the root directory, you can put all files that should be accessible in your web application.
    - If your web application is mapped to the URL **`http://localhost:8080/mywebapp/`**, then the index.html page will be accessible by the URL **`http://localhost:8080/mywebapp/index.html`**.
    - If you create any subdirectories under the root directory and place files in these subdirectories, they will be available by the subdirectory/file path in the web application.
- **The WEB-INF Directory**
    - The WEB-INF (web information) directory is located just below the web app root directory.
    - This directory is a meta information directory.
    - Files stored here are not supposed to be accessible from a browser (although your web app can access them internally, in your code).
    - There are two important directories inside the WEB-INF directory:
        - classes and lib
        - web.xml
- **web.xml**
    - The web.xml file contains information about the web application, which is used by the Java web server / servlet container in order to properly deploy and execute the web application.
    - For example, web.xml contains information about which servlets a web application should deploy and what URLs they should be mapped to.
- **classes directory**
    - The classes directory contains all compiled Java classes that are part of your web application.
    - The classes should be located in a directory structure matching their package name.

- **lib folder**
    - The lib directory contains all JAR files used by your web application.
    - This directory most often contains third-party libraries that your application is using.
    - You can also place your own classes into a JAR file and place it here rather than putting those classes in the classes directory.

WAR file

A WAR file is simply a packaged webapp directory or archive file like ZIP or RAR. It will store JSP, servlets, classes, meta data information, images, sound, tag libraries, etc. Its standard file extension is .war. WAR files are used to package web modules, and they are for web applications deployed to a server. For example, an application for stock trading might be distributed in an archive file called a **`Stocktrading.war`** file.

The WAR file is a standard format for web applications that has specific directories and specific files.

This includes a **`WEB-INF`** directory, a **`WEB-INF/web.xml`** file used to describe the application, a **`WEB-INF/lib`** directory for JAR files used by the application, and a **`WEB-INF/classes`** directory for class files that are not distributed in a JAR. JSP, HTML pages, images, java classes, and many more components of the application can be packaged as a single file distributed.

Deployment descriptor

The **`web.xml`** file is the deployment descriptor of the web application and contains mapping for servlets (prior to 3.0), welcome pages, security configurations, session timeout settings, and so on.

Model View Controller (MVC) Overview

The **Model-View-Controller** (**MVC**) is an architectural pattern that separates an application into three main logical components:

- A **model** represents the underlying logical structure of data in a software application and the high-level class associated with it. This object model does not contain any information about the user interface.
- A **view** represents the user interface using which the users can view and submit data.

- A **controller** represents the classes connecting the model and the view, and it is used to communicate between classes in the model and view:

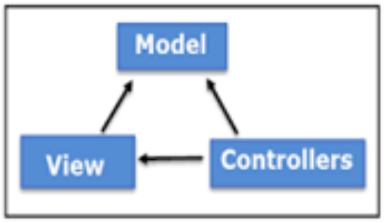


***Figure 4.10**: MVC architecture*

Conclusion

This chapter helped you gather the fundamentals knowledge required to understand client-server communication, which includes understanding web client, server and protocols used in the communication. The chapter also gave you a brief overview of TCP/IP and HTTP protocols with their role in web client-server communication. The role of container was explained in brief, and the details of how a web directory structure is organized was explained with the goal of making a web archive file. At the end of the chapter, MVC design pattern was briefly described.

The next chapter covers servlet programming to help you better understand how a server can read the request, process it and form the response.

Points to remember

- TCP/IP is a family of communication protocols used to connect computer systems in a network.
- Java Web Applications are packaged as WAR, and WAR has a defined structure.

Multiple choice questions

1. **The _____ protocol is used for communication between web server and client.**
    a. FTP
    b. HTTP
    c. SMTP
    d. None of the above

2. **The server and client are aware of each other only during a current request. This is due to which characteristics of HTTP?**
    a. Media independent
    b. Connectionless
    c. Stateless
    d. All of the above

3. **The _____ indicates the method to be performed on the resource identified by the given Request-URI.**
    a. request-line
    b. request-method
    c. request header fields n
    d. None of the above

4. **Which status code indicates a client-side error?**
    a. 2XX
    b. 5XX
    c. 3XX
    d. 4XX

5. **The _____lets you define HTTP-specific classes.**
    a. JDBC API
    b. Servlet API
    c. Swing API
    d. None of the above

Answers

1. **b**
2. **c**
3. **b**
4. **d**
5. **b**

Questions

1. What is meant by a web application?
2. Explain the terms web server and client.
3. What is the role of HTML and HTTP in a web application?
4. Explain the category of HTTP status codes.
5. Explain the structure of URL when accessing a web application.
6. What is the role of servlet and JSP in making a web application using Java.
7. What is a web container? How does a container handle a request?
8. Explain the web application directory structure.
9. What is the use of deployment descriptor file in a web application?

Key terms

- **Web server**: It is a software that can process the client request and send the response to the client.
- **Web client**: It is a software that helps in communicating with the server.
- **WAR**: A WAR "Web Archive" file is simply a packaged webapp directory or archive file like ZIP or RAR.
- **MVC**: The **Model-View-Controller** (**MVC**) is a design pattern that separates an application into three main logical components: the code for handling data, business logic and user interface.

Join our book's Discord space

Join the book's Discord Workspace for Latest updates, Offers, Tech happenings around the world, New Release and Sessions with the Authors:

https://discord.bpbonline.com

CHAPTER 5
Servlet Programming

Introduction

In this chapter, servlets are explained with their purpose and application. Servlets are a way to provide accessibility to HTTP requests and responses from the Java API. They allow you to handle server-side programming in a web application. To manage the server side concerns in a web application the chapter covers the core functionality provided by the servlets in a web application.

Structure

The chapter covers the following topics:

- Basics of the servlet
- Servlet life cycle
- Servlet programming
- Managing client-side data
- Handling the request headers
- Generating the server response
- Handling the response headers

Objectives

After studying this chapter, you should be able to understand the role of servlet in web programming, know and implement various phases of execution of a servlet, and manage client- and server-side data using HTTP headers.

Basics of the servlet

Initially, when web pages were developed, they were static in nature, meaning the pages were developed first and deployed; they were merely for the purpose of reading, and user activities had no effect on the design or development of web pages. Then came the era of dynamic pages, where the pages do need to be generated ahead of time; they are now generated on requests given by users, so the response is generated based on the request given by the client. In this chapter, we will look at how these pages can be developed using servlets.

Servlet is a special Java program that is used to handle and process the client request, prepare the response, and send it back to the client, as illustrated in *Figure 5.1*:

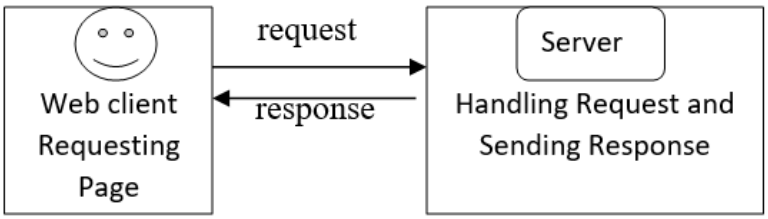


***Figure 5.1**: Client-Server Communication in a web-based application*

Role of servlets

The servlet is used to perform the following tasks:

- Read the request that is sent by the client and handle it; the request is sent as a form.
- Read the data sent as content in a form and HTTP headers.
- Handle the request and generate the desired output as response data, which may require the use of database, beans, web services, etc.
- Apart from response data, send the header data, which may include header information.

Common Gateway Interface (CGI)

Like servlets, CGI is used to handle the data passed by the client, process it and form the response. For handling each request, the CGI creates a new process.

Disadvantages of CGI

- As it creates a new process to handle each request, it is costly and makes it difficult to handle multiple processes.
- It the number of clients increases, handling each request becomes difficult. The language of implementation is platform-dependent, for example, C or C++.

Figure 5.2 illustrates the process and thread models used to implement multiprogramming in the computers:

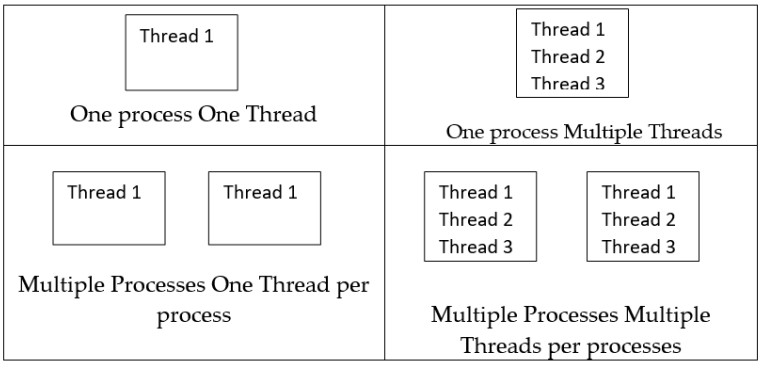


***Figure 5.2**: Process and Thread Memory Layout*

Figure 5.3 shows the address space allocation when a new process is created and when a new thread is created:

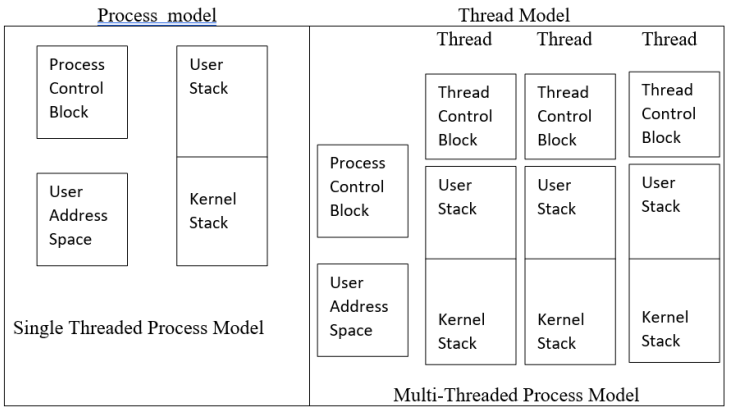


***Figure 5.3**: Process and Thread Model*

There are many advantages of servlet over CGI. The web container creates threads for handling the multiple requests to the servlet, and threads have lots of benefits over processes, such as they share a common memory area, are lightweight, and the cost of communication between the threads is low.

The Advantages of Servlets Over *Traditional* CGI:

- **Better performance**: It creates a new thread to handle each request, whereas CGI creates a new process to handle each request, creating a new thread requires quite less resources than a process.
- **Portability**: It uses the Java language.
- **Robust**: Servlets are managed by JVM, so we don't need to worry about memory leaks, garbage collection, etc.
- **Secure**: It uses the Java language.

Servlet Life cycle

A servlet life cycle can be defined as the entire process from its creation till its destruction. The life cycle methods are executed in the following order:

- The **`init()`** method is called to initialize the servlet.
- Then the **`service()`** method is called to handle the request.
- After the request is processed, the **`destroy()`** method is called and the servlet is terminated.
- The garbage collector.

Finally, servlet is garbage collected by the garbage collector of the JVM.

The init() method

The **`init`** method is called only once. It is called the first time when the servlet is created, generally when the user enters the URL leading to a servlet request.

. Once the servlet is created to handle every new request a thread is created and the appropriate method called to service the request. The **`init()`** method simply creates or loads some data that will be used throughout the life of the servlet.

```
public void init() throws ServletException {

          // Initialization code...

        }
```

The service() method

This is the method that performs the actual task. It is called by the servlet container to handle the client request and to send the output back. When the server receives the request for the servlet, it creates a new thread to service the request every time.

The **`service()`** method checks the type of request, and based on that, the appropriate method is called. The signature of method is as shown here:

```
public void service(ServletRequest request, ServletResponse response)
throws ServletException, IOException{

}
```

The doGet() method

The **`GET`** request is generated when the form data is submitted without passing any value to the form attribute of the form tag, and the request is serviced by the **`doGet`** method. The signature of the method is shown here:

```
public void doGet(HttpServletRequest request,HttpServletResponse

                        response) throws ServletException, IOException {

                    // Servlet code

        }
```

The doPost() method

When the method attribute of the form tag is set to post, the **`POST`** request is generated, and the form data s processed by the **`doPost()`** method. The signature of the method is shown here:

```
    public void doPost(HttpServletRequest request,

                               HttpServletResponse response)

        throws ServletException, IOException {

            // Servlet code

    }
```

The destroy() method

Like the **`init()`** method, the **`destroy()`** method is also called once during the life cycle of the servlet. It is used to perform cleanup and closure activities like close file or database connections, write data to disk, etc. After the **`destroy()`** method is called, the servlet object is marked for garbage collection.

The signature of method is shown here:

```
public void destroy() {
      // Finalization code...
}
```

Servlet life cycle scenario

When the servlet is invoked, the request is assigned to the servlet container. For each request, the servlet container creates a thread, and each thread executes the service method of a single instance of the servlet. Then, the servlet container loads the servlet.

The following figure shows the processing of servlet request:

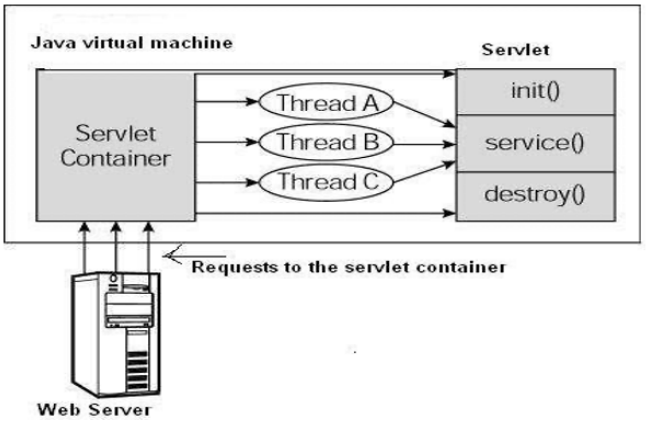


***Figure 5.4**: Servlet Processing*

Servlet programming

Following is the basic servlet structure:

- It is regular Java code.
- It has new **`import`** statements.
- It extends a standard class (**`HttpServlet`**).
- It overrides the **`doGet`** method.

The following code outlines the basic servlet that handles **`GET`** requests. **`GET`** requests, with HTTP, are the usual type of browser requests for web pages. A browser generates this request when the user enters a URL on the address line, follows a link from a web page, or submits an HTML form that either does not specify a **`METHOD`**

or specifies **METHOD="GET"**. Servlets can also easily handle **POST** requests, which are generated when someone submits an HTML form that specifies **METHOD="POST"**.

```
ServletTemplate.java
import java.io.*;
import javax.servlet.*;
import javax.servlet.http.*;

public class ServletTemplate extends HttpServlet {
  public void doGet(HttpServletRequest request, HttpServletResponse response)
      throws ServletException, IOException {
    // Use "request" to read incoming HTTP headers (e.g., cookies) and query data from
               HTML forms.
    // Use "response" to specify the HTTP response status, code and headers (e.g., the content
               type, cookies).
    PrintWriter out = response.getWriter();
    // Use "out" to send content to browser.
  }
}

```

Figure 5.5: *Using Get method*

Servlets typically extend **HttpServlet** and override **doGet** or **doPost**, depending on whether the data is being sent by **GET** or by **POST**. Both **doGet** and **doPost** take two arguments:

HttpServletRequest

HttpServletResponse

HttpServletRequest lets you get all the incoming data; the class has methods by which you can find out information like form (query) data, HTTP request headers, and the client's hostname.

HttpServletResponse lets you specify outgoing information like HTTP status codes (**200**, **404**, etc.) and response headers (Content-Type, Set-Cookie, etc.). Most importantly, it lets you obtain a **PrintWriter** that you use to send document content back to the client.

Since **doGet** and **doPost** throw two exceptions (**ServletException** and **IOException**), you are required to include them in the method declaration.

Finally, you must import classes in **java.io** (for **PrintWriter**, etc.), **javax.servlet** (for **HttpServlet**, etc.), and **javax.servlet.http** (for **HttpServletRequest** and **HttpServletResponse**).

Servlet to generate plaintext

This section shows how to write a servlet to generate plaintext in the browser:

```
01. PlainTextServlet.java
02. import java.io.*;
03. import javax.servlet.*;
04. import javax.servlet.http.*;
05. public class PlainTextServlet extends HttpServlet {
06.   public void doGet(HttpServletRequest request,HttpServletResponse response)
07.                                                                 throws ServletException, IOException
08.       {
09.                PrintWriter out = response.getWriter();
10.                 out.println("Hello M.Sc.(IT) semester III students");
11.       }
12.    }
13.
```

Figure 5.6: *Generate plaintext in the browser*

Servlet to generate HTML

To generate HTML, add three steps to the following process:

1. Tell the browser that you're sending it HTML.
2. Modify the **`println`** statements to build a legal web page.
3. Check your HTML with a formal syntax validator. Set the HTTP Content-Type response header to **`text/html`**. :

   ```
   response.setContentType("text/html");
   ```

The second step in writing a servlet that builds an HTML document is to have your **`println`** statements output HTML, not plaintext. The final step is to verify that your HTML has no syntax errors that could cause unpredictable results on different browsers:

```
01. HTMLServlet.java
02.  import java.io.*;
03.  import javax.servlet.*;
04.  import javax.servlet.http.*;
05.  public class HTMLServlet extends HttpServlet
06.  {
07.    public void doGet(HttpServletRequest request, HttpServletResponse response)
08.        throws ServletException, IOException
09.         {
10.           response.setContentType("text/html");
11.           PrintWriter out = response.getWriter();
12.           String docType = "<!DOCTYPE  html">\n";
13.           out.println(docType +"<HTML>\n" +
14.                                          "<HEAD><TITLE>Hello</TITLE> </HEAD>\n" +
15.                                          "<BODY>\n" +
16.                                          "<H1>Hello M.Sc.(IT)  III students</H1>\n" +
17.                                          "</BODY></HTML>");
18.    }
19.  }
20.
```

Figure 5.7: *Generate HTML in the browser*

Handling the client-side form data

To read the data from the client side, use the FORM element to create an HTML form. Use the **ACTION** attribute to designate the address of the servlet or JSP page that will process the results; you can use an absolute or relative URL. For example: **<FORM ACTION="...">...</FORM>**.

If **ACTION** is omitted, the data is submitted to the URL of the current page.

Use input elements to collect user data. Place the elements between the start and end tags of the **FORM** element and give each input element a **NAME**. Textfields are the most common input element; they are created with the following:

```
<INPUT TYPE="TEXT" NAME="...">
```

Place a submit button near the bottom of the form. Consider the following example:

```
<INPUT TYPE="SUBMIT">
```

When the button is pressed, the URL designated by the form's **ACTION** is invoked. With **GET** requests, a question mark and name/value pairs are attached to the end of the URL, where the names come from the **NAME** attributes in the HTML input elements and the values come from the end user. With **POST** requests, the same data is sent, but on a separate request line instead of being attached to the URL. Extracting the needed information from this form data is traditionally one of the most tedious parts of server-side programming. You have to invoke the data either from GETor POSTmethod, and then read the request parameters and parse them if required.

Methods to read the data

This section explains the methods used to read the form data.

Reading single values: getParameter

To read a request (form) parameter, you simply call the **getParameter** method of **HttpServletRequest**, supplying the case-sensitive parameter name as an argument. You supply the parameter name exactly as it appeared in the HTML source code, and you get the result exactly as the end user entered it; any necessary URL-decoding is done automatically. Unlike with many alternatives to servlet technology, you use **getParameter** exactly the same way when the data is sent by **GET** (that is, from within the **doGet** method) as you do when it is sent by **POST** (that is, from within **doPost**); the servlet knows which request method the client used and automatically uses the appropriate method to read the data. An empty **String** is returned if the parameter exists but has no value (that is, the user left the corresponding **textfield** empty when submitting the form), and null is returned if there was no such parameter.

Parameter names are case sensitive; consider the following example:

`request.getParameter("paramname1")` and **`request.getParameter("paramName1")`** are not interchangeable.

Reading multiple values: getParameterValues

When there are many parameters to be read from the form, we can use the **`getParameterValues`** method. It returns the array of all the values, so instead of reading the parameters one by one using **`getParameter`**, which returns a value for each parameter, we can read an array of values. If the parameter names do not exist, the **`getParameterValues`** method will return null. In the case of single parameter value, it will return an array with one element, so we can use this method when we are reading multiple form values.

Accessing parameter names: getParameterNames and getParameterMap

When we write the servlet code, we assume the parameters names, but if we don't know them, it becomes a problem to access the parameter value. So, the solution is to use the **`getParameterNames`** method to get the list of parameter names in the form of enumeration. Each value received in the Enumeration can be cast to String and then used in the **`getParameter`** or **`getParameterValues`** method. If there are no parameters in the current request, **`getParameterValues`** will return an empty enumeration list. Then, we can use the Enumeration interface and use the methods provided by the interface, for example, the **`hasMoreElements`** and **`nextElement`** methods, to iterate and get the values in a loop.

Accessing the form fata : getReader or getInputStream

In the previous section, we looked at how to read individual or multiple parameters. In this section, we will study another approach to read the form parameters using a stream and then parse the raw data. This can be achieved through the **`getReader`** or **`getInputStream`** method. Reading the raw data is difficult as there is no delimiter use to separate each parameter value, and the URL is also replaced by ASCII or ISO Latin characters. The parsing of raw data is useful when we need to parse the data manually, for example the data comes from a custom client rather than by an HTML form and read the data yourself, or the data is from an uploaded file, then parsing is necessary as servlet doesn't have API support to read a file.

Reading input in multiple character sets: setCharacterEncoding

When the request parameters are read, they are interpreted using the server's current character set. We can use the **`setCharacterEncoding`** method of **`ServletRequest`**.

We may have a situation where the request parameters are required to be read in more than one character set. In this case, we cannot use **`setCharacterEncoding`** with the name of one character set, so we have two options to continue with: one is to read the parameter in one character set and convert it to another, and the second is to use an autodetect option provided with some character sets.

For the first approach, we can read the parameter using **`getBytes`** to read the raw bytes and then pass the bytes to the **`String`** constructor. The following code shows the steps to carry out this approach. In this example, we read and convert the parameter to Japanese language:

```
String firstNameEncoding = request.getParameter("firstName");

String firstName = new String(firstNameEncoding.getBytes(), "Shift_JIS");
```

The second approach allows you to autodetect the character set. For example, to allow input in either English or Japanese, you might use the following:

```
request.setCharacterEncoding("JISAutoDetect");

String firstName = request.getParameter("firstName");
```

Reading form data

Using the preceding methods, we will demonstrate examples to understand the usage of these methods:

Example: We create a **`StudentInformation.html`** form to input the student details and submit the data to a Servlet, namely, **`StudentServlet`**.

```
StudentInformation.html

<html>
<head>
<title>TODO supply a title</title>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
</head>
<body>
<form method="get" action=" StudentServlet ">
            Student ID     : <input type="text" name="studentid"/><br/>
            Student Name   : <input type="text" name="studentname"/><br/>
            Student Programme and Semester :<input type="text" name="progsem"/><br/>
<input type="submit" value="Submit"/>
</form>
</body>
</html>

```

Figure 5.8: *Code for inputting three fields*

The Servlet reads the parameters send by the form and displays them:

```
StudentServlet.java
package src;
import java.io.*; import javax.servlet.*; import javax.servlet.http.*;

public class StudentServlet extends HttpServlet
{
    @Override
    public void doGet( HttpServletRequest request, HttpServletResponse response) throws ServletException,IOException
     {
 System.out.println("Inside get method ");
 PrintWriter out=response.getWriter();
 String id, name, prg;
 id = request.getParameter("studentid");
        name = request.getParameter("studentname");
        prg = request.getParameter("progsem");
        out.println(" <HTML> "
                              +  " <HEAD> "
                              + " <TITLE> HTML SERVLET </TITLE> "
                              + " </HEAD> "
                              + " <BODY> "
                              + " <H1> "
                              + " Student Details "
                              + " </H1> "
                              +" <h3> <span style = color:blue > Student ID : </span>
                                          <span style = color:maroon> " + id + " </span> <br/> "
                                     + " <span style = color:blue > Student Name : </span>
                                          <span style = color:maroon > "  + name + "
                                          </span> <br/>" + "<span style = color:blue >
                                          Student Programme and Semester : </span>
                                        <span style = color:maroon >" + prg + "</span><br/> "
                            + " </BODY> "
                            + " </HTML> ");
    }
```

Figure 5.9: *Display data in the browser*

Example: We read the data from the same HTML form **`StudentInformation.html`**, we find total number of parameters and iterate over it to generate the values.

```
ShowParameters.java
public class ShowParameters extends HttpServlet
 {
  public void doGet( HttpServletRequest request,  HttpServletResponse response )
  {
    response.setContentType("text/html");
    PrintWriter out = response.getWriter();
     out.println(" <!DOCTYPE  html> "
                            + " <HTML> \n "
                            + " <HEAD> <TITLE> Parameters </TITLE> </HEAD> \n "
                            +" <BODY > \n "
                            +" <H1> Parameters  </H1> " ) ;
                Enumeration paramNames = request.getParameterNames();
      while( paramNames.hasMoreElements() )
                  {
                                String paramName = (String)paramNames.nextElement();
                                out.print(paramName);
                                String paramValue = request.getParameter(paramName);
                                out.println(paramValue);
                      }
              }
    }
```

Figure 5.10: *Using enumeration to retrieve field values*

Handling the request headers

This category of methods provides a way to read and process the request headers.

Methods to read the request headers

To read the headers, we should provide the name of header we wish to read in the **`getHeader()`** method. For example, if wish to access the **`Connection`** header, we may access it through **`request.getHeader("Connection")`**. Apart from the **`getHeader`**, we can access some headers directly with getter methods specially created to access the headers.

The following is a list of a few methods for such **`headers`**:

- **`getCookies`**: This method returns the content of the **`Cookie`** header as an array of **`Cookie`** object.
- **`getAuthType`** and **`getRemoteUser`**: These methods are used to divide the Authorization header into small parts.
- **`getContentLength`**: This header is used to retrieve the length of content and return a value in integer for the header.
- **`Content-Length.getContentType`**: This header is used to retrieve the **`Content-Type header`** as a **`String`**.

- **`value.getDateHeader`** and **`getIntHeader`**: These headers are used to convert the date and integer values.
- **`getHeaderNames`**: Instead of reading an individual header, we can use the **`getHeaderNames`** method to get a list of all the header names received as an Enumeration.
- **`getHeaders`**: Most of the time, each header appears only once in the request line, but a header may occur multiple times with separate values. In that case, we can use **`getHeaders`** to retrieve all the values as an enumeration.

Apart from calling methods for request headers, we can use methods for request line given by **`HttpServletRequest`**:

- **`getMethod`**: This method returns the main request method, like **`Get`**, **`Post`**, **`Delete`**, etc.
- **`getRequestURI`**: This method returns the part of URL that comes after the host name and port number and before the form data.
- **`getQueryString`**: This method returns the form data, for example, with **http://BankApp/CustomerName?id=1**, the method would return **`id=1`**.
- **`getProtocol`**: This method returns the protocol name, followed by the version number.

HTTP request headers

The request headers allow the servlets to perform a number of operations; here are a few of the most commonly used headers:

- **Accept-Charset**: This header lists the character sets that are allowed to be used by the browser.
- **Accept-Encoding**: This header is used to specify the encoding type required to process the content. It suggests the encoding that the client can handle. The server can process this header by encoding the page with the formats specified by the client and then sending the Content-Encoding response header to specify that it is applied. This header is useful when sending gzip or compress values.
- **Accept-Language**: If the server can produce output in more than one language, then this header can be used to mention the client's preferred language.
- **Authorization**: This header can be used by the clients to identify themselves when they are accessing password-protected web pages.

- **Connection**: This header is used to handle persistent HTTP connections, which means the client can retrieve multiple files with a single socket connection. This saves the overhead of several independent connections. With HTTP 1.1. onward, the connection is persistent by default; we must specify the close value to change it.
- **Content-Length**: This header is used when we user **`POST`** request, it gives the size of the POST data in bytes.
- **Cookie**: This header returns the cookies to servers that were previously sent to the client; instead of this header, we can use the **`request.getCookies()`** method, which gives the values directly; otherwise, in reading the header, we need to perform low-level parsing.
- **Host**: This header specifies the host and port number as provided by the original URL; due to virtual hosting, it may happen that the server could not reply.
- **If-modified-since**: This header is used to specify that the client wants the page only if the page is changed after the specified date. If the page is not modified, the server sends the status code as **`304`**, which means "not modified". This header is useful when the browser wants to reload the cache only when updated. Generally, servlets use the **`getLastModified`** method to retrieve the dates automatically.
- **If-Unmodified-Since**: This header specifies that the document should be retrieved if it is older than the specified date. Generally, the If-Modified-Since header is used when we use the **`GET`** request, and If-Unmodified-Since is used when we use the **`PUT`** request.
- **Referer**: This header stores the predecessor page **`url`** from which the request for the current page came in. For example, suppose the user has opened the Google page link and searched for a particular content in Java; now, if the user opens any page from the search result display list, the referrer page for the current page will be the URL of Google. This header is used to note the hierarchy of pages visited by the user.
- **User-Agent**: This header is used to identify the browser or client used to request the server.

The following example shows the usage of various headers.

Example:

```
01. public class ShowParameters extends HttpServlet
02.  {
03.    public void doGet(HttpServletRequest request,   HttpServletResponse response)
04.   {
05.      response.setContentType("text/html");
06.      PrintWriter out = response.getWriter();
07.                 String title=" Request Headers ";
08.          String docType =   "<!DOCTYPE  html>\n";
09.      out.println(docType +"<HTML>\n" +"<HEAD><TITLE>" + title + "</TITLE>
10.                 </HEAD>\n" +"<BODY >\n" +"<H1>" + title + "</H1>" );
11.                out.println("Accept-Encoding : "+request.getHeader("Accept-Encoding"));
12.                out.println("Accept-Language : "+request.getHeader("Accept-Language"));
13.                out.println("Host : "+request.getHeader("Host"));
14.                out.println("Referer : "+request.getHeader("Referer"));
15.                out.println("User-Agent : "+request.getHeader("User-Agent"));
16.           }
17. }
18.
19.
```

Figure 5.11: *Retrieve various Header values and display it.*

The preceding code outputs the following:

```
Accept-Encoding : gzip, deflate

Accept-Language : en-US,en;q=0.5

Host : localhost:8083

Referer : http://localhost:8083/RequestHeaders/
User-Agent : Mozilla/5.0 (Windows NT 10.0; WOW64; rv:47.0)
Gecko/20100101 Firefox/47.0
```

Generate the server response

The response code sent by the server specifies the type of operation performed by the server. The server responses can be accessed by various methods. The HTTP response status line consists of various fields like, version, status code and a message. All the messages are set for each status code, so the server just need to set the right status code, and the message is set automatically. By default, the status code **`200`** is set, so servlets need not specify it; but in case a different status code is to be set, the servlet can use the **`response.setStatus`**, **`response.sendRedirect`**, or **`response.sendError`** method for setting the specific status code.

Setting random status codes: setStatus

To set the status code of response explicitly, we can use the **`setStatus`** method. The method can take an integer number representing the status code as an argument, and a constant is defined in the **`HttpServletResponse`** to present each status code. For the sake of readability, it is always preferable to use the constants in the programmingprogramming. The name of each constant starts with SC, indicating the status code, followed by the underscore and the message; for example, **`404`** represents file not found, so the equivalent constant in **`HttpServletResponse`** is **`SC_NOT_FOUND`**.

Setting 302 and 404 status codes: sendRedirect and sendError

In general, we can set the status code by calling the **`response.setStatus(int)`** method, but there are two other shortcut methods we can use to perform the common operations of sending an error and redirecting:

```
public void sendRedirect(String url)
```

The **`302`** status code guides the browser to connect to a new location. The **`sendRedirect`** method generates a **`302`** response along with a Location header giving the URL of the new resource to access. The URL can be either an absolute or a relative URL. The system automatically translates the relative URL into the absolute URL by puttingURL in the location header.

```
public void sendError(int code, String message)
```

This method is used to send the status code and error message to the client, for example, the **`404`**-status code is used when the required resource is not found on the server. The **`sendError`** method sends a status code with a brief message that is formed and sent to the client.

To send the output, the server has to call the **`setStatus`** or **`sendError`** method first.

HTTP 1.1 status codes

The status codes fall in five major categories:

- **100–199**: The code falling in this range provides information that the client should respond with some other action.
- **200–299**: The code falling in this range indicates that the request was successful.

- **300–399**: The code falling in this range is used to indicate that the required resource has been relocated, and it generally includes a location header indicating the new address of the resource.
- **400–499**: The code falling in this range indicates the client-side errors.
- **500–599**: The code falling in this range mention the server-side errors.

The most common status code used are explained as follows:

- **100 (Continue):** This status code is used to indicate whether a client can continue to send a follow up resource or not. If the server sends 100 as a status code, it means that the client can send the request; otherwise, the server will send the 417-status code to tell the client that it will not accept the resource.
- **200 (OK):** This status code is used to indicate that everything is fine, and the document follows for the **`GET`** and **`POST`** request.
- **202 (Accepted):** This header tells the client that the request is under process and is not completed yet.
- **204 (No Content):** This status code is used to suggest that the required resource is not updated, and the previous resource can be used as it is. This is useful when the user reloads the page frequently and the server wants to notify that the page is already new and need not be changed.
- **205 (Reset Content):** This status code is used to reset the form fields and clear reset the form. The document is not new, but the form fields are cleared.
- **301 (Moved Permanently):** This status code indicates that the requested resource has been permanently moved to a new location, and the new URL for the resource is given in the Location response header. The browsers should consider the new URL for the required resource.
- **302 (Found):** This status code is similar to 301, where the required resource is moved to the new location but only temporarily. The location header should be read for the temporary replacement, not a permanent one.
- **303 (See Other):** This status code indicates that in case the original request for the resource was POST, the new document should be retrieved with **`GET`**.
- **304 (Not Modified):** This status code indicates that the file is not modified.
- **307 (Temporary Redirect):** This status code indicates temporary redirection of the required resource. It is similar to 302, but it is used to suggest the browser that follows redirected **`GET`** but not **`POST`** request.
- **400 (Bad Request):** This status code indicates wrong syntax in the client request.

- **401 (Unauthorized):** This status code indicates that the client tried to access an authorized page, but the request did not have proper identifying information in the Authenticate header.
- **403 (Forbidden):** This status code is used by the server to refuse to provide the required resource as permission is not allowed, regardless of the authorization.
- **404 (Not Found):** This status code is used to suggest that the required resource is not found at the given address.
- **405 (Method Not Allowed):** This status code is used to suggest that the methods `GET`, `POST`, `PUT`, `DELETE`, etc. are not allowed for the given resource.
- **415 (Unsupported Media Type):** This status code is used to signify that the request has an attachment whose format is not known to the server.
- **417 (Expectation Failed):** When the server receives a request header with a value 100, it means that the client is asking to send an attached document in a follow up request. In reply to that, the server can respond with 417, indicating that it cannot accept the document, or use 100 to indicate that the server is ready to accept and the client can continue to send.
- **500 (Internal Server Error):** It is used by the sever to indicate that the servlets have been crashed or a header formatted improperly has been returned.
- **501 (Not Implemented):** This status code is used by the server to indicate that the functionality required to process the request is not supported by the server.
- **503 (Service Unavailable):** This status code is used to represent that the server cannot respond to the client due to ongoing maintenance or high network traffic.
- **505 (HTTP Version Not Supported):** This status code is used to indicate that the version of HTTP mentioned in the request line is not supported by the server.

Handling the response headers

After the request is read and handled, the server creates the response and sends it to the client, so various response headers can be used to read the information a header can provide.

Setting response headers from servlets

The **setHeader** method can be used to specify the response header; it takes two arguments: header name and header value. It assigns the value to the given header **name.setHeader(String headerName, String headerValue)**.

Apart from the general-purpose method, other methods can be used to set the headers that can contain dates and **integers.setDateHeader(String header, long milliseconds)**. The method is used to convert a Java date into GMT time **string.setIntHeader(String header, int headerValue)**.The merhod is sed to convert an integer to a String.

```
setContentType(String mimeType)
```

It is used to specify the content type header: **setContentLength(int length)**.

It is used in persistent connections to set the content length header by the browser.

```
addCookie(Cookie c)
```

This method is used to insert a cookie into the **Set Cookie** header.

```
sendRedirect(String address)
```

This method is used to set the **Location** header and set the status code to **302**.

HTTP response headers

This section explains the most common response header that can be used from servlets to work more effectively. Response headers are case insensitive, but it is a common convention to write the first letter in capital:

- **Allow**: This header specifies the request methods that are supported by the server; it is used to respond to the status code **405.Cache-Control**. This header guides the browser with regard to the situation the response document can be cached in; it holds the following possible values:
    - **public**: It suggests that the document can be cached even when it should not be cached.
    - **Private**: It suggests that the document can be stored in the private cache only.
    - **no-cache**: It suggests that the document should never be cached. Browsers generally don't catch the documents with form data, but in case documents without form data need to not be cached, then this value should be used.
    - **no-store**: It suggests that the document should never be cached and not even stored in any temporary location on the disk.

  - **must-revalidate**: It suggests that the client must revalidate the document with the original server document each time it is used.
  - **proxy-revalidate**: It suggests that the client must revalidate the document with the original server document each time it is used when it is using the shared **caches.max-age=xxx**. It suggests that the document will be considered as old and not used after **xxx** seconds. If the header has max age and expires header both in the response, then max age will take precedence over expires header.
  - **s-max-age=xxx**: It is used for the shared caches to mention that the document will get old and unused after **xxx** seconds.
  - **Connection**: It suggests that use of *close* value for this header will instruct the browser to stop using persistent connections. As persistent connections use a **Content-Length** header, if we omit this header, we indicate that we are not using persistent connections.

- **Content-Disposition**: This header is used by the browser to ask the user whether they want to save the response to the disk in a file with a given name.
- **Content-Encoding**: This header is used to know the encoding style of the document being transmitted. The browser needs to reverse the encoding of the document before processing it.
- **Content-Language**: This header specifies language codes, for example, en, da, etc., in which the document is written.
- **Content-Length**: This header specifies the number of bytes in the response; it is useful when the browser is using a persistent connection.
- **Content-Type**: This header is used to give the MIME type of the document. MIME stands for Multipurpose Internet Mail Extension, which is used to set the main type and the subtype of the document.
- **setContentType**: This method can be used from the servlet to mention the type of response to be generated as an output; generally, the default type is **text/html**. Using this header, we can also specify the character encoding.

```
response.setContentType("text/html; charset=Shift_JIS");
```

The following is a list of some of the most common MIME types used by servlets:

Common MIME Types

Application main type / subtype	Description
`application/msword`	Microsoft Word document
`application/pdf`	Acrobat (.pdf) file
`application/vnd.ms-excel`	Excel spreadsheet
`application/vnd.ms-powerpoint`	PowerPoint presentation
`application/x-gzip`	Gzip archive
`application/x-java-archive`	JAR file
`application/x-java-vm`	Java bytecode (.class) file
`application/zip`	ZIP archive
`audio/basic`	Sound file in .au or .snd format
`audio/midi`	MIDI sound file
`audio/x-aiff`	AIFF sound file
`audio/x-wav`	Microsoft Windows sound file
`image/gif`	GIF image
`image/jpeg`	JPEG image
`image/png`	PNG image
`image/tiff`	TIFF image
`image/x-xbitmap`	X Windows bitmap image
`video/mpeg`	MPEG video clip
`video/quicktime`	QuickTime video clip
`text/css`	HTML cascading style sheet
`text/html`	HTML document
`text/plain`	Plaintext
`text/xml`	XML

***Table 5.1**: Common MIME types*

Expires

This header stipulates the time at which the content should be considered out-of-date and thus, no longer be cached. A servlet might use this header for a document that changes relatively frequently, to prevent the browser from displaying a stale cached value. For example, the following would instruct the browser not to cache the document for more than 10 minutes:

```
long currentTime = System.currentTimeMillis();
long tenMinutes = 10*60*1000; // In milliseconds
response.setDateHeader("Expires",  currentTime + tenMinutes);
```

Also, see the **`max-age`** value of the Cache-Control header.

Last-modified

This very useful header indicates when the document was last changed. The client can then cache the document and supply a date by an If-Modified-Since request header in later requests.

Location

This header, which should be included with all responses that have a status code in the 300s, notifies the browser of the document address. The browser automatically reconnects to this location and retrieves the new document. This header is usually set indirectly, along with a 302-status code, by the **`sendRedirect`** method of **`HttpServletResponse`**.

Pragma

Supplying this header with a value of no-cache instructs HTTP 1.0 clients not to cache the document. However, support for this header was inconsistent with HTTP 1.0 browsers, so expires with a date in the past is often used instead. In HTTP 1.1, Cache-Control: no-cache is a more reliable replacement.

Refresh

This header indicates how soon (in seconds) the browser should ask for an updated page. For example, to tell the browser to ask for a new copy in 30 seconds, you would specify a value of 30 with the following:

```
response.setIntHeader("Refresh", 30);
```

Instead of having the browser just reload the current page, you can specify the page to load by adding a semicolon and a URL after the refresh time. For example, to tell the browser to go to **`http://host/path`** after 5 seconds, you would write the following:

```
response.setHeader("Refresh", "5; URL=http://host/path/");
```

This setting is useful for "splash screens" on which an introductory image or message is displayed briefly before the real page is loaded.

Note that this header is commonly set indirectly by putting the following:

```
<META HTTP-EQUIV="Refresh"
      CONTENT="5; URL=http://host/path/">
```

This is put in the **HEAD** section of the HTML page rather than as an explicit header from the server. That usage came about because automatic reloading or forwarding is something often desired by authors of static HTML pages. For servlets, however, setting the header directly is easier and clearer.

Retry-after

This header can be used in conjunction with a **503 (Service Unavailable)** response to tell the client how soon it can repeat its request.

Set-Cookie

The **Set-Cookie** header specifies a cookie associated with the page. Each cookie requires a separate **Set-Cookie** header.

WWW-Authenticate

This header always includes a **401 (Unauthorized)** status code. It tells the browser what authorization type (**BASIC** or **DIGEST**) and realm the client should supply in its Authorization header.

Conclusion

The chapter covered the concept of servlet and its vital role in handling the web application. It also covered the request and response headers of HTTP, and how the HTML form data can be accessed in native Java environment.

The next chapter covers advanced features like session management, handling cookies, etc. from servlets.

Points to remember

- A servlet life cycle can be defined as the process from its creation till its destruction.
- The servlet is initialized by calling the **init ()** method.
- The servlet calls the **service()** method to process a client's request.
- The servlet is terminated by calling the **destroy()** method.
- The request headers provide the methods to read the headers for processing them.

- Server responses are formed of the HTTP response status line, including the HTTP version, a status code, and an associated message.

Multiple choice questions

1. **The server and client are aware of each other only during a current request. This is due to which of the following characteristics of HTTP?**
    a. Connectionless
    b. Stateless
    c. Media independent
    d. None of these
2. **Status code starting with 4XX indicates_____.**
    a. a server-side error
    b. information
    c. a client-side error
    d. a successful request
3. **_____ sets the response header with the designated name to the given value.**
    a. setHeader
    b. setIntHeader
    c. setDateHeader
    d. None of these
4. **_____ header indicates how soon (in seconds) the browser should ask for an updated page.**
    a. Retry-After
    b. Set-Cookie
    c. Pragma
    d. Refresh

Answers

1. **b**
2. **c**
3. **a**
4. **d**

Questions

1. What is servlet?
2. What is the benefit of servlet over CGI?
3. List and explain the request methods provided by HTTP briefly.
4. List and explain the basic characteristics of HTTP.
5. Which packages provide the servlet API?
6. Explain the servlet life cycle.
7. Explain the contentType header.

Key terms

- **Servlets**: They are Java programs that run on web or application servers, acting as a middle layer between requests coming from web browsers or other HTTP clients and databases or applications on the HTTP server.
- **HttpServletRequest**: It is a predefined interface in the **`javax.servlet.http`** package that describes and defines the contracts between a servlet class running under the HTTP protocol and the runtime environment provided for an instance of such a class by a conforming servlet container.
- **HttpServletResponse**: It is a predefined interface present in the **`javax.servlet.http`** package that will help write information about the data it will send back.

Join our book's Discord space

Join the book's Discord Workspace for Latest updates, Offers, Tech happenings around the world, New Release and Sessions with the Authors:

https://discord.bpbonline.com

Chapter 6
Managing Advance Features in Servlet

Introduction

In this chapter, we will cover advanced features like managing cookies and sessions by using servlets in depth. The chapter mainly focuses on the ways in which the state of the user can be maintained, which is known as session management. Handling cookies is important for identifying requests coming from the same client. Cookies can be persistent or non-persistent. In case the cookies are disabled from the browser, this approach will not work; so, we will look at session tracking using other techniques like Hidden Form field, URL rewriting and **`HttpSession`**.

Structure

In this chapter, we will discuss the following topics:

- Managing cookies
- Managing session

Objectives

After studying this chapter, you should be able to understand sending and receiving cookies, cookie attributes and utilities. You should also understand session and

persistent cookies. Additionally, you should be familiar with managing cookies to handle user accounts and preferences. Session management will be done by understanding the API for session tracking, types of sessions, and URL encoding.

Managing cookies

A web server sends small bits of text information to the browser, and the browser uses the same information when visiting that site; this textual information is called cookie. By using cookies, the users visiting site can be provided with several services. Cookies are useful in providing various benefits, as listed here:

- They help in recognizing a user during an e-commerce session. As HTTP is stateless when a user makes a request, it is not known that same user has made the request; keep-alive connections could be applied only to the requests generated in near time sessions. Cookies can be read and manipulated for further processing.
- Cookies are useful in remembering usernames and passwords. They let a user log in to a site automatically, ensuring convenience for users of unshared computers.
- When a user sets certain preferences while accessing the sites, cookies can be used to remember customizing sites.
- While browsing, displaying advertisement based on users' choice is one of the marketing practices, so cookies can be used for that, to let the site remember which topics interest certain users and show advertisements relevant to those interests.
- Cookies ensure convenience for the users and help add value to the site owners.
- Cookies do not create any security threat by reading its values and interpreting it; they cannot be used to insert viruses or launch other denial of service attacks. Also, they do not occupy much space. Yes, but cookies can present a threat to privacy by remembering the previously searched pages; the type of data accessed or searched by users is known, and advertisements can be shown based on that; users may not like this in case they view or search for personal or sensitive data. So due to privacy issues, some users turn off the cookies so that sensitive data is not open for somebody to access it.

Sending and receiving cookies

Servlet can use the **`Cookie`** constructor to send cookies to the client. Apart from it, any attribute can be set by calling the set method for that attribute, like **`cookie.`**

setXxx. Then the cookies can be inserted into the HTTP response headers with **response.addCookie**. To read the cookies, a servlet can use **request.getCookies**, which returns an array of **Cookie** objects related to the site you have accessed from the browser; from this array, the servlet can find the cookie it is looking for using the **getName** method.

Sending cookies to the client

Sending cookies to the client involves three steps:

1. The first step is to create a cookie object by calling the parameterized constructor with the cookie name and value.
2. The second step is to set the maximum age of the cookies; to save the cookie on disk rather than keeping it just in memory, you can specify the time interval in seconds using **setMaxAge**.
3. The third step is to place the cookie in the **Http** response header using the **addCookie** method.

To apply the first step for creating the cookie, we can call the constructor to and pass the cookie name and value, for example, to create a cookie named **userID** with a value **user1**, we can use the parameterized constructor as follows:

```
Cookie c = new Cookie("userID", "user1");
```

Setting the maximum age

When we create the cookie and send it to the browser, it is preserved in the browser's memory and removed when the user closes the browser, so it is stored for the current session. To store the cookie on disk, we can use **setMaxAge** by specifying time in seconds, for example, to store the cookie for 1 week, we can write the method as follows:

```
c.setMaxAge(60*60*24*7);
```

Setting the maximum age to 0 instructs the browser to delete the cookie.

Placing the Cookie in the Response Headers

We can send the cookie using the **addCookie** method. Here's an example:

```
Cookie userCookie = new Cookie("user", "uid");
userCookie.setMaxAge(60*60*24*365); // Store cookie for 1 year
response.addCookie(userCookie);
```

Reading cookies from the client

To read the cookies that come back from the client, we can call **request.getCookies**, which returns an array of **Cookie** objects, from the array we can retrieve each cookie name using getName method and its value using getValue method..

Consider this example:

```
String cookieName = "userID";
Cookie[] cookies = request.getCookies();
if (cookies != null) {
  for(int i=0; i<cookies.length; i++) {
    Cookie cookie = cookies[i];
    if (cookieName.equals(cookie.getName())) {
              cookie.getValue();
    }
  }
}
```

Using cookies, we can differentiate whether a client is new or a repeated visitor. We can check for the cookie, and if it is there, the client is a repeated visitor; otherwise, the client is new.

```
Cookie[] cookies = request.getCookies();
if (cookies == null) {
      // Code to handle new client.
} else {
 // Code to handle repeated visitor.
}
```

Cookie attributes

We can add various characteristics to the cookie by calling various setter methods. These methods are of the **setXxx** form, where **Xxx** is the attribute we want to set. And we can retrieve the attribute values using **getXxx** methods. Here are the methods that set the cookie attributes:

```
public void setComment(String comment)
public String getComment()
```

These methods specify or look up a comment associated with the cookie:

```
public void setDomain(String domainPattern)
public String getDomain()
```

These methods set or retrieve the domain to which the cookie applies:

```
public void setMaxAge(int lifetime)
public int getMaxAge()
```

This method is used to specify time interval in seconds for which the cookie will remain alive on disk. A negative value is used to indicate that the cookie will be alive only for the current browsing session.

```
public String getName()
```

The **`getName`** method retrieves the name of the cookie. The name and the value are the two pieces you always care about. However, since the name is supplied to the **`Cookie`** constructor, there is no **`setName`** method; you cannot change the name once the cookie is created.

```
public void setPath(String path)
public String getPath()
```

These methods set or get the path to which the cookie applies. If you don't specify a path, the browser returns the cookie only to URLs in or below the directory containing the page that sent the cookie.

```
public void setSecure(boolean secureFlag)
public boolean getSecure()
```

This pair of methods sets or gets the Boolean value indicating whether the cookie should only be sent over encrypted (that is, SSL) connections. The default is false; the cookie should apply to all connections.

```
public void setValue(String cookieValue)
public String getValue()
```

The **`setValue`** method specifies the value associated with the cookie; **`getValue`** looks it up.

```
public void setVersion(int version)
public int getVersion()
```

These methods **`set`** and **`get`** the cookie protocol version with which the cookie complies.

Session and cookies

To understand the behavior of cookies with and without setting the maximum age of the cookie, we use a **`TestCookie`** servlet to understand this concept. The servlet is written to perform two main tasks:

- The first task is to set the six cookies, out of which three cookies have implicit age of negative value, and the age of the other three is set to 1 hour. The

cookies with negative value will persist only for the current browsing session, and the other three cookies will persist for 1 hour and their value will be preserved on the disk, irrespective of whether or not the browser is closed.

- The second task is to display all the cookies on a table with their name and values.

Example: Write a servlet to create six cookies to set three with negative time interval and three with 1 hour, that is, 3600 seconds persistent, and display the cookie name and value in a table format.

TestCookie.java

```
01. package coreservlets;
02. import java.io.*;
03. import javax.servlet.*;
04. import javax.servlet.http.*;
05.
06. public class TestCookie extends HttpServlet {
07.   public void doGet(HttpServletRequest request,HttpServletResponse response)
08.       throws ServletException, IOException {
09.     for(int i=0; i<3; i++) {
10.       Cookie cookie = new Cookie("Session-Cookie-" + i,"Cookie-Value-S" + i);
11.       response.addCookie(cookie);
12.       cookie = new Cookie("Persistent-Cookie-" + i,"Cookie-Value-P" + i);
13.       cookie.setMaxAge(3600);
14.       response.addCookie(cookie);
15.     }
16.     response.setContentType("text/html");
17.     PrintWriter out = response.getWriter();
18.     String title = "Active Cookies";
19.     out.println("<html>\n" + "<head><title>" + title + "</title></head>\n" +
20.                 "<body bgcolor=\"#fdf5e6\">\n" +
21.                 "<h1 align=\"center\">" + title + "</h1>\n" +
22.                 "<table border=1 align=\"center\">\n" +
23.                 "<tr bgcolor=\"#ffad00\">\n" +
24.                 "  <th>cookie name\n" +
25.                 "  <th>cookie value");
26.     Cookie[] cookies = request.getCookies();
27.     if (cookies == null) {
28.       out.println("<TR><TH COLSPAN=2>No cookies");
29.     } else {
30.       Cookie cookie;
31.       for(int i=0; i<cookies.length; i++) {
32.         cookie = cookies[i];
33.         out.println("<TR>\n" +
34.                     "  <TD>" + cookie.getName() + "\n" +
35.                     "  <TD>" + cookie.getValue());
36.       }
37.     }
38.     out.println("</TABLE></BODY></HTML>");
39.   }
40. }
41.
```

Figure 6.1: *Generate Table from servlet*

The result of the initial visit to the **`TestCookie`** servlet when visiting the servlet, quitting the browser, waiting an hour, and revisiting the servlet will be none, after one hour of the original visit of TestCookie on revisit the browser will display the cookie name and its value.

Cookie utilities

This section presents some simple but useful utilities for dealing with cookies.

Finding cookies with specified names

The following code specifies the use of static methods in the **`Utilities`** class to retrieve the cookie value if we know the cookie name. In case the cookie name does not match, the default value is returned:

```
String color = Utilities.getCookieValue(request, "color", "black");
String font =  Utilities.getCookieValue(request, "font", "Arial");
```

The **`getCookie`** method is used to return the entire cookie object, not just the value so that it can be used when you require more operations to be performed on the cookie other than the value.

Example: Write a program to retrieve the cookie object by locating the name of cookie; in case no matching cookie is found, return null value.

Utilities.java

```
01. package coreservlets;
02. import javax.servlet.*;
03. import javax.servlet.http.*;
04.
05. public class Utilities {
06.     public static String getCookieValue(HttpServletRequest request, String cookieName, String defaultValue) {
07.     Cookie[] cookies = request.getCookies();
08.     if (cookies != null) {
09.       for(int i=0; i<cookies.length; i++) {
10.         Cookie cookie = cookies[i];
11.         if (cookieName.equals(cookie.getName())) {
12.           return(cookie.getValue());
13.         }
14.       }
15.     }
16.     return(defaultValue);
17.   }
18.
19.   public static Cookie getCookie(HttpServletRequest request,String cookieName) {
20.     Cookie[] cookies = request.getCookies();
21.     if (cookies != null) {
22.       for(int i=0; i<cookies.length; i++) {
23.         Cookie cookie = cookies[i];
24.         if (cookieName.equals(cookie.getName())) {
25.           return(cookie);
26.         }
27.       }
28.     }
29.     return(null);
30.   }
31. }
```

Figure 6.2: *Manage cookies*

Creating long-lived cookies

The following code shows a small class that you can use instead of **`Cookie`** if you want your cookie to automatically persist for a year when the client quits the browser. This class (**`LongLivedCookie`**) merely extends **`Cookie`** and calls **`setMaxAge`** automatically.

Example: Cookie that persists 1 year; a default cookie doesn't persist past the current browsing session.

LongLivedCookie.java

```
01. package coreservlets;
02.
03. import javax.servlet.http.*;
04.
05. public class LongLivedCookie extends Cookie {
06.   public static final int SECONDS_PER_YEAR = 60*60*24*365;
07.
08.   public LongLivedCookie(String name, String value) {
09.     super(name, value);
10.     setMaxAge(SECONDS_PER_YEAR);
11.   }
12. }
13.
14.
```

Figure 6.3: *Set the max age of cookies*

Cookie utilities

The preceding code is written again to add the functionality; when a user visits for the first time, it displays **`Welcome`**, and it displays **`Welcome Back`** all other times. Instead of calling **`request.getCookies`** and looping down that array examining each name, it merely calls **`Utilities.getCookieValue`**. Instead of creating a **`Cookie`** object, calculating the number of seconds in a year, and then calling **`setMaxAge`**, it merely creates a **`LongLivedCookie`** object.

Example: A variation of the **`RepeatVisitor`** servlet that uses **`Utilities.getCookieValue`** and **`LongLivedCookie`** to simplify the code:

RepeatVisitor.java

```
01. package coreservlets;
02. import java.io.*;
03. import javax.servlet.*;
04. import javax.servlet.http.*;
05.
06. public class RepeatVisitor extends HttpServlet {
07.   public void doGet(HttpServletRequest request, HttpServletResponse response)
08.       throws ServletException, IOException {
09.     boolean newbie = true;
10.     String value =
11.       Utilities.getCookieValue(request, "repeatVisitor",
12.                                "no");
13.     if (value.equals("yes")) {
14.       newbie = false;
15.     }
16.     String title;
17.     if (newbie) {
18.       LongLivedCookie returnVisitorCookie =
19.         new LongLivedCookie("repeatVisitor", "yes");
20.       response.addCookie(returnVisitorCookie);
21.       title = "Welcome";
22.     } else {
23.       title = "Welcome Back";
24.     }
25.     response.setContentType("text/html");
26.     PrintWriter out = response.getWriter();
27.     String docType =
28.       "<!doctype html> \n";
29.     out.println(doctype +
30.                 "<html>\n" +
31.                 "<head><title>" + title + "</title></head>\n" +
32.                 "<body bgcolor=\"#fdf5e6\">\n" +
33.                 "<h1 align=\"center\">" + title + "</h1>\n" +
34.                 "</body></html>");
35.   }
36. }
37.
```

Figure 6.4: *Manage cookies*

Here, on the first visit of the client, it prints **`Welcome Abroad`**, and for the subsequent visits, it prints **`Welcome Back`**.

Using cookies to handle user accounts and preferences

In the previous examples, we set the cookie and never changed it; in this section, we will see how we can change the value of cookie periodically. To replace the

previous cookie value, send the same cookie name with a different cookie value. In the following code, we present a servlet that keeps track of how many times each client has visited the page. It does this by making a cookie whose name is count and whose value is the actual count. To accomplish this task, the servlet needs to repeatedly replace the cookie value by resending a cookie with the identical name.

AccessCounts.java

```
01. package coreservlets;
02. import java.io.*;
03. import javax.servlet.*;
04. import javax.servlet.http.*;
05.
06. public class AccessCounts extends HttpServlet {
07.   public void doGet(HttpServletRequest request,
08.                     HttpServletResponse response)
09.       throws ServletException, IOException {
10.     String countString =
11.     Utilities.getCookieValue(request,"count","1");
12.     int c = 1;
13.     try {
14.       c = Integer.parseInt(countString);
15.     } catch(NumberFormatException nfe) { }
16.     LongLivedCookie co =
17.       new LongLivedCookie("count",
18.                           String.valueOf(c+1));
19.     response.addCookie(co);
20.     response.setContentType("text/html");
21.     PrintWriter out = response.getWriter();
22.     String title = "Access Count Servlet";
23.     String docType =
24.       "<!doctype html">\n";
25.     out.println(doctype +
26.                 "<html>\n" +
27.                 "<head><title>" + title + "</title></head>\n" +
28.                 "<body bgcolor=\"#fdf5e6\">\n" +
29.                 "<center>\n" +
30.                 "<h1>" + title + "</h1>\n" +
31.                 "<h2>this is visit number " +
32.                 count + " by this browser.</h2>\n" +
33.                 "</center></body></html>");
34.   }
35. }
36.
37.
```

Figure 6.5: *Manage access counts using cookies*

The program displays the user count, that is, the total number of times the user visited the page; this count is maintained even in different browsers. One of the most common applications of cookies is to use them to *remember* user preferences. For simple user settings, as here, the preferences can be stored directly in the cookies. For more complex applications, the cookie typically contains a unique user identifier, and the preferences are stored in a database.

The code in figure 6.6 presents a servlet that creates an input form The form is displayed again if the data is incomplete; otherwise, the parameter values are

displayed. If a parameter is missing, the second servlet redirects the user to the original servlet so that the form can be redisplayed. The original servlet maintains the user's previously entered values by extracting them from the cookies. The form remembers the previous entries. The fields are prepopulated with whatever values the user entered on the most recent request.

Example: A servlet that displays an HTML form to collect a user's first name, last name, and email address, it uses cookies to determine the initial values of each of those form fields.

RegistrationForm.java

```
01. package coreservlets;
02. import java.io.*;
03. import javax.servlet.*;
04. import javax.servlet.http.*;
05.
06. public class RegistrationForm extends HttpServlet {
07.   public void doGet(HttpServletRequest request,
08.                     HttpServletResponse response)
09.       throws ServletException, IOException {
10.     response.setContentType("text/html");
11.     PrintWriter out = response.getWriter();
12.     String actionURL =
13.       "/servlet/ RegistrationServlet";
14.     String firstName =
15.       Utilities.getCookieValue(request, "firstName", "");
16.     String lastName =
17.      Utilities.getCookieValue(request, "lastName", "");
18.     String emailAddress =
19.       Utilities.getCookieValue(request, "emailAddress",
20.                                "");
21.     String docType =
22.       "<!doctype html">\n";
23.     String title = "please register";
24.     out.println
25.       (doctype +
26.        "<html>\n" +
27.        "<head><title>" + title + "</title></head>\n" +
28.        "<body bgcolor=\"#fdf5e6\">\n" +
29.        "<center>\n" +
30.        "<h1>" + title + "</h1>\n" +
31.        "<form action=\"" + actionurl + "\">\n" +
32.        "first name:\n" +
33.        "   <input type=\"text\" name=\"firstname\" " +
34.                  "value=\"" + firstname + "\"><br>\n" +
35.        "last name:\n" +
36.        "   <input type=\"text\" name=\"lastname\" " +
37.                    "value=\"" + lastname + "\"><br>\n" +
38.        "email address: \n" +
39.        "   <input type=\"text\" name=\"emailaddress\" " +
40.                  "value=\"" + emailaddress + "\"><p>\n" +
41.        "<input type=\"submit\" value=\"register\">\n" +
42.        "</form></center></body></html>");
43.   }
44. }
45.
```

Figure 6.6: *Handle registration page*

Example: A servlet that processes a registration form containing a user's first name, last name, and email address. If all the values are present, the servlet displays the values. If any of the values are missing, the input form is redisplayed. Either way, the values are put into cookies so that the input form can use the previous values.

RegistrationServlet.java

```
01. package coreservlets;
02. import java.io.*;  import javax.servlet.*;  import javax.servlet.http.*;
03.
04. public class RegistrationServlet extends HttpServlet {
05.   public void doGet(HttpServletRequest request, HttpServletResponse response) throws ServletException, IOException {
06.     response.setContentType("text/html");
07.     boolean isMissingValue = false;
08.     String firstName = request.getParameter("firstName");
09.     if (isMissing(firstName)) {
10.       firstName = "Missing first name";
11.       isMissingValue = true;
12.     }
13.     String lastName = request.getParameter("lastName");
14.     if (isMissing(lastName)) {
15.       lastName = "Missing last name";
16.       isMissingValue = true;
17.     }
18.     String emailAddress = request.getParameter("emailAddress");
19.     if (isMissing(emailAddress)) {
20.       emailAddress = "Missing email address";
21.       isMissingValue = true;
22.     }
23.     Cookie c1 = new LongLivedCookie("firstName", firstName);
24.     response.addCookie(c1);
25.     Cookie c2 = new LongLivedCookie("lastName", lastName);
26.     response.addCookie(c2);
27.     Cookie c3 = new LongLivedCookie("emailAddress",emailAddress);
28.     response.addCookie(c3);
29.     String formAddress ="/servlet/RegistrationForm";
30.     if (isMissingValue) {
31.       response.sendRedirect(formAddress);
32.     } else {
33.       PrintWriter out = response.getWriter();
34.       String title = "Thanks for Registering";
35.       String docType = "<!doctype html">\n";
36.         out.println(doctype +"<html>\n" +"<head><title>" + title + "</title></head>\n" +
37.         "<body bgcolor=\"#fdf5e6\">\n" + "<center>\n" + "<h1 align>" + title + "</h1>\n" +
38.         "<ul>\n" + "  <li><b>first name</b>: " + firstname + "\n" + "  <li><b>last name</b>: " +
39.         lastname + "\n" + "  <li><b>email address</b>: " + emailaddress + "\n" + "</ul>\n" +
40.          "</center></body></html>");
41.     }
42.   }
43.   private boolean isMissing(String param) {
44.     return((param == null) ||  (param.trim().equals("")));
45.   }
46. }
```

Figure 6.7: *Manage registration servlet*

Managing session

This section explains how session tracking can be carried out.

Overview of session tracking

HTTP is a *stateless* protocol: each time a client retrieves a web page, the client opens a separate connection to the web server, and the server does not automatically maintain contextual information about the client. Even with servers that support persistent (keep-alive) HTTP connections and keep sockets open for multiple client requests that occur in rapid succession, there is no built-in support for maintaining contextual information. This lack of context causes several difficulties. For example, when clients at an online store add an item to their shopping carts, how does the server know what's already in the carts? Similarly, when clients decide to proceed to checkout, how can the server determine which previously created shopping carts are theirs? These questions seem very simple, yet, because of the inadequacies of HTTP, answering them is surprisingly complicated.

There are three typical solutions to: to manage the stateless properties of the protocol. cookies, URL rewriting, and hidden form fields. The following subsections quickly summarize what would be required if you had to implement session tracking yourself (without using the built-in session-tracking API) for each of the three ways.

Cookies

You can use cookies to store an ID for a shopping session; with each subsequent connection, you can look up the current session ID and use it to extract information about that session from a lookup table on the server machine. So, there would really be two tables: one that associates session IDs with user tables, and the user tables themselves that store user-specific data. For example, on the initial request, a servlet could do something like the following:

```
String sessionID = makeUniqueString();
HashMap sessionInfo = new HashMap();
HashMap globalTable = findTableStoringSessions();
globalTable.put(sessionID, sessionInfo);
Cookie sessionCookie = new Cookie("JSESSIONID", sessionID);
sessionCookie.setPath("/");
response.addCookie(sessionCookie);
```

In later requests, the server could use the **`globalTable`** hash table to associate a session ID from the **`JSESSIONID`** cookie with the **`sessionInfo`** hash table of user-specific data.

Using cookies in this manner is an excellent solution and is the most widely used approach for session handling. Still, it is nice that servlets have a higher-level API that handles all this and the following tedious tasks:

- Extracting the cookie that stores the session identifier from the other cookies
- Determining when idle sessions have expired and reclaiming them
- Associating the hash tables with each request
- Generating the unique session identifiers

URL rewriting

With this approach, the client appends some extra data on the end of each URL. That data identifies the session, and the server associates that identifier with user-specific data it has stored. For example, with **`http://host/path/file.html;jsessionid=s123`**, the session identifier is attached as **`jsessionid=s123`**, so **`s123`** is the ID that uniquely identifies the table of data associated with that user.

URL rewriting is a moderately good solution for session tracking and also has the advantage that it works when browsers don't support cookies or when the user has disabled them. However, if you implement session tracking yourself, URL rewriting has the same drawback as cookies, namely, that the server-side program has a lot of straightforward but tedious processing to do. Even with a high-level API that handles most of the details for you, you must be very careful that every URL that references your site and is returned to the user (even by indirect means like location fields in server redirects) has the extra information appended. This restriction means you cannot have any static HTML pages on your site (at least not any that links back to dynamic pages at the site). So, every page must be dynamically generated with servlets or JSP. Even when all the pages are dynamically generated, if the user leaves the session and comes back via a bookmark or link, the session information can be lost because the stored link contains the wrong identifying information.

Hidden form fields

HTML forms can have an entry that looks as follows:

```
<input type="hidden" name="session" value="s1">
```

The mentioned field indicates the hidden form field that can be used when the form is submitted, and the specified name and value can be automatically included in the **`GET`** and **`POST`** data. The hidden field can be used to hold the session value, but it has a limitation: it will only work when the page is dynamically generated by submitting the form; using just anchor tag to click on the link will not work, so hidden field can be used to track the session only for particular operations, not for generic session tracking.

Session tracking in servlets

The **HttpSession** API provides a very good way of session tracking with a high lever interface that is built on the top of cookies or URL rewriting. Most servers have support for session tracking with cookies, and most of them can move to URL rewriting. Using the servlets will provide an easy way to manage session without explicitly manipulating cookies or the values that are appended to the URL. With servlets, different scopes are provided to store the information regarding each session.

Using session handling through servlets is very simple, and it has four basic steps:

- Accessing the session object associated with the current request. Call **request.getSession** to get an **HttpSession** object, which is a simple hash table for storing user-specific data.
- Looking up information associated with a session. Call **getAttribute** on the **HttpSession** object, cast the return value to the appropriate type, and check whether the result is null.
- Storing information in a session. Use **setAttribute** with a key and a value.
- Discarding session data. Call **removeAttribute** to discard a specific value. Call invalidates to discard an entire session. Call logout to log the client out of the web server and invalidate all sessions associated with that user.

Accessing the session object associated with the current request

Session objects are of type **HttpSession**, but they are just hash tables that can store arbitrary user objects (each associated with a key). You look up the **HttpSession** object by calling the **getSession** method of **HttpServletRequest**, as follows:

```
HttpSession session = request.getSession();
```

The working of session tracking involves fetching the user ID from a cookie or attached URL data, and then the ID is used as a key to find the user in the previously created **HttpSession** objects. This is done by invoking the **getSession** method. If no session ID is found in an incoming cookie or attached URL information, the system creates a new, empty session. And if cookies are being used (the default situation), the system also creates an outgoing cookie named **JSESSIONID** with a unique value representing the session ID. So, although you call **getSession** on the request, the call can affect the response. Consequently, you are permitted to call **request.getSession** only when it would be legal to set HTTP response headers: before any document content has been sent (that is, flushed or committed) to the client.

Now, if you plan to add data to the session, regardless of whether data was there already, **`getSession()`** (or, equivalently, **`getSession(true)`**) is the appropriate method call because it creates a new session if no session already exists. However, suppose you merely want to print out information on what is already in the session, as you might at a **View Cart** page at an e-commerce site. In such a case, it is wasteful to create a new session when no session exists already. So, you can use **`getSession(false)`**, which returns null if no session already exists for the current client.

```
HttpSession session = request.getSession(false);
if (session == null) {
  printMessageSayingCartIsEmpty();
} else {
  extractCartAndPrintContents(session);
}
```

Looking up information associated with a session

The **`HttpSession`** objects are created on the server, and they are automatically associated with the client using cookies or URL rewriting. The session objects have a built-in hash table data structure in which we can store any number of key-value pairs and retrieve those values using the **`getAttribute`** method. If the key is found, then the value is returned; otherwise, null is returned.

```
HttpSession session = request.getSession();
SomeClass value =
  (SomeClass)session.getAttribute("key");
if (value == null) { // No such object already in session
  value = new SomeClass(...);
  session.setAttribute("key", value);
}
codetomanipulatethevalue(value);
```

In most cases, you have a specific attribute name in mind and want to find the value (if any) already associated with that name. However, you can also discover all the attribute names in a given session by calling **`getAttributeNames`**, which returns an enumeration.

Associating information with a session

To read information associated with a session, we can use **`getAttribute`**. To specify information, use **`setAttribute`**. To let your values perform side effects when they are stored in a session, simply have the object you are associating with the session

implement the **`HttpSessionBindingListener`** interface. That way, every time **`setAttribute`** is called on one of those objects, its **`valueBound`** method is called immediately afterward.

Be aware that **`setAttribute`** replaces any previous values; to remove a value without supplying a replacement, use **`removeAttribute`**. This method executes the **`valueUnbound`** method of any values that implement **`HttpSessionBindingListener`**.

The following is an example of adding information to a session. You can add information in two ways: by adding a new session attribute or by augmenting an object that is already in the session (as in the last line of the example):

```
HttpSession session = request.getSession();
SomeClass value =
  (SomeClass)session.getAttribute("key");
if (value == null) { // No such object already in session
  value = new SomeClass(...);
  session.setAttribute("key", value);
}
codetomanipulatethevalue (value);
```

In general, primitive attributes merely have to be of type object (that is, they can be anything other than null or primitive like int, double, or boolean). However, some application servers support distributed web applications in which an application is shared across a cluster of physical servers. Session tracking needs to still work in such a case, so the system needs to be able to move session objects from machine to machine. Thus, if you run in such an environment and mark your web application as being distributable, we must implement the Serializable interface.

Discarding session data

When you are done with a user's session data, you have three options:

- Remove only the data your servlet created. You can call **`removeAttribute("key")`** to discard the value associated with the specified key. This is the most common approach.
- Delete the whole session (in the current web application). You can call invalidate to discard an entire session. Just remember that doing so causes all of that user's session data to be lost, not just the session data that your servlet or JSP page created. So, all the servlets and JSP pages in a web application have to agree on the cases for which invalidate may be called.
- Log the user out and delete all sessions belonging to them. Finally, in servers that support servlets 2.4 and JSP 2.0, you can call logout to log the client out of

the web server and invalidate all sessions (at most one per web application) associated with that user. Again, since this action affects servlets other than your own, be sure to coordinate the use of the logout command with the other developers at your site.

API for session tracking

Although the session attributes (that is, the user data) are the pieces of session information you care most about, other information is sometimes useful as well. Here's a summary of the methods available in the **`HttpSession`** class:

```
public Object getAttribute(String name)
```

This method extracts a previously stored value from a session object. It returns null if no value is associated with the given name.

```
public Enumeration getAttributeNames()
```

This method returns the names of all attributes in the session.

```
public void setAttribute(String name, Object value)
```

This method associates a value with a name. If the object supplied to **`setAttribute`** implements the **`HttpSessionBindingListener`** interface, the object's **`valueBound`** method is called after it is stored in the session. Similarly, if the previous value implements **`HttpSessionBindingListener`**, its **`valueUnbound`** method is called.

```
public void removeAttribute(String name)
```

This method removes any values associated with the designated name. If the value being removed implements **`HttpSessionBindingListener`**, its **`valueUnbound`** method is called.

```
public void invalidate()
```

This method invalidates the session and unbinds all objects associated with it. Use this method with caution; remember that sessions are associated with users (that is, clients), not with individual servlets or JSP pages. So, if you invalidate a session, you might be destroying data that another servlet or JSP page is using:

```
public void logout()
```

This method logs the client out of the web server and invalidates all sessions associated with that client. The scope of logout is the same as the scope of authentication. For example, if the server implements single sign-on, calling logout logs the client out of all web applications on the server and invalidates all sessions (associated with the client:

```
public String getId()
```

This method returns the unique identifier generated for each session. It is useful for debugging or logging or, in rare cases, for programmatically moving values out of memory and into a database (however, some J2EE servers can do this automatically):

```
public boolean isNew()
```

This method returns true if the client (browser) has never seen the session, usually because the session was just created rather than being referenced by an incoming client request. It returns false for preexisting sessions. The main reason for mentioning this method is to steer you away from it: **`isNew`** is much less useful than it appears at first glance. Many beginning developers try to use **`isNew`** to determine whether users have been to their servlet before (within the session timeout period), writing code as follows:

```
HttpSession session = request.getSession();
if (session.isNew()) {
  doStuffForNewbies();
} else {
  doStuffForReturnVisitors();  // Wrong!
}
```

If **`isNew`** returns true, then as far as you can tell, this is the user's first visit (at least within the session timeout). But if **`isNew`** returns false, it merely shows that they have visited the web application before, not that they have visited your servlet or JSP page before.

```
public long getCreationTime()
```

This method returns the time in milliseconds since midnight, January 1, 1970 (GMT) at which the session was first built. To get a value useful for printing, pass the value to the Date constructor or the **`setTimeInMillis`** method of **`GregorianCalendar`**.

```
public long getLastAccessedTime()
```

This method returns the time in milliseconds since midnight, January 1, 1970 (GMT) at which the session was last accessed by the client.

```
public int getMaxInactiveInterval()
public void setMaxInactiveInterval(int seconds)
```

These methods get or set the length of time, in seconds, that a session should go without access before being automatically invalidated. A negative value specifies that the session should never time out. Note that the timeout is maintained on the server and is not the same as the cookie expiration date. For one thing, sessions

are normally based on in-memory cookies, not persistent cookies, so there is no expiration date. Even if you intercepted the JSESSIONID cookie and sent it out with an expiration date, browser sessions and server sessions are two distinct things.

Types of sessions

By default, session tracking is based on cookies that are stored in the browser's memory, not written to disk. Thus, unless the servlet explicitly reads the incoming JSESSIONID cookie, sets the maximum age and path, and sends it back out, quitting the browser results in the session being broken: the client will not be able to access the session again. The problem, however, is that the server does not know that the browser was closed and thus, it has to maintain the session in memory until the inactive interval has been exceeded.

Consider a physical shopping trip to a Walmart store. You browse around and put some items in a physical shopping cart, then leave that shopping cart at the end of an aisle while you look for another item. A clerk walks up and sees the shopping cart. Can they reshelve the items in it? No—you are probably still shopping and will come back for the cart soon. What if you realize that you have lost your wallet, so you get in your car and drive home? Can the clerk reshelve the items in your shopping cart now? Again, no—the clerk presumably does not know that you have left the store. So, what can the clerk do? They can keep an eye on the cart, and if nobody has touched it for some time, they can then conclude that it is abandoned and take the items out of it. The only exception is if you brought the cart to him and said, *I'm sorry, I left my wallet at home, so I have to leave*.

The analogous situation in the servlet world is one in which the server is trying to decide whether it can throw away your **`HttpSession`** object. Just because you are not currently using the session does not mean the server can throw it away. Maybe you will be back (submit a new request) soon? If you quit your browser, thus causing the browser-session-level cookies to be lost, the session is effectively broken. But, as with the case of getting in your car and leaving Walmart, the server does not know that you quit your browser. So, the server must wait for some time to see if the session has been abandoned. Sessions automatically become inactive when the amount of time between client accesses exceeds the interval specified by **`getMaxInactiveInterval`**. When this happens, objects stored in the **`HttpSession`** object are removed (unbound). Then, if those objects implement the **`HttpSessionBindingListener`** interface, they are automatically notified. The one exception to *the server waits until sessions time out* rule is if invalidate or logout is called. This is akin to your explicitly telling the Walmart clerk that you are leaving, so the server can immediately remove all the items from the session and destroy the session object.

URL encoding

By default, servlet containers (engines) use cookies as the underlying mechanism for session tracking. We can use URL rewriting instead. The core session tracking code does not need to change at all, but lots of other code has to change. In particular, if any of your pages contain links that redirect to your own site, you have to explicitly add the session data to the URL. Now, the servlet API provides methods to add this information to whatever URL you specify. The problem is that you must call these methods; it is not technically feasible for the system to examine the output of all of your servlets and JSP pages, figure out which parts contain hyperlinks back to your site, and modify those URLs. You must tell it which URLs to modify. This requirement means you cannot have static HTML pages if you use URL rewriting for session tracking, or at least you cannot have static HTML pages that refer to your own site. This is a significant burden in many applications but worth the price in a few.

If you use URL rewriting for session tracking, most or all your pages will have to be dynamically generated. You cannot have any static HTML pages that contain hyperlinks to dynamic pages at your site. There are two possible situations in which you may use URLs that refer to your own site.

The first one is where the URLs are embedded in the web page that the servlet generates. These URLs should be passed through the **`encodeURL`** method of **`HttpServletResponse`**. The method determines whether URL rewriting is currently in use and appends the session information only if necessary. The URL is returned unchanged otherwise.

Here's an example:

```
String originalURL = someRelativeOrAbsoluteURL;
String encodedURL = response.encodeURL(originalURL);
out.println("<A HREF=\"" + encodedURL + "\">...</A>");
```

We can use encodeRedirectURL method to get the URL of our own site when we are using sendRedirect call.`String originalURL = someURL;`

```
String encodedURL = response.encodeRedirectURL(originalURL);
response.sendRedirect(encodedURL);
```

If you think there is a reasonable likelihood that your web application will eventually use URL rewriting instead of cookies, it is good practice to plan ahead and encode URLs that reference your own site.

Conclusion

The chapter covered two important concepts: handling cookies and session tracking. Both concepts are very important in developing a web application. Managing cookies can help the site administrator identify users, preserve username and passwords, customize the site, and post advertisements based on user preferences. HTTP is a *stateless* protocol: each time a client retrieves a web page, the client opens a separate connection to the web server and the server does not automatically maintain contextual information about the client. Servlets provide an outstanding session tracking solution: the **`HttpSession`** API.

The next chapter covers the basics of Java server pages, which is used to form a dynamic view using Java.

Points to remember

- A collection of all the requests made by a single client is called a session.
- A small piece of information that is saved between multiple client requests is called a cookie. The information includes name, value and attributes, like maximum age and version number.
- The **`HttpSession`** interface is used to create a session between an HTTP client and server and identify users among multiple requests.

Multiple choice questions

1. **Which of the following is not a session tracking method?**
    a. URL rewriting
    b. History
    c. Cookies
    d. SSL sessions

2. **Which of the following is stored at client side?**
    a. URL rewriting
    b. Hidden form fields
    c. SSL sessions
    d. Cookies

3. **Which of the following is not true about session?**
    a. All users connect to the same session.
    b. All users have the same session variable.
    c. The default timeout value for session variable is 20 minutes.
    d. A new session cannot be created for a new user.

4. **Which method creates unique fields in the HTML that are not shown to the user?**
    a. User authentication
    b. URL writing
    c. HTML Hidden field
    d. HTML invisible field

Answers

1. **b**
2. **d**
3. **c**
4. **c**

Questions

1. How does cookie work?
2. List the types of cookies.
3. Explain the code to create, delete and get cookies.
4. What is session tracking?
5. List and explain the different approaches that can be applied for session tracking.

Key terms

- **Cookies**: They are pieces of data that are stored at the client's browser to track information regarding the user's habits.
- **Session**: It means a particular interval of time.

- **Session ID**: It is a unique identification string, usually a long, random, and alpha-numeric string, that is transmitted between the client and the server. Session IDs are usually stored in the cookies, URLs (in case URL rewriting) and hidden fields of web pages.
- **Session Tracking**: It is a way to maintain the state (data) of a user. It is also known as session management in servlet.

Join our book's Discord space

Join the book's Discord Workspace for Latest updates, Offers, Tech happenings around the world, New Release and Sessions with the Authors:

https://discord.bpbonline.com

Chapter 7
Basics of Java Server Pages (JSP)

Introduction

The chapter covers the concept of writing Java code in HTML to generate dynamic pages. The core part of the chapter is to familiarize you with how Java code can be reduced in HTML without compromising the functionality expected from the code.

Structure

The following are the topics discussed in this chapter:

- Java Server Pages
- JSP life cycle
- Overview of JSP
- Basic scripting elements
- JSP directives
- Forwarding requests with jsp:forward
- taglib directive
- XML syntax for directives

Objectives

After studying this chapter, you should be able to understand the concept of Java server pages. You should also gain the knowledge of JSP tags and its applications and be able to perform basic web page design using JSP.

Java Server Pages

Java Server Pages (**JSP**) technology enables you to mix regular, static HTML with dynamically generated content. You simply write the regular HTML in the normal manner, using familiar webpage-building tools. In servlets, we write HTML inside Java, and in JSP, we write Java inside HTML. JSP pages are translated into servlets, the servlets are compiled, and it is the compiled servlets that execute at request time. So, writing JSP pages is just another way of writing servlets.

Servlets are convenient to write and efficient to execute. They can customize their behavior based on cookies, track user-specific data with the session-tracking API and talk to relational databases with JDBC. But Servlets are not good in writing a lot of HTML as one cannot use standard HTML tools, so it is difficult for non-Java developers to write the code.

JSP life cycle

JSP pages follow these phases:

- Translation of JSP page
- Compilation of JSP page
- Class loading (class file is loaded by the classloader)
- Instantiation (object of the generated servlet is created)
- Initialization (the **`jspInit()`** method is invoked by the container)
- Request processing (the **`_jspService()`** method is invoked by the container)
- Destroy (the **`jspDestroy()`** method is invoked by the container)

Following is the JSP life cycle:

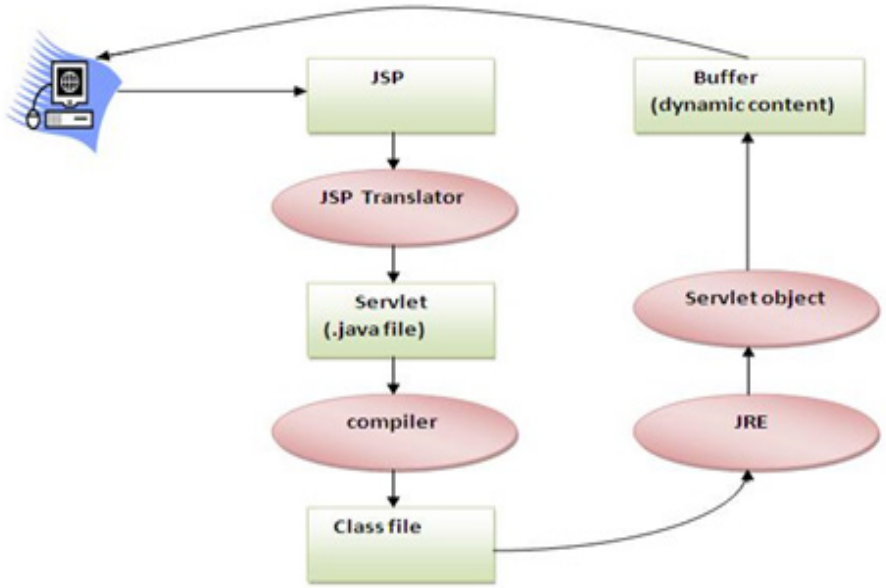


***Figure 7.1**: JSP life cycle*

The JSP API consists of two packages:

- **`javax.servlet.jsp`**
- **`javax.servlet.jsp.tagext`**

The **`javax.servlet.jsp`** package has two interfaces and multiple classes. The two interfaces are as follows:

`JspPage`
`HttpJspPage`

The classes are as follows:

`JspWriter`
`PageContext`
`JspFactory`
`JspEngineInfo`
`JspException`
`JspError`

The JSP page interface

According to the JSP specification, all the generated servlet classes must implement the **`JspPage`** interface. It extends the Servlet interface. It provides two life cycle methods:

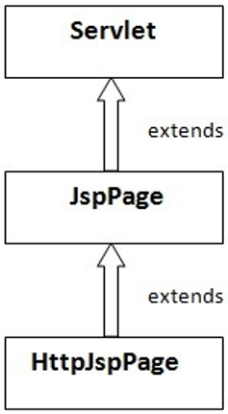


***Figure 7.2**: JspPage Interface*

Methods of JSP page interface:

- **`public void jspInit()`**: It is invoked only once during the life cycle of the JSP when JSP page is requested firstly. It is used to perform initialization and is the same as the **`init()`** method of Servlet interface.
- **`public void jspDestroy()`**: It is invoked only once during the life cycle of the JSP before the JSP page is destroyed. It can be used to perform some cleanup operation.

The HttpJspPage interface

The **`HttpJspPage`** interface provides the one life cycle method of JSP. It extends the **`JspPage`** interface.

Method of **`HttpJspPage`** interface is **`public void _jspService()`**, which is invoked each time a request for the JSP page comes to the container. It is used to process the request. The underscore **`_`** signifies that you cannot override this method.

Overview of JSP

JSP pages are translated into servlets. So, fundamentally, any task JSP pages can perform could also be accomplished by servlets. However, this underlying equivalence does not mean that servlets and JSP pages are equally appropriate in all

scenarios. The issue is not the power of technology, it is the convenience, productivity, and maintainability of one or the other.

Basic tokens of JSP

- **JSP Comment**

  Description: Developer comment that is not sent to the client

  Example: `<%-- This is a JSP comment --%>`

- **JSP Expression**

  Description: Expression that is evaluated and sent to the client each time the page is requested

  Example: `<%= Java Value %>`

- **JSP Scriptlet**

  Description: Statement or statements that are executed each time the page is requested

  Example: `<% Java Statement %>`

- **JSP Declaration**

  Description: Field or method that becomes part of class definition when the page is translated into a servlet

  Examples: `<%! Field Definition %><%! Method Definition %>`

- **JSP Directive**

  Description: High-level information about the structure of the servlet code (page), code that is included at page-translation time (include), or custom tag libraries used (taglib)

  Example: `<%@ directive att="val" %>`

- **JSP Expression Language Element**

  Description: Shorthand JSP expression

  Example: `${ EL Expression }`

- **Custom Tag (Custom Action)**

  Description: Invocation of custom tag

  Example: `<prefix:name> Body </prefix:name>`

- **Escaped Template Text**

  Description: Text that would otherwise be interpreted specially; slash is removed and the remaining text is sent to the client

  Examples: `<\%` `%\>`

Writing first JSP program

Create a program named **`first.jsp`** and write the following code:

```
<html>
 <head>
 <title> Hello World </title>
</head>
 <body>
 <h2>Hello  </h2>
 <% out.print("<p><b> Hello again </b></p>"); %>
 </body>
</html>

```

Figure 7.3: *First JSP program*

The equivalent servlet code for the preceding JSP would be as follows:

```
import java.io.*;
import javax.servlet.*;
import javax.servlet.http.*;

public class FirstServlet extends HttpServlet
{
 public void doGet(HttpServletRequest request, HttpServletResponse response)    throws ServletException, IOException
{
 response.setContentType("text/html");
 PrintWriter out = response.getWriter();

out.println("<HTML>\n" + "<HEAD><TITLE>Hello</TITLE></HEAD>\n" + "<BODY>"+"<H1>Hello</H1><P><B>Hello again</B></P>" +"</BODY></HTML>");
 }
}

```

Figure 7.4: *Equivalent servlet code*

As we can see, writing little HTML in Java is tedious; if we need to generate lots of dynamic output in HTML, then it becomes cumbersome to write the code in Java. So, JSP code comes in handy to write Java in HTML.

Basic scripting elements

In most cases, a large percentage of your JSP document consists of static text (usually HTML), known as template text. There are two minor exceptions to the *template text is passed straight through* rule:

- If you want to have **<%** or **%>** in the output, you need to put **<\%** or **%\>** in the template text.
- If you want a comment to appear in the JSP page but not in the resultant document, you must use **<%-- JSP Comment --%>**.

HTML comments of the **<!-- HTML Comment -->** form are passed through to the client normally.

There are several ways to generate dynamic content from JSP. Each of these approaches has the following:

  - a legitimate place
  - size
  - complexity

- These factors in a project are most important in deciding which approach is appropriate.

JSP scripting elements let you insert Java code into the servlet that will be generated from the JSP page:

- There are three forms:
  - Expressions of the **<%= Java Expression %>** form, which are evaluated and inserted into the servlet's output.
  - Scriptlets of the **<% Java Code %>** form, which are inserted into the servlet's **_jspService** method (called by **service**).
  - Declarations of the **<%! Field/Method Declaration %>** form, which are inserted into the body of the servlet class, outside any existing methods.

JSP expressions

A JSP expression is used to insert values directly into the output. It has the following form:

- **<%= Java Expression %>**: The expression is evaluated, converted to a string, and inserted in the page. This evaluation is performed at runtime (when the page is requested) and thus, has full access to information about the request.

  For example, the following shows the date/time that the page was requested:

  ```
  Current time: <%= new java.util.Date() %>
  ```

- To simplify these expressions, you can use several predefined variables (or *implicit objects*).
- There is nothing magical about these variables; the system simply tells you what names it will use for the local variables in **`_jspService`** (the method that replaces **`doGet`** in servlets that result from JSP pages).
- JSP changes the method name from **`doGet`** to **`_jspService`** and uses a **`Jspwriter`** instead of a **`PrintWriter`**.
- We can use lots of predefined implicit objects in the JSP expressions and scriptlets to perform various operations easily.

You are supplied with eight automatically defined local variables in **`_jspService`**, sometimes called implicit objects:

  - request
  - response
  - out
  - session
  - application
  - config
  - pageContext
  - page

- **request**: This variable is the **`HttpServletRequest`** associated with the request; it gives you access to the request parameters, the request type (for example, **`GET`** or **`POST`**), and the incoming HTTP headers (for example, cookies).
- **response**: This variable is the **`HttpServletResponse`** associated with the response to the client.
- **out**: This variable is the Writer used to send output to the client. The out variable is used almost exclusively in scriptlets since JSP expressions are automatically placed in the output stream and thus, rarely need to refer to out explicitly.
- **session**: This variable is the **`HttpSession`** object associated with the request.
- **application**: This variable is the **`ServletContext`** as obtained by **`getServletContext.Servlets`**, and JSP pages can store persistent data in the **`ServletContext`** object rather than in instance variables.
- **config**: This variable is the **`ServletConfig`** object for this page.

- **pageContext**: JSP introduced a class called **`PageContext`** to give a single point of access to many of the page attributes. The **`PageContext`** class has the **`getRequest`**, **`getResponse`**, **`getOut`**, **`getSession`**, methods and so forth. The **`pageContext`** variable stores the value of the **`PageContext`** object associated with the current page.
- **Page**: This variable is simply a synonym for this i.e. the current page and is not very useful. It was created as a placeholder for the time when the scripting language could be something other than Java.

  Example 1: JSP expression and the resultant servlet

  - Sample JSP Expression: Random Number

```
<H1>A Random Number</H1>
<%= Math.random() %>
```

```
01. <H1>A Random Number</H1>
02.    <%= Math.random() %>
03.
04.
```

Figure 7.5: *Random number*

  - Representative resulting servlet code: random number

```
01. public void _jspService(HttpServletRequest request, HttpServletResponse response) throws ServletException, IOException
02. {
03.    response.setContentType("text/html");
04.    HttpSession session = request.getSession();
05.    JspWriter out = response.getWriter();
06.    out.println("<H1>A Random Number</H1>");
07.    out.println(Math.random()); .
08.    }
```

Figure 7.6: *Representative Resulting Servlet*

Example 2: Using implicit objects in JSP expressions

```
01. <!DOCTYPE HTML PUBLIC "-//W3C//DTD HTML 4.0 Transitional//EN">
02.  <HTML><HEAD><TITLE>JSP Expressions</TITLE></HEAD>
03.  <BODY>
04.   <H2>JSP Expressions</H2>
05.   <UL><LI>Current time: <%= new java.util.Date() %>
06.    <LI>Server: <%= application.getServerInfo() %>
07. <LI>Session ID: <%= session.getId() %>
08.   </UL>
09. </BODY>
10. </HTML>
```

Figure 7.7: *Using implicit objects in JSP expressions*

Example 3: Create an HTML form to submit the input data, read this data using the servlet, and display the data

- Submit data through an HTML form

  Form to read three parameters from users and submit them to a JSP page

```
<HTML>
<HEAD><TITLE>Collecting Three Parameters</TITLE></HEAD>
<BODY>
 <H1>Input form fields</H1>
 <FORM ACTION="emp.jsp">
  Name :<INPUT TYPE="TEXT" NAME="name"><BR>
  Address :<INPUT TYPE="TEXT" NAME="address"><BR>
  Designation :<INPUT TYPE="TEXT" NAME="designation"><BR>
  <CENTER><INPUT TYPE="SUBMIT"></CENTER>
 </FORM>
</BODY>
</HTML>
```

Figure 7.8: *Data through HTML form*

- JSP page to read the request parameters submitted from the preceding form

```
<HTML>
 <HEAD><TITLE>Reading Three Request Parameters</TITLE></HEAD>
 <BODY><H1>Reading Three Request Parameters</H1>
 <UL>
  <LI><B>Name :</B>: <%= request.getParameter("name") %>
  <LI><B>Address</B>: <%= request.getParameter("address") %>
  <LI><B>Designation</B>: <%= request.getParameter("designation") %>
 </UL>
 </BODY>
 </HTML>
```

Figure 7.9: *Request parameters submitted*

There is an XML syntax to invoke every **`jsptoken`**:

XML syntax for expressions: XML authors can use the following alternative syntax for JSP expressions:

Syntax:

```
<jsp:expression>
    Java Expression
</jsp:expression>
```

Example :

```
<jsp:expression>
    request.getParameter(“designation”)
</jsp:expression>
```

JSP scriptlets

If you want to do something more complex than giving the value of a simple expression, JSP scriptlets let you insert arbitrary code into the servlet's **`_jspService`** method (which is called by service). Scriptlets have the following form:

```
<%      Java Code    %>
```

Scriptlets have access to the same automatically defined variables as expressions (request, response, session, out, etc.). So, for example, if you want to explicitly send output to the resultant page, you could use the out variable, as in the following example:

```
<% String queryData = request.getQueryString();

out.println("Attached GET data: " + queryData); %>
```

In this particular instance, you could have accomplished the same effect more easily by using a combination of a scriptlet and a JSP expression, as follows:

```
<% String queryData = request.getQueryString(); %> Attached GET data:
<%= queryData %>
```

Alternatively, you could have used a single JSP expression, as follows:

```
Attached GET data: <%= request.getQueryString() %>
```

In general, however, scriptlets can perform several tasks that cannot be accomplished with expressions alone. These tasks include setting response headers and status codes, invoking side effects like writing to the server log or updating a database, or executing code that contains loops, conditionals, or other complex constructs:

```
<% response.setContentType("application/msword"); %>
```

Example 1: Scriptlet to set a background color of a page based on user input

```
<!DOCTYPE HTML PUBLIC "-//W3C//DTD HTML 4.0 Transitional//EN">
 <HTML>
 <HEAD><TITLE>Color Testing</TITLE></HEAD>
  <% String bgColor = request.getParameter("bgColor");
   if ((bgColor == null) || (bgColor.trim().equals("")))
   { bgColor = "WHITE"; } %>
 <BODY BGCOLOR="<%= bgColor %>">
  <H2> Testing a Background of " <%= bgColor %>"</H2></BODY>
</HTML>
```

Figure 7.10: *Scriplets to set a backgrounf color*

Example 2: Using Scriptlets to make parts of the JSP page conditional

```
<!DOCTYPE HTML PUBLIC "-//W3C//DTD HTML 4.0 Transitional//EN">
<HTML>
<HEAD><TITLE>Wish for the Day</TITLE>
<LINK REL=STYLESHEET HREF="JSP-Styles.css" TYPE="text/css">
</HEAD>
<BODY>
 <% if (Math.random() < 0.5)
   { %>
   <H1>Have a <I>nice</I>day!</H1>
  <% } else { %>
   <H1>Have a <I>lousy</I>day!</H1>
 <% } %>
</BODY>
</HTML>
```

Figure 7.11*: Scriplets to make parts of JSP Page Conditional*

XML Syntax for Scriptlets: XML authors can use the following alternative syntax for JSP expressions.

Syntax:

```
<jsp:scriptlet>
    Java Code
</jsp:scriptlet>
```

Example:

```
<jsp:scriptlet>
    out.println(“Hello Good Morning”);
</jsp:scriptlet>
```

JSP declarations

JSP declaration lets you define methods or fields that get inserted into the main body of the servlet class (outside the **`_jspService`** method that is called by service to process the request).

A declaration has the following form:

```
<%! Field or Method Definition %>
```

Since declarations do not generate output, they are normally used in conjunction with JSP expressions or scriptlets. In principle, JSP declarations can contain field (instance variable) definitions, method definitions, inner class definitions, or even static initializer blocks: anything that is legal to put inside a class definition but outside any existing methods.

Example 1: To declare a method and used it in an expression

```
01. <H1> Some Heading </H1>
02. <%!
03.   private String randomNumber()
04.   {
05.     return("<H2>" + Math.random() + "</H2>");
06.   }
07.  %>
08.   <%= randomNumber() %>
```

Figure 7.12: *Declaring a method and using it in an expression*

Example 2: To declare a field and access it using expression

```
01. <HTML>
02.  <HEAD><TITLE>JSP Declarations</TITLE></HEAD>
03.  <BODY>
04.   <H1>JSP Declarations</H1>
05.   <%! private intaccessCount = 0; %>
06.   <H2>Accesses to page since server reboot: <%= ++accessCount %>
07.   </H2>
08.  </BODY>
09. </HTML>
```

Figure 7.13*: Declaring a field and accessing it using expression*

XML syntax for declarations: XML authors can use the following alternative syntax for JSP expressions:

Syntax:

```
<jsp:declaration>
    Field or Method
</jsp:declaration>
```

Example :

```
<jsp:declaration>
    int counter=0;
</jsp:declaration>
```

Comparing JSP expressions, Scriptlets, and declarations

Example 1: Generate five random numbers in the range of 1 to 100.

In the first example, the goal is to output a bulleted list of five random integers from 1 to 100. Since the structure of this page is fixed and we use a separate helper class for the **`randomNumber`** method, JSP expressions are all that is needed.

Example 1.1: Program to write the **`randomNumber`** method in the **`NumberGenerator.java`** file

```
01. package example;
02. public class NumberGenerator
03. {
04.  public static intrandomNumber(intn)
05.   {
06.   return(1 + ((int)(Math.random() * n)));
07.   }
08.
09. }
```

Figure 7.14: *Program to write randomNumber in NumberGenerator.java*

Example 1.2: Program to call the **`randomNumber`** method in the **`JspExpressions.jsp`** file

```
01. <HTML><HEAD><TITLE>Random Numbers</TITLE></HEAD>
02. <BODY><H1>Random Numbers</H1>
03.   <UL>
04.    <LI><%= example.NumberGenerator.randomNumber(100) %>
05.    <LI><%= example.NumberGenerator.randomNumber(100) %>
06.    <LI><%= example.NumberGenerator.randomNumber(100) %>
07.    <LI><%= example.NumberGenerator.randomNumber(100) %>
08.    <LI><%= example.NumberGenerator.randomNumber(100) %>
09.   </UL>
10. </BODY>
11. </HTML>
```

Figure 7.15: *Program to call method randomNumber in the file JspExpressions.jsp*

Example 2: Generate dynamic n random numbers in the range of 1 to 100

In the second example, the goal is to generate a list between 1 and 100 (numbers selected at random, each of which is a number between 1 and 100).

Since the number of entries in the list is dynamic, a JSP scriptlet is needed. But should there be a single scriptlet containing a loop that outputs the numbers, or we should use the dangling-brace approach. The choice is not clear here: the first approach yields a more concise result, but the second approach exposes the **`<LI>`** element to

the web developer, who might want to modify the type of bullet or insert additional formatting elements.

Example 2.1: Program to call the **`randomNumber`** method in the **`JspScriptlet.jsp`** file using only scriptlet.

```
<HTML>
 <HEAD><TITLE>Random Numbers</TITLE></HEAD>
 <BODY>
 <H1>Random Numbers</H1>
  <UL><%
   int n = example.NumberGenerator.randomNumber(100);
           for(inti=0; i<n; i++)
    {
    out.println("<LI>" + example.NumberGenerator.randomNumber(100));
    }
   %>
  </UL>
 </BODY>
</HTML>
```

Figure 7.16: *randomNumber in the file JspScriptlet.jsp using only scriptlet*

Example 2.2: Program to call the **`randomNumber`** method in the **`JspScriptlet.jsp`** file using scriptlet and expressions

```
<HTML>
<HEAD><TITLE>Random Numbers</TITLE></HEAD>
<BODY>
  <H1> Random Numbers</H1>
  <UL>
   <% int n = example.NumberGenerator.randomNumber(100);
   for(inti=0; i<n; i++)  {
    %>
   <LI>
    <%= example.NumberGenerator.randomNumber(100)%>
   <% } %>
  </UL>
</BODY>
</HTML>
```

Figure 7.17: *randomNumber in the JspScriptlet.jsp file using scriptlet and expressions*

Example 3: Generate semi static random numbers in the range of 1 to 100

In this example, the requirement is to generate a random number on the first request, and then show the same number to all users until the server is restarted. Instance variables (fields) are the natural way to accomplish this persistence. The reason is that instance variables are initialized only when the object is built, and servlets are built once and remain in memory between requests: a new instance is not allocated for each request. JSP expressions and scriptlets deal only with code inside the **`_jspService`** method, so they are not appropriate here. A JSP declaration is needed instead.

Example 3.1: Program to call the **`randomNumber`** method in the **`JspDeclaration.jsp`** file using declaration.

```
01. <HTML>
02. <HEAD><TITLE>Semi-Random Number</TITLE></HEAD>
03. <BODY>
04.   <%!
05.     private int n = example.NumberGenerator.randomNumber(100);
06.    %>
07.   <H1>Semi-Random Number:<BR>
08.   <%= n %>
09.   </H1>
10. </BODY>
11. </HTML>
```

Figure 7.18: *randomNumber in the file JspDeclaration.jsp using declaration*

JSP directives

A JSP directive affects the overall structure of the servlet that results from the JSP page.

Syntax: **`<%@ directive attribute="value" %>`**

In JSP, there are three main types of directives:

- **`page`**
  - The page directive lets you control the structure of the servlet by importing classes, customizing the servlet superclass, setting the content type, and the like.
  - A page directive can be placed anywhere within the document.
- **`include`**
  - The include directive lets you insert a file into the JSP page when the JSP file is translated into a servlet.
- **`taglib`**
  - The taglib directive defines custom markup tags.

Page directive

The page directive lets you define one or more of the following case-sensitive attributes:

- import
- contentType

- pageEncoding
- Session
- isELIgnored (JSP 2.0 only)
- Buffer
- autoFlush
- info
- errorPage
- isErrorPage
- isThreadSafe
- language
- extends

The import attributes

- It lets you specify the packages that should be imported by the servlet into which the JSP page gets translated.
- This allows you to call Java code indirectly and develop separate utility classes.
- Insert into JSP page only the Java code needed to invoke the utility classes.
- These utilities must use packages, and the JSP page should use the import attribute.
- When you use utility classes, remember that they should always be in packages.
- Packages are a good strategy on any large project because they help protect against name conflicts.
- With JSP, however, packages are absolutely required. The reason is that, in the absence of packages, classes you reference are assumed to be in the same package as the current class.
- By default, the servlet imports the following:

```
java.lang.*
javax.servlet.*
javax.servlet.jsp.*
javax.servlet.http.*
```

And possibly a few server-specific entries:

- Never write JSP code that relies on any server-specific classes being imported automatically; doing so makes your code nonportable.
- Use of the import attribute takes one of the following two forms:

```
<%@ page import="package.class" %>
<%@ page import="package.class1,...,package.classN" %>
```

- For example, the following directive signifies that all classes in **`java.utilpackage`** should be available to use without explicit package identifiers.

```
<%@ page import="java.util.*" %>
```

- The import attribute is the only page attribute that is allowed to appear multiple times within the same document.
- Although page directives can appear anywhere within the document, it is traditional to place import statements either near the top of the document or just before the first place that the referenced package is used.

The contentType and pageEncoding attributes

The **`contentType`** attribute sets the **`Content-Type`** response header, indicating the MIME type of the document being sent to the client.

Type	Meaning
`application/msword`	Microsoft Word document
`application/pdf`	Acrobat (.pdf) file
`application/vnd.ms-excel`	Excel spreadsheet
`application/vnd.ms-powerpoint`	PowerPoint presentation
`application/x-java-archive`	JAR file
`application/x-java-serialized-object`	Serialized Java object
`application/x-java-vm`	Java bytecode (.class) file
`application/zip`	ZIP archive
`audio/x-wav`	Microsoft Windows sound file
`image/gif`	GIF image
`text/css`	HTML cascading style sheet
`text/html`	HTML document
`text/plain`	Plaintext
`text/xml`	XML
`video/mpeg`	MPEG video clip

Table 7.1: *MIME types*

Use of the **contentTypeattribute** takes one of the following two forms:

```
<%@ page contentType="MIME-Type" %>
<%@ page contentType="MIME-Type; charset=Character-Set" %>
```

For example, the directive:

```
<%@ page contentType="application/vnd.ms-excel" %> has the same basic
effect as the scriptlet
<% response.setContentType("application/vnd.ms-excel"); %>
```

The first difference between the two forms is that **response.setContentType** uses explicit Java code (an approach some developers try to avoid), whereas the page directive uses only JSP syntax.

The second difference is that directives are parsed specially; they don't directly become **_jspService** code at the location they appear. This means **response.setContentType** can be invoked conditionally, whereas the page directive cannot be.

Unlike regular servlets, for which the default MIME type is text/plain, the default for JSP pages is **text/html** (with a default character set of ISO-8859-1).

If you want to change both the content type and the character set, you can do the following:

```
<%@ page contentType="someMimeType; charset=someCharacterSet" %>
```

However, if you only want to change the character set, it is simpler to use the **pageEncoding** attribute. For example, Japanese JSP pages might use the following:

```
<%@ page pageEncoding="Shift_JIS" %>
```

Example 1: In this example, we create a HTML page to invoke a JSP page that carries the course credit details and open it in the Excel file. Here, we are using page directive with the **contentType** attribute to generate the Excel file.

1. HTML code for index page

```
01. <!DOCTYPE html>
02.  <html>
03.      <head>
04.          <title>Page directive with contentType attribute </title>
05.      </head>
06.      <body>
07.       <h2>
08.             <a href="CourseCredit.jsp">Open the file in excel</a>
09.        </h2>
10.      </body>
11.  </html>
```

Figure 7.19: *HTML code for index page*

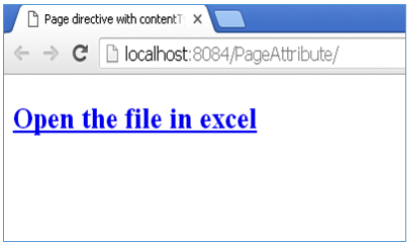


***Figure 7.20**: Index page to provide link for JSP page*

2. The **`CourseCredit.jsp`** file to generate the data in Excel

```
<!--
JSP page to record the course code, course name and
credits assigned to the course.
-->

<%-- Set the content type header with the JSP directive --%>
<%@ page contentType="application/vnd.ms-excel" %>

<%-- Set the content disposition header --%>
<% response.setHeader("Content-Disposition", "attachment; filename=\"mult-table.xls\""); %>

<%-- Enter tab to separate the  columns. --%>
<table>
    <tr>
        <th>Course Code</th>
        <th>Course Name</th>
        <th>Course Credit</th>
    </tr>
    <tr>
        <td>MS101</td>
        <td>JAVA</td>
        <td>6</td>
    </tr>
    <tr>
        <td>MS102</td>
        <td>PHP</td>
        <td>6</td>
    </tr>
     <tr>
        <td>MS103</td>
        <td>.NET </td>
        <td>6</td>
    </tr>
</table>
```

***Figure 7.21**: CourseCredit.JSP file to generate the data in Excel*

The preceding code will set the content type to be Excel, so instead of opening the content in the browser, it will display it in Excel.

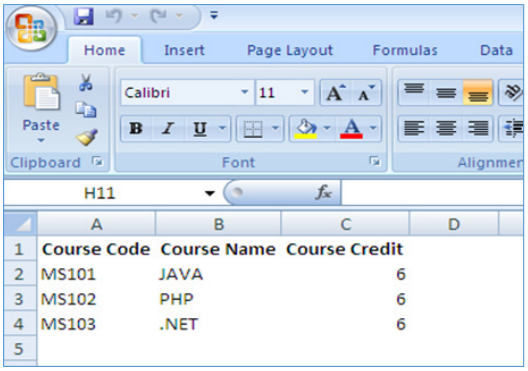


***Figure 7.22**: JSP page when executed directly, downloads the file in Excel and opens it in Excel*

3. Resultant servlet for the JSP page

```
01. package org.apache.jsp;
02.
03. import javax.servlet.*;
04. import javax.servlet.http.*;
05. import javax.servlet.jsp.*;
06.
07. public final class CourseCredit_jsp extends org.apache.jasper.runtime.HttpJspBase
08.     implements org.apache.jasper.runtime.JspSourceDependent {
09.
10.   private static final JspFactory _jspxFactory = JspFactory.getDefaultFactory();
11.
12.   private static java.util.List<String> _jspx_dependants;
13.
14.   private org.glassfish.jsp.api.ResourceInjector _jspx_resourceInjector;
15.
16.   public java.util.List<String> getDependants() {
17.     return _jspx_dependants;
18.   }
19.
20.   public void _jspService(HttpServletRequest request, HttpServletResponse response)
21.         throws java.io.IOException, ServletException {
22.
23.     PageContext pageContext = null;
24.     HttpSession session = null;
25.     ServletContext application = null;
26.     ServletConfig config = null;
27.     JspWriter out = null;
28.     Object page = this;
29.     JspWriter _jspx_out = null;
30.     PageContext _jspx_page_context = null;
31.
32.     try {
33.       response.setContentType("application/vnd.ms-excel");
34.       response.setHeader("X-Powered-By", "JSP/2.2");
35.       pageContext = _jspxFactory.getPageContext(this, request, response,
36.          null, true, 8192, true);
37.       _jspx_page_context = pageContext;
38.       application = pageContext.getServletContext();
39.       config = pageContext.getServletConfig();
40.       session = pageContext.getSession();
```

***Figure 7.23:** Resultant servlet for the JSP page*

This is the resultant servlet for the JSP code we mentioned earlier:

```
41.  out = pageContext.getOut();
42.  _ jspx_out= out;
43.  _jspx_resourceInjector =
     (org.glassfish.jsp.api.ResourceInjector) application.getAttribute("com.sun.appserv.jsp.resource.injector");
44.  out.write("<!--\n");
45.  out.write("JSP page to record the course code, course name and \n");
46.  out.write('credits assigned to the course.\n");
47.  out.write("-->\n");
48.  out.write("\n");
49.  out.write("\n");
50.  out.write("\n");
51.  out.write("
52.  out.write('n');
53.  response.setHeader("Content-Disposition", "attachment; filename=\"mult-table.xls"");
54.  out.write('\n');
55.  out.write('\n');
56.  out.write('\n');
57.  out.write("<table>\n");
58.  out.write("     <tr>\n");
59.  out.write("   <th>Course Code</th>\n");
60.  out.write("   cth Course Name</th>\n");
61.  out.write("   <th>Course Credit</th>\n");
62.  out.write("     </tr>\n");
63.  out.write("  <tr>\n");
64.  out.write("   <td><MS101c</td>\n");
65.  out.write("   <td>JAVA</td>\n");
66.  out.write("   <td>6c</td>\n");
67.  out.write("  </tr>\n");
68.  out.write("  <tr>\n");
69.  out.write("   <td><MS102c</td>\n");
70.  out.write("   <td>PHP</td>\n");
71.  out.write("   <td>6c</td>\n");
72.  out.write("  </tr>\n");
73.  out.write("  <tr>\n");
74.
75.  out.write("   <td><MS103c</td>\n");
76.  out.write("   <td>.NET</td>\n");
77.  out.write("   <td>6c</td>\n");
78.  out.write("  </tr>\t\n");
79.  out.write("  <tr>\n");
80.  out.write("</table>");
81.  } catch (Throwable t) {
82.  if (!(t instanceof SkipPageException)){
```

***Figure 7.24**: Resultant servlet for the JSP code*

This is the resultant servlet for the JSP code we mentioned earlier:

```
82.         out = _jspx_out;
83.         if (out != null && out.getBufferSize() != 0)
84.           out.clearBuffer();
85.         if (_jspx_page_context != null) _jspx_page_context.handlePageException(t);
86.         else throw new ServletException(t);
87.       }
88.     } finally {
89.       _jspxFactory.releasePageContext(_jspx_page_context);
90.     }
91.   }
92. }
93.
```

***Figure 7.25:** Resultant servlet for the JSP code*

Note that the content type of file is mandatorily set to Excel in the service method.

Example 2: In this example, we create an HTML page to invoke a JSP page that carries the course credit details and opens it in either Excel or browser, based on user choice. Here, we are using the **`response.setContentType`** method to conditionally generate the output either in an Excel file or the browser.

- HTML code for index page

```
01. <!DOCTYPE html>
02. <html>
03.     <head>
04.         <title>Page directive with contentType attribute </title>
05.     </head>
06.     <body>
07.
08.         <h2>
09.             <a href="CourseCredit.jsp">Open the file in excel</a> <br>
10.             <a href="CourseCredit_1.jsp">Open the file in excel or in browser</a>
11.         </h2>
12.     </body>
13. </html>
14.
```

Figure 7.26: *HTML code for index page*

The code will result in the following output:

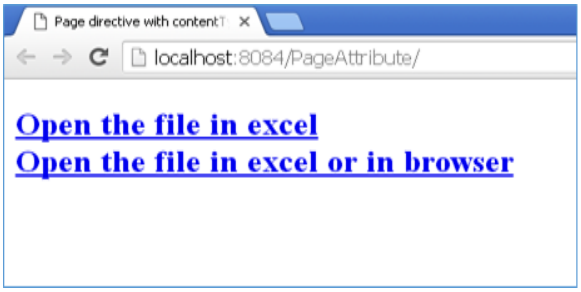


Figure 7.27: *Index page to show link for the new JSP page*

- JSP code to read a request parameter, namely, *view*, and open the file in Excel if view is set to Excel, or in the browser

```
<% String view = request.getParameter("view");
   if ((view != null) && (view.equals("excel")))
                    { response.setContentType("application/vnd.ms-excel"); }
%>
<table>
    <tr>
        <th>Course Code</th>
        <th>Course Name</th>
        <th>Course Credit</th>
    </tr>
    <tr>
        <td>MS201</td>
        <td>ADVANCED JAVA</td>
        <td>6</td>
    </tr>
    <tr>
        <td>MS202</td>
        <td>YII FRAMEWORK</td>
        <td>6</td>
    </tr>
     <tr>
        <td>MS203</td>
        <td>WINDOWS COMMUNICATION FOUNDATION </td>
        <td>6</td>
    </tr>
</table>
```

***Figure 7.28**: JSP code to read a request parameter namely "view"*

The preceding code will generate the following output:

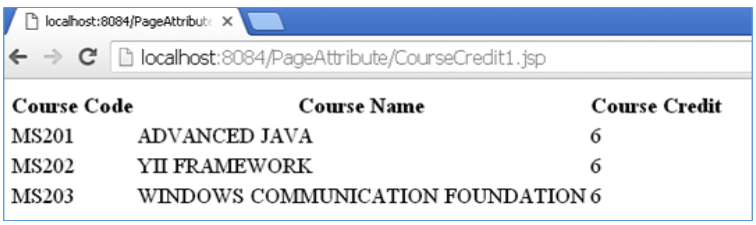


***Figure 7.29:** Content displayed in browser when view is not set to Excel*

- Modified URL in the HTML code to set the view to Excel in the index page

```
<!DOCTYPE html>
<html>
    <head>
        <title>Page directive with contentType attribute </title>
    </head>
    <body>

        <h2>
            <a href="CourseCredit.jsp">Open the file in excel</a> <br>
            <a href="CourseCredit_1.jsp?view=excel">Open the file in excel or in browser</a>
        </h2>
    </body>
</html>

```

Figure 7.30: *Set the view to Excel in the index page*

The following output will be generated by the preceding code:

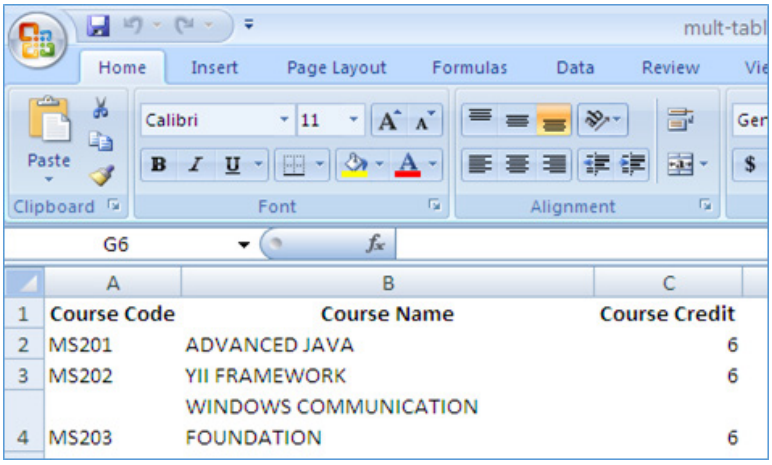


Figure 7.31: *Content displayed in Excel when view is set to Excel*

- Resultant servlet generated from the JSP page

```
01. package org.apache.jsp;
02. import javax.servlet.*;
03. import javax.servlet.http.*;
04. import javax.servlet.jsp.*;
05.
06. public final class CourseCredit1_jsp extends org.apache.jasper.runtime.HttpJspBase
07.     implements org.apache.jasper.runtime.JspSourceDependent {
08.
09.   private static final JspFactory _jspxFactory = JspFactory.getDefaultFactory();
10.   private static java.util.List<String> _jspx_dependants;
11.   private org.glassfish.jsp.api.ResourceInjector _jspx_resourceInjector;
12.   public java.util.List<String> getDependants() {
13.     return _jspx_dependants;
14.   }
15.
16.   public void _jspService(HttpServletRequest request, HttpServletResponse response)
17.         throws java.io.IOException, ServletException {
18.
19.     PageContext pageContext = null;
20.     HttpSession session = null;
21.     ServletContext application = null;
22.     ServletConfig config = null;
23.     JspWriter out = null;
24.     Object page = this;
25.     JspWriter _jspx_out = null;
26.     PageContext _jspx_page_context = null;
27.
28.     try {
29.       response.setContentType("text/html");
30.       response.setHeader("X-Powered-By", "JSP/2.2");
31.       pageContext = _jspxFactory.getPageContext(this, request, response,null, true, 8192, true);
32.       _jspx_page_context = pageContext;
33.       application = pageContext.getServletContext();
34.       config = pageContext.getServletConfig();
35.       session = pageContext.getSession();
36.       out = pageContext.getOut();
37.       _jspx_out = out;
38.       _jspx_resourceInjector = (org.glassfish.jsp.api.ResourceInjector) application.getAttribute("com.sun.appserv.jsp
```

Figure 7.32: *Servlet generated from the JSP page*

This is the resultant servlet for the JSP code we mentioned earlier:

```
41.  String view = request.getParameter("view");
42.    if ((view != null) && (view.equals("excel")))
43.    { response.setContentType("application/vnd.ms-excel"); }
44.
45.
46.        out.write("<table>\n");
47.        out.write("    <tr> \n");
48.        out.write("        <th>Course Code</th>\n");
49.        out.write("        <th>Course Name</th>\n");
50.        out.write("        <th>Course Credit</th>\n");
51.        out.write("    </tr>\n");
52.        out.write("    <tr>\n");
53.        out.write("        <td>MS201</td>\n");
54.        out.write("        <td>ADVANCED JAVA</td>\n");
55.        out.write("        <td>6</td>\n");
56.        out.write("    </tr>\n");
57.        out.write("    <tr>\n");
58.        out.write("        <td>MS202</td>\n");
59.        out.write("        <td>YII FRAMEWORK</td>\n");
60.        out.write("        <td>6</td>\n");
61.        out.write("    </tr>\n");
62.        out.write("     <tr>\n");
63.        out.write("        <td>MS203</td>\n");
64.        out.write(" <td>WINDOWS COMMUNICATION FOUNDATION </td>\n");
65.        out.write("        <td>6</td>\n");
66.        out.write("    </tr>\t\n");
67.        out.write("</table>");
68.      } catch (Throwable t) {
69.        if (!(t instanceof SkipPageException)){
70.          out = _jspx_out;
71.          if (out != null && out.getBufferSize() != 0)
72.            out.clearBuffer();
73.          if (_jspx_page_context != null) _jspx_page_context.handlePageException(t);
74.          else throw new ServletException(t);
75.        }
76.      } finally {
77.        _jspxFactory.releasePageContext(_jspx_page_context);
78.      }
79.    }
80.  }
```

Figure 7.33: *Resultant servlet for the JSP code*

To generate the servlet in NetBeans, you can select the JSP page and right-click on it; then, select view servlet; note that the servlet will be generated only if the JSP page has been successfully compiled and executed:

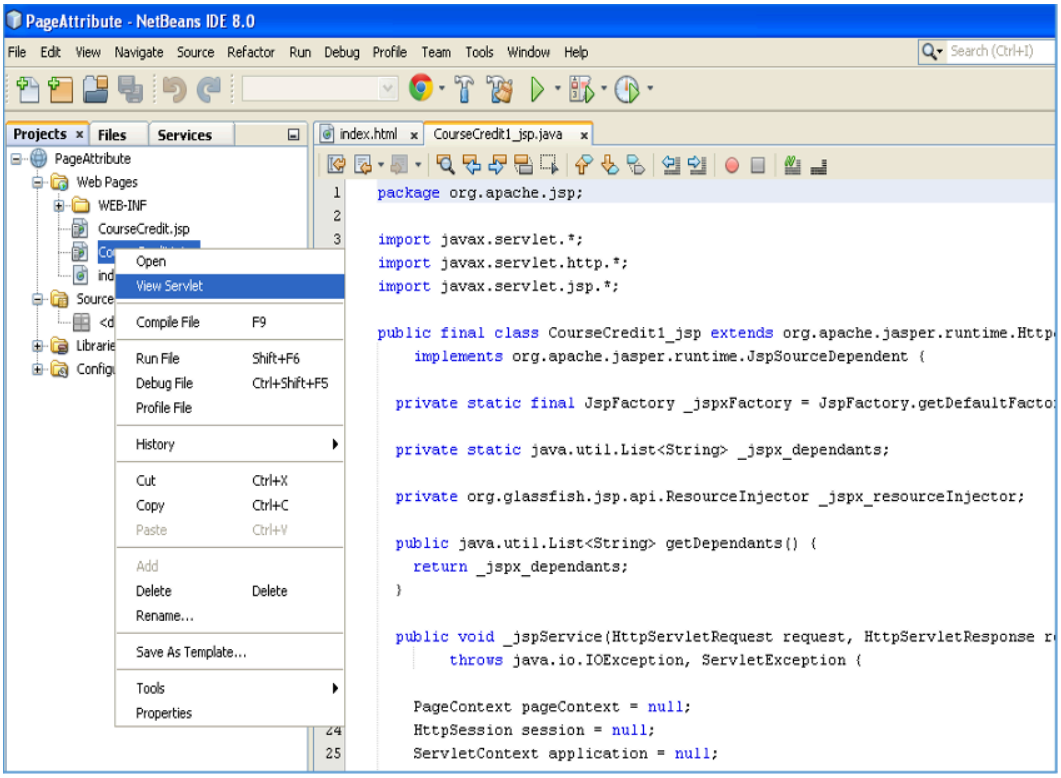


Figure 7.34: *Snapshot to generate servlet from a JSP page in NetBeans*

When you want to generate dynamic content using JSP as a view in a format other than HTML, you can use both page directive with the **`contentType`** attribute and the **`setContentType`** response method. However, when you know the content type in advance, you should use page directive with the **`contentType`** attribute, and when the format of output, that is, the content type, is based on user choice or some situation, it is preferable to use the **`response.setContentType`** method.

The session attributes

The session attribute controls whether the page participates in HTTP sessions. Use of this attribute takes one of the following two forms:

```
<%@ page session="true" %>       <%-- Default --%>
<%@ page session="false" %>
```

A value of true (the default) signifies that the predefined variable session (of type **`HttpSession`**) should be bound to the existing session if one exists; otherwise, a new session should be created and bound to session.

A value of false means that no sessions will be automatically created and that attempting to access the variable session will result in errors when the JSP page is translated into a servlet.

The isELIgnored attribute

The **`isELIgnored`** attribute controls whether the JSP 2.0 **expression language** (**EL**) is ignored (true) or evaluated normally (false). This attribute is new in JSP 2.0; it is illegal to use it in a server that supports only JSP 1.2 or earlier. Use of this attribute takes one of the following two forms.

```
<%@ page isELIgnored="false" %>
<%@ page isELIgnored="true" %>
```

Expressions in the JSP EL take the **`${expression}`** form.

The buffer and autoFlush attributes

The buffer attribute specifies the size of the buffer used by the out variable, which is of type **`JspWriter`**. The use of this attribute takes one of two forms:

```
<%@ page buffer="sizeinkb" %>

<%@ page buffer="none" %>
```

Servers can use a larger buffer than you specify, but not a smaller one. The default buffer size is server-specific but must be at least 8 kilobytes.

The info attributes

The info attribute defines a string that can be retrieved from the servlet using the **`getServletInfo`** method. Its use takes the following form:

```
<%@ page info="Message" %>
```

The errorPage and isErrorPage attributes

The **`isErrorPage`** attribute indicates whether the current page can act as the error page for another JSP page. The use of **`isErrorPage`** takes one of the following two forms:

```
<%@ page isErrorPage="true" %>
<%@ page isErrorPage="false" %>  <%-- Default --%>
```

Example 1: To understand the usage of attributes, we take an example of student marks entry form that inputs the marks, and another page displays the marks with grade; the problem occurs when there is no client-side validation to input marks in numbers; and when the server-side code converts the text to number, it generates exception in Java, so we handle the exception using the **`errorPage`** and **`isErrorPage`** attributes.

- HTML code of index page

```
<!DOCTYPE html>
<html>
    <head>
        <title>Page directive with isErrorPage and errorPage attribute </title>
    </head>
    <body>
        <h2>
            <a href="MarksInput.jsp">Input Students Marks</a> <br>
        </h2>
    </body>
</html>
```

Figure 7.35: *HTML code of index page*

The following output will be generated by the preceding code:

Figure 7.36*: Index page to provide link to MarksInput.jsp*

- Student marks input form **`MarksInput.jsp`**

```
<h2>Student Marks Input Form </h2>
<form action="DisplayMarks.jsp">
  <fieldset>
    <legend> Student Marks out of 100</legend>
       <table>
        <tr>
  <td>Student ID : </td>
  <td><input type="text" name="studentid"/> <br/></td>
       </tr>
        <tr>
  <td>Student Name : </td>
  <td><input type="text" name="studentname"/><br/></td>
       </tr>
       <tr>
  <td> Java Marks  :</td>
  <td><input type="text" name="marks1"/><br/></td>
      </tr>
       <tr>
  <td>PHP Marks  : </td>
            <td><input type="text" name="marks2"/><br/></td>
      </tr>
      <tr>
  <td>.Net Marks  :</td>
   <td><input type="text" name="marks3"/><br/></td>
     </tr>
      <tr>
  <td><input type="submit" value="Calculate Grade"/></td>
     </tr>
        </table>
  </fieldset>
  </form>
```

Figure 3.37: *Marks input form MarksInput.jsp*

The following output will be generated by the preceding code:

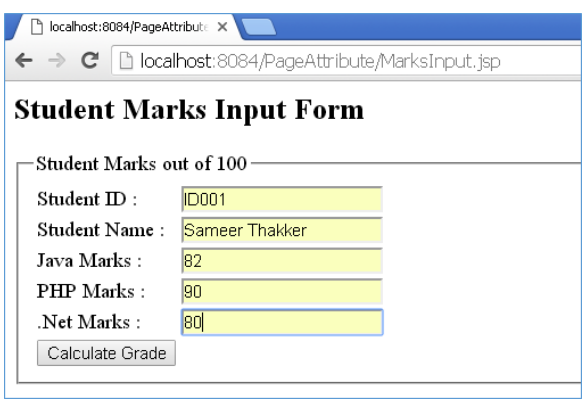


Figure 7.38: *MarksInput.jsp page to input student marks*

- Display student marks **`DisplayMarks.jsp`**

```
<h2>Student Marks Input Form </h2>
<form>
    <table>
        <tr>
  <td>Student ID : </td>
  <td><%=request.getParameter("studentid")%> <br/></td>
      </tr>
      <tr>
  <td>Student Name : </td>
  <td><%=request.getParameter("studentname")%> <br/></td>
      </tr>
    <%
     Double m1=Double.parseDouble(request.getParameter("marks1"));
   Double m2=Double.parseDouble(request.getParameter("marks2"));
   Double m3=Double.parseDouble(request.getParameter("marks3"));
   Double per=(m1+m2+m3)/3;
   String grade="";
    if (m1<=35 || m2<=35 || m3<=35)
          grade="FF";
      else if(per>=80)
          grade="AA";
      else if(per>=70)
          grade="BB";
      else if(per>=60)
          grade="CC";
      else
          grade="DD";
    %>
    <tr>
  <td>Java Marks  : </td><td><%=m1%><br/></td>
  </tr>
  <tr>
  <td>PHP Marks  : </td><td><%=m2%><br/></td>
  </tr>
    <tr>
  <td>.Net Marks  : </td><td><%=m3%><br/></td>
  </tr>
    <tr>
  <td>Grade  : </td><td><%=grade%><br/></td>
  </tr>
</form>
```

Figure 7.39: *Display Student marks DisplayMarks.jsp*

The following output will be generated by the preceding code:

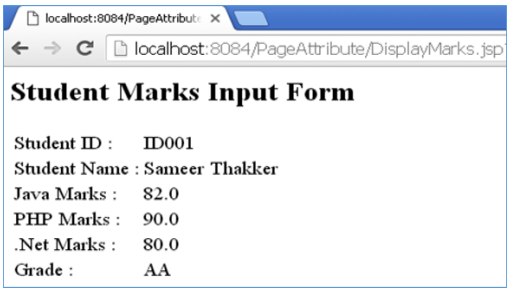


Figure 7.40: *DisplayMarks.jsp page to display student marks and grade*

In this code, we have not written the code to handle the exception, so if we enter one of the course marks other than a number, it will throw the exceptions shown in the following figure:

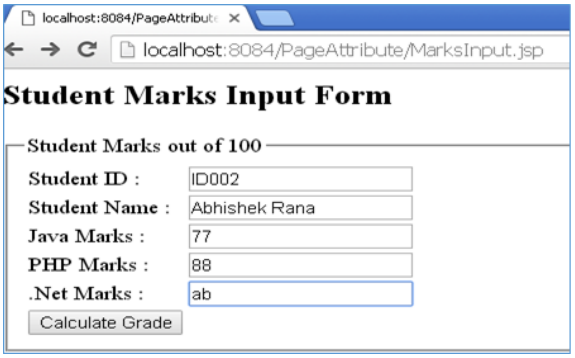


***Figure 7.41**: MarksInput.jsp page to input student marks as text in one of the marks field*

The following figure shows the NumberFormat Exception generated by parseDouble method:

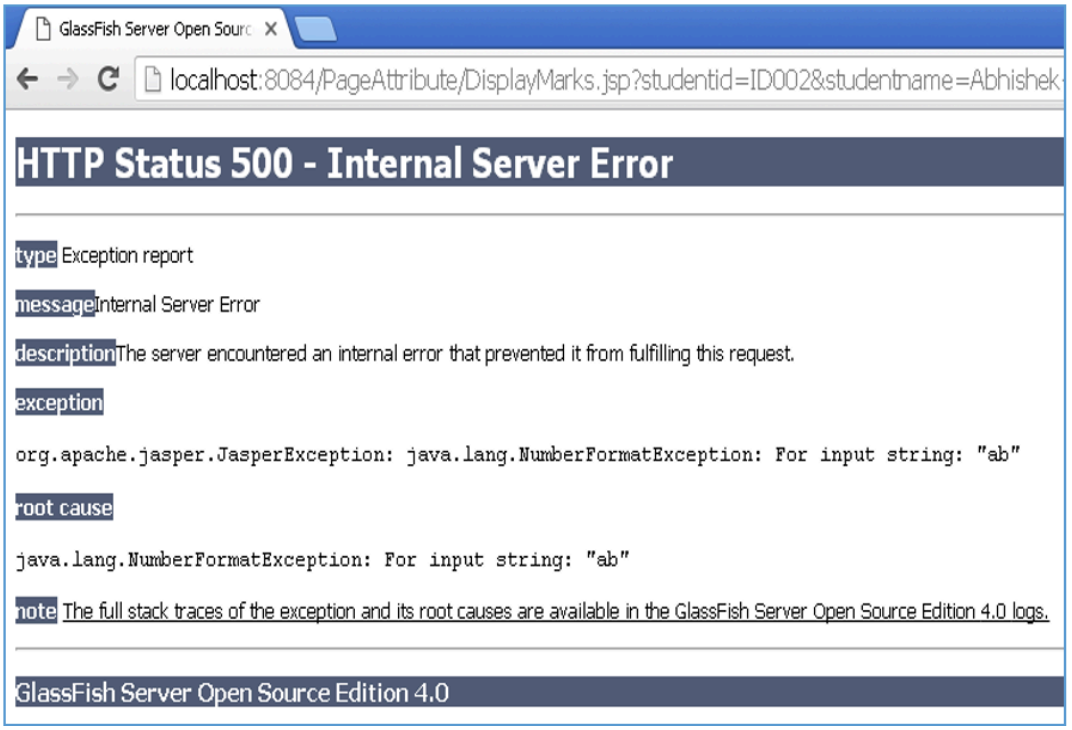


***Figure 7.42:** NumberFormat Exception generated by parseDouble method while converting text to double in the DisplayMark.jsp page*

We can handle the exception in the code; we can use the attribute **errorPage** to indicate that this page may generate an exception and create another JSP page to handle the exception, in which we set the **isErrorPage** attribute to true to catch and handle the exceptions using the implicit object exception.

- Updated **DisplayMarks.jsp** page to include the **errorPage** attribute

```
<%@ page errorPage="HandleExceptions.jsp" %>
<h2>Student Marks Input Form </h2>
<form>
    <table>
        <tr><td>Student ID : </td><td><%=request.getParameter("studentid")%> <br/></td></tr>
    <tr><td>Student Name : </td><td><%=request.getParameter("studentname")%>
    <br/>
    <%
    Double m1=Double.parseDouble(request.getParameter("marks1"));
    Double m2=Double.parseDouble(request.getParameter("marks2"));
    Double m3=Double.parseDouble(request.getParameter("marks3"));
    Double per=(m1+m2+m3)/3;
    String grade="";
    if (m1<=35 || m2<=35 || m3<=35)
        grade="FF";
    else if(per>=80)
        grade="AA";
    else if(per>=70)
        grade="BB";
    else if(per>=60)
        grade="CC";
    else
        grade="DD";
    %>
   <tr><td>Java Marks  : </td><td><%=m1%><br/></td></tr>
   <tr><td>PHP Marks  : </td><td><%=m2%><br/></td></tr>
   <tr><td>.Net Marks  : </td><td><%=m3%><br/></td></tr>
   <tr><td>Grade  : </td><td><%=grade%><br/></td></tr>
</form>
```

Figure 7.43: *DisplayMarks.jsp page to include the errorPage attribute*

- Page to Handle exceptions if occurred **HandleExceptions.jsp**

```
<%@ page isErrorPage="true" %>
<h2>Page to Handle Exceptions </h2>
<span style="color:red"><%= exception %></span>
<h5>Exception Stack Trace </h5>
<%@ page import="java.io.*" %>
<% exception.printStackTrace(new PrintWriter(out)); %>
```

Figure 7.44: *Page to Handle exceptions if occurred HandleExceptions.jsp*

After applying these changes in the **`DisplayMarks.jsp`** file and creating a new **`HandleExceptions.jsp`** file to handle the exception, the following output will be generated:

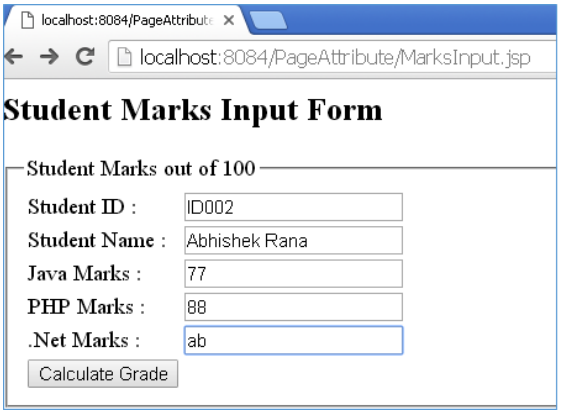


***Figure 7.45**: MarksInput.jsp page to input student marks as text in one of the marks field*

Exception is handled by the **`HandleException.jsp`** page and displays the name and stack trace of exception occurred:

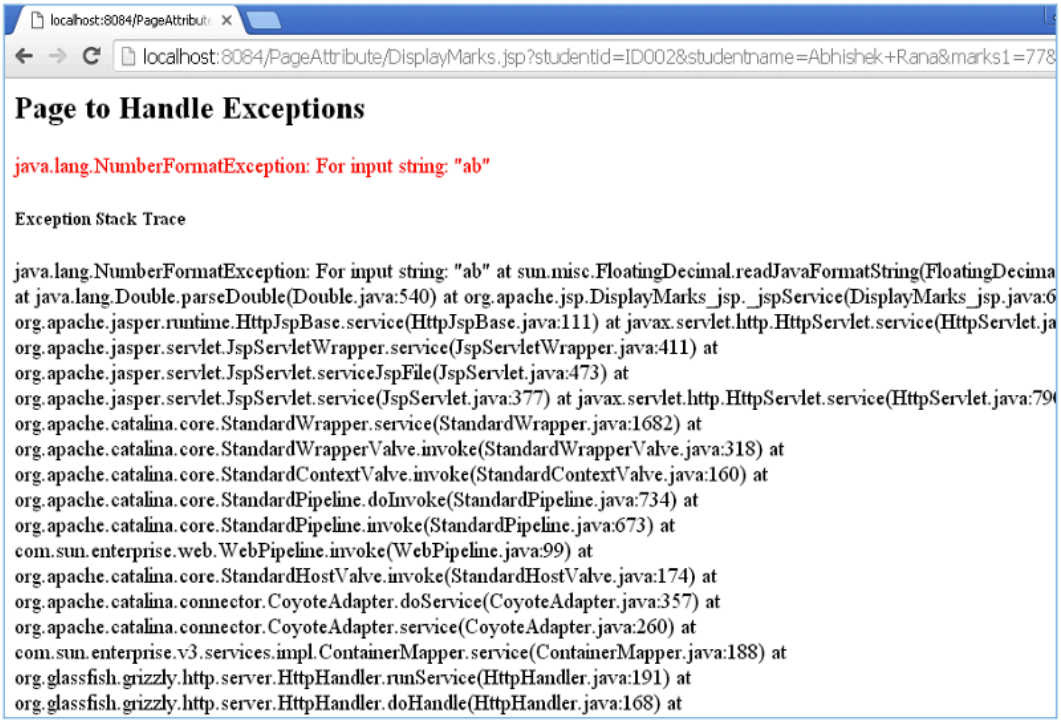


***Figure 7.46:** Exception is handled by the HandleException.jsp page and displays the name and stack trace of exception occurred*

It is always a good practice to handle the exceptions in the Java code, but these attributes can be used for debugging purposes during the development and testing phase, where several pages can generate the exception and one designated page catches and displays the exceptions.

The **`isThreadSafe`** attribute

The **`isThreadSafe`** attribute controls whether the servlet that results from the JSP page will allow concurrent access (the default) or will guarantee that no servlet instance processes more than one request at a time (**`isThreadSafe="false"`**).

Use of the isThreadSafe attribute takes one of the following two forms:

```
<%@ page isThreadSafe="true" %>  <%-- Default --%>
<%@ page isThreadSafe="false" %>
```

Do not use **`isThreadSafe`**. Use explicit synchronization instead.

Example 1: We have a counter that is incremented on every user request, so a thread is created to handle each request. We want to make sure only one thread at a time is incrementing and accessing the **`counter`** variable.

a. JSP code to increment a counter by multiple threads without applying mutual exclusion:

```
01. <%! private int count = 0; %>
02.   <%
03.    out.println("Counter  is : " +count);
04.    count = count + 1;
05.   %>
```

Figure 7.47:*JSP code to increment a counter by multiple threads*

b. JSP code to increment a counter by multiple threads applying mutual exclusion:

```
01. <%! private int count = 0; %>
02.   <%
03.  synchronized(this)
04.  {
05.     out.println("Counter  is : " +count);
06.     count = count + 1;
07.    }
08.  %>
```

Figure 7.48: *JSP code to increment a counter by multiple threads applying mutual exclusion*

Using the synchronized method, only one thread at a time will execute the code written within the method, so mutual exclusion is applied to deal with the problems faced due to concurrency.

The extends attribute

The extends attribute designates the superclass of the servlet that will be generated for the JSP page. It takes the following form:

```
<%@ page extends="package.class" %>
```

This attribute is normally reserved for developers or vendors that implement fundamental changes to the way in which pages operate (for example, to add in personalization features).

The language attributes

The language attribute is intended to specify the scripting language being used, as follows:

```
<%@ page language="c" %>
```

We don't use this attribute since Java is both the default and the only legal choice.

Include files

JSP has two main capabilities for including external files into a JSP document:

- The jsp:include action
- The include directive

The jsp:include action

The **`jsp:include`** action lets you include the output of a page at request time. Its main advantage is that it saves you from changing the main page when the included pages change. Another advantage is that since it includes the output of the secondary page, not the secondary page's actual code as with the include directive, the included pages cannot use any JSP constructs that affect the main page as a whole.

`jsp:include` lets you insert any of the following into the JSP output:

- The content of an HTML page
- The content of a plain text document
- The output of JSP page
- The output of a servlet

The **jsp:include** action includes the output of a secondary page when the main page is requested. Although the output of the included pages cannot contain JSP, the pages can be the result of resources that use servlets or JSP to create the output. So, the URL that refers to the included resource is interpreted in the normal manner by the server and thus, can be a servlet or JSP page. The server runs the included page in the usual way and places the output into the main page.

The page attributes

You designate the included page with the page attribute, as follows. This attribute is required; it should be a relative URL referencing the resource whose output should be included.

```
<jsp:include page="relative-path-to-resource" />
```

Relative URLs that do not start with a slash are interpreted relative to the location of the main page. Relative URLs that start with a slash are interpreted relative to the base web application directory, not relative to the server root.

The flush attribute

In addition to the required **page** attribute, **jsp:include** has a second attribute: flush, as shown below. This attribute is optional. It specifies whether the output stream of the main page should flush before the inclusion of the page (the default is false).

```
<jsp:include page="relative-path-to-resource" flush="true" />
<jsp:param> element
```

The included page uses the same request object as the originally requested page. As a result, the included page normally sees the same request parameters as the main page. If, however, you want to add to or replace those parameters, you can use the **jsp:param** element (which has name and value attributes) to do so. For example, consider the following snippet:

```
<jsp:include page="secondary.jsp">
            <jsp:param name="coursename" value="java" />
</jsp:include>
```

Now, suppose that main page is invoked by means of **http://localhost:8084/IncludeAction/mainpage.jsp?programname=mca**.

In such a case, the following list summarizes the results of various **getParameter** calls:

- In main page (**mainpage.jsp**):
  `request.getParameter("programename") returns "mca".`
  `request.getParameter("coursename") returns null.`

- In included page (**`secondary.jsp`**):

  `request.getParameter("programename ") returns "mca".`

  `request.getParameter("coursename ") returns "java".`

XML syntax and jsp:include

The **`jsp:include`** action is one of the first JSP constructs we have seen that has only XML syntax; when using XML syntax, we need to note three things about XML:

- XML element names can contain colons. The element name is not **`jsp:include`**; in fact, the XML-compatible version of all standard JSP elements starts with the JSP prefix (or namespace).
- XML tags are case-sensitive. Be sure to use **`jsp:include`** in all lower case.
- XML tags must be explicitly closed. In XML, all elements are container elements, and end tags are never optional.

Example 1: To understand the use of the **`jsp:include`** action, we are creating three HTML pages and one JSP page, and adding all these pages in a single JSP page to form the content of the page.

1. HTML page for index page

```
01. <!DOCTYPE html>
02. <html>
03.     <head>
04.         <title>Include action demonstration</title>
05.         <meta charset="UTF-8">
06.         <meta name="viewport" content="width=device-width, initial-scale=1.0">
07.     </head>
08.     <body>
09.     <h2>
10.             <a href="mainpage.jsp">
11.   Include Action to include four intermediate pages
12.         </a> <br>
13.     </h2>
14.     </body>
15. </html>
```

Figure 7.49: *HTML page for index page*

The following output will be generated by the preceding code:

Figure 7.50: *Index page to click the JSP page*

2. HTML page for first intermediate page, **`page1.html`**

```
<!DOCTYPE html>
<html>
    <head>
        <title>Page 1</title>
        <meta charset="UTF-8">
        <meta name="viewport" content="width=device-width, initial-scale=1.0">
    </head>
    <body>
        <h3> Welcome to the world of java. </h3>
    </body>
</html>
```

Figure 7.51: *HTML page for first intermediate page*

3. HTML page for second intermediate page, **`page2.html`**

```
<!DOCTYPE html>
<html>
    <head>
        <title>Page 2</title>
        <meta charset="UTF-8">
        <meta name="viewport" content="width=device-width, initial-scale=1.0">
    </head>
    <body>
        <h3> Welcome to the world of php. </h3>
    </body>
</html>
```

Figure 7.52: *HTML page for second intermediate page*

4. HTML page for third intermediate page, **`page3.html`**

```
<!DOCTYPE html>
<html>
    <head>
        <title>Page 3</title>
        <meta charset="UTF-8">
        <meta name="viewport" content="width=device-width, initial-scale=1.0">
    </head>
    <body>
        <h3> Welcome to the world of .net . </h3>
    </body>
</html>
```

Figure 7.53*: HTML page for third intermediate page*

5. JSP page for fourth intermediate page, **`page4.jsp`**

```
<%@page contentType="text/html" pageEncoding="UTF-8"%>
<!DOCTYPE html>
<html>
    <head>
        <meta http-equiv="Content-Type" content="text/html; charset=UTF-8">
        <title>JSP Page</title>
    </head>
    <body>
        <h3> Welcome to the world of JSP</h3>
    </body>
</html>
```

Figure 7.54: *JSP page for fourth intermediate page*

6. Main page to include all the intermediately pages **`mainpage.jsp`**

```
<%@page contentType="text/html" pageEncoding="UTF-8"%>
<!DOCTYPE html>
<html>
    <head>
        <meta http-equiv="Content-Type" content="text/html; charset=UTF-8">
        <title>JSP Page</title>
    </head>
    <body>
        <h1>Programmers World</h1>

        <h2> Java Programming </h2>
        <jsp:include page="page1.html"/>
        <hr/>

        <h2> Php Programming </h2>
        <jsp:include page="page2.html"/>
        <hr/>

        <h2> .Net Programming </h2>
        <jsp:include page="page3.html"/>
        <hr/>

        <h2> Dynamic page Programming </h2>
        <jsp:include page=" page4.jsp"/>
        <hr/>
    </body>
</html>
```

Figure 7.55: *Main page to include all the intermediately pages mainpage.jsp*

The following output will be generated by the preceding code:

Figure 7.56: *Main page with content of four intermediate JSP pages*

7. Resultant servlet for the **`mainpage.jsp`**:

```
01. import org.apache.jsp;
02. import javax.servlet.*;
03. import javax.servlet.http.*;
04. import javax.servlet.jsp.*;
05.
06. public final class mainpage_jsp extends org.apache.jasper.runtime.HttpJspBase
07.     implements org.apache.jasper.runtime.JspSourceDependent {
08.
09.
10.   private static final JspFactory _jspxFactory = JspFactory.getDefaultFactory();
11.   private static java.util.List<String> _jspx_dependants;
12.   private org.glassfish.jsp.api.ResourceInjector _jspx_resourceInjector;
13.   public java.util.List<String> getDependants() {
14.     return _jspx_dependants;
15.   }
16.
17.   public void _jspService(HttpServletRequest request, HttpServletResponse response)
18.         throws java.io.IOException, ServletException {
19.
20.     PageContext pageContext = null;
21.     HttpSession session = null;
22.     ServletContext application = null;
23.     ServletConfig config = null;
24.     JspWriter out = null;
25.     Object page = this;
26.     JspWriter _jspx_out = null;
27.     PageContext _jspx_page_context = null;
28.
29.     try {
30.       response.setContentType("text/html;charset=UTF-8");
31.       response.setHeader("X-Powered-By", "JSP/2.2");
32.       pageContext = _jspxFactory.getPageContext(this, request, response,
33.          null, true, 8192, true);
34.       _jspx_page_context = pageContext;
35.       application = pageContext.getServletContext();
36.       config = pageContext.getServletConfig();
37.       session = pageContext.getSession();
38.       out = pageContext.getOut();
39.       _jspx_out = out;
40.       _jspx_resourceInjector = (org.glassfish.jsp.api.ResourceInjector) application.getAttribute("com.sun.appserv.jsp
41.
42.       out.write("\n");
43.       out.write("<!DOCTYPE html>\n");
```

Figure 7.57: *Resultant servlet for the mainpage.jsp*

This is the resultant servlet for the JSP code we mentioned earlier:

```
01.       out.write("<!DOCTYPE html>\n");
02.       out.write("<html>\n");
03.       out.write("    <head>\n");
04.       out.write("        <meta http-equiv=\"Content-Type\" content=\"text/html; charset=UTF-8\">\n");
05.       out.write("        <title>JSP Page</title>\n");
06.       out.write("    </head>\n");
07.       out.write("    <body>\n");
08.       out.write("        <h1>Programmers World</h1>\n");
09.       out.write("        \n");
10.       out.write("        <h2> Java Programming </h2>\n");
11.       out.write("        ");
12.       org.apache.jasper.runtime.JspRuntimeLibrary.include(request, response, "page1.html", out, false);
13.       out.write("\n");
14.       out.write("        <hr/>\n");
15.       out.write("        \n");
16.       out.write("        <h2> Php Programming </h2>\n");
17.       out.write("        ");
18.       org.apache.jasper.runtime.JspRuntimeLibrary.include(request, response, "page2.html", out, false);
19.       out.write("\n");
20.       out.write("        <hr/>\n");
21.       out.write("        \n");
22.       out.write("        <h2> .Net Programming </h2>\n");
23.       out.write("        ");
24.       org.apache.jasper.runtime.JspRuntimeLibrary.include(request, response, "page3.html", out, false);
25.       out.write("\n");
26.       out.write("        <hr/>\n");
27.       out.write("        \n");
28.       out.write("        <h2> Dynamic page Programming </h2>\n");
29.       out.write("        ");
30.       org.apache.jasper.runtime.JspRuntimeLibrary.include(request, response, "page4.jsp", out, false);
31.       out.write("\n");
32.       out.write("        <hr/>\n");
33.       out.write("    </body>\n");
34.       out.write("</html>\n");
35.     } catch (Throwable t) {
36.       if (!(t instanceof SkipPageException)){
37.         out = _jspx_out;
38.         if (out != null && out.getBufferSize() != 0)
39.           out.clearBuffer();
40.         if (_jspx_page_context != null) _jspx_page_context.handlePageException(t);
41.         else throw new ServletException(t);
```

Figure 7.58: *Servlet for the JSP code*

This is the resultant servlet for the JSP code we mentioned earlier.

8. Resultant servlet for intermediate page (**page4.jsp**)

```
01. package org.apache.jsp;
02. import javax.servlet.*;
03. import javax.servlet.http.*;
04. import javax.servlet.jsp.*;
05.
06. public final class page4_jsp extends org.apache.jasper.runtime.HttpJspBase
07.     implements org.apache.jasper.runtime.JspSourceDependent {
08.
09.   private static final JspFactory _jspxFactory = JspFactory.getDefaultFactory();
10.   private static java.util.List<String> _jspx_dependants;
11.   private org.glassfish.jsp.api.ResourceInjector _jspx_resourceInjector;
12.   public java.util.List<String> getDependants() {
13.     return _jspx_dependants;
14.   }
15.
16.   public void _jspService(HttpServletRequest request, HttpServletResponse response)
17.         throws java.io.IOException, ServletException {
18.
19.     PageContext pageContext = null;
20.     HttpSession session = null;
21.     ServletContext application = null;
22.     ServletConfig config = null;
23.     JspWriter out = null;
24.     Object page = this;
25.     JspWriter _jspx_out = null;
26.     PageContext _jspx_page_context = null;
27.
28.     try {
29.       response.setContentType("text/html;charset=UTF-8");
30.       response.setHeader("X-Powered-By", "JSP/2.2");
31.       pageContext = _jspxFactory.getPageContext(this, request, response,
32.          null, true, 8192, true);
33.       _jspx_page_context = pageContext;
34.       application = pageContext.getServletContext();
35.       config = pageContext.getServletConfig();
36.       session = pageContext.getSession();
37.       out = pageContext.getOut();
38.       _jspx_out = out;
39.       _jspx_resourceInjector = (org.glassfish.jsp.api.ResourceInjector) application.getAttribute("com.sun.appserv.jsp
40.
41.       out.write("\n");
42.       out.write("\n");
43.       out.write("<!DOCTYPE html>\n");
```

Figure 7.59: *Servlet for the intermediate page page4.jsp*

This is the resultant servlet for the JSP code we mentioned earlier.

```
41.       out.write("<!DOCTYPE html>\n");
42.       out.write("<html>\n");
43.       out.write("    <head>\n");
44.       out.write("        <meta http-equiv=\"Content-Type\" content=\"text/html; charset=UTF-8\">\n");
45.       out.write("        <title>JSP Page</title>\n");
46.       out.write("    </head>\n");
47.       out.write("    <body>\n");
48.       out.write("        <h3> Welcome to the world of JSP</h3>\n");
49.       out.write("    </body>\n");
50.       out.write("</html>\n");
51.     } catch (Throwable t) {
52.       if (!(t instanceof SkipPageException)){
53.         out = _jspx_out;
54.         if (out != null && out.getBufferSize() != 0)
55.           out.clearBuffer();
56.         if (_jspx_page_context != null) _jspx_page_context.handlePageException(t);
57.         else throw new ServletException(t);
58.       }
59.     } finally {
60.       _jspxFactory.releasePageContext(_jspx_page_context);
61.     }
62.   }
63. }
64.
```

Figure 7.60: *Resultant servlet for the JSP code*

This is the resultant servlet for the JSP code we mentioned earlier.

As shown in the servlet, the intermediate file output is taken at request time and the content is built for the main page. Here, two servlets are generated: one for **`mainpage.jsp`** and another for **`page4.jsp`**. In the **`jsp:include`** action, one servlet will be generated for the **`main.jsp`** page, and if the intermediate file carries a JSP page, then another servlet will be generated for all the JSP intermediate files.

The include directive

This construct lets you insert JSP code into the main page before that main page is translated into a servlet. Its main advantage is that it is powerful: the included code can contain JSP constructs like field definitions and content-type settings that affect the main page as a whole. Its main disadvantage is that it is hard to maintain; you have to update the main page whenever any of the included pages change. We can include files at page translation time with the include directive. For file inclusion, use **`jsp:include`** whenever possible. Reserve the **`include`** directive (**`<%@ include ... %>`**) for cases in which the included file defines fields or methods that the main

page uses or when the included file sets response headers of the main page. The following are a few maintenance problems with the **`include`** directive:

- With the include directive (**`<%@ include ... %>`**), if the included file changes, all the JSP files that use it may need to be updated.
- Servers are required to detect when a JSP page changes and translate it into a new servlet before handling the next request.

The file attributes

The file attribute is used to indicate the file whose content is to be taken and added in the main file; the syntax is as follows:

```
<%@include file="file name" %>
```

The file content is added during page translation.

XML syntax for the include directive

The XML-compatible equivalent of the following:

```
<%@ include file="..." %> is
<jsp:directive.include file="..." />
```

When this form is used, both the main page and the included file must use XML-compatible syntax throughout.

Example 1: To understand the use of include directive, we are creating three HTML pages and one JSP page, adding all these pages in a single JSP page to form the content of the page.

1. HTML page for index page

```
01. <!DOCTYPE html>
02. <html>
03.     <head>
04.         <title>Include action demonstration</title>
05.         <meta charset="UTF-8">
06.         <meta name="viewport" content="width=device-width, initial-scale=1.0">
07.     </head>
08.     <body>
09.     <h2>
10.             <a href="mainpage.jsp">Include Directive to include four intermediate  pages</a> <br>
11.     </h2>
12.     </body>
13. </html>
```

Figure 7.61: *HTML page for index page*

The following output will be generated by the preceding code:

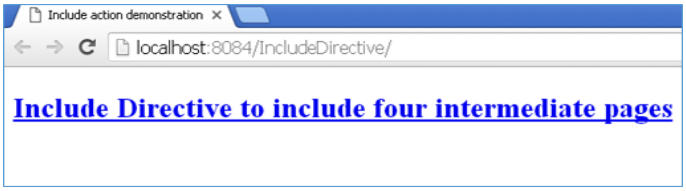


Figure 7.62: *Index page to click the main JSP page*

2. HTML page for first intermediate page (**`page1.html`**)

```
<!DOCTYPE html>

<html>
    <head>
        <title>Page 2</title>
        <meta charset="UTF-8">
        <meta name="viewport" content="width=device-width, initial-scale=1.0">
    </head>
    <body>
        <h3> Welcome to the world of Frontend designing. </h3>
    </body>
</html>
```

Figure 7.63: *HTML page for first intermediate page*

3. HTML page for second intermediate page (**`page2.html`**)

```
<!DOCTYPE html>

<html>
    <head>
        <title>Page 2</title>
        <meta charset="UTF-8">
        <meta name="viewport" content="width=device-width, initial-scale=1.0">
    </head>
    <body>
        <h3> Welcome to the world of Frontend designing. </h3>
    </body>
</html>
```

Figure 7.64: *HTML page for second intermediate page*

4. HTML page for third intermediate page (**page3.html**)

```
<!DOCTYPE html>

<html>
    <head>
        <title>Page 2</title>
        <meta charset="UTF-8">
        <meta name="viewport" content="width=device-width, initial-scale=1.0">
    </head>
    <body>
        <h3> Welcome to the world of Frontend designing. </h3>
    </body>
</html>
```

Figure 7.65: *HTML page for third intermediate page*

5. HTML page for fourth intermediate page (**page4.html**)

```
<%@page contentType="text/html" pageEncoding="UTF-8"%>
<!DOCTYPE html>
<html>
    <head>
        <meta http-equiv="Content-Type" content="text/html; charset=UTF-8">
        <title>JSP Page</title>
    </head>
    <body>
        <h3> Welcome to the world of android.</h3>
    </body>
</html>
```

Figure 7.66*: HTML page for fourth intermediate page*

6. Main page to include all the intermediary files **mainpage.jsp**

```
<%@page contentType="text/html" pageEncoding="UTF-8"%>
<!DOCTYPE html>
<html>
    <head>
        <meta http-equiv="Content-Type" content="text/html; charset=UTF-8">
        <title>JSP Page</title>
    </head>
    <body>
        <h1>Software Development World</h1>

        <h2> Testing Zone </h2>
        <%@include file="page1.html" %>
        <hr/>

        <h2> Designing Zone </h2>
        <%@include file="page2.html" %>
        <hr/>

        <h2> Backend Zone </h2>
        <%@include file="page3.html" %>
        <hr/>

        <h2> Mobile Application Zone </h2>
        <%@include file="page4.jsp" %>
        <hr/>
    </body>
</html>
```

Figure 7.67: *Main page to include all the intermediary files*

The following output will be generated by the preceding code:

Figure 7.68: *Main page to include the content of four intermediate pages*

Resultant servlet for the Main page, **`mainpage.jsp`**

```
01. package org.apache.jsp;
02. import javax.servlet.*;
03. import javax.servlet.http.*;
04. import javax.servlet.jsp.*;
05.
06. public final class mainpage_jsp extends org.apache.jasper.runtime.HttpJspBase
07.     implements org.apache.jasper.runtime.JspSourceDependent {
08.
09.   private static final JspFactory _jspxFactory = JspFactory.getDefaultFactory();
10.   private static java.util.List<String> _jspx_dependants;
11.   static {
12.     _jspx_dependants = new java.util.ArrayList<String>(4);
13.     _jspx_dependants.add("/page1.html");
14.     _jspx_dependants.add("/page2.html");
15.     _jspx_dependants.add("/page3.html");
16.     _jspx_dependants.add("/page4.jsp");
17.    }
18.
19.   private org.glassfish.jsp.api.ResourceInjector _jspx_resourceInjector;
20.
21.   public java.util.List<String> getDependants() {
22.     return _jspx_dependants;
23.   }
24.
25.   public void _jspService(HttpServletRequest request, HttpServletResponse response)
26.         throws java.io.IOException, ServletException {
27.
28.     PageContext pageContext = null;
29.     HttpSession session = null;
30.     ServletContext application = null;
31.     ServletConfig config = null;
32.     JspWriter out = null;
33.     Object page = this;
34.     JspWriter _jspx_out = null;
35.     PageContext _jspx_page_context = null;
36.
37.     try {
38.       response.setContentType("text/html;charset=UTF-8");
39.       response.setHeader("X-Powered-By", "JSP/2.2");
40.       pageContext = _jspxFactory.getPageContext(this, request, response,
41.          null, true, 8192, true);
42.       _jspx_page_context = pageContext;
```

Figure 7.69: *Servlet for the Main page*

This is the resultant servlet for the JSP code we mentioned earlier:

```
41.     _jspx_page_context = pageContext;
42.     application = pageContext.getServletContext();
43.     config = pageContext.getServletConfig();
44.     session = pageContext.getSession();
45.     out = pageContext.getOut();
46.     _jspx_out = out;
47.     _jspx_resourceInjector = (org.glassfish.jsp.api.ResourceInjector) application.getAttribute("com.sun.appserv.jsp.resource.injector");
48.
49.     out.write("\n");
50.     out.write("<!DOCTYPE html>\n");
51.     out.write("<html>\n");
52.     out.write("    <head>\n");
53.     out.write("        <meta http-equiv=\"Content-Type\" content=\"text/html; charset=UTF-8\">\n");
54.     out.write("        <title>JSP Page</title>\n");
55.     out.write("    </head>\n");
56.     out.write("    <body>\n");
57.     out.write("        <h1>Software Development World</h1>\n");
58.     out.write("        \n");
59.     out.write("        <h2> Testing Zone </h2>\n");
60.     out.write("        ");
61.     out.write("<!DOCTYPE html>\r\n");
62.     out.write("\r\n");
63.     out.write("<html>\r\n");
64.     out.write("    <head>\r\n");
65.     out.write("        <title>Page 1</title>\r\n");
66.     out.write("        <meta charset=\"UTF-8\">\r\n");
67.     out.write("        <meta name=\"viewport\" content=\"width=device-width,       initial-scale=1.0\">\r\n");
68.     out.write("    </head>\r\n");
69.     out.write("    <body>\r\n");
70.     out.write("        <h3> Welcome to the world of testing. </h3>\r\n");
71.     out.write("    </body>\r\n");
72.     out.write("</html>\r\n");
73.     out.write("\n");
74.     out.write("        <hr/>\n");
75.     out.write("        \n");
76.     out.write("        <h2> Designing Zone </h2>\n");
77.     out.write("        ");
78.     out.write("<!DOCTYPE html>\r\n");
79.     out.write("\r\n");
80.     out.write("<html>\r\n");
81.     out.write("    <head>\r\n");
```

***Figure 7.70**: Servlet for the JSP code*

This is the resultant servlet for the JSP code we mentioned earlier:

```
82.         out.write("        <title>Page 2</title>\r\n");
83.         out.write("        <meta charset=\"UTF-8\">\r\n");
84.         out.write("        <meta name=\"viewport\" content=\"width=device-width, initial-scale=1.0\">\r\n");
85.         out.write("    </head>\r\n");
86.         out.write("    <body>\r\n");
87.         out.write("        <h3> Welcome to the world of Frontend designing. </h3>\r\n");
88.         out.write("    </body>\r\n");
89.         out.write("</html>\r\n");
90.         out.write("\n");
91.         out.write("        <hr/>\n");
92.         out.write("        \n");
93.         out.write("        <h2> Backend Zone </h2>\n");
94.         out.write("        ");
95.         out.write("<!DOCTYPE html>\r\n");
96.         out.write("\r\n");
97.         out.write("<html>\r\n");
98.         out.write("    <head>\r\n");
99.         out.write("        <title>Page 3</title>\r\n");
100.        out.write("        <meta charset=\"UTF-8\">\r\n");
101.        out.write("        <meta name=\"viewport\" content=\"width=device-width, initial-scale=1.0\">\r\n");
102.        out.write("    </head>\r\n");
103.        out.write("    <body>\r\n");
104.        out.write("        <h3> Welcome to the world of query optimization . </h3>\r\n");
105.        out.write("    </body>\r\n");
106.        out.write("</html>\r\n");
107.        out.write("\n");
108.        out.write("        <hr/>\n");
109.        out.write("        \n");
110.        out.write("        <h2> Mobile Application Zone </h2>\n");
111.        out.write("        ");
112.        out.write("\n");
113.        out.write("\n");
114.        out.write("<!DOCTYPE html>\n");
115.        out.write("<html>\n");
116.        out.write("    <head>\n");
117.        out.write("        <meta http-equiv=\"Content-Type\" content=\"text/html; charset=UTF-8\">\n");
118.        out.write("        <title>JSP Page</title>\n");
119.        out.write("    </head>\n");
120.        out.write("    <body>\n");
121.        out.write("        <h3> Welcome to the world of android.</h3>\n");
```

***Figure 7.71**: Servlet for the JSP code*

This is the resultant servlet for the JSP code we mentioned earlier:

```
122.       out.write("    </body>\n");
123.       out.write("</html>\n");
124.       out.write("\n");
125.       out.write("        <hr/>\n");
126.       out.write("    </body>\n");
127.       out.write("</html>\n");
128.     } catch (Throwable t) {
129.       if (!(t instanceof SkipPageException)){
130.         out = _jspx_out;
131.         if (out != null && out.getBufferSize() != 0)
132.           out.clearBuffer();
133.         if (_jspx_page_context != null) _jspx_page_context.handlePageException(t);
134.         else throw new ServletException(t);
135.       }
136.     } finally {
137.       _jspxFactory.releasePageContext(_jspx_page_context);
138.     }
139.   }
140. }
```

***Figure 7.72**: Servlet for the JSP code*

This is the resultant servlet for the JSP code we mentioned earlier. As you can see in the generated servlet, the code of the intermediate files is placed in the main servlet. Here, the intermediate servlet for **`page4.jsp`** is not created, as the page is not executed individually:

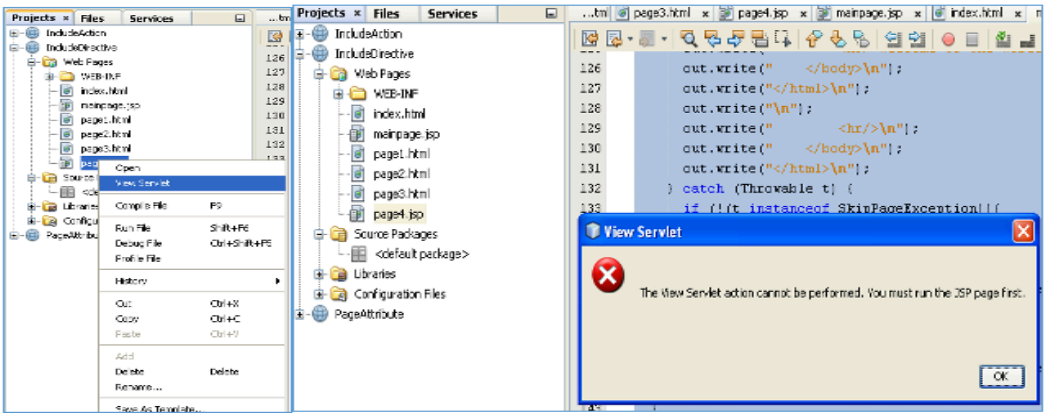


***Figure 7.73:** Snapshot to generate servlet for the intermediate page that throws an error as the page is not executed*

In the preceding figure, you can see that trying to view servlet for the intermediately page results in an error:

- The include directive is more powerful.

- With the **`include`** directive, the included file is permitted to contain JSP code, such as response header settings and field definitions that affect the main page.

Forwarding requests with jsp:forward

You use **`jsp:include`** to combine output from the main page and the auxiliary page. Instead, you can use **`jsp:forward`** to obtain the complete output from the auxiliary page. Using **`jsp:forward`**, you can select the view dynamically based on some conditions.

Example 1: In this example, we have a login page, user inputs username and password; if the username and password are valid, the view layer selected is **`successe.jsp`**; if the username or password is invalid, then the view layer selected is **`error.jsp`**.

1. Login page to enter the credentials **`index.html`**

```
<!DOCTYPE html>

<html>
    <head>
        <title>TODO supply a title</title>
        <meta charset="UTF-8">
        <meta name="viewport" content="width=device-width, initial-scale=1.0">
    </head>
    <body>
        <h2>Login</h2>
        <form action="Login.jsp" method="post">
          Username :  <input type="text" name="uname"/><br/>
          Password :  <input type="password" name="pwd"/><br/>
          <input type="submit" value="Login"/>
        </form>
    </body>
</html>
```

Figure 7.74: *Page to enter the credentials index.html*

The following output will be generated by the preceding code:

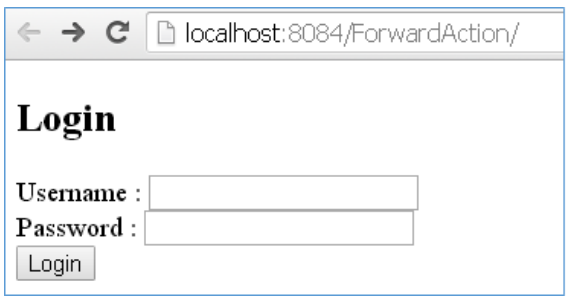


Figure 7.75*: Index page to input the username and password*

2. Page to read the credentials and navigate to the **`Login.jsp`** page

```
<%@page contentType="text/html" pageEncoding="UTF-8"%>
<!DOCTYPE html>
<html>
    <head>
        <meta http-equiv="Content-Type" content="text/html; charset=UTF-8">
        <title>JSP Page</title>
    </head>
    <body>
        <% String address,uname,pwd;
           uname=request.getParameter("uname");
           pwd=request.getParameter("pwd");
   if (uname.equals("admin") && pwd.equals("admin"))
                          {
    address = "success.jsp";
   }
   else
   {
   address = "error.jsp";
   }
                  request.setAttribute("u",uname);
  %>
  <jsp:forward page="<%= address %>" />

    </body>
</html>
```

Figure 7.76: *Credentials and navigate to the page Login.jsp*

3. When valid credentials are entered, the **`success.jsp`** page is selected

```
<%@page contentType="text/html" pageEncoding="UTF-8"%>
<!DOCTYPE html>
<html>
    <head>
        <meta http-equiv="Content-Type" content="text/html; charset=UTF-8">
        <title>JSP Page</title>
    </head>
    <body>
        <h1>
                Hello
              <span style="color:blue">
                          <%=request.getAttribute("u")%>
             </span>
        </h1>
    </body>
</html>
```

Figure 7.77*: Valid credentials entered; success.jsp page is selected*

The following output will be generated by the preceding code:

***Figure 7.78**: Index page to input valid username and password; the success page is rendered in the browser*

4. When invalid credentials are entered, the **`error.jsp`** page is selected

```
<%@page contentType="text/html" pageEncoding="UTF-8"%>
<!DOCTYPE html>
<html>
    <head>
        <meta http-equiv="Content-Type" content="text/html; charset=UTF-8">
        <title>JSP Page</title>
    </head>
    <body>
        <h1>Invalid username
                <span style="color:red">
                        <%=request.getAttribute("u")%>
                </span>
        </h1>
    </body>
</html>
```

***Figure 7.79:** Invalid credentials entered; error.jsp page is selected*

The following output will be generated by the preceding code:

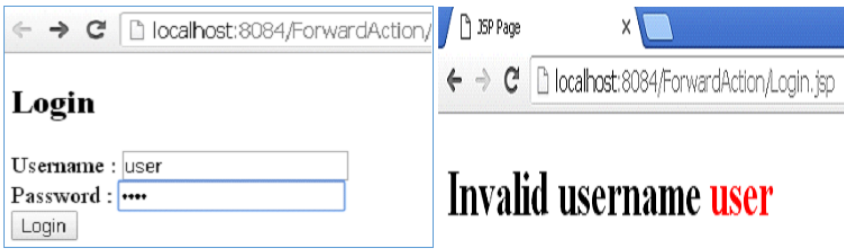


***Figure 7.80:** Index page to input invalid username and password; the error page is rendered in the browser*

The **`setAttribute`** method is used to share the username in the request scope, and the **`getAttribute`** is used to retrieve the username. Using the **`forward`** method, the page to which the navigation has occurred is not shown in the URL in the browser.

taglib directive

This directive allows user to use custom tags in JSP. We will discuss custom tags in detail in the upcoming JSP tutorials. The taglib directive helps you declare custom tags in JSP page.

Syntax of Taglib Directive:

```
<%@taglib uri ="taglibURI" prefix="tag prefix"%>
```

Where URI is uniform resource locator that is used to identify the location of custom tag, and tag prefix is a string that can identify the custom tag in the location.

Example to include the JSTL library:

```
<%@ taglib prefix="c" uri="http://java.sun.com/jsp/jstl/core"%>
```

Invalid credentials are entered, and the **`error.jsp`** page is selected. If you are writing XML-compatible JSP pages, you can use an alternative XML-compatible syntax for directives, as long as you don't mix the XML syntax and the classic syntax in the same page.

These constructs take the following form:

```
<jsp:directive.directiveType attribute="value" />
```

For example, the XML equivalent of the following:

```
<%@ page import="java.io.*" %> is
<jsp:directive.page import="java.io.*" />
```

Conclusion

In this chapter, we covered the basic syntax of JSP. JSP is used to write Java in HTML, but how, where, and how much we write matters a lot. So, understanding the basic scripting elements like scriptlets, expressions and declarations is very important. Then, we looked at the usage of directives like page, **`include`** and **`taglib`** directive. Additionally, we covered the include action, param element, forward action, and the XML syntax of using all the JSP tokens.

In the next chapter, we will study the concise way of accessing Java syntax using expression language, and we will explore JavaBeans functionality.

Points to remember

- Java Server Pages (JSP) technology enables you to mix regular, static HTML with dynamically generated content.

- JSP pages are translated into servlets. So, fundamentally, any task JSP pages can perform could also be accomplished by servlets.
- When the page contains lots of HTML, it is always preferable to use JSP for achieving the dynamic content on the page.

Multiple choice questions

1. **JSP embeds _____ in _____.**
    a. Servlet, HTML
    b. HTML, Java
    c. HTML, Servlet
    d. Java, HTML

2. **_____ are evaluated and inserted into the servlet's output.**
    a. Expressions
    b. Scriptlets
    c. Declarations
    d. Directives

3. **_____ are used when you need to write more than one line of Java code in HTML.**
    a. Expressions
    b. Scriptlets
    c. Declarations
    d. Directives

4. **The _____ directive is used to change the content type of the page displayed.**
    a. page
    b. include
    c. taglib
    d. None of these

5. **The _____ attribute lets you specify the packages that should be imported by the servlet into which the JSP page gets translated.**
    a. info
    b. extends
    c. import
    d. session

Answers

1. d **2. a** **3. b** **4. a** **5. c**

Questions

1. What is JSP? Why do we need JSP for developing a web application using Java?
2. Explain the life cycle of JSP.
3. Explain the use of JSP expressions with suitable examples.
4. Explain the use of JSP scriptlet with suitable examples.
5. Explain the use of JSP declaration with suitable examples.
6. List the directives provided by JSP.
7. Explain the page directive with any six attributes.
8. Explain the include directive.
9. Differentiate between the following:
    a. Include directive and include action
    b. **`response.setContentType`** and page directive attribute **`contentType`**

Key terms

- **JSP expression**: Expressions of the form **`<%= Java Expression %>`**, which are evaluated and inserted into the servlet's output
- **JSP scriptlets**: Scriptlets of the form **`<% Java Code %>`**, which are inserted into the servlet's **`_jspService`** method (called by **service**)
- **JSP declarations**: Declarations of the form **`<%! Field/Method Declaration %>`**, which are inserted into the body of the servlet class, outside any existing methods

Join our book's Discord space

Join the book's Discord Workspace for Latest updates, Offers, Tech happenings around the world, New Release and Sessions with the Authors:

https://discord.bpbonline.com

Chapter 8

Handling Beans and Using Expression Language

Introduction

The chapter covers the concept of Java bean creation and retrieval, and how to place the beans in different scope so that they can be shared. The second half of the chapter covers the formation of expressions for various aspects from the expression language.

Structure

In this chapter, we will learn the following topics:

- Introduction to JavaBeans
- JSP syntax for creating and accessing Beans
- Sharing Beans in different scopes
- Introduction to expression language
- Retrieving scoped variables
- Retrieving collections
- Using Implicit objects

- Retrieving Bean properties
- Using operators
- Using functions

Objectives

After studying this chapter, you should be able to understand the concept of accessing the Java bean. The chapter will provide an insight into how beans are shared and retrieved from the JSP page without writing the Java code.

Introduction to JavaBean

A JavaBean is a Java class that should follows these conventions:

- It should have a no-arg constructor.
- It should be serializable.
- It should provide methods to set and get the values of the properties, known as getter and setter methods.

It is a reusable software component. A bean encapsulates many objects into one object, so we can access this object from multiple places. Moreover, it provides easy maintenance.

Example 1: In this example, we create a bean for **`Student`** with **`id`**, **`name`**, **`program`**, **`semester`**, and **`marks`** of three courses and access the properties using a Java application.

a. Student bean class to hold the properties of a student (**`StudentBean.java`**)

```
01. package studentinformation;
02. public class StudentBean
03. {
04.     String id,name,program;
05.     int semester;
06.     double marks1,marks2,marks3;
07.
08.     public String getId() {         return id;     }
09.     public void setId(String id) {         this.id = id;     }
10.     public String getName() {      return name;    }
11.     public void setName(String name) {  this.name = name;    }
12.     public String getProgram() {    return program;    }
13.     public void setProgram(String program) {this.program = program;     }
14.     public int getSemester() {     return semester;    }
15.     public void setSemester(int semester) {this.semester = semester;     }
16.     public double getMarks1() {        return marks1;     }
17.     public void setMarks1(double marks1) {this.marks1 = marks1;  }
18.     public double getMarks2() {  return marks2; }
19.     public void setMarks2(double marks2) {this.marks2 = marks2; }
20.     public double getMarks3() {  return marks3;  }
21.     public void setMarks3(double marks3) {this.marks3 = marks3;  }
22. }
```

Figure 8.1: *Student bean class to hold the properties of a student, StudentBean.java*

b. Main class to instantiate the object of **`StudentBean`** class, and set and get the properties (**`StudentInformation.java`**)

```
01. package studentinformation;
02. import java.util.Scanner;
03. 
04. public class StudentInformation {
05.      public static void main(String[] args) {
06.          StudentBean s1=new StudentBean();
07.          String input;
08.          Scanner in= new Scanner(System.in);
09.          System.out.println("Enter Student ID :");
10.          input=in.nextLine();
11.          s1.setId(input);
12.          System.out.println("Enter Student Name :");
13.          input=in.nextLine();
14.          s1.setName(input);
15.          System.out.println("Enter Enrolled Program Name :");
16.          input=in.nextLine();
17.          s1.setProgram(input);
18.          System.out.println("Enter Semester :");
19.          s1.setSemester(in.nextInt());
20.          System.out.println("Enter Marks of Java :");
21.          s1.setMarks1(in.nextDouble());
22.          System.out.println("Enter Marks of Php :");
23.          s1.setMarks2(in.nextDouble());
24.          System.out.println("Enter Marks of .Net :");
25.          s1.setMarks3(in.nextDouble());
26.          System.out.println("Student Details");
27.          System.out.println("----------------");
28.          System.out.println("Student ID :" +s1.getId());
29.          System.out.println("Student Name :" +s1.getName());
30.          System.out.println("Student Enrolled Program :" +s1.getProgram());
31.          System.out.println("Student Semester :" +s1.getSemester());
32.          System.out.println("Student Marks of Java :" +s1.getMarks1());
33.          System.out.println("Student Marks of Php  :" +s1.getMarks2());
34.          System.out.println("Student Marks of .Net :" +s1.getMarks3());
35.      }
36. }
```

Figure 8.2: *Instantiate the object of StudentBean class*

c. Output of executing the program run

```
01. Enter Student ID :
02. ID001
03. Enter Student Name :
04. SAMEER THAKKER
05. Enter Enrolled Program Name :
06. MSCIT
07. Enter Semester :
08. 3
09. Enter Marks of Java :
10. 60
11. Enter Marks of Php :
12. 80
13. Enter Marks of .Net :
14. 78
15. 
16. Student Details
17. ----------------
18. Student ID :ID001
19. Student Name :SAMEER THAKKER
20. Student Enrolled Program :MSCIT
21. Student Semester :3
22. Student Marks of Java :60.0
23. Student Marks of Php  :80.0
24. Student Marks of .Net :78.0
25. 
26. BUILD SUCCESSFUL (total time: 42 seconds)
27. 
```

Figure 8.3: *Output after executing the program*

Here, we use the Scanner class of util package to input from console and assign the values to the properties of a bean class using the setter methods. Later, we retrieve the properties of bean using the getter methods. Apart from parameterized constructors, we can use setter methods to assign values to the properties of a class. This is example of using simple bean class to make a separate reusable component in a Java application.

Advantages of using Java bean

Following are the advantages of using Java bean:

- Although it is true that beans are merely Java classes that are written in a standard format, there are several advantages to their use.
- With beans in general, visual manipulation tools and other programs can automatically discover information about classes that follow this format, and can create and manipulate the classes without the user having to explicitly write any code.
- In JSP in particular, use of JavaBeans components provides three advantages over scriptlets and JSP expressions that refer to normal Java classes:
    - No Java syntax
    - Simpler object sharing
    - Convenient correspondence between request parameters and object properties
- A software component architecture provides standard mechanisms to deal with software building blocks. The following list enumerates some of the specific benefits that Java technology provides for a component developer:
    - A Bean obtains all the benefits of Java's *write-once, run-anywhere* paradigm.
    - The properties, events, and methods of a Bean that are exposed to an application builder tool can be controlled.
    - A Bean may be designed to operate correctly in different locales, which makes it useful in global markets.
    - Auxiliary software can be provided to help a person configure a Bean. This software is only needed when the design-time parameters for that component are being set. It does not need to be included in the runtime environment.
    - The configuration settings of a Bean can be saved in persistent storage and restored later.

- A Bean may register to receive events from other objects and can generate events that are sent to other objects.

Using Java beans in a web application

In the previous section, we saw the use of beans in a simple Java application. In this section, we will see how we can use beans in a web application.

Example 1: In this example, we create a bean for **`Student`** with **`id`**, **`name`**, **`program`**, **`semester`**, **`marks`** of three courses and grade. We create an **`index`** page that gives link to access the student details entry form and enter the details in the student entry form. The form data is passed to another JSP page whose task is to read the form data, set the bean properties and then retrieve the bean properties and display it. The web application directory structure is as follows:

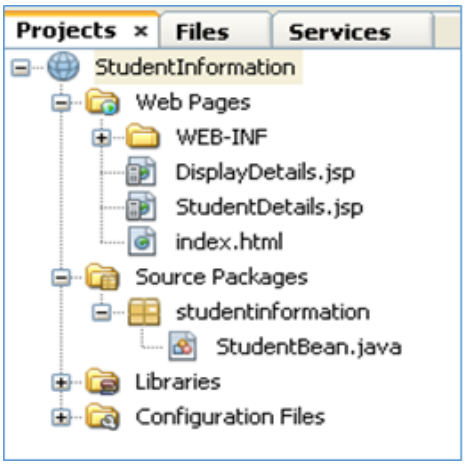


Figure 8.4: *Directory Structure of the application*

a. Index page of the application **`index.html`**

```
<!DOCTYPE html>
<html>
<head>
<title>Student Information</title>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
</head>
<body>
<h3><a href="StudentDetails.jsp">Students entry form</a></h3>
</body>
</html>

```

Figure 8.5*: Index page of the application index.html*

The following output will be generated by the preceding code:

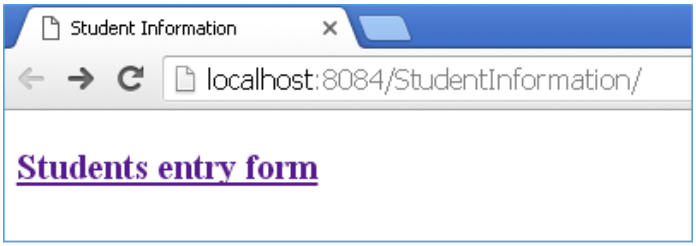


***Figure 8.6**: Index page of the application*

b. Student bean class to hold the properties of a student (**`StudentBean.java`**)

```
01. package studentinformation;
02. public class StudentBean {
03. 
04.     String id, name, program;
05.     int semester;
06.     double marks1, marks2, marks3;
07.     String grade;
08. 
09.     public String getId() {   return id;    }
10.     public void setId(String id) {   this.id = id;    }
11.     public String getName() {  return name;   }
12.     public void setName(String name) {  this.name = name;  }
13.     public String getProgram() { return program;  }
14.     public void setProgram(String program) {  this.program = program;  }
15.     public int getSemester() {  return semester;   }
16.     public void setSemester(int semester) { this.semester = semester;   }
17.     public double getMarks1() {  return marks1;   }
18.     public void setMarks1(double marks1) {this.marks1 = marks1;     }
19.     public double getMarks2() {  return marks2;   }
20.     public void setMarks2(double marks2) {this.marks2 = marks2;    }
21.     public double getMarks3() {     return marks3;   }
22.     public void setMarks3(double marks3) {this.marks3 = marks3;    }
23.     public String getGrade() {    return grade;   }
24.     public void setGrade() {
25.        double per = (marks1 + marks2 + marks3) / 3; // each subject is of 100 marks
26.        if (marks1 <= 35 || marks2 <= 35 || marks3 <= 35) {   grade = "FF";  }
27.        else if (per >= 80) { grade = "AA";  }
28.        else if (per >= 70) { grade = "BB";  }
29.        else if (per >= 60) { grade = "CC";  }
30.        else { grade = "DD"; }
31.     }
32. }
33. 
```

***Figure 8.7:** Student bean class to hold the properties of a student*

c. Student detail input **`formStudentDetails.jsp`**

```
<!DOCTYPE html>
<html>
<head>
<title>Student Information</title>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
</head>
<body>
<h2>Student Input Form </h2>
<form action="DisplayDetails.jsp">
<table>
<tr>
<td>Student ID :</td>
<td><input type="text" name="studentid" required/><br/></td>
</tr>
<tr>
<td>Student Name :</td>
<td><input type="text" name="studentname" required/><br/></td>
</tr>
<tr>
<td>Student Enrolled Programm :</td>
<td><input type="text" name="studentprogram" required/><br/></td>
</tr>
<tr>
<td>Student Semester :</td>
<td><input type="text" name="studentsemester" required/><br/></td>
</tr>
<tr>
<td> Java Marks out of 100 :</td>
<td><input type="text" name="marks1" required/><br/></td>
</tr>
<tr>
<td>PHP Marks  out of 100: </td>
<td><input type="text" name="marks2" required/><br/></td>
</tr>
<tr>
<td>.Net Marks  out of 100:</td>
<td><input type="text" name="marks3" required/><br/></td>
</tr>
<tr>
<td><input type="submit" value="Calculate Grade"/></td>
</tr>
</table>
</form>
</body>
</html>

```

Figure 8.8: *Student Detail input formStudentDetails.jsp*

The following output will be generated by the preceding code:

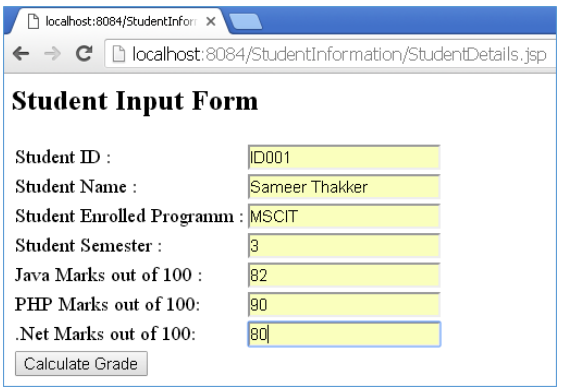


Figure 8.9: *Student entry form*

d. Read data and display the data submitted through the form by setting and getting the bean **propertiesDisplayDetails.jsp**

```
<!DOCTYPE html>
<html>
<head>
<title>Student Information</title>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
</head>
<body>
<h2>Student Details </h2>
<%
studentinformation.StudentBean s1 = new studentinformation.StudentBean();
            s1.setId(request.getParameter("studentid"));
            s1.setName(request.getParameter("studentname"));
            s1.setProgram(request.getParameter("studentprogram"));

            try {
                s1.setSemester(Integer.parseInt(request.getParameter("studentsemester")));
                s1.setMarks1(Double.parseDouble(request.getParameter("marks1")));
                s1.setMarks2(Double.parseDouble(request.getParameter("marks2")));
                s1.setMarks3(Double.parseDouble(request.getParameter("marks3")));
                s1.setGrade();
               } catch (NumberFormatException nfe) {
            }
        %>
<table>
<tr><td>Student ID :</td><td><%=s1.getId()%><br/></td></tr>
<tr><td>Student Name :</td><td><%=s1.getName()%><br/></td></tr>
<tr><td>Student Enrolled Program: </td><td><%=s1.getProgram()%><br/></td></tr>
<tr><td>Java Marks  :</td><td><%=s1.getMarks1()%><br/></td></tr>
<tr><td>PHP Marks  :</td><td><%=s1.getMarks2()%><br/></td></tr>
<tr><td>.Net Marks  :</td><td><%=s1.getMarks3()%><br/></td></tr>
<tr><td>Grade  :</td><td><%=s1.getGrade()%><br/></td></tr>
</table>
</body>
</html>

```

Figure 8.10: *Getting the bean propertiesDisplayDetails.jsp*

Following is the display form screen:

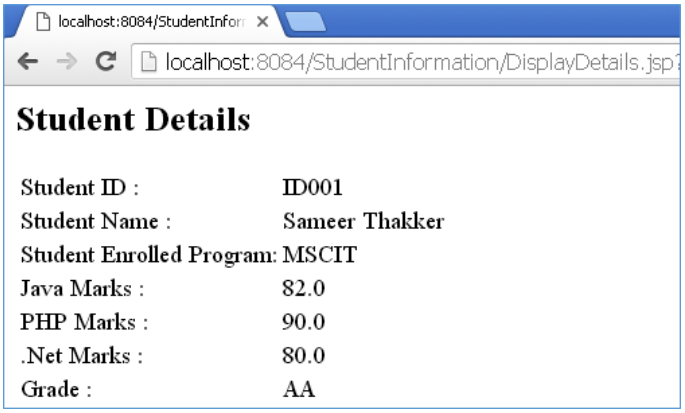


Figure 8.11*: Display form*

e. Resultant Servlet for **`DisplayDetails.jsp`**

In the **`DisplayDetails.jsp`**, we write the code to instantiate the bean object, read the request parameters and set the bean properties using the JSP scriptlet; after the properties are set, we write the code to access the properties using the getter methods in the JSP expression. In this example, we are creating, assigning and retrieving beans using core Java syntax.

JSP syntax for creating and accessing beans

JSP provides three main components to create bean, and to assign and retrieve values of a bean. They are listed as follows:

`jsp:useBean`

- This element is used to build a new bean.
- It is normally used as follows:

  Syntax: **`<jsp:useBeanid="beanName" class="package.Class" />`**

`jsp:getProperty`

- This element is used to read and output the value of a bean property.
- Reading a property is a shorthand notation for calling a method of the **`getXxx`** form. This element is used as follows:

  Syntax: **`<jsp:getProperty name="beanName" property="propertyName" />`**

`jsp:setProperty`

- This element is used to assign a value to the bean property, and it is equivalent to calling a setter method of the **`setXxx`** form.
- This element is used as follows:

  Syntax: **`<jsp:setProperty name ="beanName" property="propertyName" value="propertyValue" />`**

Creating beans

The **`jsp:useBean`** action lets you load a bean to be used in the JSP page. Beans provide a very useful capability because they let you exploit the reusability of Java classes without sacrificing the convenience that JSP adds over servlets alone.

The **`jsp:useBean`** action tag is used to locate or instantiate a bean class. If the bean object of the **`Bean`** class is already created, it doesn't create the bean depending on the scope. But if the object of bean is not created, it instantiates the bean. The simplest syntax for specifying that a bean should be used is the following:

```
<jsp:useBean id="name" class="package.Class" />
```

This statement usually means "instantiate an object of the class specified by class and bind it to a variable in **_jspService** with the name specified by **id**". For example, the following JSP action:

```
<jsp:useBean id="s1" class="studentinformation.StudentBean " />
```

It can normally be thought of as equivalent to the following scriptlet:

```
<% studentinformation.StudentBean s1 = new studentinformation.
StudentBean(); %>
```

Here are the attributes of **jsp:useBean** action tag:

- **id**: It is used to identify the bean in the specified scope.
- **scope**: It represents the scope of the bean. It may be a page, request, session, or application. The default scope is page.
    - **page**: It specifies that you can use this bean within the JSP page. The default scope is page.
    - **request**: It specifies that you can use this bean from any JSP page that processes the same request. It has a wider scope than page.
    - **session**: It specifies that you can use this bean from any JSP page in the same session, regardless of whether or not it processes the same request. It has wider scope than request.
    - **application**: It specifies that you can use this bean from any JSP page in the same application. It has wider scope than session.
- **class**: It instantiates the specified bean class (that is, creates an object of the bean class) but it must have no-arg or no constructor and must not be abstract.
- **type**: It provides the bean a data type if the bean already exists in the scope. It is mainly used with class or **beanName** attribute. If you use it without class or **beanName**, no bean is instantiated.
- **beanName**: It instantiates the bean using the **java.beans.Beans.instantiate()** method.

Retrieving values from beans

Once you have a bean, you can output its properties with **jsp:getProperty**, which takes a **name** attribute that should match the **id** given in **jsp:useBean** and a property attribute that names the property of interest.

Instead of using **jsp:getProperty**, you could use a JSP expression and explicitly call a method on the object with the variable name specified by the **id** attribute.

For example, assuming that the **Book** class has a **String** property called title and that you've created an instance called **book1** using the **jsp:useBean** example given earlier in this section, you could insert the value of the **title** property into the JSP page in either of the following two ways:

```
<jsp:getProperty name="s1" property="id" />
<%= s1.getId() %>
```

Assigning values to beans

To modify bean properties, you normally use **jsp:setProperty**. This action has several forms, but with the simplest form, you supply three attributes: **name** (which should match the ID given by **jsp:useBean**), **property** (the name of the property to change), and **value** (the new value).

An alternative to using the **jsp:setProperty** action is to use a scriptlet that explicitly calls methods on the bean object. For example, given the **book1** object shown earlier in this section, you could use either of the following two forms to modify the **title** property:

```
<jsp:setProperty name="s1" property="program" value="MSCIT" />
<% s1.setTitle("MSCIT"); %>
```

Example1: We modify the **DisplayDetails.jspfile** with a new **DisplayDetails_1.jsp** file to include the **jsp:useBean**, **jsp:setProperty** and **jsp:getProperty** for creating, reading, and writing a bean. We modify the bean class to update the **grade** method to include a **String** parameter; the updated code is as follows:

a. Index page to **index.html**

```
01. <!DOCTYPE html>
02. <html>
03. <head>
04. <title>Student Information</title>
05. <meta charset="UTF-8">
06. <meta name="viewport" content="width=device-width, initial-scale=1.0">
07. </head>
08. <body>
09. <h3><a href="StudentDetails.jsp">Students entry form</a></h3>
10. </body>
11. </html>
12.
```

Figure 8.12: *Index page to index.html*

The following output will be generated by the preceding code:

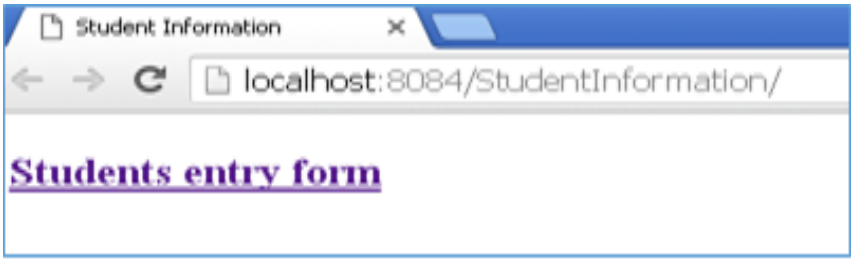


Figure 8.13: *Student information index form*

b. Student bean class to hold the properties of a student (**`StudentBean.java`**)

```
01. package studentinformation;
02.
03. public class StudentBean {
04.     String id, name, program;
05.     int semester;
06.     double marks1, marks2, marks3;
07.     String grade;
08.
09.     public String getId() {   return id;   }
10.     public void setId(String id) {   this.id = id;   }
11.     public String getName() {  return name;   }
12.     public void setName(String name) { this.name = name;   }
13.     public String getProgram() {   return program;    }
14.     public void setProgram(String program) {this.program = program;    }
15.     public int getSemester() {        return semester;    }
16.     public void setSemester(int semester) {this.semester = semester;    }
17.     public double getMarks1() {   return marks1;    }
18.     public void setMarks1(double marks1) {this.marks1 = marks1;    }
19.     public double getMarks2() {    return marks2;   }
20.     public void setMarks2(double marks2) {this.marks2 = marks2;    }
21.     public double getMarks3() {    return marks3;   }
22.     public void setMarks3(double marks3) {this.marks3 = marks3;    }
23.     public String getGrade() {     return grade;   }
24.     public void setGrade(String g) {
25.        double per = (marks1 + marks2 + marks3) / 3; // each subject is of 100 marks
26.        if (marks1 <= 35 || marks2 <= 35 || marks3 <= 35) {  grade = "FF";  }
27.        else if (per >= 80) {   grade = "AA";   }
28.        else if (per >= 70) {  grade = "BB";    }
29.        else if (per >= 60) {   grade = "CC";    }
30.        else {   grade = "DD";   }
31.     }
32. }
```

Figure 8.14: *Student bean class to hold the properties of a student*

c. Student detail input form **`StudentDetails.jsp`**

```
<!DOCTYPE html>
<html>
<head>
<title>Student Information</title>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
</head>
<body>
<h2>Student Input Form </h2>
<form action="DisplayDetails_1.jsp">
<table>
<tr><td>Student ID :</td><td><input type="text" name="studentid" required/><br/></td></tr>
<tr><td>Student Name :</td><td><input type="text" name="studentname" required/><br/></td></tr>
<tr><td>Student Enrolled Programm :</td><td><input type="text" name="studentprogram" required/><br/></td></tr>
<tr><td>Student Semester :</td><td><input type="text" name="studentsemester" required/><br/></td></tr>
<tr><td> Java Marks out of 100 :</td><td><input type="text" name="marks1" required/><br/></td></tr>
<tr><td>PHP Marks  out of 100: </td><td><input type="text" name="marks2" required/><br/></td></tr>
<tr><td>.Net Marks  out of 100:</td><td><input type="text" name="marks3" required/><br/></td></tr>
<tr><td><input type="submit" value="Calculate Grade"/></td></tr>
</table>
</form>
</body>
</html>
```

Figure 8.15: *Student detail input form StudentDetails.jsp*

The following output will be generated by the preceding code:

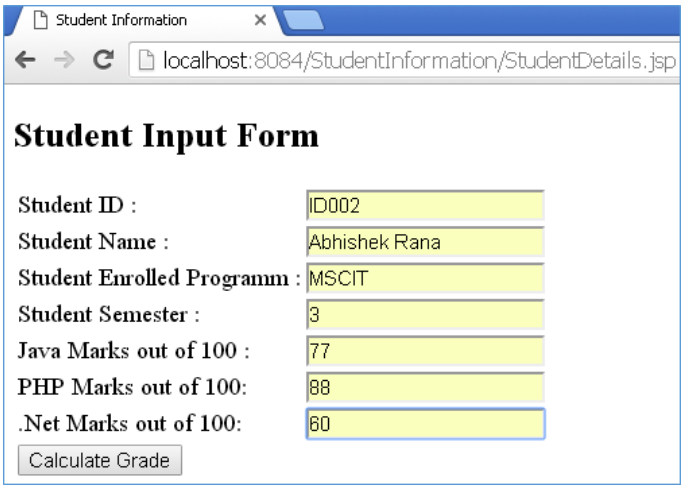


Figure 8.16: *Student information input form*

d. Read and display the data submitted through the form by setting and getting the bean properties using **`jsp:useBean`**, **`jsp:setProperty`**, and **`jsp:getProperty`** in **`DisplayDetails_1.jsp`**

```
<!DOCTYPE html>
<html>
<head>
<title>Student Information</title>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
</head>
<body>
<h2>Student Details </h2>
<jsp:useBean id="s1" class="studentinformation.StudentBean"/>
<jsp:setProperty  name="s1" property="id"
                                    value='<%=request.getParameter("studentid")%>'/>
<jsp:setProperty  name="s1" property="name"
                                    value='<%=request.getParameter("studentname")%>'/>
<jsp:setProperty  name="s1" property="program"
                                    value='<%=request.getParameter("studentprogram")%>'/>
<%      int sem=0;
            double marks1=0, marks2=0, marks3=0;
            try {
                sem = Integer.parseInt(request.getParameter("studentsemester"));
                marks1 = Double.parseDouble(request.getParameter("marks1"));
                marks2 = Double.parseDouble(request.getParameter("marks2"));
                marks3 = Double.parseDouble(request.getParameter("marks3"));
            } catch (NumberFormatException nfe) {

            }
        %>
<jsp:setProperty name="s1" property="semester" value='<%=sem%>'/>
<jsp:setProperty name="s1" property="marks1" value='<%=marks1%>'/>
<jsp:setProperty name="s1" property="marks2" value='<%=marks2%>'/>
<jsp:setProperty name="s1" property="marks3" value='<%=marks3%>'/>
<jsp:setProperty name="s1" property="grade" value="Default" />

<table>
<tr><td>Student ID :</td><td><jsp:getProperty name="s1" property="id" /><br/></td></tr>
<tr><td>Student Name :</td><td><jsp:getProperty name="s1" property="name" /><br/></td></tr>
<tr><td>Student Enrolled Program: </td><td><jsp:getProperty name="s1" property="program" /><br/></td></tr>
<tr><td>Java Marks  :</td><td><jsp:getProperty name="s1" property="marks1" /><br/></td></tr>
<tr><td>PHP Marks  :</td><td><jsp:getProperty name="s1" property="marks2" /><br/></td></tr>
<tr><td>.Net Marks  :</td><td><jsp:getProperty name="s1" property="marks3" /><br/></td></tr>
<tr><td>Grade  :</td><td><jsp:getProperty name="s1" property="grade" /><br/></td></tr>
</table>
</body>
</html>
```

Figure 8.17: *Getting the bean properties using jsp:useBean, jsp:setProperty and jsp:getProperty*

The following output will be generated by the preceding code:

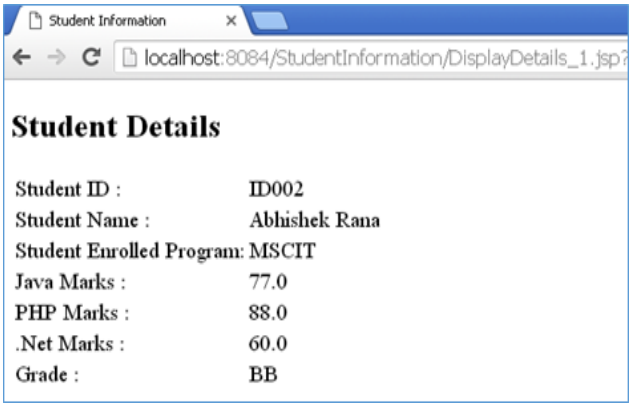


Figure 8.18: *Grade calculation and display form*

e. Resultant servlet for **`DisplayDetails_1.jsp`**

```
01. package org.apache.jsp;
02. import javax.servlet.*;
03. import javax.servlet.http.*;
04. import javax.servlet.jsp.*;
05. public final class DisplayDetails_005f1_jsp extends org.apache.jasper.runtime.HttpJspBase
06. implements org.apache.jasper.runtime.JspSourceDependent {
07.   private static final JspFactory _jspxFactory = JspFactory.getDefaultFactory();
08.   private static java.util.List<String> _jspx_dependants;
09.   private org.glassfish.jsp.api.ResourceInjector _jspx_resourceInjector;
10.   public java.util.List<String> getDependants() {  return _jspx_dependants;  }
11. public void _jspService(HttpServletRequest request, HttpServletResponse response)
12.         throws java.io.IOException, ServletException {
13.     PageContext pageContext = null;     HttpSession session = null;
14.     ServletContext application = null;     ServletConfig config = null;
15.     JspWriter out = null;                         Object page = this;
16.     JspWriter _jspx_out = null;               PageContext _jspx_page_context = null;
17.     try {
18.             response.setContentType("text/html");
19.             response.setHeader("X-Powered-By", "JSP/2.2");
20.             pageContext = _jspxFactory.getPageContext(this, request, response,null, true, 8192, true);
21.            _jspx_page_context = pageContext;      application = pageContext.getServletContext();
22.             config = pageContext.getServletConfig();      session = pageContext.getSession();
23.             out = pageContext.getOut();      _jspx_out = out;
24.            _jspx_resourceInjector = (org.glassfish.jsp.api.ResourceInjector) application.getAttribute("com.sun.appserv.jsp.resource.injector");
25.             out.write("<!DOCTYPE html>\n");
26.             out.write("<html>\n");
27.             out.write("    <head>\n");
28.             out.write("        <title>Student Information</title>\n");
29.             out.write("        <meta charset=\"UTF-8\">\n");
30.             out.write("        <meta name=\"viewport\" content=\"width=device-width, initial-scale=1.0\">\n");
31.             out.write("    </head>\n");
32.             out.write("    <body>\n");
33.             out.write("        <h2>Student Details </h2>\n");
34.             out.write("        ");
35.             studentinformation.StudentBean s1 = null;
36.             synchronized (_jspx_page_context) {
37.             s1 = (studentinformation.StudentBean) _jspx_page_context.getAttribute("s1", PageContext.PAGE_SCOPE);
38.             if (s1 == null){
39.             s1 = new studentinformation.StudentBean();
40.            _jspx_page_context.setAttribute("s1", s1, PageContext.PAGE_SCOPE);
41.         }
```

Figure 8.19: *Resultant Servlet for DisplayDetails_1.jsp*

This is the resultant servlet for the JSP code we mentioned earlier:

```
42.     }
43.         out.write("\n");
44.         out.write("        ");
45.         org.apache.jasper.runtime.JspRuntimeLibrary.handleSetProperty(_jspx_page_context.findAttribute("s1"), "id",request.getParameter("studentid"));
46.         out.write("\n");
47.         out.write("        ");
48.         org.apache.jasper.runtime.JspRuntimeLibrary.handleSetProperty(_jspx_page_context.findAttribute("s1"),"name",request.getParameter("studentname"));
49.         out.write("\n");
50.         out.write("        ");       org.apache.jasper.runtime.JspRuntimeLibrary.handleSetProperty(_jspx_page_context.findAttribute("s1"),"program",request.getParameter("studentprogram
51.         out.write("\n");
52.         out.write("        ");
53.         int sem=0;
54.         double marks1=0, marks2=0, marks3=0;
55.         try {
56.             sem = Integer.parseInt(request.getParameter("studentsemester"));
57.             marks1 = Double.parseDouble(request.getParameter("marks1"));
58.             marks2 = Double.parseDouble(request.getParameter("marks2"));
59.             marks3 = Double.parseDouble(request.getParameter("marks3"));
60.          } catch (NumberFormatException nfe) {
61.          }
62.         out.write("\n");
63.         out.write("        ");
64.         org.apache.jasper.runtime.JspRuntimeLibrary.handleSetProperty(_jspx_page_context.findAttribute("s1"), "semester",sem);
65.         out.write("\n");
66.         out.write("        ");
67.         org.apache.jasper.runtime.JspRuntimeLibrary.handleSetProperty(_jspx_page_context.findAttribute("s1"), "marks1",marks1);
68.         out.write("\n");
69.         out.write("        ");
70.         org.apache.jasper.runtime.JspRuntimeLibrary.handleSetProperty(_jspx_page_context.findAttribute("s1"), "marks2",marks2);
71.         out.write("\n");
72.         out.write("        ");
73.         org.apache.jasper.runtime.JspRuntimeLibrary.handleSetProperty(_jspx_page_context.findAttribute("s1"), "marks3",marks3);
74.         out.write("\n");
75.         out.write("        ");
76.         org.apache.jasper.runtime.JspRuntimeLibrary.introspecthelper(_jspx_page_context.findAttribute("s1"), "grade", "Default", null, null, false);
77.         out.write("\n");
78.         out.write("        \n");
79.         out.write("        <table>\n");
80.         out.write("            <tr><td>Student ID : </td><td> ");
81.         out.write(org.apache.jasper.runtime.JspRuntimeLibrary.toString((((studentinformation.StudentBean)_jspx_page_context.findAttribute("s1")).getId())));
82.         out.write(" <br/></td></tr> \n");
```

***Figure 8.20**: Resultant servlet for the JSP code*

This is the resultant servlet for the JSP code we mentioned earlier:

```
          out.write("           <tr><td>Student Name : </td><td> ");
        out.write(org.apache.jasper.runtime.JspRuntimeLibrary.toString((((studentinformation.StudentBean)_jspx_page_context.findAttribute("s1")).getName())));
          out.write("<br/></td></tr> \n");
          out.write("           <tr><td>Student Enrolled Program: </td><td> ");


out.write(org.apache.jasper.runtime.JspRuntimeLibrary.toString((((studentinformation.StudentBean)_jspx_page_context.findAttribute("s1")).getProgram())));
          out.write("<br/></td></tr> \n");
          out.write("           <tr><td>Java Marks  : </td><td>");
      out.write(org.apache.jasper.runtime.JspRuntimeLibrary.toString((((studentinformation.StudentBean)_jspx_page_context.findAttribute("s1")).getMarks1())));
          out.write("<br/></td></tr> \n");
          out.write("           <tr><td>PHP Marks  : </td><td>");
      out.write(org.apache.jasper.runtime.JspRuntimeLibrary.toString((((studentinformation.StudentBean)_jspx_page_context.findAttribute("s1")).getMarks2())));
          out.write("<br/></td></tr> \n");
          out.write("           <tr><td>.Net Marks  : </td><td>");
      out.write(org.apache.jasper.runtime.JspRuntimeLibrary.toString((((studentinformation.StudentBean)_jspx_page_context.findAttribute("s1")).getMarks3())));
          out.write("<br/></td></tr> \n");
          out.write("           <tr><td>Grade  : </td><td>");
        out.write(org.apache.jasper.runtime.JspRuntimeLibrary.toString((((studentinformation.StudentBean)_jspx_page_context.findAttribute("s1")).getGrade())));
        out.write("<br/></td></tr> \n");
        out.write("       </table>\n");
        out.write("    </body>\n");
        out.write("</html>\n");
    } catch (Throwable t) {
      if (!(t instanceof SkipPageException)){
        out = _jspx_out;
        if (out != null && out.getBufferSize() != 0)
          out.clearBuffer();
        if (_jspx_page_context != null) _jspx_page_context.handlePageException(t);
        else throw new ServletException(t);
      }
    } finally {
      _jspxFactory.releasePageContext(_jspx_page_context);
    }
  }
}
```

Figure 8.21: *Resultant servlet for the JSP code*

Assigning properties of bean with input parameter values

We use **`jsp:setProperty`** to set bean properties. The simplest form of this action takes three attributes:

- **`name`** (which should match the id given by **`jsp:useBean`**)
- **`property`** (the name of the property to change)
- **`value`** (the new value)

Setting the program property is easy since its value is a String.

```
<jsp:setProperty name="s1" property="program" value='<%=request.
getParameter("studentprogram")%>'/>
```

Setting the **semester**, **marks1**, **marks2** and **marks3** properties is a bit more problematic since their values must be numbers, whereas **getParameter** returns a **String**. Here's the somewhat cumbersome code required to set **numItems**:

```
<%
            int sem=0;
            double marks1=0, marks2=0, marks3=0;
            try {
                sem = Integer.parseInt(request.getParameter("studentsemester"));
                marks1 = Double.parseDouble(request.getParameter("marks1"));
                marks2 = Double.parseDouble(request.getParameter("marks2"));
                marks3 = Double.parseDouble(request.getParameter("marks3"));
            } catch (NumberFormatException nfe) {

            }
 %>
<jsp:setProperty name="s1" property="semester" value='<%=sem%>'/>
<jsp:setProperty name="s1" property="marks1" value='<%=marks1%>'/>
<jsp:setProperty name="s1" property="marks2" value='<%=marks2%>'/>
<jsp:setProperty name="s1" property="marks3" value='<%=marks3%>'/>
<jsp:setProperty name="s1" property="grade" value="Default" />

```

Figure 8.22: *Code required to set numItems*

Fortunately, JSP has a nice solution to this problem. It lets you associate a property with a request parameter and automatically perform type conversion from strings to numbers, characters, and boolean values. Instead of using the value attribute, you use param to name an **input** parameter.

The value of the named request parameter is automatically used as the value of the **bean** property, and type conversions from **String** to primitive types (**byte**, **int**, **double**, etc.) and wrapper classes (**Byte**, **Integer**, **Double**, etc.) are automatically performed. For example, setting the semester property can be simplified to the following:

```
<jsp:setProperty name="s1"  property="semester"  param="studentsemester"
/>
```

You can simplify the code slightly if the request parameter name and the bean property name are the same. In that case, you can omit param as in the following example:

```
<jsp:setPropertyname="s1" property="semester" />
<%-- param="semester" is assumed. --%>
```

Example 1: In the previous example, we make changes to read the request parameters and make changes in the **StudentBean** properties as shown in the following code.

a. Index page to **`index.html`**

```
<!DOCTYPE html>
<html>
<head>
<title>Student Information</title>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
</head>
<body>
<h3><a href="StudentDetails.jsp">Students entry form</a></h3>
</body>
</html>

```

Figure 8.23: *Index page to index.html*

b. Student bean class to hold the properties of a student (**`StudentBean.java`**)

```
package studentinformation;

public class StudentBean
{
    String id, name, program;
    int semester;
    double marks1, marks2, marks3;
    String grade;

    public String getId() {    return id;    }
    public void setId(String id) {        this.id = id;    }
    public String getName() {        return name;    }
    public void setName(String name) {        this.name = name;    }
    public String getProgram() {        return program;    }
    public void setProgram(String program) {this.program = program;    }
    public int getSemester() {        return semester;    }
    public void setSemester(int semester) {this.semester = semester;    }
    public double getMarks1() {        return marks1;    }
    public void setMarks1(double marks1) {this.marks1 = marks1;    }
    public double getMarks2() {        return marks2;    }
    public void setMarks2(double marks2) {this.marks2 = marks2;    }
    public double getMarks3() {        return marks3;    }
    public void setMarks3(double marks3) {this.marks3 = marks3;    }
    public String getGrade() {
        double per = (marks1 + marks2 + marks3) / 3; // each subject is of 100 marks
        if (marks1 <= 35 || marks2 <= 35 || marks3 <= 35) {      grade = "FF";   }
        else if (per >= 80) {  grade = "AA";   }
        else if (per >= 70) {  grade = "BB";    }
        else if (per >= 60) {  grade = "CC";   }
        else { grade = "DD";  }
        return grade;
    }
    public void setGrade() {    }
}
```

Figure 8.24: *Student bean class to hold the properties of a student*

c. Student detail input form **`StudentDetails.jsp`**

```
<!DOCTYPE html>
<html>
<head>
<title>Student Information</title>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
</head>
<body>
<h2>Student Input Form </h2>
<form action="DisplayDetails_2.jsp">
<table>
<tr><td>Student ID :</td><td><input type="text" name="studentid" required/><br/></td></tr>
<tr><td>Student Name :</td><td><input type="text" name="studentname" required/><br/></td></tr>
<tr><td>Student Enrolled Programm :</td><td><input type="text" name="studentprogram" required/><br/></td></tr>
<tr><td>Student Semester :</td><td><input type="text" name="studentsemester" required/><br/></td></tr>
<tr><td> Java Marks out of 100 :</td><td><input type="text" name="marks1" required/><br/></td></tr>
<tr><td>PHP Marks  out of 100: </td><td><input type="text" name="marks2" required/><br/></td></tr>
<tr><td>.Net Marks  out of 100:</td><td><input type="text" name="marks3" required/><br/></td></tr>
<tr><td><input type="submit" value="Calculate Grade"/></td></tr>
</table>
</form>
</body>
</html>
```

Figure 8.25: *Student detail input form StudentDetails.jsp*

The following output will be generated by the preceding code:

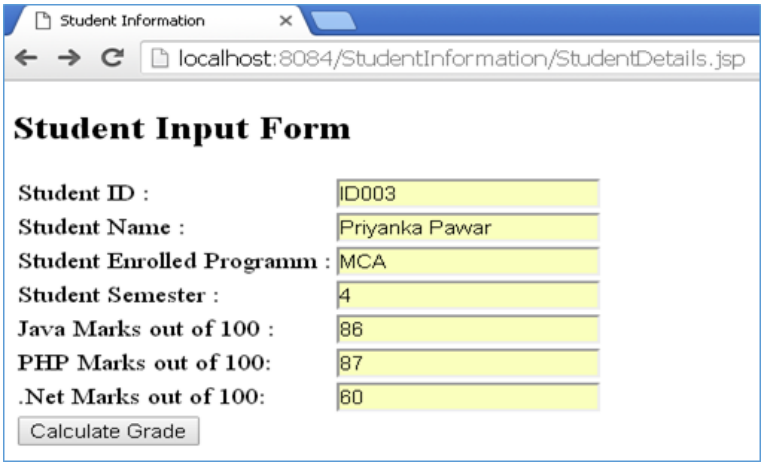


Figure 8.26: *Input form*

d. Read and display the data submitted through the form by setting and getting the bean properties **`DisplayDetails_2.jsp`**

```
<!DOCTYPE html>
<html>
<head>
<title>Student Information</title>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
</head>
<body>
<h2>Student Details </h2>
<jsp:useBean id="s1" class="studentinformation.StudentBean"/>
<jsp:setProperty name="s1" property="id" param="studentid"/>
<jsp:setProperty name="s1" property="name" param="studentname"/>
<jsp:setProperty name="s1" property="program" param="studentprogram"/>
<jsp:setProperty name="s1" property="semester" param="studentsemester"/>
<jsp:setProperty name="s1" property="marks1" param="marks1"/>
<jsp:setProperty name="s1" property="marks2" param="marks2"/>
<jsp:setProperty name="s1" property="marks3" param="marks3"/>

<table>
<tr><td>Student ID :</td><td><jsp:getProperty name="s1" property="id" /><br/></td></tr>
<tr><td>Student Name :</td><td><jsp:getProperty name="s1" property="name" /><br/></td></tr>
<tr><td>Student Enrolled Program: </td><td><jsp:getProperty name="s1" property="program" /><br/></td></tr>
<tr><td>Java Marks  :</td><td><jsp:getProperty name="s1" property="marks1" /><br/></td></tr>
<tr><td>PHP Marks  :</td><td><jsp:getProperty name="s1" property="marks2" /><br/></td></tr>
<tr><td>.Net Marks  :</td><td><jsp:getProperty name="s1" property="marks3" /><br/></td></tr>
<tr><td>Grade  :</td><td><jsp:getProperty name="s1" property="grade" /><br/></td></tr>
</table>
</body>
</html>
```

Figure 8.27: *Display data submitted through the form by setting and getting the bean properties DisplayDetails_2.jsp*

The following output will be generated by the preceding code:

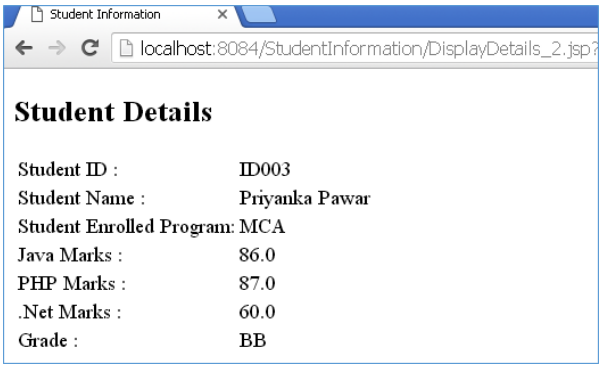


Figure 8.28: *Student grade display form*

Example 2: In the preceding example, we make changes to the names of **`input`** tag and make them the same as Java properties, so while reading the **`request`** parameters, we need not use the **`param`** attribute as follows.

a. Student detail input form **`StudentDetails_1.jsp`**

```
<!DOCTYPE html>
<html>
<head>
<title>Student Information</title>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
</head>
<body>
<h2>Student Input Form </h2>
<form action="DisplayDetails_3.jsp">
<table>
<tr><td>Student ID :</td><td><input type="text" name="id" required/><br/></td></tr>
<tr><td>Student Name :</td><td><input type="text" name="name" required/><br/></td></tr>
<tr><td>Student Enrolled Programm :</td><td><input type="text" name="program" required/><br/></td></tr>
<tr><td>Student Semester :</td><td><input type="text" name="semester" required/><br/></td></tr>
<tr><td> Java Marks out of 100 :</td><td><input type="text" name="marks1" required/><br/></td></tr>
<tr><td>PHP Marks  out of 100: </td><td><input type="text" name="marks2" required/><br/></td></tr>
<tr><td>.Net Marks  out of 100:</td><td><input type="text" name="marks3" required/><br/></td></tr>
<tr><td><input type="submit" value="Calculate Grade"/></td></tr>
</table>
</form>
</body>
</html>
```

Figure 8.29: *Student detail input form StudentDetails_1.jsp*

The following output will be generated by the preceding code:

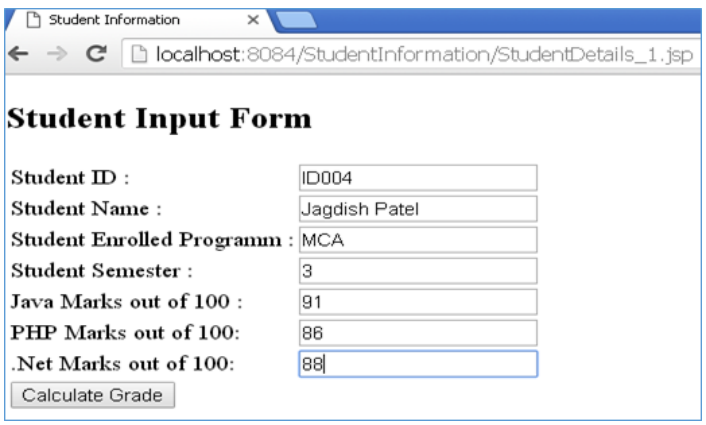


Figure 8.30: *Student marks input form*

b. Read and display the data submitted through the form by setting and getting the bean properties **`DisplayDetails_2.jsp`**

```
<!DOCTYPE html>
<html>
<head>
<title>Student Information</title>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
</head>
<body>
<h2>Student Details </h2>
<jsp:useBean id="s1" class="studentinformation.StudentBean"/>
<jsp:setProperty name="s1" property="id" />
<jsp:setProperty name="s1" property="name" />
<jsp:setProperty name="s1" property="program" />
<jsp:setProperty name="s1" property="semester" />
<jsp:setProperty name="s1" property="marks1" />
<jsp:setProperty name="s1" property="marks2" />
<jsp:setProperty name="s1" property="marks3" />

<table>
<tr><td>Student ID :</td><td><input type="text" name="id" required/><br/></td></tr>
<tr><td>Student Name :</td><td><input type="text" name="name" required/><br/></td></tr>
<tr><td>Student Enrolled Programm :</td><td><input type="text" name="program" required/><br/></td></tr>
<tr><td>Student Semester :</td><td><input type="text" name="semester" required/><br/></td></tr>
<tr><td> Java Marks out of 100 :</td><td><input type="text" name="marks1" required/><br/></td></tr>
<tr><td>PHP Marks  out of 100: </td><td><input type="text" name="marks2" required/><br/></td></tr>
<tr><td>.Net Marks  out of 100:</td><td><input type="text" name="marks3" required/><br/></td></tr>
<tr><td><input type="submit" value="Calculate Grade"/></td></tr>
</table>
</form>
</body>
</html>
```

Figure 8.31: *Display data submitted through the form by setting and getting the bean properties DisplayDetails_2.jsp*

The following output will be generated by the preceding code:

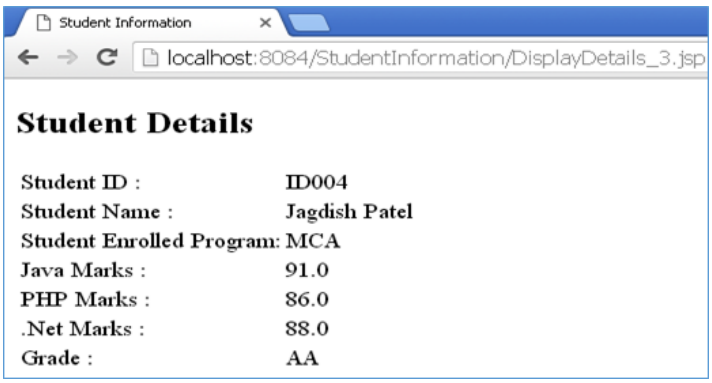


Figure 8.32: *Display of marks and grade*

Assigning all properties of bean with input parameter values

Associating a property with a request parameter saves you the bother of performing conversions for many of the simple built-in types. JSP lets you take the process one step further by associating all properties with identically named request parameters. All you have to do is to supply "*" for the property parameter. We can replace the code as follows:

```
01. <jsp:useBean id="s1" class="studentinformation.StudentBean"/>
02. <jsp:setProperty name="s1" property="id" />
03. <jsp:setProperty name="s1" property="name" />
04. <jsp:setProperty name="s1" property="program" />
05. <jsp:setProperty name="s1" property="semester" />
06. <jsp:setProperty name="s1" property="marks1" />
07. <jsp:setProperty name="s1" property="marks2" />
08. <jsp:setProperty name="s1" property="marks3" />
09.
```

Figure 8.33: *Replacing the code*

With the following code:

```
01. <jsp:useBean id="s1" class="studentinformation.StudentBean"/>
02. <jsp:setProperty name="s1" property="*" />
```

Figure 8.34: *Replace the code with this*

This approach lets you define simple *form beans* whose properties correspond to the request parameters and get populated automatically. Although this approach is simple, following three things should be taken care of:

- No action is taken when an input parameter is missing.
- Automatic type conversion does not guard against illegal values as effectively as manual type conversion.
- Bean property names and request parameters are case sensitive.

Sharing Beans in different scopes

The availability of a JSP object for use from a particular place of the application is defined as the scope of that JSP object. Every object created in a JSP page will have a scope. Object scope in JSP is segregated into four parts: page, request, session, and application.

We have used **jsp:useBean** to create objects, which are bound to local variables in the **_jspService** method. With the scope attribute, we can change the behavior of bounding the local variables depending on one of the four locations.

When you use scope, the system first looks for an existing bean of the specified name in the designated location. Only when the system fails to find a preexisting bean does it create a new one. The scope attribute has four possible values:

- **Page scope**

  ```
  <jsp:useBean ... scope="page" />
  ```

  - This is the default value; we get the same behavior if we omit the scope attribute entirely.
  - The page scope indicates that in addition to being bound to a local variable, the bean object should be placed in the **PageContext** object for the duration of the current request.
  - Storing the object there means that servlet code can access it by calling **getAttribute** on the predefined **pageContext** variable.
  - Since every page and every request has a different **PageContext** object, using **scope="page"** (or omitting scope) indicates that the bean is not shared and thus, a new bean will be created for each request.

- **Request scope**

  ```
  <jsp:useBean ... scope="request" />
  ```

  - This value signifies that in addition to being bound to a local variable, the bean object should be placed in the **HttpServletRequest** object for the duration of the current request, where it is available by means of the **getAttribute** method.
  - Although at first glance, it appears that this scope also results in unshared beans, two JSP pages or a JSP page and a servlet will share request objects when you use **jsp:include**, **jsp:forward**, or the include or forward methods of **RequestDispatcher**.
  - Storing values in the request object is common when the MVC (Model 2) architecture is used.

- **Session scope**

  ```
  <jsp:useBean ... scope="session" />
  ```

  - This value means that in addition to being bound to a local variable, the bean will be stored in the **HttpSession** object associated with the current request, where it can be retrieved with **getAttribute**.

  - Thus, this scope lets JSP pages easily perform session tracking.

- **Application scope**

```
<jsp:useBean ... scope="application" />
```

  - This value means that in addition to being bound to a local variable, the bean will be stored in the **`ServletContext`** available through the predefined application variable or by a call to **`getServletContext`**.
  - The **`ServletContext`** is shared by all servlets and JSP pages in the web application.
  - Values in the **`ServletContext`** can be retrieved with the **`getAttribute`** method.

When we store beans in different scopes, we have to be sure to use different names for each bean. Otherwise, servers can get confused and retrieve the incorrect bean.

Creating beans conditionally

- To make bean sharing more convenient, you can conditionally evaluate bean-related elements in two situations:
  - First, a **`jsp:useBean`** element results in a new bean being instantiated only if no bean with the same **`id`** and scope can be found. If a bean with the same **`id`** and scope is found, the preexisting bean is simply bound to the variable referenced by id.
  - Second, instead of **`<jsp:useBean ... />`**, you can use **`<jsp:useBean ...>statements</jsp:useBean>`**.
- The point of using the second form is that the statements between the **`jsp:useBean`** start and end tags are executed only if a new bean is created, not if an existing bean is used.
- As **`jsp:useBean`** invokes the default (zero-argument) constructor, you frequently need to modify the properties after the bean is created.
  - **Example 1**: We take a bean, namely, **`PageAccessCount`**, which has two properties: count and **`pagename`**. The **`count`** property records cumulative access counts to any of a set of related pages and thus, should be executed for all requests. The **`pagename`** property stores the name of the first page that was accessed and thus, should be executed only by the page that is first accessed. To fulfill the purpose of these two properties, we place the **`jsp:setProperty`** statement that updates the **`count`** property in unconditional code, and we place the **`jsp:setProperty`** statement for **`pagename`** between the start and end tags of **`jsp:useBean`**.

a. Standard Deployment Descriptor **`web.xml`**

```
<?xml version="1.0" encoding="UTF-8"?>

<web-app xmlns="http://xmlns.jcp.org/xml/ns/javaee"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://xmlns.jcp.org/xml/ns/javaee http://xmlns.jcp.org/xml/ns/javaee/web-app_3_1.xsd"
  version="3.1">
<session-config>
<session-timeout>
           30
</session-timeout>
</session-config>
<welcome-file-list>
<welcome-file>index.jsp</welcome-file>
</welcome-file-list>
</web-app>

```

Figure 8.35: *Standard deployment descriptor web.xml*

b. Bean class **`PageAccessCount`** with properties count a **`pagename`**

```
package src;

public class PageAccessCount
{
    static private int count=0;
private String pagename;

    public int getCount() {
        return count;
    }

    public void setCount(int i) {
        count = count + i;
    }

    public String getPagename() {
        return pagename;
    }

    public void setPagename(String pagename) {
this.pagename = pagename;
    }
}
```

Figure 8.36*: Bean class PageAccessCount with properties count a pagename*

c. Index page of application **index.jsp**

```
<%@page contentType="text/html" pageEncoding="UTF-8"%>
<!DOCTYPE html>
<html>
<head>
<meta http-equiv="Content-Type" content="text/html; charset=UTF-8">
<title>JSP Page</title>
</head>
<body>
<a href="Page1.jsp?fp=Page1.jsp" > First Page  </a><br/>
<a href="Page2.jsp?fp=Page2.jsp" > Second Page </a><br/>
<a href="Page3.jsp?fp=Page3.jsp" > Third Page </a><br/>
</body>
</html>

```

Figure 8.37: *Index page of application index.jsp*

d. First page content **Page1.jsp**

```
<%@page contentType="text/html" pageEncoding="UTF-8"%>
<!DOCTYPE html>
<html>
<head>
<meta http-equiv="Content-Type" content="text/html; charset=UTF-8">
<title>JSP Page</title>
</head>
<body>
<h1>Hello from page 1</h1>


<h3>
<jsp:useBean id="c" class="src.PageAccessCount" scope="application">
<jsp:setProperty name="c" property="pagename" value='<%=request.getParameter("fp")%>'/>
</jsp:useBean>

The <jsp:getProperty name="c" property="pagename"/> page is accessed first. All the pages have been accessed <jsp:getProperty name="c" property="count"/> times.
<jsp:setProperty name="c" property="count" value="1"/>
</h3>
</body>
</html>

```

Figure 8.38: *First page content Page1.jsp*

e. Second page content **`Page2.jsp`**

```
<%@page contentType="text/html" pageEncoding="UTF-8"%>
<!DOCTYPE html>
<html>
<head>
<meta http-equiv="Content-Type" content="text/html; charset=UTF-8">
<title>JSP Page</title>
</head>
<body>
<h1>Hello from page 2.</h1>
<h3>
<jsp:useBean id="c" class="src.PageAccessCount" scope="application">
<jsp:setProperty name="c" property="pagename" value='<%=request.getParameter("fp")%>'/>
</jsp:useBean>

The <jsp:getProperty name="c" property="pagename"/> page is accessed first. All the pages have been accessed <jsp:getProperty name="c" property="count"/> times.
<jsp:setProperty name="c" property="count" value="1"/>
</h3>
</body>
</html>
```

***Figure 8.39**: Second page content Page2.jsp*

f. Third page content **`Page3.jsp`**

```
<%@page contentType="text/html" pageEncoding="UTF-8"%>
<!DOCTYPE html>
<html>
<head>
<meta http-equiv="Content-Type" content="text/html; charset=UTF-8">
<title>JSP Page</title>
</head>
<body>
<h1>Hello from page 3.</h1>
<h3>
<jsp:useBean id="c" class="src.PageAccessCount" scope="application">
<jsp:setProperty name="c" property="pagename"
value='<%=request.getParameter("fp")%>'/>
</jsp:useBean>

The <jsp:getProperty name="c" property="pagename"/> page is accessed first. All the pages have been accessed <jsp:getProperty name="c" property="count"/> times.
<jsp:setProperty name="c" property="count" value="1"/>
</h3>
</body>
</html>
```

***Figure 8.40:** Third page content Page3.jsp*

The following output will be generated by the preceding code:

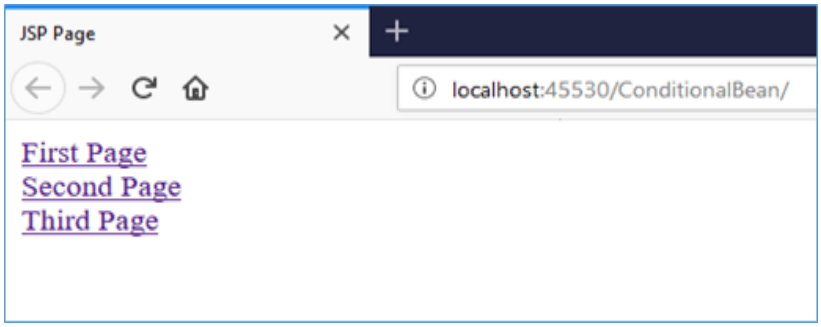


Figure 8.41: *Index page of application*

g. Page 3 accessed for the first time

Figure 8.42: *Page 3 access for first time*

h. Page 1 accessed for the first time

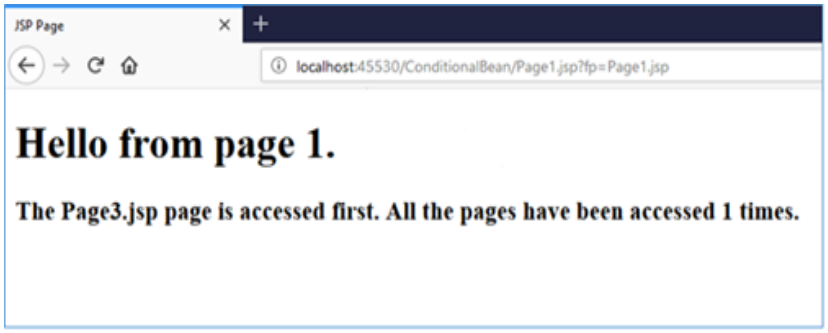


Figure 8.43*: Page 1 access for first time*

i. Page 2 accessed for first time and refreshed, so the count of all the pages has increased:

Figure 8.44: *Page 2 accessed for the first time and refreshed, so the count of all the pages has increased*

In this example, the page accessed for the first time is set to the property of **`pagename`**, and the first page that executes creates the object and then the other pages use this object. So, when the different pages are accessed, the counter increments, but the page first page name remains the same as it is set only once and shared in the application scope.

Unshared bean in Page scope

Page scope means the JSP object can be accessed only from within the page where it was created. The default scope for JSP objects created using the **`<jsp:useBean>`** tag is page. JSP implicit objects **`out`**, **`exception`**, **`response`**, **`pageContext`**, **`config`** and page have **`page`** scope.

Example 1: In this example, we create a bean class **`MusicBean`** with properties **`musicgenre`** and **`artistname`**. We create a JSP page to instantiate bean and access it in a page scope. The value of bean is preserved only for the current request; it is not preserved between multiple requests.

a. Index page of application, **`index.html`**

```
<html>
<head>
<title>TODO supply a title</title>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
</head>
<body>
<h2>
            Page Scope sharing <br/></h2>
<form action="pagescopesharing.jsp">
                Music Genere <input type="text" name="musicgenre"/><br/>
                Artist Name <input type="text" name="artistname"/><br/>
<input type="submit" value="Submit"/><br/>
</form>

</body>
</html>
```

Figure 8.45: *Index page of application, index.html*

b. Bean class **`MusicBean.java`** with two properties **`musicgenre`** and **`artistname`**

```
01. package src;
02. public class MusicBean
03. {
04.     private String musicgenre="Pop Music";
05.     private String artistname="Michael Jackson";
06.
07.     public String getMusicgenre() {
08.         return musicgenre;
09.     }
10.
11.     public void setMusicgenre(String musicgenre) {
12.          this.musicgenre = musicgenre;
13.     }
14.
15.     public String getArtistname() {
16.         return artistname;
17.     }
18.
19.     public void setArtistname(String artistname) {
20.         this.artistname = artistname;
21.     }
22.
23. }
```

***Figure 8.46**: Bean class MusicBean.java with two properties: musicgenre and artistname*

c. JSP page to set the property and retrieve the bean properties **`pagescopesharing.jsp`**

```
01. <%@page contentType="text/html" pageEncoding="UTF-8"%>
02. <!DOCTYPE html>
03. <html>
04. <head>
05. <meta http-equiv="Content-Type" content="text/html; charset=UTF-8">
06. <title>JSP Page</title>
07. </head>
08. <body>
09. <h1>Page Scope Sharing</h1>
10. <jsp:useBean id="pagebean" class="src.MusicBean" scope="page"/>
11.         This is page scope example.
12.         The bean value is available for the current request.
13.
14. <jsp:setProperty name="pagebean" property="*"/>
15.
16. <h3>
17.             Music Genre :<jsp:getProperty name="pagebean" property="musicgenre"/><br/>
18.             Artist Name :<jsp:getProperty name="pagebean" property="artistname"/><br/>
19. </h3>
20. </body>
21. </html>
```

***Figure 8.47**: JSP page to set the property and retrieve the bean properties pagescopesharing.jsp*

The following output will be generated by the preceding code:

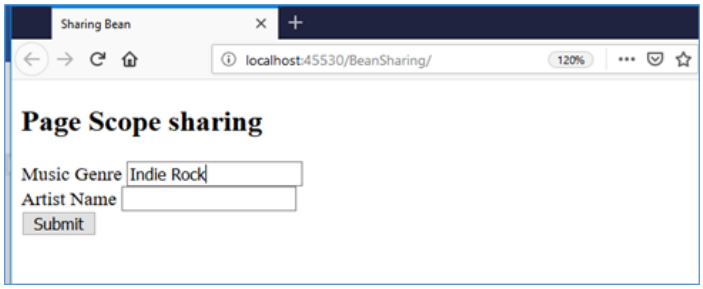


***Figure 8.48**: Index page to input the properties; here, we input music genre*

JSP page to set and retrieve the bean properties

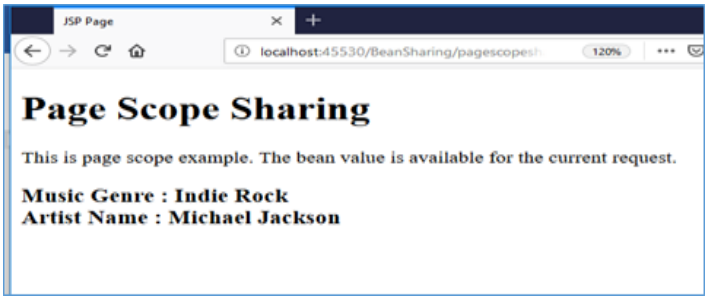


***Figure 8.49**: JSP page to set and retrieve the bean properties*

Index page without input values

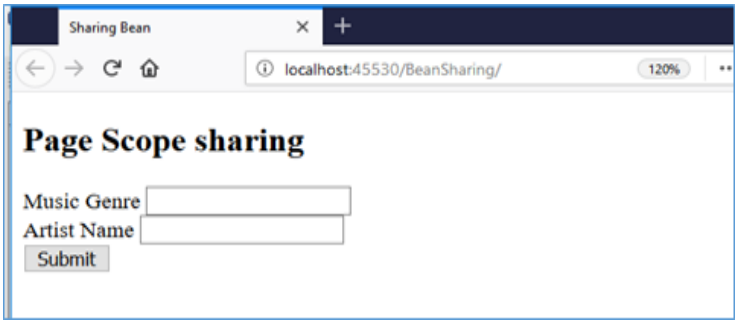


***Figure 8.50**: Index page without input values*

Default values are displayed on the page

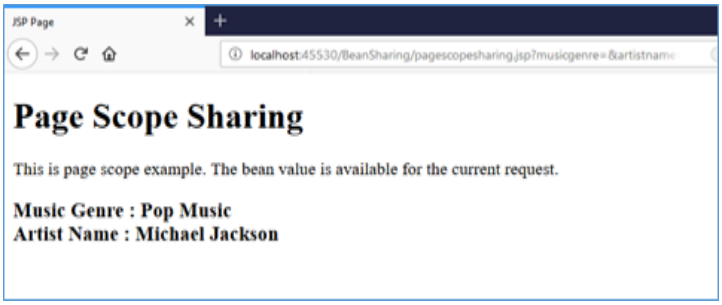


Figure 8.51: *Default values are displayed in the page*

In JSP page, the values between the request is not preserved; the default value set is displayed.

Here, we show that the value placed in page scope is not shared between the request; it is only available for the current request. As the default scope is page, skipping the scope attribute in the **`jsp:useBean`** will set the scope to page.

Sharing bean in Request scope

A JSP object created using the **`request`** scope can be accessed from any page that serves that request. More than one page can serve a single request. The JSP object will be bound to the **`request`** object. Implicit object request has the **`request`** scope.

Example 1: In this example, we use the same bean class **`MusicBean`** with properties **`musicgenre`** and **`artistname`**. We create a JSP page to instantiate bean and access it in a request scope. The value of bean is preserved among the two pages: the main page and the intermediator included page; the value of bean is not preserved between multiple page requests. We use the same bean class listed earlier, so the code for it is not included here.

a. Index page of application, **`index.html`**

```
<!DOCTYPE html>
<html>
<head>
<title>Sharing Bean</title>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
</head>
<body>
<h2>
            Request Scope sharing <br/></h2>
<form action="requestscopesharing.jsp">
                Music Genre <input type="text" name="musicgenre"/><br/>
                Artist Name <input type="text" name="artistname"/><br/>
<input type="submit" value="Submit"/><br/>
</form>

</body>
</html>

```

Figure 8.52: *Index page of application, index.html*

b. JSP page to create and share the object in request scope with the secondary page included

```
<%@page contentType="text/html" pageEncoding="UTF-8"%>
<!DOCTYPE html>
<html>
<head>
<meta http-equiv="Content-Type" content="text/html; charset=UTF-8">
<title>JSP Page</title>
</head>
<body>
<h1>Request Scope Sharing</h1>
<h2>Page 1</h2>
<jsp:useBean id="requestbean" class="src.MusicBean" scope="request"/>
        This is request scope example.
        The bean value is available for the current request between the two pages.
<jsp:setProperty name="requestbean" property="*"/>
<h3>
           Music Genre :<jsp:getProperty name="requestbean" property="musicgenre"/><br/>
           Artist Name :<jsp:getProperty name="requestbean" property="artistname"/><br/>
</h3>
<hr/>
<h2>Page 2</h2>
<jsp:include page="page2.jsp" />
</body>
</html>
```

Figure 8.53: *JSP page to create and share the object in request scope*

c. Secondary JSP page where bean is shared for the current request

```
01. The bean value is available for the current request between the two pages.
02. <jsp:setProperty name="requestbean" property="*"/>
03. <h3>
04. Music Genre :<jsp:getProperty name="requestbean" property="musicgenre"/><br/>
05. Artist Name :<jsp:getProperty name="requestbean" property="artistname"/><br/>
06. </h3>
```

Figure 8.54: *Secondary JSP page where bean is shared for the current request*

The following output will be generated by the preceding code:

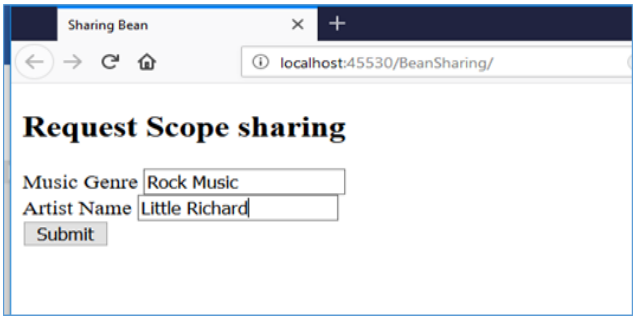


Figure 8.55: *Index page with values set for both the properties*

The following output will be generated by the preceding code:

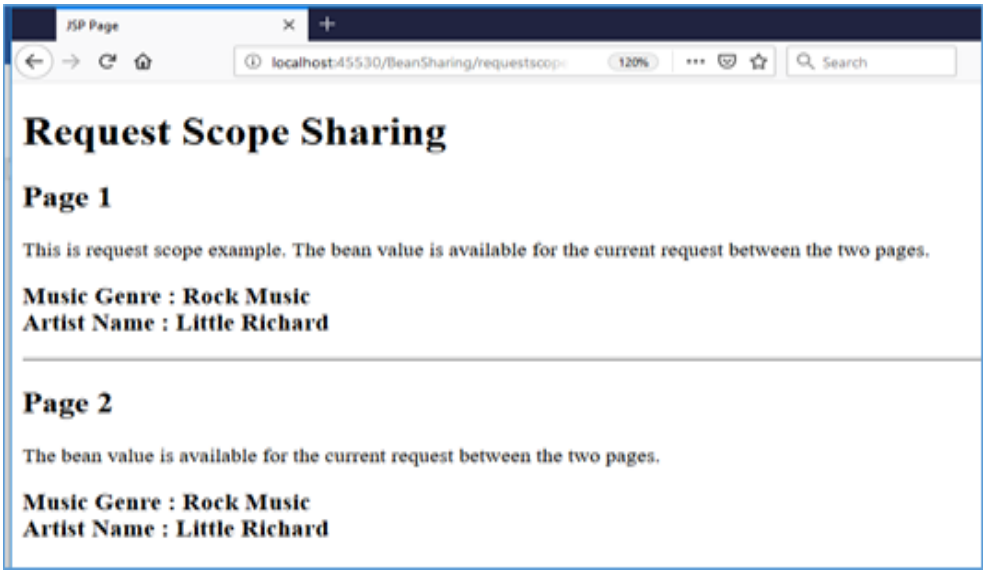


Figure 8.56*: JSP page sharing values between both the pages*

Index page with null values set for both the properties:

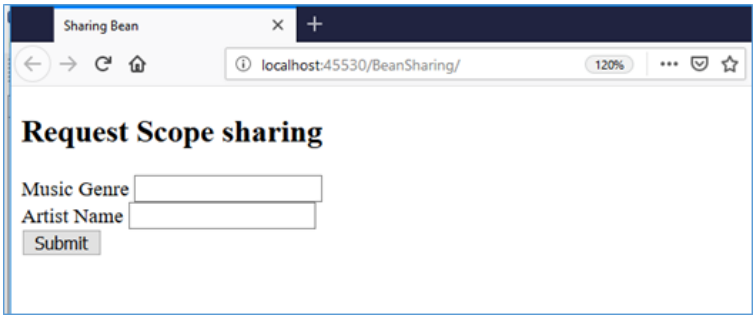


Figure 8.57: *Index page with null values set for both the properties*

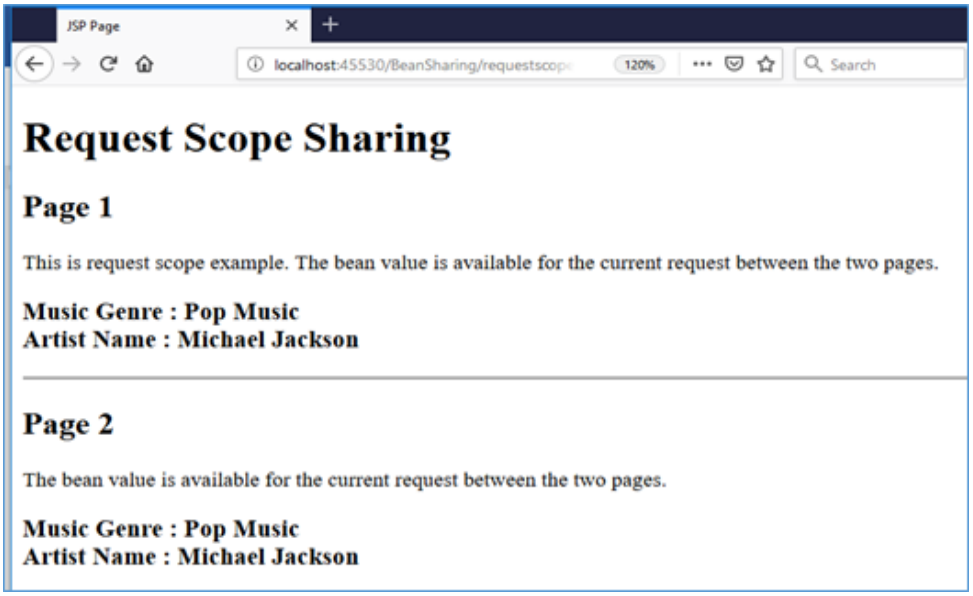


Figure 8.58: *JSP page not holding the values between the request among both pages*

Sharing bean in Session scope

The **`session`** scope means the JSP object is accessible from pages that belong to the same session where it was created. The JSP object that is created using the session scope is bound to the session object. Implicit object session has the **`session`** scope.

Example 1: In this example, we use the same bean class **`MusicBean`** with properties **`musicgenre`** and **`artistname`**. We create a JSP page to instantiate bean and access it in a session scope. The value of bean is set and retrieved in the page, and it is preserved between multiple page requests for a given session. We use the same bean class listed earlier, so the code for it is not included here.

a. Index page of the application, **index.html**

```
<!DOCTYPE html>
<html>
<head>
<title>Sharing Bean</title>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
</head>
<body>
<h2>
            Session Scope sharing <br/></h2>
<form action="sessionscopesharing.jsp">
                Music Genre <input type="text" name="musicgenre"/><br/>
                Artist Name <input type="text" name="artistname"/><br/>
<input type="submit" value="Submit"/><br/>
</form>

</body>
</html>
```

Figure 8.59: *Index page of the application, index.html*

b. JSP page to create and display the bean values during the session

```
<%@page contentType="text/html" pageEncoding="UTF-8"%>
<!DOCTYPE html>
<html>
<head>
<meta http-equiv="Content-Type" content="text/html; charset=UTF-8">
<title>JSP Page</title>
</head>
<body>
<h1>Session Scope Sharing</h1>

<jsp:useBean id="sessionbean" class="src.MusicBean" scope="session"/>
        This is session scope example.
        The bean value is available between multiple requests.
<jsp:setProperty name="sessionbean" property="*"/>
<h3>
            Music Genre :<jsp:getProperty name="sessionbean" property="musicgenre"/><br/>
            Artist Name :<jsp:getProperty name="sessionbean" property="artistname"/><br/>
</h3>

</body>
</html>
```

Figure 8.60: *JSP page to create and display the bean values during the session*

The following output will be generated by the preceding code:

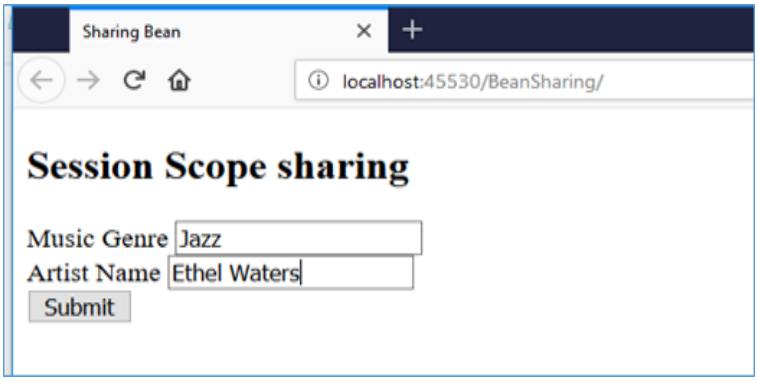


Figure 8.61*: Index page to input the two values*

JSP page to display bean properties

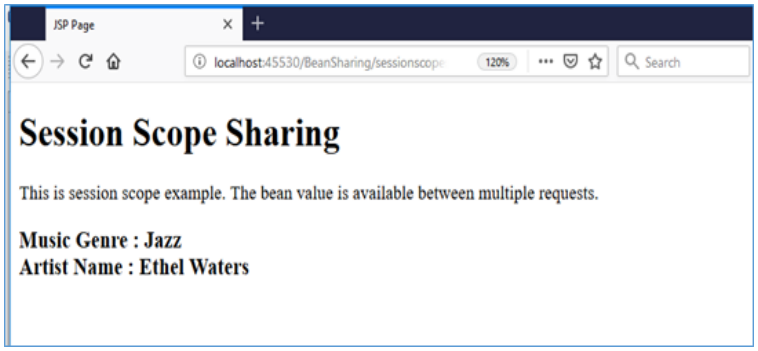


Figure 8.62: *JSP page to display bean properties*

Index page with null values set for both the properties

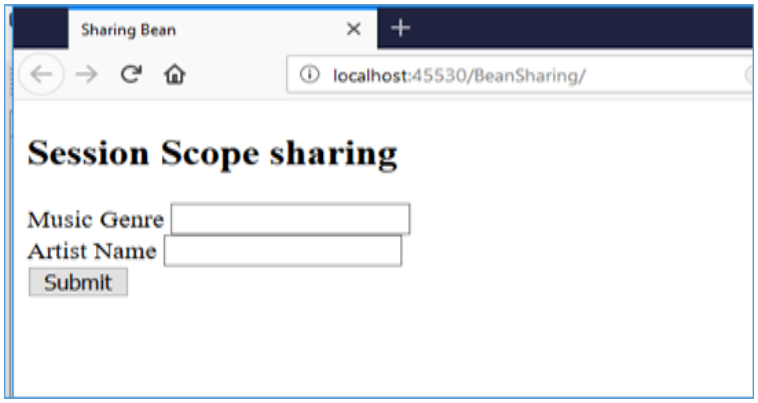


Figure 8.63: *Index page with null values set for both the properties*

JSP page showing the values when null parameters are passed; the value is preserved between requests

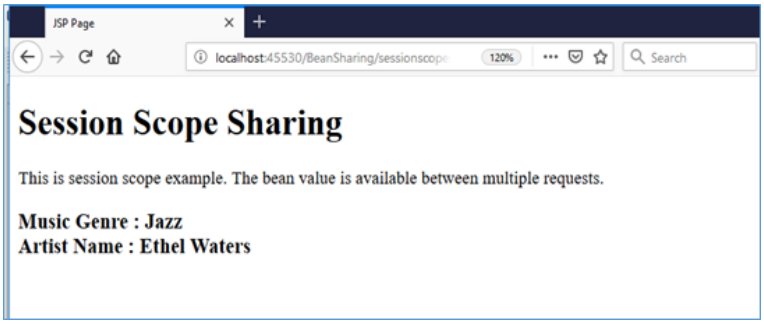


***Figure 8.64**: JSP page showing the values when null parameters are passed; the value is preserved between requests*

In this example, the values between the page are preserved between multiple page requests, until the browser is changed or session timeout has reached.

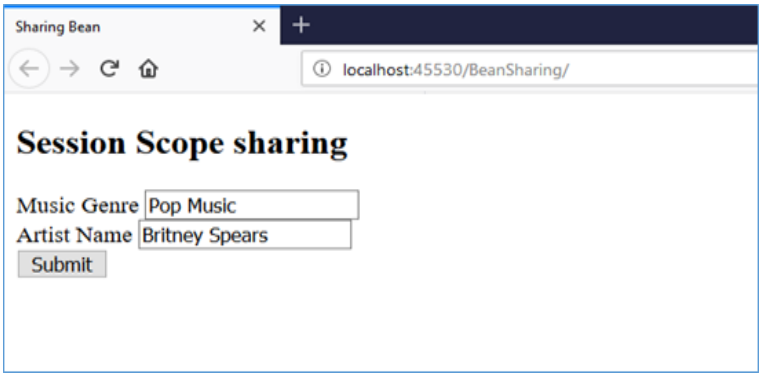


***Figure 8.65:** Index page to input two values*

The values shared and displayed in the JSP page

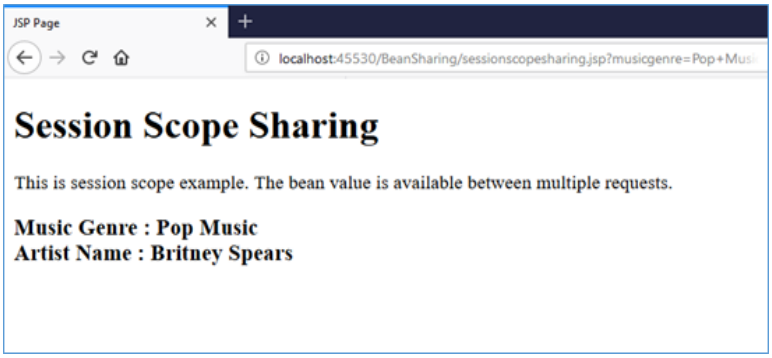


***Figure 8.66**: The values shared and displayed in the JSP page*

Input null values in the index page

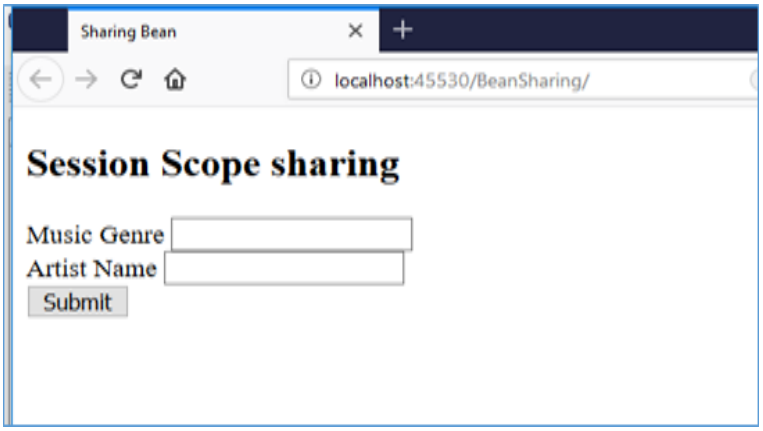


Figure 8.67: *Input null values in the index page*

The values preserved in the page between requests in the same browser (Mozilla Firefox)

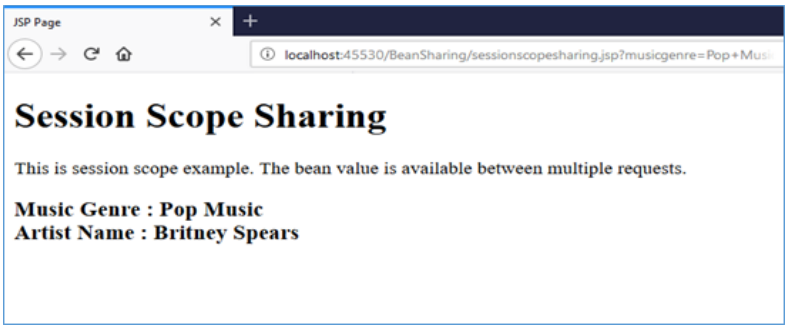


Figure 8.68: *The values preserved in the page between requests in the same browser (Mozilla Firefox)*

The values lost in the new browser (Chrome) as browser was changed and so, the session is over

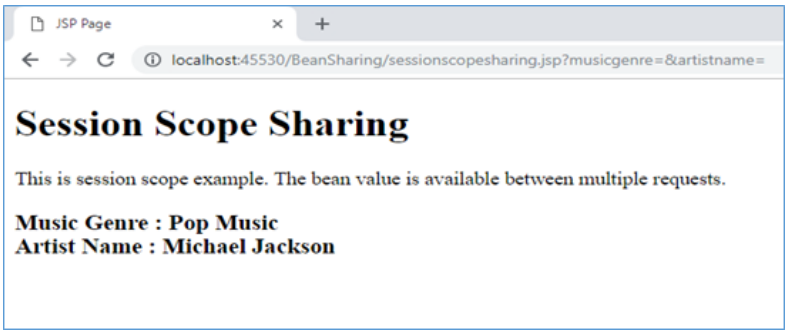


Figure 8.69: *The values lost in the new browser (Chrome) as browser was changed and so, the session is over*

Sharing bean in application scope

A JSP object created using the **`application`** scope can be accessed from any pages across the application. The JSP object is bound to the application object. Implicit object application has the **`application`** scope.

Example 1: In this example, we use the same bean class **`MusicBean`** with properties **`musicgenre`** and **`artistname`**. We create a JSP page to instantiate bean and access it in an application scope. The value of bean is set and retrieved in the page, and it is preserved between multiple page requests for a given session and throughout the application. This scope is the broadest scope. We use the same bean class listed above, so the code for it is not included here:

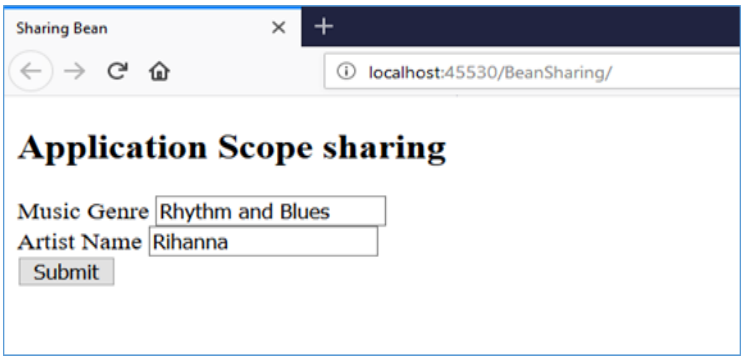


Figure 8.70: *Index page to input the two values of bean class*

JSP page to display values of bean

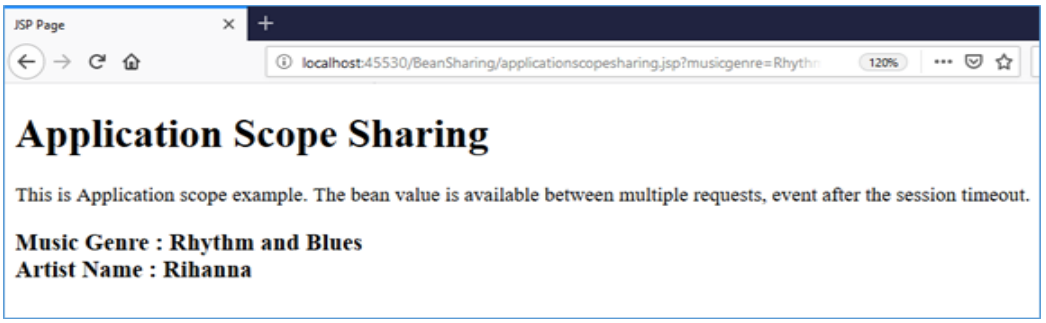


Figure 8.71: *JSP page to display values of bean*

JSP page to input null values

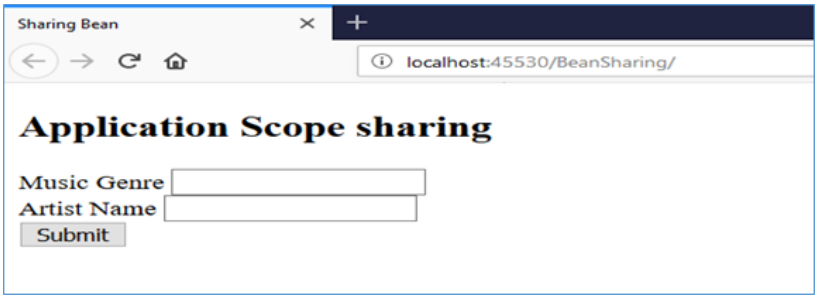


Figure 8.72: *JSP page to input null values*

JSP page preserves values between multiple requests in the same browser (Mozilla Firefox)

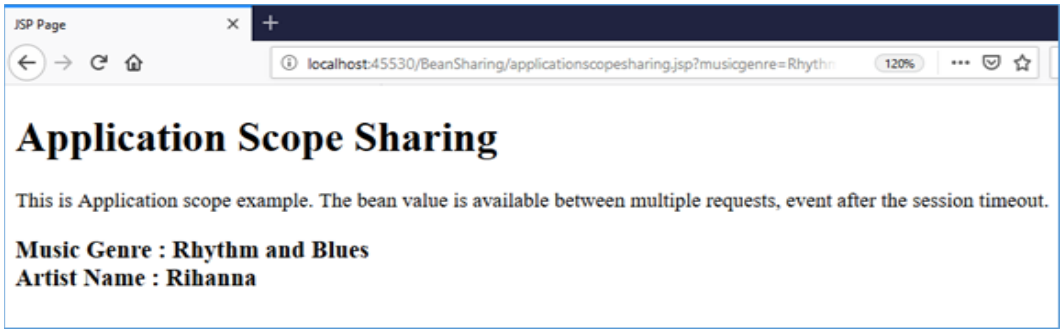


Figure 8.73*: JSP page preserves values between multiple requests in the same browser*

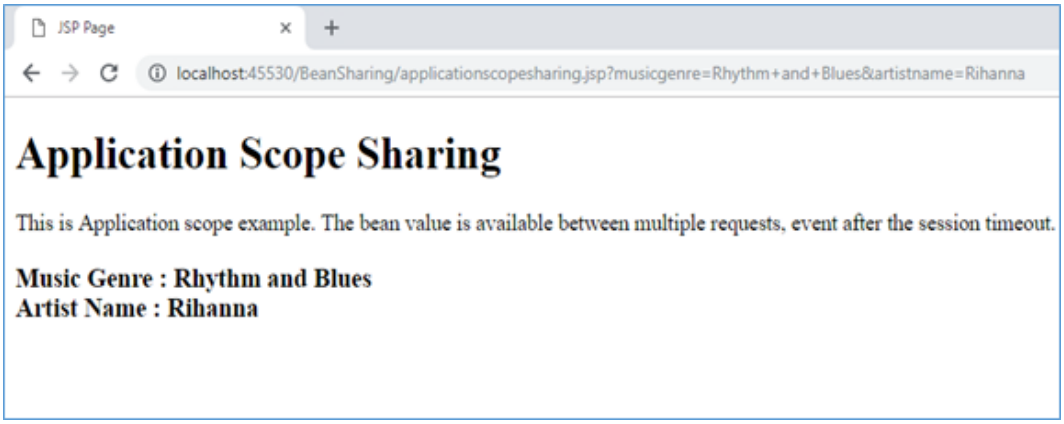


Figure 8.74: *JSP page preserves values in the different browser (Chrome)*

Index page of the application

```
<!DOCTYPE html>
<html>
<head>
<title>Sharing Bean</title>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
</head>
<body>
<h2>
                Application Scope sharing <br/></h2>
<form action="applicationscopesharing.jsp">
                    Music Genre <input type="text" name="musicgenre"/><br/>
                    Artist Name <input type="text" name="artistname"/><br/>
<input type="submit" value="Submit"/><br/>
</form>
</body>
</html>
```

***Figure 8.75**: Index page of the application*

JSP page to create and display the bean values throughout the application

```
<%@page contentType="text/html" pageEncoding="UTF-8"%>
<!DOCTYPE html>
<html>
<head>
<meta http-equiv="Content-Type" content="text/html; charset=UTF-8">
<title>JSP Page</title>
</head>
<body>
<h1>Application Scope Sharing</h1>

<jsp:useBean id="applicationbean" class="src.MusicBean" scope="application"/>
        This is Application scope example.
        The bean value is available between multiple requests, event after the session timeout.
<jsp:setProperty name="applicationbean" property="*"/>
<h3>
            Music Genre :<jsp:getProperty name="applicationbean" property="musicgenre"/>
<br/>
            Artist Name :<jsp:getProperty name="applicationbean" property="artistname"/><br/>
</h3>
</body>
</html>
```

***Figure 8.76:** JSP page to create and display the bean values throughout the application*

The bean shared in the application scope, which is placed in **`ServletContext`**, is available to all the resources throughout the application.

Introduction to expression language

JSP specification provides another convenient way (Expression Language) to minimize the use of scripting tag in JSP because writing Java code is not easy for web

designers. JSP 2.0 introduced a shorthand language for evaluating and outputting the values of Java objects that are stored in standard locations. The JSP 2.0 expression language lets you simplify the presentation layer by replacing hard-to-maintain Java scripting elements with short and readable entries.

The general syntax of expression language is **`${expression}`**.

If the given attribute is not found or expression returns null instead of throwing any exception. For arithmetic operations, EL treats null as 0 and for logical operations, EL treats null as false.

Retrieving scoped variables

In the previous sections, we covered instantiating a bean object, assigning and retrieving bean value, and sharing the bean in different scopes available. Like a bean, we can share any object or variable in the available scopes. Generally, the sharing of object will be done by servlet when we apply the MVC pattern, and the JSP page will access it.

To allow the JSP page to access the object, the servlet needs to use the **`setAttribute`** method to store the data in one of the scopes available:

- **`HttpServletRequest`**
- **`HttpSession`**
- **`ServletContext`**

The objects/variables stored in these scopes are known as *scoped variables*, and the expression language provides a short and easy way to access them.

When the JSP page needs to find the object to access from the different scopes, we need to use the **`findAttribute`** method. For example, to find an object/variable, namely, **`objectname`**, from the JSP page, we need to write **`<%=pageContext.findAttribute("objectname")%>`** or use **`jsp:useBean`**, **`<jsp:useBean id="objectname" type="somePackage.SomeClass" scope="...">`**, and then **`<%=objectname%>`** the equivalent expression language syntax is **`${objectname }`**.

The general syntax for accessing the scoped variables using expression language is as follows:

```
${scoped variable}
```

Example 1: We share three variables in the three different scopes with different names and then share variables with the same name in all the three scopes to show how to access them:

a. Index page for the application to invoke the servlet

```
<!DOCTYPE html>
<html>
<head>
<title> Expression Language </title>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
</head>
<body>
<h1><a href="ScopedVariables">Accessing Scoped variables</a></h1><br>
</body>
</html>
```

Figure 8.77: *Index page for the application to invoke the servlet*

b. Servlet code to create and share the scope variables

```
package core;
import java.io.IOException;
import java.io.PrintWriter;
import java.util.ArrayList;
import java.util.HashMap;
import javax.servlet.RequestDispatcher;
import javax.servlet.ServletContext;
import javax.servlet.ServletException;
import javax.servlet.annotation.WebServlet;
import javax.servlet.http.HttpServlet;
import javax.servlet.http.HttpServletRequest;
import javax.servlet.http.HttpServletResponse;
import javax.servlet.http.HttpSession;

@WebServlet(name = "ScopedVariables", urlPatterns = {"/ScopedVariables"})
public class ScopedVariables extends HttpServlet {

@Override
    protected void doGet(HttpServletRequest request, HttpServletResponse response)
            throws ServletException, IOException {

request.setAttribute("requestscopevariable", "Hello from Request Scope");

                HttpSession session = request.getSession();
session.setAttribute("sessionscopevariable", "Hello from Session Scope");

                ServletContext application = getServletContext();
application.setAttribute("applicationscopevariable", "Hello from Application
Scope");

request.setAttribute("commonscopevariable", "Request Scope");
session.setAttribute("commonscopevariable", "Session Scope");
application.setAttribute("commonscopevariable", "Application Scope");

                RequestDispatcher dispatcher = request.getRequestDispatcher("scopedvars.jsp");
                dispatcher.forward(request, response);
        }
    }
```

Figure 8.78: *Servlet code to create and share the scope variables*

c. JSP page to access and display the scope variables

```
<%@page contentType="text/html" pageEncoding="UTF-8"%>
<!DOCTYPE html>
<html>
<head>
<meta http-equiv="Content-Type" content="text/html; charset=UTF-8">
<title>JSP Page</title>
</head>
<body>
<h1><center> Accessing Scoped Variables using Expression Language </center></h1>
<br/>
        Accessing three different variables from different scopes
<ul>
<li>${requestscopevariable}</li>
<li>${sessionscopevariable}</li>
<li>${applicationscopevariable}</li>
</ul>
Accessing same variable from different scopes
<ul>
<li>${commonscopevariable}</li>
</ul>
</body>
</html>
```

Figure 8.79: *JSP page to access and display the scope variables*

The following output will be generated by the preceding code:

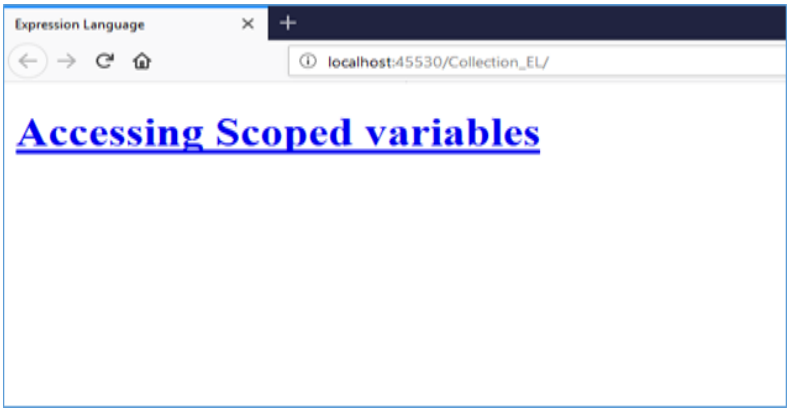


Figure 8.80: *Index page to provide link to the servlet whose role is to create and share the scope variables*

JSP page to access and display the scoped variables

***Figure 8.81**: JSP page to access and display the scoped variables*

In this example, we create a servlet to create and share three variables in different scopes using the **`setAttribute`** method, and then we write the code to navigate to the JSP page,that access and display the scoped variables using expression language.

The syntax of accessing scoped variables using expression language is easy to use, and it provides two benefits: you need not write the scope of the object/variable you are wishing to access, and you need not write core Java syntax even though you are using core Java.

Retrieving Collections

The JSP 2.0 expression language lets you access different types of collections:

- For instance, if attribute **`Name`** is a scoped variable referring to an array, **`List`**, or **`Map`**, you access an item in the collection with the syntax **`${attributeName[keyName]}`**.
- If the scoped variable is an array, the entry name is the index, and the value is obtained with **`theArray[index]`**.

  For example, if **`artistnames`** refers to an array of **`Strings`**, **`${artistnames[0]}`** would output the first item in the array.
- If the scoped variable is an object that implements the **`List`** interface, the item name is the index, and the value is obtained with **`listobject.get(index)`**.

  For example, if **`musicgenre`** refers to an **`ArrayList`**, **`${musicgenre[0]}`** would output the first item in the **`ArrayList`**.
- If the scoped variable is an object that implements the Map interface, the entry name is the key, and the value is obtained with **`mapobject.get(key)`**.

For example, if origin refers to a HashMap whose keys are genre of music and whose values are origin of the music genre, we can access the map using the syntax **`${origin["Jazz"]}`**, where **`Jazz`** is one of the music genres. It will give the value as origin of **`Jass`** music genre, that is, **`African-American`**.

Example 1: In this example, we are sharing the objects in different scopes from a servlet and then accessing them in a JSP page using the expression language syntax for scoped variables.

a. Index page for the application to invoke the servlet

```
<!DOCTYPE html>
<html>
<head>
<title> Expression Language </title>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
</head>
<body>
<h1><a href="Collections">Accessing Collections </a></h1><br>
</body>
</html>
```

***Figure 8.82**: Index page for the application to invoke the servlet*

b. Servlet for creating and sharing the objects known as *scoped variables*

```
package core;
import java.io.IOException;
import java.io.PrintWriter;
import java.util.ArrayList;
import java.util.HashMap;
import javax.servlet.RequestDispatcher;
import javax.servlet.ServletException;
import javax.servlet.annotation.WebServlet;
import javax.servlet.http.HttpServlet;
import javax.servlet.http.HttpServletRequest;
import javax.servlet.http.HttpServletResponse;
@WebServlet(name = "Collections", urlPatterns = {"/Collections"})
public class Collections extends HttpServlet {
    @Override
    protected void doGet(HttpServletRequest request, HttpServletResponse response)
            throws ServletException, IOException {
String[] artistnames = {"Michael Jackson", "Rihanna", "Abbey Lincoln","Little Richard"};
            ArrayList musicgenre = new ArrayList();
musicgenre.add("Pop Music");
musicgenre.add("Rhythm and Blues");
            musicgenre.add("Jazz");
            musicgenre.add("Rock");
            HashMap origin = new HashMap();
origin.put("Pop Music", "United States and United Kingdom");
origin.put("Rhythm and Blues", "African American communities");
origin.put("Jazz", " African-American ");
origin.put("Rock", " United States ");
request.setAttribute("artist", artistnames);
request.setAttribute("genre", musicgenre);
request.setAttribute("origin", origin);
            RequestDispatcher dispatcher = request.getRequestDispatcher("collections.jsp");
            dispatcher.forward(request, response);
        }
    }
```

***Figure 8.83**: Servlet for creating and sharing the objects known as "scoped variables"*

c. JSP page to access the scoped variables using expression language

```
<%@page contentType="text/html" pageEncoding="UTF-8"%>
<!DOCTYPE html>
<html>
<head>
<meta http-equiv="Content-Type" content="text/html; charset=UTF-8">
<title>JSP Page</title>
</head>
<body>
<h1><center>Accessing Collections using Expression Language</center></h1>
<br/>
<table border="3" width="500">
<tr>
<th> Artist Name </th>
<th> Music Genre </th>
<th> Origin of Music </th>
</tr>
<tr>
<td>${artist[0]}</td>
<td>${genre[0]}</td>
<td>${origin["Pop Music"]}</td>
</tr>
<tr>
<td>${artist[1]}</td>
<td>${genre[1]}</td>
<td>${origin["Rhythm and Blues"]}</td>
</tr>
<tr>
<td>${artist[2]}</td>
<td>${genre[2]}</td>
<td>${origin["Jazz"]}</td>
</tr>
<tr>
<td>${artist[3]}</td>
<td>${genre[3]}</td>
<td>${origin["Rock"]}</td>
</tr>
</table>
</body>
</html>
```

***Figure 8.84**: JSP page to access the scoped variables using expression language*

The following output will be generated by the preceding code:

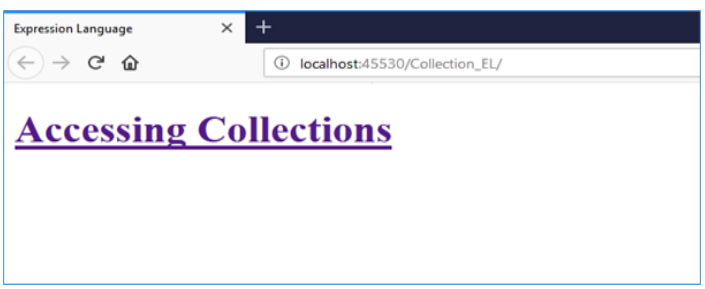


***Figure 8.85:** Index page to provide the link to invoke the servlet to create and share the scoped variables*

JSP page to access and display the scoped variables

***Figure 8.86**: JSP page to access and display the scoped variables*

In this example, we created a servlet whose role is to create objects of String, ArrayList and HashMap and add values to them. Then, we shared the objects in the request scope using the setAttribute value, which uses a key and value pair. At the end, we wrote to code to navigate to the JSP page who role is to access and display the scoped variables. In the JSP page, we used the key name along with the index number to access the String and ArrayList objects; for HashMap objects, we need to use the key name used in setAttribute along with the name of key we have used to create the map objects.

Using Implicit objects

Implicit objects are the Java objects that the **`JSPContainer`** makes available to the developers in each page, and the developer can call them directly without being explicitly declared. JSP Implicit Objects are also called pre-defined variables. Using expression language, we can access the following implicit variables:

- **pageContext**: The **`pageContext`** object refers to the **`PageContext`** of the current page. The **`PageContext`** class, in turn, has request, response, session, out, and **`servletContext`** properties (i.e., the **`getRequest`**, **`getResponse`**, **`getSession`**, **`getOut`**, and **`getServletContext`** methods). To access and display the current session ID, we can write the following code:

  ```
  Current Session ID :${pageContext.session.id}
  ```

- **param and paramValues**: These objects allow us to access the request parameter value using param and the array of request parameter values using **`paramValues`**. To read a request parameter, namely, **`firstname`**, we can write the code as follows:

  ```
  First Name : ${param.firstname}
  ```

- **header and headerValues**: These objects allows us to access the complete HTTP request header values. We can use a dot notation or **[]** to access the properties. The dot notation cannot be used when the value after the dot would be a constant value and not a name of property. To access the **Accept** header, you could use either **${header.Accept}** or **${header["Accept"]}**. But to access the **Accept-Encoding** header, you must use **${header["Accept-Encoding"]}**. To read a request header, namely, **user-agent**, we can write the code as follows:

```
User Agent :${header["user-agent"]}
```

- **Cookie**: This object allows you to refer to the incoming cookies. To access the value, we can use the **value** property and the **getValue** method of the **Cookie** class. To access the cookie named **mycookie**, we can write the code as follows; it will display the value of cookie if found or will display nothing if the cookie not found.

```
Cookie Value :${cookie.mycookie.value}
```

- **pageScope, requestScope, sessionScope, and applicationScope**: These objects allow us to specify where the system should search for the scoped variables. For example, with **${commonscopevariable}**, the system searches for **commonscopevariable** in the **PageContext**, **HttpServletRequest**, **HttpSession**, and **ServletContext**, returning the first match it finds. But if write **${sessionScope.commonscopevariable}**, the system searches only in the **HttpSession** scope. To access the scoped variable **commonscopevariable** from **ServletContext**, we can write the code as follows:

```
CommonScopedVariable:${applicationScope.commonscopevariable}
```

Example 1: In this example, we create a JSP page that reads and displays the **requestheaders**

a. Index page to provide link for the JSP page

```
01. <!DOCTYPE html>
02. <html>
03. <head>
04. <title>Expression Language</title>
05. <meta charset="UTF-8">
06. <meta name="viewport" content="width=device-width, initial-scale=1.0">
07. </head>
08. <body>
09. <h1><a href="implicitobjects.jsp">Accessing implicit objects</a></h1><br>
10. </body>
11. </html>
```

Figure 8.87: *Index page to provide link for the JSP page*

b. JSP page to access various request headers

```
<%@page contentType="text/html" pageEncoding="UTF-8"%>
<!DOCTYPE html>
<html>
<head>
<meta http-equiv="Content-Type" content="text/html; charset=UTF-8">
<title>JSP Page</title>
</head>
<body>
<h1>Accessing implicit objects</h1>
<ul>
<li><B>Request Parameter:</B> ${param.firstname}</li>
<li><B>Host Header:</B> ${header["Host"]}</li>
<li><B>Server:</B> ${pageContext.servletContext.serverInfo} </li>
</ul>
</body>
</html>
```

***Figure 8.88**: JSP page to access various request headers*

Below is the index page to invoke the JSP page.

***Figure 8.89:** Expression Language page*

The following output will be generated by the preceding code:

***Figure 8.90**: JSP page to display the request headers*

In this example, we access three request headers. The first is **`param.firstname`**, which is used to read the request parameter **`firstname`**; if the parameter is set to

some value, it displays the value, otherwise it displays nothing; note that it doesn't throw a **NullPointer** exception. The second header we use is to know the **Host** name, which shows the localhost with port number. The third header we use is to display the name of the server on which we are running the application; it can be displayed by calling the **getServerInfo** method. To access this method using core Java in JSP using scriptlet, we need to write **ServletContext application = getServletContext();application.getServerInfo();**, which is replaced by short and easy syntax of expression language.

Retrieving Bean properties

Using expression language, we can access the bean properties with very easy and short notation; the general syntax is as follows:

```
${object.propertyname}
```

For example, we have a bean class **MusicBean**, with properties **musicgenre** and **artistname**. In a servlet, we create the bean object **mbean** and share it in the request scope with the **keyname m**; to access the object in the JSP page, we can simple write **${m.musicgenre}**. Instead of expression language, if we use scriptlet, then we have to **import** package, create object, find the attribute, and then invoke the getter method for the required property. If we use **jsp:useBean**, we have to mention the scope from which we want to find the object and then use **jsp:getProperty** to access the values; using expression language syntax makes it easy to work in a simple and compact way.

When a bean class has properties that point to another bean class, we can use the dot operator multiple times to access the last property:

```
${object.propertyObject.propertyname}
```

Finally, note that the expression language lets you replace dot notation with array notation (square brackets).

So, for example, you can replace the following:

```
${name.property} with ${name["property"]}
```

The second form is rarely used with bean properties. However, it does have two advantages:

- It lets you compute the name of the property at request time. With the array notation, the value inside the brackets can itself be a variable; with dot notation the property name must be a literal value.
- The array notation lets you use values that are illegal as property names. This is of no use when accessing bean properties, but it is very useful when accessing collections or request headers.

Example 1: In this example, we take three bean classes: **`TimeTable`**, **`Faculty`** and **`CourseDetails`**; each class carries the following properties:

- The **`Faculty`** bean class contains the **`facultyId`** and **`facultyName`** properties.
- The **`CourseDetails`** bean class contains the **`programName`**, **`semesterNumber`**, **`courseId`** and **`courseName`** properties.
- The **`TimeTable`** bean class contains the **`dayofweek`**, **`dateofweek`**, **`hourofday`**, **`faculty`**, and **`course`** properties, where faculty and course are of type **`Faculty`** and **`CourseDetails`** class.

We create an input form to enter the time table details; a servlet reads the form data and sets the properties of various bean classes; and then it shares the bean object in the request scope and navigates to a JSP page that retrieves the bean properties using expression language and displays them.

a. Input form to enter the timetable details **`index.html`**

```
01. <!DOCTYPE html>
02. <html>
03. <head>
04. <title> Expression Language </title>
05. <meta charset="UTF-8">
06. <meta name="viewport" content="width=device-width, initial-scale=1.0">
07. </head>
08. <body>
09. <h1> Time Table Entry Form </h1>
10. <form action="BeanServlet" method="post">
11. <table>
12. <tr><th>Date :</th><td><input type="text" name="txtdate" /></td></tr>
13. <tr><th>Time :</th><td><input type="text" name="txthour"/></td></tr>
14. <tr><th> Program Name :</th><td><input type="text" name="txtprg"/></td></tr>
15. <tr><th>Semester :</th><td><input type="text" name="txtsem"/></td></tr>
16. <tr><th> Course ID :</th><td><input type="text" name="txtcid"/></td></tr>
17. <tr><th> Course Name :</th><td><input type="text" name="txtcname" size="50"/></td></tr>
18. <tr><th> Faculty ID :</th><td><input type="text" name="txtfid"/></td></tr>
19. <tr><th> Faculty Name: </th><td><input type="text" name="txtfname"/></td></tr>
20. <tr><td colspan="2" align="center" ><input type="submit" value="Submit"/></td></tr>
21. </table>
22. </form>
23. </body>
24. </html>
```

Figure 8.91: *Input form to enter the timetable details index.html*

b. **`Faculty`** Bean class

```
01. package core;
02. 
03. public class Faculty
04. {
05.     String facultyId,facultyName;
06. 
07.     public String getFacultyId() {
08.         return facultyId;
09.     }
10. 
11.     public String getFacultyName() {
12.         return facultyName;
13.     }
14. 
15.     public void setFacultyId(String facultyId) {
16. this.facultyId = facultyId;
17.     }
18. 
19.     public void setFacultyName(String facultyName) {
20. this.facultyName = facultyName;
21.     }
22. }
```

Figure 8.92: *Faculty Bean class*

c. **`CourseDetails`** Bean class

```
01. package core;
02. public class CourseDetails
03. {
04.     String programName,semesterNumber,courseId,courseName;
05.     public String getProgramName() {
06.         return programName;
07.     }
08.     public void setProgramName(String programName) {
09.         this.programName = programName;
10.     }
11.     public String getSemesterNumber() {
12.         return semesterNumber;
13.     }
14.     public void setSemesterNumber(String semesterNumber) {
15.         this.semesterNumber = semesterNumber;
16.     }
17.     public String getCourseId() {
18.         return courseId;
19.     }
20.     public void setCourseId(String courseId) {
21.        this.courseId = courseId;
22.     }
23.     public String getCourseName() {
24.         return courseName;
25.     }
26.     public void setCourseName(String courseName) {
27.        this.courseName = courseName;
28.     }
29. }
```

Figure 8.93*: Course Details Bean class*

d. `TimeTable` bean class

```
01. package core;
02. public class TimeTable
03. {
04.     String dayofweek,dateofweek,hourofday;
05.     Faculty faculty;
06.     CourseDetails course;
07.     public Faculty getFaculty() {
08.         return faculty;
09.     }
10.     public void setFaculty(Faculty faculty) {
11.        this.faculty = faculty;
12.     }
13.     public CourseDetails getCourse() {
14.         return course;
15.     }
16.     public void setCourse(CourseDetails course) {
17.       this.course = course;
18.     }
19.     public String getDayofweek() {
20.         return dayofweek;
21.     }
22.     public void setDayofweek(String dayofweek) {
23.        this.dayofweek = dayofweek;
24.     }
25.     public String getDateofweek() {
26.         return dateofweek;
27.     }
28.     public void setDateofweek(String dateofweek) {
29.        this.dateofweek = dateofweek;
30.     }
31.     public String getHourofday() {
32.         return hourofday;
33.     }
34.     public void setHourofday(String hourofday) {
35.        this.hourofday = hourofday;
36.     }
37. }
```

Figure 8.94: *Time Table Bean class*

e. Servlet to read the form data and set the bean properties

```
01. package core;
02. import java.io.IOException;
03. import java.util.Date;
04. import javax.servlet.RequestDispatcher;
05. import javax.servlet.ServletException;
06. import javax.servlet.http.HttpServlet;
07. import javax.servlet.http.HttpServletRequest;
08. import javax.servlet.http.HttpServletResponse;
09. public class BeanServlet extends HttpServlet {
10.     protected void processRequest(HttpServletRequest request, HttpServletResponse response) throws ServletException, IOException {
11.         Faculty f = new Faculty();
12.         CourseDetails c = new CourseDetails();
13.         TimeTable t = new TimeTable();
14.         f.setFacultyId(request.getParameter("txtfid"));
15.         f.setFacultyName(request.getParameter("txtfname"));
16.         c.setProgramName(request.getParameter("txtprg"));
17.         c.setSemesterNumber(request.getParameter("txtsem"));
18.         c.setCourseId(request.getParameter("txtcid"));
19.         c.setCourseName(request.getParameter("txtcname"));
20.         String dt = request.getParameter("txtdate");
21.         Date d = new Date(dt);
22.         String[] strDays = new String[]{"Sunday", "Monday", "Tuesday", "Wednesday", "Thursday","Friday", "Saturday"};
23.         t.setDateofweek(dt);
24.         t.setDayofweek(strDays[d.getDay()]);
25.         t.setHourofday(request.getParameter("txthour"));
26.         t.setFaculty(f);
27.         t.setCourse(c);
28.         request.setAttribute("t", t);
29.         RequestDispatcher dispatcher = request.getRequestDispatcher("beanvalues.jsp");
30.         dispatcher.forward(request, response);
31.     }
32.     @Override
33.     protected void doGet(HttpServletRequest request, HttpServletResponse response) throws ServletException, IOException {
34.                     processRequest(request, response);
35.     }
36.     @Override
37.     protected void doPost(HttpServletRequest request, HttpServletResponse response) throws ServletException, IOException {
38.                     processRequest(request, response);
39.     }
40.     @Override
41.     public String getServletInfo() {
42.         return "Short description";
43.     }
44. }
```

Figure 8.95: *Servlet to read the form data and set the bean properties*

f. Enter the details of servlet name and mapping in the standard deployment descriptor file **`web.xml`** if we don't use annotations in the servlet

```
01. <?xml version="1.0" encoding="UTF-8"?>
02. <web-app version="3.1" xmlns="http://xmlns.jcp.org/xml/ns/javaee" xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance" xsi:schemaLocation="http://xmlns.jcp.org/xml/ns/javaee http://xmlns.jcp.org/xml/ns/javaee/web
03. <servlet>
04. <servlet-name>BeanServlet</servlet-name>
05. <servlet-class>core.BeanServlet</servlet-class>
06. </servlet>
07. <servlet-mapping>
08. <servlet-name>BeanServlet</servlet-name>
09. <url-pattern>/BeanServlet</url-pattern>
10. </servlet-mapping>
11. <session-config>
12. <session-timeout>
13.         30
14. </session-timeout>
15. </session-config>
16. </web-app>
```

Figure 8.96: *Enter details of servlet name and mapping in the standard deployment descriptor file web.xml*

g. JSP page to retrieve the bean properties using expression language

```
<%@page contentType="text/html" pageEncoding="UTF-8"%>
<!DOCTYPE html>
<html>
<head>
<meta http-equiv="Content-Type" content="text/html; charset=UTF-8">
<title>JSP Page</title>
</head>
<body>
<h1>
<center>
Accessing Bean Values using Expression Language
</center>
</h1>

<br/>

<h2><center> Time Table </center></h2>

<table border="1">
<tr>
<th>Date </th>
<th>Day  </th>
<th>Hour </th>
<th>Program </th>
<th>Semester </th>
<th>Faculty ID </th>
<th>Faculty Name </th>
<th>Course ID  </th>
<th>Course Name </th>
</tr>
<tr>
<td> ${t.dateofweek} </td>
<td> ${t.dayofweek}  </td>
<td>  ${t.hourofday}   </td>
<td> ${t.course.programName}  </td>
<td>  ${t.course.semesterNumber}  </td>
<td>  ${t.faculty.facultyId}            </td>
<td>  ${t.faculty.facultyName}       </td>
<td>  ${t.course.courseId}             </td>
<td> ${t.course.courseName}         </td>
</tr>
</table>
</body>
</html>
```

***Figure 8.97**: JSP page to retrieve the bean properties using expression language*

The following output will be generated by the preceding code:

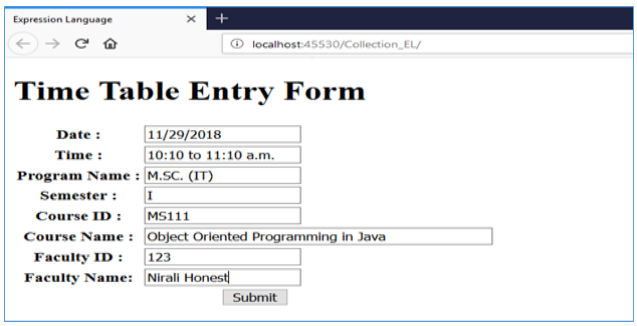


***Figure 8.98**: Input page to input the TimeTable details*

JSP page to display the bean properties using expression language

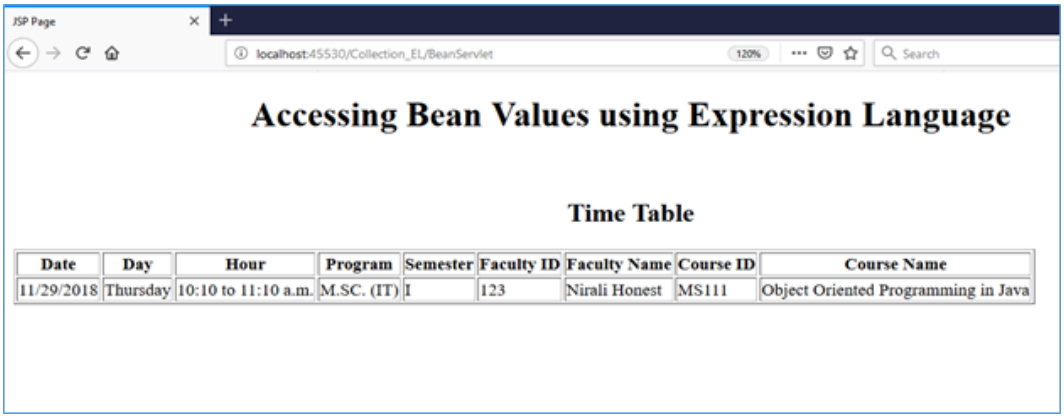


Figure 8.99: *JSP page to display the bean properties using expression language*

In this example, we used the servlet to read the request parameters and set the bean properties. After setting the beans, we share the timetable bean **`t`** in the request scope, and then navigate to the JSP page to display the bean properties using the expression language. We use **`${t.dateofweek}`** to access the **`dateofweek`** property from the **`TimeTable`** class; we use **`${t.course.programName}`** to access the **`programName`** property of the **`CourseDetails`** class, with the property course of the **`TimeTable`** class. Note that in this example, we did set the value and read the request parameter to set the **`dayofweek`** property, as we use a **`String`** array to get the day of week name, and we set the property using it.

Using Operators

Various operators are supported in expression language notation. The different operators are listed as follows:

- **Arithmetic operators**
  - + , –, *, / and div,% and mod.
- **Relational operators**
  - == and eq, != and ne, < and lt, > and gt , <= and le, >= and ge
- **Logical pperators**
  - &&, and, | |, or, !, not
  - These are the standard logical AND, OR, and NOT operators.
- **The empty operator**
  - empty

- This operator returns true if its argument is null, an empty string, an empty array, an empty Map, or an empty collection. Otherwise, it returns false.

- **The ternary operator**
  - ?:
  - This is used to test a condition and evaluate the expressions.

Example: In this example, we use the basic operators in expression language and display the expression by escaping the **$** symbol **\$** so the expression is printed and not evaluated. We use the following expressions in our example:

Expressions	Results
`${10 mod 3)}`	1
`${6.0 >= 3}`	true
`${7 > 3}`	true
`${'a' < 'b'}`	true
`${7 > (3 / 2)}`	true
`${10.0 == 10}`	true
`${(16*10) ne 300}`	true
`${'abc' gt 'aef'}`	false
`${1.6E4 + 1.2}`	16001.2
`${10 div 7}`	1.4285714285714286
`${10 mod 7}`	3
`${!empty param.firstname}`	false

Table 8.1: *Expressions used in our example*

a. Index page to invoke the JSP page with expressions

```
<!DOCTYPE html>
<html>
<head>
<title> Expression Language </title>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
</head>
<body>
<h1>
<a href="operators.jsp">Operators in Expression Language</a>
</h1>
</body>
</html>
```

Figure 8.100: *Index page to invoke the JSP page with expressions*

b. JSP page to write the expressions and evaluate it

```
<%@page contentType="text/html" pageEncoding="UTF-8"%>
<!DOCTYPE html>
<html>
<head>
<meta http-equiv="Content-Type" content="text/html; charset=UTF-8">
<title>JSP Page</title>
</head>
<body>
<h1> Operators in expression language </h1>
<table border="1">
<tr><th> Expressions </th><th> Results </th></tr>
<tr><td> \${10 mod 3)} </td><td> ${10 mod 3} </td></tr>
<tr><td> \${6.0 >= 3} </td><td> ${6.0 >= 3} </td></tr>
<tr><td> \${7 > 3} </td><td> ${7 > 3} </td></tr>
<tr><td> \${'a' < 'b'} </td><td> ${'a' < 'b'} </td></tr>
<tr><td> \${7 > (3 / 2)} </td><td> ${7 > (3 / 2)} </td></tr>
<tr><td> \${10.0 == 10} </td><td> ${10.0 == 10} </td></tr>
<tr><td> \${(16*10) ne 300} </td><td> ${(16*10) ne 300} </td></tr>
<tr><td> \${'abc' gt 'aef'} </td><td> ${'abc' gt 'aef'} </td></tr>
<tr><td> \${1.6E4 + 1.2} </td><td> ${1.6E4 + 1.2} </td></tr>
<tr><td> \${10 div 7} </td><td> ${10 div 7} </td></tr>
<tr><td> \${10 mod 7} </td><td> ${10 mod 7} </td></tr>
<tr><td> \${!empty param.firstname} </td><td> ${!empty param.firstname} </td></tr>
</table>
</body>
</html>
```

Figure 8.101: *JSP page to write the expressions and evaluate it*

The following output will be generated by the preceding code:

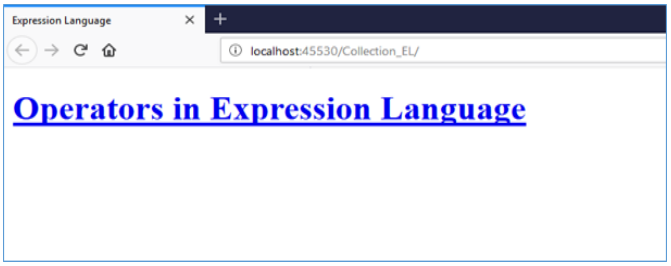


Figure 8.102: *Index page to display the link*

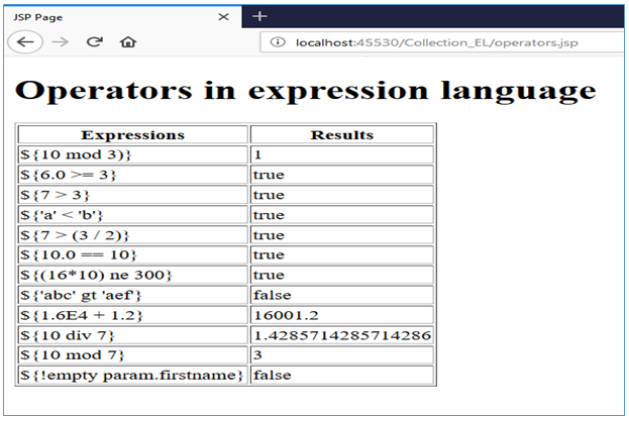


Figure 8.103*: JSP page to display and evaluate the expressions*

In the example, the first column of the table displays the expression by escaping using backslash **\${expression}**, and the second column of table displays the result after evaluating the expressions. The arithmetic operators display the numerical results based on the values, the relational operators display the Boolean results based on the values used in the expression, and the last row displays the use of empty operator by reading a request parameter **firstname** using the param implicit object. The empty operator will display **false** if the **firstname** parameter is not set to some value; otherwise, it will **show the property** displayas **true**. In the expression, we have used the **!** symbol before empty, so even though the parameter is empty, it displays **false**.

Example 2: In this example, we show the use of the ternary operator. We take a student's marks entry form and show the result with grade. We create an HTML form to input the marks of student; these marks are read and grade is calculated by servlet, which sets the bean class **Student** properties and navigates to the JSP page. The JSP page then reads and displays the properties but applies styling of color to marks and grades conditionally using the ternary operator.

a. Index page to input the student marks and submit the form to the servlet

```
01. <!DOCTYPE html>
02. <html>
03. <head>
04. <title>Ternary Operator usage</title>
05. <meta charset="UTF-8">
06. <meta name="viewport" content="width=device-width, initial-scale=1.0">
07. </head>
08. <body>
09. <h1>Student Marks Entry form</h1>
10. <form action="StudentMarks" method="post" >
11. <table>
12. <tr><td>Student ID :</td><td><input type="text" name="txtid"/></td></tr>
13. <tr><td>Student Name :</td><td><input type="text" name="txtname"/></td></tr>
14. <tr><td>Java :</td><td><input type="text" name="txtmarks1"/></td></tr>
15. <tr><td>Php :</td><td><input type="text" name="txtmarks2"/></td></tr>
16. <tr><td>.Net :</td><td><input type="text" name="txtmarks3"/></td></tr>
17. <tr><td colspan="2" align="center"><input type="submit" value="Calulate Grade"/></td></tr>
18. </table>
19. </form>
20. </body>
21. </html>
```

Figure 8.104: *Index page to input the student marks and submit the form to the servlet*

b. **`Student`** bean class

```
01. package src;
02. public class Student
03. {
04.     String studentid,studentname;
05.     double marks1,marks2,marks3;
06.     String grade;
07.
08.     public String getStudentid() {          return studentid;    }
09.     public void setStudentid(String studentid) {this.studentid = studentid;    }
10.     public String getStudentname() {         return studentname;    }
11.     public void setStudentname(String studentname) {this.studentname = studentname;    }
12.     public double getMarks1() {         return marks1;    }
13.     public void setMarks1(double marks1) { this.marks1 = marks1;    }
14.     public double getMarks2() {         return marks2;    }
15.     public void setMarks2(double marks2) {this.marks2 = marks2;    }
16.     public double getMarks3() {         return marks3;    }
17.     public void setMarks3(double marks3) {  this.marks3 = marks3;    }
18.     public String getGrade() {          return grade;    }
19.     public void setGrade(String grade) {  this.grade = grade;   }
20. }
```

Figure 8.105: *Student bean class*

c. Servlet to handle and process the request

```
01. package src;
02. import java.io.IOException;
03. import java.io.PrintWriter;
04. import javax.servlet.RequestDispatcher;
05. import javax.servlet.ServletException;
06. import javax.servlet.annotation.WebServlet;
07. import javax.servlet.http.HttpServlet;
08. import javax.servlet.http.HttpServletRequest;
09. import javax.servlet.http.HttpServletResponse;
10. @WebServlet(name = "StudentMarks", urlPatterns = {"/StudentMarks"})
11. public class StudentMarks extends HttpServlet {
12.     protected void processRequest(HttpServletRequest request, HttpServletResponse response) throws ServletException, IOException {
13.          Student s=new Student();       Double m1=0.0,m2=0.0,m3=0.0,total=0.0,per=0.0;    String grade;
14.          s.setStudentid(request.getParameter("txtid"));
15.          s.setStudentname(request.getParameter("txtname"));
16.         try {
17.           m1=Double.parseDouble(request.getParameter("txtmarks1"));
18.           m2=Double.parseDouble(request.getParameter("txtmarks2"));
19.           m3=Double.parseDouble(request.getParameter("txtmarks3"));
20.         }
21.         catch(NumberFormatException nfe) { System.out.println(nfe);   }
22.          s.setMarks1(m1);           s.setMarks2(m2);          s.setMarks3(m3);
23.          total=s.getMarks1()+s.getMarks2()+s.getMarks3();          per=total/3;
24.         if ( s.getMarks1()<35 || s.getMarks2()<35 || s.getMarks3()<35 )           grade="FF";
25.         else if ( per>= 80 )              grade="AA";
26.         else if (per >= 70)               grade="BB";
27.         else if (per >= 60)               grade="CC";
28.         else if ( per>= 50)               grade="DD";
29.         else                                    grade="FF";
30.         System.out.println("grade : "+grade);
31.         s.setGrade(grade);          request.setAttribute("s",s);
32.         RequestDispatcher dispatcher = request.getRequestDispatcher("studentresult.jsp");
33.         dispatcher.forward(request, response);
34.     }
35.     @Override
36.     protected void doGet(HttpServletRequest request, HttpServletResponse response) throws ServletException, IOException {
37.          processRequest(request, response);
38.     }
39.     @Override
40.     protected void doPost(HttpServletRequest request, HttpServletResponse response) throws ServletException, IOException {
41.          processRequest(request, response);
42.     }
43.     @Override
44.     public String getServletInfo() {   return "Student Marks Handler";    }
45. }
```

Figure 8.106*: Servlet to handle and process the request*

d. JSP page to display the bean properties

```
<%@page contentType="text/html" pageEncoding="UTF-8"%>
<!DOCTYPE html>
<html>
<head>
<meta http-equiv="Content-Type" content="text/html; charset=UTF-8">
<title>Student Result</title>
<style>
.fail{  color: red; font-weight: bold; }
.pass{ color: forestgreen; font-weight: bold; }
.gradeaa{ color: blueviolet; font-weight: bold; }
.gradebb{ color: aqua; font-weight: bold; }
.gradecc{ color: deeppink; font-weight: bold; }
.gradedd{ color: rosybrown; font-weight: bold; }
.gradeff{ color: tomato; font-weight: bold; }
</style>
</head>
<body>
<h1>Student Marksheet</h1>
<table>
<tr><td>Student ID :</td><td> ${s.studentid}</td></tr>
<tr><td>Student Name :</td><td> ${s.studentname}</td></tr>
<tr><td>Java :</td><td class="${(s.marks1<35)?"fail":"pass"}"> ${s.marks1}</td></tr>
<tr><td> Php :</td><td class="${(s.marks2<35)?"fail":"pass"}"> ${s.marks2}</td></tr>
<tr><td>.Net :</td><td class="${(s.marks3<35)?"fail":"pass"}">${s.marks3}</td></tr>
<tr><td> Grade:</td>
<td class="${ (s.grade=='AA') ? "gradeaa" : (s.grade=='BB') ? "gradebb" :(s.grade=='CC') ?
 "gradecc" : (s.grade=='DD') ? "gradedd" : "gradeff"}">      ${s.grade}</td></tr>
</table>
</body>
</html>
```

***Figure 8.107**: JSP page to display the bean properties*

The application will generate the following outputs:

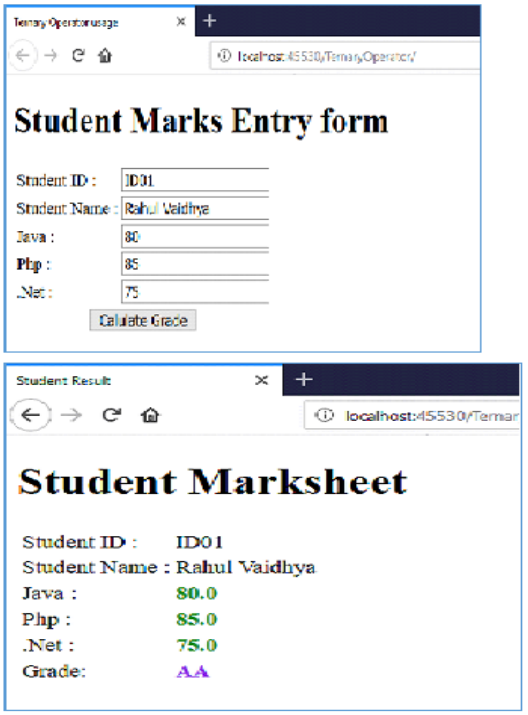


***Figure 8.108:** Student record with ID 1 and grade AA*

Student record with ID **2** and grade **FF**

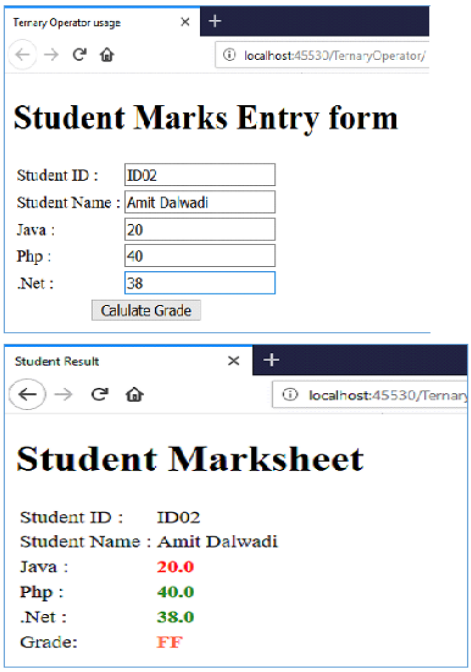


***Figure 8.109**: Student record with ID 2 and grade FF*

Student record with ID **3** and grade **BB**:

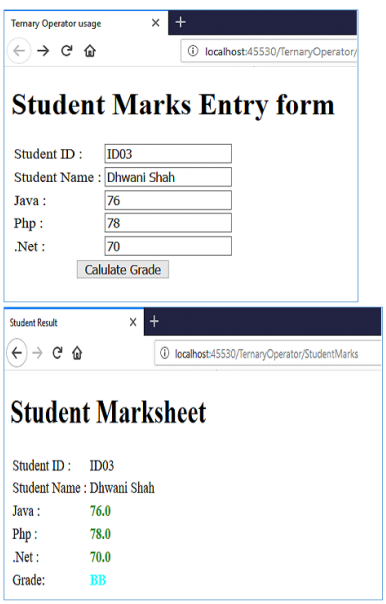


***Figure 8.110:** Student record with ID 3 and grade BB*

The application will generate the following outputs:

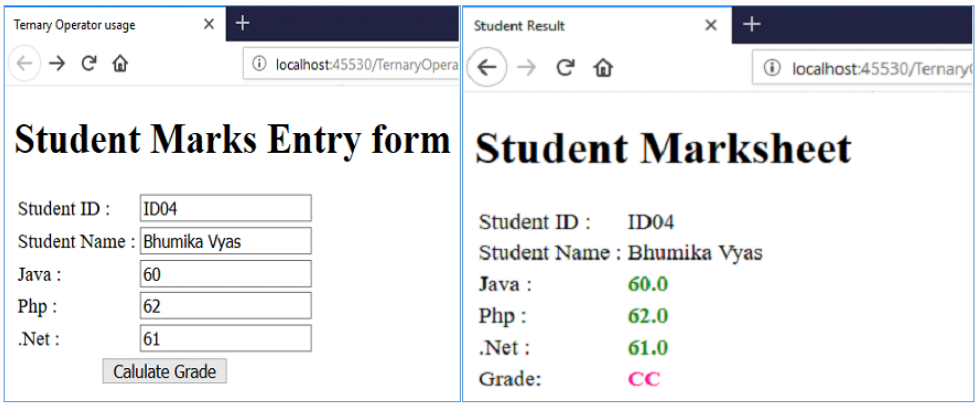


Figure 8.111: *Student record with ID 4 and grade CC*

Student record with ID **5** and grade **DD**:

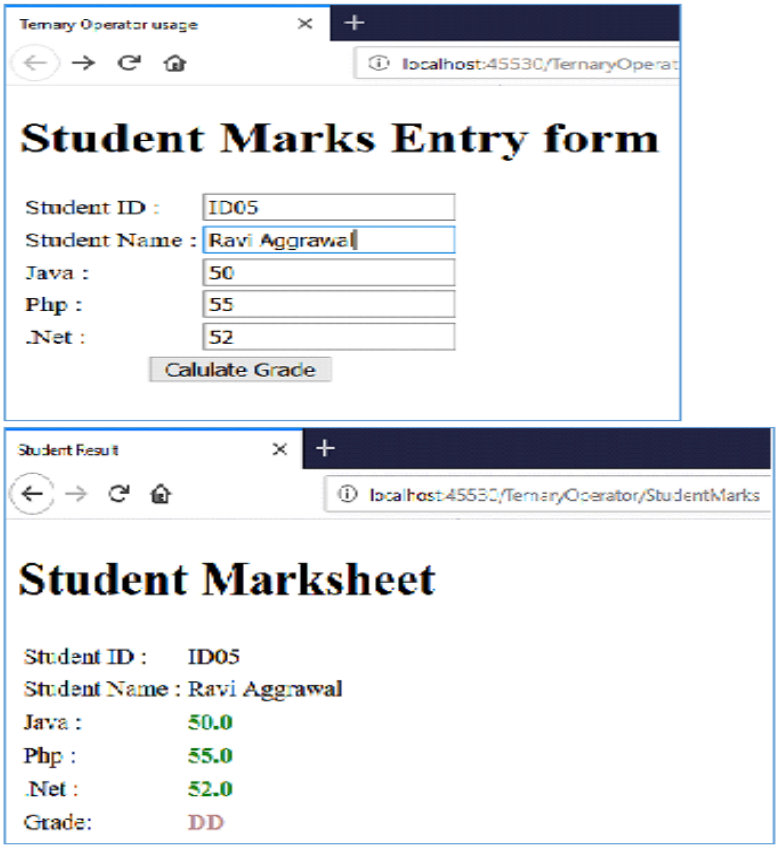


Figure 8.112: *Student record with ID 5 and grade DD*

In this example, we showed how ternary operators can be used to execute based on some condition. We combined styling with cascading style sheets applied internally using a class selector, for pass, fail and all grades, and then we read the marks and apply the style for font color and boldness; if marks are less than **35**, we apply the **fail** class; otherwise, we apply the **pass** class. Likewise, for each grade, we apply different font color set in each class; if grade is **AA**, we apply the **gradeaa** class; for grade **BB**, we apply the **gradebb** class; for grade **CC**, we apply the **gradecc** class; for grade **DD**, we apply the **gradedd** class; and for grade **FF**, we apply the **gradeff** class, conditionally using the nested ternary operator.

Using Functions

JSP EL allows you to use functions in expressions as well. These functions must be defined in the custom tag libraries. A function usage has the following syntax:

```
${ns:func(param1, param2, ...)}
```

Where ns is the namespace of the function, **func** is the name of the function and param1 is the first parameter value. For example, the **fn:length** function, which is part of the JSTL library, can be used as follows to get the length of a string:

```
${fn:length("Get my length")}
```

To use a function from any **tag** library (standard or custom), you must install that library on your server and must include the library in your JSP using the **<taglib>** directive as explained in the JSTL

Conclusion

In this chapter, we covered the concept of bean class; how to create and use it in a simple Java application and in a web application; and how to use jsp:useBean, jsp:getProperty and **jsp:getSetproperty**. We covered the sharing of beans in different locations like request, session and application. In the second half part of the chapter, we studied the usage of expression language, which is a short and easy way to invoke Java code without writing core Java syntax in HTML.

The next chapter covers the concept of MVC pattern and how it can be implemented using JSP, servlets and beans.

Points to remember

- Beans provide a very useful capability because they let you exploit the reusability of Java classes without sacrificing the convenience that JSP adds over servlets alone.

- Using expression language, we can access the bean properties with very easy and short notation.

Multiple choice questions

1. **What is the benefit of using JavaBeans components over scriptlets and JSP expressions that refer to normal Java classes?**
    a. No Java syntax
    b. Simpler object sharing
    c. Convenient correspondence between request parameters and object properties
    d. All of these

2. **The _____ attribute is used in jsp:useBean to mention the name of the object to be created.**
    a. name
    b. class
    c. id
    d. property

3. **Instead of using the _____ attribute in jsp:setProperty, you use param to name an input parameter.**
    a. id
    b. value
    c. name
    d. class

4. **The _____ attribute is used to retrieve a bean from particular place when shared with other beans.**
    a. place
    b. class
    c. name
    d. scope

5. **The _____ syntax is used to access the properties of bean using the expression language.**
    a. ${object.property}
    b. %{object.property}
    c. #{object.property}
    d. @{object.property}

Answers

1. **d**
2. **c**
3. **b**
4. **d**
5. **a**

Questions

1. What is JavaBean?
2. Explain the syntax of jsp:useBean, jsp:getProperty and jsp:setProperty.
3. Explain jsp:setProperty with name, value, param and property attributes.
4. In how many ways can bean be shared on a JSP page? Which is the default scope?
5. Write the step to store a bean in session scope from a servlet and access it on a JSP page.
6. What is expression language (EL)?
7. Explain the syntax for accessing scoped variables using EL.
8. Explain the syntax for accessing bean properties using EL.
9. Explain the syntax for accessing collections using EL.
10. Explain the syntax for referencing Implicit objects using EL.

Key terms

- **jsp:useBean**: The action tag is used to locate or instantiate a bean class.
- **jsp:getProperty**: The action tag is used to read and output the value of a bean property.

- **jsp:setProperty**: The action tag is used to assign a value to the bean property; this is equivalent to calling a setter method of the **`setXxx`** form.
- **Implicit objects**: Implicit objects are the Java objects that **`JSPContainer`** makes available to the developers in each page; the developer can call them directly without being explicitly declared. They are also called predefined variables.

Join our book's Discord space

Join the book's Discord Workspace for Latest updates, Offers, Tech happenings around the world, New Release and Sessions with the Authors:

https://discord.bpbonline.com

Chapter 9
Understand Model, View and Controller Pattern

Introduction

The chapter focuses on understanding a design pattern for developing code with a concern of separation of model, design and view that is shortly known as MVC pattern.. The chapter shows the implementation of this pattern using servlets, beans and JSP.

Structure

In this chapter, we will discuss the following topics:

- Understanding the use of technology
- MVC design pattern, need for MVC and MVC frameworks
- Implementing MVC with Servlets and JSP
- Introduction to RequestDispatcher interface
- Data sharing in various scope
- MVC examples

Objectives

After studying this chapter, you should be able to understand the concepts of model, view and controller, along with their implementation using JSP, servlets and JavaBeans. You will also understand the purpose and application of sharing the beans in various scope.

Understanding the use of technology

Technology helps us develop applications to fulfill user requirements, and selecting the right approach to develop the application will help the team to maintain the software easily. One such approach in developing applications very effectively is known as the design pattern.

Design pattern

A software design pattern is a general reusable solution to a commonly occurring problem within a given context in software design. A design pattern isn't a finished design that can be transformed directly into code. It is a description or template for how to solve a problem that can be used in several situations. Design patterns were started as best practices that were applied again and again to similar problems encountered in different contexts. They become popular after they were collected in a formalized form in the *Gang of Four* book in 1994. Originally published with C++ and Smalltalk code samples, design patterns are very popular in Java and C# can be applied in all object-oriented languages. In functional languages like Scala, certain patterns are not necessary anymore.

Uses of design patterns

Understanding and applying design patterns has various benefits; the following points mention the usage of design patterns:

- Design patterns help in organizing the code in a structured way, which helps improve the development process in terms of speed of development and locating issues generated during the implementation. The pattern provides code readability and maintenance in a well-organized way.
- Design patterns provide the solution not for a specific problem in a particular domain; they provide a general solution in a well-defined, documented structured way to a broader range of problems.
- Different design patterns are developed for different purposes, so programmers can identify the right design pattern for solving their problems. Common design patterns are improved over a period, making them more useful and stronger.

MVC design pattern

MVC stands for Model-View-Controller, and this pattern is used to separate the application's concerns. Thus, every layer in MVC is assigned with a unique responsibility. So, the view is for the look and feel and for positioning what the end user will actually see. The model supplies the data and business logic. The controller is actually the heart of MVC, as it deals with invoking the right view and deciding the navigational part in traversing the application. The controller has to coordinate with the model and view layers.

Need for MVC

The MVC architectural pattern is not directly related to web applications. In fact, it's quite common in Smalltalk applications, which generally have nothing to do with the web. MVC is concerned with separating responsibilities in web applications. Allowing a JSP page to handle the responsibilities of receiving the request, executing some business logic, and then determining the next view to display can make for an unattractive JSP page, not to mention the maintenance and extensibility problems this entanglement causes. Application development and maintenance are much easier if the different components of a web application have clear and distinct responsibilities. The following list mentions the need for MVC pattern in an application:

- Servlets are good at data processing: reading and checking data, communicating with databases, invoking business logic, and so on. JSP pages are good at presentation: building HTML to represent the results of requests. This chapter describes how to combine servlets and JSP pages to best make use of the strengths of each technology.
- Servlets are great when your application requires a lot of real programming to accomplish its task. But generating HTML with servlets can be tedious and can yield a result that is hard to modify. That's where JSP comes in. It lets you separate much of the presentation from the dynamic content. That way, you can write the HTML in the normal manner, even using HTML-specific tools and putting your web content developers to work on your JSP documents.
- The assumption behind a JSP document is that it provides a single overall presentation.
- What if you want to give totally different results depending on the data that you receive?
    - Scripting expressions, beans, and custom tags, although extremely powerful and flexible, don't overcome the limitation that the JSP page defines a relatively fixed, top-level page appearance.

- What if you need complex reasoning just to determine the type of data that applies to the current situation?
    - JSP is poor at this type of business logic.
- The solution is to use both servlets and JSP.
- In this approach, known as the Model View Controller (MVC) or Model 2 architecture, you let each technology concentrate on what it excels at.
    - The original request is handled by a servlet.
    - The servlet invokes the business-logic and data-access code and creates beans to represent the results (that's the model).
    - Then, the servlet decides which JSP page is appropriate to present those results and forwards the request there (the JSP page is the view).
    - The servlet decides what business logic code applies and which JSP page should present the results (the servlet is the controller).

MVC frameworks

The key motivation behind the MVC approach is the desire to separate the code that creates and manipulates the data from the code that presents the data. The basic tools needed to implement this presentation-layer separation are standard in the servlet API. However, in very complex applications, a more elaborate MVC framework is sometimes beneficial. The most popular of these frameworks is Apache Struts. Although Struts is useful and widely used, you should not feel that you must use Struts in order to apply the MVC approach. For simple and moderately complex applications, implementing MVC from scratch with RequestDispatcher is straightforward and flexible.

Implementing MVC with servlets and JSP

The most important point about MVC is the idea of separating the business logic and data access layers from the presentation layer. To implement the MVC pattern, we use servlets, JSP and Java bean class. The role of the model is performed by bean class, the role of the controller is performed by servlet, and the role of the presentation layer is performed by JSP page.

The following steps are performed to achieve the MVC pattern:

1. List of steps to implement the MVC pattern.
    a. Create beans to handle the data
    b. Use a servlet to listen to and process the requests

c. Fill the beans with data

d. Store the bean in the request, session, or servlet context scope

e. Forward the request to a JSP page

f. Retrieve the data from the beans

Each step is explained in detail in the following section

2. Create beans to handle the data

    a. Beans are Java objects that follow a few simple conventions.

    b. A servlet or other Java routine (never a JSP page) will create the beans.

    c. The JSP page will only access the beans, not create or modify them.

3. Use a servlet to listen and process the requests

    a. Once the bean classes are defined, the next task is to write a servlet to read the request information.

    b. Since, with MVC, a servlet responds to the initial request, the normal approaches are to read request parameters and request headers, respectively.

    c. Although the servlets use the normal techniques to read the request information and generate the data, they do not use the normal techniques to output the results.

    d. In fact, with the MVC approach, the servlets do not create any output; the output is completely handled by the JSP pages. So, the servlets do not call **`response.setContentType`**, **`response.getWriter`**, or **`out.println`**.

4. Fill the beans with data

    a. After you read the form parameters, you use them to determine the results of the request.

    b. These results are determined in a completely application-specific manner.

    c. You might call some business logic code, invoke an Enterprise JavaBeans component, or query a database.

    d. No matter how you come up with the data, you need to use that data to fill in the value object beans that you defined in the first step.

5. Store the bean in the request, session, or servlet context scope

a. Storing data that the JSP page will use only in this request

   i. First, the servlet would create and store data as follows:

```
ValueObject value = new ValueObject(...);
request.setAttribute("key", value);
```

   ii. Next, the servlet would forward the request to a JSP page that uses the following to retrieve the data:

```
<jsp:useBean id="key" type="Package.ValueObject"
scope="request" />
```

b. Storing data that the JSP page will use in this and later requests from the same client

   i. First, the servlet would create and store data as follows:

```
ValueObject value = new ValueObject(...);
HttpSession session = request.getSession();
session.setAttribute("key", value);
```

   ii. Next, the servlet would forward to a JSP page that uses the following to retrieve the data:

```
<jsp:useBean id="key" type="somePackage.
ValueObject"
scope="session" />
```

6. Forward the request to a JSP page

   a. You forward request with the forward method of **`RequestDispatcher`**.

   b. You obtain a **`RequestDispatcher`** by calling the **`getRequestDispatcher`** method of **`ServletRequest`**, supplying a relative address.

   c. Once you have a **`RequestDispatcher`**, you use forward to transfer control to the associated address.

   d. You supply **`HttpServletRequest`** and **`HttpServletResponse`** as arguments.

   e. When you use the forward method of **`RequestDispatcher`**, the client sees the URL of the original servlet, not the URL of the final JSP page.

7. Retrieve the data from the beans

   a. Once the request arrives at the JSP page, the JSP page uses **`jsp:useBean`** and **`jsp:getProperty`** to extract the data.

b. The JSP page never creates objects. The servlet, not the JSP page, should create all the data objects. So, to guarantee that the JSP page will not create objects, you should use the following:

```
<jsp:useBean ... type="package.Class" />
```

Instead of the following:

```
<jsp:useBean ... class="package.Class" />.
```

c. The JSP page should not modify the objects. So, you should use **`jsp:getProperty`** but not **`jsp:setProperty`**.

d. The scope you specify should match the storage location used by the servlet. For example, the three forms request-, session-, and application- would be used for sharing, respectively.

Introduction to RequestDispatcher interface

We are using the method of **`RequestDispatcher`** interface in servlet to apply the concept of navigation; in this section, we will discuss the syntax and usage of methods provided by this interface.

In the **`javax.servlet`** package, there is an interface named **`RequestDispatcher`**; it defines an object that receives requests from the client and sends them to any resource (such as a servlet, HTML file, or JSP file) on the server. The servlet container creates the **`RequestDispatcher`** object, which is used as a wrapper around a server resource located at a particular path or given by a particular name. This interface is intended to wrap servlets, but a servlet container can create **`RequestDispatcher`** objects to wrap any type of resource.

The interface consists of two methods:

- **Forward method**: It forwards a request from a servlet to another resource (servlet, JSP file, or HTML file) on the server.

  Syntax:

  ```
  public void forward(ServletRequest request, ServletResponse
  response) throws ServletException,java.io.IOException
  ```

  It forwards a request from a servlet to another resource (servlet, JSP file, or HTML file) on the server. This method allows one servlet to do preliminary processing of a request and another resource to generate the response.

  For a **`RequestDispatcher`** obtained via **`getRequestDispatcher()`**, the **`ServletRequest`** object has its path elements and parameters adjusted to match the path of the target resource.

forward should be called before the response has been committed to the client (before response body output has been flushed). If the response has already been committed, this method throws an **`IllegalStateException`**. Uncommitted output in the response buffer is automatically cleared before the forward.

The request and response parameters must be either the same objects as were passed to the calling servlet's service method or be subclasses of the **`ServletRequestWrapper`** or **`ServletResponseWrapperclasses`** that wrap them.

Parameters:

- **`request`**: A **`ServletRequest`** object that represents the request the client makes of the servlet
- **`response`**: A **`ServletResponse`** object that represents the response the servlet returns to the client

Throws:

- **`ServletException`**: If the target resource throws this exception
- **`java.io.IOException`**: If the target resource throws this exception
- **`java.lang.IllegalStateException`**: If the response was already committed

- **Include method**: It includes the content of a resource (servlet, JSP page, HTML file) in the response.

  Syntax:

  ```
  public void include(ServletRequest request, ServletResponse response) throws ServletException, java.io.IOException
  ```

  It includes the content of a resource (servlet, JSP page, HTML file) in the response. In essence, this method enables programmatic server-side includes.

  The **`ServletResponse`** object has its path elements and parameters remain unchanged from the caller's. The included servlet cannot change the response status code or set headers; any attempt to make a change is ignored.

  The request and response parameters must be either the same objects as were passed to the calling servlet's service method or be subclasses of the **`ServletRequestWrapper`** or **`ServletResponseWrapperclasses`** that wrap them.

Parameters:

- **`request`**: A **`ServletRequest`** object that contains the client's request
- **`response`**: A **`ServletResponse`** object that contains the servlet's response

Throws:

- **`ServletException`**: If the included resource throws this exception
- **`java.io.IOException`**: If the included resource throws this exception

Data sharing in request scope

With request-based sharing, the servlet stores the beans in the **`HttpServletRequest`**, where they are accessible only to the destination JSP page.

- **Servlet**

```
ValueObject value = new ValueObject(...);

request.setAttribute("key", value); RequestDispatcher dispatcher =
request.getRequestDispatcher("/WEB-INF/SomePage.jsp"); dispatcher.
forward(request, response);
```

- **JSP Page**

```
<jsp:useBean id="key“ type="somePackage.ValueObject"
scope="request" />

<jsp:getProperty name="key" property="someProperty" />
```

Data sharing in session scope

With session-based sharing, the servlet stores the beans in the **`HttpSession`**, where they are accessible to the same client in the destination JSP page or in other pages.

- **Servlet**

```
ValueObject value = new ValueObject(...);

HttpSession session = request.getSession();

session.setAttribute("key", value);

RequestDispatcher dispatcher = request.getRequestDispatcher("/WEB-
INF/SomePage.jsp");

dispatcher.forward(request, response);
```

- **JSP Page**

```
<jsp:useBeanid="key“ type="somePackage.ValueObject“
scope="session" />

<jsp:getProperty name="key" property="someProperty" />
```

Data sharing in application scope

With application-based sharing, the servlet stores the beans in the **`ServletContext`**, where they are accessible to any servlet or JSP page in the web application. To guarantee that the JSP page extracts the same data that the servlet inserted, you should synchronize your code as follows:

- Servlet

```
synchronized(this)

{

ValueObject value = new ValueObject(...);

getServletContext().setAttribute("key", value);

RequestDispatcher dispatcher =

request.getRequestDispatcher("/WEB-INF/SomePage.jsp"); dispatcher.
forward(request, response);

}
```

- JSP Page

```
<jsp:useBeanid="key" type="Package.ValueObject"
scope="application"/>

<jsp:getProperty name="key" property="someProperty" />
```

MVC example

In the following example, we try to provide the application of MVC using JSP, servlets and beans. In this example, we apply the MVC approach to an application that displays bank account balances:

- The controller servlet reads a customer ID and passes that to some data-access code that returns a **`BankCustomer`** value bean.
- The servlet then stores the bean in the **`HttpServletRequest`** object, where it will be accessible from destination JSP pages but nowhere else.

- If the account balance of the resulting customer is negative, the servlet forwards to a page designed for delinquent customers.
- If the customer has a positive balance of less than 10,000 INR, the servlet transfers to the standard balance-display page.
- If the customer has a balance of 10,000 INR or more, the servlet forwards the request to a page reserved for elite customers.
- Finally, if the customer ID is unrecognized, an error page is displayed.

Example 1: We cover the example using the collection framework class **`HashMap`** to create map of the customers of a bank. We create a bean class **`Customer`**, a servlet to handle the request and navigate to the right page. We create views like unknown customer, regular customer, prestigious customer and unknown customer. The view layer is selected based on the id of the customer and the balance of the customer. The following figure specifies the directory structure of the application:

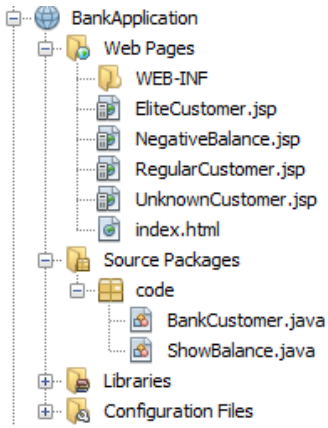


Figure 9.1: *Directory structure of the application*

- Index page of the application

```
<!DOCTYPE html>
<html>
<head>
<title>Bank Applcation</title>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
</head>
<body>
<h1>Customer Balance Inquiry </h1>
<form action="ShowBalance">
<input type="text" name="id">
<input type="submit" value="Submit"/>
</form>
</body>
</html>
```

Figure 9.2: *Code for HTML index page*

This is the index page that will load when the application executes.

- Bean class **`BankCustomer.java`**

```
01. package code;
02. import java.util.HashMap;
03.
04. public class BankCustomer
05. {
06.  private String id, firstName, lastName;
07.  private double balance;
08.
09.   public BankCustomer(String id,StringfirstName,StringlastName,double balance)
10.   {
11.     this.id = id;
12.     this.firstName = firstName;
13.     this.lastName = lastName;
14.     this.balance = balance;
15.   }
16.
17.   public String getId() {  return(id);  }
18.
19.   public String getFirstName() {  return(firstName);  }
20.
21.   public String getLastName() {  return(lastName);  }
22.
23.   public double getBalance() {     return(balance);  }
24.
25.   public double getBalanceNoSign() {   return(Math.abs(balance));  }
26.
27.   public void setBalance(double balance) {  this.balance = balance;  }
28.
29.  private static HashMap customers;
30.   static
31.   {
32.    customers = new HashMap();
33.    customers.put("id001",new BankCustomer("id001","RONAK","PATEL",-3456.78));
34.    customers.put("id002",new BankCustomer("id002","AMISH","PATEL",1234.56));
35.    customers.put("id003",new BankCustomer("id003","MEET","PATEL",987654.32));
36.   }
37.
38. public static BankCustomergetCustomer(String id) { return((BankCustomer)customers.get(id));  }
39. }
```

Figure 9.3: *Code for bean class*

Bean class manages the **`Customer`** properties.

- Servlet code to manage and store the data and navigate to the view layer

```
01. package code;
02. import java.io.IOException;
03. import javax.servlet.RequestDispatcher;
04. import javax.servlet.ServletException;
05. import javax.servlet.http.HttpServlet;
06. import javax.servlet.http.HttpServletRequest;
07. import javax.servlet.http.HttpServletResponse;
08.
09. public class ShowBalance extends HttpServlet {
10.      public void doGet(HttpServletRequest request, HttpServletResponse response)
11.              throws ServletException, IOException
12.      {
13.         BankCustomer customer = BankCustomer.getCustomer(request.getParameter("id"));
14.         String address;
15.
16.         if (customer == null) {
17.             address = "UnknownCustomer.jsp";
18.         }
19.         else if (customer.getBalance() < 0) {
20.             address = "NegativeBalance.jsp";
21.             request.setAttribute("badCustomer", customer);
22.         }
23.         else if (customer.getBalance() < 10000)
24.         {
25.             address = "/RegularCustomer.jsp";
26.             request.setAttribute("regularCustomer", customer);
27.         }
28.         else
29.         {
30.             address = "/EliteCustomer.jsp";
31.             request.setAttribute("eliteCustomer", customer);
32.         }
33.
34.            RequestDispatcher dispatcher =request.getRequestDispatcher(address);
35.           dispatcher.forward(request, response);
36.     }
37.
38. }
```

Figure 9.4: *Code for servlet class*

The servlet code is used as a controller.

- View layer for unknown customer

```
01. <%@page contentType="text/html" pageEncoding="UTF-8"%>
02. <!DOCTYPE html>
03. <html>
04. <head>
05. <meta http-equiv="Content-Type" content="text/html; charset=UTF-8">
06. <title>Unknown Customer</title>
07. </head>
08. <body>
09. <table>
10. <tr>
11.       Unknown Customer
12. </tr>
13. </table>
14. <h3>
15.     Invalid Customer, Unrecognized customer ID.</h3>
16. </body>
17.
18. </html>
19.
```

Figure 9.5: *Code for handing unknown customer*

This code is used to manage presentation view for the unknown customer.

- View layer for negative balance customer

```
<%@page contentType="text/html" pageEncoding="UTF-8"%>
<!DOCTYPE html>
<html>
<head>
<meta http-equiv="Content-Type" content="text/html; charset=UTF-8">
<title>Negative Balance</title>
</head>
<body>
<table border=1 align="center">
<tr><th> Customer Balance</table></th></tr>
<p>
<span style="color:blue">
<jsp:useBean id="badCustomer" scope="request" type="code.BankCustomer" /></span>
Your balance is negative,
<span style="color:blue">
<jsp:getProperty name="badCustomer" property="firstName" />
</span>
we need to take action against you.
<p>
pay us the<span style="color:blue"><jsp:getProperty name="badCustomer" property="balance" /> INR </span>
before it is too late.
</body>
</html>

```

Figure 9.6: *Code for handing unknown customer*

This code is for managing presentation view for customer with negative balance.

- View layer for regular customer.

```
<%@page contentType="text/html" pageEncoding="UTF-8"%>
<!DOCTYPE html>
<html>
<head>
<meta http-equiv="Content-Type" content="text/html; charset=UTF-8">
<title> Regular Customer </title>
</head>
<html>
<head>
<title> Customer Details </title>
</head>
<body>
<table border=1 align="center">
<tr>
    Customer Balance
</tr>
</table>
<p>
<span style="color:blue"><jsp:useBean id="regularCustomer" type="code.BankCustomer" scope="request" /></span>
Welcome <span style="color:blue"><jsp:getProperty name="regularCustomer" property="firstName" /></span><span style="color:blue"><jsp:getProperty name="regularCustomer" property="lastName" /></span>to the balance i
your Customer ID is <span style="color:blue"><jsp:getProperty name="regularCustomer"  property="id" /></span>
and your balance is <span style="color:blue">INR<jsp:getProperty name="regularCustomer" property="balance" /></span> , you can take benefit of bank interest by putting the fund in Fixed Deposit.
</p>
</body>
</html>

```

Figure 9.7: *Code for handing regular customer*

This code is for managing presentation view for customer with regular balance.

- View layer for prestigious customer

```
<%@page contentType="text/html" pageEncoding="UTF-8"%>
<!DOCTYPE html>
<html>
<head>
<meta http-equiv="Content-Type" content="text/html; charset=UTF-8">
<title> Elite Customer </title>
</head>
<body>
<table border=1s>
<tr><th class="title"> Customer Balance</th></tr>
</table>
<p>
<br clear="all">
<jsp:useBean id="eliteCustomer" type="code.BankCustomer"  scope="request" />
            it is an honor to serve you,
<span style="color:blue"><jsp:getProperty name="eliteCustomer"
property="firstName" />
<jsp:getProperty name="eliteCustomer" property="lastName" />!</span><br/>
<p>
            since you are one of our most valued customers, <br/> we would like
            to offer you the opportunity, <br/> to spend some amount of your balance <br/>
<span style="color:blue"><jsp:getProperty name="eliteCustomer"
property="balance" /> INR </span><br/>
            on insurance of life we offer, <br/> please visit out site <br/>
<a href="site.html">Insurance Details </a><br/>
 for more information on Life insurance. <br/>
</p>
</body>
</html>
```

Figure 9.8: *Code for handing prestigious customer*

This code is for managing presentation view for customer with high balance.

The code will result in the following output:

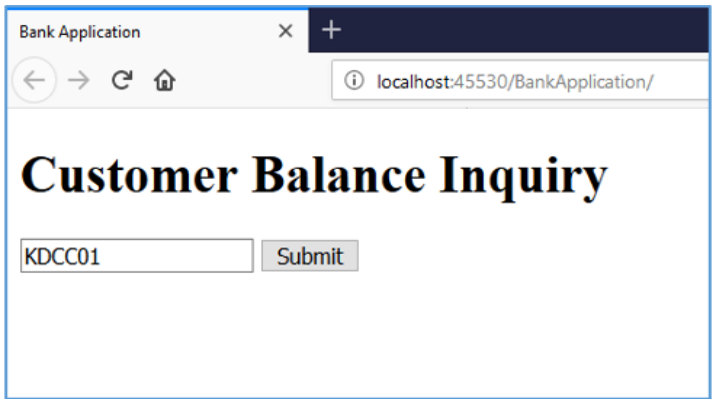


Figure 9.9*: Customer balance inquiry form*

We have entered a customer ID that does not exist:

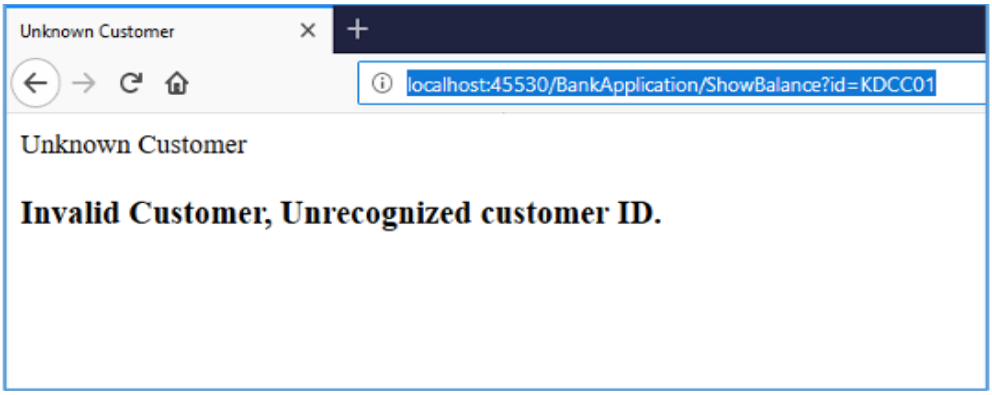


Figure 9.10: *Unknown customer view selected by the servlet as ID is not known*

The preceding view layer is displayed for an unknown customer:

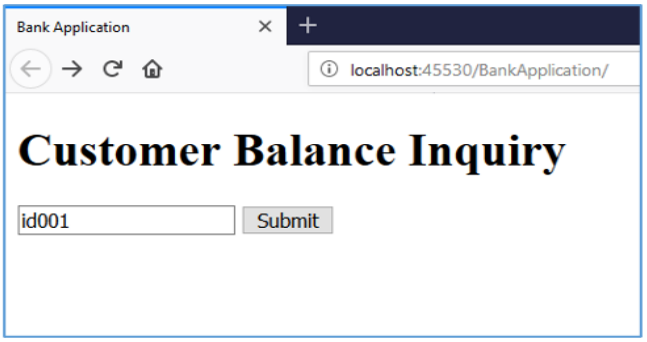


Figure 9.11: *Known Customer ID input in the inquiry form*

The code will result in the following output:

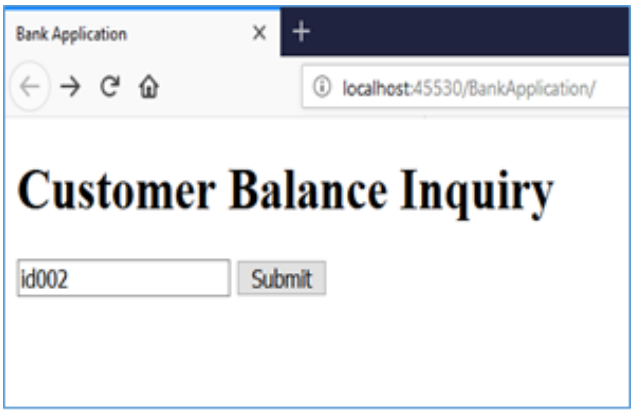


Figure 9.12*: Known Customer ID input in the inquiry form*

Following figure depicts the regular customer view:

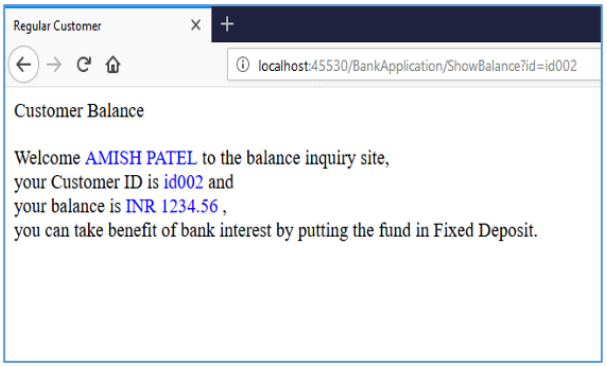


***Figure 9.13**: Regular customer view selected by the servlet as balance is less than 10,000 INR*

The code will result in the following output:

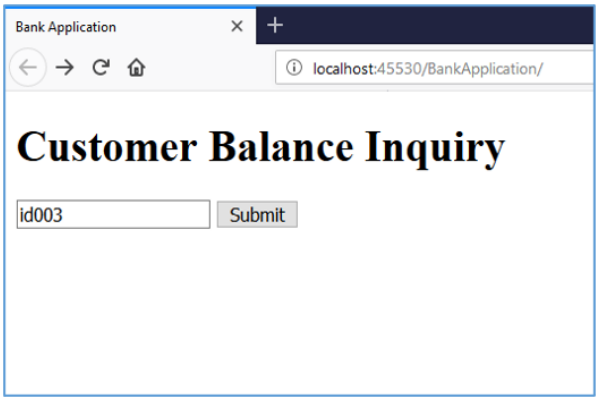


***Figure 9.14:** Known Customer ID input in the inquiry form*

We input customer ID for known customer:

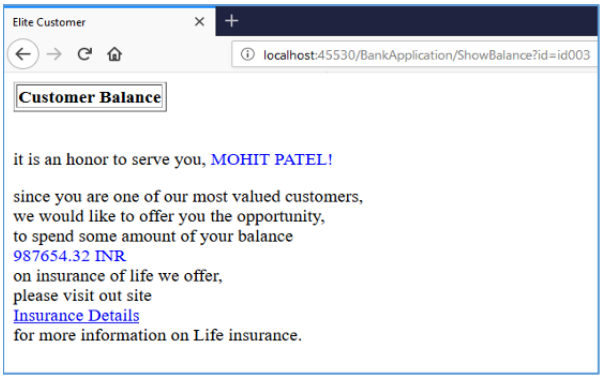


***Figure 9.15**: Elite and Prestigious customer view selected by the servlet as balance is more than 10,000 INR*

In this example, we created a small MVC application for bank customer balance inquiry. The requirement is to create a form to enter the customer id; if the user is bank customer, based on the balance, the page is viewed in the browser. If the customer's balance is negative, then the view for Negative balance is selected; if the customer balance is positive and less than 10,000 INR, the view for Regular customer is selected; and if the balance is 10,000 or more, then the view for prestigious/elite customer is selected. In this example, we have created the following classes and JSP pages:

- **Index html page**: HTML page to input the customer ID
- **BankCustomer class**: Bean class to manage the customer details; it acts as a model layer
- **ShowBalance class**: Servlet to listen and process the request, set and share bean, and navigate to correct view layer
- UnknownCustomer.jsp, NegativeBalance,jsp, RegularCustomer.jsp, EliteCustomer.jsp: JSP pages to render in the browser as a view layer to display the information to the user

The application flow is as shown in in the below *:*

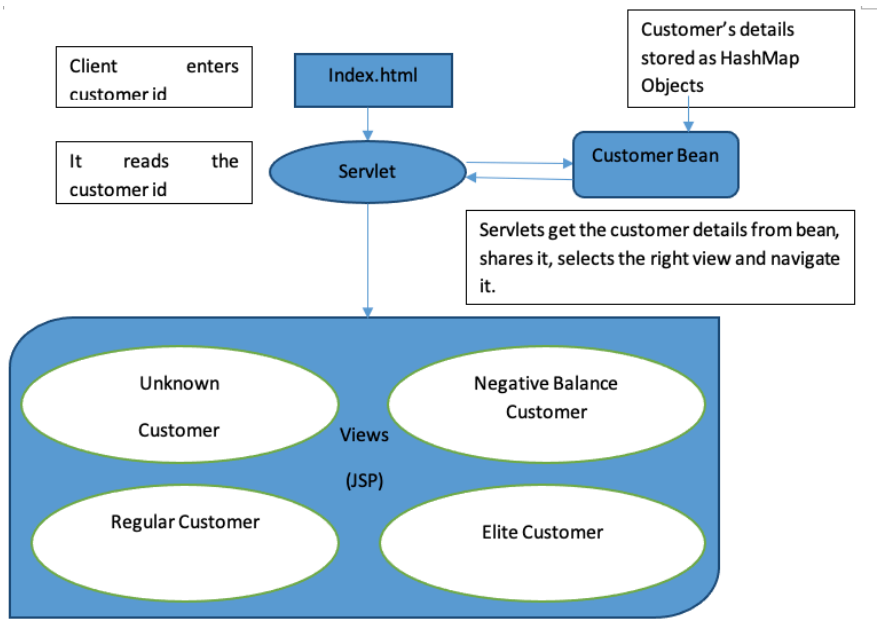


***Figure 9.16**: Bank Application execution flow*

Example 2: In this example, we include database part to store the customer details in the table instead of **`HashMap`**. We insert the customer details and search the customers. We create a bean class **`Customer`** to get and set the values of customer, and a servlet

to handle the request and navigate to the right page. We create views like unknown customer, regular customer, prestigious customer and unknown customer. The view layer is selected based on the id and balance of the customer:

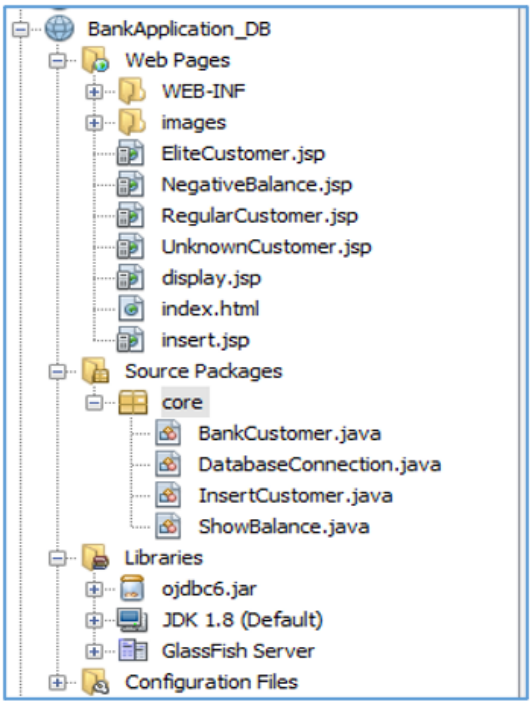


***Figure 9.17**: Directory Structure of Example 2*

a. Index page of the application

```
<!DOCTYPE html>
<html>
<head>
<title>KDCC Bank</title>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
</head>
<body>
<center>
<a href="insert.jsp" style="float:right;margin:0 10 px"> Insert  </a>
<h1>Welcome to KDCC Bank</h1>
<imgsrc="images/kdcc.jpg"/><br>
<imgsrc="images/kdcc1.jpg"/>
<h2>To Check balance online click on below link</h2>
<form action="ShowBalance">
<input type="text" name="id">
<input type="submit" value="Submit"/>
</form>
</center>
</body>
</html>
```

***Figure 9.18**: Code for Index page*

Index page loads as the first page of the application.

b. Insert page to enter the customer details

```
<%@page contentType="text/html" pageEncoding="UTF-8"%>
<!DOCTYPE html>
<html>
<head>
<meta http-equiv="Content-Type" content="text/html; charset=UTF-8">
<title>Insert Customer Details</title>
<script type="text/javascript">
          function check(v1, v2, v3, errorMsg){
              if (v1.value.length == 0)
              {
                  v1.focus();
                  return false;
              }
              if (v2.value.length == 0){
                  v2.focus();
                  return false;
              }
              if (v3.value.length == 0) {
                  v3.focus();
                  return false;
              }
              return true;
          }
</script>
</head>
<body>
<center>
<h1>Insert Customer Details</h1>
<imgsrc="images/kdcc.jpg"/><br>
<h2>
<form action="InsertCustomer" onsubmit="return check(document.getElementById('fname'),document.getElementById('lname'), document.getElementById('bal'), 'Please enter values')">
      Customer First Name <span style="color:red">*</span>:
<input type="text" name="txtfname" id="fname"/><br>
      Customer Last Name <span style="color:red">*</span>:
<input type="text" name="txtlname" id="lname"/><br>
      Initial Balance Deposit <span style="color:red">*</span>:
<input type="text" name="txtbal" id="bal"/><br><br>
<input type="submit" value="Submit"/>
</form>
</h2>
<h5> * - indicates mandatory input </h5>
</body>
</html>
```

***Figure 9.19**: Code for customer details*

This code provides the customer details.

c. **`Bean`** class **`Customer`** to handle the customer details

```
package coreservlets;
import java.util.HashMap;
public class BankCustomer {
    private String id, firstName, lastName;
    private double balance;

    public String getId() {  return id;  }
    public void setId(String id) {   this.id = id;   }
    public String getFirstName() {   return firstName;   }
    public void setFirstName(String firstName) { this.firstName = firstName;    }
    public String getLastName() {   return lastName;   }
    public void setLastName(String lastName) { this.lastName = lastName;    }
    public double getBalance() {  return balance;  }
    public void setBalance(double balance) { this.balance = balance;    }
}
```

***Figure 9.20**: Code for customer details*

This is the code for customer properties.

d. Database code to connect and write queries to the database

```
01. package core;
02. import java.sql.Connection;                import java.sql.DriverManager;
03. import java.sql.PreparedStatement;    import java.sql.ResultSet;             import java.sql.Statement;
04.
05. public class DatabaseConnection {
06.     Connection con;      Statement stmt;          ResultSet rs;
07.     public boolean connect() {
08.     boolean flag = true;
09.         try {
10.             String driver = "oracle.jdbc.OracleDriver";         String url = "jdbc:oracle:thin:@localhost:1521:xe";
11.             String username = "system";           String password = "oracle";
12.             Class.forName(driver);     con = DriverManager.getConnection(url, username, password);   stmt = con.createStatement();
13.         } catch (Exception e) {System.out.println("Exception : " + e);  flag = false;  }
14.         return flag;
15.     }
16.     public booleaninsertrecord(BankCustomer c) {  boolean flag = true;
17.        try {
18.             String pinsert_query = "INSERT INTO Customer VALUES (?,?,?,?)";
19.             PreparedStatementpstmt = con.prepareStatement(pinsert_query);
20.             pstmt.setString(1, c.getId());                  pstmt.setString(2, c.getFirstName());
21.             pstmt.setString(3, c.getLastName());     pstmt.setDouble(4, c.getBalance());  int i = pstmt.executeUpdate();
22.         } catch (Exception ex) {   flag = false;    }
23.         return flag;
24.     }
25.     public intcountRecords() { int i = 0;
26.         String str = "select count(*) as count from customer";
27.         try {  rs = stmt.executeQuery(str);
28.               while (rs.next()) {  i = rs.getInt("count");   }
29.         } catch (Exception e) { System.out.println("Exception :" + e);  }
30.            System.out.println("out i : " + i);   return i;
31.     }
32.     public BankCustomerselectRecords(String sid) {
33.         String select_query = "SELECT * FROM customer where id=" + "'"+sid+ "'";
34.         System.out.println("ID in method : " +select_query);
35.         BankCustomer c = new BankCustomer();
36.         try {
37.               stmt = con.createStatement();     rs = stmt.executeQuery(select_query);
38.             while (rs.next()) {
39.                 c.setId(rs.getString(1));                  c.setFirstName(rs.getString(2));
40.                 c.setLastName(rs.getString(3));      c.setBalance(rs.getDouble(4));
41.                 }
42.              } catch (Exception e) { System.out.println("Exception :" + e);     }
43.             return c;
44.           }   } catch (Exception e) { System.out.println("Exception :" + e);    }
45.     }  }
```

***Figure 9.21**: Code for customer details*

The preceding code is used to manage database queries.

e. **`InsertServlet`** to insert the customer details in the table

```
01. package core;
02. import java.io.IOException;
03. import javax.servlet.RequestDispatcher;
04. import javax.servlet.ServletException;
05. import javax.servlet.annotation.WebServlet;
06. import javax.servlet.http.HttpServlet;
07. import javax.servlet.http.HttpServletRequest;
08. import javax.servlet.http.HttpServletResponse;
09. @WebServlet(name = "InsertCustomer", urlPatterns = {"/InsertCustomer"})
10. public class InsertCustomer extends HttpServlet {
11. int i;  DatabaseConnection dbconnect;
12. @Override
13.     public void init() {
14.         dbconnect = new DatabaseConnection();       dbconnect.connect();
15.     }
16. Override
17.     public void doGet(HttpServletRequest request, HttpServletResponse response)
18.             throws ServletException, IOException
19.     {
20.      i = dbconnect.countRecords();
21.     System.out.println(" in B servlet i : " +i);
22.     i++;
23.     System.out.println(" in A servlet i : " +i);
24.     String id = "KDCC0" + i;
25.     System.out.println("Inside Servlet" + id);
26.     BankCustomer customer = new BankCustomer();
27.     customer.setId(id);       customer.setFirstName(request.getParameter("txtfname"));
28.     customer.setLastName(request.getParameter("txtlname"));      customer.setBalance(Double.parseDouble(request.getParameter("txtbal")));
29.     System.out.println("Customer value set : " + customer.getId() + "  " +
30.     customer.getFirstName() + "  " + customer.getBalance());
31.     request.setAttribute("customer", customer);
32.    dbconnect.insertrecord(customer);
33.    RequestDispatcher dispatcher = request.getRequestDispatcher("display.jsp");
34.    dispatcher.forward(request, response);
35.     }
36. }
```

***Figure 9.22**: Code for customer details*

Code for handling customer.

f. **`ShowServlet`** to show the customer details of known customer, otherwise display unknown customer details

```
01. package core;
02. import java.io.IOException;
03. import javax.servlet.RequestDispatcher;
04. import javax.servlet.ServletException;
05. import javax.servlet.annotation.WebServlet;
06. import javax.servlet.http.HttpServlet;
07. import javax.servlet.http.HttpServletRequest;
08. import javax.servlet.http.HttpServletResponse;
09.
10. @WebServlet(name = "ShowBalance", urlPatterns = {"/ShowBalance"})
11. public class ShowBalance extends HttpServlet
12. {
13.       int i;
14.       DatabaseConnection dbconnect;
15.      @Override
16.      public void init() {
17.      dbconnect = new DatabaseConnection();
18.      dbconnect.connect();
19.     }
20.     @Override
21.     protected void doGet(HttpServletRequest request, HttpServletResponse response)
22.             throws ServletException, IOException
23.     {
24.         String id=request.getParameter("id");
25.         BankCustomer customer=dbconnect.selectRecords(id);
26.         System.out.println("customer: "+customer.getId()+ " "+customer.getBalance());
27.         String address;
28.         if (customer.getId() == null) {
29.             address = "/UnknownCustomer.jsp";
30.          }
31.        else if (customer.getBalance() < 0) {
32.             address = "/NegativeBalance.jsp";
33.             request.setAttribute("badCustomer", customer);
34.           }
35.  else if (customer.getBalance() < 10000) {
36.             address = "/RegularCustomer.jsp";
37.             request.setAttribute("regularCustomer", customer);
38.         }
39.  else {
40.                 address = "/EliteCustomer.jsp";
41.                 request.setAttribute("eliteCustomer", customer);
42. }
43.        RequestDispatcher dispatcher =request.getRequestDispatcher(address);
44.       dispatcher.forward(request, response);
45.     }
46. }
```

Figure 9.23: *Code for customer details*

This is the servlet code that acts as a controller.

g. Unknown customer page

```
01. <%@page contentType="text/html" pageEncoding="UTF-8"%>
02.
03. <!DOCTYPE html>
04.
05. <html>
06. <head>
07. <meta http-equiv="Content-Type" content="text/html; charset=UTF-8">
08. <title>KDCC Bank</title>
09. </head>
10. <body>
11. <a href="index.html" style="float:right;margin:0 20 px"> Home</a>
12. <table border=1 align="center">
13. <tr><th class="title">Unknown Customer</th>
14. </table>
15. <center><imgsrc="images/kdcc.jpg"/><br>
16. <h3>unrecognized customer id.</h3></center>
17. </body>
18.
19. </html>
```

Figure 9.24*: Code for presentation view of unknown customer*

This is the view layer for unknown customer.

h. Negative balance customer page

```
<%@page contentType="text/html" pageEncoding="UTF-8"%>
<!DOCTYPE html>
<html>
<head>
<meta http-equiv="Content-Type" content="text/html; charset=UTF-8">
<title>KDCC Bank</title>
</head>
<body>
<a href="index.html" style="float:right;margin:0 20 px"> Home</a>
<table border=1 align="center">
<tr><th>  Customer Balance</th></tr>
</table>
<center><imgsrc="images/kdcc.jpg"/><br></center><p>

<span style="color:blue">
<jsp:useBean id="badCustomer" scope="request" type="core.BankCustomer" />
</span>
            Your balance is negative,
<span style="color:blue">
<jsp:getProperty name="badCustomer" property="firstName" /></span>
            we need to take action against you.
<p>     pay us the
<span style="color:blue">
<jsp:getProperty name="badCustomer" property="balance" /> INR
</span>
            before it is too late.
</body>
</html>
```

***Figure 9.25**: Code for presentation view of negative customer*

This is the view layer for negative balance customer.

i. Regular customer page

```
<%@page contentType="text/html" pageEncoding="UTF-8"%>
<!DOCTYPE html>
<html>
<head>
<meta http-equiv="Content-Type" content="text/html; charset=UTF-8">
<title>JSP Page</title>
</head>
<html>
<head>
<title>KDCC Bank</title>
</head>
<body>
<a href="index.html" style="float:right;margin:0 20 px"> Home</a>
<table border=5 align="center">
<tr>
<th class="title">
                        Customer Balance
</th>
</tr>
</table>
<center><imgsrc="images/kdcc.jpg"/><br></center>
<p>
<span style="color:blue">
<jsp:useBean id="regularCustomer" type="core.BankCustomer" scope="request" /></span>
 Welcome <span style="color:blue"><jsp:getProperty name="regularCustomer" property="firstName" /></span>
<span style="color:blue"><jsp:getProperty name="regularCustomer" property="lastName" /></span> to the balance inquiry site,<br/>
your Customer ID is <span style="color:blue"><jsp:getProperty name="regularCustomer"  property="id" /></span>
and </br/> your balance is <span style="color:blue"> INR <jsp:getProperty name="regularCustomer" property="balance" />
</span> ,</br/> you can take benefit of bank interest by putting the fund in Fixed Deposit.
</p>
</body>
</html>

```

***Figure 9.26**: Code for presentation view of regular customer*

This is the view layer for regular customer.

j. Elite customer page

```
<%@page contentType="text/html" pageEncoding="UTF-8"%>
<!DOCTYPE html>
<html>
<head>
<meta http-equiv="Content-Type" content="text/html; charset=UTF-8">
<title>KDCC Bank</title>
</head>
<body>
<a href="index.html" style="float:right;margin:0 20 px"> Home</a>
<table border=1 align="center">
<tr>
<th class="title">
                    Customer Balance
</th>
</tr>
</table>
<p>
<br clear="all">
<jsp:useBean id="eliteCustomer" type="core.BankCustomer"  scope="request" />
            it is an honor to serve you,
<span style="color:blue">
<jsp:getProperty name="eliteCustomer" property="firstName" />
<jsp:getProperty name="eliteCustomer" property="lastName" />!</span><br/>
<p>
            since you are one of our most valued customers, <br/> we would like
            to offer you the opportunity, <br/> to spend some amount of your balance <br/>
<span style="color:blue">
<jsp:getProperty name="eliteCustomer"  property="balance" /> INR </span><br/>
            on insurance of life we offer, <br/> please visit out site <br/>
<a href="site.html">Insurance Details </a>
<br/> for more information on Life insurance. <br/>
</p>
</body>
</html>
```

***Figure 9.27**: Code for presentation view of prestigious customer*

This is the view layer for prestigious customer.

k. Display customer page

```
<%@page contentType="text/html" pageEncoding="UTF-8"%>
<!DOCTYPE html>
<html>
<head>
<meta http-equiv="Content-Type" content="text/html; charset=UTF-8">
<title>KDCC Bank</title>
</head>
<body>
<a href="index.html" style="float:right;margin:0 20 px"> Home</a>
<center><h1>Customer Details</h1>
<center><imgsrc="images/kdcc.jpg"/></center><br>
<h3>
<jsp:useBean id="customer" type="core.BankCustomer" scope="request"/>
        Customer ID : <span style="color:gray">
<jsp:getProperty name="customer" property="id"/></span><br>
        Customer Name :<span style="color:blueviolet">
<jsp:getProperty name="customer" property="firstName"/>
<jsp:getProperty name="customer" property="lastName"/></span><br>
         Customer Balance : <span style="color:green">
<jsp:getProperty name="customer" property="balance"/></span>
</h3>
</center>
</body>
</html>
```

***Figure 9.28**: Code for presentation view of prestigious customer*

Customer details are displayed using the preceding page:

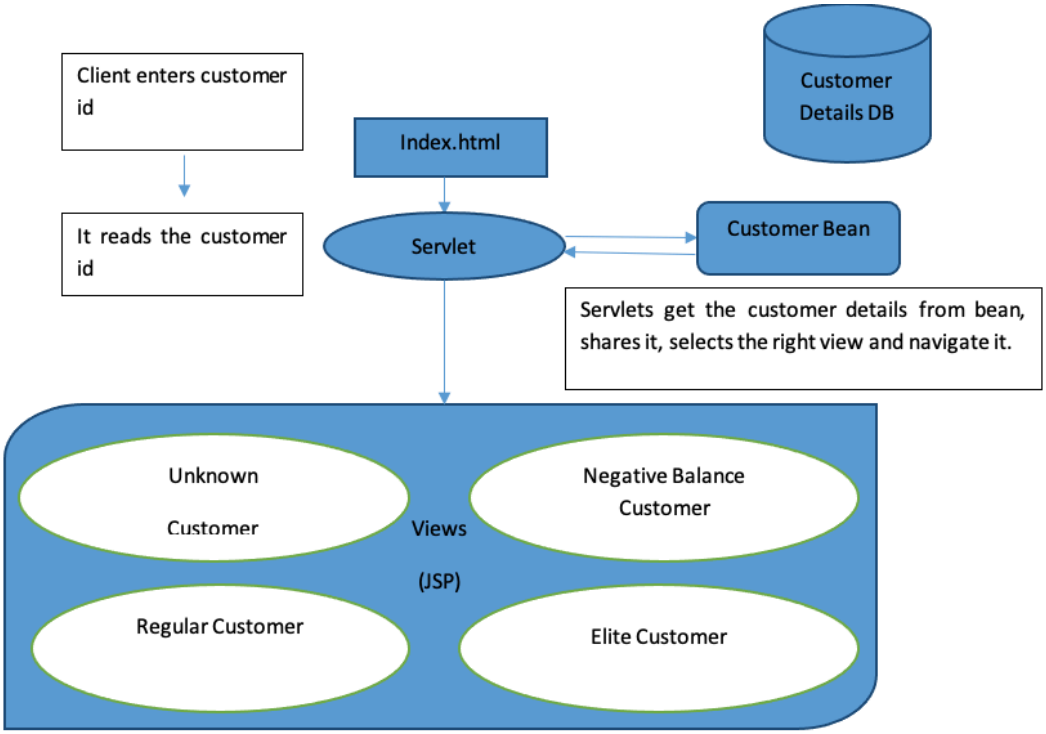


Figure 9.29: *Bank application with database execution flow*

In this example, we integrate the database concept discussed in *Chapter 2*, *Performing Insert, Update, Delete and Select Operations* and *Chapter 3,Creating Properties File and Performing Batch Operations* . Instead of taking a **`HashMap`** of customers to hold the **`Customer`** details, we create a **`Customer`** table in Oracle and create a **`DatabaseConnection`** class to handle to code to register the driver, establish the connection and queries to handle the database. We create a search form to input the customer ID; then, the customer ID is read by the servlet and passed on to the database method to search and return the customer details; if customer is not found in the database, then the unknown customer page is displayed in the browser, and if the customer is found from the database, then based on the balance of customer, the regular customer or elite customer page is displayed in the browser. The insert page is used to input the customer data and pass it to servlet, whose role is to call the database query to insert the data. In the servlet's code, the method to connect the database is called in the **`init`** method, as the connection is to be established only once, and the query to process the request is written in the **`processrequest`** method, which is called in **`doGet`** and **`doPost`** methods.

The following figures show the search for **`Customer ID`** found with negative balance:

Figure 9.30: *Customer ID search page*

The preceding input navigates to the page of the customer having negative balance in the account:

Figure 9.31*: ID found with negative balance*

The following figures show the search for Customer ID with more balance the elite customer page:

Figure 9.32: *Customer ID search page*

The preceding input navigates to the page of the customer having very good balance in the account:

Figure 9.33: *Customer ID search page; ID found with more balance, elite customer page*

The following figures show the search for Customer ID with regular balance:

Figure 9.34: *Customer ID search page*

The preceding input navigates to the page of the customer having regular balance in the account:

Figure 9.35: *ID found with regular balance*

The following figures show the search for unknown customer ID:

Figure 9.36: *Customer ID search page*

The preceding input navigates to the page of the **`Unknown Customer`** as the customer ID is not known.

Figure 9.37*: ID not found, so unknown customer page*

The following output shows the customer details input form:

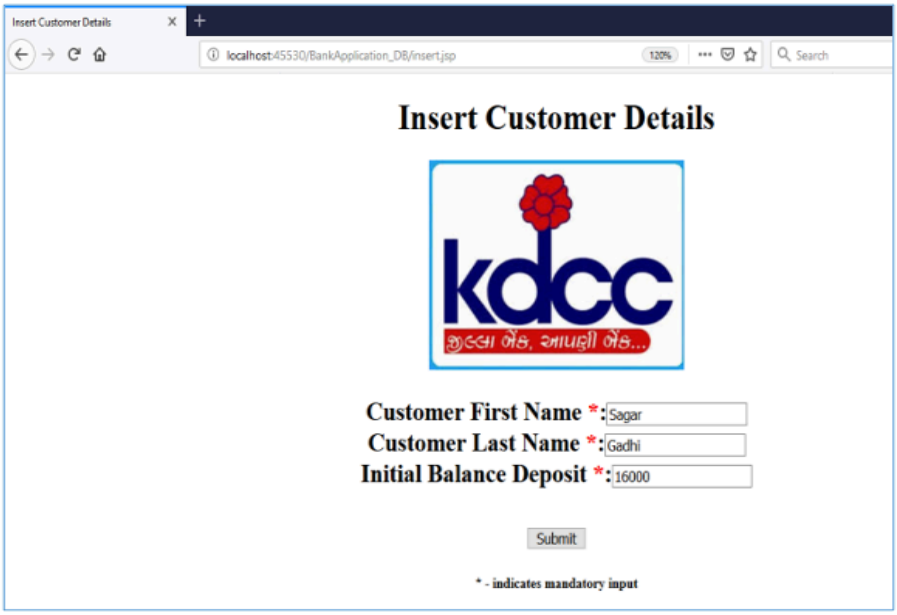


Figure 9.38: *Customer details input form*

The preceding input navigates to the page of the customer details where ID is autogenerated.

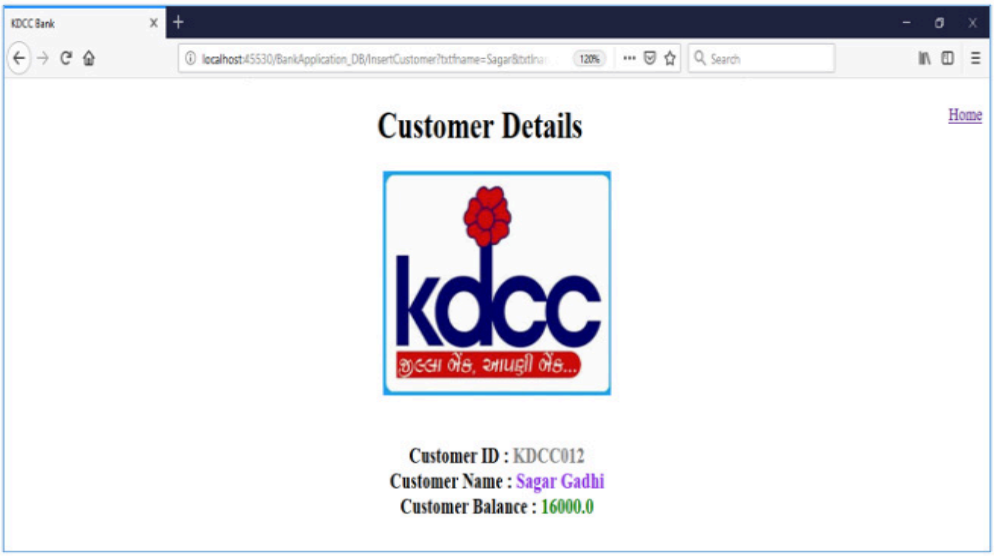


Figure 9.39: *ID is autogenerated and the details are displayed in the page*

The following figures show the table structure:

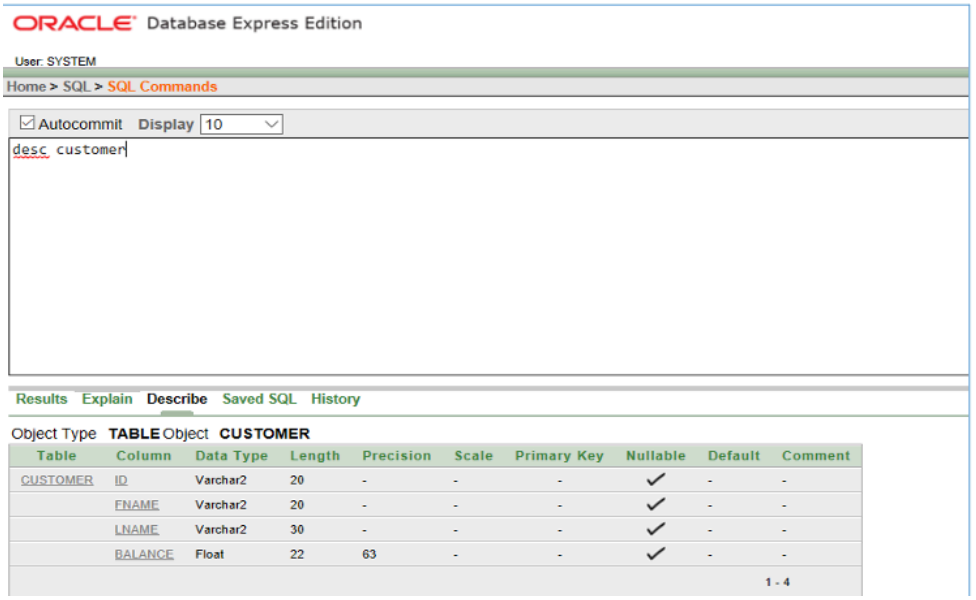


***Figure 9.40**: Oracle table structure*

The following figures show the entry of record in Oracle:

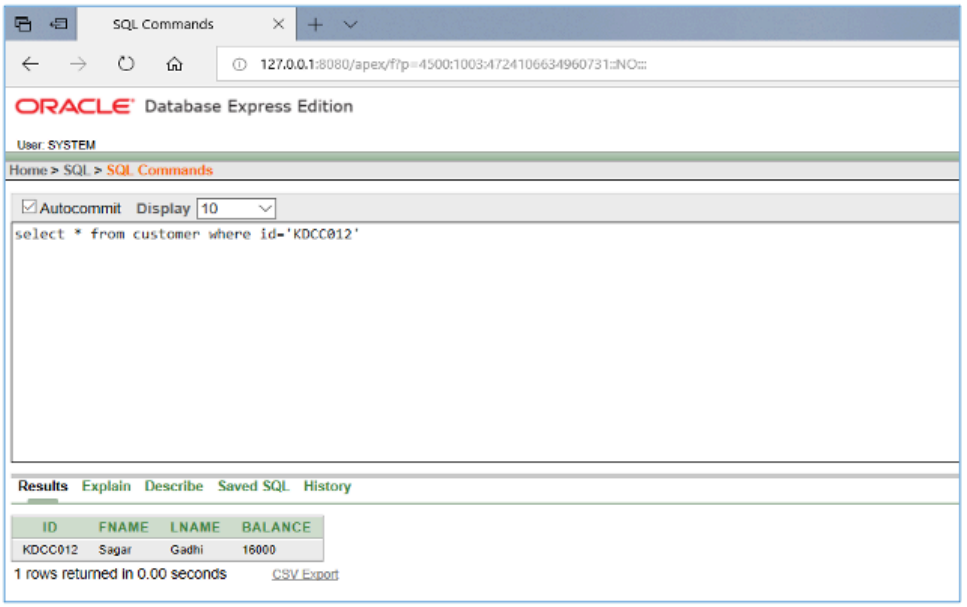


***Figure 9.41:** Newly inserted record in the table*

Database entry of new customer

Conclusion

In this chapter, we covered the concept of model, view and controller in the development of the application. The MVC pattern suggests the separation of code based on the different role and purpose, so the error solving and managing of code become easier as the application size and complexity increase. Servlet is good at processing, JSP is good at presentation, and bean class and database can be used to represent the entity class. Technologies can be optimized in using it for right purpose, so Servlet is used as a Controller, JSP is used as a View layer and, Java Beans are used as Model layer.

The next chapter covers the use of JSP Standard Tag library for performing various operations using functions provided by this library.

Points to remember

In the MVC pattern implemented by using servlets and JSP, the role of model is performed by bean class, the role of controller is performed by servlet and the role of presentation layer is performed by JSP page.

We are using the method of **`RequestDispatcher`** interface in servlet to apply the concept of navigation.

Multiple choice questions

1. **Servlets are not good in which of these?**
    a. Data processing
    b. Communicating with databases
    c. Invoking business logic
    d. Presentation by building HTML to represent the results of requests

2. **The key motivation behind the MVC approach is the desire to separate the code that creates and manipulates the data from the code that presents the data.**
    a. True
    b. False
    c. Not sure

3. **For storing data that the JSP page will use in this request and in later requests from the same client, the data must be stored in the _____ scope.**
    a. request
    b. session
    c. application

4. **In which servlet method is the control transferred entirely on the server and no network traffic is involved.**
    a. forward
    b. sendRedirect
    c. None of these

Answers

1. **d**
2. **a**
3. **b**
4. **a**

Questions

1. In how many ways can bean be shared on a JSP page? Which is the default scope?
2. Write the steps of implementing MVC with **`RequestDispatcher`**.
3. Write the step to store a bean in session scope from a servlet and access it on a JSP page.
4. Differentiate between forward and the **`sendRedirect`** method.
5. Explain data sharing in application scope with an example.

Key terms

- **Design pattern**: A software design pattern is a general reusable solution to a commonly occurring problem within a given context in software design.
- **MVC pattern**: It stands for Model-View-Controller Pattern and is used to separate an application's concerns.

Join our book's Discord space

Join the book's Discord Workspace for Latest updates, Offers, Tech happenings around the world, New Release and Sessions with the Authors:

https://discord.bpbonline.com

Chapter 10
Overview of JSP Standard Tag Library (JSTL)

Introduction

This chapter covers the tag library specially designed to work with HTML-like syntax to achieve various functionality related to programming, database, XML, etc. It covers the usage and syntax of tags with examples.

Structure

The chapter includes the following topics:

- Introduction to JSTL
- Core tag library
- SQL tags
- XML tags
- JSTL functions
- Formatting tags

Objectives

After studying this chapter, you should be able to understand and use readymade library for various purposes. The built in functions will help programmers code easily with HTML like tags work with complex functions.

Introduction to JSTL

JSTL stands for **JSP Standard Tag Library**. JSTL is a collection of the standard JSP tags, which includes the core functionality common to control the JSP page behavior. JSTL tags can be used for iteration and control statements, and for manipulating XML documents, internationalization tags, and SQL tags. It also provides a framework for integrating the existing custom tags with the JSTL tags. In the previous chapters, we saw the use of JSP tags and JSP EL to perform complex actions in a simple way using Java language in HTML. But when it comes to nested loops, they have lots of SQL statements; we cannot skip writing core Java in HTML. That is where the use of JSP is misused as it has to include a lot of Java in HTML, which is not a good programming practice. To avoid this problem, the JSTL library was introduced to perform complex tasks in a simple way using HTML-like tags that perform the core language functionality work.

Installing JSTL library

JSTL is part of the Java EE API and is included in most servlet containers. But to use JSTL in our JSP pages, we need to download the JSTL jars for your servlet container. You need to include these libraries in your web application project **`WEB-INF/lib`** directory. JSTL jars are container specific, for example, in Tomcat, we need to include **`jstl.jar and standard.jar`** JAR files in project build path. If they are not present in the container lib directory, you should include them in your application. To begin working with JSTL tags you need to install the JSTL library. If you are using the Apache Tomcat container, follow these two steps:

- Download the binary distribution from Apache Standard Taglib and unpack the compressed file. You can find the link from **https://tomcat.apache.org/taglibs/index.html**.
- To use the Standard Taglib from its Jakarta Taglibs distribution, simply copy the JAR files in the distribution's **`lib`** directory to your application's **`webapps\ROOT\WEB-INF\lib`** directory.
- To use any of the libraries, you must include a **`<taglib>`** directive at the top of each JSP that uses the library.

Classification of the JSTL tags

JSTL includes a wide variety of tags that fit into discrete functional areas. To reflect this and to give each area its own namespace, JSTL is exposed as multiple tag libraries. The URIs for the libraries are as follows:

- **Core: http://java.sun.com/jsp/jstl/core**
- **XML: http://java.sun.com/jsp/jstl/xml**
- **Internationalization: http://java.sun.com/jsp/jstl/fmt**
- **SQL: http://java.sun.com/jsp/jstl/sql**
- **Functions: http://java.sun.com/jsp/jstl/functions**

The tags include the functionalities listed in *Table 10.1*:

Tag Group Name	Functionality Provided	Prefix
Core	Variable support, Flow control, URL management	`c`
XML	Core, Flow control, Transformation	`x`
I18N	Locale, Message formatting ,Number and date formatting	`fmt`
Database	SQL connection and writing queries	`sql`
Functions	Message formatting, number and date formatting, etc.	`fn`

***Table 10.1**: JSTL tag groups based on functionalities*

The JSTL represents a set of tags to simplify the JSP development, which includes the following advantages:

- Fast development
- Code reusability
- No need to use scriptlet tag

Core tag library

JSTL Core provides several core tags, such as Iteration, conditional logic, catch exception, URL forward, and Redirect, to support some basic scripting tasks. To use core tags, we need to define tag library first; the following is the syntax to include a tag library by using the **`taglib`** directive:

Syntax: **`<%@ taglib prefix="c" uri=http://java.sun.com/jsp/jstl/core%>`**

Here:

- prefix can be used to define all the core tags **c**
- **uri** is the library of **taglib** from which it is imported

c:out tag

The **<c:out>** tag displays the result of an expression. This is almost similar to the way **<%= %>** works. The difference here is that the **<c:out>** tag lets you use the simpler "**.**" notation to access properties. For example, to access **student.address.city**, use the **<c:out value = "student.address.city "/>** tag. Apart from this, the **<c:out>** tag can automatically escape XML tags, so they aren't evaluated as actual tags.

The **<c:out>** tag has the following attributes:

Attribute	Description	Required	Default
Value	Information to output	Yes	None
default	To display value when not set	No	body
escapeXml	True if the tag should escape special XML characters	No	true

***Table 10.2**: Attributes of <c:out> tag*

Example 1:

```
<%@page contentType="text/html" pageEncoding="UTF-8"%>
<%@ taglib prefix = "c" uri = "http://java.sun.com/jsp/jstl/core" %>
<!DOCTYPE html>
<html>
   <head>
      <title> Tag Example </title>
   </head>

   <body>
       <h2> <c:out value = " <c:out> Tag Example "/> </h2>
      <h3>
          Student ID : <c:out value="${studentid}" default="MSIT001"/>
      </h3>
   </body>
</html>

```

***Figure 10.1**: Code to demonstrate the use of cout tag*

The code will have the following output:

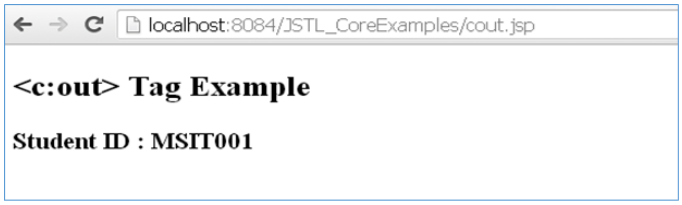


***Figure 10.2**: <c:out> tag example*

c:set and c:remove tags

The **`<c:set>`** tag is JSTL-friendly version of the **`setPropertyaction`**. The tag is helpful because it evaluates an expression and uses the results to set a value of a JavaBean or a **`java.util.Map`** object.

The **`<c:set>`** tag has the following attributes:

Attribute	Description
`Value`	Information to save
`target`	Name of the variable whose property should be modified
`property`	Property to modify
`var`	Name of the variable to store information
`scope`	Scope of variable to store information

***Table 10.3**: Attributes of <c:set> tag*

If target is specified, property must also be specified.

Example:

```
01. <%@ page contentType="text/html" pageEncoding="UTF-8"%>
02. <%@ taglib prefix = "c" uri = "http://java.sun.com/jsp/jstl/core" %>
03. <!DOCTYPE html>
04. <html>
05.    <head>
06.       <title> Tag Example </title>
07.    </head>
08.
09.    <body>
10.        <h2> <c:out value = " <c:set> Tag Example "/> </h2>
11.        <c:set var="i" value="<%=(int)(1+(Math.random()*10))%>" />
12.        <c:set var = "studentid" scope = "session" value = "MSIT0${i}"/>
13.       <h3>
14.
15.       <c:out value = "${studentid}"/>
16.       </h3>
17.    </body>
18. </html>
```

***Figure 10.3**: Code to demonstrate the use of cset tag*

The following output is generated from the preceding code:

***Figure 10.4**: <c:set> tag example*

c:if tag

The **<c:if>** tag evaluates an expression and displays its body content only if the expression evaluates to true. The condition is evaluated through its required attribute test.

The **<c:if>** tag has the following attributes:

Attribute	Description
test	Condition to evaluate
var	Name of the variable to store the condition's result
scope	Scope of the variable to store the condition's result

***Table 10.4**: Attributes of <c:if> tag*

Example: In this example, a student ID is generated, and if the ID is greater than 10, it is displayed with a message; otherwise, the student ID is displayed without message.

```
<%@page contentType="text/html" pageEncoding="UTF-8"%>
<%@ taglib prefix = "c" uri = "http://java.sun.com/jsp/jstl/core" %>
<!DOCTYPE html>
<html>
   <head>
      <title> Tag Example </title>
   </head>

   <body>
       <h2>Student ID </h2>
       <c:set var="id" value="<%=(int)(1+(Math.random()*16))%>" />
       <c:set var = "studentid" scope = "session" value = "MSIT0${id}"/>
       <c:if test="${id >=10}">
           <h3> Student ID is greater than 10, ID = ${studentid} </h3>
       </c:if>

      <h3>
          <c:out value = "${studentid}"/>
      </h3>
   </body>
</html>
```

***Figure 10.5**: Code to demonstrate the use of if tag*

The following output is generated from the preceding code:

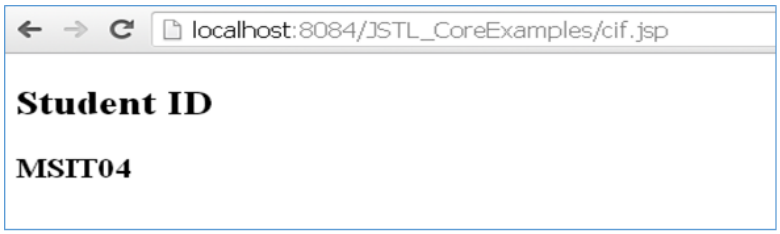


***Figure 10.6**: <c:if> tag output not displayed as id is less than 10*

The following output is generated from the preceding code:

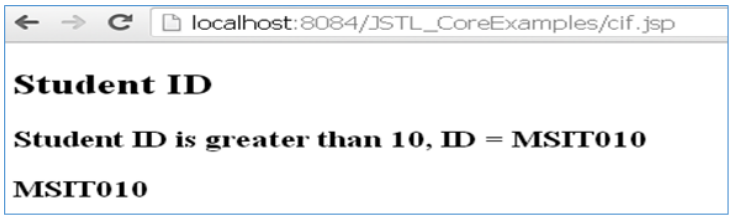


***Figure 10.7**: <c:if> tag output displayed as id is equal to 10*

<c:if> is like an if statement in Java that evaluates a condition and executes a block of code if the result is true. This is the basic syntax of the **<c:if>** core tag. The set of statements enclosed within the **<c:if>** tag is executed if **test="true"**. For using this tag, we generally use expression language to evaluate an relational expression. We use EL because it returns boolean value(true/false) after evaluating the condition and we need the Boolean value for test attribute. There are two other

optional attributes for this tag: var and scope. Using these attributes, you can simply store the test results in a variable within a specified scope.

- **`var`**: Variable name in which the test result would be stored
- **`scope`**: The scope for storing the value, for example, if it's session, the stored **`var`** value can be accessed till the session is active

Storing the test result in variable res in request scope. For printing the value, we have given **`requestScope.res`** as the variable is stored in request; however, you can give variable name (**`res`**) alone, and it would still work fine.

```
<c:if test="${17 >= 18}" var="res" scope="request">

</c:if>

<c:out value="${requestScope.res}"/>
```

c:choose, c:when, and c:otherwise tag

The **`<c:choose>`** tag works like a Java switch statement in that it lets you choose between a number of alternatives. While the switch statement has case statements, the **`<c:choose>`** tag has **`<c:when>`** tags. Just as a **`switch`** statement has the default clause to specify a default action, **`<c:choose>`** has **`<c:otherwise>`** as the default clause. The **`<c:choose>`** tag does not have any attribute. The **`<c:when>`** tag has one attribute, which is listed below. The **`<c:otherwise>`** tag does not have any attribute.

The **`<c:when>`** tag has the following attributes:

Attribute	Description
`test`	Condition to evaluate

***Table 10.5**: Attributes of the <c:when> tag*

Example: The student ID is generated and compared with 10; likewise, the message is displayed along with student ID.

```
01.
02. <%@ taglib prefix = "c" uri = "http://java.sun.com/jsp/jstl/core" %>
03. <!DOCTYPE html>
04. <html>
05.    <head>
06.       <title> Tag Example </title>
07.    </head>
08.    <body>
09.        <h2> Student ID  </h2>
10.        <c:set var="id" value="<%=(int)(1+(Math.random()*30))%>" />
11.        <c:set var = "studentid" scope = "session" value = "MSIT0${id}"/>
12.
13.       <c:choose>
14.          <c:when test = "${id < 10}">
15.            <h3> Student ID is less than 10, ID = ${studentid} </h3>
16.          </c:when>
17.          <c:when test = "${id >=10}}">
18.              <h3> Student ID is greater than 10, ID = ${studentid} </h3>
19.          </c:when>
20.          <c:otherwise>
21.               <h3> Student ID is greater than 20, ID = ${studentid} </h3>
22.          </c:otherwise>
23.       </c:choose>
24.    </body>
25. </html>
```

***Figure 10.8**: Code to demonstrate the use of choose and when tags*

The following output is generated from the preceding code:

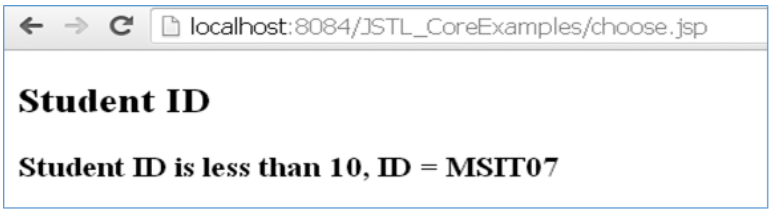


***Figure 10.9**: <c:choose> and <c:when> tag output displayed as id is less than 10*

The following output is generated from the preceding code:

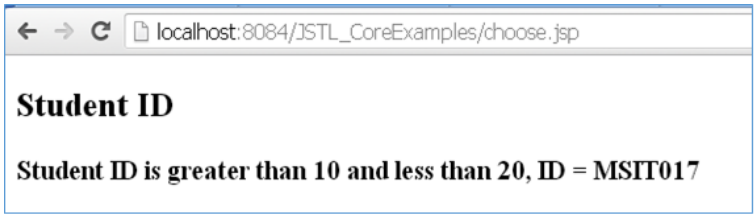


***Figure 10.10**: <c:choose> and <c:when> tag output displayed as id is greater than 10 and less than 20*

The following output is generated from the preceding code:

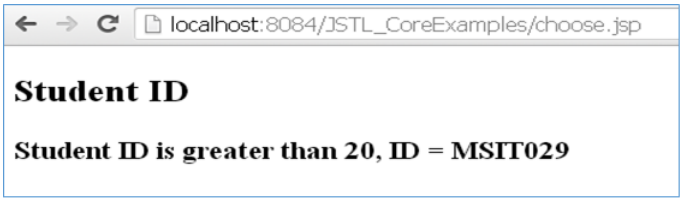


***Figure 10.11**: <c:choose> and <c:otherwise> tag output displayed as id is greater than 20*

These tags can be used for testing an expression and checking for correct values; unlike switch, multiple expressions can be tested in the **`<c:when>`** tag; and like switch statement, if all the options are wrong, the default value is selected using the **`<c:otherwise>`** tag.

c:forEach and c:forTokens tags

The **`<c:forEach>`** tag in JSTL is used for executing the same set of statements for a finite number of times. It's similar to the **`for`**, **`while`**, and **`do-while`** loop in Java. This is used when we need to perform and execute a set of statements again and again for a specified number of times. The **`<c:forEach>`** tag has the following attributes:

Attribute	Description
`items`	Information to loop over
`begin`	Element to start with (0 = first item, 1 = second item, ...)
`end`	Element to end with (0 = first item, 1 = second item, ...)
`step`	Process every step items
`var`	Name of the variable to expose the current item
`varStatus`	Name of the variable to expose the loop status

***Table 10.6**: Attributes of <c:forEach> tag*

Example: Generate a list of five numbers using the **<c:forEach>** tag

```
<%@page contentType="text/html" pageEncoding="UTF-8"%>
<%@ taglib prefix = "c"  uri = "http://java.sun.com/jsp/jstl/core" %>
<!DOCTYPE html>
<html>
  <head>
      <title> Tag Example </title>
   </head>
   <body>
       <h1> List of numbers </h1>
       <ul>
        <c:forEach var = "i" begin = "1" end = "5">
            <li> <c:out value = "${i}"/> </li>
        </c:forEach>
       </ul>
   </body>
</html>
```

***Figure 10.12**: <c:choose> and <c:otherwise> tag output displayed as id is greater than 20*

The following output is generated from the preceding code:

***Figure 10.13**: <c:forEach> tag used to generate a list of five numbers*

<c:forTokens> is also used for iteration, but it only works with delimiter, which means using this tag can break the input data into multiple parts based on the delimiter. We will understand this with the help of an example in this post.

The **<c:forTokens>** tag has similar attributes as that of the **<c:forEach>** tag, except one additional attribute: delims; it specifies the characters to use as delimiters.

Attribute	Description
delims	Characters to use as delimiters

***Table 10.7**: Attributes of the <c:forTokens> tag*

Example: Generate a list of courses using the **<c:forEach>** tag

```
<%@page contentType="text/html" pageEncoding="UTF-8"%>
<%@ taglib prefix = "c" uri = "http://java.sun.com/jsp/jstl/core" %>
<!DOCTYPE html>
<html>
    <head>
        <title> Tag Example </title>
    </head>
    <body>
        <h1> List of Cities </h1>
        <ul>
            <c:forTokens
                                items = "Object Oriented Programming, Front End Designing, System
                                                Analysis and Design"
                               delims = ","  var = "coursename">
                <li>
                               <c:out value = "${coursename}"/>
                </li>
            </c:forTokens>
        </ul>
    </body>
</html>

```

***Figure 10.14**: Code to demonstrate the use of forTokens tag*

The following output is generated from the preceding code:

***Figure 10.15**: <c:forTokens> tag used to generate a list of courses separated by comma delimiter*

Example: Generate a list of cities using the **<c:forEach>** tag

```
<%@page contentType="text/html" pageEncoding="UTF-8"%>
<%@ taglib prefix = "c" uri = "http://java.sun.com/jsp/jstl/core" %>
<!DOCTYPE html>

<html>
    <head>
        <title> Tag Example </title>
    </head>
    <body>
     <h1> List of Cities </h1>
     <ul>
     <c:forTokens items = "Ahmedabad Anand Baroda Nadiad Surat" delims = " " var = "cityname">
                <li> <c:out value = "${cityname}"/> </li>
     </c:forTokens>
     </ul>
    </body>
</html>
```

***Figure 10.16**: Code to demonstrate the use of forTokens tag with default delimiter space character*

The following output is generated from the preceding code:

***Figure 10.17**: <c:forTokens> tag usage*

The following **<c:forTokens>** tag used to generate a list of cities separated by space delimiter. This tag acts as a string tokenizer, where we can generate tokens based on different delimiters.

c:import tag

The JSTL **<c:import>** tag is used for importing the content from another file/page to the current JSP page. The **<c:import>** tag provides all functionalities of the **<include>** action but also allows for the inclusion of absolute URLs. For example, using the import tag allows for the inclusion of content from a different website or an FTP server. The **<c:import>** tag has the following attributes:

Attribute	Description
url	URL to retrieve and import into the page
context	/ followed by the name of a local web application
charEncoding	Character set to use for imported data
var	Name of the variable to store imported text
scope	Scope of the variable used to store imported text
varReader	Name of an alternate variable to expose `java.io.Reader`

***Table 10.8:** Attributes of <c:import> tag*

Example: Import data from a local JSP file using the **<c:import>** tag

```
<%@page contentType="text/html" pageEncoding="UTF-8"%>
<%@ taglib prefix = "c" uri = "http://java.sun.com/jsp/jstl/core" %>
<!DOCTYPE html>

<html>
  <head>
      <title> Tag Example </title>
   </head>
   <body>
   <center>
        <h1>Importing data from another local file </h1>

       <c:import var="jspdata" url="cout.jsp"/>
              <c:out value="${jspdata}"/>
   </center>
   </body>
</html>
```

***Figure 10.18**: <c:imports> tag usage*

The following output is generated from the preceding code:

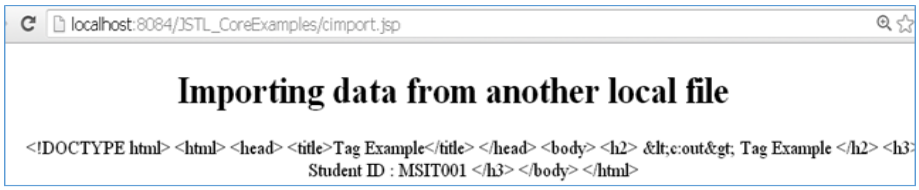


***Figure 10.19**: <c:import> tag used to import the content of a JSP page into a variable*

Example: Import data from a server page into a variable

```
<%@ taglib prefix = "c" uri = "http://java.sun.com/jsp/jstl/core" %>
<!DOCTYPE html>

<html>
  <head>
      <title> Tag Example </title>
   </head>
   <body>
   <center>

       <h1> Importing data from server file </h1>

       <c:import var="serverdata"
                      url="https://en.wikipedia.org/wiki/JavaServer_Pages_Standard_Tag_Library"/>
       <c:out value="${serverdata}"/>
   </center>
   </body>
</html>
```

***Figure 10.20**: <c:import> tag used to import the content of a JSP page into a variable*

The following output is generated from the preceding code:

***Figure 10.21**: <c:import> tag used to import the content from the server*

In the previous examples, the first example showed how to import the data from the local machine and store it into a variable; in the second example, the content was imported from the file on server and stored in a variable.

Example: Import output from a local file into the current JSP page

```
01. <%@page contentType="text/html" pageEncoding="UTF-8"%>
02. <%@ taglib prefix = "c" uri = "http://java.sun.com/jsp/jstl/core" %>
03. <!DOCTYPE html>
04.
05. <html>
06.   <head>
07.       <title>Tag Example</title>
08.    </head>
09.    <body>
10.    <center>
11.    <h1>Importing data from server file </h1>
12.    <c:import url="cout.jsp"/>
13.   </center>
14.   </body>
15. </html>
```

***Figure 10.22**: <c:import> tag*

The following output is generated from the preceding code:

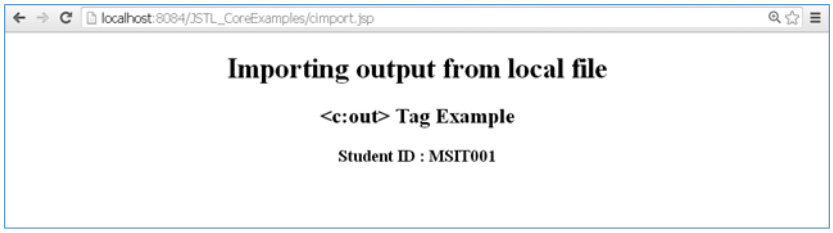


***Figure 10.23**: <c:import> tag used to import the output from the a local file*

In the previous example, we have not used the "**var**" attribute, so the output from the file is imported in the page at the place where we have used the **<c:import>** tag.

c:url and c:param tags

The **<c:url>** tag formats a URL into a string and stores it into a variable. This is mainly used when we need to open a JSP page based on the user input or based on the value of a variable. It converts a relative **url** into an application context's **url**. This tag automatically performs URL rewriting when necessary. The **var** attribute specifies the variable that will contain the formatted URL. The JSTL **url** tag is just an alternative method of writing the call to the **response.encodeURL()** method. The only real advantage that the **url** tag provides is proper URL encoding, including any parameters specified by children param tag.

The **<c:url>** tag has the following attributes:

Attribute	Description
Value	Base URL
context	/ followed by the name of a local web application
var	Name of the variable to expose the processed URL
scope	Scope of the variable to expose the processed URL

***Table 10.9:** Attributes of the <c:url> tag*

Example: Rewriting a **`url`** using the **`<c:url>`** tag

```
<%@page contentType="text/html" pageEncoding="UTF-8"%>
<%@ taglib prefix = "c" uri = "http://java.sun.com/jsp/jstl/core" %>
<!DOCTYPE html>
<html>
   <head>
      <title> Tag Example </title>
   </head>

   <body>
       <h2> URL Rewriting </h2>
    <h3>
       <c:url value="/index.html"/>
      </h3>
   </body>
</html>
```

***Figure 10.24**: <c:import> tag used to import the content from the server*

The following output is generated from the preceding code:

***Figure 10.25**: <c:url> tag used to rewrite the URL of the given page by adding application name*

Example: **`<c:url>`** tag used to provide the path in the **`anchor`** tag

```
<%@page contentType="text/html" pageEncoding="UTF-8"%>
<%@ taglib prefix = "c" uri = "http://java.sun.com/jsp/jstl/core" %>
<!DOCTYPE html>
<html>
   <head>
      <title> Tag Example </title>
   </head>

   <body>
       <h2> URL Rewriting </h2>
    <h3>

        <a href = "<c:url value = "/cset.jsp"/>"> c:set example </a>
      </h3>
   </body>
</html>
```

***Figure 10.26**: Using anchor tag with url tag*

The following output is generated from the preceding code:

- Link of page
- The page is opened using the relative URL

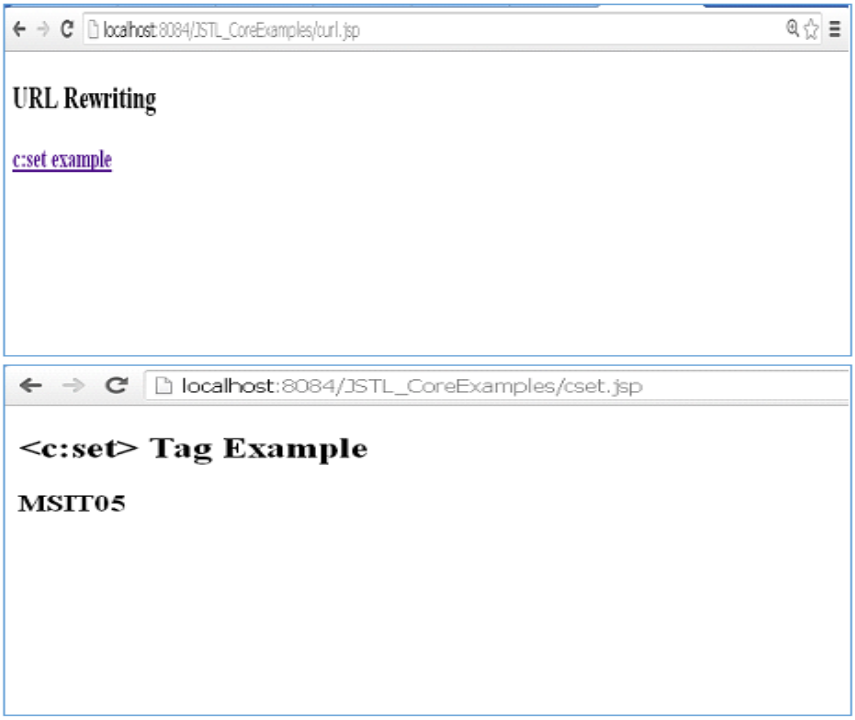


***Figure 10.27**: <c:url> tag used in the link tag to provide the path of the page*

In the previous examples, the **`<c:url>`** tag is used to provide the path of the page to open.

The **`c:param`** tag is mostly used with the **`<c:url>`** and **`<c:redirect>`** tags. It adds parameters and their values to the output of these tags. The **`<c:param>`** tag allows proper URL request parameter to be specified with URL and also does the necessary URL encoding required. Within a **`<c:param>`** tag, the name attribute indicates the parameter name, and the value attribute indicates the parameter value . The **`<c:param>`** tag has the following attributes:

Attribute	Description
`name`	Name of the request parameter to set in the URL
`value`	Value of the request parameter to set in the URL

***Table 10.10**: Attributes of <c:param> tag*

Example: Using the **`<c:param>`** tag to add parameters to the **`<c:url>`** tag

```
<%@page contentType="text/html" pageEncoding="UTF-8"%>
<%@ taglib prefix = "c" uri = "http://java.sun.com/jsp/jstl/core" %>
<!DOCTYPE html>
<html>
    <head>
        <title>Tag Example</title>
    </head>

    <body>
        <h2> cparam Tag  </h2>
        <c:url value = "cout_parameters.jsp" var = "urlname">
            <c:param name = "fname" value = "Indira"/>
            <c:param name = "lname" value = "Gandhi"/>
        </c:url>
        <c:import url = "${urlname}"/>
    </body>
</html>
```

***Figure 10.28**: Using url tag with param attribute*

Example: Using the **`<c:out>`** tag to display parameter

```
<%@page contentType="text/html" pageEncoding="UTF-8"%>
<%@ taglib prefix = "c" uri = "http://java.sun.com/jsp/jstl/core" %>
<!DOCTYPE html>
<html>
   <head>
      <title> Tag Example </title>
   </head>
   <body>
       <h2> Parameter Values  </h2>
      <h3>
          First Name  : <c:out value="${param.fname}" default="Mahatma"/> <br/>
          Last Name : <c:out value="${param.lname}" default="Gandhi"/> <br/>
      </h3>
   </body>
</html>
```

***Figure 10.29**: Using cout tag with default attribute*

The following output is generated from the preceding code:

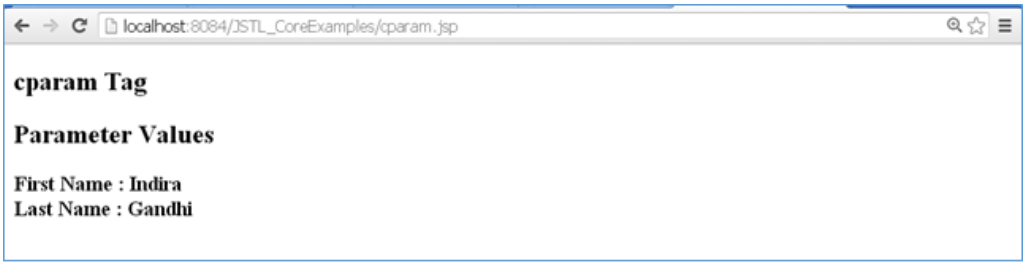


***Figure 10.30**: <c:param> tag used set parameters fname and lname, in <c:url> tag*

Example: Using the default values when the **`<c:param>`** tag is not used

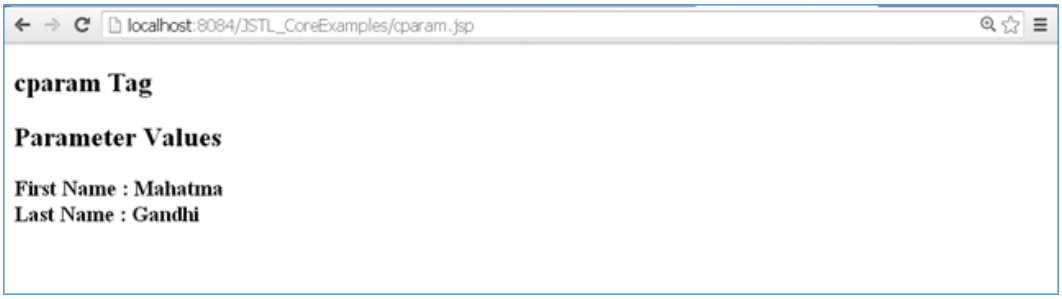


***Figure 10.31**: Default values for the parameters fname and lname used when <c:param> tag not used*

In the previous examples, we have used the **`<c:param>`** tag to set the parameter values for the **`<c:url>`** tag. While displaying the values using expression language, we have used the implicit object param with the parameter name, and default values are displayed when the parameter values are not passed.

c:redirect tag

The **`<c:redirect>`** tag is used for redirecting the current page to another URL. The **`<c:redirect>`** tag redirects the browser to an alternate URL by facilitating automatic URL rewriting, and it supports context-relative URLs. It also supports the **`<c:param>`** tag. The **`<c:redirect>`** tag has the following attributes:

Attribute	Description
`url`	URL to redirect the user's browser to
`context`	/ followed by the name of a local web application

***Table 10.11:** Attributes of the <c:redirect> tag*

Example: Using the **`<c:redirect>`** tag to redirect from one page to another

```
01. <%@page contentType="text/html" pageEncoding="UTF-8"%>
02. <%@ taglib prefix = "c" uri = "http://java.sun.com/jsp/jstl/core" %>
03. <!DOCTYPE html>
04. <html>
05.    <head>
06.       <title> Tag Example </title>
07.    </head>
08.
09.    <body>
10.        <h2>  Redirecting to Google </h2>
11.       <h3>
12.          <c:redirect url = "http://www.google.com"/>
13.       </h3>
14.    </body>
15. </html>
16.
```

***Figure 10.32**: Using cout tag with default attibute*

The following output is generated from the previous code.

a. Index page to open the **`credirect.jsp`** page

b. Google page is redirected

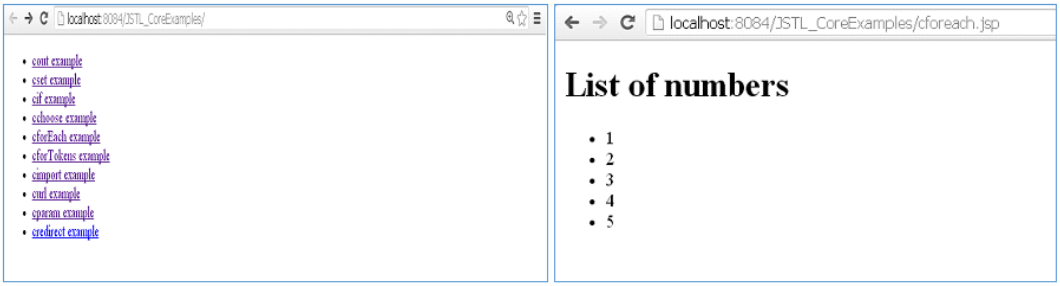
***Figure 10.33**: Redirected to Google page from credirect.jsp*

Example: Using **<c:redirect>** tag to forward from one page to another

```
<%@page contentType="text/html" pageEncoding="UTF-8"%>
<%@ taglib prefix = "c" uri = "http://java.sun.com/jsp/jstl/core" %>
<!DOCTYPE html>
<html>
   <head>
      <title> Tag Example </title>
   </head>
   <body>
       <h2>  Redirecting to cforEach Page </h2>
      <h3>
                  <c:redirect url = "cforeach.jsp"/>
      </h3>
   </body>
</html>
```

***Figure 10.34**: Using redirect tag*

The following output is generated from the preceding code:



a. index page to open the **credirect.jsp** *page* *b. Page forwarded to* **cforeach.jsp**

***Figure 10.35**: Forwarded to cforeach.jsp from credirect.jsp*

c:catch tag

The **c:catch** tag acts like the **try/catch** construct in Java, except that in the case of **c:catch**, there is no try. If you surround part of the page that may throw an exception with the **c:catch** tag and an exception occurs, the page will still render up until the point where the exception occurred and then continue rendering from the **c:catch** end tag on.

You can store the thrown exception in a page scoped attribute specified by the optional var attribute. The **c:catch** tag should be used only for debugging or experimentation. Exception handling should be part of the business logic code generally using core Java, not in the JSP page.

Example

```
01. <%@ taglib prefix="c" uri="http://java.sun.com/jsp/jstl/core"%>
02. <!doctype html >
03. <html>
04. <head><title><c:out value="<c:catch> tag"/></title></head>
05. <body>
06.  <h1> Catch Tag example  </h1>
07.  <h3>Division By Zero Exception</h3>
08.  <c:catch var="dbzexp">
09.   <% int  n = 10 / 0; %>
10.  </c:catch>
11.  <h3>Exception Occurred </h3>
12.  Exception  :  ${ dbzexp.message}
13. </body>
14. </html>
```

***Figure 10.36**: Catch tag example*

SQL tags

The **JSP Standard Tag Library** (**JSTL**) adds essential features to JSP that enable JSP programming without the need for embedded Java code. It is a collection of standard JSP tags that perform several common tasks. This frees us from having to develop custom tags for these tasks, or from using a mix of tags from several organizations to do our work.

The SQL JSTL contains tags for interacting with relational databases. This can be, for instance, MySQL, PostgreSQL, Oracle or Microsoft SQL server. The SQL JSTL tags are designed for lightweight web application concerning database access. Writing queries using SQL JSTL is easy, but complex queries are more flexible writing through database access, for example JDBC.

Before using the SQL JSTL, the following directive needs to be added to the JSP page:

```
<%@ taglib prefix="sql" uri="http://java.sun.com/jsp/jstl/sql" %>
```

sql:setDataSource tag

The **`<sql:setDataSource>`** tag provides a **`DataSource`** for connecting to a database used by all actions in the SQL tag library. The syntax is as follows:

```
<sql:setDataSource [var="variable name"]
  [scope="page|request|session|application"]
  [driver="JDBC driver"]
  [url="JDBC URL for database"]
  [user="Username to connect to the database"]
```

```
  [password="Password to connect to the database"]
  [dataSource="DataSource"]/>
```

Attribute:

- **`var`**: Variable that contains the specified **`DataSource`**
- **`scope`**: Scope of the variable specified in the var attribute; it can be page, request, session, or application
- **`driver`**: JDBC driver class name
- **`url`**: JDBC URL for database
- **`user`**: Username to connect to the database
- **`password`**: Password to connect to the database
- **`dataSource`**: The **`DataSource`** for the database to be accessed

Example:

```
<sql:setDataSource
  var="dataSource"
  driver="org.acme.sql.driver"
  url="jdbc:msql://localhost/tempDB"
  user="Dan"
  password="pwd"/>
```

sql:query tag

The **`<sql:query>`** tag executes an SQL query statement. It's possible to pass strings of SQL to the tag either through body content or using an SQL attribute.

The syntax is as follows:

```
<sql:query var="variable name"
  [scope="page|request|session|application"]
  [sql="SQL query statement"]
  [startRow="starting row"]
  [maxRows="maximum number of rows"]
  [dataSource="DataSource"]>
```

JSP body (Instead of using the SQL attribute, the SQL query statement can be entered here.):

```
</sql:query>
```

Attribute:

- **`var`**: The variable that contains the result of the SQL query statement
- **`scope`**: Scope of the variable specified in the var attribute; it can be page, request, session, or application
- **`sql`**: The SQL query statement to execute
- **`startRow`**: The starting row for the returned results
- **`maxRows`**: The maximum number of rows to return; the default is no maximum limit
- **`dataSource`**: A **`DataSource`** that was configured by using the **`</sql:setDataSource>`** action

Example:

```
<sql:query var="sqlresult" sql="select * from bookstore  where price <
?"
     dataSource="jdbc:mysql://localhost:3306/customerdb, com.mysql.jdbc.
Driver, root, root">
   <sql:param value="${fromPrice}" />
</sql:query>
```

The result of the query (here, **`sqlresult`**) is of the **`javax.servlet.jsp.jstl.sql.Result`** type. From this, you can retrieve data with core JSTL.

Result information to retrieve:

- **`rows`**: Returns the result of the query as an array of **`SortedMap`** objects
- **`rowsByIndex`**: Returns the result of the query as an array of arrays
- **`columnNames`**: Returns the names of the columns in the result as an **`String`** array
- **`rowCount`**: Returns the number of rows in the cached **`ResultSet`**
- **`limitedByMaxRows`**: Returns true if the query was limited by a maximum row setting

An example of how to use this:

```
<c:forEach var = "row" items = "${sqlresult.rows}">
  <tr>
     <td> <c:out value="${row.fieldName1}"/></td>
     <td> <c:out value="${row.fieldName2}"/></td>
```

```
    <td> <c:out value="${row.fieldName3}"/></td>
    <td> <c:out value="${row.fieldName4}"/></td>
  </tr>
</c:forEach>
```

sql:update tag

The **<sql:update>** executes an SQL update statement specified either via an attribute or embedded in the body of the tag. An update tag can have nested **param** tags to **pass** parameters to the SQL statement.

The syntax is as follows:

```
<sql:update [var="variable name"]
  [scope="page|request|session|application"]
  [sql="SQL update statement"]
  [dataSource="DataSource"]>
  JSP body
</sql:update>
```

Attribute(s):

- **var**: The variable that contains the result of the SQL update statement
- **scope**: Scope of the variable specified in the var attribute; it can be page, request, session, or application
- **sql**: The SQL update statement to execute
- **dataSource**: The **DataSource** for the database to be accessed

Example:

```
<sql:update>
  UPDATE orders SET deliver_date = ?
  WHERE order_id = ?
  <sql:param value="${param.deliver_date}" />
  <sql:param value="${param.order_id}" />
</sql:update>
```

sql:param tag

The **<sql:param>** is a body tag to a query or update tag. It is used to pass parameters to an SQL statement.

The syntax is as follows:

```
<sql:param [value="parameter value"]>
    JSP body
</sql:param>
```

Attribute:

- **value**: No (the parameter value for the SQL statement)

Example:

```
<sql:query sql="select * from bookstore  where price > ?">
   <sql:param value="${lowPrice}"/>
</sql:query>
```

sql:dateParam tag

The **<sql:dateParam>** is a body tag to a query or update tag. It is used to pass **java.util.Date** parameters to an SQL statement.

The syntax is as follows:

```
<sql:dateParam value="date type value"
[type="date|time|timestamp"] />
```

Attribute:

- **value**: The parameter value for the SQL statement; it can be a date, time, or timestamp
- **type**: The type of parameter; it can be date, time, or timestamp

Example:

```
<sql:query sql="select * from employee where join_date > ?">
<sql:dateParam value="${cutoff_date}" type="date"/>
</sql:query>
```

sql:transaction tag

The **<sql:transaction>** tag will have nested SQL tags and cause them to be executed as a transaction. The syntax is as follows:

```
<sql:transaction [dataSource="DataSource"]
       [isolation="read_committed|read_uncommitted|repeatable_
read|serializable"] >
    JSP body
</sql:transaction>
```

Attribute:

- **`dataSource`**: The **`DataSource`** for the database to be accessed
- **`isolation`**: Isolation level for the transaction; it can be **`read_committed`**, **`read_uncommitted`**, **`repeatable_read`**, or
- **`serializable:`** This is an optional attribute that defaults to the **`DataSource`** isolation level

Example:

```
<sql:transaction>
  <sql:update sql="update bookstore set price = price+? where isbn_no =
?">
    <sql:param value="${price_increase}"/>
    <sql:param value="${id}"/>
  </sql:update>
  <sql:update sql="update bookstore set price = price-? where isbn_no =
?">
    <sql:param value="${price_reduction}"/>
    <sql:param value="${id}"/>
  </sql:update>
</sql:transaction>
```

XML tags

The **JSP Standard Tag Library** (**JSTL**) adds essential features to JSP that enable JSP programming without the need for embedded Java code. It is a collection of standard JSP tags that perform several common tasks. This frees us from having to develop custom tags for these tasks, or from using a mix of tags from several organizations to do our work.

The XML JSTL contains tags for interacting with XML data. This includes flow control based on XPath expression, parsing XML data, and transforming XML data using XSLT stylesheet.

Before using the XML JSTL, the following directive needs to be added to the JSP page:

```
<%@ taglib prefix="x" uri="http://java.sun.com/jsp/jstl/xml" %>
```

The equivalent XML syntax used in tag files for this directive is as follows:

```
<jsp:directive.taglib prefix="x" uri="http://java.sun.com/jsp/jstl/xml"
/>
```

x:out tag

The **<x:out>** tag is used to output the result of an expression. It functions like the **<%= ... =>** JSP syntax. The syntax is as follows:

```
<x:out select="XPath expression" [escapeXml="true|false"] />
```

Attribute:

- **select**: The XPath expression to be evaluated; the result of the expression is output to the page
- **escapeXml**: Determines whether the **<**, **>**, **"** and ' characters in the resultant string should be converted to their corresponding character entity codes

Example:

```
<x:forEach select="$doc/customers/customer" >
  <x:out select="firstName" />
</x:forEach>
```

x:parse tag

The **<x:parse>** tag is used to parse XML data specified either via an attribute or in the tag **body**. The syntax is as follows:

```
<x:parse [var="variable name"]
  [varDom="variable name"]
  [scope="page|request|session|application"]
  [scopeDom="page|request|session|application"]
  [doc="XML document"]
  [systemId="system identifier URI"]
  [filter="filter"]>
JSP body content
</x:parse>
```

Attribute:

- **var**: A variable that contains the parsed XML data; the type of this data is implementation dependent; use this or **varDom**
- **varDom**: Variable that contains the parsed XML data; the type of this data is **org.w3c.dom.Document**
- **scope**: Scope of the variable specified in the **var** attribute; it can be page, request, session, or application, and defaults to page if not specified

- **scopeDom**: Scope of the variable specified in the **varDom** attribute; it can be page, request, session, or application, and defaults to page if not specified
- **doc**: XML document to be parsed
- **systemId**: The system identifier URI for parsing the document
- **filter**: The filter to be applied to the source document

Example:

```
<c:import url="customers.xml" var="xml" />
<x:parse doc="${xml}" var="doc" />
```

x:set tag

The **<x:set>** tag sets a variable or a property to the value of an expression. The syntax is as follows:

```
<x:set [var="variable name"]
  [value="expression"]
  [target="JavaBean or Map object name"]
  [property="target property name"]
  [scope="page|request|session|application"] >
  JSP body
</x:set>
```

Attribute:

- **var**: The variable that is set to the value of the expression
- **value**: The expression to be evaluated
- **target**: The target object (JavaBean or Map object) whose property will be set
- **property**: Name of the property to be set in the **target** attribute
- **scope**: Scope of the variable specified in the **var** attribute; it can be page, request, session, or application

Example:

```
<x:set var="counter" value="${counter + 1}" />
```

x:if tag

The **<x:if>** tag evaluates a test condition. If it is true, it processes its body; else, the body is ignored.

The syntax is as follows:

```
<x:if select="XPath expression"
  [var="variable name"]
  [scope="page|request|session|application"]>
JSP body content
</x:if>
```

Attribute:

- **`select`**: If the test condition is true, it processes its body; else, the body is ignored.
- **`var`**: It is the variable that holds the result (true or false) of the test condition evaluation. This variable can be used for future test conditions (depending on the scope of the variable), thus avoiding reevaluation of the test condition.
- **`scope`**: This is the scope of the variable specified in the var attribute. This can be page, request, session, or application.

Example:

```
<x:if select="author = 'Solan'"
  Book is written by Solan.
</x:if>

<x:if select="author = 'Willcox'"
  Book is written by Willcox.
</x:if>
```

x:forEach tag

The **`<x:forEach>`** tag is used for iterating over collections based on XPath expressions.

The syntax is as follows:

```
<x:forEach select="XPath expression"
  [begin="begin index"]
  [end="end index"]
  [step="step size"]
  [var="variable name"]
  [varStatus="status variable name"]>
JSP body content
</x:forEach>
```

Attribute:

- **`select`**: This is the XPath expression to be evaluated.
- **`begin`**: This is the start index for the iteration.
- **`end`**: This is the end index for the iteration.
- **`step`**: This is the size of the index increment while iterating over the collection. The iteration will process every step item of the collection, starting from the begin index (or first item if begin is not specified). For example, if step is 5, the index is incremented by 5 every iteration.
- **`var`**: This is the name of the variable in which the current item in the iteration is stored.
- **`varStatus`**: This is the name of the variable in which the status of the iteration is stored :
  - **`current =>`**: Retrieves the current item in the iteration
  - **`index =>`**: Retrieves the index of the current round of the iteration
  - **`count =>`**: Retrieves the **`count`** of the current round of the iteration
  - **`first =>`**: Returns true for the first item in the iteration
  - **`last =>`**: Returns true for the last item in the iteration
  - **`begin =>`**: Returns the value of the **`begin`** attribute for the associated tag, or null if no **`begin`** attribute was specified
  - **`end =>`**: Returns the value of the **`end`** attribute for the associated tag, or null if no **`end`** attribute was specified
  - **`step =>`**: Returns the value of the **`step`** attribute for the associated tag, or null if no **`step`** attribute was specified

Example:

```
<ul>
  <x:forEach select="${list}" var="item">
    <li><x:out value="${item}"/></li>
  </x:forEach>
</ul>
```

x:choose, x:when, x:otherwise tags

The **`<x:choose>`** action enables you to provide handling for a set of mutually exclusive conditions instead of just one.

The syntax is as follows:

```
<x:choose>
  <x:when select="XPath expression">
    JSP body
  </x:when>
...
  <x:otherwise>
    JSP body
  </x:otherwise>
</x:choose>
```

Unlimited **<x:when>** actions may exist within a **<x:choose>** action to provide handling for a wide range of conditions. The **<x:otherwise>** action may appear only once because it covers all remaining alternatives and must therefore, appear after all **<x:when>** actions.

Attribute:

- **select**: If the test is true, it processes the tags body.

Example:

```
<x:choose>
  <x:when select="author = 'Solan'"
    Book is written by Solan.
  </x:when>
  <x:when select="author = 'Willcox'"
    Book is written by Willcox.
  </x:when>
  <x:otherwise>
    The book writer is unknown.
  </x:otherwise>
</x:choose>
```

x:transform tag

The **<x:transform>** tag conducts a transformation given a source XML document and an XSLT stylesheet.

The syntax is as follows:

```
<x:transform [var="variable name"]
  [scope="page|request|session|application"]
```

```
  [result=""]
  [xml=""]
  [doc=""]
  [xmlSystemId=""]
  [docSystemId=""]
  [xslt=""]
  [xsltSystemId=""]>
  JSP body content
</x:transform>
```

Attributes:

- **var**: Name of the exported scoped variable for the transformed XML document; the type of the scoped variable is **org.w3c.dom.Document**
- **scope**: Scope of the variable specified in the **var** attribute; it can be page, request, session, or application
- **result**: Result Object that captures or processes the transformation result
- **xml**: Deprecated; use the **doc** attribute instead
- **doc**: Source XML document to be transformed (If exported by **<x:set>**, it must correspond to a well-formed XML document, not a partial document.)
- **xmlSystemId**: Deprecated; use the **docSystemId** attribute instead
- **docSystemId**: The system identifier (URI) for parsing the XML document
- **xslt**: **javax.xml.transform.Source** Transformation stylesheet as a String, Reader, or Source object
- **xsltSystemId**: The system identifier (URI) for parsing the XSLT stylesheet

Example:

```
<x:transform xml="${books}" xslt="${xslt}"/>
```

```
10.4.8x:param tag
```

The **<x:param>** tag is used along with the transform tag to add a parameter to the transformer. The syntax is as follows:

```
<x:param name="parameter name"
    [value="parameter value"]>
  JSP body content
</x:param>
```

Attribute:

- **`name`**: The name of the parameter
- **`value`**: The value for the parameter

Example:

```
<x:transform xml="${doc}" xslt="${styleSheet}">
  <param name="id" value="${idvalue}" />
</x:transform>
```

JSTL functions

The JSP Standard Tag Library (JSTL) adds essential features to JSP that enable JSP programming without the need for embedded Java code. It is a collection of standard JSP tags that perform several common tasks. This frees us from having to develop custom tags for these tasks, or from using a mix of tags from several organizations to do our work.

The Function JSTL contains a number of string manipulation functions. These include functions for splitting and concatenating strings, returning substrings, and determining whether a string contains a specified substring.

Before using the function JSTL, the following directive needs to be added to the JSP page:

```
<%@ taglib prefix="fn" uri="http://java.sun.com/jsp/jstl/functions" %>
```

The equivalent XML syntax used in tag files for this directive is as follows:

```
<jsp:directive.taglib prefix="fn" uri="http://java.sun.com/jsp/jstl/
functions" />
```

fn:contains() function

The **`fn:contains()`** function checks whether the given string is present in the input as substring. It does a case-sensitive check, which means it considers the case while checking for the substring.

Syntax:

```
booleanfn:contains(String inputstring, String checkstring)
```

The return type of this function is boolean. It returns true when the **`checkstring`** is present in **`inputstring`**, else it returns false. It has two string arguments – first one has **`input`** string and second argument has the string that needs to be checked in input string.

Example:

```
<%@ tagliburi="http://java.sun.com/jsp/jstl/core" prefix="c" %>
<%@ tagliburi="http://java.sun.com/jsp/jstl/functions" prefix="fn" %>
<%@page contentType="text/html" pageEncoding="UTF-8"%>
<!DOCTYPE html>
<html>
<head>
<meta http-equiv="Content-Type" content="text/html; charset=UTF-8">
<title>fn:contains example </title>
</head>
<body>
<h3>fn:contains example </h3>
<c:setvar="studentid" value="18MSIT001"/>
<c:setvar="programname" value="MSIT" />
<c:if test="${fn:contains(studentid, programnames)}">
<c:out value="M.Sc(IT) Student"/>
</c:if>
<c:if test="${!fn:contains(studentid, programnames)}">
<c:out value="Student from Other Program."/>
</c:if>
</body>
</html>
```

***Figure 10.37**: Using function contains with arguments*

The following output is generated from the previous code:

***Figure 10.38**: The pattern is not contained, so the message is not displayed*

The pattern is contained, so the message is displayed

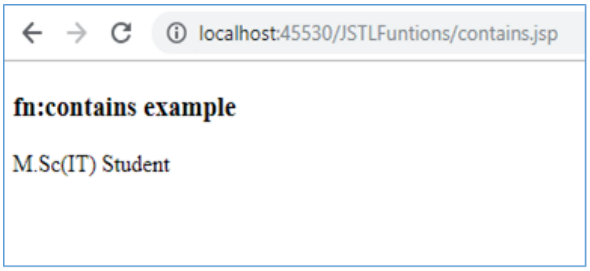


***Figure 10.39**: The pattern is contained, so the message is displayed*

Using multiple **`if`** tags

```
<%@ tagliburi="http://java.sun.com/jsp/jstl/core" prefix="c" %>
<%@ tagliburi="http://java.sun.com/jsp/jstl/functions" prefix="fn" %>
<%@page contentType="text/html" pageEncoding="UTF-8"%>
<!DOCTYPE html>
<html>
<head>
<meta http-equiv="Content-Type" content="text/html; charset=UTF-8">
<title>fn:contains example </title>
</head>
<body>
<h3>fn:contains example </h3>
<c:setvar="studentid" value="18MCAL001"/>
<c:setvar="programname" value="MSIT" />
<c:if test="${fn:contains(studentid, programname)}">
<c:out value="M.Sc(IT) Student"/>
</c:if>
<c:if test="${!fn:contains(studentid, programname)}">
<c:out value="Student from Other Program."/>
</c:if>
</body>
</html>
```

***Figure 10.40**: Using multiple if tags*

The following output is generated from the previous code:

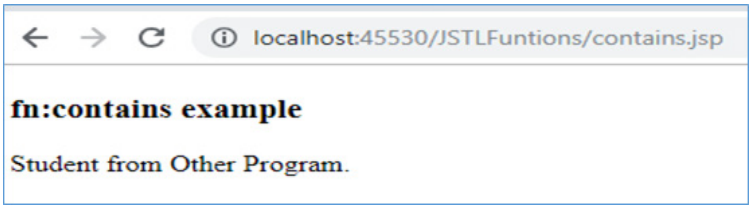


***Figure 10.41**: The pattern is not contained, so the new message is displayed*

fn:containsIgnoreCase() function

The **`fn:containsIgnoreCase()`** function is used to test whether an **`input`** string contains the specified substring as a case insensitive way. While searching the specified substring, it ignores the case.

Syntax:

```
booleancontainsIgnoreCase(java.lang.String, java.lang.String)
```

Example:

```
<%@ tagliburi="http://java.sun.com/jsp/jstl/core" prefix="c" %>
<%@ tagliburi="http://java.sun.com/jsp/jstl/functions" prefix="fn" %>
<%@page contentType="text/html" pageEncoding="UTF-8"%>
<!DOCTYPE html>
<html>
<head>
<meta http-equiv="Content-Type" content="text/html; charset=UTF-8">
<title>fn:contains example </title>
</head>
<body>
<h3>fn:containsignorecase example </h3>
<c:setvar="studentid" value="18MSIT001"/>
<c:setvar="programname" value="msit" />
<c:if test="${fn:contains(studentid, programname)}">
<c:out value="M.Sc(IT) Student ."/>
</c:if>
<c:if test="${fn:containsIgnoreCase(studentid, programname)}">
<c:out value="M.Sc(IT) Student."/>
</c:if>
</body>
</html>
```

***Figure 10.42**: Using multiple if tags*

The following output is generated from the preceding code:

***Figure 10.43**: The message is displayed*

fn:endsWith() function

The **`fn:endsWith()`** function is used for checking the suffix of a string. It checks whether the given string ends with a particular string. The validation is case sensitive.

Syntax:

```
booleanfn:endsWith(input_string, suffix_string)
```

It validates whether the **`input_string`** ends with **`suffix_string`**. If the test is successful, the function returns true; else, it returns false. The return type of the function is boolean, and it receives both the strings as arguments.

Example:

```
<%@ taglib uri="http://java.sun.com/jsp/jstl/core" prefix="c" %>
<%@ taglib uri="http://java.sun.com/jsp/jstl/functions" prefix="fn" %>
<%@page contentType="text/html" pageEncoding="UTF-8"%>
<!DOCTYPE html>
<html>
<head>
<meta http-equiv="Content-Type" content="text/html; charset=UTF-8">
<title>fn:contains example </title>
</head>
<body>
<h3>fn:containsignorecase example </h3>
<c:set var="emailid1" value="18msit001@charusat.ac.in"/>
<c:set var="emailid2" value="18msit001@gmail.com"/>
<c:set var="universityemail" value="@charusat.ac.in" />
<h4>
<c:if test="${fn:endsWith(emailid1, universityemail)}">
<c:out value="${emailid1} is University email ID."/>
</c:if>
<c:if test="${!fn:endsWith(emailid1, universityemail)}">
<c:out value="${emailid1} is Personal email ID."/>
</c:if>
</h4>
<br/>
<h4>
<c:if test="${fn:endsWith(emailid2, universityemail)}">
<c:out value="${emailid2} is University email ID."/>
</c:if>
<c:if test="${!fn:endsWith(emailid2, universityemail)}">
<c:out value="${emailid2} is Personal email ID."/>
</c:if>
</h4>

</body>
</html>
```

***Figure 10.44**: Using multiple if tags*

The following output is generated from the preceding code:

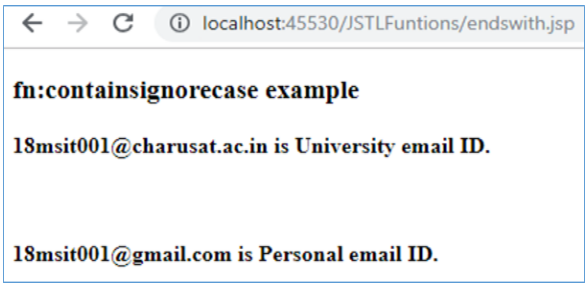


***Figure 10.45**: The personal as well as university ID is displayed*

fn:escapeXml() function

The **`fn:escapeXml()`** JSTL function is used for HTML/XML character escaping, which means it treats HTML/XML tags as a string rather than markup tags. It is similar to the **`escapeXml`** attribute of the **`<c:out>`** tag. Let's understand this with the help of an example.

Syntax:

```
String escapeXml(String input_string)
```

This converts the **`input_string`** into an output string after escaping HTML/XML markup tags.

Example:

```
<%@ taglibуri="http://java.sun.com/jsp/jstl/core" prefix="c" %>
<%@ taglibуri="http://java.sun.com/jsp/jstl/functions" prefix="fn" %>
<html>
<head>
<title>fn:escapeXml() example</title>
</head>
<body>
<b> Hello Students.</b> :  ${fn:escapeXml("<b>Hello Students.</b>")}
<br/>
<i>Welcome to the world of programming.</i> :${fn:escapeXml("<i>Welcome to the world of programming.</i>")}
</body>
</html>
```

***Figure 10.46**: Using multiple if tags*

The following output is generated from the preceding code:

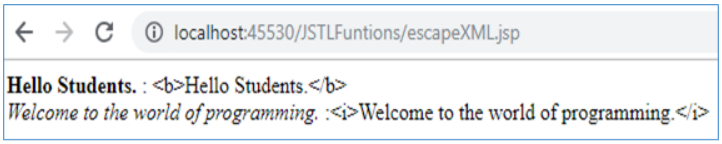


***Figure 10.47**: The executed tags and printed tags are displayed*

fn:indexOf()

The **`fn:indexOf()`** function is used for determining the start position (index) of a string in the provided string.

Syntax:

```
intindexOf(String,  String )
```

The return type of this function is int. It returns the starting position (or index) of the second string (second argument of the function) in the first string (first argument of the function).

The function returns -1 when the string is not found in the **`input`** string. It is case sensitive and treats uppercase and lowercase characters of the same letter as different. It returns the index of the first occurrence, which means if the string is present in input more than once, then the function would return the index of the first occurrence.

Example:

```
<%@ tagliburi="http://java.sun.com/jsp/jstl/core" prefix="c" %>
<%@ tagliburi="http://java.sun.com/jsp/jstl/functions" prefix="fn" %>
<html>
<head>
<title>fn:indexOf() example</title>
</head>
<body>
<h3>Welcome to the <span style="color:blue">world </span>of Programming  :
${fn:indexOf("Welcome to the world of Programming", "world")} </h3><br/>
<h3>Programing using java <span style="color:blue">,</span>php, and c# world of Programming
${fn:indexOf("Programing using java, php, and c#", ",")} </h3><br/>
<h3>Learn to know <span style="color:blue">errors</span> for better programming :
    ${fn:indexOf("Learn to know errors for better programming", "Errors")} </h3>
</body>
</html>
```

***Figure 10.48**: Using the indexOf function*

The following output is generated from the preceding code:

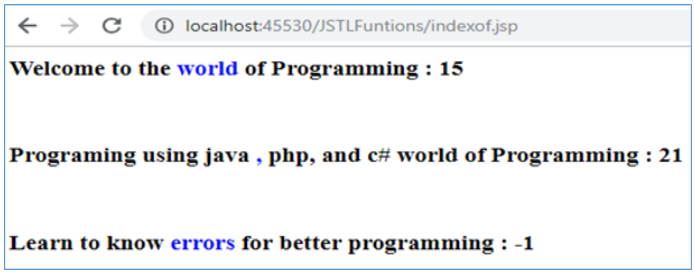


***Figure 10.49**: Displays the index of "world", "," and "error"*

fn:join()

It concatenates the strings with a given separator and returns the output string.

Syntax:

```
String fn:join(String arrayofstrings, String separator)
```

It concatenates all the elements of the input array, along with the provided separator in between. The return type of this function is **`String`**; it returns the output string after concatenation.

Example:

```
<%@ taglibUri="http://java.sun.com/jsp/jstl/core" prefix="c" %>
<%@ taglibUri="http://java.sun.com/jsp/jstl/functions" prefix="fn" %>
<html>
<head>
<title>fn:join() example</title>
</head>
<body>
<h3>
<%
String l[]={"Java", "Php", "Python", "C#"};
session.setAttribute("languages", l);
%>
Programming Languages known : ${fn:join(languages, " & ")}
</h3>
</body>
</html>
```

***Figure 10.50**: Using the join function*

The following output is generated from the preceding code:

***Figure 10.51**: All the values of array are joined with the symbol "&"*

fn:length()

`fn:length()` is used for computing the length of a string or to determine the number of elements in a collection.

Syntax:

```
int length(Object)
```

The return type of this function is **`int`**. It returns the length of the object provided in it as an argument.

Example:

```
<%@ taglib uri="http://java.sun.com/jsp/jstl/core" prefix="c" %>
<%@ taglib uri="http://java.sun.com/jsp/jstl/functions" prefix="fn" %>
<html>
<head>
<title>fn:length() example</title>
</head>
<body>
<c:set var="str1" value="Hello Students."/>
<c:set var="str2" value="This is programming session using java."/>
<c:set var="str3" value="Enjoy working with java."/>
<h3>Length of Strings
Length of "<span style="color:blue;text-decoration:underline">${str1}</span>"  is : ${fn:length(str1)}<br/>
Length of "<span style="color:blue;text-decoration:underline">${str2}</span>" is : ${fn:length(str2)}<br/>
Length of "<span style="color:blue;text-decoration:underline">${str3}</span>"  is : ${fn:length(str3)} <br/>
</h3>
</body>
</html>
```

***Figure 10.52**: Using multiple if tags*

The following output is generated from the preceding code:

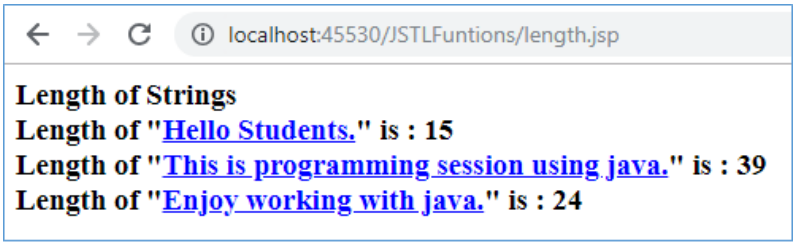


***Figure 10.53**: Displays the length of three strings*

fn:replace()

It searches for a string in the input and replaces it with the provided string. The following is the basic syntax of the **`fn:replace()`** function.

Syntax:

```
String fn:replace(String input, String search_for, String replace_with)
```

The arguments and return type is **`String`**. It searches the **`earch_for`** string in input and replaces it with the **`replace_with`** string. If the string is not found, it returns the actual input as is. It does case sensitive processing.

Example: To greet students based on the current time

```
<%@ tagliburi="http://java.sun.com/jsp/jstl/core" prefix="c" %>
<%@ tagliburi="http://java.sun.com/jsp/jstl/functions" prefix="fn" %>
<%@page import="java.util.Date" %>
<html>
<head>
<title>fn:replace() example</title>
</head>
<body>
<c:setvar="msg" value="Hello Students Good Morning"/>
<%
      Date d= new Date();
int hour=d.getHours();
      String greeting="";

      if(hour>=4 && hour<=11)
          greeting="Good Morning";
      else if(hour>=12 && hour<=16)
          greeting="Good Afternoon";
      else if(hour>=17 && hour<=22)
          greeting="Good Evening";
      else
          greeting="Good Night";
request.setAttribute("greeting",greeting);
%>

${fn:replace(msg, "Good Morning", greeting)};
</body>
</html>
```

***Figure 10.54**: Using the replace function*

The following output is generated from the preceding code:

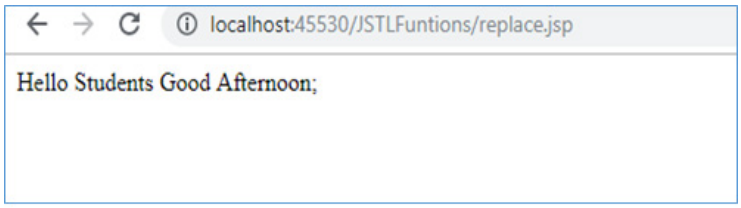


***Figure 10.55**: Displays the greeting message based on the time of execution of the program*

fn:split()

It splits a given string into an array of substrings. Splitting process considers the delimiter string that we provide during function call, that is, we provide the string and delimiter as arguments to the function, and it returns the array of strings after splitting the input based on the delimiter string.

Syntax:

```
String[] fn:split(String inputstring, String delimiterstring)
```

The function returns an array of string, and based on the delimiter present in the input string, the array elements are formed.

Example: To split the string into pieces based on the space delimiter

```
<%@ tagliburi="http://java.sun.com/jsp/jstl/core" prefix="c" %>
<%@ tagliburi="http://java.sun.com/jsp/jstl/functions" prefix="fn" %>
<%@page import="java.util.Date" %>
<html>
<head>
<title>fn:replace() example</title>
</head>
<body>
<c:setvar="msg" value="Hello Students Welcome to the world of programming."/>
<c:setvar="arr" value="${fn:split(msg,' ')}"/>
<ul>
<c:forEachvar="i" items="${arr}">
<li>${i}</li>
</c:forEach>
</ul>
</body>
</html>
```

***Figure 10.56**: Using the replace function*

The following output is generated from the preceding code:

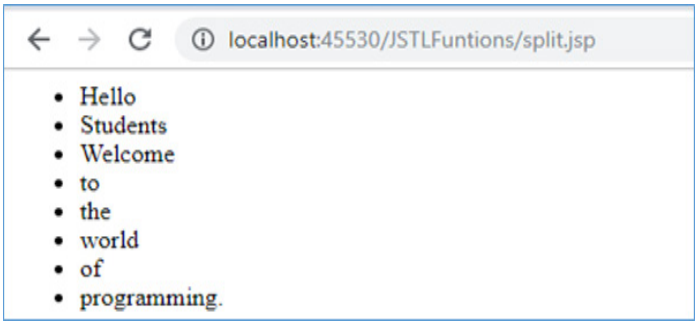


***Figure 10.57**: Splits the string on space character and displays the tokes in list*

fn:startsWith()

It checks whether the given string starts with a particular string value.

Syntax:

```
booleanfn:startsWith(String input, String prefix)
```

This function returns a boolean value. It returns true when the string starts with the given prefix; else, it returns false.

Example: To check whether the message starts with **`Hello`** or **`Hi`**

```
<%@ tagliburi="http://java.sun.com/jsp/jstl/core" prefix="c" %>
<%@ tagliburi="http://java.sun.com/jsp/jstl/functions" prefix="fn" %>
<%@page import="java.util.Date" %>
<html>
<head>
<title>fn:startswith() example</title>
</head>
<body>
<c:setvar="msg" value="Hello Students Welcome to the world of programming."/>
<c:setvar="b1" value="${fn:startsWith(msg,'Hi')}"/>
<c:setvar="b2" value="${fn:startsWith(msg,'Hello')}"/>
<c:if test="${b1}">
<c:out value="Message Starts with Hi"/>
</c:if>
<c:if test="${b2}">
<c:out value="Message Starts with Hello"/>
</c:if>
</body>
</html>
```

***Figure 10.58**: Using the startswith function*

The following output is generated from the preceding code:

***Figure 10.59**: Displays that the message starts with "Hello"*

fn:substring()

This function returns a substring of the given input string as per the given start and end position.

Syntax

```
String fn:substring(String inputstring, int start, int end)
```

- **Return type of function**: String
- **inputstring**: The string from which a substring needs to be taken
- **start**: Starting position of substring
- **end**: End position of substring

Example: To retrieve the **`Students`** from the message **`Hello Students welcome to the world of Java Programming`**

```
01. <%@ tagliburi="http://java.sun.com/jsp/jstl/core" prefix="c" %>
02. <%@ tagliburi="http://java.sun.com/jsp/jstl/functions" prefix="fn" %>
03. <html>
04. <head>
05. <title>fn:substring() example</title>
06. </head>
07. <body>
08. <c:setvar="msg" value="Hello Students welcome to the world of Java Programming"/>
09. <h2><c:out value="${msg}"/></h2>
10. <h3>Sub String function :${fn:substring(msg, 6, 13)} </h3>
11. </body>
12. </html>
```

***Figure 10.60**: Using the substring function*

The following output is generated from the preceding code:

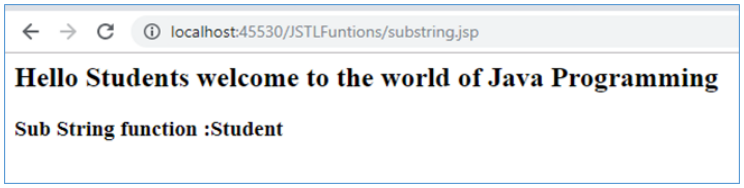


***Figure 10.61**: Display the sub string "Student" from the entire string*

fn:substringAfter()

It returns the part of a given string that lies after a provided string value.

Syntax:

```
String fn:substringAfter(String input, String afterstring)
```

Whatever is present in the input after the **`afterstring`** is being returned by this function.

Example: The string after the keyword **`using`** is returned

```
<%@ taglibуri="http://java.sun.com/jsp/jstl/core" prefix="c" %>
<%@ taglibуri="http://java.sun.com/jsp/jstl/functions" prefix="fn" %>
<html>
<head>
<title>substringAfter() example</title>
</head>
<body>
<c:setvar="coursename" value="Web Development using Advanced Java"/>
<h2><c:out value="${coursename}"/></h2>
<h3> Applying substringAfter function value after "using":
    ${fn:substringAfter(coursename, "using ")}
</h3>
</body>
</html>
```

***Figure 10.62**: Using the substringAfter function*

The following output is generated from the preceding code:

***Figure 10.63**: Display the substring after Advanced Java*

fn:substringBefore()

It is just the opposite of the **`fn:substringAfter`** function. It returns the part of original string that lies before a specified string value.

Syntax:

```
String fn:substringBefore(String input, String beforestring)
```

The part of **`input`** before the **`beforestring`** will be returned as the output of this function.

Example: The string before the keyword **`using`** is returned

```
<%@ tagliburi="http://java.sun.com/jsp/jstl/core" prefix="c" %>
<%@ tagliburi="http://java.sun.com/jsp/jstl/functions" prefix="fn" %>
<html>
<head>
<title>substringBefore() example</title>
</head>
<body>
<c:setvar="coursename" value="Web Development using Advanced Java"/>
<h2><c:out value="${coursename}"/></h2>
<h3> Applying substringBefore function
    ${fn:substringBefore(coursename, "using ")}
</h3>
</body>
</html>
```

***Figure 10.64**: Using the substringBefore function*

The following output is generated from the preceding code:

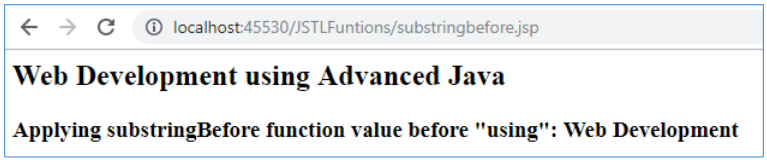


***Figure 10.65**: Display the substring before Web Development*

fn:toLowerCase()

This function converts a string into lower case string. Any upper case character in the input string is replaced with the corresponding lower case character.

Syntax:

```
String fn:toLowerCase(String  input)
```

- **Return type**: **`String;`** Argument: Single argument of **`String`** type. It returns a **`String`** after converting the input string to lower case.

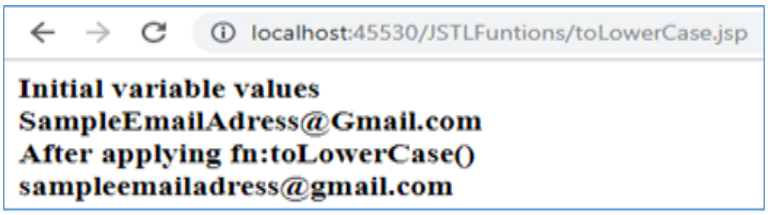


***Figure 10.66**: Convert an email address to lower case*

fn:toUpperCase()

It converts the input string to an upper case string. All the characters of input string get replaced with the corresponding upper case characters. The string that needs to be changed is provided as an argument to the function, and the converted string is returned by the function. We can pass the string as a variable or by simply hardcoding it during function call.

Syntax:

```
String fn:toUpperCase(String input)
```

It returns the **`String`** after converting the input string to upper case.

Example: To convert a string to upper case for a given course code and course name

***Figure 10.67**: Convert a course code and course name to upper case*

fn:trim()

It removes the space characters from start and end of the provided string.

Syntax:

```
String fn:trim(String input)
```

The function returns the string after removing the white spaces from the start and end of the input **`String`**.

Example:

```
01. <%@ tagliburi="http://java.sun.com/jsp/jstl/core" prefix="c" %>
02. <%@ tagliburi="http://java.sun.com/jsp/jstl/functions" prefix="fn" %>
03. <html>
04. <head>
05. <title>fn:trim() example</title>
06. </head>
07. <body>
08. <c:setvar="coursename" value="          Web Development using Advanced Java    "/>
09. <h3>
10.     After applying fn:trim() <br/>
11.             ${fn:trim(coursename)}
12. </h3>
13. </body>
14. </html>
```

***Figure 10.68**: Using the trim function*

The following output is generated from the preceding code:

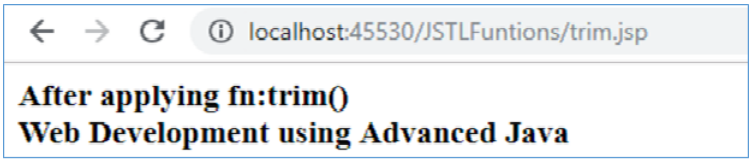


***Figure 10.69**: Removing space before and after string*

Formatting tags

The JSP Standard Tag Library (JSTL) adds essential features to JSP that enable JSP programming without the need for embedded Java code. It is a collection of standard JSP tags that perform several common tasks.

This frees us from having to develop custom tags for these tasks, or from using a mix of tags from several organizations to do our work. The formatting JSTL provides support for internationalization, which is to enable the formatting of data (dates, numbers, time) in different locales.

Before using the formatting JSTL, the following directive needs to be added to the JSP page:

```
<%@ taglib prefix="fmt" uri="http://java.sun.com/jsp/jstl/fmt" %>
```

fmt:formatNumber ()

The **`<fmt:formatNumber>`** tag is flexible and capable of formatting a numeric value in a locale-sensitive or custom format as a number, currency, or percent.

The syntax is as follows:

```
<fmt:formatNumber [value="numeric value"]
  [type="number|currency|percent"]
  [pattern="pattern"]
  [currencyCode="currency code"]
  [currencySymbol="currency symbol"]
  [groupingUsed="true|false"]
  [maxIntegerDigits="max integer digits"]
  [minIntegerDigits="min integer digits"]
  [maxFranctionDigits="max fractional digits"]
  [minFractionDigits="min fractional digits"]
  [var="variable name"]
```

```
  [scope="page|request|session|application"]>
  JSP body
</fmt:formatNumber>
```

Attributes:

- **`value`**: The numeric value to be formatted
- **`type`**: Specifies whether the numeric value is to be formatted as a number, currency, or a percent
- **`pattern`**: Patterns as specified by the **`java.text.DecimalFormat`** class
- **`currencyCode`**: Currency codes as defined by the ISO 4217 standard; this is applicable only when the type attribute is currency
- **`currencySymbol`**: This is applicable only when the type attribute is currency; it is used to override the defaults
- **`groupingUsed`**: Specifies whether the output format contains any grouping separators; will be commas, periods or spaces depending on the local language
- **`maxIntegerDigits`**: Maximum number of digits in the integer portion of the formatted output
- **`minIntegerDigits`**: Minimum number of digits in the integer portion of the formatted output
- **`maxFractionDigits`**: Maximum number of digits in the fractional portion of the formatted output
- **`minFractionDigits`**: Minimum number of digits in the fractional portion of the formatted output
- **`var`**: Variable that contains the formatted numeric value as a string
- **`scope`**: Scope of the **`var`** variable; an optional attribute tha can be page, request, session, or application

Various symbols can be used to form the custom pattern, as listed here:

- **`0`**: Digit
- **`#`**: Digit, zero indicates absent
- **`.`**: Decimal separator or monetary decimal separator
- **`-`**: Minus sign
- **`,`**: Grouping separator

- **E**: Separates mantissa and exponent in scientific notation; need not be quoted in prefix or ssuffix
- **;**: Separates positive and negative subpatterns
- **%**: Multiply by 100 and show as percentage
- **\u2030**: Multiply by 1000 and show as per mille value
- **\u00A4**: Currency sign, replaced by currency symbol; replaced by international currency symbol if doubled; if present in a pattern, the monetary decimal separator is used instead of the decimal separator
- **'**: Used to quote special characters in a prefix or suffix, for example, '**#'#** formats 123 to **#123**; to create a single quote itself, use two in a row: **# o''clock**

Example:

```
<%@page contentType="text/html" pageEncoding="UTF-8"%>
<%@ taglib prefix="c" uri="http://java.sun.com/jsp/jstl/core"%>
<%@ taglib prefix="fmt" uri="http://java.sun.com/jsp/jstl/fmt"%>
<!DOCTYPE html>
<html>
<head>
<meta http-equiv="Content-Type" content="text/html; charset=UTF-8">
<title>formatNumber Tag </title>
</head>
<body>
<c:setvar="cost" value="5244.234" />
<h3> Minimum fraction digits 3:
<fmt:formatNumber type="number" minFractionDigits="3" value="${cost}"/><br/>
    Percent with maximum integer digits 2:
<fmt:formatNumber type="percent" maxIntegerDigits="2" value="${cost}"/><br/>
    Using pattern:
<fmt:formatNumber type="number" pattern="#.###E0" value="${cost}"/><br/>
    Using variable and maximum integer digits 3 :
<fmt:formatNumber type="number" maxIntegerDigits="3" value="${cost}" var="value" scope="page"/>
    ${value}
</h3>
</body>
</html>
```

***Figure 10.70**: Using the formatNumber function*

The following output is generated from the preceding code:

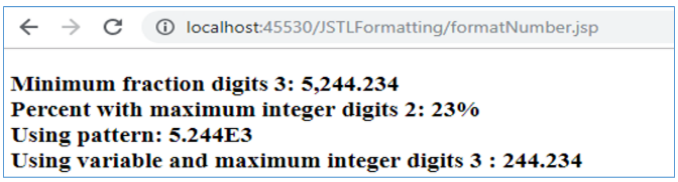


***Figure 10.71**: Displaying different values using the formatNumber tag*

fmt:parseNumber

This tag parses a string representation of a number, currency, or a percent specified either via an attribute or in its body content.

The syntax is as follows:

```
<fmt:parseNumber [value="string representation"]
  [type="number|currency|percent"]
  [pattern="pattern"]
  [parseLocale="locale"]
  [integerOnly="true|false"]
  [var="variable name"]
  [scope="page|request|session|application"]>
  JSP body content
</fmt:parseNumber>
```

Attribute

- **`value`**: The string to be parsed
- **`type`**: Specifies whether the value is to be parsed as a number, currency, or a percent
- **`pattern`**: Custom formatting pattern; patterns are specified by the **`java.text.DecimalFormat`** class
- **`parseLocale`**: Locale to be used for parsing
- **`integerOnly`**: Specifies that only the integer part needs to be parsed
- **`var`**: Variable that contains the parsed numeric value
- **`scope`**: Scope of the **`var`** variable; an optional attribute that can be page, request, session, or application

Various symbols discussed in the previous sectionformat and parse number can be used to create custom patterns.

Example:

```
<%@page contentType="text/html" pageEncoding="UTF-8"%>
<%@ taglib prefix="c" uri="http://java.sun.com/jsp/jstl/core"%>
<%@ taglib prefix="fmt" uri="http://java.sun.com/jsp/jstl/fmt"%>
<!DOCTYPE html>
<html>
<head>
<meta http-equiv="Content-Type" content="text/html; charset=UTF-8">
<title>parseNumber Tag </title>
</head>
<body>
<c:setvar="budget" value="1628.31610" />
<h3>
<fmt:parseNumbervar="parsedNumber" type="number" value="${budget}" />
<p>Parsed budget using value :<strong><c:out value="${parsedNumber}" /></strong></p>
<fmt:parseNumbervar="parsedNumber" integerOnly="true" type="number" >33022.660</fmt:parseNumber>
<p>Parsed budget using body :<strong><c:out value="${parsedNumber}" /></strong></p>
</h3>
</body>
</html>
```

***Figure 10.72**: Using the integerOnly function*

The following output is generated from the preceding code:

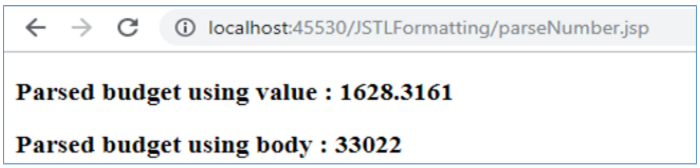


***Figure 10.73**: Displaying parsed budget*

fmt:formatDate

This tag provides flexible, time zone-aware formatting of Java. The syntax is as follows:

```
<fmt:formatDate value="string representation"
                [type="date|time|both"]
                [dateStyle="formatting style"]
                [timeStyle="formatting style"]
                [pattern="formatting pattern"]
                [timeZone="timezone"]
                [var="variable name"]
                [scope="page|request|session|application"] />
```

The **`formatDate`** tag formats date and time data according to a specified style and pattern.

Attributes

- **`value`**: The date/time to be formatted
- **`type`**: Specifies whether the data to be formatted contains a date, time, or both
- **`dateStyle`**: Formatting style used for dates; valid style values are defined by the **`java.text.DateFormat`** class (that is, default, short, medium, long, and full); valid only if the type attribute is date or both
- **`timeStyle`**: Formatting style used for time values; valid style values are defined by the **`java.text.DateFormat`** class (that is, default, short, medium, long, and full); valid only if the type attribute is time or both
- **`pattern`**: Custom formatting pattern for dates and times; patterns are specified by the **`java.text.SimpleDateFormat`** class
- **`timeZone`**: Time zone to be used for the time information
- **`var`**: Variable containing the parsed date/time value; variable is of type **`java.util.Date`**
- **`scope`**: Scope of the var variable; an optional attribute that can be page, request, session, or application

Various symbols are used to form a pattern for date/time component:

G: Era designator

y: Year

Y: Week year, A week year can be different from the calendar year depending on which day the first of January falls.

M: Month in year

w: Week in year

W: Week in month

D: Day in year

d: Day in month

F: Day of week in month

E: Day name in week

u: Day number of week (1 = Monday, ..., 7 = Sunday)

a: Am/pm marker

H: Hour in day (0-23)

k: Hour in day (1-24)

K: Hour in am/pm (0-11)

h: Hour in am/pm (1-12)

m: Minute in hour

s: Second in minute

S: Millisecond

z: Time zone - General time zone

Z: Time zone - RFC 822 time zone

X: Time zone - ISO 8601 time zone

Example:

```
<!DOCTYPE html>
<html>
<head>
<title>JSTL Formatting Tags<</title>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
</head>
<body>
<h1> JSTL Formatting Functions </h1>
<h3><a href="formatNumber.jsp">fmt:formatNumber example </a></h3>
<h3><a href="parseNumber.jsp">fmt:parseNumber example </a></h3>
<h3><a href="formatDate.jsp">fmt:formatDate example </a></h3>
</body>
</html>
```

***Figure 10.74**: Using the formatNumber function*

The following output is generated from the preceding code:

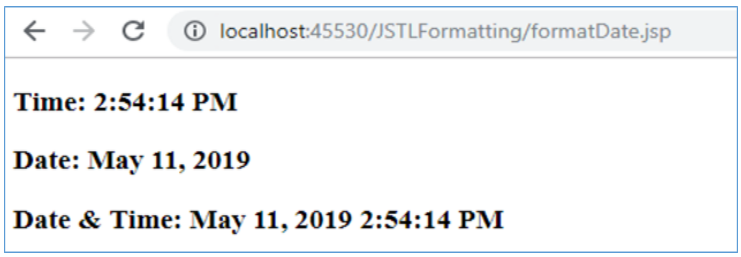


***Figure 10.75**: Display date and time in different formats*

fmt:parseDate

This tag parses string representations of date and time specified either via an attribute or in its body content. The syntax is as follows:

```
<fmt:parseDate [value="string representation"]
  [type="date|time|both"]
  [dateStyle="formatting style"]
  [timeStyle="formatting style"]
  [pattern="formatting pattern"]
  [timeZone="timezone"]
  [parseLocale="locale"]
  [var="variable name"]
  [scope="page|request|session|application"]>
  JSP body
</fmt:parseDate>
```

Attribute

- **`value`**: This is the string to be parsed.
- **`type`**: It specifies whether the value is to be parsed; contains a date, time, or both.
- **`dateStyle`**: The valid style values are defined by the **`java.text.DateFormat`** class. (that is, default, short, medium, long, and full). It is valid only if the type attribute is date or both.
- **`timeStyle`**: The valid style values are defined by the **`java.text.DateFormat`** class (that is, default, short, medium, long, and full). It is valid only if the type attribute is time or both.
- **`pattern`**: It specifies the custom formatting pattern. The patterns are as specified by the **`java.text.DecimalFormat`** class.
- **`timeZone`**: It is the time zone to be used for the time information.
- **`parseLocale`**: It is the locale to be used for formatting date and time values.
- **`var`**: It is the variable that contains the parsed date/time value. This variable is of type **`java.util.Date`**.
- **`scope`**: It gives the scope of the **`var`** variable. This is an optional attribute and can be page, request, session, or application.

Various symbols discussed in the previous section to format and parse dates, can be used to create custom patterns using various symbols.

Example:

```
<%@page contentType="text/html" pageEncoding="UTF-8"%>
<%@ taglib prefix="c" uri="http://java.sun.com/jsp/jstl/core"%>
<%@ taglib prefix="fmt" uri="http://java.sun.com/jsp/jstl/fmt"%>
<!DOCTYPE html>
<html>
<head>
<meta http-equiv="Content-Type" content="text/html; charset=UTF-8">
<title>parseNumber Tag </title>
</head>
<body>
<c:setvar="today" value="01-01-2019" />
<h3>Parsing date, ${today}, to:
<fmt:parseDate pattern="dd-MM-yyyy" var="thedate" value="${today}" />
<fmt:formatDate type="both" value="${thedate}" />
</h3>
</body>
</html>
```

***Figure 10.76**: Using the parseDate function*

The following output is generated from the preceding code:

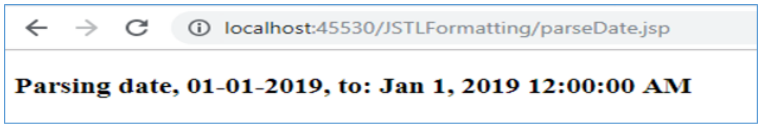


***Figure 10.77**: Parsing date as per requirement*

fmt:bundle and fmt:setbundle

This tag loads a resource bundle for the body of the tag. The body usually contains a nested **<fmt:message>** tag to display a localized message.

The syntax is as follows:

```
<fmt:bundlebasename="resource bundle basename"
            [prefix="message key prefix"]>
            JSP body
</fmt:bundle>
```

Attribute

- **basename**: The basename of the resource bundle
- **prefix**: The prefix to be applied to the message key for any nested **<fmt:message>** tags

The **<fmt:setBundle>** tag loads a resource bundle for a specified scope. The syntax is as follows:

```
<fmt:setBundlebasename="resource bundle basename"
[var="Localization context variable"]
[scope="page|request|session|application"] />
```

Attribute(s)

- **basename**: The **basename** of the resource bundle
- **var**: The name of the localization context variable
- **scope**: Scope of the **var** variable; it is an optional attribute and can be page, request, session, or application

Example:

```
<fmt:setBundlebasename = "mess.messages" var = "lang"/>
```

Example: To create a list of cities from different states of India in Java and access them using the **bundle** and **message** tags.

Java code to create list:

```
package src;
import java.util.ListResourceBundle;
public class StateList extends ListResourceBundle
{
       public Object[][] getContents()
         {
          return contents;
         }
       static final Object[][] contents = {
{ "state.gujarat", "Ahmedabad" },
{ "state.maharastra", "Pune" },
{ "state.HimachalPradesh", "Shimla" },
{ "state.bihar", "Arwal" },
{ "state.goa", "Panji" },
{ "state.kerala", "Calicut" },
        };
}
```

HTML and JSP code to access the values

```
<%@page contentType="text/html" pageEncoding="UTF-8"%>
<%@ taglib prefix="c" uri="http://java.sun.com/jsp/jstl/core"%>
<%@ taglib prefix="fmt" uri="http://java.sun.com/jsp/jstl/fmt"%>
<!DOCTYPE html>
<html>
<head>
<meta http-equiv="Content-Type" content="text/html; charset=UTF-8">
<title>bundle Tag </title>
</head>
<body>
<h3>
          List of Cities from different states of India<br/>
<fmt:setBundlebasename="src.StateList" var="s"/>
<fmt:message key="state.gujarat" bundle="${s}"/><br/>
<fmt:message key="state.maharastra" bundle="${s}"/><br/>
<fmt:message key="state.HimachalPradesh" bundle="${s}"/><br/>
<fmt:bundlebasename="src.StateList">
<fmt:message key="state.bihar"/><br/>
<fmt:message key="state.goa" /><br/>
<fmt:message key="state.kerala"/><br/>
</fmt:bundle>
</h3>
</body>
</html>
```

***Figure 10.78**: Using the setBundlebasename.tag*

The following output is generated from the previous code:

***Figure 10.79**: Display data from different bundles*

fmt:setLocale

The **`<fmt:setLocale>`** tag sets the locale value in the locale configuration variable. The syntax is as follows:

```
<fmt:setLocale value="Locale string"
  [variant="Vendor or browser specific variant"]
  [scope="page|request|session|application"]
/>
```

Attribute(s)

- **value**: The name of the character encoding for the request; examples of request encodings include UTF-8 and ISO-8859-1
- **variant**: This is a vendor/browser-specific value; examples of the variants are WIN (for Windows), MAC (for Macintosh), and POSIX (for POSIX)
- **scope**: Scope of the variable specified in the **var** attribute; it can be page, request, session, or application

Example:

```
<fmt:setLocale value = "es_ES"/>
```

fmt:timeZone and fmt:setTimeZone

The JSTL **<fmt:timeZone>** formatting tag is used to specify the time zone.

Syntax: **<fmt:timeZone value=”timeZone” />**

Attributes

- **value**: It specifies the time zone to apply to the body.

The **<fmt:setTimeZone>** tag sets the time zone value in the time zone configuration variable for a specified scope. The syntax is as follows:

```
<fmt:setTimeZone value="Time Zone string"
  [var="time zone variable"]
  [scope="page|request|session|application"]
  />
```

Attribute

- **value**: Name of the time zone
- **var**: Name of the time zone variable
- **scope**: Scope of the variable specified in the **var** attribute; it can be page, request, session, or application

Example: To display various time zones

```
<%@ tagliburi="http://java.sun.com/jsp/jstl/core" prefix="c" %>
<%@ taglib prefix="fmt" uri="http://java.sun.com/jsp/jstl/fmt" %>
<html>
 <head>
  <title>fmt:timeZone JSTL formatting tag example</title>
 </head>
 <body>
 <c:setvar="currentDate" value="<%=new java.util.Date()%>"/>
 <table>
 <c:forEachvar="timeZone"
                        items="<%=java.util.TimeZone.getAvailableIDs()%>">
   <tr>
   <td>
   <c:out value="${timeZone}" />
   </td>
   <td>
   <fmt:timeZone value="${timeZone}">
   <fmt:formatDate
 value="${currentDate}" type="both" />
   </fmt:timeZone>
   </td>
   </tr>
  </c:forEach>
  </table>
 </body>
</html>
```

***Figure 10.80**: Using date and time related tags*

The following output is generated from the previous code:

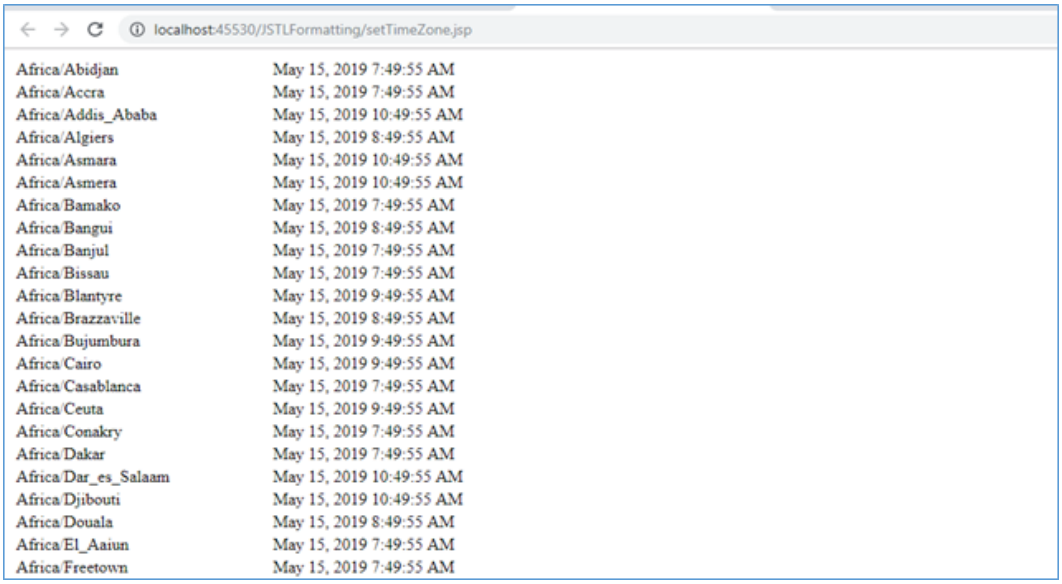


***Figure 10.81**: Display different time zones*

fmt:message

The **<fmt:message>** tag displays a localized message. The body can contain a nested **fmt:param** tag to pass parameterized values to the message.

The syntax is as follows:

```
<fmt:message [key="message key"]
    [bundle="localization bundle where key is looked up"]
    [var="variable that stores localized message"]
    [scope="page|request|session|application"]>
    JSP body
</fmt:message>
```

Attribute

- **key**: The message key corresponding to the message
- **bundle**: The localization context in which the message key is looked up
- **var**: The variable that stores the localized message
- **scope**: The scope of the **var** variable; it is an optional attribute and can be page, request, session, or application

Example:

```
<fmt:bundlebasename="mess.messages">
    <fmt:message key="key.welcome"/><br/>
    <fmt:message key="key.description"/>
</fmt:bundle>
```

fmt:requestEncoding

The **<fmt:requestEncoding>** tag is used to specify the character encoding of the request and to decode the incoming forms data entered by the user. It is used if the encoding used is different from the ISO-8859-1.

The syntax is as follows:

```
<fmt:requestEncoding [value="character encoding name"] />
```

Attribute

value: The name of the character encoding for the request. Examples of request encodings include UTF-8 and ISO-8859-1.

Example:

```
<fmt:requestEncoding value="UTF-8"/>
```

Conclusion

The chapter covered the use of JSTL for using different tags and functions provided by the library. The library provides different category of tags for example re tags for doing the Java routine work, functions for performing various tasks, parsing and using XML, using SQL tags to perform database operations, and formatting tags.

In the next chapter, we will cover Java Server Faces for developing user interfaces and performing web programming.

Points to remember

- Core tag library supports variables, flow control, URL management.
- XML tag library supports core, flow control, and transformation.
- I18N tag library supports locale, message formatting, and number and date formatting.
- Database tag library supports SQL connection and writing queries.
- Functions tag library supports message formatting, number and date formatting, etc.

Multiple choice questions

1. **What is the full form of JSTL?**
    a. Java Standard Tag Library
    b. JSP Standard Tag Library
    c. JSON Standard Tag Library
    d. None of these

2. **Basic functionalities of language, like looping, importing data from external sources, etc., are provided by which tag library?**
    a. Core
    b. XML
    c. SQL
    d. Functions

3. **Consider the string "Hello Students"; which tag can be used to separate each word from the string?**
    a. c:forEach
    b. c:forTokens
    c. c:choose
    d. c:set

4. **The _____ tag displays a localized message.**
    a. fmt:requestEncoding
    b. fmt:timeZone
    c. fmt:message
    d. fmt:setLocale

Answers

1. b **2. a** **3. b** **4. c**

Questions

1. Explain data sharing in application scope with an example.
2. What is JSTL?
3. Explain the **`c:out`** tag.
4. Explain the **`c:if`** tag.
5. Explain the **`c:forEach`** tag.
6. Explain the **`c:forTokens`** tag.
7. Explain the **`x:parse`** tag.
8. Explain the **`fmt:bundle`** tag.

Key terms

- **JSTL**: It is a collection of the standard JSP tags that includes the core functionality common to control the JSP page behavior.
- **taglib**: To use any of the libraries, you must include a **`<taglib>`** directive at the top of each JSP that uses the library.

Join our book's Discord space

Join the book's Discord Workspace for Latest updates, Offers, Tech happenings around the world, New Release and Sessions with the Authors:

https://discord.bpbonline.com

Chapter 11

Outline of JSF for Building Component-Based, Event-Oriented Web Interfaces

Introduction

The chapter explains the importance and requirement of JSF over JSP and servlets. JSF uses web interfaces to bind data dynamically with easy, component-based integration. The chapter also covers web interfaces used in JSF; the separation of concerns using MVC design pattern; and the use of **`PrimeFaces`** and **`DataList`** to handle database, manage event handling in JSF, and understand application of JSF.

Structure

In this chapter, you will learn about the following topics:

- Introduction to JSF
- Web interfaces in JSF
- MVC in JSF
- Managing PrimeFaces and DataList
- Events in JSF
- Applications of JSF

Objectives

After reading this chapter, you should be able to use events in JSF, grasp MVC, and construct dynamic web apps using JSF.

Introduction to JSF

Compared to the technologies available at the time like PHP, ASP, Perl, and even earlier, CGI scripts; Java Servlets was a fairly clean approach. Although it gained a lot of popularity, it began to lose its usefulness as most online applications needed to generate quite big HTML code inside Java.

Due to this, a new industry standard known as **JavaServer Pages** (**JSP**) was developed in 1999. JSP changed the idea of dynamic HTML markup generation inside Java classes to writing Java code using scriptlets inside HTML pages.

JSP and Servlets coexisted and were widely used in applications at all application levels. Most applications were created so that JSP handled dynamic view management while Servlets handled business logic. The model view and controller helped the business logic and the presentation code be separated in the different parts of code.

The technical needs for these applications, however, were numerous and time-consuming for the application developers to manually address. These requirements included separation of concerns for data, business logic, presentation of data, navigating among pages, defining overall pages connection and flow, using various user interface components to increase usability and make validations easier and more manageable. As a result, numerous frameworks and APIs have been developed by vendors, communities, and third-party organizations to address these problems. The most common among them are the Struts framework, the facility of layout management using Tiles, creating highly scalable web applications using Tapestry framework, and the Spring MVC framework.

Sun chose to start from scratch with a new web technology in 2004, which became known as JSF 1.0. Later versions, JSF 1.1 and 1.2, were released in 2006 and included all the functionalities that had become standard for the majority of web applications.

In JSF 1.2, there were performance issues and security vulnerabilities, and IDE integration was not supported, so it was not so common and easily accepted.

JSF 2.0 was released to fix the performance issues with a host of new capabilities and improvements, such as the choice of Facelets as the default template engine and JSP as the default view declaration language; and support for GET requests, AJAX, and many more. Later, JSF 2.1 was made available. The version that is accessible right now is 2.2, which was released in 2013. Many JSF versions were created since its inception, and JFS is now often used for dynamic web applications.

User interface design and web programming are done using **JavaServer Faces** (**JSF**). It is used to create dynamic content for web pages and to perform user input validation and dynamic page refreshing. Although JSP offers built-in support for these activities, making the development process quicker and simpler, all these functionalities can be offered with other technologies like servlets.

Since Java EE 5, JSP has been a part of the specification. JSF features a defined life cycle for processing requests, an expandable component architecture for user interfaces, and a robust event handling system. With the help of these characteristics, Java EE-based apps can use the **Rapid Application Development** (**RAD**) principles that are typical of traditional UI development.

The **Model, View, and Controller** (**MVC**) paradigm is perfectly implemented by JSF technology. For instance, pure servlet and JSP technologies combine the programming (Model and Controller) logic with the HTML presentation (View) logic. This calls for the designer of the application's HTML layout to also be a programmer, or vice versa. The designer of a JSF application uses XML-like elements, the majority of which are analogous to HTML tags and some of their characteristics. The Java programmer creates JavaBeans, which represent the Model and Controller logic. Only a limited selection of bean properties (variables in the source code), which the designer binds to in the designed pages, are provided by the programmer.

JSF is used to create component-based and event-driven web pages to access server-side data and logic. JSF is an XML document that holds the page in hierarchically ordered tags, and JSP is an HTML page with server-side capabilities. The Java objects that support JSF components have full Java capabilities, including access to remote APIs and databases, and are independent of the HTML.

JSF's primary goal is to integrate various technologies like HTML, CSS, JavaScript, etc., which are the most common client-side aspects, so that programmers may create web interfaces with little to no interaction with these platforms.

Web interfaces in JSF

The main concept of JSF is to turn functionality into reusable parts, so JSF components are used for this purpose.

JSF pages can be used within JSP; Facelets are more often used to develop separate JSF pages. Facelets are made specifically for JSF interface definition, and they are XHTML pages. Using Facelets, you may build a component tree that serves as the framework for a JSF user interface by using XML elements.

The key components of a straightforward JSF page created with Facelets are shown in listing 1. In this example, we're using a bean that has been put in scope via CDI to access Java's server-side functionality.

```
<lt;!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Transitional//EN"
"http://www.w3.org/TR/xhtml1/DTD/xhtml1-transitional.dtd">
<lt;html xmlns=http://www.w3.org/1999/xhtml  xmlns:h="http://xmlns.jcp.
org/jsf/html" >
   <lt;h:head>
      <lt;title>Hello JavaWorld!<lt;/title>
   <lt;/h:head>
   <lt;body>
      #{javaBean.content}
   <lt;/body>
```

We can see a typical XHTML page in listing 1. On top of XHTML, a Facelets view is constructed. Another namespace is specified and referenced in addition to the XHTML namespace.

Standard elements for usage in JSF HTML pages may be found in the **h** library.

The **http://xmlns.jcp.org/jsf/html** library defines a collection of JSF components, in this case, a collection of common HTML elements. One of these components is the **<h:head>** element.

HTML components in JSF

The prefix **h** in the **<h:head>** tag references the **jsf/html** library, and then it refers the specific tag within the library, for example, the head tag. The **<h:head>** tag works like the HTML head tag.

Nested tags

In HTML, we can use child tags, for example, we can use the **<title>** tag inside the head tag. Similarly, we can use the child tags nested inside the **<h:head>** tag. In JSP, we can use **${}** as a short way of using expression; in JSF, we use **#{}** to provide JSF expressions. In JSF, we build an XML tree with Facelets that references a component library or libraries, and we then utilize the library's components to convert Java objects into HTML.

Using Java objects in JSF

In listing 1, the JavaBean object is in scope when this markup is evaluated because of the JSF expression (**${javaBean.content**) inside it. The content attribute on the JavaBean object is accessed by the XHTML of Facelets. The result is a web interface that combines Java's server-side data and logic capabilities with Facelets' view structure. We can use JSF expression to retrieve Java application data from a JSF user

interface. JSF builds a component tree that outputs HTML depending on the data present in Java objects using XML tags or annotations. With JSF 2.3, annotations can now be used to specify JSF components instead of XML metadata. Without modifying any XML, a JSF app may be defined and deployed.

Structure of a JSF application

JavaServer Faces needs a common directory structure and metadata, just like JavaServer Pages and the Servlet API. These are distributed in **`.war`** format. The **`.war`** file's structure is comparable to that of a servlet or JSP application. It has a / web-app directory that houses the markup files for the application and a **`/WEB-INF`** directory that displays the metadata for the application.

Glassfish is a Java EE container and can execute JSF, but for server-side Java technologies like JSF and others, Tomcat is a well-liked container.

MVC in JSF

JSF uses the **Model-view-controller** (**MVC**) design pattern. In the MVC approach, the three UI concerns are divided into independent components to make them easier to handle. In general, the controller sets up the model and directs the user to the appropriate view, while the view is in charge of showing data in the model.

The Facelets page with its set of XML tags serves as the view in a JSF implementation. These specify how the user interface is laid out. The use of JSF on the serverside is where Java classes support those UI components. In JSF 2.3, managed bean annotations were replaced by CDI. Developers create a context with CDI and then inject things into it. The annotation syntax is a little different for those who are used to managed beans, but the semantics are exactly the same.

Controller beans

The controller component of the MVC equation is provided by controller beans in JSF 2.3. The model is provided via regular Java objects, also known as POJOs or Plain Old Java Objects. The controller beans' process flow is as follows:

- Select the correct user request
- Prepare the model's POJOs
- Render the Facelets view using the model

The component tree and model are then combined by JSF to produce the HTML output. Listing 2 demonstrates how to use CDI to define the JavaBean object from listing 1; **`cdi-api-1.2.jar`** is assumed to be one of the application's dependencies in this listing.

Listing 2: A JavaBean that has been CDI-defined.

```
import javax.inject.Named;
import javax.enterprise.context.SessionScoped;
@Named
@ViewScoped
public class JavaBean  implements Serializable {
  private String content = ìWelcome to JSF!î
  // getters/setters
}
```

Managing PrimeFaces and DataList

Event-driven messaging, reusable components, and the MVC pattern implementation in JSF are all done using PrimeFaces. Start by launching the PrimeFaces Showcase and selecting **`DataList`** from the left-side column's Data link. This will display the PrimeFaces **`DataList`** demo code.

Figure 11.1 identifies the locations of these samples:

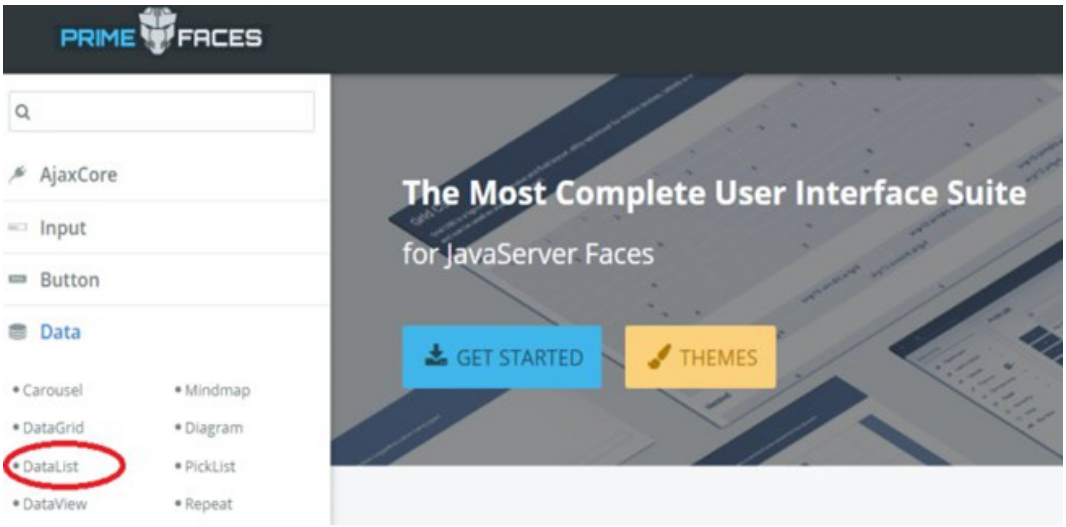


Figure 11.1: *PrimeFaces showcase*

PrimeFaces DataList:

We can access the data model using DataList. The markup for this **`dataList`** presentation is shown in Listing 2. The markup in the **`dataList.xhtml`** tab can be seen if you scroll to the bottom of the PrimeFaces showcase.

Facelet for PrimeFaces DataList, Listing 2.

```
<p:dataList value="#{dataListView.cars1}" var="car" type="ordered">
  <f:facet name="header">
    Basic
  </f:facet>
  #{car.brand}, #{car.year}
</p:dataList>
```

Take note of the **dataList** component's value attribute in listing 2. You can see that this reads the **cars1** attribute on a **dataListView** object and references that object. The model object that field returned will be used by the component. Tokens in JSF employ standard accessors to refer to object properties, so cars1 will refer to the object's **getCars()** getter.

Next, take note of the **var="car"** property. When the **dataList** component iterates through the list of cars returned by the value field, this instructs it which variable to utilize. Although the value property is quite popular, these characteristics are exclusive to the **dataList** component. For parts that iterate over lists, the **var** property is also typical.

You can see that the **vehicle** variable is accessible in the component's body using JSF expressions like {#car.brand}. The **dataListView.cars1** instance will output the **car.brand** property on each iteration.

You'll see that the **< f:facet...>** tag displays how you can alter the way components are displayed. The header is defined as basic in this instance.

Combining the data with the markup allows us to visualize how the Facelets XML will drive this result.

We can use the server-side elements of **DataList,** shown below. The Java class that the markup in listing 3 uses, **DataListView**, is displayed in listing 3:

DataListView class in Listing 3

```
package org.primefaces.showcase.view.data;
import java.io.Serializable;
import java.util.List;
import javax.annotation.PostConstruct;
import javax.inject.Named;
import javax.faces.bean.ManagedBean;
import javax.inject.Inject;
import javax.faces.bean.ViewScoped;
```

```
import org.primefaces.showcase.domain.Car;
import org.primefaces.showcase.service.CarService;

@Named
@ViewScoped
public class DataListView implements Serializable {
    private List<Car> cars1;
    private Car selectedCar;
    @Inject("#{carService}")
    private CarService service;

    @PostConstruct
    public void init() {
        cars1 = service.createCars(10);
    }
    public List getCars1() {
        return cars1;
    }
    public void setService(CarService service) {
        this.service = service;
    }
}
```

Dependency injection and annotations

The **`DataListView`** class has the **`@Named`** annotation by looking at the **`import javax.inject`** a component of JSF. The **`@Named`** annotation informs JSF that the bean belongs to the application. The **`@ViewScoped`** annotation tells JSF that the bean will only exist for the duration of the view. The **`@Inject`** annotation is present on the **`CarService`** property. Another JSF feature that enables beans to be *wired together* is this one. JSF will essentially locate the **`carService`** object inside scope and link it to the service field on the **`DataListView`** object automatically.

Lifecycle methods

The **`@PostConstruct`** annotation is present on the **`init()`** method. Developers can access the JSF component life cycle using the **`@PostConstruct`** annotation, which is a life cycle function. After the component is built but before it is rendered, the **`PostConstruct`** instructs JSF to execute this code. The line **`cars1 = service.`**

`createCars(10);` is used in the **`init()`** method's body to add items to the **`cars1`** list using the **`CarService`** object.

Serving levels

A class that offers services to other classes often does so by offering services to one or more controllers. The concept of a service class is widespread and not exclusive to JSF, much like the MVC design.

Listing 4. The class DataListView

```
package org.primefaces.showcase.service;
import java.util.ArrayList;
import java.util.List;
import java.util.UUID;
import javax.faces.bean.ApplicationScoped;
import javax.inject.Named;
import org.primefaces.showcase.domain.Car;

@Named(name = "carService")
@ApplicationScoped

public class CarService {
    private final static String[] colors;
    private final static String[] brands;
    static {
        colors = new String[10];
        colors[0] = "Black";
        colors[1] = "White";
        // ...other colors...
    }

    public List createCars(int size) {
        List list = new ArrayList();
        for(int i = 0 ; i < size ; i++)
          {
            list.add(new Car(getRandomId(), getRandomBrand(),
getRandomYear(), getRandomColor(), getRandomPrice(),
getRandomSoldState()));
```

```
        }
        return list;
    }
    // Utility methods like getRandomId() have been removed
}
```

First, observe how the code tells JSF that this is a managed class: (**`@Named(name = "carService"`**). The name **`carService`** is assigned to the bean in this instance and is used as a handle to refer to the bean in the **`DataListView`** controller. The class is also designated as **`@ApplicationScoped`**, which essentially indicates that the object instance will continue to exist during the course of the program.

Bean information

To find the beans it requires, JSF leverages the application's metadata, which formerly resided mostly in distinct XML files, the **`/WEB-INF/beans.xml`**.

Events in JSF

JSF is a component-based MVC framework and an event handling framework. This means interactions with users and components are handled using events. By separating event creation code from event handling code, events should offer a clear separation of concerns. Server-side events and client-side events are the two categories into which JSF events fall. Java programmers would write an event handler to configure a server-side event. The next step would be to establish an event generator or listener whose purpose is to keep an eye out for life cycle events. The server's event handlers are transparently translated to client-side events, such as a user pressing a button in the browser. This is how component events are handled. The actual wiring of the events is mostly done by the JSF framework.

Component-managed events

By enclosing such features inside one or more components, JSF conceals the specifics of how an event operates. Let us return to the **`DataList`** example and locate Paginator, a **`dataList`** that uses events, to see how these functions. work This list generates the grid using the same type of structure as the **`DataListView`**, but it has a unique feature: by clicking on the magnifying glass icon, a pop-up window displaying a list of links will appear. Listing 6 displays the element responsible for the linkages.

Listing 6. Magnifying glass links

```
<p:commandLink update=":form:carDetail" oncomplete="PF('carDialog').
show()" title="View Detail" styleClass="ui-icon ui-icon-search"
style="float:left;margin-right:10px">
```

```
  <f:setPropertyActionListener value="#{car}" target="#{dataListView.
selectedCar}" />
  <h:outputText value="#{car.brand}, #{car.year}" />
</p:commandLink>
```

Listing 7 has the component that supplies the actual popup.

Listing 7. Popup component

```
<p:dialog header="Car Info" widgetVar="carDialog" modal="true"
showEffect="blind" hideEffect="explode" resizable="false">
 <!-- Dialog Content -->
</p:dialog>
```

By closely analyzing listings 6 and 7, it becomes evident that in JSF, JavaScript event is handled by components. JavaScript code is fired by the JavaScript **`oncomplete`** event in listing 6: **`oncomplete="PF('carDialog').show()"`**.

It should be noted that the function that will be called is already defined by the internals of the component.

With **`widgetVar="carDialog"`**, the **`p:dialog`** component in Listing 7 has given itself a name. The PrimeFaces framework and the components themselves are in charge of combining these two into a useful JavaScript dialogue.

Application of JavaServer faces

The Java standard for developing web-based UIs is called JavaServer Faces. It's wonderful news for Java developers who seek a standardized method to create cutting-edge, Java-based web UIs since JSF has been chosen for inclusion and continued development in Jakarta EE. **Rapid Application Development (RAD)** has been a goal of JSF's design from the outset. It offers fantastic third-party components and significant community support. Building JSF-based applications still involves many intricate tasks that are necessary at all stages of the typical **Software Development Life Cycle (SDLC**). While some of these duties are supported by the third party, others are part of the JSF standard implementation. The following categories apply to JSF

- Project preparation and environment setup
- Pages prototype and design
- Development and debugging
- Performance tuning

Project preparation and environment setup

Numerous configurations, libraries, and setups are required for developing new JSF applications. Since they are already included with any Java EE compliant application server, the JSF libraries (that is, API and implementation) exist solely during the compilation phase. On the contrary, JSF libraries must be included in the project class-path for servlets containers like Jetty and Tomcat, which results in a portability problem. The following configurations need to be in Web.xml file:

- JSF Servlets
- Map JSF pages
- Encoding servlets to solve encoding issues (in the case of Tomcat and Jetty)
- Set project mode
- Set JSF time zone to system time zone
- Handle view expired exception by setting the compatibility mode

`Beans.xml` must add CDI support and include CDI API and implementation to the project-class path in case of servlet containers.

Pages prototype and design

When traditional desktop and outdated web apps were still being developed, it was simple for business analysts to create application prototypes using any modelling tool, such as Microsoft Visio. The emergence of new web design and user experience trends like responsive, lightweight, flat, and metro designs has significantly altered this though. Since UI designs have evolved into one of the primary determining factors of awarding criteria in many software projects, these trends have made it necessary to create a comprehensive application prototype in the early stages of projects, sometimes even before awarding.

In JSF applications, this can be done in one of three ways:

- **JSF components and themes**: The simplest method for implementing the prototype is to use pure JSF components and themes. This strategy, however, forces JSF developers to rely entirely on the components and themes that are already in use, which are heavyweight and limited in possibilities as compared to HTML5 modern themes and don't take advantage of emerging trends. Additionally, as the labor cost of Java developers is higher than the cost of web designers, this results in an increase in project costs. Additionally, it results in a strong coupling between applications and the components library, which increases maintenance costs (that is, the change of PrimeFaces dialogue component between 4.0 and 5). Additionally, it makes switching to a

different component difficult, much like when many applications converted from ICE-Faces and Rich faces components to PrimeFaces (PF).

- **Pure JSF components with customization support by web designers**: Even while it might be beneficial to have a few small design adjustment possibilities, hiring web designers in this situation won't be very beneficial, which is the cause of a lot of difficulties. First of all, it is difficult and not very flexible to customize a theme and override its styles. Second, the process takes a long time and does not guarantee success. Thirdly, one of the key reasons this solution is not advised and is unfriendly for designers is the obvious lack of design tools to facilitate such modification. Facelets as the declaration language won't fix the problem because utilizing JSF components for everything will result in the same output from conventional JSF syntax, and there will still be a significant degree of coupling with the components.
- **Pure HTML 5**: This method of prototyping and application design will be the most economical and useful. We can get the results faster using pure HTML5. Additionally, the readily available design frameworks and libraries make it simple to create contemporary designs (for example, Bootstrap). Utilizing pre-made web design templates is another viable and useful option. In this method, JSF components will only be utilized to bind input components; HTML5 will be used for the entire design.

Development and debugging

Following features are used for developing applications:

- **Model-View-Controller** (**MVC**) through views and managed beans is one of the numerous technical needs for contemporary web applications that JSF has been developed to include.
- Expression language is used automatically for binding between client-side and server-side components of the user interface (EL).
- Built-in template engine (Facelets)
- Navigation and workflow management (Navigation rules)
- Ready-made user-interface components
- Built-in validation for input fields (Validators)
- Built-in conversion for data types (Converters)
- Transparent AJAX support

Although JSF was designed with a high level of abstraction and a large list of fancy features, which should, in theory, shorten development time, it comes at a hefty

price in terms of performance, debugging, and performance, especially given the lack of tools and IDE support. The main causes of this are the limitations in JSF specifications and implementations, which will be covered later in this chapter; and the complexity of client-side code in sophisticated web applications, which is the case for most modern web applications.

Despite being available since 2004, JSF lacks numerous technical functionalities and features, which makes creating and debugging JSF applications extremely difficult. The difficulties include the following:

- The design issues caused when using the **`<head>`** tag instead of **`<h:head>`**
- The no-action of commands if not surrounded with the **`<h:form>`** tag
- Missing exception handling for AJAX requests
- Converters of list data types
- The handling of view expired exceptions
- Using the **`<html>`** tag to define composite components and include files, which could lead to an unconventional HTML DOM structure
- The resources that complicated CSS files use as references
- The difficulty of using Java classes to create composite components

Many JSF developers are compelled to employ third-party solutions and workarounds to fix these problems. But using non-standard libraries and APIs results in significant maintainability problems.

Performance Tuning

Although JSF has been designed on a high level of abstraction and a long list of fancy features, it includes many serious performance issues, such as the following:

- When we add images, css, javascripts, etc. they add overhead in JSF components tree. JSF has some configurations that can be used for performance tuning, including refresh-period and project-stage context parameters.
- Due to the absence of transparent and standard caching functions for user-interface resources they are handled less frequently in the component JSF tree

Conclusion

The usage of JSF for fast application development and for creating enterprise-level web apps using Java was covered in this chapter. We discussed event management, the MVC model, and the primary user interfaces that are used to create applications.

The use and implementation of web services are covered in the next chapter.

Points to remember

- JSF is a framework for building web applications; it simplifies development by providing a component-centric approach to developing user interfaces.
- JSF uses a component-based structure that provides better security over JSP.
- JSF is a front-end framework that makes the creation of user interface components easier by reusing the UI components.
- Facelets is a lightweight page declaration language that is used to build JavaServer Faces views using HTML style templates and to build component trees.
- Primefaces is an UI library for JSF-based applications.

Multiple choice questions

1. **What is the full form of JSF?**
    a. JavaScript Framework
    b. Java Server Faces
    c. Java Server Facility
    d. None of these

2. **Which of the following is/are the features of JSF?**
    a. Component-Based Framework
    b. Support HTML5 & Internationalization
    c. Ease and Rapid Web Development
    d. All of the above

3. **Which of the following converter tags are supported in JSF?**
    a. convertNumber
    b. convertDateTime
    c. Custom converter
    d. All of the above

Answers

1. **b**
2. **d**
3. **b**

Questions

1. What is JSF?
2. What are the benefits of using JSF?
3. What is managed bean in JSF?
4. What are user interface components in JSF?
5. What is Face lets?
6. What are web resources in JSF?
7. Explain how JSF is different from conventional JSP.

Key terms

- **JSF**: It is a user interface framework with server-side components.
- **Facelets**: Views in Facelets are XHTML pages. By adding components to the page; connecting the components to backing bean values and properties; and registering converters, validators, or listeners on the components, they are used to build web pages or views.

Join our book's Discord space

Join the book's Discord Workspace for Latest updates, Offers, Tech happenings around the world, New Release and Sessions with the Authors:

https://discord.bpbonline.com

Chapter 12
Working with Web Services (SOAP and RESTful)

Introduction

In this chapter, the idea of web services and its significance will be explored. It is important to understand the service-oriented architecture when we wish to create web services, use and consume already-existing web services, and create a web service in one environment and access it in the other environment using the Java platform. The chapter also looks at the differences between web applications and various types of web services.

Structure

In this chapter we will learn the following topics:

- Service-oriented architecture
- Introduction to web services
- Types of web services
- Java web services
- Create a web service
- Consume a web service
- Web service interoperability

Objectives

You should be able to distinguish between SOAP-based and RESTful services, apply logic to decide when to use SOAP-based or RESTful services, and comprehend the Java API to carry out the implementation of web services after reading this chapter. You should also be able to understand the service-oriented architecture; the role and applications of web services; and how to create, consume, and deploy the web services.

Service-oriented architecture

A **service-oriented architecture** (**SOA**) is a design pattern whose purpose is to create distributed systems that offer services to other applications. It is a concept and is not limited to any programming language or platform.

A service is a standalone module, a function that represents a unit of functionality. A service can exchange information from another service. It is independent in behavior and uses a loosely coupled, message-based communication model to communicate with applications and other services.

Service connections

The following figure shows the communication between the consumer and the service provider. The consumer sends a request to the provider, and the provider handles the request, processes it and sends the service response. The service connection is established between the consumer and the provider.

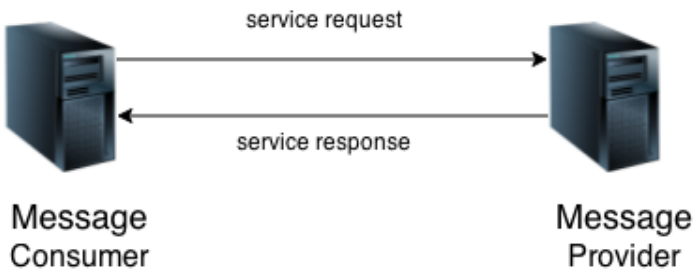


Figure 12.1: *Service-oriented architecture*

Service-oriented terminologies

The following terms are the major concepts used in service-oriented architecture:

- **Services**: They are small modules created to provide some functionality.
- **Service provider**: It provides a mechanism to use the service.
- **Service consumer**: It is a client that requests to use the service; it can be an application or another service.

- **Service locator**: It is a registry that looks for the service, its interface and locations.
- **Service broker**: It passes service requests to one or more additional service providers.

The following figure demonstrates the role of the previously mentioned terms:

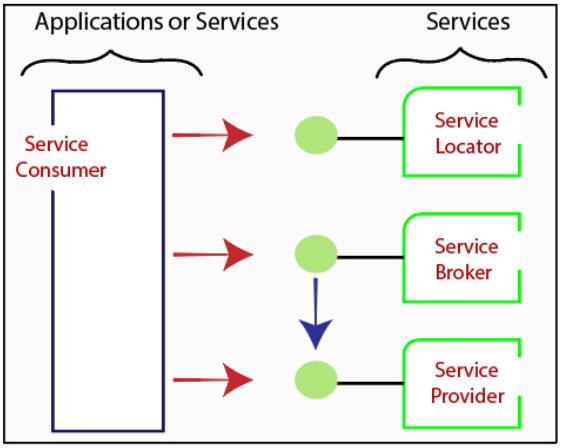


Figure 12.2: *Role of service locator, broker, provider and consumer*

Workings of service-oriented architecture

The service-oriented architecture can be visualized as qualitative operations:

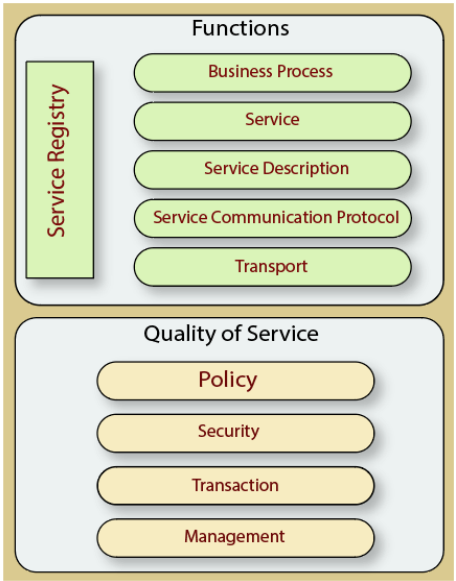


Figure 12.3: *Basic operations of service-oriented architecture*

Functional operations

Functional operations contain the following:

- **Transport**: This operation takes care of transfer of request from the consumer to the provider and the response from the provider to the consumer.
- **Service communication protocol**: It gives the specification for communication between the consumer and the provider.
- **Service description**: It gives the details of the service and data required to start the service.
- **Service**: It is the functionality.
- **Business process**: It provides the list of services to be called in a particular order associated with the specific rules to meet the business requirements.
- **Service registry**: It contains the details of data used by service providers to publish their services.

Qualitative operations

The qualitative operations include the following:

- **Policy**: It specifies the set of protocols using which the providers and consumers can communicate.
- **Security**: It specifies the set of protocols that are used to provide identification and authorization.
- **Transaction**: It is used to provide consistent result; it ensures that if more than one service are used, all of them are executed in the required order, so the results are complete and correct.
- **Management**: It specifies the set of attributes used to manage the services.

Introduction to web services

A new approach to implementing and using a standalone functionality that can communicate between client and server applications is the use of web services. Web services provide a common platform that allows multiple applications built on different programming languages to have the ability to communicate with each other.

The services that can be accessed over network are called web services. So how does it differ from a web application, they are also services that are accessed over a network. There are a few attributes that clarify this difference:

- Web applications are meant for users and can be accessed in a browser having human-readable format, whereas web services are meant for applications to access data in the format of XML, JSON, etc.
- Traditional web services use the SOAP protocol, whereas web apps always use the HTTP/HTTPS protocol. REST is an architectural style that has gained prominence recently. It almost always uses the HTTP/HTTPS protocol.
- Reusability is one of the advantages of web services but is not intended for web applications. Different types of applications might use the same web service.
- Web services cannot access web apps to retrieve data, while web applications can access web services to accomplish tasks.
- Web services are stateless, while web apps are able to preserve user sessions.

A web service can be defined as below,. A client-server application, also known as an application component, is a way for two devices to communicate with one another through a network. It is a software platform that enables machine-to-machine interoperability.

Interface is a group of rules or procedures for transferring data between two programs or devices. Mainly, the following concepts are used with service-oriented architecture.

Simple Object Access Protocol (SOAP)

It is a web service access protocol built on the XML standard. W3C recommends the use of SOAP for application communication.

- **Web Services Description Language (WSDL)**: A WSDL is an XML document that contains details about web services, including the method name, method parameter, and access information. UDDI includes WSDL. It serves as an interface for web service programs.
- **Universal Description, Discovery, and Integration (UDDI)**: A framework for describing, finding, and integrating online services based on XML is known as UDDI. A directory of WSDL-described web service interfaces called UDDI contains details about web services.

Types of web services

Web services are externally accessible distributed application components. They can be utilized to combine computer programs created in several languages and operating systems. Because vendors have agreed upon common web service standards, web

services are agnostic of language and platform. Developers of web services have access to several programming models that are divided into two groups, both of which are supported by the IDE. There are mainly two types of web services:

- SOAP-based implementation
- RESTful-based implementation

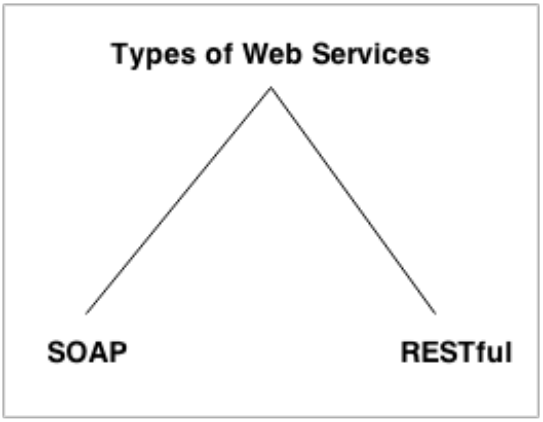


Figure 12.4: *Types of web services*

- **SOAP/WSDL-based**: Traditional web service models expose web service interfaces through URL-enabled WSDL documents, a kind of XML. The subsequent message exchange uses SOAP, a different XML document format.
- **REST-based**: Web services can now be created and communicated with using REpresentational State Transfer. Resources in REST have URIs and can be modified via HTTP header actions.

SOAP-based web services

It is a web service access protocol built on the XML standard. A W3C recommendation for inter-application communication is SOAP. It is independent of both platforms and languages. You can communicate with other programming language applications by utilizing SOAP. Java utilities use the Java code in the web service to generate a WSDL file for SOAP-based web services. The WSDL is available online and is used by those who want to use the service using a client. The format of the messages is SOAP. Particularly in terms of security, the variety of actions that can be passed in SOAP is far wider than what is accessible in REST. Heavyweight applications that use complex operations and applications needing advanced security, reliability, or other WS-* standards-supported characteristics should use SOAP-based web services. When a transport protocol other than HTTP must be used, SOAP and REST are also appropriate. Numerous web services provided by Amazon, notably

those involving business transactions, and those accessed by banks and government organizations, are SOAP-based.

SOAP Web services benefits

SOAP offers the following benefits:

- **WS security**: WS Security is a type of security that SOAP defines.
- **Independent of programming language and platform**: SOAP web services can be created in any language and run on any platform.

Problems with SOAP web services

- **Slow**: In order to be read, SOAP requires an XML format that must be parsed. It outlines several requirements that must be met while creating SOAP applications. As a result, it is slow and uses up more resources.
- **WSDL dependent**: SOAP relies on WSDL as its only method of service discovery.

RESTful web services

REpresentational State Transfer is the abbreviation for REST. REST is not a protocol but an architectural approach. URIs are used to identify groups of web resources that make up REST-based ("RESTful") web services. Every process and document is represented as a web resource with its own URI. The actions that can be defined in an HTTP header affect these online pages. None of the WSDL, SOAP, or WS-* standards are utilized. Instead, any format, including XML, JSON, and HTML, can be used for message exchange. A web browser can frequently act as the client. The REST protocol is HTTP. There are only four options: GET, PUT, POST, and DELETE. Requests can be bookmarked, and replies can be cached. A network administrator can quickly understand and fix what is going on in the network. For applications where HTTP is the right protocol and additional security is not required beyond that provided by the HTTP infrastructure, REST is a suitable solution. REST services are still capable of providing complex functionality. Amazon, Flickr, and Google Maps all offer RESTful web services. You can integrate services from third parties like Facebook and Zillow into your own applications using the **Software as a Service** (**SaaS**) capabilities of the NetBeans IDE.

Advantages of RESTful web services

- **Quick**: Because there is not a rigid specification as there is with SOAP, RESTful web services are quick, and less bandwidth and resources are used.

- **Independent of programming language and platform**: RESTful web services can be created in any language and run on any platform.
- **Can use SOAP**: RESTful web services can implement SOAP web services.
- **Supports several data formats**: RESTful web services support a variety of data formats, including plaintext, HTML, XML, and JSON.

Web services in Java

Applications that wish to use web services in Java use **Web Services Description Language** (**WSDL**) for communication. Web services using Java as implementation can be written in one of two ways: SOAP or RESTful.

API for Java Web Services

Since JavaEE 6, there are two primary APIs that have been defined by Java for creating web service applications:

- For SOAP web services, use JAX-WS. Writing JAX-WS application code can be done in one of two ways: RPC style or Document style.
- For RESTful web services, use JAX-RS. Currently, Jersey and RESTeasy are the two primary implementations used to build JAX-RS applications:

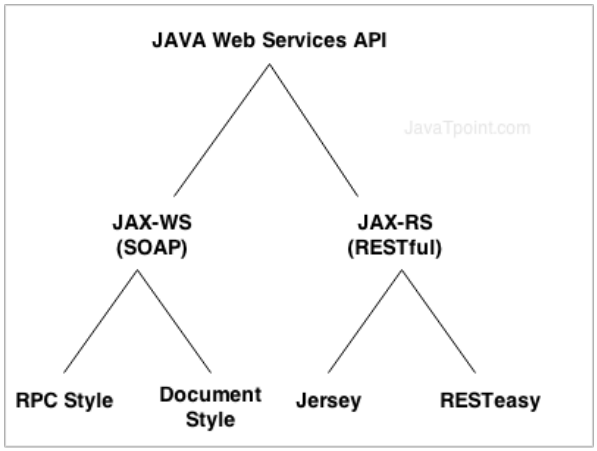


Figure 12.5: *Java API for web services.*

Using JAX-WS to create a web service and client

We can create a web service and a client to access the service as follows,

- The `@WebService` annotation defined in the `javax.jws` package is used to declare a class as a JAX-WS web service; now this class is a web service endpoint.

- The methods that a client can call on the service are declared via a Java interface or class, respectively, known as a **Service Endpoint Interface** (**SEI**) or implementation. Building a JAX-WS endpoint is not required to include an interface.
- An SEI is defined implicitly by the web service implementation class.
- We can create an interface by including the endpoint **`Interface`** element in the **`@WebService`** annotation of the class where we wish to implement the web service.
- Next, you must present users with an interface that lists the open functions that the endpoint implementation class exposes.
- Using the classes where endpoint implementation is done and the **`wsgen`** tool, the linking of client and JAX-WS runtime is done.
- The server's implementation of JAX-WS is provided by the **`wsgen`** utility in conjunction with the Application Server.

The steps for creating the web service and client are as follows:

1. Write the code for the implementation class and compile it.
2. Use **`wsgen`** to generate the classes that are required to implement the service.
3. Create a web archive by putting all the files into it.
4. Install the WAR file.
5. Write the code for the client.
6. Use the **`wsimport`** tool for creating and compiling the web service artefacts needed to connect to the service.
7. Compile and run the client.

Creating a web service

In this section, we will create a project and then create a web service in it.

Choosing a container

Either a web container or an EJB container can be used to deploy your web service. Your choice of implementation will determine this. Use a web container regardless of whether you are developing a Java EE application because you may include EJBs directly in an application. Create an application rather than an EJB module, for instance, if you intend to deploy to the Tomcat Web Server, which only has a web container.

Follow these steps to add a EJB module:

1. Choose EJB Module from the subcategory or web application from the Java web category.
2. The project should be called **`CalculatorWSApplication`**. The project's location should be decided. Choose **Next**.
3. Click on **Finish** after choosing your server and Java EE version.

Creating a web service from a Java class

1. Click on the **New** | **Web Service** option from the context menu of the **`CalculatorWSApplication`** node.
2. Give the web service the name **`CalculatorWS`** and include the package name **`org.me.calculator`**. Do not deselect **Create Web Service from Scratch**.
3. Choose the option to **Implement Web Service as a Stateless Session Bean** if you are building a Java EE project using the Glassfish server or Weblogic:

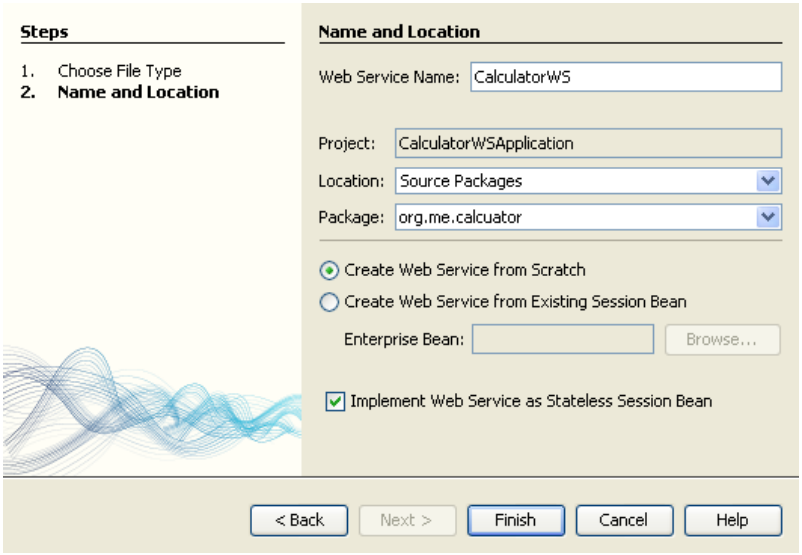


Figure 12.6: *Interface to provide service name*

4. Press **Finish**. The new web service's structure can be seen in the **Projects** window, and the editor area also displays the source code.

Adding an operation to the web service

The objective of this exercise is to add an operation that adds two numbers that were received from a client to the web service. A dialogue for adding an operation to a

web service is provided by the NetBeans IDE. Either the web service context menu or the web service visual designer can be used to open this dialogue.

To add an operation to the web service:

You can either change to the design view in the editor:

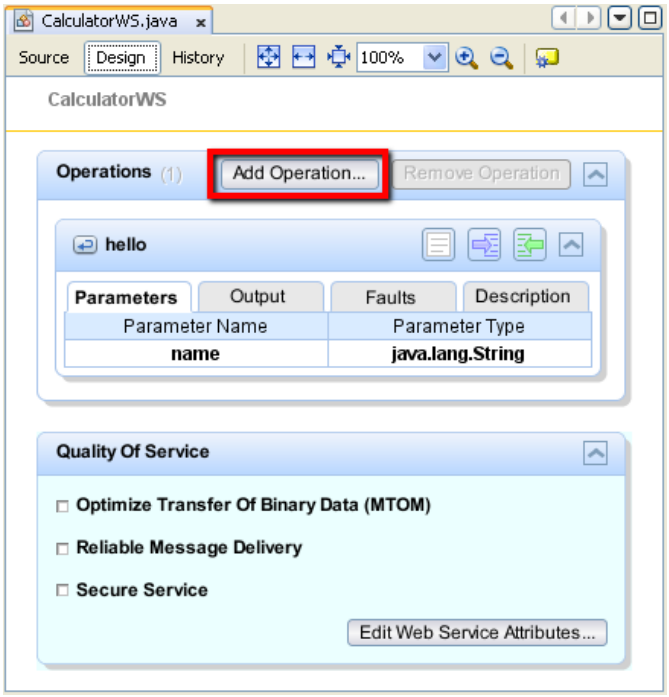


Figure 12.7: *Interface to add operations*

Or in the project window, find the web service's option and right-click on that option. A context menu opens, as follows:

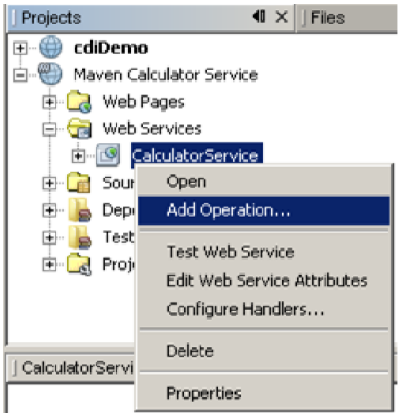


Figure 12.8*: Add operations through coding*

1. Either in the context menu or the visual designer, click on **Add Operation**. The **Add Operation** dialog opens.
2. Type add in **Name** and select any number datatype, for example, integer, from the Return Type drop-down list in the **Add Operation** dialog box's upper section.
3. Click on **Add** and provide the name I for the int-type parameter you just created in the lower section of the **Add Operation** dialog box.
4. Create a parameter of datatype integer named j by clicking on **Add** once more.

   You now see the following:

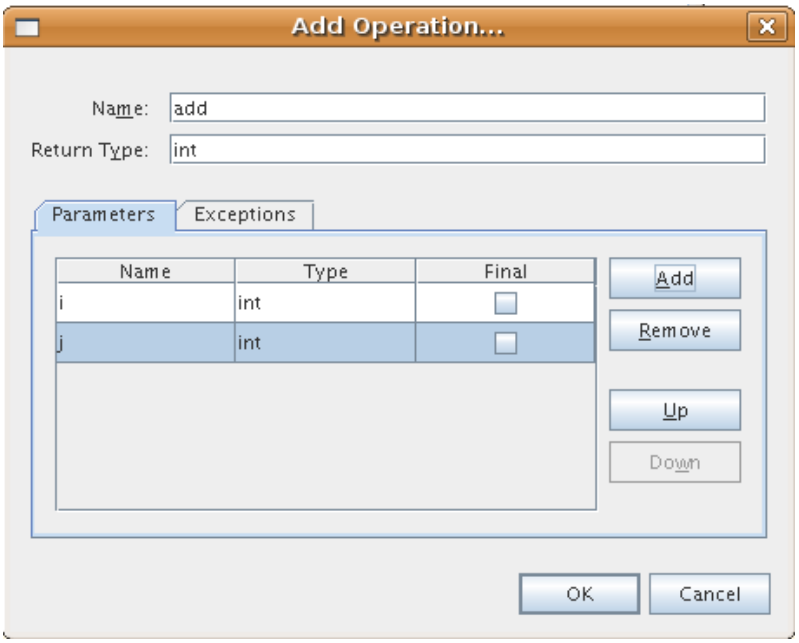


Figure 12.9: *Add the signature of operations*

5. Click on **OK** in the dialog box. You return to the editor.

6. Remove the default hello operation, either by deleting the **`hello()`** method in the source code or by selecting the hello operation in the visual designer and clicking on **Remove Operation**. The visual designer now displays the following:

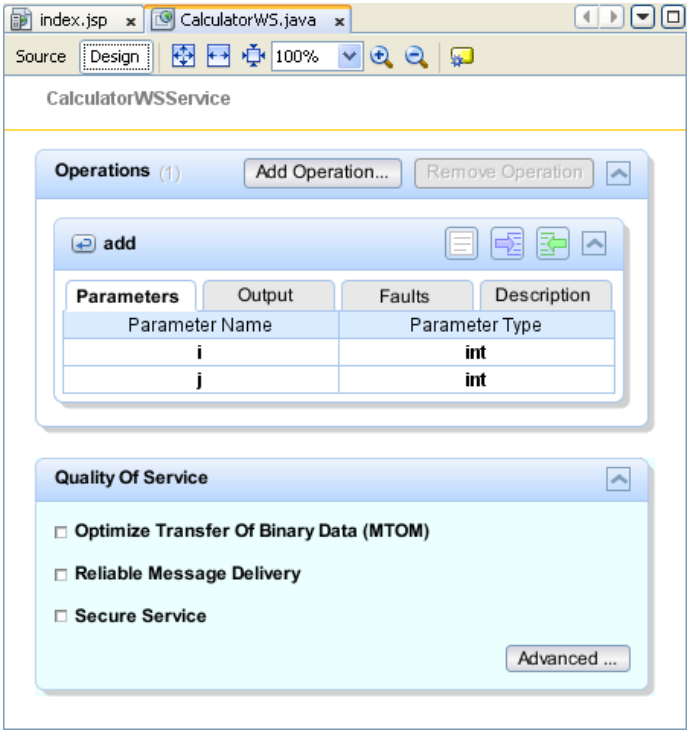
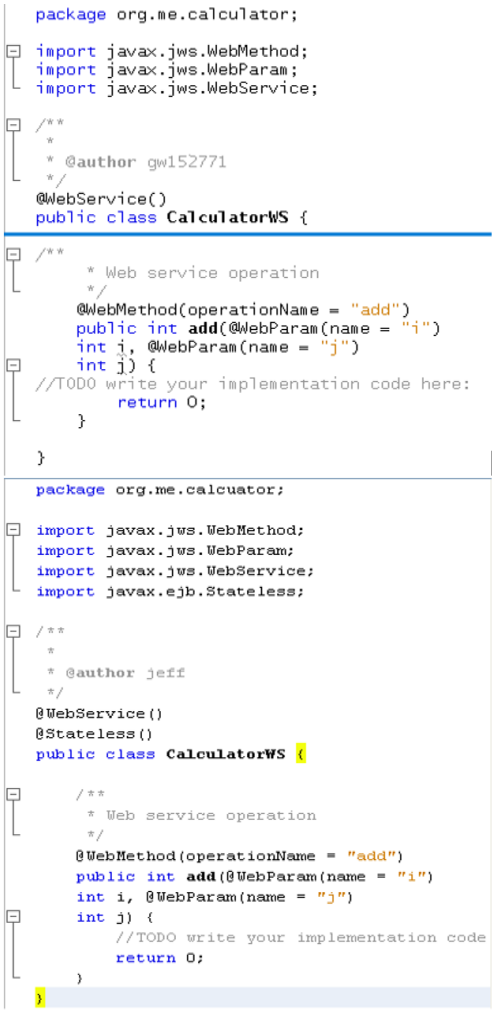


Figure 12.10: *Parameterized web service creation*

7. Click on **Source** and view the code that you generated in the previous steps. It differs based on whether or not you created the service as a Java EE stateless bean:



```
package org.me.calculator;

import javax.jws.WebMethod;
import javax.jws.WebParam;
import javax.jws.WebService;

/**
 *
 * @author gw152771
 */
@WebService()
public class CalculatorWS {

    /**
     * Web service operation
     */
    @WebMethod(operationName = "add")
    public int add(@WebParam(name = "i")
    int i, @WebParam(name = "j")
    int j) {
//TODO write your implementation code here:
        return 0;
    }

}
```

```
package org.me.calcuator;

import javax.jws.WebMethod;
import javax.jws.WebParam;
import javax.jws.WebService;
import javax.ejb.Stateless;

/**
 *
 * @author jeff
 */
@WebService()
@Stateless()
public class CalculatorWS {

    /**
     * Web service operation
     */
    @WebMethod(operationName = "add")
    public int add(@WebParam(name = "i")
    int i, @WebParam(name = "j")
    int j) {
        //TODO write your implementation code
        return 0;
    }
}
```

***Figure 12.11**: Java code of web services*

8. In the editor, extend the skeleton add operation to the following:

```
    @WebMethod
    public int add(@WebParam(name = "i") int i, @WebParam(name =
"j") int j) {
        int k = i + j;
        return k;
      }
```

As you can see from the preceding code, the web service simply receives two numbers and then returns their sum.

Executing and testing the web service

If a server has a test client, you can use the IDE to launch it after you've deployed a web service to it. The servers running GlassFish and WebLogic offer test clients.

There is no test client when using the Tomcat web server. Only the Tomcat web services page may be opened by running the project. In this scenario, you must make the web service your application's entry point before launching the project. Choose **Properties** from the context menu when you right-click on the **`CalculatorWSApplication`** project node to make the web service your application's entry point. Enter **`/CalculatorWS`** in the relative URL field of the **Run** properties.

To test successful deployment to a GlassFish or WebLogic server

1. Click on the **project's** right-click menu and select **Deploy**. The application server is started, the application is built, and it is deployed to the server by the IDE. The **`CalculatorWSApplication`** (run-deploy) and GlassFish server or **Tomcat** tabs in the **Output** view allow you to monitor the status of these actions.

2. Expand the **`CalculatorWSApplication`** project's **Web Services** node in the **Projects** tab of the IDE. Go to the **`CalculatorWS`** node's context menu and select **Test Web Service**.

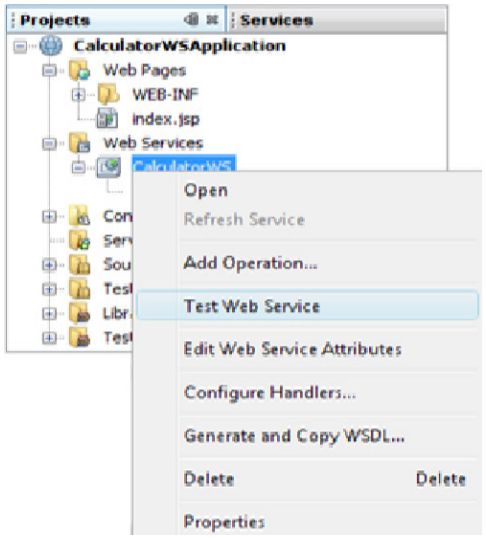


***Figure 12.12**: Interface to test the web service*

The IDE opens the tester page in your browser if you deployed a web application to the GlassFish server. For the Tomcat Web Server and deployment of EJB modules, the situation is different.

3. If you deployed to the GlassFish server, enter the following two numbers in the tester page:

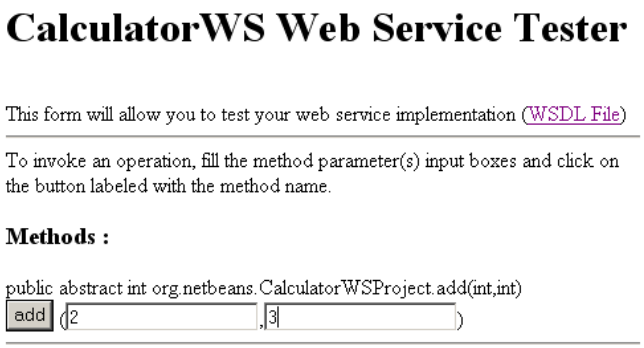


Figure 12.13: *Interface to provide two numbers to invoke the web service*

The sum of the two numbers is displayed, as shown in the following figure:

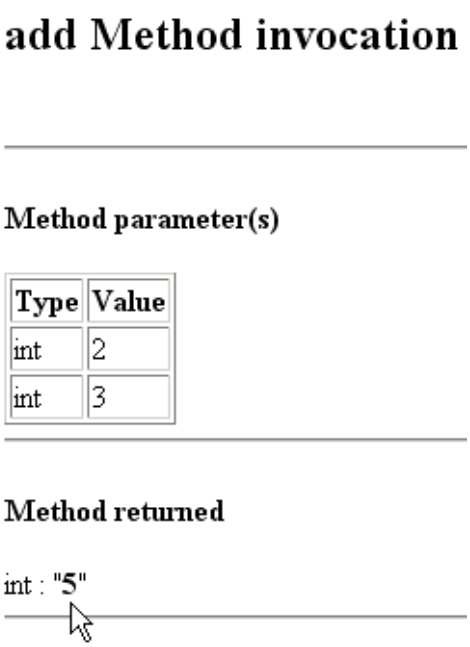


Figure 12.14: *Interface to execute the web service*

Let us consume another web service to convert the temperature from Celsius to Fahrenheit; the general steps are as follows.

Consume a web service in Java

Following are the steps to consume a web service in Java:

1. Select **File** menu and select **New Project**:

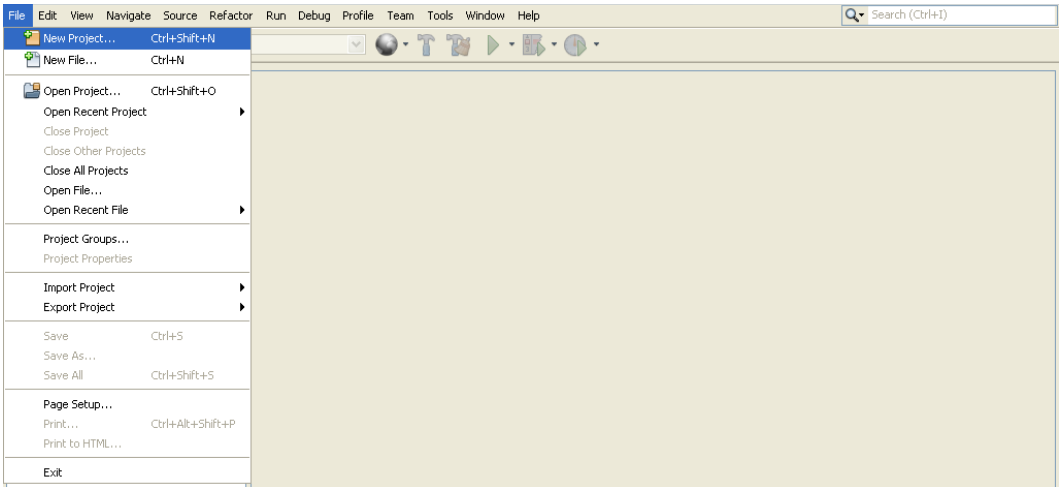


***Figure 12.15**: Interface to select a Project from File menu*

2. Select **Java web** in categories; in **Project**, select **web application**, and specify name and other details:

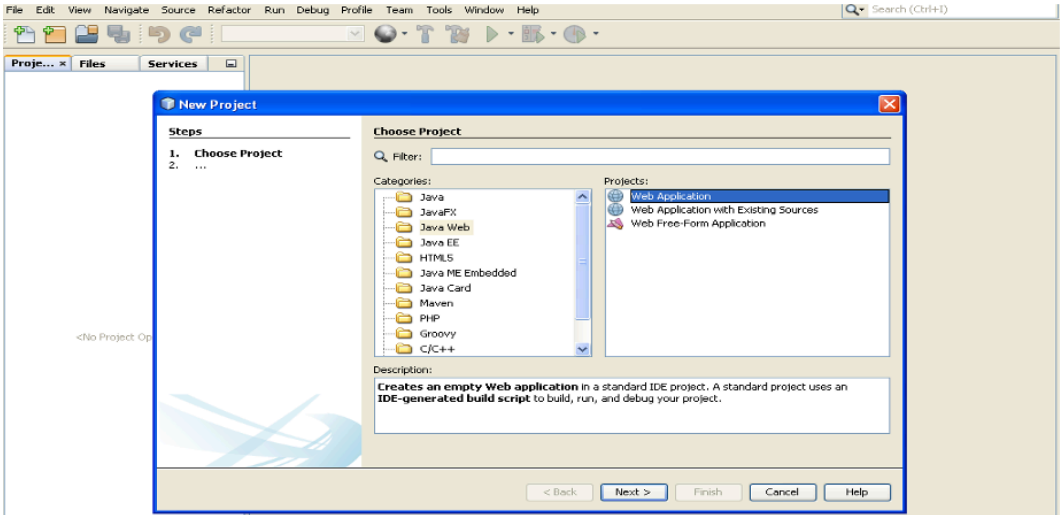


***Figure 12.16**: Interface to provide type of project*

Provide the name and location of the project and click on the **Next** button:

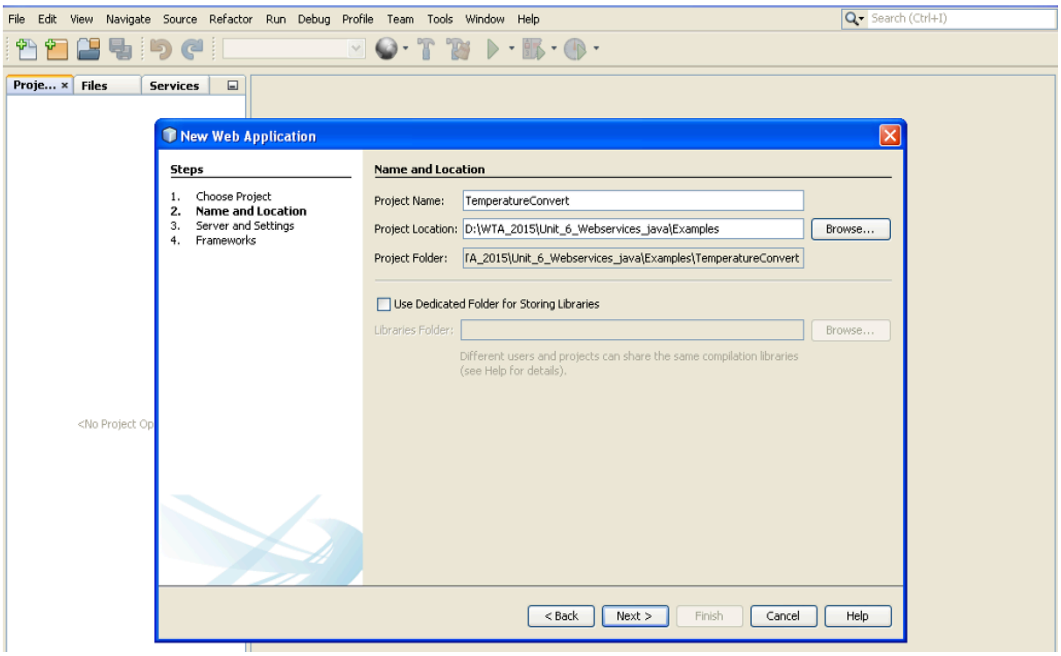


***Figure 12.17**: Interface to provide project name*

Provide the server name, Java enterprise edition version and name of the context path and select **Next**:

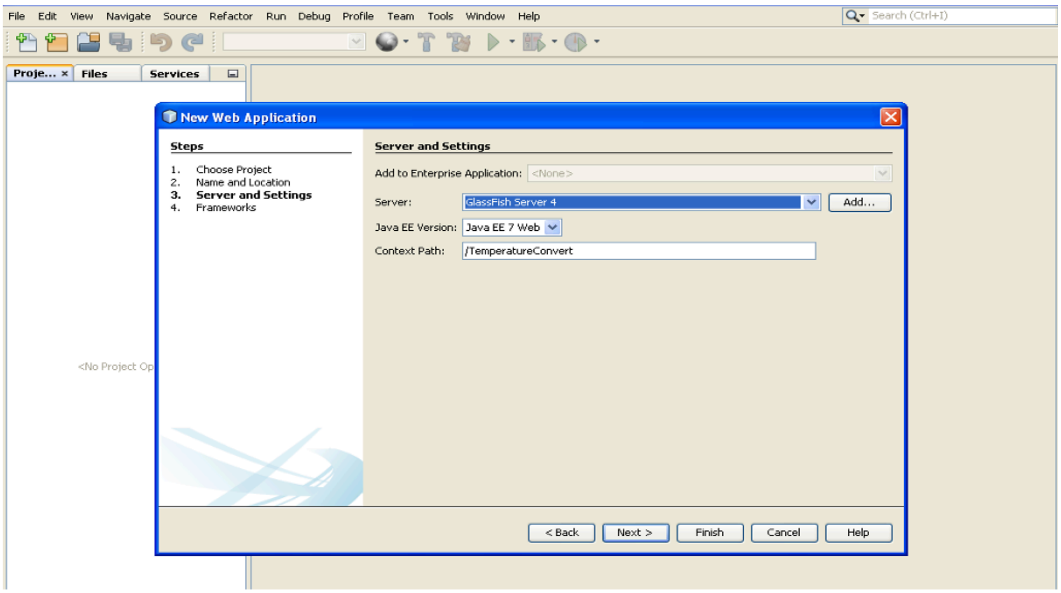


***Figure 12.18**: Interface to server settings*

Select the name of framework in case we wish to create the default directory structure for a project using a framework:

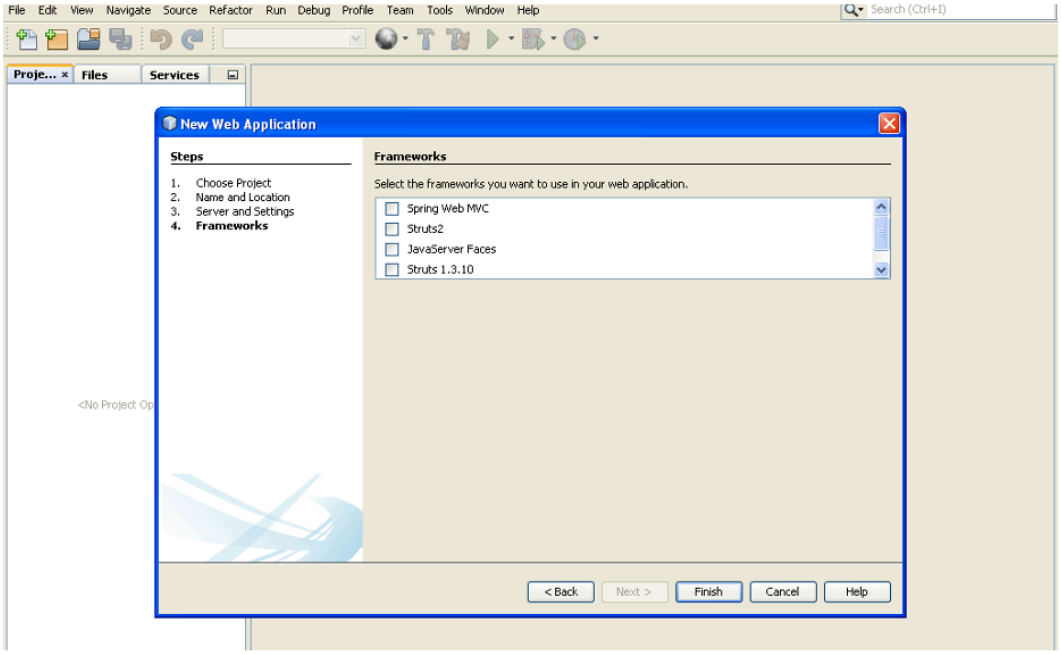


***Figure 12.19**: Interface to select framework for creating a web application*

3. Add new web service client by right-clicking source package and selecting web service client.

   (**Others** | **Web Services** | **Web service client**)

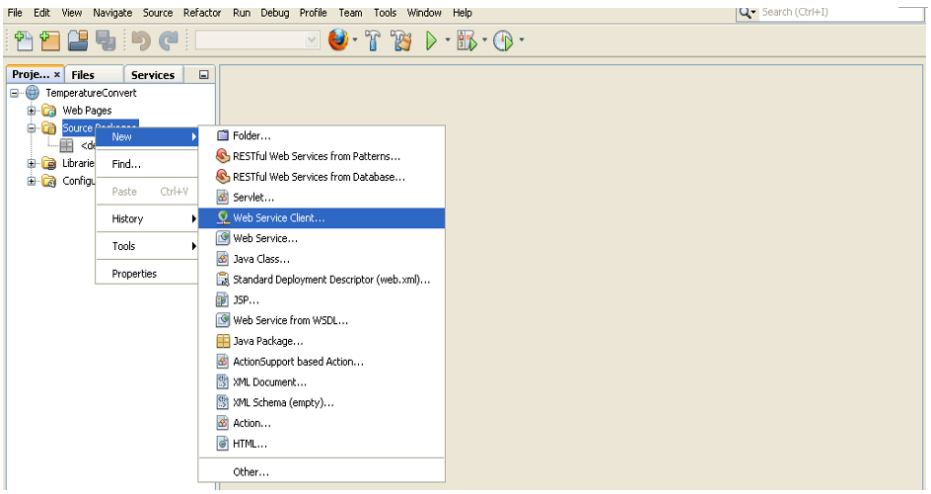


***Figure 12.20:** Interface to create a web service client*

4. Specify the location of WSDL for the local file:

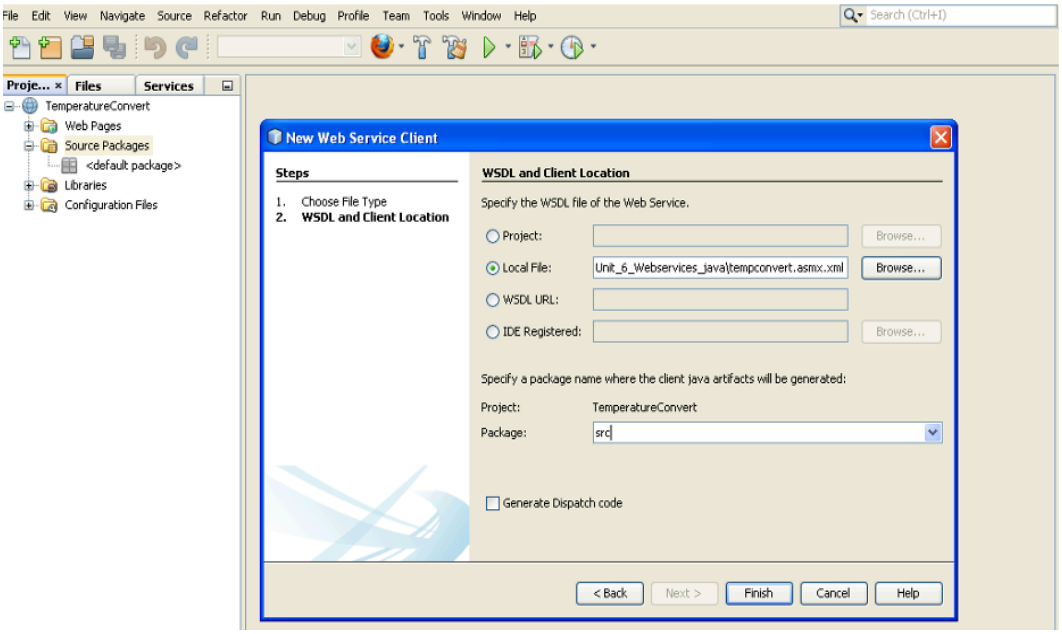


***Figure 12.21**: Interface to create a web service client*

After successful parsing, all the **Generated Source** code will be available in the project:

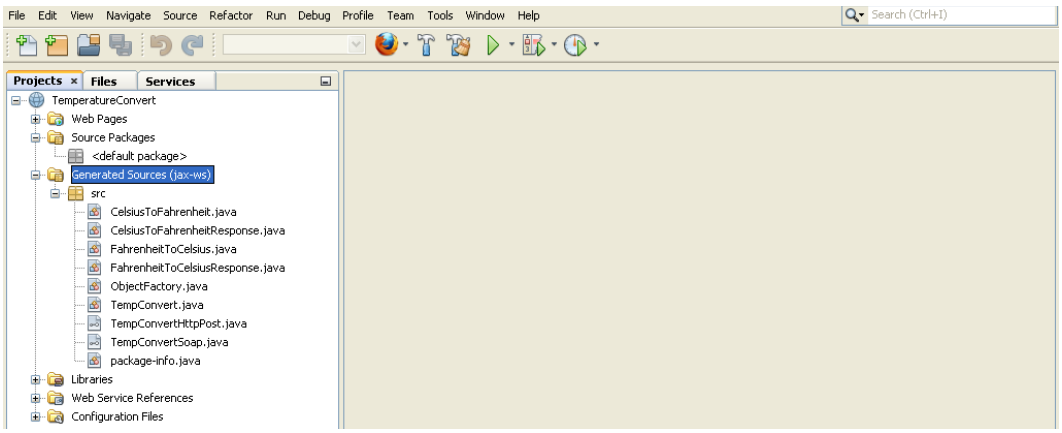


***Figure 12.22:** Extract the web services*

5. Create a **Servlet...** to consume the web service and display the result

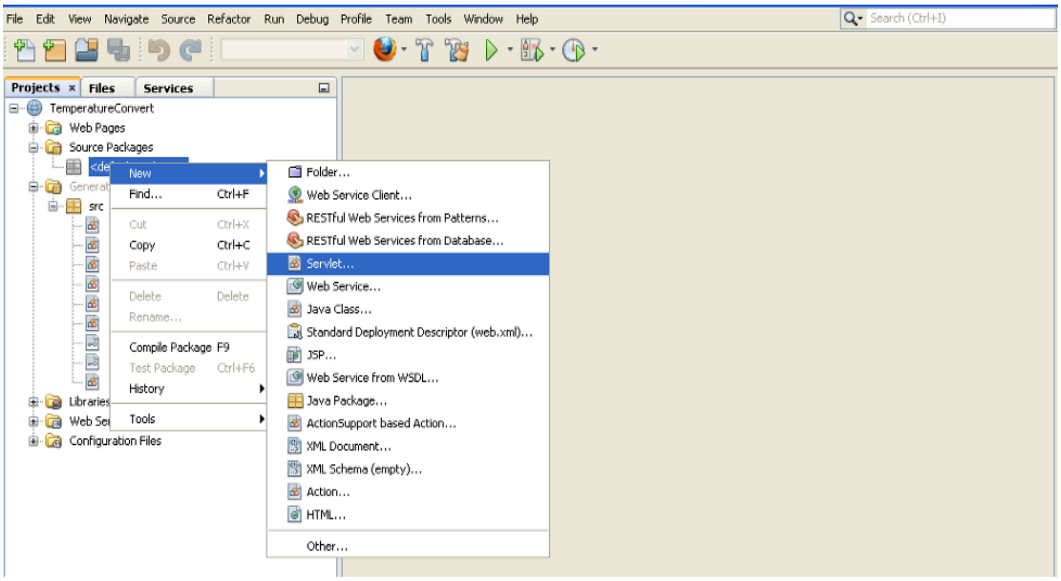


Figure 12.23: *Interface to add a new servlet*

6. In the wizard, provide the servlet class name, location and package name.

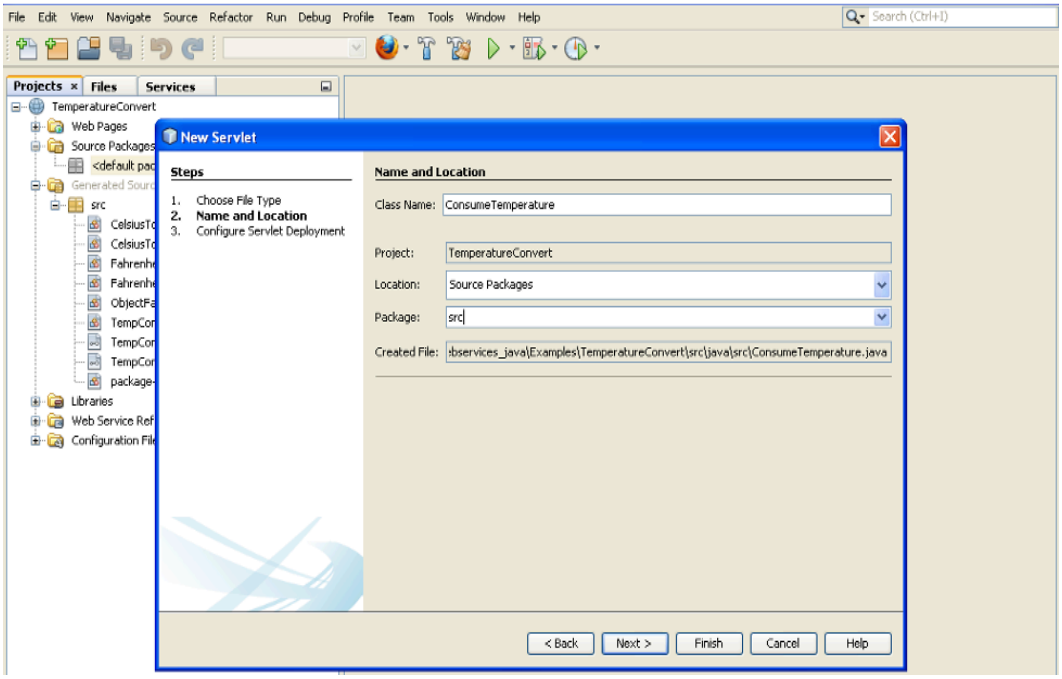


Figure 12.24: *Interface to provide servlet name and package*

Provide servlet name and the URL pattern to access the servlet:

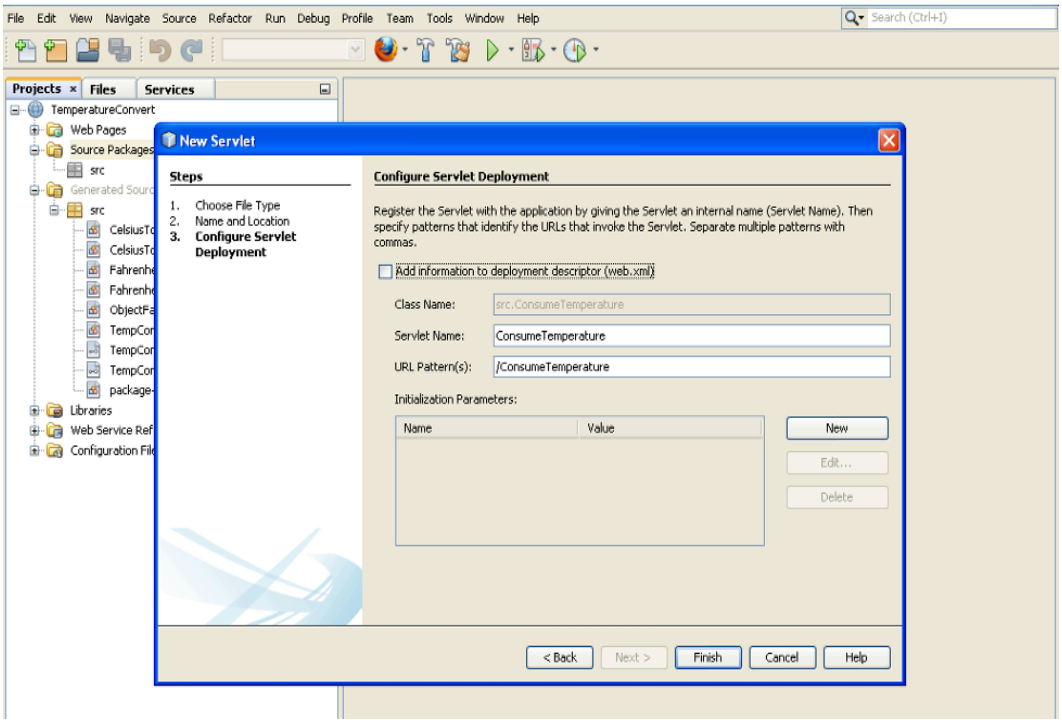


Figure 12.25: *Interface to configure servlet deployment*

7. Update **`index.jsp`** to read the temperature and invoke the servlet.

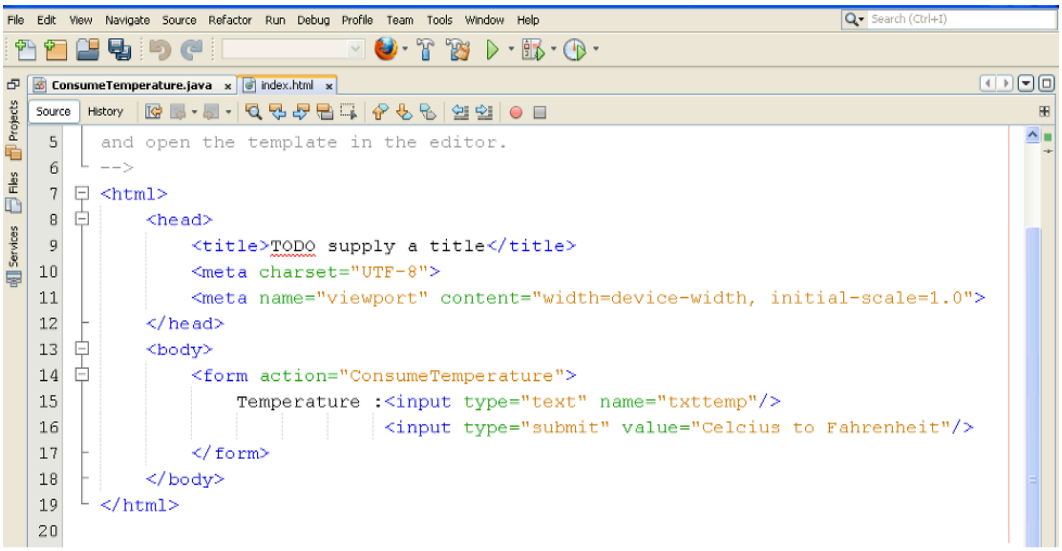


Figure 12.26: *Interface to write the index page of the application*

8. Update the code in Servlet to get the reference of objects and retrieve the method to display the result in JSP:

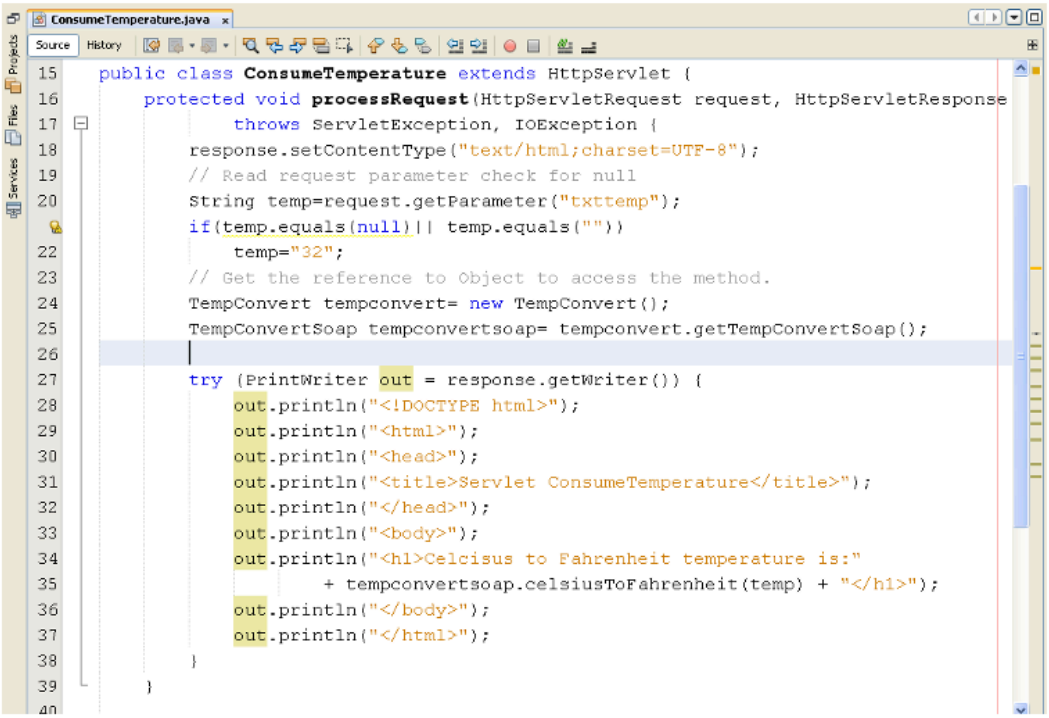


Figure 12.27: *Servlet code to handle the request*

9. Deploy and check the output, as shown in the following figure:

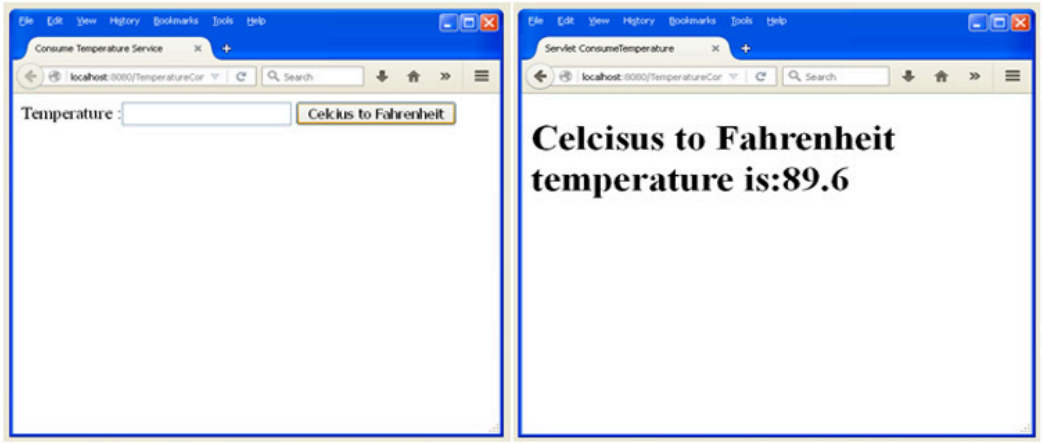


Figure 12.28*: Execution of application*

Web service interoperability

Web services typically operate without a hitch on a variety of platforms, apps, and coding languages. One of the main objectives of web services is interoperability. Although they can help, web service standards cannot guarantee interoperability. To achieve complete interoperability, a number of challenges and considerations must be handled. A web service has a wide range of potential clients, and these clients can connect to the service via various libraries and providers. Other services may end up as clients of other services. The success of the service depends on enabling clients based on various implementations of the standards to communicate with it. Here, we'll create basic mathematical operations like addition and subtraction using **`JSimpCalcWebService`**, a Java web service. The process for calling these web service methods through the **`JSimpCalcWebServiceWebSite`** ASP.net application will then be examined. We need to use two IDEs to construct the two distinct applications. In this example, the asp.net client calls the Java web service.

Creating web service using Java

Basic steps for creating a service are listed as follows:

- Select a new project of type web application
- Create a new web service and add operations and implement them
- Deploy and test the web service

Select a new project of type web application

1. In Netbeans IDE, select the **New Project** under **File** menu. The **New Project** wizard opens.
2. In the categories, select **Web** and **Web application** from project section and then press the **Next** button.

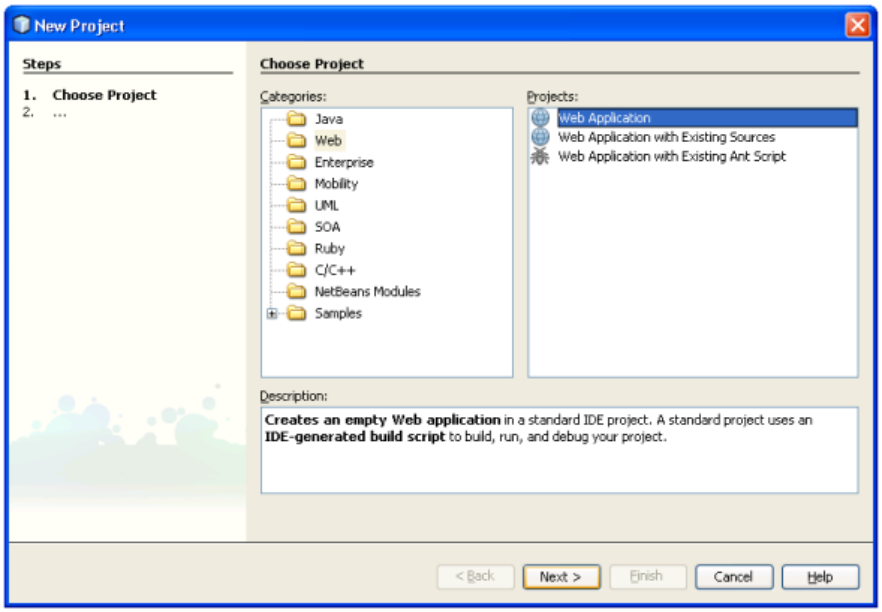


***Figure 12.29**: New Project Selection*

3. In the next step of wizard, provide the name of project and browse the location where we want to save our project. We can also specify and select the server we want to deploy our application.

4. Specify the project name as **`JSimpCalcWebService`** and keep the context path the same as the project name. We use GlassFish server for deployment.

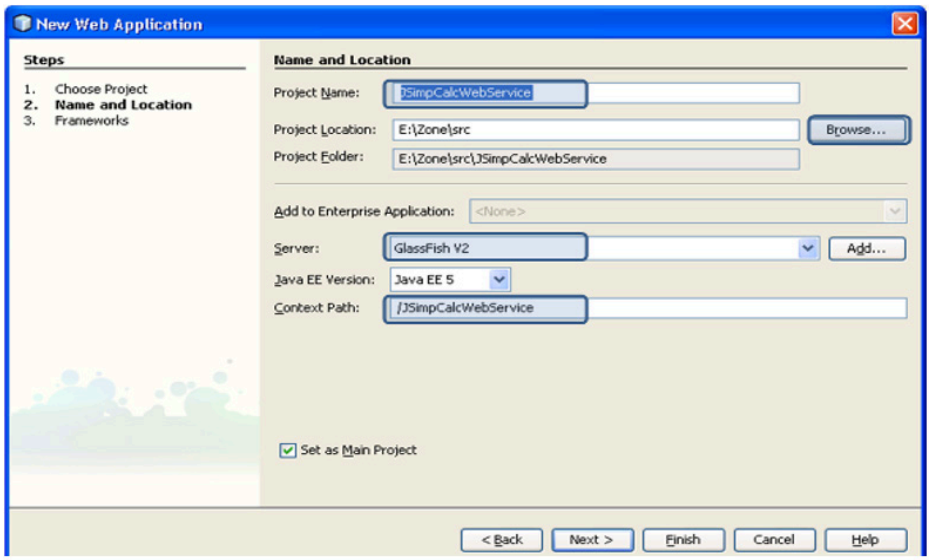


***Figure 12.30**: Provide Project name and location, along with other attributes*

5. After providing all the information required to create the project, click on the **finish** button.

Create a new web service and add operations and implement them

1. In the **Projects** explorer window, right-click on the project name.

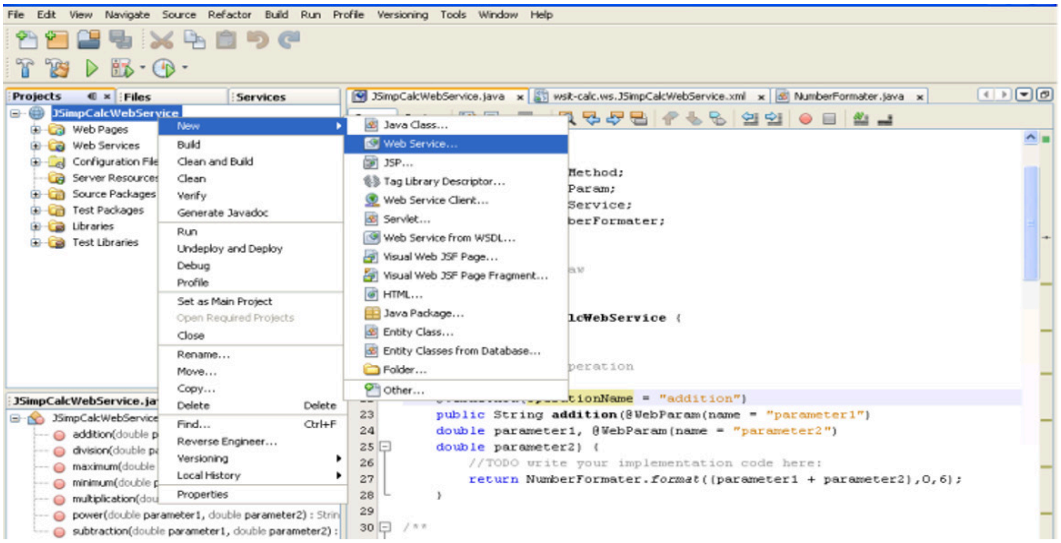


***Figure 12.31**: Add a web service*

2. From the context menu options, select the **Web Service** menu; web service dialog will open.

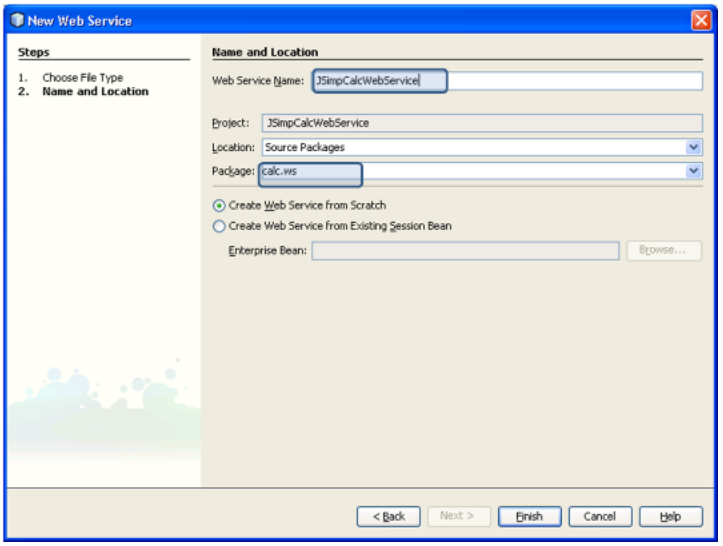


***Figure 12.32**: Provide a web service name and other attributes*

3. Specify the web service and the package name, and then click on the **finish** button.

4. Specify the service name as **`JSimpCalcWebService`** and the package name as **`calc.ws.`**

5. We can add the web method in two ways: through design mode and through source mode. Here, we use the design mode for creating the basic structure of **`WebMethod`**.

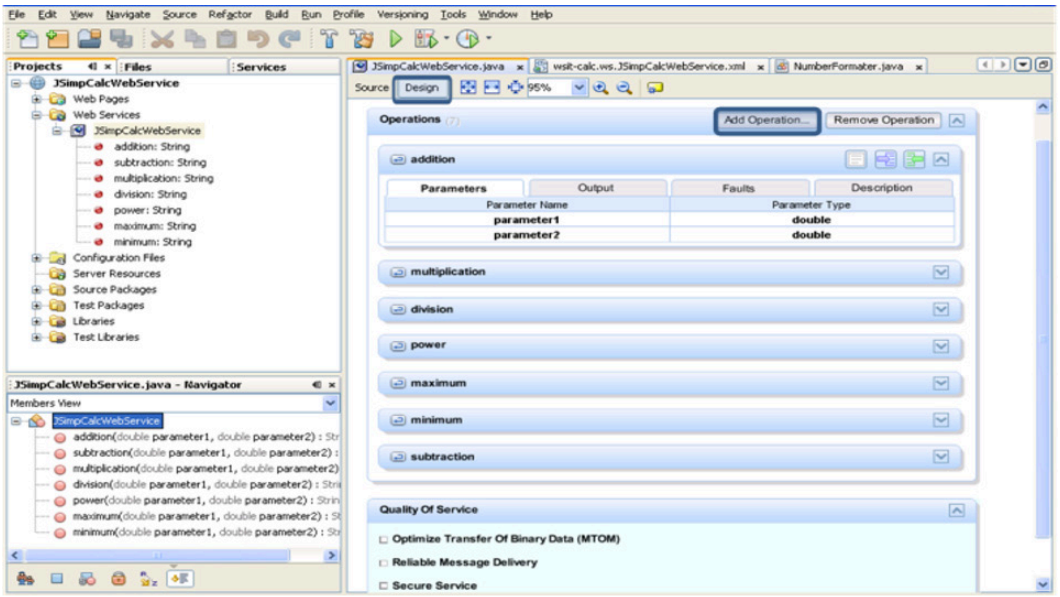


***Figure 12.33**: Add web methods*

6. Add web service operations by clicking on the **Add Operation** button; it will open an **Add Operation** dialog box.

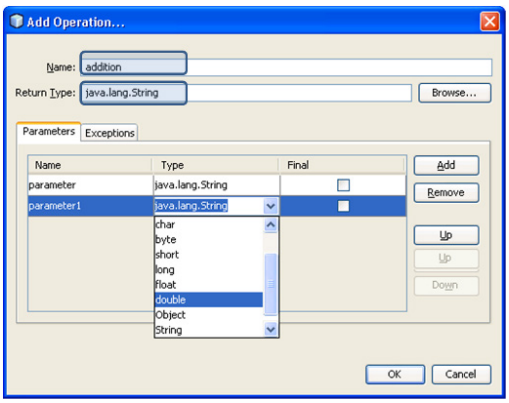


***Figure 12.34**: Provide signature of method*

7. In the **Add Operation** dialog, specify the name; if we use parameters, then we can also specify the parameter names and their types.

8. In the example, we specify the **`WebMethod`** name as "addition" whose return type is String, and it takes two parameters of type double. We create more operations like subtraction, multiplication, division, power, maximum and minimum.

   Here, a basic structure of our web service is created. Then, we switch from design mode to source mode, as shown in the following figure, to do rest of the implementation.

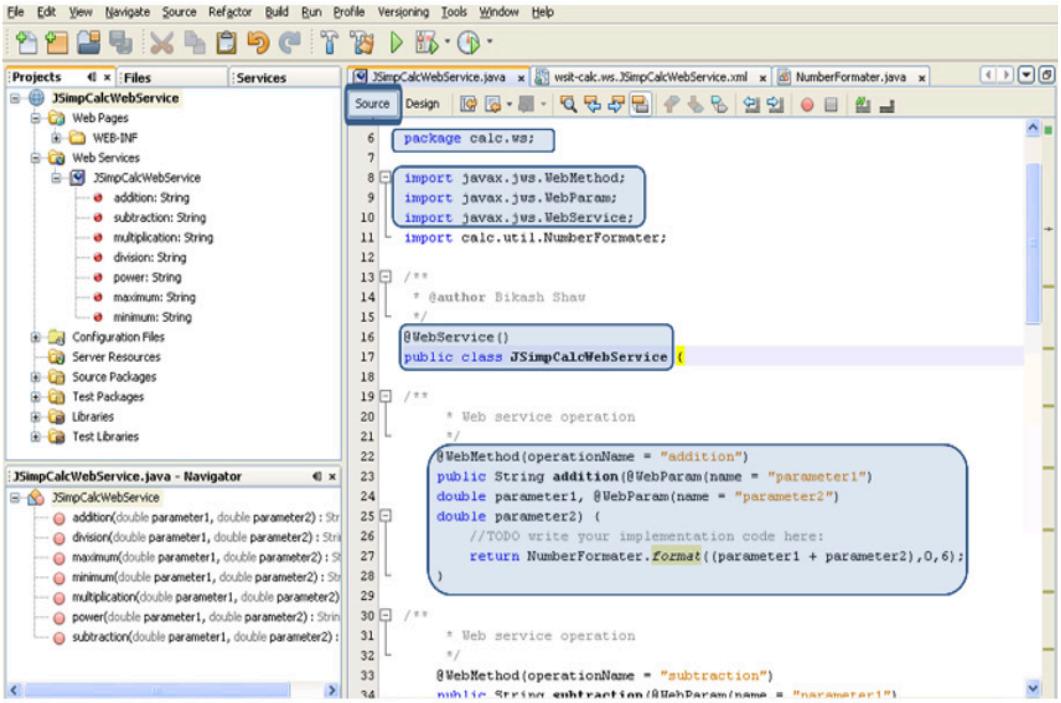


***Figure 12.35**: Code view of the web method*

The code should look as follows:

```
01. package calc.ws;
02. import javax.jws.WebMethod;
03. import javax.jws.WebParam;
04. import javax.jws.WebService;
05. import calc.util.NumberFormater;
06.
07. @WebService()public class JSimpCalcWebService { @WebMethod(operationName = "addition") public String addition(@WebParam(name = "parameter1")
08. double parameter1, @WebParam(name = "parameter2")   double parameter2) {
09.               return NumberFormater.format((parameter1 + parameter2),0,6);
10.     }
11.
12. @WebMethod(operationName = "subtraction") public String subtraction(@WebParam(name = "parameter1")double parameter1, @WebParam(name = "parameter2")    double parameter2) {
13.       return NumberFormater.format((parameter1 - parameter2),0,6);
14.     }
15.
16.  @WebMethod(operationName = "multiplication") public String multiplication(@WebParam(name = "parameter1") double parameter1, @WebParam(name = "parameter2")   double parameter2) {
17.         return NumberFormater.format((parameter1 * parameter2),0,6);
18.     }
19.
20. @WebMethod(operationName = "division") public String division(@WebParam(name = "parameter1")    double parameter1, @WebParam(name = "parameter2")  double parameter2) {
21.         return NumberFormater.format((parameter1 / parameter2),0,6);
22.     }
23.
24. @WebMethod(operationName = "power") public String power(@WebParam(name = "parameter1") double parameter1, @WebParam(name = "parameter2") double parameter2) {
25.         return NumberFormater.format(Math.pow(parameter1, parameter2),0,6);
26.     }
27.
28. @WebMethod(operationName = "maximum")   public String maximun(@WebParam(name = "parameter1")
29.     double parameter1, @WebParam(name = "parameter2")
30.     double parameter2) {
31.         return NumberFormater.format(Math.max(parameter1, parameter2),0,6);
32.     }
33.
34. @WebMethod(operationName = "minimum") public String minimum(@WebParam(name = "parameter1") double parameter1, @WebParam(name = "parameter2") double parameter2) {
35.         return NumberFormater.format(Math.min(parameter1, parameter2),0,6);
36.     }
37. }
```

***Figure 12.36**: Code view of the web service*

The following code shows method format to apply the **`Number`** format:

```
01. package calc.util;
02.
03. import java.text.NumberFormat;
04.
05. public class NumberFormater {
06.     public static String format(double number, int minFractionDigits,  int maxFractionDigits)
07.     {
08.         NumberFormat format = NumberFormat.getInstance();
09.         format.setMaximumFractionDigits(maxFractionDigits);
10.         format.setMinimumFractionDigits(minFractionDigits);
11.         return format.format(number);
12.     }
13. }
```

***Figure 12.37**: Code view of the main method*

Here, we use the **`NumberFormater.format`** method to format the double value to **`String`** up to six decimal point.

Deploy and test the web service

Start the GlassFish server using the following steps:

1. In the explorer window, go to **Services**.
2. Open the servers option.

3. Right-click on the server name.
4. Select **Start**.

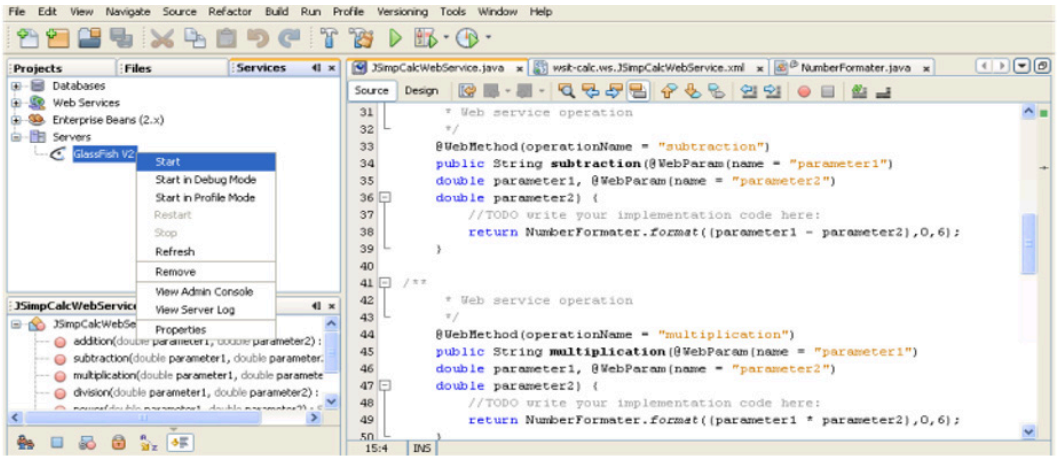


***Figure 12.38**: Start GlassFish V2 Application Server*

After the server gets started, we can deploy the service:

5. Go to **Projects** in the explorer window.
6. Expand the **Web Services** node.
7. Right-click on the web services.
8. Click on the **Test Web Service** menu.

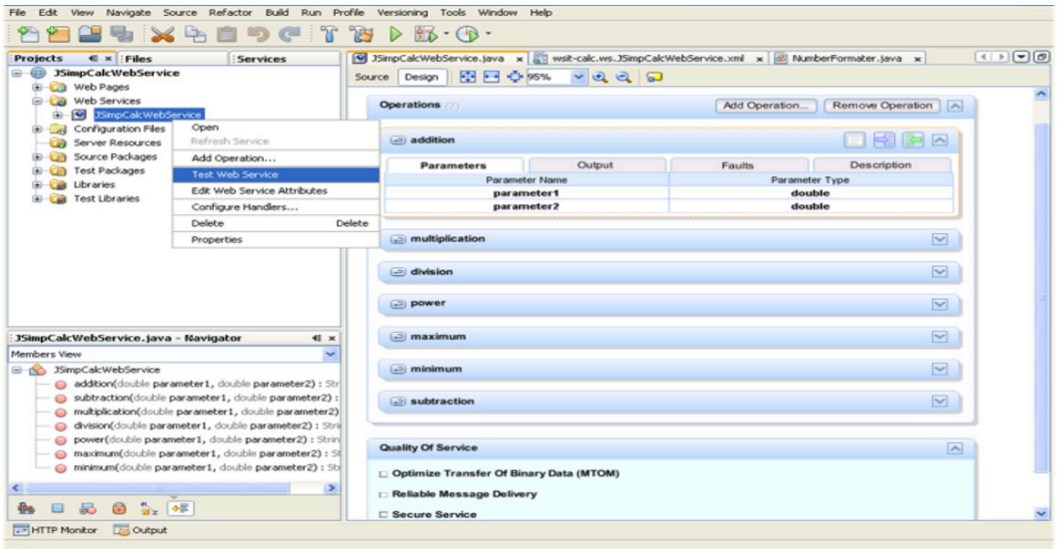


***Figure 12.39**: Test Web Service*

The application is deployed and opened in the default browser, and the service can be tested with a SOAP request and response.

***Figure 12.40**: Test Web Service*

We can also deploy and test the service in GlassFish application server's admin console, as shown in the following figure:

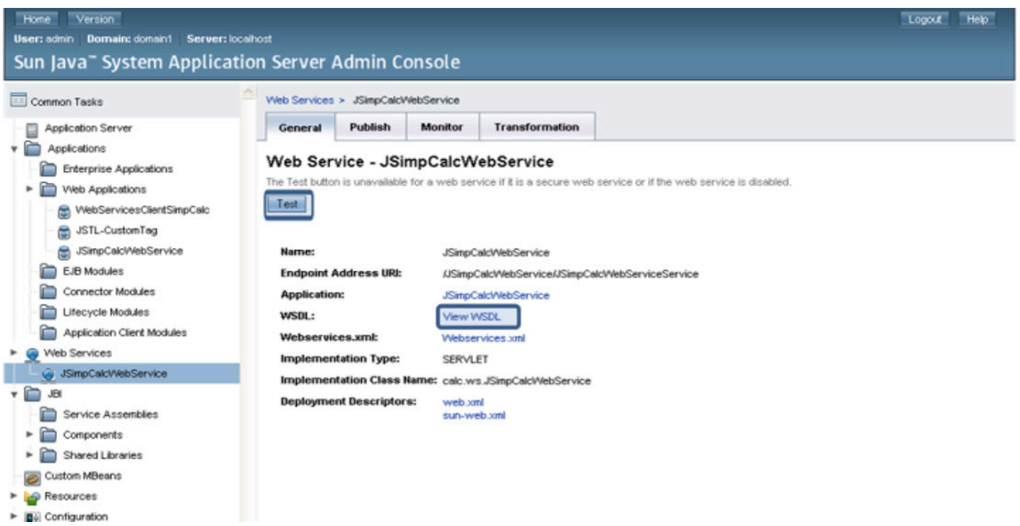
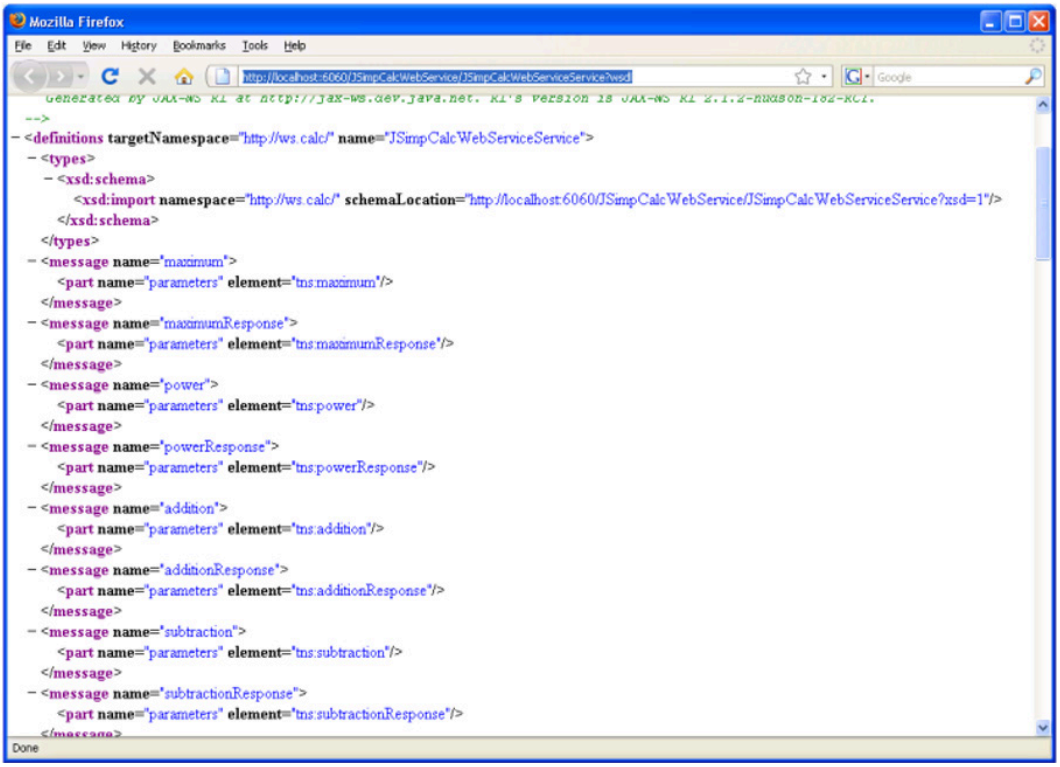


***Figure 12.41**: Test Web Service via Admin Console*

We can view and click on the WSDL link for our web service. The WSDL file will be as follows:




```
<!--
- <definitions targetNamespace="http://ws.calc/" name="JSimpCalcWebServiceService">
  - <types>
    - <xsd:schema>
        <xsd:import namespace="http://ws.calc/" schemaLocation="http://localhost:6060/JSimpCalcWebService/JSimpCalcWebServiceService?xsd=1"/>
      </xsd:schema>
    </types>
  - <message name="maximum">
      <part name="parameters" element="tns:maximum"/>
    </message>
  - <message name="maximumResponse">
      <part name="parameters" element="tns:maximumResponse"/>
    </message>
  - <message name="power">
      <part name="parameters" element="tns:power"/>
    </message>
  - <message name="powerResponse">
      <part name="parameters" element="tns:powerResponse"/>
    </message>
  - <message name="addition">
      <part name="parameters" element="tns:addition"/>
    </message>
  - <message name="additionResponse">
      <part name="parameters" element="tns:additionResponse"/>
    </message>
  - <message name="subtraction">
      <part name="parameters" element="tns:subtraction"/>
    </message>
  - <message name="subtractionResponse">
      <part name="parameters" element="tns:subtractionResponse"/>
```

***Figure 12.42**: WSDL File*

We can use the URL given in the address bar to invoke the web service.

Create ASP.net client

Our web service is now prepared to be called from development environments that do not use Java. We create a prototype ASP.net-based client in this post. It may be done using Visual Studio 2008 in a few easy steps:

- Build an ASP.net website
- Include a web address
- Create a web service call using code
- Test the client application for web services

Create the client

Start Visual Studio; in the file menu, select **New** and then **Web Site**, and select **ASP.net Web Site**:

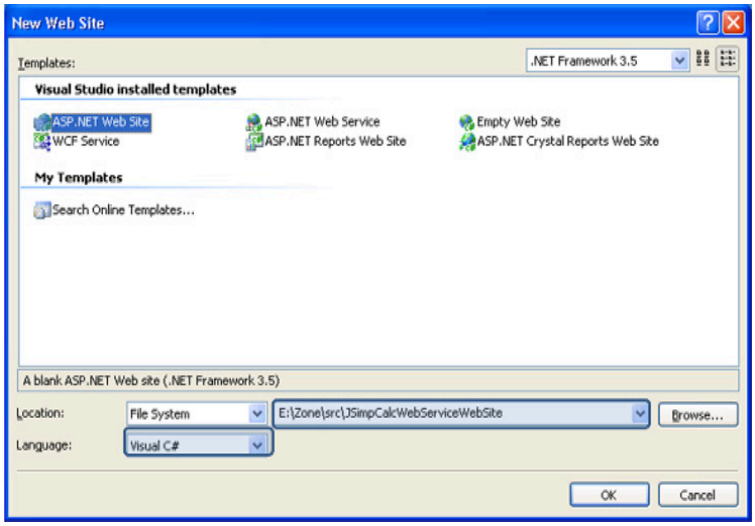


Figure 12.43: *New Website Window*

1. Specify the location and name of the folder where we want to save the site, in our case, it is **`JSimpCalcWebServiceWebSite`**.
2. Select the language as **Visual C#** and click on the **OK** button.

Provide the web reference

We have to specify the WSDL file in our website using the following steps:

1. In the **Explorer** window, select the **Solution** tab.
2. Right-click on the project name.

3. Click on the **Add Web Reference** menu. The **Add Web Reference** dialog opens.

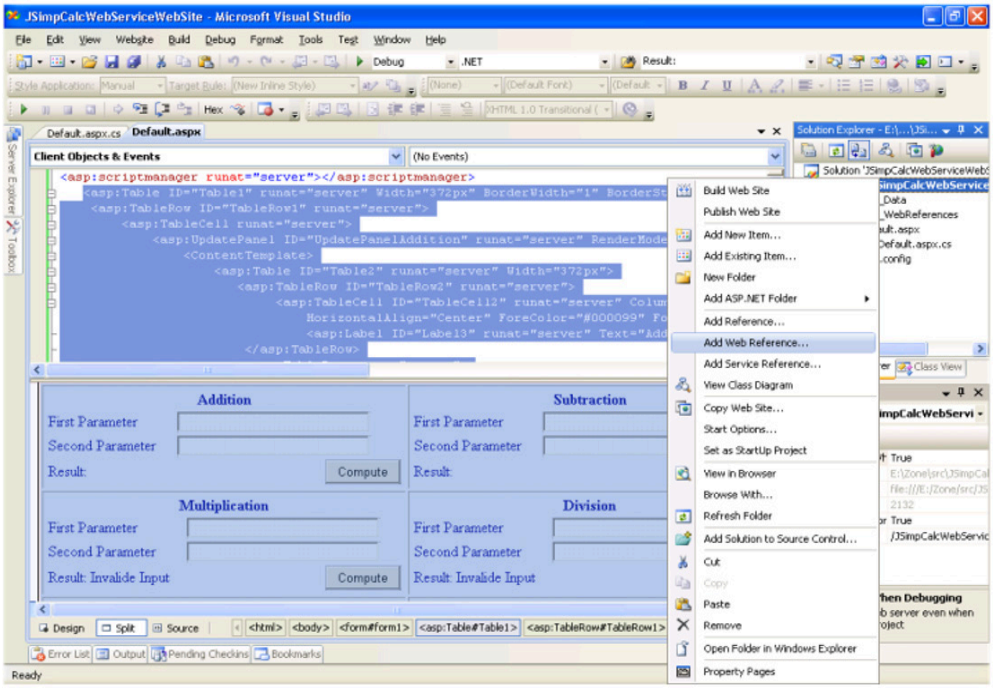


***Figure 12.44**: Web Reference Menu*

4. Select the WSDL URL from the browser's address bar and **Add Web Reference** dialog's address bar, and press the **Go** button:

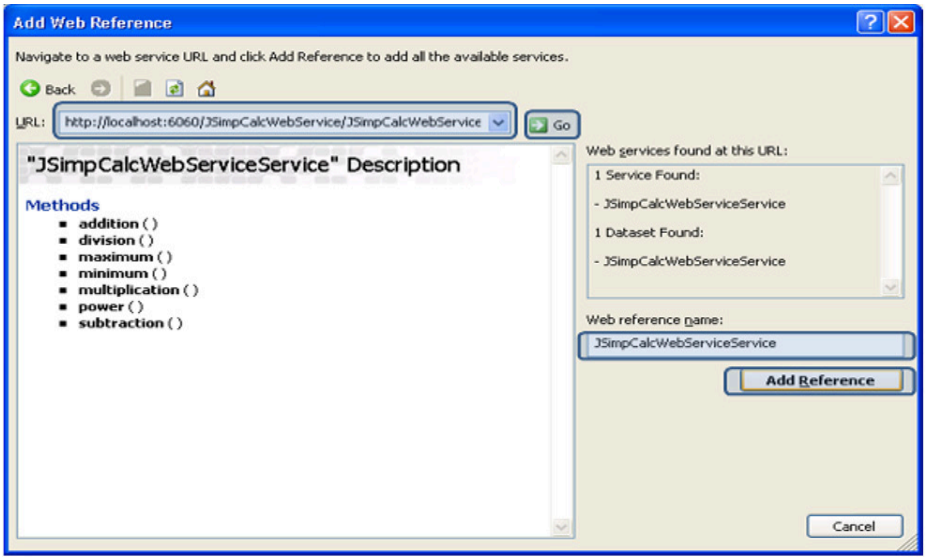


***Figure 12.45**: Web Reference Dialog box*

In the above figure it displays all the methods of our web service, we need to provide name of the web reference and click button.

Call a web service

Let us understand the steps to invoke the service:

1. Design an ASP.Net page and add the web reference in the page; consider the following example:

   ```
   using JSimpCalcWebServiceService;
   ```

2. Now, create the object of the web reference:

   ```
   JSimpCalcWebServiceService.JSimpCalcWebServiceService proxy = new
   JSimpCalcWebServiceService.JSimpCalcWebServiceService();
   ```

3. Retrieve the **`WebMethod`** as any simple method call. Consider this example:

   ```
   proxy.addition(10,20);
   ```

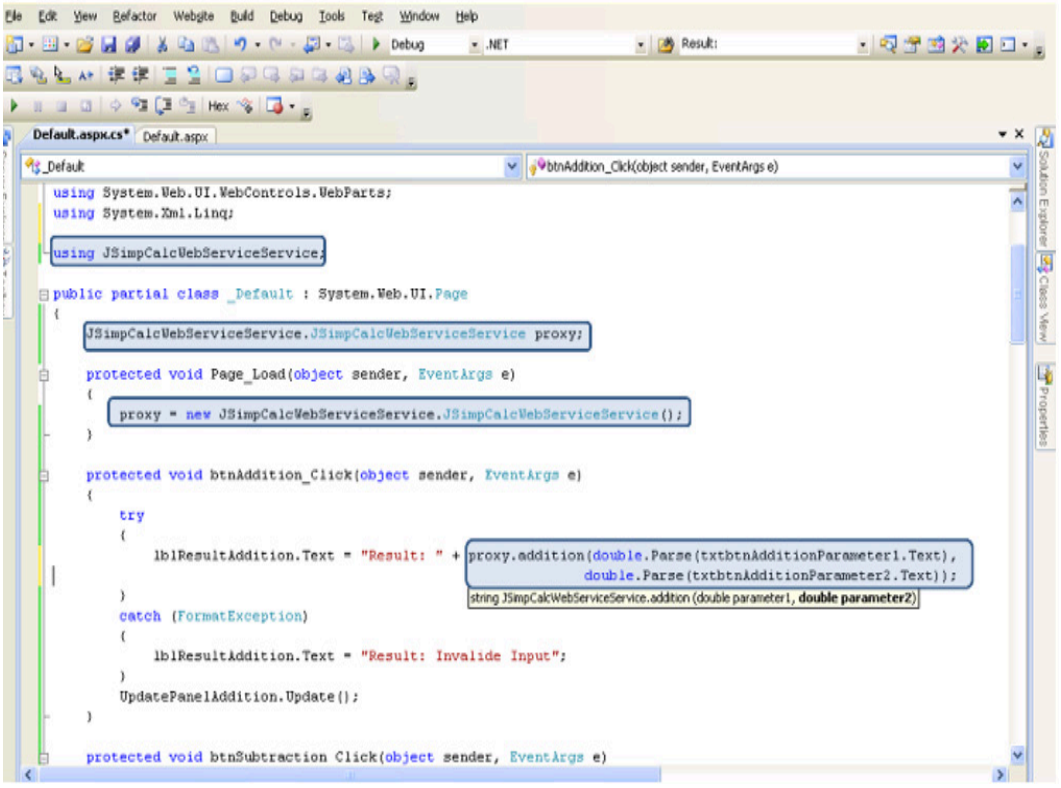


***Figure 12.46**: Web Methods call*

The code will look as follows:

Hide Shrink Copy Code

```
using System; using System.Configuration; using System.Data; using System.Linq; using System.Web;using System.Web.Security;
using System.Web.UI; using System.Web.UI.HtmlControls; using System.Web.UI.WebControls;
using System.Web.UI.WebControls.WebParts; using System.Xml.Linq; using JSimpCalcWebServiceService;

public partial class _Default : System.Web.UI.Page { JSimpCalcWebServiceService.JSimpCalcWebServiceService proxy;
    protected void Page_Load(object sender, EventArgs e) {
        proxy = new JSimpCalcWebServiceService.JSimpCalcWebServiceService();   }
    protected void btnAddition_Click(object sender, EventArgs e)  {
     try{   lblResultAddition.Text = "Result: " + proxy.addition(double.Parse(txtbtnAdditionParameter1.Text),
       double.Parse(txtbtnAdditionParameter2.Text));       }
        catch (FormatException){ lblResultAddition.Text = "Result: Invalide Input";   }
        UpdatePanelAddition.Update();    }
   protected void btnSubtraction_Click(object sender, EventArgs e) {
        try {   lblResultSubtraction.Text = "Result: " + proxy.subtraction(double.Parse(txtSubtractionParameter1.Text),
         double.Parse(txtSubtractionParameter2.Text));    }
        catch(FormatException) { lblResultSubtraction.Text = "Result: Invalide Input";  }
        UpdatePanelSubtraction.Update();    }
    protected void btnMultiplication_Click(object sender, EventArgs e) {
        try{  lblResultMultiplication.Text = "Result: " + proxy.multiplication(double.Parse(txtMultiplicationParameter1.Text),
         double.Parse(txtMultiplicationParameter2.Text));   }
        catch (FormatException){   lblResultMultiplication.Text = "Result: Invalide Input";  }
        UpdatePanelMultiplication.Update();    }
    protected void btnDivision_Click(object sender, EventArgs e) {
        try {  lblResultDivision.Text = "Result: " + proxy.division(double.Parse(txtDivisionParameter1.Text),
        double.Parse(txtDivisionParameter2.Text));     }
        catch (FormatException){   lblResultDivision.Text = "Result: Invalide Input";    }
        UpdatePanelDivision.Update();    }
     protected void btnMaximum_Click(object sender, EventArgs e)  {
        try {   lblResultMaxMin.Text = "Result: " + proxy.maximum(double.Parse(txtMaxMinParameter1.Text),
       double.Parse(txtMaxMinParameter2.Text));       }
      catch (FormatException){ lblResultMaxMin.Text = "Result: Invalide Input";   }
     UpdatePanelMaxMin.Update();    }
    protected void btnMinimum_Click(object sender, EventArgs e) {
        try {   lblResultMaxMin.Text = "Result: " + proxy.minimum(double.Parse(txtMaxMinParameter1.Text),
        double.Parse(txtMaxMinParameter2.Text));    }
        catch (FormatException){ lblResultMaxMin.Text = "Result: Invalide Input";   }
        UpdatePanelMaxMin.Update();    }
    protected void btnPower_Click(object sender, EventArgs e) {
        try{ lblResultPower.Text = "Result: " +
            proxy.power(double.Parse(txtPowerParameter1.Text), double.Parse(txtPowerParameter2.Text));   }
        catch (FormatException){ lblResultPower.Text = "Result: Invalide Input";  }
        UpdatePanelPower.Update(); }
}
```

***Figure 12.47**: Asp.net page Source Code*

Test web service client application

To test the web service, we can start debugging or press the *F5* key. The web page will look as follows:

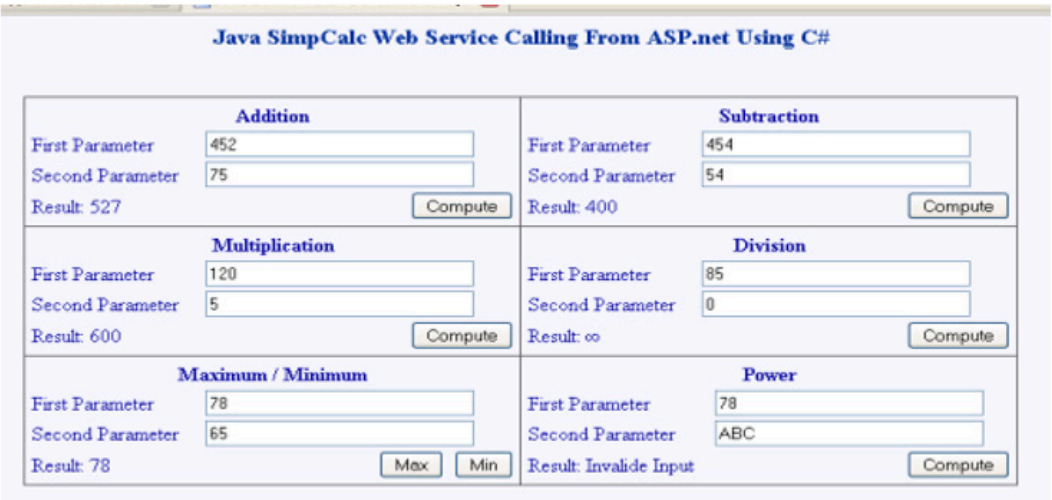


***Figure 12.48**: Web Site Output*

Conclusion

The chapter introduced the fundamentals of web services and discussed comprehending various web service kinds, producing web services, utilizing web services, and evaluating the interoperability offered by web services.

The use and applications of the spring framework will be covered in the next chapter.

Points to remember

- Web services are self-sufficient; no additional software is needed.
- A programming language with XML/HTML support is needed on the client side for client support. A web server and a SOAP server are required on the server side.
- Client and server are just aware of messages since they are loosely connected; a low level of coordination enables more flexible reconfiguration.
- Web services are distributed, located, and called using well-established, minimal internet standards.
- Different environments and languages may be used to implement the client and server.
- Workflow approaches can be used to combine web services and carry out higher-level business operations.
- Web service binding and discovery can be automated with UDDI and WSDL.

Multiple choice questions

1. **_____ is a software that implements a service specification.**
    a. Services
    b. Service Provider
    c. Service Consumer
    d. Service Broker

2. **In service-oriented architecture the, _____ service aspect provides the surety of consistent result.**
    a. Policy
    b. Security
    c. Transaction
    d. Management

3. **Which of the following points about web applications is not true?**
    a. Meant for applications to access data
    b. Always use the HTTP/HTTPS protocol
    c. Capable of maintaining user session
    d. Can access web services to access some data or to perform some tasks

4. **What point is true about Restful web services**
    a. It is a standard communication protocol.
    b. It only permits XML data formats.
    c. It supports both SSL security and WS security.
    d. REST is a better choice for exposing data; SOAP WS might be a better choice for logic.

Answers

1. **b**
2. **c**
3. **a**
4. **d**

Questions

1. What is a web service?
2. What is the difference between SOA and a web service?
3. What is the web service protocol stack?
4. What is SOAP?
5. What is WSDL?
6. What is UDDI?
7. What are the differences between both SOAP WS and RESTful WS?

Key terms

- **Services**: They are self-contained modules that can be used by one or more published interfaces.
- **Service provider**: It implements a service specification.
- **Service consumer**: It requests to call a service provider. A service consumer can be another service or an end user application.
- **Service locator**: It acts as a registry and is responsible for examining service provider interfaces and service locations.
- **Service broker**: It is a service provider that pass service requests to one or more additional service providers.

Join our book's Discord space

Join the book's Discord Workspace for Latest updates, Offers, Tech happenings around the world, New Release and Sessions with the Authors:

https://discord.bpbonline.com

Chapter 13
Aspect-Oriented Approach with Spring Framework

Introduction

This chapter gives basic knowledge of how to use the Spring framework so that one can understand and use the framework to its best. It is important to understand the spring architecture and its directory structure to start working with the spring application, and this chapter gives a brief introduction to it. The other significant concepts include dependency injection, creating and understanding bean and bean scopes, applying validation, and performing internationalization.

Structure

In this chapter, the following topics will be covered:

- Software framework
- Spring introduction
- Ioc container
- Dependencies
- Bean scopes
- Bean definition inheritance

- ApplicationContext
- Applying validation
- Aspect-oriented programming

Objectives

After studying this chapter, you should be able to understand the basic concepts of spring framework and apply them to create the application using these concepts. You should also be able to understand basic application structure, create beans and keep them in required scopes, apply dependency injections using different approaches, apply validations, and understand aspect-oriented programming.

Software framework

The purpose of a framework is to provide basic functionalities in terms of different project structures, drag and drop pallets, autogeneration of basic routine code in a default structure based on the type of file, provide easy compile and build facility, etc. These functionalities enable programmers to develop the applications easily with less effort and time needed for writing and creating the routine code.

Application frameworks offer several advantages:

- Already existing code can be reused, which saves a lot of time and money.
- Based on functionality, different teams can work parallelly; for example, software framework configuration and application development teams can work separately.
- Security is implicitly provided by the framework, mostly with the use of secure session management and escaping database input.
- Low-level tasks are taken care of by the framework.
- Frameworks offer all the basic and advance operations, for example, a web application framework mostly offers session management, validations, configuring payment gateways, etc.
- Frameworks can assist in programming design patterns.
- Application functionality can be enhanced automatically, without writing extra code when the framework is upgraded.
- The software framework provides several facilities in software development, so it saves lots of time and effort of the programmers.

There are also a few drawbacks of using a framework:

- When common code is used, performance can be degraded sometimes.
- To use the framework effectively, prior knowledge of using its feature to the best is required.
- Functionality that needs to bypass or work around deficiencies in a framework can cause more programming issues than developing the full functionality in the first place. This is not the fault of the framework, but few frameworks are more generally suited and flexible than others.
- Bugs and security issues in a framework can affect every application using that framework.
- When working with a framework for the first time, one may find it difficult to understand how to develop an application using it, but once the flow and structure of framework is understood, it allows faster development with less effort.
- The framework needs to be extended with specific code to develop any specific software.

Types of Java frameworks:

- The Java collection framework consists of various classes and interfaces that are used to perform operations on various data.
- A Java logging framework is used to record transactions; the most common framework includes JCL, log4j, etc.
- Several frameworks have been developed for the Java platform, like servlets API, JSP, Struts, Tiles, and Spring Framework.
- The **Java Media Framework** (**JMF**) enables adding audio, video and other media to the Java applications and applets.
- Spring is a data persistence framework.
- Spring framework allows us to handle transactions, so programmers need not perform low-level programming.
- Spring provides strong database error handling support by giving JDBC exception handling facility, which allows us to identify and handle database exceptions.
- Hibernate is a strong feature while developing web application; spring provides integration with Hibernate, so it eases many tasks.
- Spring framework works on the concept of aspect-oriented programming.

- Based on the essential Spring capabilities, Spring includes an MVC web application framework. This framework offers several views, including JSP, velocity, and tiles.

Spring introduction

Spring is used to develop applications that can support transaction management, web services, mailing facilities, and persistence database operations, and perform all routine tasks with minimum configurations.

Spring architecture

There are seven different modules that are well-organized and form the Spring architecture:

- **Spring AOP**: This layer offers the Declarative enterprise services like EJB declarative services. It manages the Declarative transaction management, which expands on Spring's transaction abstraction.
- **Spring ORM**: This layer offers access to databases. For well-known object-relational mapping APIs like JDO, Hibernate, and iBatis, it offers integration layers.
- **Spring Web**: This layer offers the Spring web application development stack, which contains Spring MVC, and includes the Spring Web module.
- **Spring DAO**: This layer offers data access object support to standardize the data access work that uses JDBC, Hibernate, or JDO.
- **Spring Context**: This layer offers support for message sources, the observer design pattern, and the capability for application objects to access resources using a unified API; this package expands upon the beans package.
- **Spring Web MVC**: This layer provides the MVC implementations for web applications.
- **Spring Core**: This layer provides the basic and important functionalities.

The characteristics of dependency injection are provided by this component. `BeanFactory` offers a factory pattern that isolates your actual programme logic from dependencies like object initialization, creation, and access.

The following diagram represents the components that forms the framework:

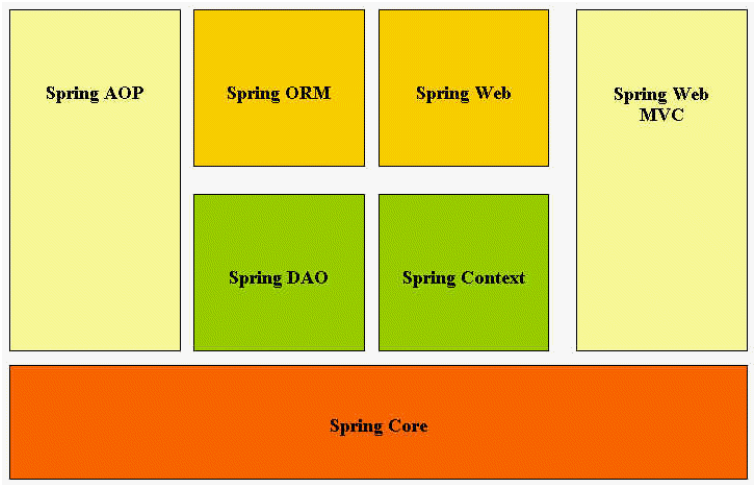


Figure 13.1: *Architecture of Spring Framework*

Spring directory structure

When a new Spring project is created, a specific directory structure is created to hold and organize the files. The following files are included in the directory structure: **`build.bat`**, **`build.xml`**, **`build-continuous.xml`**, **`changelog.txt`**, **`license.txt`**, **`maven.xml`**, **`notice.txt`**, **`project.properties`**, **`readme.txt`**, and **`tests.bat`**. The following is a screenshot of the directory structure:

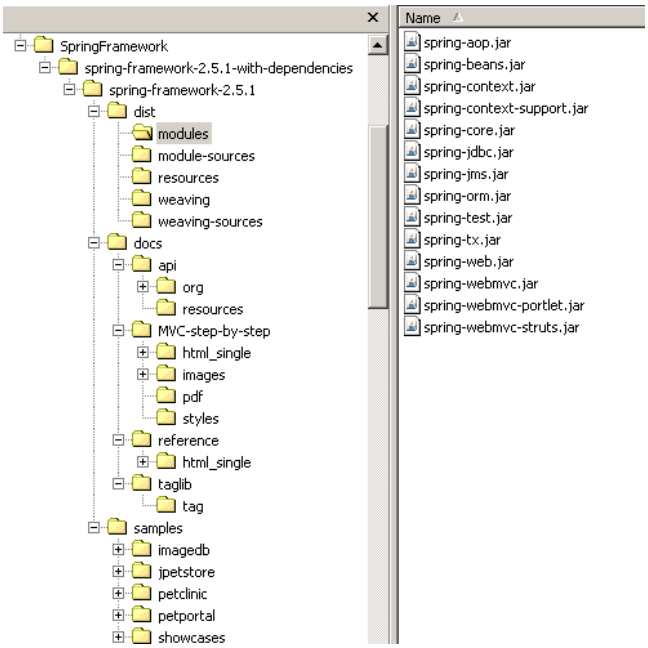


Figure 13.2: *Spring Project Directory Structure*

The library files for the spring modules are in the spring framework's **`dist`** directory. These JAR files are our focus. These JAR files will be copied into our development environment. The following JAR files make up the Spring framework: **`spring-aop.jar`**, **`spring-beans.jar`**, **`spring-context.jar`**, **`spring-context-support.jar`**, **`spring-core.jar`**, **`spring-jdbc.jar`**, **`spring-jms.jar`**, **`spring-orm.jar`**, **`spring-test.jar`**, **`spring-tx.jar`**, **`spring-web.jar`**, **`spring-webmvc-portlet.jar`**, and **`spring-webmvc-strut`**. The lib directory, which holds the library files needed by the application, is another crucial directory.

IoC container

Beans are created and configured using a bean factory. In addition to doing that, an application context offers the infrastructure necessary to enable many enterprise-specific capabilities like transactions and AOP. The IoC container for the Spring Framework is built on top of the beans and context packages that are part of the **`org.springframework`** package. The **`BeanFactory`** interface offers a sophisticated setup system that can handle objects of any kind. On top of the **`BeanFactory`**, the **`ApplicationContext`** interface offers new features, including event handling, managing resource bundles, and application-specific features like **`WebApplicationContext`** for usage in online applications. It also makes it simpler to integrate with Spring's AOP features.

Containers and beans

Spring containers manage objects using beans. The **`BeanFactory`** interface is used to store and maintain the beans. It finds or creates application objects, configures those objects, and assembles the dependencies between those components. The **`XmlBeanFactory`** class uses the XML configuration metadata to create a fully organized application:

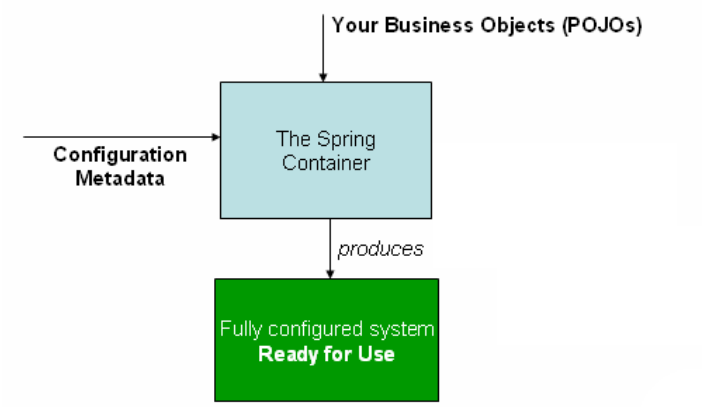


Figure 13.3: *XML configuration metadata creating organized application*

The container checks and looks for configuration metadata to start with an application.

Resources

The path to an **`ApplicationContext`** is actually resource strings, that enable the container to provide the details from different resources, like, the file system present on the local machine, the Java **`CLASSPATH`** and etc. The container must manage at least one bean definition as part of the spring setup; however, there will usually be more than one. The following XML file shows the configuration details:

```
<?xml version="1.0" encoding="UTF-8"?>
<beans xmlns="http://www.springframework.org/schema/beans"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="http://www.springframework.org/schema/beans http://
www.springframework.org/schema/beans/spring-beans-2.0.xsd">
  <bean id="..." class="...">
  </bean>
  <bean id="..." class="...">
  </bean>
</beans>
```

Container settings

To provide the details of the container, we can use the following code:

```
        <?xml version="1.0" encoding="UTF-8"?>
<beans xmlns="http://www.springframework.org/schema/beans"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="http://www.springframework.org/schema/beans http://
www.springframework.org/schema/beans/spring-beans-2.0.xsd">
  <bean id="..." class="...">
  </bean>
  <bean id="..." class="...">
  </bean>
</beans>
```

We can provide the details regarding the resources to be used in the application using XML files:

```
<beans>
    <import resource="services.xml"/>
```

```
    <import resource="resources/messageSource.xml"/>
    <import resource="/resources/themeSource.xml"/>
    <bean id="bean1" class="..."/>
    <bean id="bean2" class="..."/>
</beans>
```

The beans

The beans are handled by the container, and they are identified using ID or name attribute, as shown here:

```
<bean id="bookBean" class="examples.BookBean"/>
<bean name="libraryBean" class="examples.ExampleLibraryBean"/>
```

Using the container

The **`BeanFactory`** methods can be used in XML format:

```
BeanFactory factory = new XmlBeanFactory(is);
InputStream factory = new FileInputStream("beans.xml");
```

Following methods are provided by the interface to manage the beans:

```
boolean containsBean(String)
Object getBean(String)
Object getBean(String, Class)
class getType(String name)
boolean isSingleton(String)
String[] getAliases (String)
```

The first method returns Boolean values based on the occurrence of the bean present in **`BeanFactory`**. The second method returns bean on the occurrence of the bean or else throws an exception. The third method returns a bean or else throws an exception when the casting of bean fails. The fourth method returns the bean class or throws an exception. The fifth method returns Boolean value when the bean is mapped in the singleton scope or else throws an exception. The sixth method returns the alias name for the given bean.

Dependencies

For performing various tasks when working on a common project, we can inject dependencies to the object using setter methods or parameterized constructors.

Using setter methods and parameterized constructor for injecting dependencies

We can use setter methods on the beans as shown here:

```
public class BookLister {
       private BookFinder bookFinder;
    public void setBookFinder(BookFinder bookFinder) {
        this.bookFinder = bookFinder;
    }
}
```

We can use constructors as shown here:

```
public class BookLister {
    private BookFinder bookFinder;
    public BookLister(BookFinder bookFinder) {
        this.bookFinder = bookFinder;
    }
   }
```

Both ways of implementing DI are good, but when we have large number of arguments, it is better to use setter methods.

Ambiguous dependencies

Sometimes we may write code that may lead to unresolvable dependencies; the following code shows the relations of multiple beans:

```
<bean id="exampleBean" class="examples.ExampleBean">
<property name="beanOne"><ref bean="BeanOne"/></property>
  <property name="beanTwo" ref="BeanTwo"/>
  <property name="integerProperty" value="1"/>
</bean>
<bean id="bean1" class="examples.BeanOne"/>
<bean id="bean2" class="examples.BeanTwo"/>

public class ExampleBean {
    private BeanOne beanOne;
    private BeanTwo beanTwo;
```

```
    private int i;

    public void setBeanOne(BeanOne beanOne) {
        this.beanOne = beanOne;
    }
    public void setBeanTwo(Bean beanTwo) {
        this.beanTwo = beanTwo;
    }

    public void setIntegerProperty(int i) {
        this.i = i;
    }
}
```

The following code shows the Java class, along with mapping of XML configuration:

```
<bean id="exampleBean" class="examples.ExampleBean">
<constructor-arg><ref bean="beanOne"/></constructor-arg>
<constructor-arg ref="beanTwo"/>
<constructor-arg type="int" value="1"/>
</bean>
<bean id="beanOne" class="examples.BeanOne"/>
<bean id="beanTwo" class="examples.BeanTwo"/>
public class ExampleBean {

    private BeanOne beanOne;
    private BeanTwo beanTwo;
    private int i;

    public ExampleBean(
    BeanOne beanOne, BeanTwo beanTwo, int i) {
        this.beanOne = beanOne;
        this.beanTwo = beanTwo;
        this.i = i;
    }
}
```

The **exampleBean** will take the arguments as input, and the call to static method will return an instance as shown here:

```
<bean id="exampleBean" class="examples.ExampleBean"
      factory-method="createInstance">
  <constructor-arg ref="beanOne"/>
  <constructor-arg ref="beanTwo"/>
  <constructor-arg value="1"/>
</bean>

<bean id=" beanOne " class="examples.BeanOne"/>
<bean id=" beanTwo " class="examples.BeanTwo"/>
public class ExampleBean {
    private ExampleBean(...) {
    }

  public static ExampleBean createInstance (
   BeanOne beanOne, BeanTwo beanTwo, int i) {
        ExampleBean eb = new ExampleBean (...);
        return eb;
    }
  }
}
```

Collections

The tags like list, set, map and props allow Java Collections like List, Set, Map, and Properties to be defined and set.

```
<bean id="collectionsObject" class="utils.CollectionsExample">
<property name="emailList">
  <props>
    <prop key="administrator">admin@mycompany.org</prop>
    <prop key="helpdesk">helpdesk@mycompany.org</prop>
    <prop key="tester">tester@mycompany.org</prop>
  </props>
</property>
  <property name="list1">
```

```
    <list>
        <value>List element</value>
        <ref bean="myDataSource" />
    </list>
  </property>
  <property name="map1">
    <map>
        <entry>
            <key>
                <value>value as an entry </value>
            </key>
            <value>value as a string</value>
        </entry>
        <entry>
            <key>
                <value>value as a ref</value>
            </key>
            <ref bean="myDataSource" />
        </entry>
    </map>
  </property>
  <property name="set1">
    <set>
        <value>value as just a string</value>
        <ref bean="myDataSource" />
    </set>
  </property>
</bean>
```

We can apply properties inheritance by specifying the parent relationship in child tag using the **parent** attribute:

```
<beans>
<bean id="parent" abstract="true"
                  class="utils.CollectionsExample ">
    <property name=" emailList ">
      <props>
        <prop key="administrator">admin@company.com</prop>
```

```
        <prop key="helpdesk">helpdesk@company.com</prop>
      </props>
    </property>
</bean>
<bean id="child" parent="parent">
    <property name="adminEmails">
        <props merge="true">
            <prop key="sales">sales@company.com</prop>
            <prop key="helpdesk">support@company.co.in</prop>
        </props>
    </property>
</bean>
<beans>
```

We can merge the collection by setting the merge attribute to true; the following property will take the parent tag values:

```
administrator=admin@company.com

sales=sales@company.com

helpdesk=helpdesk@company.co.in
```

The merge attribute is applied to the child tag, the values are retrieved in hierarchy, so the parents value are followed by child values.

Checking for dependencies

The dependencies are checked for various mappings manually. Using XML approach, we can check with different modes. Like **`none`** means the dependencies are not checked, the **`simple`** mode is used to check dependency for primitive types and collections, the **`object`** mode is used to check dependency on collaborations, and **`all`** is used to check dependency for everything mentioned earlier.

Method injection

The beans are checked for dependencies and their life cycle, as shown here:

```
package company.mypackage;
import org.springframework.beans.BeansException;
import org.springframework.beans.factory.BeanFactory;
import org.springframework.beans.factory.BeanFactoryAware;
```

```
public class CommandManager implements BeanFactoryAware {
   private BeanFactory beanFactory;
   public Object process(Map commandState) {
      Command command = createCommand();
      command.setState(commandState);
      return command.execute();
   }

   protected Command createCommand() {
      return (Command) this.beanFactory.getBean("command");
   }

   public void setBeanFactory(BeanFactory beanFactory) throws
BeansException {
      this.beanFactory = beanFactory;
   }
}
```

Bean scopes

Beans can be set on different scopes like singleton, prototype, request, session, and global session.

The singleton scope

In singleton scope, we set one bean for an object instance. This is the default scope:

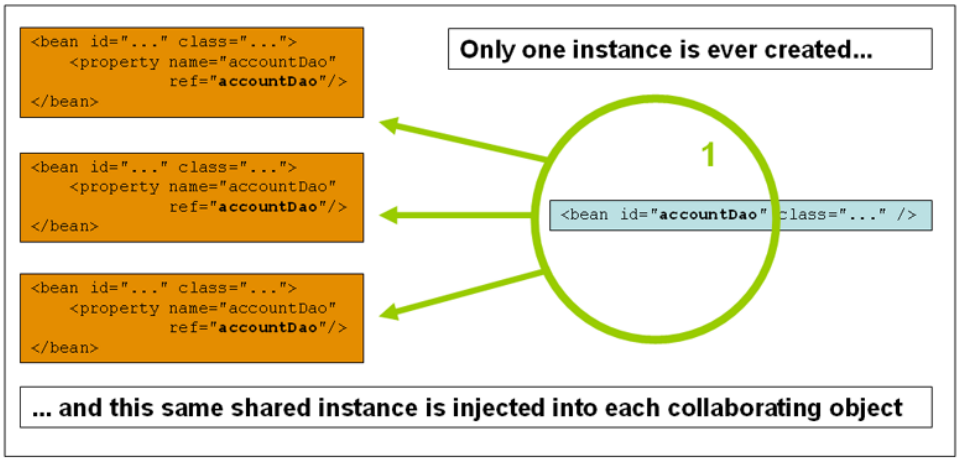


***Figure 13.4**: Default scope*

We can explicitly define the bean as follows:

```
<bean id="accountService" class="com.mypackage.DefaultAccountService"/>
<bean id="accountService" class="com.mypackage.DefaultAccountService"
scope="singleton"/>
<bean id="accountService" class="com.mypackage.DefaultAccountService"
singleton="true"/>
```

The prototype scope

In prototype scope, a single bean can be defined for more than one objects, as required:

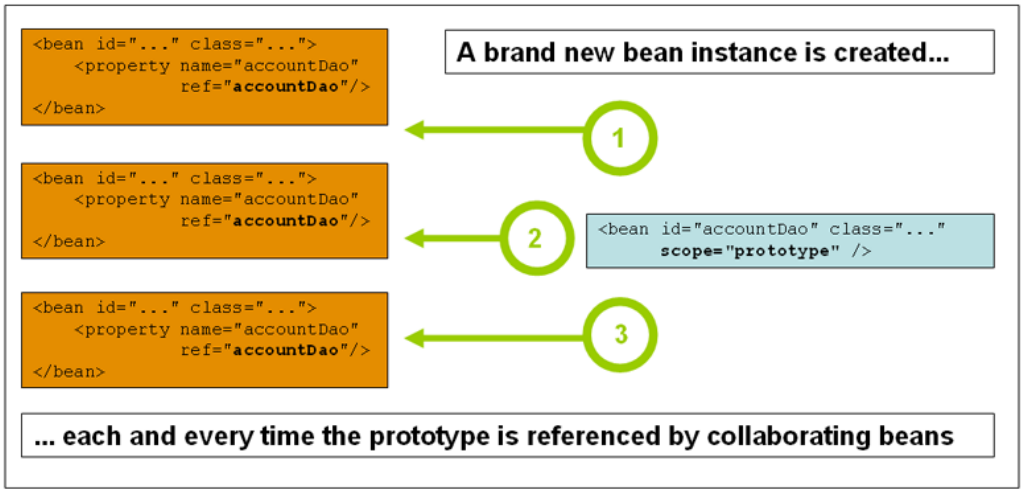


***Figure 13.5**: Scope of bean*

We can set the scope of bean as prototype using the following syntax:

```
<bean id="accountService" class="com.mypackage.DefaultAccountService"
scope="prototype"/>
<bean id="accountService" class="com.mypackage.DefaultAccountService"
singleton="false"/>
```

After the use of beans, they should be removed from the scope and all the resources should be released; this is the programmer's responsibility.

The request scope

In request scope, a bean for each HTTP request is created; we set it as shown here:

```
<bean id="loginAction" class="com.mybean.LoginAction" scope="request"/>
```

The **loginAction** bean has a scope for HTTP request, so whatever changes we make are applied to a single request; they will not affect other requests that use instances from the **loginAction** bean. The bean will be destroyed after the request has been processed.

The session scope

In session scope, a bean is defined for the lifetime of a session, as shown here:

```
<bean id=”userPreferences” class=”com.mybean.UserPreferences”
scope=”session”/>
```

The **userPreferences** bean is created in the session scope, so it is available between the request throughout the single session. When the session expires, the bean is destroyed.

The global session scope

In global session, the bean is defined for multiple sessions in the **ApplicationContext**, as shown here:

```
<bean id="userPreferences" class="com.mybean.UserPreferences"
scope="globalSession"/>
```

The bean defined in this scope is accessed across multiple sessions.

Custom scopes

We can also define our own scope; the interface **org.springframework.beans.factory.config.Scope** is used to create the bean. The following methods provide the handling of beans in custom scope:

- **Object get (String name,** ObjectFactory objectFactory**)**: It returns the object from the given scope.
- **Object remove (String name)**: It is used to remove the bean from the defined scope.
- **void** registerDestructionCallback**(String name, Runnable** destructionCallback**)**: It is used when the scope is destroyed or specific object is destroyed in the scope.
- **String** getConversationId**()**: It is used to get an identifier for the given scope.

Using a custom scope

When we create a new custom scope, we have to explicitly inform the container; the **`ConfigurableBeanFactory`** interface provides a method to register the new scope:

```
void registerScope(String scopeName, Scope scope);
```

The first argument is the name associated with a scope, and the second argument is the instance of the custom scope that we want to register and use. The following code is used to register the custom scope:

```
Scope customScope = new ThreadScope();
beanFactory.registerScope(thread, scope);
```

We can create bean definitions for the custom scope, as follows:

```
<bean id="..." class="..." scope="thread"/>
```

We can register the custom scope programmatically and also using declarative architecture, as shown here:

```
<?xml version="1.0" encoding="UTF-8"?>
<beans xmlns="http://www.springframework.org/schema/beans"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xmlns:aop="http://www.springframework.org/schema/aop"
xsi:schemaLocation="
http://www.springframework.org/schema/beans http://www.springframework.
org/schema/beans/spring-beans-2.0.xsd
http://www.springframework.org/schema/aop http://www.springframework.
org/schema/aop/spring-aop-2.0.xsd">


    <bean class="org.springframework.beans.factory.config.
CustomScopeConfigurer">
        <property name="scopes">
            <map>
                <entry key="thread">
                    <bean class="com.example.ThreadScope"/>
                </entry>
            </map>
        </property>
    </bean>
    <bean id="book" class="com.example.Book" scope="thread">
```

```
        <property name="name" value="Java"/>
        <aop:scoped-proxy/>
    </bean>

    <bean id="person" class="com.example.Person">
        <property name="book" ref="book"/>
    </bean>
</beans>
```

Bean definition inheritance

A child bean can inherit the parent **bean** attribute values. We can use the parent attribute to indicate the parent bean value to be used in the child bean, as shown here:

```
<bean id="inheritedTestBean" abstract="true"    class="org.
springframework.beans.TestBean">
  <property name="name" value="parent"/>
  <property name="age" value="50"/>
</bean>
<bean id="inheritsWithDifferentClass"    class="org.springframework.
beans.DerivedTestBean"
      parent="inheritedTestBean" init-method="initialize">
   <property name="name" value="override"/>
 </bean>
```

The child class bean must be compatible with the parent and accept the parent's property values.

The following code shows the use of parent child property values usage:

```
<bean id="inheritedTestBeanWithoutClass" abstract="true">
    <property name="name" value="parent"/>
    <property name="age" value="50"/>
</bean>
<bean id="inheritsWithClass" class="org.springframework.beans.
DerivedTestBean"
    parent="inheritedTestBeanWithoutClass" init-method="initialize">
  <property name="name" value="override"/>
</bean>
```

The parent is declared as abstract, so it acts as a template; we cannot create its instances.

The ApplicationContext

`ApplicationContext` is used to create beans using the declarative way. It also offers internalization that supports message organization using **`MessageSource`**; it is used to access various resources, manage events, and allow working across multiple contexts.

Internationalization using MessageSources

The facility of i18n, that is, internationalization, is provided by the interface **`MessageSource`** and **`HierarchicalMessageSource`**; the following methods are provided by the interface:

- **`String getMessage(String code, Object[] args, String default, Locale loc)`**: It is used to get the message; when message is not found, the default message is returned.
- **`String getMessage(String code, Object[] args, Locale loc)`**: It is used to get the message; when message is not found, it throws a **`NoSuchMessageException`**.
- **`String getMessage(MessageSourceResolvable resolvable, Locale locale)`**: This method returns all the properties.

While loading **`ApplicationContext`**, **`MessageSource`** is searched implicitly; when found, it is used by the method calls mentioned earlier. Otherwise it will search for parent containing the same name; when found, it uses the **`MessageSource,`** otherwise it will return empty **`StaticMessageSource`**. **`ResourceBundleMessageSource`** is also used to handle messages; **`ResourceBundleMessageSource`** is used more frequently than **`StaticMessageSource`**. The following example shows the use of **`ResourceBundleMessageSource`**:

```
<beans>
  <bean id="messageSource"
        class="org.springframework.context.support.
ResourceBundleMessageSource">
    <property name="basenames">
      <list>
        <value>format</value>
        <value>exceptions</value>
        <value>windows</value>
```

```
      </list>
    </property>
  </bean>
</beans>
```

In the example, we created three resource bundles: format, exceptions and windows. The properties file to define the messages is shown here:

```
# in 'format.properties'
message=Spring Framework
# in 'exceptions.properties'
argument.required=The '{0}' argument is required.
```

We can cast the **ApplicationContext** implementations to **MessageSource** implementations as here:

```
public static void main (String [] args) {
    MessageSource resources = new ClassPathXmlApplicationContext("beans.
xml");
    String message = resources.getMessage("message", null, "Default",
null);
    System.out.println(message);
}
```

The output of the preceding program will be as follows:

```
Spring Framework
```

The **MessageSource** is defined in a **beans.xml** file, and the **messageSource** bean refers to a number of resource bundles through its **basenames** property. To create **Tname**, for example, to set the British (en-GB) locale, we need to create files as **format_en_GB.properties**, **exceptions_en_GB.properties**, and **windows_en_GB.properties**. The appropriate locale file will be searched by the application automatically. We can also resolve the locale manually, as shown here:

```
argument.required=The,  '{0}' argument is required.
public static void main(final String[] args) {
    MessageSource resources = new ClassPathXmlApplicationContext("beans.
xml");
    String message = resources.getMessage("argument.required",
        new Object [] {"userDao"}, "Required", Locale.UK);
    System.out.println(message);
}
```

The output of the program will be as follows:

```
The, 'userDao' argument is required.
```

The **MessageSource** that has already been defined can also be referred to by the **MessageSourceAware** interface.

Events

The **ApplicationEvent** class and **ApplicationListener** interface is used to handle the events. The following list covers the built-in events:

- **ContextRefreshedEvent**: This event is triggered when the **ApplicationContext** is initialized or refreshed.
- **ContextClosedEvent**: This event is triggered when **ApplicationContext** is terminated; it is called when the **close()** method is used on **ApplicationContext**.
- **RequestHandledEvent**: This event is triggered when a HTTP request has been serviced.

Custom events can be implemented using the **publishEvent()** method on **ApplicationContext**, by specifying a parameter as an instance of the custom event class implementing the **ApplicationEvent**. In the example, **ApplicationContext** is shown as follows:

```
<bean id="emailer" class="example.EmailBean">
  <property name="blackList">
    <list>
      <value>black@list.org</value>
      <value>white@list.org</value>
      <value>john@doe.org</value>
    </list>
  </property>
</bean>
<bean id="blackListListener" class="example.BlackListNotifier">
  <property name="notificationAddress" value="spam@list.org"/>
</bean>
```

The following code shows the class:

```
public class EmailBean implements ApplicationContextAware {
    private List blackList;
```

```
    private ApplicationContext ctx;
    public void setBlackList(List blackList) {
        this.blackList = blackList;
    }
    public void setApplicationContext(ApplicationContext ctx) {
        this.ctx = ctx;
    }
    public void sendEmail(String address, String text) {
        if (blackList.contains(address)) {
            BlackListEvent evt = new BlackListEvent(address, text);
            ctx.publishEvent(evt);
            return;
        }
        // send email...
    }
}
public class BlackListNotifier implements ApplicationListener {
    private String notificationAddress;
    public void setNotificationAddress(String notificationAddress) {
        this.notificationAddress = notificationAddress;
    }
    public void onApplicationEvent(ApplicationEvent evt) {
        if (evt instanceof BlackListEvent) {
            // notify appropriate person...
        }
    }
}
```

The preceding code can also be written using the AOP features; here, we have used the basic approach to handle the events.

Applying validation

The Spring MVC validation is used to restrict the input provided by the user. To validate the user's input, Spring 4 or a higher version supports and uses Bean Validation API. It can validate both server-side and client-side applications. The Bean Validation API is a Java specification used to apply constraints on object model

via annotations. Here, we can validate a length, number, regular expression, etc. We can also provide custom validations and use various validations, as follows:

- **`@NotNull`**: It determines that the value cannot be null.
- **`@Min`**: It determines that the number must be equal or greater than the specified value.
- **`@Max`**: It determines that the number must be equal or less than the specified value.
- **`@Size`**: It determines that the size must be equal to the specified value.
- **`@Pattern`**: It determines that the sequence follows the specified regular expression.

In this example, we create a simple form that contains the input fields. Here, (**`*`**) means it is mandatory to enter the corresponding field. Otherwise, the form generates an error.

1. Add dependencies to the **`pom.xml`** file.

`pom.xml`

```
<dependency>
    <groupId>org.springframework</groupId>
    <artifactId>spring-webmvc</artifactId>
    <version>5.1.1.RELEASE</version>
</dependency>
<dependency>
    <groupId>org.apache.tomcat</groupId>
    <artifactId>tomcat-jasper</artifactId>
    <version>9.0.12</version>
</dependency>
<dependency>
    <groupId>javax.servlet</groupId>
    <artifactId>servlet-api</artifactId>
    <version>3.0-alpha-1</version>
</dependency>
<dependency>
    <groupId>javax.servlet</groupId>
    <artifactId>jstl</artifactId>
```

```
    <version>1.2</version>
</dependency>
<dependency>
    <groupId>org.hibernate.validator</groupId>
    <artifactId>hibernate-validator</artifactId>
    <version>6.0.13.Final</version>
</dependency>
```

2. Create the bean class.

Employee.java

```
package com.validationexample;
import javax.validation.constraints.Size;

public class Employee {
    private String name;
    @Size(min=1,message="required")
    private String pass;

    public String getName() {
        return name;
    }
    public void setName(String name) {
        this.name = name;
    }
    public String getPass() {
        return pass;
    }
    public void setPass(String pass) {
        this.pass = pass;
    }
}
```

3. Create the controller class.

   In the controller class, the **`@Valid`** annotation applies validation rules on the provided object, and the **`BindingResult`** interface contains the result of validation.

```
package com.validationexample;


import javax.validation.Valid;
import org.springframework.stereotype.Controller;
import org.springframework.ui.Model;
import org.springframework.validation.BindingResult;
import org.springframework.web.bind.annotation.ModelAttribute;
import org.springframework.web.bind.annotation.RequestMapping;


@Controller
public class EmployeeController {


    @RequestMapping("/hello")
    public String display(Model m)
    {
        m.addAttribute("emp", new Employee());
        return "viewpage";
    }
    @RequestMapping("/helloagain")
   public String submitForm( @Valid @ModelAttribute("emp") Employee
e, BindingResult br)
    {
        if(br.hasErrors())
        {
            return "viewpage";
        }
        else
        {
        return "final";
```

```
        }
    }
}
```

4. Provide the entry of controller in the **web.xml** file.

web.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<web-app xmlns:xsi="http://www.w3.org/2001/XMLSchema-
instance" xmlns="http://java.sun.com/xml/ns/javaee"
xsi:schemaLocation="http://java.sun.com/xml/ns/javaee http://
java.sun.com/xml/ns/javaee/web-app_3_0.xsd" id="WebApp_ID"
version="3.0">
  <display-name>SpringMVC</display-name>
   <servlet>
    <servlet-name>spring</servlet-name>
                  <servlet-class>org.springframework.web.servlet.
DispatcherServlet</servlet-class>
    <load-on-startup>1</load-on-startup>
</servlet>
<servlet-mapping>
    <servlet-name>spring</servlet-name>
    <url-pattern>/</url-pattern>
</servlet-mapping>
</web-app>
```

5. Define the bean in the XML file.

spring-servlet.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<beans xmlns="http://www.springframework.org/schema/beans"
    xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
    xmlns:context="http://www.springframework.org/schema/context"
    xmlns:mvc="http://www.springframework.org/schema/mvc"
    xsi:schemaLocation="
        http://www.springframework.org/schema/beans
```

```
        http://www.springframework.org/schema/beans/spring-beans.
xsd
        http://www.springframework.org/schema/context
            http://www.springframework.org/schema/context/spring-
context.xsd
        http://www.springframework.org/schema/mvc
       http://www.springframework.org/schema/mvc/spring-mvc.xsd">
    <!-- Provide support for component scanning -->
     <context:component-scan base-package="com.validationexample"
/>
    <!--Provide support for conversion, formatting and validation
-->
    <mvc:annotation-driven/>
    <!-- Define Spring MVC view resolver -->
         <bean id="viewResolver" class="org.springframework.web.
servlet.view.InternalResourceViewResolver">
        <property name="prefix" value="/WEB-INF/jsp/"></property>
        <property name="suffix" value=".jsp"></property>
     </bean>
</beans>
```

6. Create the requested page.

index.jsp

```
<%@ taglib prefix="form" uri="http://www.springframework.org/tags/
form" %>
<html>
<body>
<a href="hello">Click here...</a>
</body>
</html>
```

7. Create the other view components.

viewpage.jsp

```
<%@ taglib prefix="form" uri="http://www.springframework.org/tags/
form" %>
```

```
<html>
<head>
<style>
.error{color:red}
</style>
</head>
<body>
<form:form action="helloagain" modelAttribute="emp">
Username: <form:input path="name"/> <br><br>
Password(*): <form:password path="pass"/>
<form:errors path="pass" cssClass="error"/><br><br>
<input type="submit" value="submit">
</form:form>
</body>
</html>
final.jsp
<html>
<body>
Username: ${emp.name} <br><br>
Password: ${emp.pass}
</body>
</html>
```

If we do not enter username or password when we execute this example, it will display that they are required fields; when we enter username and password, it will display the entered username and password.

The **`Validator`** interface is used to apply validation in the Spring framework. When the validation fails, the **`Validator`** reports to the **`Errors`** object.

```
public class Person {
  private String name;
  private int age;
  // code for getters and setters...
}
```

The following methods are provided by the interface:

- **`Supports (Class)`**: This method is used to check whether instances of the supplied class are valid.
- **`Validate (Object, org.springframework.validation.Errors)`**: This method checks for validation, and if error exists, it maps with the **`Errors`** object.

We can apply **`Validator`** by using the **`ValidationUtils`** helper class, as shown here:

```
public class PersonValidator implements Validator {


    public boolean supports(Class clazz) {
        return Person.class.equals(clazz);
    }
    public void validate(Object obj, Errors e) {
        ValidationUtils.rejectIfEmpty(e, "name", "name.empty");
        Person p = (Person) obj;
        if (p.getAge() < 0) {
            e.rejectValue("age", "negativevalue");
        } else if (p.getAge() > 110) {
            e.rejectValue("age", "too.darn.old");
        }
    }
}
```

We can write validation logic in a single class for multiple validations, or we can write logic in multiple classes:

```
public class CustomerValidator implements Validator {
   private final Validator addressValidator;
   public CustomerValidator(Validator addressValidator) {
      if (addressValidator == null) {
          throw new IllegalArgumentException("The supplied [Validator]
is required and must not be null.");
      }
      if (!addressValidator.supports(Address.class)) {
          throw new IllegalArgumentException(
            "The supplied [Validator] must support the validation of
```

```
[Address] instances.");
      }
      this.addressValidator = addressValidator;
   }

   public boolean supports(Class clazz) {
      return Customer.class.isAssignableFrom(clazz);
   }
   public void validate(Object target, Errors errors) {
      ValidationUtils.rejectIfEmptyOrWhitespace(errors, "firstName",
"field.required");
      ValidationUtils.rejectIfEmptyOrWhitespace(errors, "surname",
"field.required");
      Customer customer = (Customer) target;
      try {
          errors.pushNestedPath("address");
          ValidationUtils.invokeValidator(this.addressValidator,
customer.getAddress(), errors);
      } finally {
          errors.popNestedPath();
      }
   }
}
```

The errors generated are handled by the **`Errors`** object passed to the validator; then, we can check the error messages using the **`<spring:bind/>`** tag or manually.

Aspect-oriented programming

All the routine and cross-cutting tasks can be separated, and a clean program structure can be implemented using aspect-oriented programming. Let us understand the concept using the following application:

`Account.java`

```
public class Account{
    public long deposit(long depositAmount){
        newAmount = existingAccount + depositAccount;
        currentAmount = newAmount;
        return currentAmount;
```

```
    }
    public long withdraw(long withdrawalAmount){
        if (withdrawalAmount <= currentAmount){
            currentAmount = currentAmount – withdrawalAmount;
        }
        return currentAmount;
    }
}
```

In the example, we create an object of the **`Account`** class that is used to provide services of deposit and withdraw operations. We add security to the **`Account`** class to specify that only users with the **`BankAdmin`** privilege are permitted to perform tasks. The following code implements this functionality:

`Account.java`

```
public class Account{
    public long deposit(long depositAmount){
        User user = getContext().getUser();
        if (user.getRole().equals("BankAdmin"){
            newAmount = existingAccount + depositAccount;
            currentAmount = newAmount;
        }
        return currentAmount;
    }

    public long withdraw(long withdrawalAmount){
        User user = getContext().getUser();
        if (user.getRole().equals("BankAdmin"){
            if (withdrawalAmount <= currentAmount){
                currentAmount = currentAmount – withdrawalAmount;
            }
        }
        return currentAmount;
    }
}
```

We modify the code to show the use of Logging and Transaction Management. The modified code is as follows:

Account.java

```
public class Account{
    public long deposit(long depositAmount){
        logger.info("Start of deposit method");
        Transaction trasaction  = getContext().getTransaction();
        transaction.begin();
        try{
            User user = getContext().getUser();
            if (user.getRole().equals("BankAdmin"){
                newAmount = existingAccount + depositAccount;
                currentAmount = newAmount;
            }
            transaction.commit();
        }catch(Exception exception){
            transaction.rollback();
        }
        logger.info("End of deposit method");
        return currentAmount;
    }

    public long withdraw(long withdrawalAmount){
        logger.info("Start of withdraw method");
        Transaction trasaction = getContext().getTransaction();
        transaction.begin();
        try{
            User user = getContext().getUser();
            if (user.getRole().equals("BankAdmin"){
                if (withdrawalAmount <= currentAmount){
                    currentAmount = currentAmount – withdrawalAmount;
                }
            }
```

```
            transaction.commit();
        }catch(Exception exception){
            transaction.rollback();
        }
        logger.info("End of withdraw method");
        return currentAmount;
    }
}
```

In the example, we are trying to show that with each new functionality that we want to add, we need to change lots of logic in method, and by changing the code, we need to follow the software development life cycle, which is not practically possible for big and complex projects. This is a major drawback shown with the preceding example.

The Solution through AOP

The purpose of the **`Account`** class is to provide deposit and withdraw services, but it also provides the facility of logging, user checking and transaction management. The following code shows these methods:

```
public void deposit(){
    // Transaction Management
    // Logging
    // Checking for the Privileged User
    // Actual Deposit Logic comes here
}

public void withdraw(){
    // Transaction Management
    // Logging
    // Checking for the Privileged User
    // Actual Withdraw Logic comes here
}
```

The code apart from the core functionality is called routine task that are known as cross cutting tasks. The main purpose of AOP is to isolate the cross-cutting concerns from the application code and use them as add-on functionality. This approach results in a cleaner code with less redundancy.

AOP terminologies

The following section covers the most common terminologies used in the AOP.

Aspects

The cross-cutting tasks are known as an aspect; for example, in the following code, logging and transaction management are the aspects:

```
public void businessOperation(BusinessData data){
    // Logging
    logger.info("Business Method Called");
    // Transaction Management Begin
    transaction.begin();
    // Do the original business operation here
    transaction.end();
}
```

JoinPoint

Join Points defines the various execution points where an aspect can be applied, as shown here:

```
public void someBusinessOperation(BusinessData data){
    //Method Start -> Possible aspect code here like logging.
    try{
        // Original Business Logic here.
    }catch(Exception exception){
        // Exception -> Aspect code here when some exception is raised.
    }finally{
        // Finally -> Even possible to have aspect code at this point too.
    }
    // Method End -> Aspect code here in the end of a method.
}
```

We can define multiple execution points, and an aspect need not be applied to all the Join Points.

Pointcut

Join Points are the logical points wherein a particular aspect or a set of aspects can be applied. A pointcut or a pointcut definition will tell exactly which join points the aspects will be applied. Let us understand the following:

```
aspect LoggingAspect {}
aspect TransactionManagementAspect {}
The preceding code is declaring aspect; now, let's consider the
following piece of code:
public void someMethod(){
    //Method Start
    try{
        // Some Business Logic Code.
    }catch(Exception exception){
        // Exception handler Code
    }finally{
        // Finally Handler Code for cleaning resources.
    }
    // Method End
}
```

The following code demonstrates the point cut definition:

```
pointcut method_start_end_pointcut(){
    // This point cut applies, before the
    // beginning and the end of the method.
}

pointcut catch_and_finally_pointcut(){
    // This point cut applies the aspects,  in the catch
    // block whenever an exception raises and the finally block.
}
```

We can define a point cut to map aspect to a single or multiple joint point.

Advice

Implementation of the Aspect is called an Advice. It defines the functionality in a more abstract manner and provides a concrete code implementation for the Aspect. Field aspect is supported by other aspect-oriented programming, but Spring provides support only for the method aspect. Before, after, throws and around advice are various aspects supported in Spring.

Conclusion

Understanding the purpose and types of software framework is very important for developing complex and large applications. This chapter provided the basic features used in the Spring framework and also familiarised you with its directory structure. Further on, the chapter explored the concepts of container, dependencies, the `ApplicationContext`, validations, events, and the aspect-oriented programming.

In the next chapter, we will learn the spring boot basics, understand its core architecture, learn how to create projects, and look at the basic annotations used in spring boot.

Points to remember

- The Spring Framework has become the most popular framework for developing Java applications as it offers speed, simplicity and a safe environment.
- Spring offers salient features with the support of plain Java objects to work with the legacy concepts.

Multiple choice questions

1. **What advanced configuration features the ApplicationContext provides ?**
    a. Message resource handing
    b. Event propagation
    c. Application layer specific context
    d. All of these

2. **The _____ injects dependencies while creating a bean.**
    a. Container
    b. Object
    c. Component
    d. Class

3. **In the _____ mode, dependency checking is performed for collaborators only.**
    a. simple
    b. object

c. none

d. both simple and object

4. **The _____ annotation is used to invoke the handler method to process the request.**

   a. @Before

   b. @After

   c. @Controller

   d. @Exception

5. **When we use @RequestMapping, all the requests are POST request.**

   a. True

   b. False

   c. Not determined

   d. None of these

Answers

1. **d**
2. **a**
3. **b**
4. **c**
5. **b**

Questions

1. What are the advantages and disadvantages of using a software framework?
2. Explain the different bean scopes.
3. Explain internationalization using **`MessageSource`** with examples.
4. What do you mean by IOC container?
5. What is autowiring?
6. Explain the events supported in the framework.
7. What is the purpose of aspect-oriented programming?

Key terms

- **Spring framework**: It is an open-source framework that is a lightweight dependency injection and an aspect-oriented container.
- **Inversion of Control (IoC)**: It separates the components into a fully working application ready for use.

Join our book's Discord space

Join the book's Discord Workspace for Latest updates, Offers, Tech happenings around the world, New Release and Sessions with the Authors:

https://discord.bpbonline.com

Chapter 14
Introduction to Spring Boot

Introduction

Spring boot is a framework used to bootstrap and develop spring applications very easily and quickly. It provides annotation-based configuration default configuration to avoid lots of boilerplate code and configuration. It also provides the opinionated development approach to quick start the applications. Spring boot takes less development and testing time compared to Spring framework, so it is mostly used when rapid application development is required.

Structure

In this chapter, we will learn the following topics:

- Basics of Spring boot
- Creating a project
- Annotations
- Dependency management
- Database connectivity

Objectives

After studying this chapter, you should be able to understand the Spring boot basics and its core architecture. You should be familiar with how to create projects and know basic annotations used in spring boot. Additionally, you should have gained knowledge of spring boot properties and dependencies, and know how to handle the database connectivity.

Basics of Spring boot

Spring is one of the most commonly used frameworks for developing Java-based applications as it provides the facility of dependency injection and inversion of control that supports the development of loosely coupled applications. Spring has many modules, and one of them is Spring boot, which allows us to build stand-alone applications with almost no configurations. It is mainly used to develop simple restful services. Spring boot is a way to provide rapid application development using the Spring framework. It is a module built as a part of the Spring framework and can be used to develop stand-alone as well as web applications. Spring boot provides support for dependency injection; it has a very good database transaction management capabilities and provides easy integration with other Java framework like JPA, Hibernate, JPA, ORM, Struts, and so on. Using spring framework can help reduce development cost and time of the application. Spring boot provides opinionated development approach, and it reduces lots of import statements, XML and extra annotation-based configuration. Following is the list of main features provided by Spring boot:

- Web application can be developed quickly and easily using this module. The application can use in-built servers like **Tomcat, Jetty,** or Undertow. The starter web module of spring boot can be used to develop and deploy the application quickly.
- The Spring Application class is used to bootstrap a Spring application by calling the main method and static **`run()`** method.
- Spring boot uses event handling by providing various listeners to handle the events.
- We can access and manage spring boot applications remotely by enabling the administrator-related features.
- Spring boot allows us to provide various configurations that allow the same application to work with a heterogenous platform, by providing configuration in YAML file.

- Based on the requirements of the application, we can specify the properties with the use of a YAML or properties file.
- Configuration of the application is provided in a type-safe way.
- Spring boot logging is managed by default.
- Spring boot applications are secure because authentication on various endpoints is provided by default.

Spring boot advantages and disadvantages

Spring boot mainly aims to increase productivity by reducing the development and testing time and increasing productivity.

Advantages

- It is easy to integrate the spring boot application with spring jdbc, orm, data. security, etc.
- It provides embedded HTTP servers to develop and test the applications.
- It provides CLI (Command Line Interface) tool to develop and test Spring boot (Java or Groovy) Applications from command prompt very easily and quickly.
- It provides multiple plugins to develop and test Spring boot applications very easily using build tools like Maven and Gradle.
- It provides many plugins to work with embedded and in-memory databases easily.

Disadvantage

- It is a time-consuming process to convert existing or legacy Spring Framework projects into Spring boot applications.

Spring boot architecture

There are different layers in the Spring boot architecture, and each layer communicates with the other layers. There are **four** layers in Spring boot, as follows:

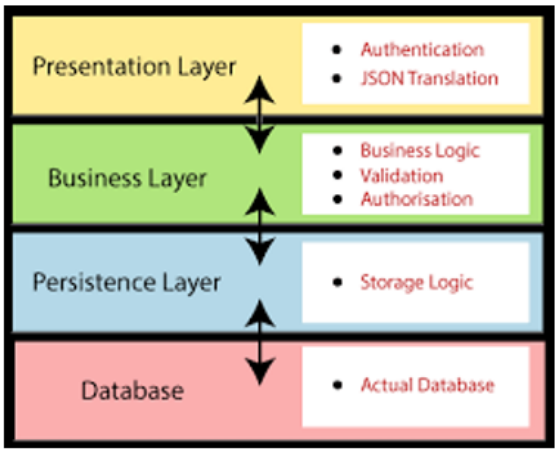


Figure 14.1: *Spring boot architecture*

- **Presentation layer:** The presentation layer is the view layer that manages and authenticates the requests, translates the JSON parameter to an object, and transfers it to the business layer.
- **Business layer:** The business layer is concerned with business logic authorization and validation. It consists of classes and services provided by data access layer.
- **Persistence layer:** The persistence layer consists of data storage part, and it manages and translates objects to and from the database.
- **Database layer:** In the database layer, the basic **CRUD** (create, retrieve, update, delete) operations are performed.

Creating projects

There are different ways in which Spring boot applications can be created; the following section covers the most common approaches used to create Spring projects:

- Spring Initializer is used to generate the directory structure of Spring boot application; it is a handy tool provided by Pivotal web service. It provides API for developing projects with metadata.
- Spring Tool Suite is an IDE to develop Spring applications. It is an Eclipse-based development environment and provides a ready-to-use environment

to implement, run, deploy, and debug the application. It validates our application and provides quick fixes for the applications.

- Spring boot CLI can be used to develop spring boot applications.

Annotations

Annotations provide extra information about the program; annotation is a form of metadata that can be provided at class and method level. It is not part of program logic, but it provides important information regarding configuration, data management, and so on from the program. Following is the list of most common annotations used in the spring boot:

1.	`@Required`**:** It is applied on the method specially on the setter, method, indicating that the bean value must be retrieved at the time of configuration, otherwise it throws an exception.
2.	`@Autowired`**:** This annotation can be applied to the setter method, instance variable and constructor. It looks for a similar datatype and matches the field.
3.	`@Configuration`**:** It is applied to the class that is used as a source of a bean definition.
4.	`@ComponentScan`**:** It is used to scan a package for beans. We can specify the current and base package to scan for Spring Components.
5.	`@Bean`**:** It is used at the method level and is used as a substitute for XML bean tag to create a bean.
6.	`@Component`**:** It is a class-level annotation. When this annotation is found in the classpath, the framework finds it and configure it in the application context as a **Spring Bean**.
7.	`@Controller`**:** It is a class-level annotation. It is used to handle web requests and map them using the control string, which suggests what route to redirect.
8.	`@Service`**:** It is used to indicate the class that applies the business logic.
9.	`@Repository`**:** It is applied to class to hold the Data Access Object to perform all the operations related to the database.
10	`@EnableAutoConfiguration`**:** The bean that is present in the classpath is configured automatically and it also configure to run the methods.

11.	`@SpringBootApplication`: Instead of using different annotations for component scan, configuration and allowing autoconfiguration, we can use this annotation to specify the run method.
12.	`@RequestMapping`: It is applied for mapping of the requests for a class and method. The annotation offers various elements that one can use or omit as per requirement; the most commonly used elements include header, name, method, params, path, and value.
13.	`@GetMapping`: It is used to map the HTTP `GET` requests on the specific method as web service endpoints. It can be used as a short way to indicate the `Get` request instead of using `RequestMapping` and indicating the `Get` method request.
14.	`@PostMapping`: It is used to map the HTTP `POST` requests on the specific method as web service endpoints. It can be used as a short way to indicate the `Post` request instead of using `RequestMapping` and indicating the `Post` method request.
15.	`@PutMapping`: It is used to map the HTTP `PUT` requests on the specific method as web service endpoints. It can be used as a short way to indicate the Put request instead of using `RequestMapping` and indicating the `Put` method request.
16.	`@DeleteMapping`: It is used to map the HTTP `DELETE` requests on the specific method as web service endpoints to remove or delete a resource. It is an alternative of using `RequestMapping` and indicating the `Delete` method request.
17.	`@PatchMapping`: It is used to map the HTTP `PATCH` requests on the specific method. It can be used as a short way to indicate the `Patch` request instead of using `RequestMapping` and indicating the `Patch` method request.
18.	`@RequestBody`: It is used map the request body with an object in a method parameter.
19.	`@ResponseBody`: It is used to bind the value that is returned by a method to the response body. It translates an object into JSON and XML format.
20.	`@PathVariable`: It is used to retrieve the values from the URI generally used with web services. Multiple path variables can be used in a method.
21.	`@RequestParam`: It is used to retrieve the query parameters from the URL; we can specify default values in case the query parameter is missing.

22.	`@RequestHeader`**:** We can get the values for HTTP request headers using this annotation. Multiple elements like name, required, value, and `defaultValue` can be used for a different purposes; we should specify separate annotations for each detail in the header. It can be used multiple times in a method.
23.	`@RestController`**:** It is a combination of the `@Controller` and `@ResponseBody` annotations. We can use this annotation instead of using `@ResponseBody` for each method.
24.	`@RequestAttribute`**:** It is used to map a method parameter to a request attribute. From the controller, we can retrieve the request attributes using this annotation.

Table 14.1: *Annotations used in Spring Boot*

Dependency management

Dependencies and configuration are managed automatically in Spring boot. With each new version, the list of dependencies supported are updated. Spring boot automatically takes the version of dependencies we need not provide most of the time. When Spring boot version is updated, all the dependencies are updated automatically. All the configuration of starter parent is adopted by the Maven project, which generally includes Java compiler version, dependency section, spring boot dependencies POM, plugin configuration, resource filtering, and UTF source encoding.

Inheriting starter parent

The following code shows how the parent inherits by default when we configure the project:

```
<parent>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-parent</artifactId>
<version>2.2.2.BUILD-SNAPSHOT</version>
<relativePath/>
</parent>
```

We can use the **`<properties>`** tag to override already injected dependency with the new values for the same dependency; for example, to manage Java version, we can use the **`<java.version>`** tag.

```
<properties>
<java.version>1.8</java.version>
</properties>
```

To manage the Maven plugin in the **pom.xml** file, it creates an executable **jar** file for the project.

```
<build>
<plugins>
<plugin>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-maven-plugin</artifactId>
</plugin>
</plugins>
</build>
```

To implement dependency management without using starter parent, we can use the scope tag that does not override, as follows:

```
<dependencyManagement>
<dependencies>
<dependency>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-dependencies</artifactId>
<version>2.2.2.RELEASE</version>
<type>pom</type>
<scope>import</scope>
</dependency>
</dependencies>
</dependencyManagement>
```

To upgrade **spring-data-releasetrain**, add the following dependency in the **pom.xml** file:

```
<dependencyManagement>
<dependencies>
<dependency>
<groupId>org.springframework.data</groupId>
```

```
<artifactId>spring-data-releasetrain</artifactId>
<version>Fowler-SR2</version>
<type>pom</type>
<scope>import</scope>
</dependency>
<dependency>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-dependencies</artifactId>
<version>2.2.2.RELEASE</version>
<type>pom</type>
<scope>import</scope>
</dependency>
</dependencies>
</dependencyManagement>
```

Spring boot application properties

All application-specific configuration can be provided in the **`application.properties`** file. Many properties have default value, many of them have Boolean values and many of them have specific values. We can define our own properties as per our requirement. By specifying different properties, we can run the same application in different environments. So, we use this file to configure the application by in-built properties; we can also customize it by defining our own properties. The following code shows the example of the **`application.properties`** file:

```
#Set application name
spring.application.name = firstApplication
#set port number
server.port = 8085
```

Here, we are setting the application name and port number for the application; we use the # symbol for adding comments in the file.

YAML properties file

Another way to configure the application properties is through the YML file. We can configure all the properties in the **`application.yml`** file and place it in the classpath.

Example of **`application.yml`**:

```
spring:
application:
name: firstApplication
server:
port: 8085
```

In the preceding example, we have configured the application name and port. Port 8085 denotes that the application runs on port **`8085`**.

Spring boot property groups

Based on some common characteristics, spring boot properties can be classified into various groups:

- Core properties
- Cache properties
- Mail properties
- JSON properties
- Data properties
- Transaction properties
- Data migration properties
- Integration properties
- Web properties
- Templating properties
- Server properties
- Security properties
- RSocket properties
- Actuator properties
- DevTools properties
- Testing properties

Common application properties

The following is a list of the most commonly used properties of spring boot:

1.	`Debug`: It is used to make an entry of debug logs. Its default value is false.
2.	`spring.application.name`: It sets the name of the application.
3.	`spring.application.admin.enabled`: It enables the admin features of the application. Its default value is false.
4.	`spring.config.name`: It sets the config file name.
5.	`spring.config.location`: It sets the file name.
6.	`server.port`: It sets the HTTP server port. Default port is `8080`.
7.	`server.servlet.context-path`: It sets the context path of the application.
8.	`logging.file.path`: It sets the location of the log file.
9.	`spring.banner.charset`: It sets the banner file encoding. Default character set is UTF-8.
10.	`spring.banner.location`: It sets the banner file location.
11.	`logging.file`: It sets the log file name, for example, data.log.
12.	`spring.application.index`: It sets the application index.
13.	`spring.mail.default-encoding`: It sets the default `MimeMessage` encoding.
14.	`spring.mail.host`: It sets the SMTP server host, for example, `smtp.example.com`.
15.	`spring.mail.password`: It is used to set the login password of the SMTP server.
16.	`spring.mail.port`: It sets the SMTP server port.
17.	`spring.mail.test-connection`: It tests that the mail server is available on startup. Its default value is false.
18.	`spring.mail.username`: It sets the login user of the SMTP server.
19.	`spring.main.sources`: It sets the sources for the application.
20.	`server.address`: It sets the network address to which the server should bind.

21.	`server.connection-timeout`: It sets the time, in milliseconds, that connectors will wait for another HTTP request before closing the connection.
22.	`server.context-path`: It sets the context path of the application.
23.	`server.server-header`: It sets the server response header.
24.	`server.servlet-path`: It sets the path of the main dispatcher servlet.
25.	`server.ssl.enabled`: It enables SSL support.
26	`spring.http.multipart.enabled`: It enables support of multi-part uploads. Its default value is true.
27.	`spring.servlet.multipart.max-file-size`: It sets max file size. Its default value is 1 MB.
28.	`spring.mvc.async.request-timeout`: It sets the time, in milliseconds.
29.	`spring.mvc.date-format`: It sets the date format, for example, dd/MM/yyyy.
30.	`spring.mvc.locale`: It sets the locale for the application.
31.	`spring.social.facebook.app-id`: It sets the application's Facebook App ID.
32.	`spring.social.linkedin.app-id`: It sets the application's LinkedIn App ID.
33.	`spring.social.twitter.app-id`: It sets the application's Twitter App ID.
34.	`security.basic.authorize-mode`: It sets the security authorize mode.
35.	`security.basic.enabled`: It enables basic authentication. Its default value is true.
36.	`spring.freemaker.content-type`: It provides the content type value.
37.	`server.server-header`: It sets the value to use for the server response header.
38.	`spring.security.filter.dispatcher-type`: It sets the security filter chain dispatcher types.
39.	`spring.security.filter.order`: It sets the security filter chain order.
40.	`spring.security.oauth2.client.registration.*`: It sets the OAuth client registrations.
41.	`spring.security.oauth2.client.provider.*`: It sets the OAuth provider details.

Table 14.2: *Application properties used in Spring Boot*

Spring boot starters

Spring Boot comes with a number of starters that are used to add JAR files in the classpath. The built-in starters make the development faster and easier. The Spring boot starter always begin with the keyword **`Spring-boot-starter`**, followed by the type of starter. For example, if we want to use Spring and JPA for database access, we can use the **`spring-boot-starter-data-jpa`** dependency in our **`pom.xml`** file of the project.

External starters

We can use external starters in our project. These external starters start with the name of the project. For example, if the third-party project name is HelloWorld**,** then the dependency name will be HelloWorld-spring-boot-starter**.**

The **`org.springframework.boot`** group provides the starters; following is the list of the most common starters:

- **`spring-boot-starter-thymeleaf`**: It gives a view layer using the **`thymeleaf`** views.
- **`spring-boot-starter-data-couchbase`**: It allows us to use the **`couchbase`** database.
- **`spring-boot-starter-artemis`**: It is used for JMS messaging.
- **`spring-boot-starter-web-services`**: It is used for web services.
- **`spring-boot-starter-mail`**: It is used to support email sending.
- **`spring-boot-starter-data-redis`**: It is used to set Redis key-value data store.
- **`spring-boot-starter-web`**: It is used for creating web and RESTful applications using spring MVC. Generally, it uses Tomcat as the default embedded container.
- **`spring-boot-starter-data-gemfire`**: It is used to GemFire distributed data store.
- **`spring-boot-starter-activemq`**: It is used in JMS messaging using Apache ActiveMQ.
- **`spring-boot-starter-data-elasticsearch`**: It is used in search and analytics engine with Spring Data Elasticsearch.
- **`spring-boot-starter-integration`**: It is used for Spring integration.

- **spring-boot-starter-test**: It is used to test Spring Boot applications with testing libraries, including JUnit and Hamcrest.
- **spring-boot-starter-jdbc**: It is used for JDBC with the Tomcat JDBC connection pool.
- **spring-boot-starter-mobile**: It is used for building web applications using Spring mobile.
- **spring-boot-starter-validation**: It is used for Java bean validation with hibernate validator.
- **spring-boot-starter-hateoas**: It is used to build a hypermedia-based RESTful web application with Spring MVC and Spring HATEOAS.
- **spring-boot-starter-jersey**: It is used to build RESTful web applications using JAX-RS and Jersey.
- **spring-boot-starter-data-neo4j**: It is used for the Neo4j graph database.
- **spring-boot-starter-data-ldap**: It is used for Spring Data LDAP.
- **spring-boot-starter-websocket**: It is used for building the WebSocket applications.
- **spring-boot-starter-aop**: It is used for aspect-oriented programming with Spring AOP and AspectJ.
- **spring-boot-starter-amqp**: It is used for Spring AMQP and Rabbit MQ.
- **spring-boot-starter-data-cassandra**: It is used for Cassandra distributed database.
- **spring-boot-starter-social-facebook**: It is used for Spring Social Facebook.
- **spring-boot-starter-jta-atomikos**: It is used for JTA transactions.
- **spring-boot-starter-security**: It is used for Spring Security.
- **spring-boot-starter-mustache**: It is used for building MVC web applications.
- **spring-boot-starter-data-jpa**: It is used for Spring Data JPA with Hibernate.
- **spring-boot-starter**: It is used for core starter, including auto-configuration support, logging, and YAML.
- **spring-boot-starter-groovy-templates**: It is used for building MVC web applications using Groovy Template views.

- **`spring-boot-starter-freemarker`**: It is used for building MVC web applications using FreeMarker views.
- **`spring-boot-starter-batch`**: It is used for Spring Batch.
- **`spring-boot-starter-social-linkedin`**: It is used for Spring Social LinkedIn.
- **`spring-boot-starter-cache`**: It is used for Spring Framework's caching support.
- **`spring-boot-starter-data-solr`**: It is used for the Apache Solr search platform.
- **`spring-boot-starter-data-mongodb`**: It is used for MongoDB database.
- **`spring-boot-starter-jooq`**: It is used for jOOQ to access SQL databases.
- **`spring-boot-starter-jta-narayana`**: It is used for Spring Boot Narayana JTA Starter.
- **`spring-boot-starter-cloud-connectors`**: It is used for Spring Cloud Connectors and simplifies connecting to services in cloud platforms like Cloud Foundry and Heroku.
- **`spring-boot-starter-jta-bitronix`**: It is used for JTA transactions using Bitronix.
- **`spring-boot-starter-social-twitter`**: It is used for Spring Social Twitter.
- **`spring-boot-starter-data-rest`**: It is used for exposing Spring Data.

Spring boot production starters

Following are the spring boot production starters:

- **`spring-boot-starter-actuator`**: It is used to monitor and manage your application.
- **`spring-boot-starter-remote-shell`**: It is used for the remote shell to monitor and manage the application over SSH. It is deprecated property.

Spring boot technical starters

Following are the spring boot technical starters:

- **`spring-boot-starter-undertow`**: It is used to set undertow as embedded servlet container.

- **spring-boot-starter-jetty**: It is used to set jetty as embedded servlet container.
- **spring-boot-starter-logging**: It is used for logging using Logback.
- **spring-boot-starter-tomcat**: It is used to set **tomcat** as embedded servlet container. It is the default servlet container.
- **spring-boot-starter-log4j2**: It is used for Log4j2 for recording logs based on the user activities and execution.

Spring boot starter web

The starter web can be configured for easy web development. We need to add the following dependency to the **pom.xml** file:

```
<dependency>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-web</artifactId>
<version>2.2.2.RELEASE</version>
</dependency>
```

As default values, the spring web uses spring MVC and Rest, and it uses tomcat as embedded server. Using starter web dependency, add all the dependencies for the web development and reduce the build time. The starter web depends on the following:

- **org.springframework.boot:spring-boot-starter**
- **org.springframework.boot:spring-boot-starter-tomcat**
- **org.springframework.boot:spring-boot-starter-validation**
- **com.fasterxml.jackson.core:jackson-databind**
- **org.springframework:spring-web**
- **org.springframework:spring-webmvc**

By default, the following tomcat server dependency is set up with the spring boot starter:

```
<dependency>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-tomcat</artifactId>
<version>2.0.0.RELEASE</version>
```

```
<scope>compile</scope>
</dependency>
```

Dispatcher servlet, default error page, and **`jar`** files for managing the dependencies and the embedded servlet container are automatically configured for the application.

Spring boot embedded web server

An application has an embedded server by default. The benefit of an embedded server is that we do not need to install the server before deploying the application. Tomcat is the default embedded server with spring boot. Spring Boot also supports two other embedded servers: Jetty Server and Undertow Server.

Using another embedded web server

For basic servlet applications, the spring boot starter uses tomcat as a default server, but we can also use jetty or undertow server. For reactive applications, the starter-webflux includes Reactor Netty by including starter-reactor-netty, but we can also use starter-tomcat, starter-jetty, or starter-undertow instead.

Jetty server

Jetty is another embedded server supported by Spring Boot. It is an HTTP server and Servlet container that can be used to generate static and dynamic content. Generally, it is used when more machine communication is required. Add spring boot starter jetty dependency in the **`pom.xml`** file to use the jetty server. We need to remove the default or any other servlet container before adding the jetty server so that we do not face server conflicts.

```
<dependency>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-web</artifactId>
<exclusions>
<exclusion>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-tomcat</artifactId>
</exclusion>
</exclusions>
</dependency>
<dependency>
```

```
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-jetty</artifactId>
</dependency>
```

We need to remove the default or any other servlet container before adding the undertow server so that we do not face server conflicts.

Undertow server

Another embedded server provided by spring boot is undertow server, which is managed by JBoss. It has Http and web socket support and provides servlets 4.x support. Before adding Undertow server in the application, remove any other added server to avoid conflict between the servers:

```
<dependency>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-web</artifactId>
<exclusions>
<exclusion>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-tomcat</artifactId>
</exclusion>
</exclusions>
</dependency>
<dependency>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-undertow</artifactId>
</dependency>
```

We can customize the behavior of the Undertow server by setting different properties in the properties file.

spring-boot-starter-web vs. spring-boot-starter-tomcat

`starter-web` contains many dependencies, including the dependency for tomcat. Other dependencies include the following:

spring-boot-starter

jackson

spring-core

spring-mvc

spring-boot-starter-tomcat

spring boot starter for **tomcat** contains core, el, logging and websocket.

The starter-tomcat has the following dependencies:

```
<dependency>
<groupId>org.apache.tomcat.embed</groupId>
<artifactId>tomcat-embed-core</artifactId>
<version>8.5.23</version>
 <scope>compile</scope>
</dependency>
<dependency>
<groupId>org.apache.tomcat.embed</groupId>
<artifactId>tomcat-embed-el</artifactId>
<version>8.5.23</version>
<scope>compile</scope>
</dependency>
<dependency>
<groupId>org.apache.tomcat.embed</groupId>
<artifactId>tomcat-embed-websocket</artifactId>
<version>8.5.23</version>
<scope>compile</scope>
</dependency>
```

Spring MVC can be used without the embedded server. To remove any embedded server, we can use the exclusion tag; in the following example, we use the exclusion tag to remove the tomcat server:

```
<dependency>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-web</artifactId>
<exclusions>
```

```
<exclusion>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-tomcat</artifactId>
</exclusion>
</exclusions>
</dependency>
```

Database connectivity

Spring data provides support for relational database and NoSQL database. While creating a new application, the main concern should be the business logic, not the routine and repetitive tasks. The Java Persistent API, known as JPA, and the Spring Data JPA are commonly used to simplify the database operations.

Spring Data JPA is a layer built on the top of JPA, so it uses all the features defined in the JPA, like managing entities, association mappings, and writing queries. Apart from using JPA features, it provides new features like implementing repository pattern with no or less code, forming database queries using method name, and so on.

Spring data repository

Spring JPA provides Crud, Paging and sorting, and JPA repository:

- **`CrudRepository`:** This repository offers elementary operations to create, read, update, and delete records. The most common methods it provides include **`findOne()`**, **`findAll()`**, **`save()`**, and **`delete()`**.
- **`PagingAndSortingRepository`:** It extends **`CrudRepository`** and provides the **`findAll`** methods to retrieve or sort data in a paginated order.
- **`JpaRepository`:** It extends **`CrudRepository`** and **`PagingAndSorting Repository`**. It offers new methods like **`flush()`** to clear the context:

  ```
  <dependency>
  <groupId>org.springframework.data</groupId>
  <artifactId>spring-data-jpa</artifactId>
  <version>2.2.3.RELEASE</version>
  </dependency>
  ```

Spring boot starter data JPA

With this dependency, we can establish a connection with application and database. The starter-data-jpa internally uses the JPA dependency.

```
<dependency>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-data-jpa</artifactId>
<version>2.2.2.RELEASE</version>
</dependency>
```

The database is designed with table or views. Using JDBC, we need to connect the database and write lots of queries. Using JPA, we can retrieve data from the objects and transfer it back when we read or write to and from tables. Before JPA, Object Relational Mapping was commonly used, so Hibernate is called the ORM framework.

JPA uses entity manager to handle all the communication with the database. It uses JPQL, which stands for Java persistence query language, to write queries; and it uses criteria API to execute queries against the database.

Hibernate vs. JPA

Hibernate is the most popular framework that supports ORM, and JPA is an API. Hibernate is a implementation, while JPA is a specification. Hibernate provides the mapping between objects and tables; it maps and checks whether the data is read or written into the database.

Conclusion

This chapter covered the use of Spring boot in reducing the development cost and time of the application. It also covered the development of projects in various ways and the application of annotations used to configure the applications. You understood dependency management for using ready-made things and learned how to use the most powerful concept of hibernate and JPA in handling database operations from.

Points to remember

- Spring Initializer is one of the approaches to create spring boot applications.
- Spring boot starters is a way to provide dependencies in the **`pom.xml`** file.

- All the application-specific properties can be set in the **`application.properties`** file.
- Gradle or Maven dependency management can be managed easily by adding them to classpath.
- We can monitor and manage an application before putting it for the production using Spring boot Actuator.

Multiple choice questions

1. **We can create Beans using the _____ property.**
    a. Scope
    b. Property
    c. Class
    d. Its own constructor

2. **To instantiate a method, we can use the _____.**
    a. default-init method
    b. static factory method
    c. destroy method
    d. lazy-init method

3. **What are spring beans?**
    a. Controller classes
    b. Service classes
    c. Repository
    d. Any class that is managed by the container

Answers

1. **d**
2. **b**
3. **d**

Questions

1. What is dependency injection?
2. What is singleton scope?

3. What is Bean in Spring?
4. Which method is used to process bean before initialization callback?
5. How does spring achieve DI or IOC?
6. What exceptions do the DAO classes used in Spring throw?
7. How can you inject Java Collection in Spring?
8. How many types of lOC are there?

Key terms

- **Properties files**: Properties files are text files that hold key-value pairs and are used to provide configuration of the application by setting or changing the default properties value with the application-specific or requirements related values.
- **Spring Initializer**: It is used to create Spring boot application easily by providing the directory structure for the project.
- **Spring Tool Suite**: It is an IDE that provides the environment to implement, run, deploy, and debug the spring boot application.

Join our book's Discord space

Join the book's Discord Workspace for Latest updates, Offers, Tech happenings around the world, New Release and Sessions with the Authors:

https://discord.bpbonline.com

Index

A

annotations, Spring boot 461-463
AOP terminologies
  Advice 453
  aspects 452
  JointPoint 452
  pointcut 452, 453
application, building with IDE 22
  records, deleting in table
    from Java 36, 37
  records, inserting from
    Java 32-34
  records, updating in table
    from Java 34, 35
  table, altering in Oracle
    from Java 32
  table, creating in Oracle
    from Java 23-31
ApplicationContext 437
  events 439, 440
  internationalization, with
    MessageSources 437, 438
application frameworks 420
  advantages 420
  drawbacks 421
application, JSF 373
  debugging 376
  development 375
  environment setup 374
  pages prototype and design 374, 375
  performance tuning 376
  project preparation 374
  structure 367
aspect 452
aspect-oriented programming
    (AOP) 448-451
  solution 451

ASP.net client
creating 410, 411
web reference, providing 411, 412
web service, calling 413

B

batch processing, in JDBC 51
advantages 52
application of methods 52-55
bean definition inheritance 436
bean properties
retrieving 242-248
bean scopes 432
custom scopes 434
global session scope 434
prototype scope 433
request scope 433, 434
session scope 434
singleton scope 432, 433

C

client-side form data
handling 89
collections
retrieving 236-239
Common Gateway Interface (CGI) 83
disadvantages 83
Constructors 44
Content-Length.getContentType method 93
cookies
attributes 110, 111
finding, with specified names 113
for handling user accounts and preferences 115, 116
long-lived cookies, creating 114
managing 108
maximum age, setting 109
placing, in response header 109
reading, from client 110
receiving 108
sending 108
sending, to client 109
TestCookie servlet, using 111, 112
cookie utilities
AccessCounts.java 116, 117
LongLivedCookie.java 114
RegistrationForm.java 117
RegistrationServlet.java 118
RepeatVisitor.java 115
Utilities.java 113
core tag library, JSTL 299
c:catch tag 318
c:choose tag 304
c:forEach tag 306-308
c:forTokens tag 307-309
c:if tag 302-304
c:import tag 309-312
c:otherwise tag 304-306
c:out tag 300, 301
c:param tag 315, 316
c:redirect tag 316-318
c:remove tag 301, 302
c:set tag 301
c:url tag 312-314
c:when tag 304
CRUD operations 460
custom scopes 434
using 435
Custom Tag 135

D

DatabaseMetaData 22
DatabaseProtocol driver 7

data sharing
  in application scope 270
  in request scope 269
  in session scope 269
dependencies 426
  ambiguous dependencies 427-429
  checking for 431
  collections 429-431
  injecting, with setter methods and parameterized constructor 427
  method injection 431
dependency management, Spring boot 463
  application properties 467, 468
  Spring boot application properties 465
  Spring boot property groups 466
  Spring boot starters 469
  starter parent, inheriting 463, 464
  YAML properties file 465, 466
deployment descriptor 77
design pattern 262
  uses 262

E

embedded web server 473
  Jetty server 473
  undertow server 474
Escaped Template Text 136
events, JSF 372
  component-managed events 372, 373
expression language 232, 233

F

file attributes 176
flush attribute 168
form data reading methods 89
  demonstration 91-93
  getInputStream 90
  getParameter 89
  getParameterMap 90
  getParameterNames 90
  getParameterValues 90
  getReader 90
  setCharacterEncoding 91
functions
  using 256

G

getAuthType method 93
getContentLength method 93
getCookies method 93
getHeaderNames method 94
getHeaders 94
getInputStream method 90
getIntHeader method 94
getMethod method 94
getParameterMap method 90
getParameter method 89
getParameterNames method 90
getParameterValues method 90
getProtocol method 94
getQueryString method 94
getReader method 90
getRemoteUser method 93
getRequestURI method 94
global session scope 434

H

HTML 62
HTTP 62, 63
  examples 63, 64
  features 62
  header fields 65, 66
  headers 65
  message body 66

message start-line 65
requests 66-70
HTTP 1.1 status codes 97-99
HTTP request headers 94
Accept-Charset 94
Accept-Encoding 94
Accept-Language 94
Authorization 94
Connection 95
Content-Length 95
Cookie 95
example 96
Host 95
If-modified-since 95
If-Unmodified-Since 95
Referer 95
User-Agent 95
HTTP response headers 100, 101
expires 102
last-modified 103
location 103
pragma 103
refresh 103, 104
retry-after 104
Set-Cookie header 104
WWW-Authenticate 104

I

implicit objects
applicationScope 240
cookie 240
header 240
headerValues 240
pageContext 239
pageScope 240
param 239
paramValues 239
requestScope 240
sessionScope 240
using 239-242
include directive 175, 176
XML syntax 176-182
IoC container 424
beans 424-426
resources 425
settings 425
using 426

J

JavaBean 190-192
advantages 192
all properties, assigning with input parameter values 212
beans, creating 197, 198
beans, creating conditionally 214-219
bean, sharing in application scope 230-232
bean, sharing in request scope 222-225
bean, sharing in session scope 225-229
beans, sharing in different scopes 212-214
JSP syntax, for creating and accessing 197
properties, assigning with input parameter values 205-211
unshared bean, in page scope 219-222
using, in web application 193-197
values, assigning 199-205
values, retrieving 198, 199
Java Database Connectivity (JDBC) 2
Java Development Kit (JDK) 2
Java frameworks
types 421, 422
Java SE 8 2

JavaServer Faces (JSF) 364, 365
  applications 373
  events 372
  MVC 367
  web interfaces 365
Java Server Pages (JSP) 71-73, 132, 364
  files, including 167
  overview 134, 135
  scripting elements 136, 137
  tokens 135, 136
JDBC API 2
JDBC API components
  connection 3
  Driver 3
  DriverManager 3
  ResultSet 3
  SQLException 3
  Statement 3
JDBC application, building 12
  CallableStatement objects, creating 16, 17
  connection, establishing 14
  data, extracting from result set 18-20
  environment, cleaning up 21
  JDBC driver, registering 12, 13
  packages, importing 12
  PreparedStatement objects, creating 15, 16
  query, executing 17
  statement, creating 15
  Statement object, creating 15
JDBC drivers 4
  DatabaseProtocol driver 7, 8
  JDBC-ODBC Bridge 4, 5
  Native-API driver 5, 6
  Network-Protocol 6, 7
  Type 5 JDBC Drivers 8
JDBC transactions 50
  commit 51
  rollback 51
  savepoints 51
Jetty server 473
JoinPoint 452
JSP API
  javax.servlet.jsp package 133
  javax.servlet.jsp.tagext package 133
JSP Comment 135
JSP Declaration 135
JSP directives 135, 146
  page directive 146
JSP Expression 135
JSP Expression Language Element 135
jsp:forward
  requests, forwarding with 183-185
jsp:include action 167-169
  using 169-175
JSP life cycle 132, 133
  HttpJspPage interface 134
  JSP page interface 134
JSP program
  writing 136
JSP Scriptlet 135
JSP Standard Tag Library (JSTL) 298
  core tag library 299
  installing 298
  tags classification 299
JSTL formatting tags 347
  fmt:bundle 355
  fmt:formatDate 351-353
  fmt:formatNumber 347-349
  fmt:message 360
  fmt:parseDate 354, 355
  fmt:parseNumber() 350, 351

fmt:requestEncoding 360
fmt:setbundle 355-357
fmt:setLocale 357, 358
fmt:setTimeZone 358, 359
fmt:timeZone 358
JSTL functions 331
fn:contains() function 331-333
fn:containsIgnoreCase() function 333, 334
fn:endsWith() function 334, 335
fn:escapeXml() function 335, 336
fn:indexOf() function 336, 337
fn:join() function 337, 338
fn:length() function 338, 339
fn:replace() function 339, 340
fn:split() function 340, 341
fn:startsWith() function 341, 342
fn:substringAfter() function 343, 344
fn:substringBefore() function 344, 345
fn:substring() function 342, 343
fn:toLowerCase() function 345
fn:toUpperCase() function 346
fn:trim() function 346, 347

M

MetaData 21
DatabaseMetaData 22
ResultSetMetaData 21
MIME-type 63
MIME types 102
Model, View, and Controller (MVC) paradigm 365
Model-View-Controller (MVC) 77, 78
example 270-293
implementing, with servlets and JSP 264-267
MVC design pattern 263
need for 263, 264
MVC frameworks 264
MVC, JavaServer Faces (JSF)
annotations 370
bean information 372
controller beans 367, 368
dependency injection 370
lifecycle methods 370
PrimeFaces DataList, managing 368, 369
serving levels 371, 372

N

Native-API driver 5
Network-Protocol driver 6

O

operators
arithmetic operators 248
empty operator 248
logical operators 248
relational operators 248
ternary operator 249
using 248-256

P

page attributes 168
page directive, JSP 146
autoFlush attribute 159
buffer attribute 159
contentType attribute 148-158
errorPage attribute 159-165
extends attribute 167
import attributes 147, 148
info attributes 159
isELIgnored attribute 159
isErrorPage attribute 159-165
isThreadSafe attribute 166

language attributes 167
pageEncoding attribute 148-158
session attributes 158, 159
pointcut 452
Properties class
methods 44, 45
properties file 44
Properties file example 45-50
prototype scope 433

R

Rapid Application Development (RAD) 373
principles 365
REpresentational State Transfer (REST) 385
RequestDispatcher interface 267, 268
request headers
handling 93
methods, for reading 93
request scope 433
response headers
handling 99
HTTP response headers 100
setting, from servlets 100
RESTful web services 385
advantages 385, 386
ResultSetMetaData 21

S

scoped variables
retrieving 233-236
scripting elements, JSP 136, 137
comparing 144-146
JSP declarations 142, 143
JSP expressions 137-140
JSP scriptlets 141, 142
sendError method 97
sendRedirect method 97
server response
302 and 404 status codes, setting 97
generating 96
HTTP 1.1 status codes 97
random status codes, setting 97
service-oriented architecture (SOA) 380
functional operations 382
operations 381
qualitative operations 382
service connections 380
terminologies 380, 381
working 381
servlet 71
advantages 84
basics 82
structure 86
tasks 82
servlet life cycle 84
destroy() method 85
doGet() method 85
doPost() method 85
init() method 84
scenario 86
service() method 85
servlet programming 86, 87
HTML, generating 88
plaintext, generating 88
session managing 118
session scope 434
session tracking
API, using for 124, 125
cookies 119
hidden form fields 120
information, associating with session 122, 123

in servlets 121
overview 119
session data, discarding 123
session object, accessing 121, 122
session types 126
URL encoding 127
URL rewriting 120
setCharacterEncoding method 91
setStatus method 97
Simple Object Access Protocol (SOAP) 383
singleton scope 432
SOAP-based web services 384
benefits 385
challenges 385
Software Development Life Cycle (SDLC) 373
Spring 422, 458
architecture 422
directory structure 423, 424
Spring AOP 422
Spring boot 458
advantages 459
annotations 461, 462, 463
architecture 460
business layer 460
database layer 460
disadvantages 459
features 458
persistence layer 460
presentation layer 460
projects, creating 460, 461
Spring boot CLI 461
Spring boot starters 469
external starters 469-471
Spring boot embedded web server 473
Spring boot production starters 471
Spring boot starter web 472
Spring boot technical starters 471
spring-boot-starter-web
versus, spring-boot-starter-tomcat 474, 475
Spring Context 422
Spring Core 422
Spring DAO 422
Spring database connectivity 476
Hibernate, versus JPA 477
Spring boot starter data JPA 477
Spring data repository 476
Spring Data JPA 476
Spring Initializer 460
Spring MVC validation 440
Spring ORM 422
Spring Tool Suite 460
Spring Web 422
Spring Web MVC 422
SQL JSTL tags 319
sql:dateParam tag 323
sql:param tag 322, 323
sql:query tag 320, 321
sql:setDataSource tag 319, 320
sql:transaction tag 323, 324
sql:update tag 322

T

taglib directive 186
TCP/IP 61, 62
technology
usage 262
TestCookie.java 112

U

undertow server 474
Universal Description, Discovery, and Integration (UDDI) 383

Universal Resource Locator (URL) 70, 71

V

validation
  applying 440-448
value.getDateHeader 94

W

WAR file 77
web application 60
  directory structure 75-77
Web Archive (WAR) 75
web client 60
web interfaces, JSF 365, 366
  HTML components 366
  Java objects, using 366, 367
  nested tags 366
web server 60
  request handling, container used 73, 74
web service client application
  testing 414
web services 382, 383
  consuming, in Java 395-401
  container, selecting 387, 388
  creating 387
  creating, from Java class 388
  creating, with Java 402-410
  deployment testing, to GlassFish or WebLogic server 393, 394
  executing 393
  in Java 386
  interoperability 402
  JAX-WS, for creating client 386, 387
  operation, adding 388-392
  RESTful web services 385
  SOAP-based web services 384
  testing 393
  types 383, 384
Web Services Description Language (WSDL) 383
web.xml file 77

X

XML JSTL tags 324
  x:choose tag 328, 329
  x:forEach tag 327, 328
  x:if tag 326, 327
  x:otherwise tag 329
  x:out tag 325
  x:parse tag 325, 326
  x:set tag 326
  x:transform tag 329, 330
  x:when tag 329